Học Pruning từ đầu

Lộ trình 50 bài

Ghi chú học tập bằng tiếng Việt

Tháng 11, 2026

Mục lục

Học Pruning từ đầu: lộ trình 50 bài . . . 4
Vì sao mạng cắt được: mạng quá tham số hoá chứa rất nhiều thứ thừa . . . 8
Ma trận thưa và chi phí thật: số không bằng 0 ít hơn chưa chắc là chạy nhanh hơn . . . 16
Độ quan trọng của trọng số: độ lớn, gradient, Hessian và kích hoạt trả lời bốn câu hỏi khác nhau . . . 24
Cắt hạt, cắt khối và mẫu 2:4: quyết định xoá "những gì" sẽ quyết định bạn được gì . . . 34
Cắt rồi huấn luyện lại: sau cú cắt, mô hình cần được chữa lành bằng cách nào . . . 44
Cắt lớp và độ sâu trong LLM: dòng dư làm cho bỏ một lớp ít phá vỡ tính toán, nhưng chỉ khi chọn đúng lớp . . . 53
Đo đúng một kết quả pruning: perplexity, benchmark, độ trễ và tỉ lệ nén nói gì và không nói gì . . . 61
00 Preface: Pruning cho người mới bắt đầu — toán nền tảng và hướng dẫn đọc . . . 70
01 Optimal Brain Damage (1989/1990) và Second Order Derivatives for Network Pruning: Optimal Brain Surgeon (1992/1993) . . . 83
02 Learning both Weights and Connections for Efficient Neural Networks (2015) . . . 90
03 To prune, or not to prune: exploring the efficacy of pruning for model compression (2017) . . . 96
04 Pruning Filters for Efficient ConvNets (2016) . . . 103
05 Learning Efficient Convolutional Networks through Network Slimming (2017) . . . 109
06 ThiNet: A Filter Level Pruning Method for Deep Neural Network Compression (2017) . . . 116
07 Importance Estimation for Neural Network Pruning (2019) . . . 123
08 The Lottery Ticket Hypothesis: Finding Sparse, Trainable Neural Networks (2018) . . . 130
09 Linear Mode Connectivity and the Lottery Ticket Hypothesis (2019) . . . 136
10 SNIP: Single-shot Network Pruning based on Connection Sensitivity (2018) . . . 142
11 Rethinking the Value of Network Pruning (2018) . . . 149
12 What is the State of Neural Network Pruning? (2020) . . . 156
13 Are Sixteen Heads Really Better than One? (2019) . . . 163
14 Analyzing Multi-Head Self-Attention: Specialized Heads Do the Heavy Lifting, the Rest Can Be Pruned (2019) . . . 169
15 The Lottery Ticket Hypothesis for Pre-trained BERT Networks (2020) . . . 176
16 Movement Pruning: Adaptive Sparsity by Fine-Tuning (2020) . . . 182
17 Structured Pruning Learns Compact and Accurate Models (2022) . . . 189
18 Accelerating Sparse Deep Neural Networks (2021) . . . 197
19 HALP: Hardware-Aware Latency Pruning (2021) . . . 203
20 DepGraph: Towards Any Structural Pruning (2023) . . . 210
21 A Survey on Model Compression for Large Language Models (2023) . . . 217
22 Optimal Brain Compression: A Framework for Accurate Post-Training Quantization and Pruning (2022) . . . 223
23 SparseGPT: Massive Language Models Can Be Accurately Pruned in One-Shot (2023) . . . 232
24 A Simple and Effective Pruning Approach for Large Language Models (2023) . . . 239
25 ZipLM: Inference-Aware Structured Pruning of Language Models (2023) . . . 245
26 LLM-Pruner: On the Structural Pruning of Large Language Models (2023) . . . 252
27 The LLM Surgeon (2023) . . . 258
28 Fluctuation-based Adaptive Structured Pruning for Large Language Models (2023) . . . 265
29 SliceGPT: Compress Large Language Models by Deleting Rows and Columns (2024) . . . 272
30 MaskLLM: Learnable Semi-Structured Sparsity for Large Language Models (2024) . . . 278
31 ShortGPT: Layers in Large Language Models are More Redundant Than You Expect (2024) . . . 284

32 The Unreasonable Ineffectiveness of the Deeper Layers (2024) . . . 289
33 Shortened LLaMA: Depth Pruning for Large Language Models with Comparison of Retraining Methods (2024) . . . 294
34 SLEB: Streamlining LLMs through Redundancy Verification and Elimination of Transformer Blocks (2024) . . . 299
35 LaCo: Large Language Model Pruning via Layer Collapse (2024) . . . 304
36 ReplaceMe: Network Simplification via Depth Pruning and Transformer Block Linearization (2025) . . . 309
37 Repair Changes Which Layers to Remove: Exact Optima for Depth Pruning of Language Models (2026) 315
38 Sheared LLaMA: Accelerating Language Model Pre-training via Structured Pruning (2023) . . . 324
39 Compact Language Models via Pruning and Knowledge Distillation (2024) . . . 330
40 LLM Pruning and Distillation in Practice: The Minitron Approach (2024) . . . 339
41 Flextron: Many-in-One Flexible Large Language Model (2024) . . . 345
42 Puzzle: Distillation-Based NAS for Inference-Optimized LLMs (2024) . . . 351
43 Small LLMs: Pruning vs. Training from Scratch (2026) . . . 358
44 Nemotron-H: A Family of Accurate and Efficient Hybrid Mamba-Transformer Models (2025) . . . 366
45 Minitron-SSM: Efficient Hybrid Language Model Compression through Group-Aware SSM Pruning (2025) . . . 372
46 Mamba-Shedder: Post-Transformer Compression for Efficient Selective Structured State Space Models (2025) . . . 377
47 Jet-Nemotron: Efficient Language Model with Post Neural Architecture Search (2025) . . . 382
48 ECoFLaP: Efficient Coarse-to-Fine Layer-Wise Pruning for Vision-Language Models (2024) . . . 387
49 Structural Pruning of Large Vision Language Models: A Comprehensive Study on Pruning Dynamics, Recovery, and Data Efficiency (2026) . . . 392

Lộ trình

Học Pruning từ đầu: lộ trình 50 bài

Thư mục này chứa ghi chú học tập bằng tiếng Việt về **pruning mạng neural**, từ các bài kinh điển năm 1989 đến các paper năm 2026. Thứ tự được sắp cho **người chưa biết gì về domain**: cơ bản trước, rồi chuyển dần sang LLM, model hybrid và VLM.

> Đây là các ghi chú diễn giải, không phải bản dịch. Hãy tham khảo nguồn gốc được dẫn ở đầu từng bài khi trích dẫn số liệu.

Giai đoạn 0: Chuẩn bị

#	File	Nội dung
00	`00_preface_toan_va_huong_dan.md`	Pruning là gì, phân loại (unstructured / N:M / structured), tiêu chí, phục hồi, toán (Taylor, Hessian, OBS, least squares), đánh giá latency, bảng thuật ngữ, câu tự kiểm tra. **Đọc trước tiên.**
01	`nen01_vi_sao_mang_cat_duoc.md`	Chương nền: Vì sao mạng cắt được. Dư thừa tham số, mặt nạ, độ thưa, vì sao chất lượng giữ được rồi mới sụp.
02	`nen02_ma_tran_thua_va_chi_phi_that.md`	Chương nền: Ma trận thưa và chi phí thật. CSR/COO, điểm hoà vốn bộ nhớ, FLOPs lý thuyết khác tốc độ thật.
03	`nen03_do_quan_trong_cua_trong_so.md`	Chương nền: Độ quan trọng của trọng số. Độ lớn, Taylor, Hessian (OBD/OBS) và Wanda, chọn cắt khác nhau ra sao.
04	`nen04_cat_hat_cat_khoi_va_2_4.md`	Chương nền: Cắt hạt, cắt khối và 2:4. Mức chi tiết của mặt nạ, mẫu 2:4, dây chuyền phụ thuộc giữa các lớp.
05	`nen05_cat_roi_huan_luyen_lai.md`	Chương nền: Cắt rồi huấn luyện lại. Một lần và lặp, lịch thưa bậc ba, distillation và LoRA để phục hồi.
06	`nen06_cat_lop_va_do_sau.md`	Chương nền: Cắt lớp và độ sâu. Dòng dư, Block Influence, bỏ lớp so với cắt chiều rộng.
07	`nen07_do_dung_chat_luong_pruning.md`	Chương nền: Đo đúng một kết quả pruning. Perplexity, benchmark, độ trễ, tỉ lệ nén và tăng tốc thật.

Giai đoạn 1: Nền tảng kinh điển (CNN era)

#	File	Bài	Ý chính
01	`01_1989_1993_optimal_brain_damage_surgeon.md`	Optimal Brain Damage / Optimal Brain Surgeon	Chọn trọng số để cắt bằng đạo hàm bậc 2 và **bù trừ** các trọng số còn lại. Gốc của SparseGPT.
02	`02_2015_learning_weights_and_connections.md`	Han et al. 2015	Cắt theo độ lớn (magnitude), xen kẽ với retrain.
03	`03_2017_to_prune_or_not_to_prune.md`	Zhu & Gupta 2017	Gradual pruning; model lớn đã thưa so với model nhỏ dày đặc.
04	`04_2016_pruning_filters_convnets.md`	Li et al. 2016	Structured pruning: cắt cả filter để tăng tốc thật.

#	File	Bài	Ý chính
05	`05_2017_network_slimming.md`	Network Slimming	Dùng hệ số BatchNorm với phạt L1 làm tiêu chí cắt kênh.
06	`06_2017_thinet.md`	ThiNet	Chọn kênh theo **sai số tái tạo** của layer kế tiếp (least squares).
07	`07_2019_importance_estimation_taylor.md`	Molchanov et al. 2019	Ước lượng importance bằng Taylor bậc 1 và 2.

Giai đoạn 2: Lý thuyết và tranh luận

#	File	Bài	Ý chính
08	`08_2018_lottery_ticket_hypothesis.md`	Lottery Ticket Hypothesis	Mạng con "trúng thưởng" train lại được từ khởi tạo ban đầu.
09	`09_2019_linear_mode_connectivity_lth.md`	Linear Mode Connectivity & LTH	Khi nào ticket tồn tại; vai trò của độ ổn định với nhiễu SGD.
10	`10_2018_snip.md`	SNIP	Cắt ngay lúc khởi tạo, trước khi train.
11	`11_2018_rethinking_value_of_pruning.md`	Rethinking the Value of Pruning	Train lại từ đầu thường ngang fine-tune; giá trị nằm ở **kiến trúc** tìm được.
12	`12_2020_state_of_neural_network_pruning.md`	What is the State of Pruning?	Cách đọc paper pruning một cách phản biện; vấn đề baseline và benchmark.

Giai đoạn 3: Pruning transformer (BERT era)

#	File	Bài	Ý chính
13	`13_2019_sixteen_heads.md`	Are Sixteen Heads Really Better than One?	Cắt được nhiều head mà gần như không mất gì.
14	`14_2019_voita_specialized_heads.md`	Voita et al. 2019	Vài head chuyên biệt làm phần lớn việc; phần còn lại cắt được.
15	`15_2020_lottery_ticket_bert.md`	LTH for pre-trained BERT	Ticket trên model pretrain; mask có chuyển giao được không.
16	`16_2020_movement_pruning.md`	Movement Pruning	Cắt theo chiều trọng số **di chuyển** khi fine-tune, thay vì theo độ lớn.
17	`17_2022_cofi_structured_pruning.md`	CoFi	Structured pruning với mask đa độ mịn kết hợp distillation từng layer.

Giai đoạn 4: Phần cứng và công cụ

#	File	Bài	Ý chính
18	`18_2021_nvidia_2to4_sparsity.md`	Accelerating Sparse DNNs (2:4)	Semi-structured sparsity được GPU hỗ trợ.
19	`19_2021_halp_latency_pruning.md`	HALP	Pruning theo **ngân sách latency** thay vì số tham số.

#	File	Bài	Ý chính
20	`20_2023_depgraph.md`	DepGraph	Xử lý phụ thuộc giữa các layer khi cắt structured cho kiến trúc bất kỳ.

Giai đoạn 5: LLM, pruning một lần (post-training)

#	File	Bài	Ý chính
21	`21_2023_survey_llm_compression.md`	Survey on LLM Compression	Bản đồ toàn cảnh: pruning, quantization, distillation, low-rank.
22	`22_2022_optimal_brain_compression.md`	Optimal Brain Compression	OBS chính xác theo từng hàng, tái tạo layer.
23	`23_2023_sparsegpt.md`	SparseGPT	Cắt 50% LLM một lần, có bù trừ trọng số.
24	`24_2023_wanda.md`	Wanda	$\|W\|\cdot\|\|X\|\|$, không bù trừ; đơn giản mà mạnh.
25	`25_2023_ziplm.md`	ZipLM	Structured, chọn theo sai số **sau khi bù trừ OBS**, có ràng buộc latency.
26	`26_2023_llm_pruner.md`	LLM-Pruner	Structured pruning LLM với nhóm phụ thuộc, phục hồi bằng LoRA.
27	`27_2023_llm_surgeon.md`	LLM Surgeon	Dùng curvature (Kronecker-factored) để cắt structured và cập nhật trọng số.
28	`28_2023_flap.md`	FLAP	Tiêu chí fluctuation, bù trừ bằng bias, không cần retrain.
29	`29_2024_slicegpt.md`	SliceGPT	Xoay residual stream bằng PCA rồi xóa hàng và cột.
30	`30_2024_maskllm.md`	MaskLLM	Học mask N:M bằng huấn luyện end-to-end.

Giai đoạn 6: Depth pruning (cắt layer)

#	File	Bài	Ý chính
31	`31_2024_shortgpt.md`	ShortGPT	Block Influence: cắt các layer ít thay đổi residual stream.
32	`32_2024_unreasonable_ineffectiveness_deeper_layers.md`	Gromov et al.	Cắt khối layer sâu theo khoảng cách góc, “heal” bằng QLoRA.
33	`33_2024_shortened_llama.md`	Shortened LLaMA	So depth với width; LoRA và continued pretraining.
34	`34_2024_sleb.md`	SLEB	Loại block theo kiểm chứng dư thừa.
35	`35_2024_laco.md`	LaCo	“Gập” các layer vào nhau thay vì xóa.
36	`36_2025_replaceme.md`	ReplaceMe	Thay các block bị cắt bằng **một phép biến đổi tuyến tính**.
37	`37_2026_repair_changes_which_layers.md`	Repair Changes Which Layers to Remove	Khi có repair, tập layer tối ưu để cắt thay đổi.

Giai đoạn 7: Prune và distill ở quy mô lớn

#	File	Bài	Ý chính
38	`38_2023_sheared_llama.md`	Sheared LLaMA	Targeted structured pruning, dynamic batch loading.
39	`39_2024_minitron.md`	Minitron	Depth so với width; thứ hạng đảo sau retrain; công thức KD.
40	`40_2024_llama31_minitron.md`	Llama-3.1-Minitron	Teacher correction; width thắng depth sau distill.
41	`41_2024_flextron.md`	Flextron	Model "nhiều trong một", chọn cấu hình theo latency.
42	`42_2024_puzzle.md`	Puzzle	NAS dựa trên distillation, tối ưu cho inference trên phần cứng thật.
43	`43_2026_small_llms_pruning_vs_scratch.md`	Small LLMs: Pruning vs. Training from Scratch	Khi nào prune có lợi hơn train từ đầu.

Giai đoạn 8: Hybrid (SSM / linear attention) và VLM

#	File	Bài	Ý chính
44	`44_2025_nemotron_h.md`	Nemotron-H	Kiến trúc hybrid Mamba-Transformer; nén 56B xuống 47B bằng MiniPuzzle.
45	`45_2025_minitron_ssm.md`	Minitron-SSM	Group-aware pruning cho Mamba head.
46	`46_2025_mamba_shedder.md`	Mamba-Shedder	Loại block, module SSM và kênh trong Mamba/hybrid.
47	`47_2025_jet_nemotron_postnas.md`	Jet-Nemotron	Nên giữ layer full-attention nào trong hybrid.
48	`48_2024_ecoflap_vlm.md`	ECoFLaP	Pruning VLM: phân bổ sparsity từ thô đến mịn.
49	`49_2026_structural_pruning_vlms.md`	Structural Pruning of Large VLMs	Width so với layer trên VLM; phục hồi bằng SFT/KD; hiệu quả dữ liệu.

Cách đọc gợi ý

- **Người mới:** đọc 00 → giai đoạn 1 → 08, 11, 12 → 21, 22, 23, 24 → giai đoạn 6 → 39, 40.
- **Đọc theo chủ đề phục hồi:** 00 → 22 → 25 → 36 → 37 → 39 → 40 → 47 → 49.
- Kết hợp đọc bài gốc với thực hành trên mô hình nhỏ để hiểu các giả định và giới hạn của từng phương pháp.

Liên kết với các thư mục khác

- `../distillation content/`: phục hồi model sau pruning bằng distillation.
- `../interp content/`: interpretability, để hiểu *bên trong* model mất gì khi bị cắt.

Nền tảng 01

Vì sao mạng cắt được: mạng quá tham số hoá chứa rất nhiều thứ thừa

Loại: chương nền tảng (không phải ghi chú paper)

Đọc trước: ma trận và phép nhân ma trận (math nền tảng 02), SVD ở mức trực giác (math nền tảng 05), mạng nơ-ron nhiều lớp (ML cơ bản nền tảng 05) và overfitting (ML cơ bản nền tảng 03)

Dùng ở: pruning 01 (OBD/OBS), pruning 02 (Han 2015), pruning 03 (cắt dần), pruning 08 (Lottery Ticket), pruning 09 (linear mode connectivity), pruning 10 (SNIP), pruning 12 (state of pruning), pruning 21 (khảo sát nén LLM)

> Chương này giải thích từ đầu, không phải bản dịch một paper. Các phép tính ghi "tự tính" đã kiểm bằng numpy. Các đường cong và histogram là mô phỏng trên một mạng rất nhỏ tự huấn luyện (seed cố định), dùng để minh hoạ khái niệm, không phải số đo trên mô hình thật.

1. Vì sao cần hiểu điều này trước khi đọc về pruning

Mạng cắt được vì nó có nhiều tham số hơn mức bài toán cần, nên một phần lớn trọng số đóng góp rất ít vào đầu ra. Mọi phương pháp trong cuốn sách này, từ cắt theo độ lớn của Han năm 2015 đến các phương pháp cắt LLM, đều dựa trên một niềm tin chung: ta bỏ được một phần đáng kể trọng số mà mô hình vẫn làm việc gần như cũ. Nếu niềm tin đó sai thì toàn bộ lĩnh vực không có lý do tồn tại.

Chương này trả lời ba câu hỏi một cách chậm rãi. Thứ nhất, "dư thừa" nghĩa là gì, nhìn bằng hình thì nó trông ra sao? Thứ hai, cắt một trọng số thì đầu ra đổi bao nhiêu, và vì sao có trọng số cắt rẻ, có trọng số cắt đắt? Thứ ba, vì sao đường chất lượng theo độ thưa thường phẳng một đoạn dài rồi mới sụp?

Với AI và LLM, câu trả lời có hậu quả thực tế. Một lớp tuyến tính 4096×4096 có 16.777.216 trọng số (tự tính); ở độ chính xác FP16 (2 byte mỗi số) nó chiếm 32 MiB. Một mô hình có hàng chục lớp như vậy tốn hàng chục GB bộ nhớ chỉ để chứa trọng số. Nếu một nửa số trọng số đó thực sự thừa, ta có cơ hội giảm bộ nhớ và, với điều kiện phần cứng phù hợp (xem chương nền 02), cả thời gian chạy.

2. Nhắc nền: trọng số, cạnh, tham số và độ thưa

2.1 Một lớp tuyến tính là một bảng trọng số

Một lớp tuyến tính nhận vector vào x có n phần tử và tạo vector ra $z = Wx$ có m phần tử. Ma trận W cỡ $m \times n$ chứa mn **trọng số**. Mỗi trọng số w_{ij} là hệ số nhân đặt trên đường nối từ đầu vào j đến đầu ra i. Vì vậy người ta vẽ mạng thành sơ đồ nút và cạnh: mỗi nút là một giá trị, mỗi cạnh là một trọng số, và "cắt một trọng số" nghĩa là xoá một cạnh khỏi sơ đồ. Cắt xong, w_{ij} coi như bằng 0 và phép nhân $w_{ij}x_j$ biến mất khỏi tổng.

Các tham số khác của mạng (hệ số bias, chuẩn hoá) có số lượng nhỏ hơn rất nhiều. Phần lớn tham số nằm trong các ma trận trọng số, nên pruning chủ yếu nhắm vào chúng.

2.2 Mạng hai lớp với ReLU

Mạng nhỏ nhất đủ để quan sát hiện tượng gồm một lớp ẩn:

$$h = \mathrm{ReLU}(W_1 x), \qquad y = w_2^\top h,$$

trong đó $\mathrm{ReLU}(a) = \max(a, 0)$ áp dụng từng phần tử. ReLU cho phép một neuron "tắt" (đầu ra bằng 0) với những đầu vào làm tổng của nó âm. Neuron tắt không truyền gì sang lớp sau, và điều đó sẽ quan trọng ở mục 6.

2.3 Độ thưa

Độ thưa (sparsity) của một ma trận là tỉ lệ phần tử bằng 0:

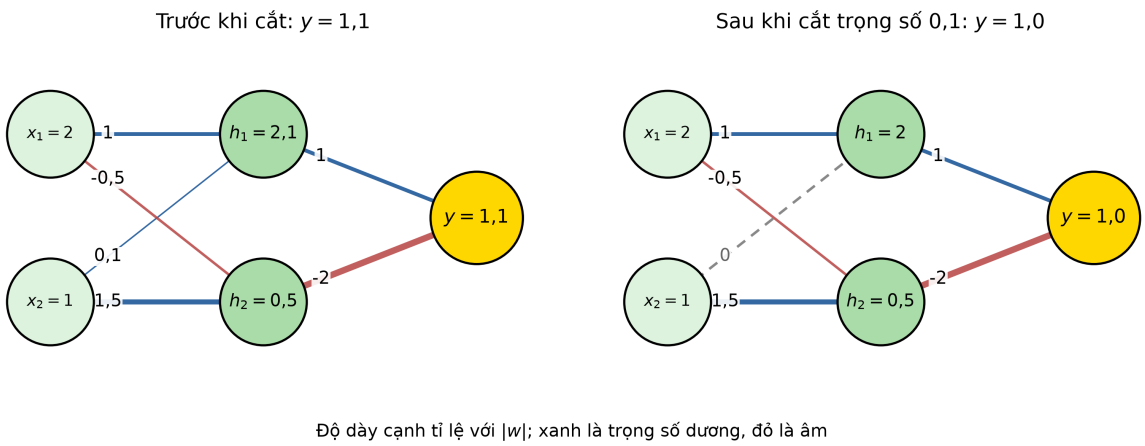
$$s = \frac{\text{số phần tử bằng } 0}{\text{tổng số phần tử}}.$$

Ví dụ (tự tính): ma trận 1000×1000 có $1.000.000$ phần tử; độ thưa 90% nghĩa là còn đúng 100.000 phần tử khác 0, tức tỉ lệ còn lại là $1 - s = 10\%$. Hai cách nói này hay bị lẫn trong bài báo: "cắt 90%" và "còn 10%" là cùng một mô hình, đừng đọc nhầm.

3. Trực giác bằng hình: cắt một cạnh và nhìn trọng số

3.1 Cắt một cạnh trong mạng 2-2-1

Ta bắt đầu bằng mạng nhỏ đến mức tính tay được: hai đầu vào, hai neuron ẩn, một đầu ra, sáu trọng số. Đầu vào là $x = (2, 1)$, ma trận lớp ẩn là $W_1 = \begin{pmatrix} 1 & 0{,}1 \\ -0{,}5 & 1{,}5 \end{pmatrix}$ và vector ra là $w_2 = (1, -2)$.



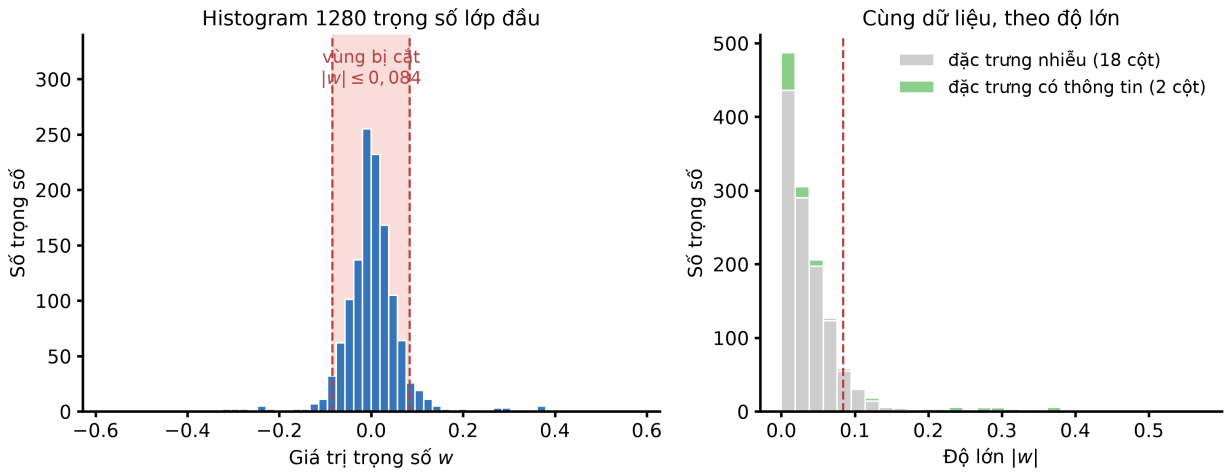
Hình 1: Cắt trọng số 0,1 trong mạng 2-2-1 chỉ làm đầu ra đổi từ 1,1 xuống 1,0 (minh hoạ, tự tính)

Cách đọc hình: bên trái là mạng nguyên vẹn, độ dày cạnh tỉ lệ với độ lớn trọng số, xanh là dương và đỏ là âm. Cạnh mảnh nhất nối x_2 với h_1 mang trọng số $0{,}1$. Bên phải ta xoá đúng cạnh đó (nét đứt, nhãn 0); các cạnh dày không bị động tới. Đầu ra chỉ đổi từ $1{,}1$ xuống $1{,}0$ (tự tính), trong khi nếu xoá một cạnh dày thì đầu ra đổi nhiều hơn rất nhiều (bảng ở mục 5).

Đây chính là trực giác cốt lõi: **không phải trọng số nào cũng đóng góp ngang nhau**. Đóng góp của w_{ij} vào đầu ra là $w_{ij}x_j$, nên một trọng số nhỏ nhân với một đầu vào cũng nhỏ thì gần như không để lại dấu vết. Chương nền 03 sẽ bàn cái bẫy ngược lại: trọng số nhỏ nhưng đầu vào rất lớn thì vẫn quan trọng.

3.2 Histogram trọng số của một mạng đã huấn luyện

Hình tiếp theo cho thấy trọng số của một mạng thật sau huấn luyện phân bố thế nào. Mạng ở đây rất nhỏ, ta tự huấn luyện bằng numpy: dữ liệu có 20 đặc trưng vào nhưng nhãn chỉ phụ thuộc vào 2 đặc trưng đầu (18 đặc trưng còn lại là nhiễu), lớp ẩn có 64 neuron, nên ma trận lớp đầu cỡ 64×20 có 1280 trọng số. Đây là mô phỏng để minh hoạ, không phải mô hình thật.



Hình 2: Histogram 1280 trọng số lớp đầu của mạng nhỏ tự huấn luyện, và vùng bị cắt khi độ thưa 90% (minh hoạ, tự tính)

Cách đọc hình: bên trái là histogram toàn bộ trọng số; vùng đỏ ở giữa là những trọng số có độ lớn không quá $0{,}084$, tức 90% trọng số nhỏ nhất, nơi ngưỡng cắt đặt ra. Gần như toàn bộ khối lượng tập trung quanh 0; chỉ một đuôi mỏng kéo dài sang hai bên. Bên phải là cùng dữ liệu theo độ lớn $|w|$ và tô màu theo cột: xanh là 2 cột đặc trưng có thông tin, xám là 18 cột nhiễu. Cột xám dồn hết về bên trái ngưỡng, còn các cột xanh chiếm phần lớn đuôi dài.

Theo số đếm của mô phỏng (tự tính): sau khi cắt độ lớn ở $s = 90\%$, còn lại 128 trọng số, trong đó 49 thuộc hai cột có thông tin (trên 128, tức khoảng 38%) và chỉ 79 thuộc 18 cột nhiễu (trên 1152, khoảng $6{,}9\%$). Tức là cắt độ lớn, vốn không biết gì về nhãn, đã tự dồn việc giữ lại về các trọng số "có việc làm". Khoảng $72{,}7\%$ trọng số có độ lớn dưới $0{,}05$. Histogram nhọn quanh 0 là điều pruning theo độ lớn khai thác: theo ghi chú pruning 02, Han và cộng sự đặt ngưỡng của mỗi tầng bằng một hệ số nhân với độ lệch chuẩn trọng số của tầng đó rồi bỏ mọi trọng số dưới ngưỡng.

4. Định nghĩa chính thức

4.1 Mặt nạ và mạng thưa

Cắt tỉa trọng số được viết gọn bằng một **mặt nạ** nhị phân $M \in \{0,1\}^{m \times n}$ cùng cỡ với W:

$$\tilde{W} = M \odot W,$$

với $\odot$ là nhân từng phần tử. $M_{ij} = 0$ nghĩa là cạnh (i, j) bị cắt. Độ thưa là $s = 1 - \frac{\|M\|_0}{mn}$, với $\|M\|_0$ là số phần tử bằng 1 của mặt nạ.

4.2 Độ quan trọng (saliency)

Câu hỏi trung tâm của pruning: **bỏ trọng số nào thì thiệt hại nhỏ nhất?** Ghi chú pruning 01 định nghĩa saliency của một tham số là mức tăng của hàm mục tiêu khi xoá tham số đó. Viết thành công thức, với L là hàm mất mát và $W^{(-ij)}$ là W sau khi đặt $w_{ij} = 0$:

$$\Delta L_{ij} = L\big(W^{(-ij)}\big) - L(W).$$

Tính ΔL_{ij} cho mọi trọng số bằng cách thử từng cái một đòi hỏi mn lần chạy mạng, quá đắt. Mọi tiêu chí trong chương nền 03 là cách ước lượng ΔL_{ij} rẻ hơn. Chương này chỉ cần ý rằng ước lượng đó có thể rất khác

nhau giữa các trọng số.

4.3 Quá tham số hoá

Một mạng được gọi là **quá tham số hoá** (overparameterized) khi số tham số lớn hơn nhiều so với mức tối thiểu cần để biểu diễn hàm mong muốn. Dư tham số có công dụng thực tế: nó làm bài toán tối ưu dễ hơn, nên ta thường huấn luyện mạng lớn rồi mới nén. Nhưng chính phần dư đó khiến một số tham số thành thừa khi huấn luyện xong, và đó là chỗ pruning khai thác.

5. Ví dụ tay làm chậm: cắt từng trọng số một và xem đầu ra đổi bao nhiêu

Lấy lại mạng ở mục 3.1: $x = (2, 1)$, $W_1 = \begin{pmatrix} 1 & 0{,}1 \\ -0{,}5 & 1{,}5 \end{pmatrix}$, $w_2 = (1, -2)$. Tất cả phép tính dưới đây đều tự tính.

Bước 1: lan truyền xuôi mạng nguyên vẹn. Tổng đầu vào lớp ẩn là $z = W_1 x = (1 \cdot 2 + 0{,}1 \cdot 1,\ -0{,}5 \cdot 2 + 1{,}5 \cdot 1) = (2{,}1;\ 0{,}5)$. Cả hai đều dương nên ReLU giữ nguyên: $h = (2{,}1;\ 0{,}5)$. Đầu ra $y = 1 \cdot 2{,}1 + (-2) \cdot 0{,}5 = 1{,}1$.

Bước 2: cắt $w_{12} = 0{,}1$ (trọng số nhỏ nhất). Khi đó $z_1 = 2$, $z_2 = 0{,}5$ không đổi, nên $y = 2 - 1 = 1{,}0$. Đầu ra đổi $-0{,}1$.

Bước 3: cắt $w_{21} = -0{,}5$. Khi đó $z_2 = -0{,}5 \cdot 0 + 1{,}5 = 1{,}5$, tức $h_2 = 1{,}5$ và $y = 2{,}1 - 2 \cdot 1{,}5 = -0{,}9$. Đầu ra đổi $-2{,}0$. Trọng số này có độ lớn trung bình $(0{,}5)$ mà đầu ra đổi rất nhiều vì nó đang "kìm" neuron thứ hai.

Bước 4: làm nốt bốn trọng số còn lại. Kết quả đầy đủ:

Trọng số bị cắt	Đầu ra sau khi cắt	Đổi so với 1,1
W1[1,2] = 0,1	1,0	-0,1
W1[1,1] = 1	-0,9	-2,0
W1[2,1] = -0,5	-0,9	-2,0
W1[2,2] = 1,5	2,1	+1,0
w2[1] = 1	-1,0	-2,1
w2[2] = -2	2,1	+1,0

Ba điều rút ra. Một, cắt trọng số nhỏ nhất đổi đầu ra ít nhất (chỉ $0{,}1$ so với $1{,}1$). Hai, thiệt hại không tỉ lệ đơn giản với độ lớn: trọng số $-0{,}5$ nhỏ hơn $1{,}5$ nhưng cắt nó đổi đầu ra tới $2{,}0$, vì ReLU và tương tác giữa các lớp. Ba, "đầu ra đổi" không phải lúc nào cũng là "tệ đi": cắt $w_2[2]$ làm y tăng từ $1{,}1$ lên $2{,}1$; có tệ hay không tuỳ vào đầu ra đúng phải là bao nhiêu. Đo thiệt hại thật sự phải dựa vào hàm mất mát, không chỉ độ lệch đầu ra.

Ví dụ này chỉ có một mẫu đầu vào. Với dữ liệu thật ta lấy trung bình trên nhiều mẫu, và hai trọng số có thể đổi vai: trọng số cắt rẻ với mẫu này lại đắt với mẫu khác. Vì vậy phần "trên dữ liệu nào" luôn là một phần của định nghĩa độ quan trọng.

6. Biến thể và chi tiết hay bị bỏ qua

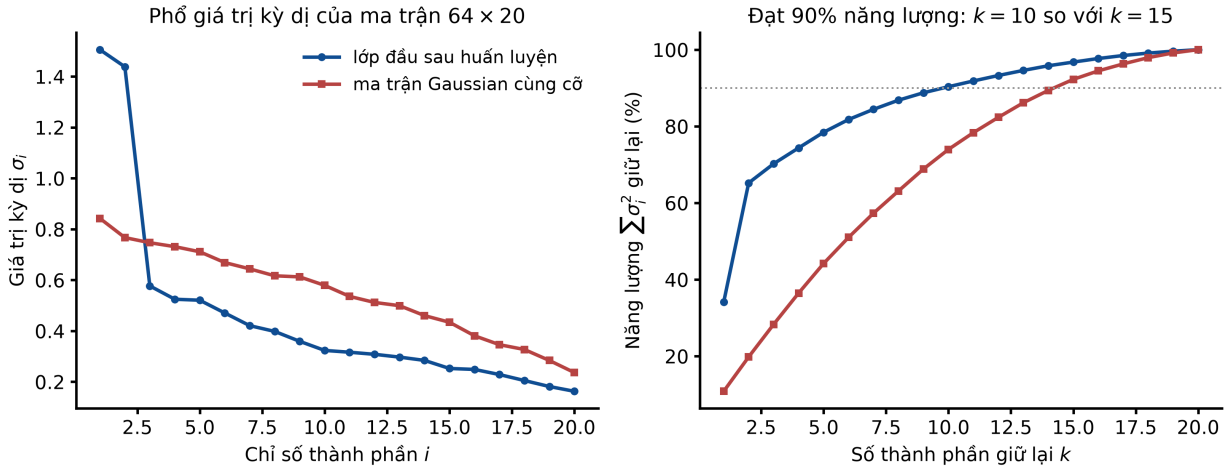
6.1 Dư thừa dạng trùng lặp: hai neuron làm cùng một việc

Dạng dư thừa dễ hình dung nhất là hai neuron học ra gần như cùng một hàm. Ví dụ cực đoan (tự tính): lớp ẩn có hai neuron với hàng trọng số giống hệt $(1, 2)$, vector ra $w_2 = (3, 3)$, đầu vào $x = (1, 1)$. Mỗi neuron cho $h = 3$, nên $y = 3 \cdot 3 + 3 \cdot 3 = 18$. Xoá neuron thứ hai rồi gấp đôi trọng số ra của neuron còn lại ($w_2 = 6$) cho $y = 6 \cdot 3 = 18$. Hàm không đổi dù số tham số của lớp giảm một nửa. Trong mạng thật không bao giờ

có hai neuron trùng khít, nhưng "gần trùng" thì rất thường gặp, và kỹ thuật cắt có bù (OBS, SparseGPT) khai thác chính điều này bằng cách chỉnh các trọng số còn lại sau khi cắt.

6.2 Dư thừa dạng hạng thấp

Cách nhìn thứ hai là **hạng hiệu dụng thấp**: dù ma trận 64×20 có 20 hướng độc lập, chỉ một vài hướng mang phần lớn thông tin. Ta kiểm bằng SVD (xem math nền tảng 05), $W = \sum_i \sigma_i u_i v_i^\top$, nơi các giá trị kỳ dị σ_i đo "độ mạnh" của từng thành phần.



Hình 3: Phổ giá trị kỳ dị của lớp đầu mạng nhỏ đã huấn luyện so với ma trận Gaussian cùng cỡ (minh hoạ, tự tính)

Cách đọc hình: bên trái là các giá trị kỳ dị xếp giảm dần; đường xanh là lớp đầu của mạng nhỏ sau huấn luyện, đường đỏ là ma trận Gaussian ngẫu nhiên cùng cỡ và cùng độ lệch chuẩn phần tử. Đường xanh có một "vách đá": hai giá trị đầu ($1{,}505$ và $1{,}438$) rồi rơi xuống $0{,}577$ ở giá trị thứ ba (tự tính), khớp với việc nhãn chỉ phụ thuộc vào 2 đặc trưng. Đường đỏ giảm từ từ, không có cấu trúc. Bên phải là phần năng lượng $\sum \sigma_i^2$ giữ lại khi chỉ giữ k thành phần đầu: hai thành phần đầu đã chứa $65{,}2\%$ năng lượng, và cần $k = 10$ để đạt 90%, so với $k = 15$ của ma trận ngẫu nhiên.

Lưu ý ranh giới: mạng nhỏ này chỉ có một cấu trúc rất đơn giản nên "vách đá" rõ. Ở mạng thật, phổ thường rơi chậm hơn, và hạng thấp là một cách nén khác (phân rã ma trận thành tích hai ma trận nhỏ) chứ không phải pruning. Chương này dùng nó như bằng chứng trực giác rằng ma trận trọng số không dùng hết số chiều mà nó có.

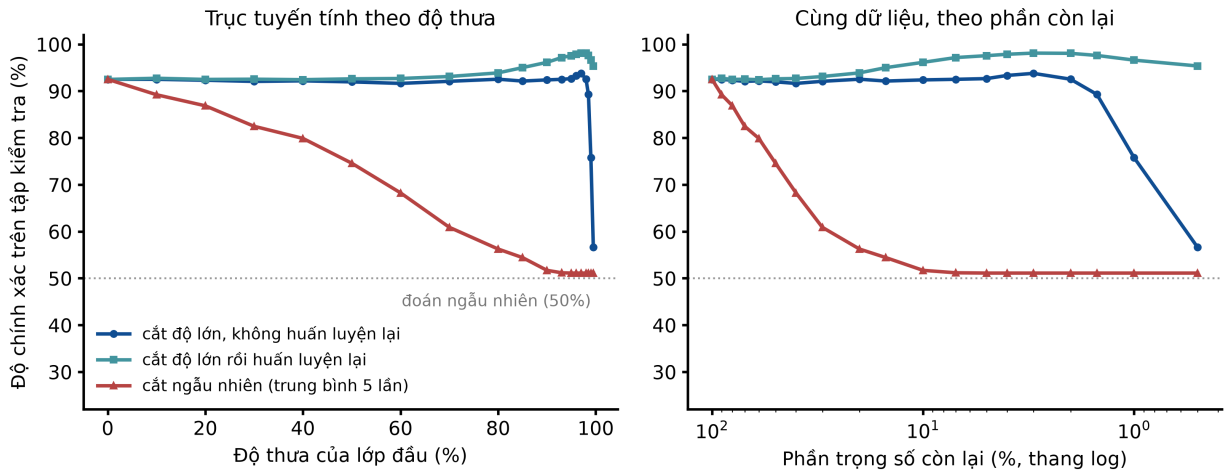
6.3 Neuron chết

Neuron ReLU mà tổng đầu vào luôn âm với mọi mẫu dữ liệu cho đầu ra 0 mãi mãi. Ví dụ (tự tính): một neuron có hàng trọng số $(-1, -0{,}5)$ trên đầu vào $x = (2, 1)$ cho $z = -2{,}5$, nên $\mathrm{ReLU}(z) = 0$. Cắt toàn bộ neuron đó không đổi đầu ra, vì nó chưa từng truyền gì. Đây là kiểu dư thừa miễn phí nhất, nhưng cần kiểm tra trên đủ dữ liệu: một neuron im lặng trên tập hiệu chỉnh vẫn có thể bật với một đầu vào hiếm.

6.4 Chất lượng theo độ thưa: phẳng rồi sụp

Hình sau là kết quả của một thí nghiệm nhỏ trên chính mạng 64 neuron ở trên. Ta cắt lớp đầu ở nhiều mức độ thưa, đo độ chính xác trên một tập kiểm tra riêng (4000 mẫu) và so ba cách: cắt độ lớn không huấn luyện lại, cắt độ lớn rồi huấn luyện lại 300 bước với mặt nạ cố định, và cắt ngẫu nhiên (trung bình 5 lần). Đây là mô phỏng trên mạng nhỏ, không phải kết quả đo trên mô hình thật.

Cách đọc hình: trục ngang bên trái là độ thưa, bên phải là phần trọng số còn lại theo thang log, để thấy rõ đoạn cuối. Đường xanh đậm (cắt độ lớn, không huấn luyện lại) gần như nằm ngang quanh 92% cho đến độ



Hình 4: Độ chính xác theo độ thưa của lớp đầu trong mạng nhỏ tự huấn luyện: đường phẳng rồi sụp (minh hoạ, tự tính)

thưa khoảng 98%, rồi rơi nhanh: 89,3% ở 98,5% thưa, 75,8% ở 99%, 56,6% ở 99,5% (tự tính). Đường đỏ (cắt ngẫu nhiên) đi xuống ngay từ đầu: 74,6% ở độ thưa 50% và về mức đoán ngẫu nhiên khoảng 51% từ khoảng 90%. Đường xanh lơ (có huấn luyện lại) giữ trên 95% đến tận 99,5%.

Ba nhận xét. Một, **phẳng rồi sụp** là hình dạng điển hình: dư thừa là một khoản "dự trữ", cắt trong khoản dự trữ thì gần như miễn phí, vượt khỏi nó thì mất nhanh. Hai, **chọn đúng trọng số quan trọng hơn cắt bao nhiêu**: cùng độ thưa 50%, cắt độ lớn giữ 91,9% còn cắt ngẫu nhiên chỉ 74,6%. Ba, độ chính xác thậm chí **tăng** ở đoạn giữa (đạt 93,8% khi thưa 97%, so với 92,5% của mạng đầy đủ), vì mạng đầy đủ ở thiết lập này đã quá khớp (độ chính xác trên tập huấn luyện là 100%, trên tập kiểm tra 92,5%) và cắt đóng vai trò điều chuẩn. Kết quả này đặc thù của mô phỏng nhỏ; đừng suy ra rằng cắt luôn làm mô hình tốt hơn.

6.5 Ý tưởng vé số ở mức khái niệm

Từ các quan sát trên có một câu hỏi tự nhiên: nếu mạng thưa sau khi cắt vẫn tốt, sao không huấn luyện thẳng mạng thưa đó từ đầu cho rẻ? Thực nghiệm cũ cho thấy mạng thưa khởi tạo ngẫu nhiên thường khó huấn luyện hơn mạng đầy đủ. Giả thuyết vé số (Frankle và Carbin) trả lời: trong mạng dày khởi tạo ngẫu nhiên có những mạng con thưa "trúng số", tức nếu giữ đúng mặt nạ và đúng giá trị khởi tạo ban đầu của các trọng số sống sót, mạng con đó huấn luyện được ngang mạng đầy đủ. Theo ghi chú pruning 08, trên Lenet/MNIST, mạng con còn 21% trọng số học nhanh hơn (dừng sớm sớm hơn 38% so với mạng gốc), độ chính xác tăng hơn 0,3 điểm phần trăm ở 13,5% còn lại, rồi về bằng mạng gốc ở 3,6%; nếu giữ mặt nạ nhưng khởi tạo lại ngẫu nhiên thì kết quả kém rõ rệt. Chi tiết và các điều kiện nằm ở pruning 08; chương này chỉ cần ý tưởng: dư thừa không chỉ là "trọng số thừa" mà còn có thể là "mạng con tốt nằm sẵn trong mạng dày".

7. Nối sang pruning và LLM

Đường chất lượng phẳng rồi sụp là hình ảnh nền của cả cuốn sách. Các bài trong pruning đều đang tìm cách đi xa hơn trên đoạn phẳng, hoặc làm đoạn phẳng dài thêm: chọn trọng số tốt hơn (pruning 01, 07, 22, 24), cắt rồi bù (pruning 22, 23), cắt dần trong lúc huấn luyện (pruning 03), huấn luyện lại sau khi cắt (pruning 02, 38 đến 40).

Han 2015 là minh chứng quy mô lớn đầu tiên. Theo ghi chú pruning 02, quy trình train, prune, retrain nén AlexNet từ 61M xuống 6,7M tham số (khoảng 9 lần) và VGG-16 từ 138M xuống 10,3M (khoảng 13 lần) mà không mất độ chính xác. Con số này nhất quán với hình dạng phẳng của mục 6.4, dù thang đo khác xa mạng nhỏ ở đây.

LLM có đoạn phẳng ngắn hơn với một số cách cắt đơn giản. Theo ghi chú pruning 24, trên LLaMA-7B ở độ thưa 50%, cắt độ lớn đẩy perplexity từ $5{,}68$ (mô hình đầy đủ) lên $17{,}29$, trong khi Wanda (có tính đến độ lớn kích hoạt) chỉ lên $7{,}26$. Nói cách khác ở LLM, tiêu chí chọn quyết định đoạn phẳng dài hay ngắn nhiều hơn so với mạng nhỏ ở trên; chương nền 03 giải thích vì sao.

Dư thừa không đồng nghĩa với tốc độ. Có 90% trọng số bằng 0 chưa có nghĩa là chạy nhanh hơn 10 lần. Việc đổi dư thừa thành bộ nhớ và thời gian phụ thuộc cách lưu trữ và phần cứng; đó là chủ đề của chương nền 02, và là lý do pruning 04 trở đi chuyển sang cắt có cấu trúc.

Dư thừa có thể nằm ở nhiều cấp độ. Từng trọng số (unstructured), từng neuron hay kênh (structured), từng đầu attention, thậm chí cả một lớp: cùng một niềm tin "mạng quá tham số hoá" được áp dụng ở mỗi cấp. Các bài về cắt lớp trong LLM (pruning 31 đến 37) kiểm tra niềm tin này ở cấp thô nhất.

8. Sai lầm hay gặp

- **"Trọng số nhỏ thì vô dụng."** Đóng góp của w_{ij} là $w_{ij}x_j$; trọng số nhỏ trên đầu vào lớn có thể rất quan trọng, và trọng số nhỏ trong mạng có chuẩn hoá có thể được khuếch đại ở lớp sau. Ví dụ ở mục 5: trọng số $-0{,}5$ nhỏ hơn $1{,}5$ nhưng cắt nó đổi đầu ra mạnh hơn.
- **Lẫn "cắt 90%" với "còn 90%".** Độ thưa và tỉ lệ còn lại cộng lại bằng 100%. Đọc kỹ bài báo xem con số nào được dùng.
- **Cho rằng cắt nhiều là chuyện tốt tuyệt đối.** Đường chất lượng phẳng chỉ trong một khoảng; đoạn sụp có thể đến rất đột ngột (mô phỏng: từ $92{,}5\%$ xuống $75{,}8\%$ chỉ trong vài điểm phần trăm độ thưa cuối cùng).
- **Suy ra từ một mạng nhỏ rằng mạng lớn cũng vậy.** Mô phỏng ở đây có cấu trúc rất đơn giản (2 đặc trưng thật trong 20). Điểm sụp của mạng thật phụ thuộc kiến trúc, dữ liệu và cách chọn trọng số, nên phải đo lại.
- **Nhầm giảm tham số với giảm thời gian chạy.** Số trọng số khác 0 và độ trễ là hai đại lượng khác nhau (chương nền 02).
- **Đo thiệt hại chỉ bằng độ lệch đầu ra.** Đầu ra đổi mà mất mát không tăng (hoặc giảm) vẫn là cắt tốt; ngược lại, đầu ra đổi ít trên mẫu này nhưng nhiều trên mẫu khác vẫn là cắt xấu.

9. Bảng tóm tắt

Khái niệm	Công thức và ý nghĩa
Độ thưa	$s = 1 - \dfrac{\|M\|_0}{mn}$: tỉ lệ trọng số bằng 0
Mạng sau khi cắt	$\tilde{W} = M \odot W$: mặt nạ nhị phân nhân từng phần tử
Saliency	$\Delta L_{ij} = L(W^{(-ij)}) - L(W)$: mức tăng mất mát khi xoá một trọng số
Dư thừa trùng lặp	Hai neuron cho cùng một hàm: xoá một neuron rồi gấp đôi trọng số ra của neuron còn lại
Dư thừa hạng thấp	Vài giá trị kỳ dị lớn nhất chứa phần lớn năng lượng $\sum \sigma_i^2$: ma trận không dùng hết số chiều
Phẳng rồi sụp	Chất lượng gần như hằng đến một độ thưa, sau đó giảm nhanh: dư thừa là khoản dự trữ hữu hạn
Vé số	Mạng con thưa kèm khởi tạo ban đầu huấn luyện được ngang mạng đầy đủ (xem pruning 08)

10. Bài tập tự tính

Các bài là phép tính độc lập, không phải số liệu paper. Mỗi đáp án đã kiểm bằng numpy.

Bài 1. Đếm trọng số

Ma trận 1000×1000 có độ thưa 90%. Còn bao nhiêu trọng số khác 0? Nếu chuyển sang độ thưa 99% thì còn bao nhiêu?

Đáp án (tự tính): Tổng $1.000.000$ phần tử. Độ thưa 90% còn 100.000 phần tử; độ thưa 99% còn 10.000 phần tử.

Bài 2. Cắt trọng số trong mạng 2-2-1

Dùng mạng ở mục 5 với $x = (2, 1)$. Tính đầu ra khi cắt $w_2[2] = -2$ (trọng số thứ hai của vector ra), và đầu ra đổi bao nhiêu so với $1{,}1$.

Đáp án (tự tính): Khi $w_2[2] = 0$, đầu ra là $1 \cdot 2{,}1 + 0 \cdot 0{,}5 = 2{,}1$, đổi $+1{,}0$.

Bài 3. Neuron trùng lặp

Lớp ẩn có hai neuron cùng hàng trọng số $(2, 1)$, vector ra $w_2 = (1, 1)$, đầu vào $x = (1, 3)$. Tính y; rồi xoá neuron thứ hai và đặt lại trọng số ra của neuron còn lại để y giữ nguyên.

Đáp án (tự tính): Mỗi neuron cho $h = 2 \cdot 1 + 1 \cdot 3 = 5$ nên $y = 5 + 5 = 10$. Sau khi xoá neuron thứ hai, đặt $w_2 = 2$ cho neuron còn lại: $y = 2 \cdot 5 = 10$.

Bài 4. Hạng thấp và độ thưa cùng số tham số

Ma trận 4096×4096 được xấp xỉ bằng tích AB với A cỡ $4096 \times r$, B cỡ $r \times 4096$. Với $r = 256$, số tham số là bao nhiêu và bằng bao nhiêu phần của ma trận đầy đủ? Cùng số tham số đó ứng với độ thưa bao nhiêu nếu cắt từng trọng số?

Đáp án (tự tính): Số tham số là $2 \cdot 4096 \cdot 256 = 2.097.152$, bằng $12{,}5\%$ của $16.777.216$. Cùng số tham số ứng với độ thưa $87{,}5\%$. Để bằng độ thưa 90% (còn 10%) thì $2 \cdot 4096 \cdot r = 0{,}1 \cdot 4096^2$, tức $r = 204{,}8$, làm tròn xuống $r = 204$.

Bài 5. Trọng số nhỏ trên đầu vào lớn

Một neuron có $w = (0{,}05;\ 0{,}8;\ -0{,}03;\ 0{,}6)$ và đầu vào $x = (100, 1, 1, 1)$. Tính $y = w \cdot x$. Cắt hai trọng số có độ lớn nhỏ nhất, tính y mới; rồi cắt hai trọng số có $|w_j x_j|$ nhỏ nhất, tính y mới. So độ thay đổi.

Đáp án (tự tính): $y = 5 + 0{,}8 - 0{,}03 + 0{,}6 = 6{,}37$. Hai độ lớn nhỏ nhất là $0{,}05$ và $-0{,}03$; cắt chúng còn $0{,}8 + 0{,}6 = 1{,}4$, đổi $-4{,}97$ (khoảng 78%). Hai tích $|w_j x_j|$ nhỏ nhất là $0{,}03$ (kênh 3) và $0{,}6$ (kênh 4); cắt chúng còn $5 + 0{,}8 = 5{,}8$, đổi $-0{,}57$ (khoảng 9%). Cắt theo độ lớn của w bỏ mất đóng góp 5 của kênh đầu vào lớn; đây là điều chương nền 03 sẽ hệ thống hoá.

Tài liệu tham khảo nền: LeCun, Denker và Solla, Optimal Brain Damage; Han, Pool, Tran và Dally, Learning both Weights and Connections; Frankle và Carbin, The Lottery Ticket Hypothesis; các ghi chú pruning 01, 02, 08, 12 và 24 trong sách.

Nền tảng 02

Ma trận thưa và chi phí thật: số không bằng 0 ít hơn chưa chắc là chạy nhanh hơn

Loại: chương nền tảng (không phải ghi chú paper)

Đọc trước: ma trận và phép nhân ma trận-vector (math nền tảng 02), chương nền tảng 01 của cuốn này (độ thưa, mặt nạ)

Dùng ở: pruning 02 (Han 2015, chi phí bộ nhớ), pruning 04 (cắt filter), pruning 12 (state of pruning, FLOPs và tốc độ), pruning 18 (2:4 trên GPU), pruning 19 (HALP, độ trễ thật), pruning 23 (SparseGPT), pruning 24 (Wanda), pruning 30 (MaskLLM)

> Chương này giải thích từ đầu, không phải bản dịch một paper. Các phép tính ghi "tự tính" đã kiểm bằng numpy. Các đường cong tăng tốc là mô hình đơn giản với tham số phần cứng và hiệu suất GIẢ ĐỊNH để minh hoạ (ghi rõ ngay trong chương), không phải số đo trên GPU thật.

1. Vì sao cần hiểu điều này

Đặt một nửa số trọng số bằng 0 chưa làm mô hình nhỏ hơn hay nhanh hơn: nó chỉ làm nhỏ hơn hoặc nhanh hơn khi ta có cách lưu và cách tính tận dụng được những số 0 đó. Một ma trận đầy đủ vẫn lưu cả các số 0 và vẫn nhân cả chúng, vì phần cứng không "biết" số nào đó là 0 nếu không được báo trước.

Với AI và LLM, điều này quyết định pruning có đáng làm không. Suy luận LLM sinh từng token thường bị giới hạn bởi tốc độ đọc trọng số từ bộ nhớ, không phải bởi số phép nhân. Một định dạng thưa tốn nhiều byte hơn ma trận đầy đủ (vì phải lưu chỉ số) có thể làm chậm đi dù số phép nhân giảm. Chương này xây từ hai thứ giản dị: đếm byte, và so sánh thời gian đọc dữ liệu với thời gian tính.

2. Nhắc nền: byte, FLOPs và hai kiểu nghẽn

2.1 Bộ nhớ của một ma trận đầy đủ

Ma trận $m \times n$ lưu từng phần tử, mỗi phần tử b_v byte, nên cần $m\,n\,b_v$ byte. Độ chính xác FP16 dùng $b_v = 2$, FP32 dùng 4, INT8 dùng 1. Ví dụ (tự tính): ma trận 4096×4096 ở FP16 có $16.777.216$ phần tử nên chiếm $33.554.432$ byte, tức 32 MiB. Ma trận đầy đủ không quan tâm giá trị là 0 hay không: lưu 0 cũng tốn đủ b_v byte.

2.2 FLOPs của phép nhân ma trận-vector

Tính $y = Wx$ với W cỡ $m \times n$ cần mn phép nhân và gần mn phép cộng, tức khoảng $2mn$ phép tính dấu phẩy động (FLOPs). Với $m = n = 4096$: $2 \cdot 4096^2 = 33.554.432$ FLOPs (tự tính). Nếu 90% phần tử bằng 0 và ta bỏ qua được các phép nhân với số 0 thì còn khoảng $3.355.443$ FLOPs. Con số lý thuyết này là $1/(1-s)$ lần ít hơn với độ thưa s.

2.3 Nghẽn tính toán và nghẽn bộ nhớ

Một phép tính chạy mất thời gian của khâu chậm hơn trong hai khâu: tính toán (số FLOPs chia tốc độ tính cực đại P) và chuyển dữ liệu (số byte chia băng thông bộ nhớ BW). Mô hình đơn giản này gọi là **roofline**:

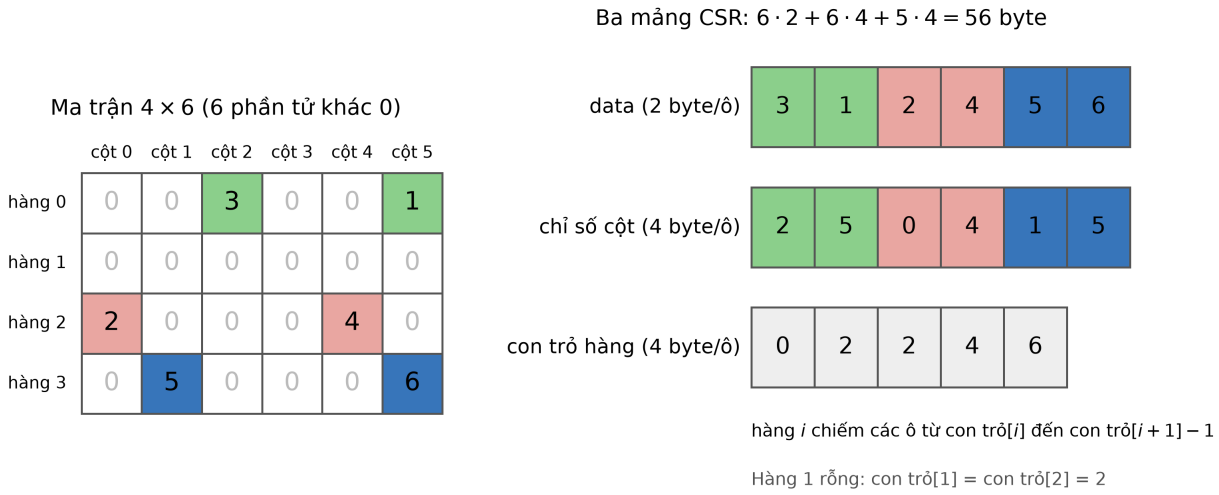
$$t \approx \max\left(\frac{\text{FLOPs}}{P},\ \frac{\text{byte}}{BW}\right).$$

Khi khâu tính toán chậm hơn, ta nói phép tính **bị chặn bởi tính toán**; ngược lại là **bị chặn bởi bộ nhớ**. Tỉ lệ FLOPs trên byte của phép tính gọi là **cường độ số học**. Phép nhân ma trận-vector có cường độ rất thấp: mỗi trọng số được đọc 1 lần chỉ dùng cho khoảng 2 phép tính. Theo ghi chú pruning 02, Han và cộng sự từ năm 2015 đã nhấn mạnh chi phí chính của mạng lớn là truy cập bộ nhớ chứ không phải phép nhân: bảng năng lượng 45nm trong bài đó cho một lần đọc DRAM 32-bit khoảng 640 pJ so với khoảng $3{,}7$ pJ cho một phép nhân số thực 32-bit, gấp hơn 170 lần.

3. Trực giác bằng hình: CSR lưu gì, và nó tốn bao nhiêu byte

3.1 Ba mảng của CSR

Cách phổ biến nhất để lưu ma trận thưa là **CSR** (compressed sparse row, hàng thưa nén). Thay vì lưu mọi ô, CSR chỉ lưu các phần tử khác 0, kèm hai mảng chỉ số để biết chúng nằm đâu.



Hình 5: Ma trận 4x6 có 6 phần tử khác 0 và ba mảng CSR: giá trị, chỉ số cột, con trỏ hàng (minh hoạ, tự tính)

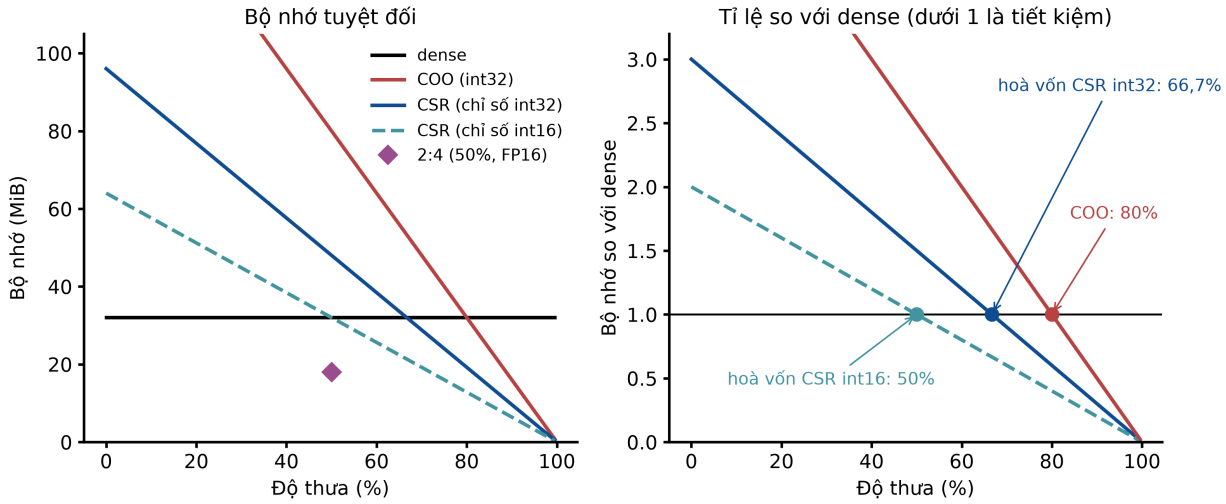
Cách đọc hình: bên trái là ma trận 4×6, mỗi hàng được tô một màu riêng (hàng 1 toàn số 0 nên để trắng). Bên phải là ba mảng. Mảng **data** liệt kê các giá trị khác 0 theo thứ tự đọc từng hàng: $3, 1, 2, 4, 5, 6$. Mảng **chỉ số cột** cho biết mỗi giá trị nằm ở cột nào: 3 ở cột 2, 1 ở cột 5, và cứ thế. Mảng **con trỏ hàng** có $m + 1 = 5$ ô: hàng i chiếm các ô từ con trỏ$[i]$ đến con trỏ$[i + 1] - 1$ trong hai mảng trên. Hàng 1 rỗng nên con trỏ$[1]$ bằng con trỏ$[2]$. Màu trong hai mảng đầu khớp màu hàng ở ma trận, giúp theo dõi.

Điểm cần thấy: mỗi giá trị khác 0 phải đi cùng một chỉ số cột. Với giá trị FP16 (2 byte) và chỉ số int32 (4 byte), mỗi phần tử khác 0 tốn 6 byte chứ không chỉ 2 byte. Cách lưu khác tên là **COO** (coordinate) lưu thẳng bộ ba (hàng, cột, giá trị) cho mỗi phần tử nên còn tốn hơn: $2 + 4 + 4 = 10$ byte mỗi phần tử.

3.2 Bộ nhớ theo độ thưa

Hình sau vẽ bộ nhớ của ma trận 4096×4096 FP16 theo độ thưa, cho dense, COO, CSR với chỉ số int32, CSR với chỉ số int16 (đủ khi số cột nhỏ hơn 65.536) và mẫu 2:4 của NVIDIA.

Cách đọc hình: đường đen ngang là dense, luôn 32 MiB bất kể độ thưa. Mọi đường định dạng thưa đều xuất phát từ trên đường đen (ở độ thưa 0, CSR int32 tốn gấp ba lần: 96 MiB) rồi giảm tuyến tính. Bên phải vẽ tỉ lệ so với dense, nên chỗ cắt qua đường ngang bằng 1 là **điểm hoà vốn**: CSR int16 hoà vốn ở 50%, CSR int32 ở $66{,}7\%$, COO ở 80%. Dưới độ thưa đó, định dạng thưa tốn nhiều byte hơn dense. Kim cương tím là mẫu 2:4 ở độ thưa 50% với FP16: 18 MiB (tức $56{,}25\%$ của dense), thấp hơn hẳn các định dạng thưa tổng quát vì metadata chỉ 2 bit cho mỗi giá trị giữ lại.



Hình 6: Bộ nhớ của ma trận 4096x4096 FP16 theo độ thưa cho dense, COO, CSR và mẫu 2:4, cùng các điểm hoà vốn (minh hoạ, tự tính)

4. Định nghĩa chính thức

4.1 Công thức bộ nhớ

Cho ma trận $m \times n$ có z phần tử khác 0. Gọi b_v là số byte của một giá trị và b_i là số byte của một chỉ số. Khi đó:

$$\text{Dense} = m\,n\,b_v, \qquad \text{COO} = z\,(b_v + 2b_i), \qquad \text{CSR} = z\,(b_v + b_i) + (m+1)\,b_i.$$

Số hạng $(m+1)b_i$ của CSR là mảng con trỏ hàng; nó nhỏ khi ma trận lớn nhưng đáng kể khi ma trận bé.

4.2 Điểm hoà vốn

CSR nhỏ hơn dense khi $z(b_v + b_i) + (m+1)b_i < m\,n\,b_v$. Bỏ qua số hạng con trỏ (đúng khi $m \ll mn$), đặt mật độ $d = z/(mn) = 1 - s$:

$$d < \frac{b_v}{b_v + b_i} \quad \Longleftrightarrow \quad s > \frac{b_i}{b_v + b_i}.$$

Với FP16 và chỉ số int32: $s > 4/6 = 66{,}7\%$. Với FP16 và chỉ số int16: $s > 2/4 = 50\%$. Với FP32 và int32: $s > 50\%$. COO có $2b_i$ nên ngưỡng cao hơn: $s > 2b_i/(b_v + 2b_i) = 80\%$ với FP16 và int32. Hoà vốn về bộ nhớ chưa phải hoà vốn về tốc độ; xem mục 6.

4.3 Các định dạng có cấu trúc

Các mẫu cố định theo khối tránh chi phí chỉ số lớn. Mẫu **N:M** (ví dụ 2:4: trong mỗi nhóm 4 phần tử liên tiếp giữ tối đa 2) lưu giá trị cộng chỉ 2 bit vị trí cho mỗi giá trị giữ lại. Với FP16: một nhóm 4 phần tử dense tốn $4 \cdot 16 = 64$ bit, còn định dạng 2:4 tốn $2 \cdot 16 + 2 \cdot 2 = 36$ bit (tự tính), khớp với số 36 bit và tiết kiệm khoảng 44% trong ghi chú pruning 18. Với giá trị INT8: $2 \cdot 8 + 2 \cdot 2 = 20$ bit trên 32 bit của dense, tiết kiệm 37,5% (tự tính), phù hợp với con số khoảng 38% ghi chú pruning 18 nêu. Ghi chú đó cũng nêu overhead metadata là 12,5% với giá trị 16-bit và 25% với 8-bit, so với tới 200% của CSR với giá trị 8-bit: kiểm lại (tự tính) $4/32 = 12{,}5\%$, $4/16 = 25\%$, và CSR int32 trên giá trị INT8 là $4/1 = 400\%$, còn với chỉ số int16 là $2/1 = 200\%$; con số 200% khớp với trường hợp chỉ số 16 bit (đây là suy ra của chương này, ghi chú không nói rõ loại chỉ số).

5. Ví dụ tay làm chậm từng bước

5.1 Ma trận 4×6 ở FP16

Dùng đúng ma trận ở hình mục 3.1, giá trị FP16 ($b_v = 2$) và chỉ số int32 ($b_i = 4$). Tất cả phép tính là tự tính.

Bước 1: đếm phần tử. Ma trận có $4 \cdot 6 = 24$ ô, trong đó $z = 6$ ô khác 0 (hàng 0 có 2, hàng 1 có 0, hàng 2 có 2, hàng 3 có 2). Độ thưa $s = 1 - 6/24 = 75\%$.

Bước 2: bộ nhớ dense. $24 \cdot 2 = 48$ byte.

Bước 3: bộ nhớ CSR. Mảng data: $6 \cdot 2 = 12$ byte. Mảng chỉ số cột: $6 \cdot 4 = 24$ byte. Mảng con trỏ hàng: $(4 + 1) \cdot 4 = 20$ byte. Tổng $12 + 24 + 20 = 56$ byte.

Bước 4: bộ nhớ COO. $6 \cdot (2 + 4 + 4) = 60$ byte.

Kết quả đáng chú ý: dù độ thưa 75%, cả CSR (56 byte) lẫn COO (60 byte) đều **tốn hơn** dense (48 byte). Nguyên nhân là ma trận nhỏ: con trỏ hàng tốn 20 byte, chiếm gần nửa dense.

Bước 5: tìm hoà vốn. CSR nhỏ hơn dense khi $6z + 20 < 48$, tức $z < 4{,}67$, nên $z \leq 4$. Độ thưa tương ứng ít nhất $1 - 4/24 = 83{,}3\%$. COO cần $10z < 48$, cũng $z \leq 4$. Như vậy ma trận bé này phải thưa trên 83% mới có lợi, cao hơn nhiều so với ngưỡng 66,7% của công thức bỏ qua con trỏ.

5.2 Ma trận lớn 4096×4096

Với $m = n = 4096$, FP16 và int32. Dense: 33.554.432 byte (32 MiB).

Ở độ thưa 90%: $z = \text{round}(0{,}1 \cdot 16.777.216) = 1.677.722$. CSR: $1.677.722 \cdot 6 + 4097 \cdot 4 = 10.066.332 + 16.388 = 10.082.720$ byte, khoảng 9,62 MiB, bằng 30,0% dense (tự tính). Con trỏ hàng chỉ 16.388 byte, không đáng kể.

Hoà vốn chính xác: CSR bằng dense khi $6z + 4 \cdot 4097 = 33.554.432$, tức $z = 5.589.674$ và độ thưa 66,83% (tự tính), rất gần ngưỡng 66,7% của công thức rút gọn. Với ma trận lớn con trỏ hàng không còn quan trọng, khác hẳn ví dụ 4×6.

5.3 Nhân ma trận-vector bằng CSR, từng bước

Để thấy vì sao CSR khó chạy nhanh, hãy tính $y = Ax$ cho ma trận 4×6 ở trên với $x = (1, 2, 3, 4, 5, 6)$. Mã giả của CSR là: với mỗi hàng i, đi từ con trỏ$[i]$ đến con trỏ$[i + 1] - 1$, mỗi lần lấy giá trị ở mảng data, lấy chỉ số cột c ở mảng chỉ số, rồi nhân với x_c và cộng dồn. Con trỏ hàng là $(0, 2, 2, 4, 6)$, data là $(3, 1, 2, 4, 5, 6)$, chỉ số cột là $(2, 5, 0, 4, 1, 5)$ (tự tính).

- Hàng 0 (ô 0 và 1): $3 \cdot x_2 + 1 \cdot x_5 = 3 \cdot 3 + 1 \cdot 6 = 15$.
- Hàng 1 (không có ô): 0.
- Hàng 2 (ô 2 và 3): $2 \cdot x_0 + 4 \cdot x_4 = 2 \cdot 1 + 4 \cdot 5 = 22$.
- Hàng 3 (ô 4 và 5): $5 \cdot x_1 + 6 \cdot x_5 = 5 \cdot 2 + 6 \cdot 6 = 46$.

Kết quả $y = (15, 0, 22, 46)$, trùng với nhân ma trận đầy đủ (tự tính). Số phép nhân giảm từ 24 xuống 6, đúng như độ thưa 75% hứa hẹn. Nhưng nhìn kỹ: vector x được đọc **theo chỉ số cột** (x_2, x_5, x_0, x_4, x_1, x_5), tức truy cập nhảy cóc, không liền kề. Mỗi phép nhân còn kèm một lần đọc chỉ số. Ngoài ra số phần tử mỗi hàng khác nhau (hàng 0 có 2, hàng 1 có 0), nên nếu giao mỗi hàng cho một luồng song song thì có luồng làm nhiều, có luồng làm ít. Ba chi phí ẩn này (đọc chỉ số, truy cập không liền, mất cân bằng tải) không có trong con số FLOPs và là lý do hệ số hiệu suất η và κ ở mục 6 nhỏ hơn 1.

6. FLOPs lý thuyết và độ trễ thật

6.1 Hai nghẽn, hai kết luận khác nhau

Giả sử một GPU đại diện có tốc độ tính $P = 300$ TFLOP/s và băng thông $BW = 2$ TB/s. **Các số này là giả định để minh hoạ**, không phải đo trên một GPU cụ thể. Tính $y = Wx$ với W cỡ 4096×4096 FP16 và một vector x (một token):

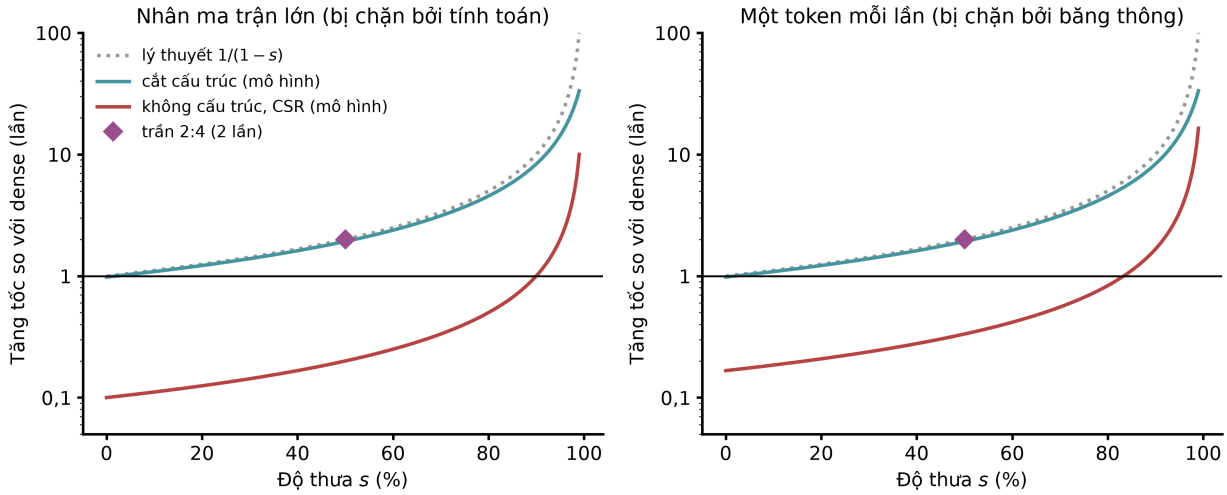
- Thời gian tính toán: $33.554.432/(300 \cdot 10^{12}) \approx 0{,}11\ \mu s$.
- Thời gian đọc trọng số: $33.554.432/(2 \cdot 10^{12}) \approx 16{,}8\ \mu s$.

Khâu đọc bộ nhớ chậm hơn khoảng 150 lần, nên phép tính bị chặn bởi bộ nhớ và thời gian khoảng $16{,}8\ \mu s$ (tự tính). Tăng cỡ batch B (số token xử lý cùng lúc) tái sử dụng mỗi trọng số B lần nên FLOPs tăng theo B còn byte gần như đứng yên. Hai khâu bằng nhau ở $B = P/BW \cdot b_v/2 = 150$ (tự tính, với các giả định trên); dưới ngưỡng đó phép tính bị chặn bởi bộ nhớ, trên ngưỡng đó bị chặn bởi tính toán.

6.2 Mô hình tăng tốc theo độ thưa

Ta thêm ba giả định hiệu suất, cũng chỉ để minh hoạ, và so ba cách tận dụng độ thưa s trên hai chế độ.

- **Cắt có cấu trúc** (xoá hẳn hàng hoặc cột, ma trận dense nhỏ lại): giữ nguyên hiệu suất kernel dense, chỉ cộng một chi phí cố định bằng 2% thời gian dense. Tăng tốc $1/((1-s)+0{,}02)$.
- **Không cấu trúc dùng CSR**: kernel gather không dùng được lõi nhân ma trận, giả sử chỉ đạt $\eta = 0{,}1$ tốc độ tính cực đại và $\kappa = 0{,}5$ băng thông.
- **2:4**: trần 2 lần ở độ thưa 50% (theo ghi chú pruning 18, tối đa 2 lần throughput GEMM).



Hình 7: Mô hình tăng tốc theo độ thưa cho cắt có cấu trúc, CSR không cấu trúc và trần 2:4, ở chế độ chặn tính toán và chặn băng thông (minh hoạ, tự tính)

Cách đọc hình: trục dọc là tăng tốc theo thang log, đường ngang đen là bằng 1 (không đổi). Đường chấm xám là lý thuyết $1/(1-s)$. Đường xanh lơ (cắt có cấu trúc) bám sát lý thuyết, chỉ thấp hơn một chút ở độ thưa rất cao vì chi phí cố định. Đường đỏ (CSR không cấu trúc) nằm **dưới** 1 ở phần lớn dải: bên trái (chặn tính toán) nó chỉ vượt 1 khi $s > 90\%$ (tức $\eta/(1-s) > 1$ với $\eta = 0{,}1$); bên phải (chặn băng thông) nó vượt 1 khi $s > 83\%$ (tự tính). Kim cương tím là trần 2 lần của mẫu 2:4.

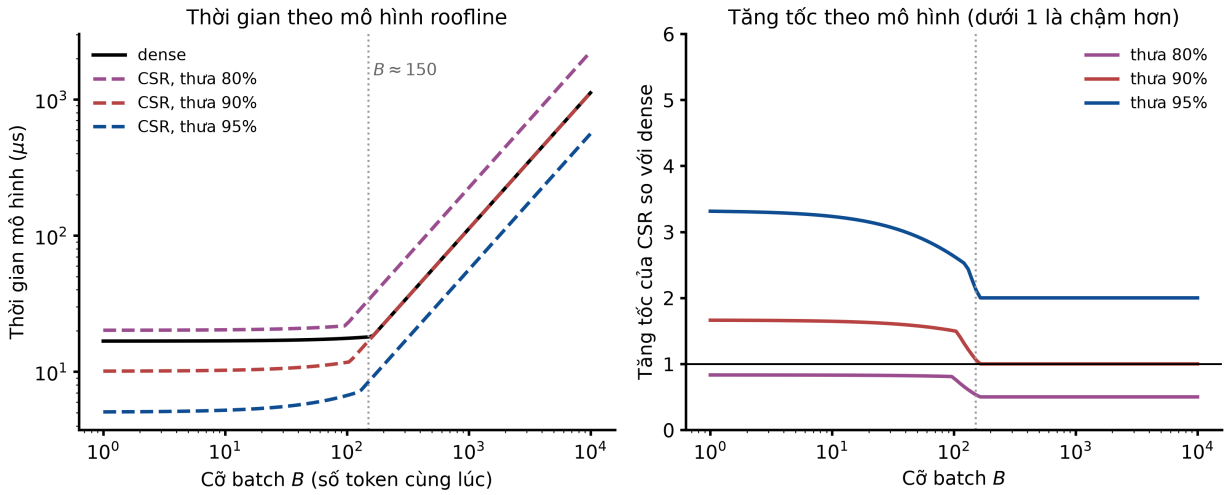
Một vài giá trị từ mô hình (tự tính): ở $s = 90\%$, lý thuyết cho 10 lần, cắt có cấu trúc cho khoảng $8{,}3$ lần, CSR chặn tính toán cho $1{,}0$ lần và CSR chặn băng thông cho khoảng $1{,}66$ lần. Ở $s = 50\%$, CSR chỉ đạt $0{,}2$ lần (chậm hơn 5 lần) khi chặn tính toán và khoảng $0{,}33$ lần khi chặn băng thông.

Đây là mô hình, không phải phép đo. Ghi chú pruning 18 nêu nhận định tương ứng bằng lời: lợi ích tốc độ của sparsity không cấu trúc là “negligible and at times negative” trên GPU có đơn vị nhân ma trận. Ngưỡng

cụ thể (90%, 83%) phụ thuộc kernel, kích thước ma trận và phần cứng; hãy xem chúng như ví dụ, không phải quy tắc.

6.3 Roofline theo cỡ batch

Hình sau cho thời gian mô hình của $y = Wx$ (với B token cùng lúc) của dense và CSR ở ba độ thưa, cùng các giả định ở trên.



Hình 8: Thời gian và tăng tốc theo mô hình roofline của dense và CSR theo cỡ batch (minh hoạ, tự tính)

Cách đọc hình: bên trái, đường đen là dense, đường nét đứt là CSR ở độ thưa 80%, 90% và 95%; đường chấm đứng đánh dấu $B \approx 150$. Với B nhỏ, tất cả đều nằm ngang (chặn bởi bộ nhớ, thời gian theo số byte), và các đường CSR thấp hơn đường đen vì ít byte hơn, miễn độ thưa đủ lớn. Từ $B \approx 150$ trở đi mọi đường tăng theo B (chặn bởi tính toán). Bên phải là tăng tốc của CSR: với độ thưa 90% khoảng 1,66 lần ở $B = 1$ rồi tụt về 1,0 ở B lớn; với 95% từ 3,3 lần về 2,0; với 80% luôn dưới 1 (0,83 rồi 0,5) (tự tính). Điểm cần nhớ: chỗ nào bị chặn bởi bộ nhớ, giảm byte mới giúp; chỗ nào chặn bởi tính toán, hiệu suất kernel thưa mới quyết định.

6.4 Đừng nhầm FLOPs với tốc độ

Từ mục 6.1 đến 6.3 ta thấy hai điều. FLOPs lý thuyết là cận trên rất lỏng cho tăng tốc của sparsity không cấu trúc. Và tăng tốc phụ thuộc vào chế độ sử dụng (batch nhỏ hay lớn). Theo ghi chú pruning 12, nhiều bài báo báo "speedup" bằng tỉ lệ FLOPs, không phải thời gian đo, và nhóm tác giả chỉ ra hai metric là số tham số và FLOPs chỉ là proxy lỏng cho latency, throughput, bộ nhớ và năng lượng thực tế. Ghi chú pruning 19 (HALP) đi theo hướng ngược lại: đưa độ trễ đo được vào ràng buộc cắt tỉa.

7. Nối sang pruning và LLM

Tại sao pruning 04 trở đi chuyển sang cắt có cấu trúc. Xoá cả hàng, cột hoặc filter làm ma trận dense nhỏ lại, nên mọi kernel dense có sẵn chạy nhanh hơn ngay, như đường xanh lơ ở hình mục 6.2. Cái giá là chất lượng: cắt thô làm mất nhiều hơn cắt từng trọng số ở cùng độ thưa.

2:4 là điểm giữa. Mẫu 2:4 (pruning 18) cố định cấu trúc nhỏ đến mức chỉ tốn 2 bit metadata mỗi giá trị, đủ đều để Sparse Tensor Core xử lý, và đủ mịn để giữ chất lượng. Theo ghi chú pruning 18, tốc độ thực phụ thuộc kích thước GEMM: GEMM lớn tiến gần 2 lần, còn mô hình hidden nhỏ hoặc batch nhỏ không đạt 2 lần. Ghi chú đó cũng cảnh báo rằng khi suy luận từng token ở batch 1 (GEMV), lợi ích chính của 2:4 có thể là giảm khoảng 38 đến 44% dung lượng và băng thông trọng số chứ không phải gấp đôi tính toán, đúng với chế độ chặn bộ nhớ ở hình mục 6.3.

LLM suy luận thường ở vùng chặn bộ nhớ. Khi sinh token ở batch nhỏ, thời gian gần bằng số byte trọng số chia băng thông. Nén bộ nhớ (lượng tử hoá, 2:4, cắt cấu trúc) có lợi tức thời, còn sparsity không cấu trúc ở 50% vì CSR còn tốn thêm byte chỉ số nên thường không có lợi nếu không có kernel chuyên dụng.

Ở huấn luyện và prefill batch lớn, vùng chặn tính toán. Lúc đó hiệu suất kernel thưa mới là yếu tố, và lõi nhân ma trận dense thường thắng; theo ghi chú pruning 24, Wanda báo cáo chất lượng (perplexity) ở độ thưa 50% không cấu trúc và cả mẫu 2:4; chuyện tăng tốc thật phụ thuộc kernel và định dạng, như mục 6 đã phân tích.

Nén bộ nhớ là một mục tiêu độc lập. Dù không nhanh hơn, mô hình nhỏ hơn có thể vừa một GPU nhỏ hơn hoặc chạy trên thiết bị biên. Đây là động cơ ban đầu của Han 2015 (pruning 02).

8. Sai lầm hay gặp

- **"Thưa 50% là nhanh gấp 2 lần."** Đó là FLOPs lý thuyết; với CSR ở 50% thưa trên GPU, mô hình ở mục 6.2 cho chậm đi (khoảng $0{,}2$ đến $0{,}33$ lần), và paper pruning 18 mô tả lợi ích là không đáng kể hoặc âm.
- **Quên chi phí chỉ số.** Mỗi phần tử CSR tốn $b_v + b_i$ byte, nên với FP16 và int32 cần thưa trên $66{,}7\%$ mới nhỏ hơn dense. Ma trận nhỏ còn phải cộng con trỏ hàng (ví dụ 4×6 ở độ thưa 75% vẫn tốn 56 byte so với 48).
- **Dùng thưa không cấu trúc cho suy luận batch nhỏ mà không có kernel chuyên dụng.** Truy cập bộ nhớ gián tiếp làm giảm băng thông hiệu dụng, đúng hệ số κ trong mô hình.
- **So sánh "speedup" mà không nói đo gì.** Tỉ lệ FLOPs, thời gian đo trên một GPU, hay thời gian đo trên CPU là ba thứ khác nhau (theo ghi chú pruning 12).
- **Quên rằng số liệu tăng tốc phụ thuộc kích thước ma trận và batch.** Kết luận ở $B = 1$ có thể đảo ngược ở B lớn (hình roofline).
- **Đo kích thước mô hình bằng số trọng số khác 0.** Kích thước thật là số byte trên đĩa hoặc trong bộ nhớ GPU, gồm cả metadata.

9. Bảng tóm tắt

Đại lượng	Công thức và ý nghĩa
Bộ nhớ dense	$m\,n\,b_v$: không phụ thuộc độ thưa
Bộ nhớ COO	$z\,(b_v + 2b_i)$: mỗi phần tử một bộ ba
Bộ nhớ CSR	$z\,(b_v + b_i) + (m+1)\,b_i$: giá trị, chỉ số cột, con trỏ hàng
Hoà vốn CSR (ma trận lớn)	$s > \dfrac{b_i}{b_v + b_i}$: bằng $66{,}7\%$ với FP16 và int32, 50% với FP16 và int16
Mẫu 2:4, FP16	$2 \cdot 16 + 2 \cdot 2 = 36$ bit trên 64 bit của dense, tiết kiệm khoảng 44%
FLOPs của phép nhân ma trận-vector	$2mn$; giảm theo $1/(1-s)$ chỉ về lý thuyết
Roofline	$t \approx \max(\text{FLOPs}/P,\ \text{byte}/BW)$; ngưỡng batch $B \approx P/BW \cdot b_v/2$

10. Bài tập tự tính

Các bài là phép tính độc lập, không phải số liệu paper hay phép đo. Mỗi đáp án đã kiểm bằng numpy.

Bài 1. Dense và CSR của ma trận nhỏ

Ma trận 3×4 FP16 ($b_v = 2$) với chỉ số int32 ($b_i = 4$) có đúng 3 phần tử khác 0, ví dụ $\begin{pmatrix} 0 & 7 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 2 \end{pmatrix}$. Tính bộ nhớ dense, CSR và COO.

Đáp án (tự tính): Dense $12 \cdot 2 = 24$ byte. CSR $3 \cdot 6 + (3+1) \cdot 4 = 18 + 16 = 34$ byte. COO $3 \cdot 10 = 30$ byte. Cả hai đều lớn hơn dense dù độ thưa là 75%.

Bài 2. Hoà vốn của ma trận bé

Với ma trận ở bài 1, số phần tử khác 0 tối đa để CSR không lớn hơn dense là bao nhiêu và độ thưa tương ứng?

Đáp án (tự tính): Cần $6z + 16 < 24$, tức $z < 1{,}33$, nên $z \leq 1$. Độ thưa tối thiểu $1 - 1/12 \approx 91{,}7\%$.

Bài 3. Ngưỡng hoà vốn theo định dạng số

Hãy tính ngưỡng độ thưa (bỏ qua con trỏ hàng) của CSR khi giá trị là FP32 và chỉ số int32, và khi giá trị là INT8 và chỉ số int16.

Đáp án (tự tính): FP32 và int32: $s > 4/(4+4) = 50\%$. INT8 và int16: $s > 2/(1+2) = 66{,}7\%$. Giá trị càng ít byte thì chỉ số càng chiếm tỉ lệ lớn, nên ngưỡng càng cao.

Bài 4. Ma trận lớn ở độ thưa 90%

Ma trận 4096×4096 FP16 với chỉ số int32 và độ thưa 90%. Tính bộ nhớ CSR theo MiB, tỉ lệ so với dense, và số phần tử khác 0 ở điểm hoà vốn.

Đáp án (tự tính): $z = 1.677.722$, CSR $= 10.082.720$ byte $\approx 9{,}62$ MiB, bằng $30{,}0\%$ dense (32 MiB). Điểm hoà vốn ở $z = 5.589.674$, độ thưa $66{,}83\%$.

Bài 5. Mô hình tăng tốc

Dùng mô hình chặn tính toán của mục 6.2 với $\eta = 0{,}1$: tính tăng tốc của CSR ở độ thưa 80%, 90%, 95%; rồi dùng mô hình chặn băng thông ($\kappa = 0{,}5$, FP16, int32, 4096×4096) để tính tăng tốc ở 90%.

Đáp án (tự tính): Chặn tính toán: $\eta/(1-s)$ cho $0{,}5$, $1{,}0$, $2{,}0$. Chặn băng thông ở 90%: thời gian dense $= 33.554.432/(2 \cdot 10^{12}) \approx 16{,}78\ \mu s$; thời gian CSR $= 10.082.720/(0{,}5 \cdot 2 \cdot 10^{12}) \approx 10{,}08\ \mu s$; tăng tốc $\approx 1{,}66$ lần. Nhắc lại đây là mô hình với tham số giả định, không phải đo.

Tài liệu tham khảo nền: Williams, Waterman và Patterson, Roofline: an insightful visual performance model; sách giáo khoa đại số tuyến tính số về định dạng ma trận thưa CSR và COO; các ghi chú pruning 02, 12, 18 và 19 trong sách.

Nền tảng 03

Độ quan trọng của trọng số: độ lớn, gradient, Hessian và kích hoạt trả lời bốn câu hỏi khác nhau

Loại: chương nền tảng (không phải ghi chú paper)

Đọc trước: chương nền tảng 01 của cuốn này (mặt nạ, saliency), đạo hàm riêng, gradient và Hessian (math nền tảng 06), khai triển Taylor, ma trận nghịch đảo và chuẩn vector (math nền tảng 02 đến 04, 13)

Dùng ở: pruning 01 (OBD/OBS), pruning 02 (độ lớn), pruning 07 (Taylor), pruning 10 (SNIP), pruning 22 (OBC), pruning 23 (SparseGPT), pruning 24 (Wanda), pruning 26 và 28 (tiêu chí cho nhóm và cho kênh)

> Chương này giải thích từ đầu, không phải bản dịch một paper. Các phép tính ghi "tự tính" đã kiểm bằng numpy. Các đường cong sai số là mô phỏng với dữ liệu ngẫu nhiên (seed cố định), không phải đo trên mô hình thật.

1. Vì sao cần một "độ quan trọng"

Độ quan trọng là con số ta gán cho mỗi trọng số để quyết định bỏ cái nào trước. Chương nền 01 cho thấy cắt trọng số này rẻ, cắt trọng số kia đắt. Vấn đề còn lại là: làm sao biết cái nào rẻ mà không thử từng cái?

Đáp án không duy nhất, vì có nhiều cách hiểu "quan trọng". Trọng số lớn thì quan trọng (độ lớn). Trọng số mà loss rất nhạy với nó thì quan trọng (gradient, Taylor). Trọng số mà loss cong mạnh quanh nó thì quan trọng, kể cả khi nó đang gần tối ưu (Hessian, OBD, OBS). Trọng số nhân với đầu vào lớn thì quan trọng (Wanda). Bốn tiêu chí này thường chọn ra các tập trọng số khác nhau khi áp dụng trên cùng một lớp. Trong LLM, chọn tiêu chí nào là khác biệt giữa mô hình còn dùng được và mô hình hỏng: theo ghi chú pruning 24, trên LLaMA-7B ở độ thưa 50%, perplexity của mô hình đầy đủ là $5{,}68$, cắt độ lớn cho $17{,}29$ còn Wanda cho $7{,}26$.

2. Nhắc nền: ba công cụ toán

2.1 Taylor một chiều

Với hàm mất mát một biến $L(t)$ và bước nhảy δ, khai triển Taylor quanh t_0:

$$L(t_0+\delta) \approx L(t_0) + g\,\delta + \frac{1}{2}\,h\,\delta^2, \qquad g = L'(t_0),\ h = L''(t_0).$$

Số hạng bậc một $g\delta$ nói mất mát đổi bao nhiêu nếu đi theo độ dốc; số hạng bậc hai $\frac{1}{2}h\delta^2$ nói độ cong làm thêm bao nhiêu. Cắt trọng số w nghĩa là $\delta = -w$, nên $\Delta L \approx -g\,w + \frac{1}{2}h\,w^2$. Ví dụ (tự tính) với $L(t) = \big(e^{1{,}5(t-0{,}3)} - 1{,}5(t-0{,}3) - 1\big)/2{,}25$ và $t_0 = 0{,}5$: $L(0{,}5) = 0{,}0222$, $g = 0{,}2332$, $h = 1{,}3499$. Cắt về 0: mất mát thật tăng $0{,}0168$, xấp xỉ bậc một cho $-0{,}1166$, xấp xỉ bậc hai cho $+0{,}0521$.

2.2 Gradient và Hessian nhiều chiều

Với nhiều trọng số $w \in \mathbb{R}^d$, gradient $g_i = \partial L/\partial w_i$ và Hessian $H_{ij} = \partial^2 L/\partial w_i \partial w_j$ là ma trận đối xứng các đạo hàm bậc hai. Hessian đường chéo H_{ii} đo độ cong theo từng trọng số riêng; các phần tử ngoài đường chéo H_{ij} đo hai trọng số "kéo" loss cùng hướng ra sao. Hai trọng số có H_{ij} lớn là hai trọng số có thể bù cho nhau.

2.3 Mất mát tái tạo của một lớp tuyến tính

Cho lớp $z = wx$ (một hàng w của W) và ma trận kích hoạt X cỡ $d \times N$ (cột là N mẫu hay token, hàng j là kênh đầu vào j). Nếu thay w bằng w' thì sai lệch đầu ra là $(w' - w)X$ và mất mát tái tạo là

$$L(w') = \|(w' - w)X\|_2^2 = (w' - w)\, M\, (w' - w)^\top, \qquad M = XX^\top.$$

Đây là hàm bậc hai chính xác theo w', nên Taylor bậc hai là chính xác và gradient tại w bằng 0 (mất mát tái tạo bằng 0 ngay tại w). Ghi chú pruning 22 viết cùng thứ này với $H = 2XX^\top$ để $L = \frac{1}{2}\delta^\top H \delta$; chương này dùng $M = XX^\top$ cho gọn, và $H = 2M$. Ví dụ nhỏ (tự tính): hai kênh với $X = \begin{pmatrix} 2 & 1 & 0 \\ 2 & 0 & 1 \end{pmatrix}$ cho $M = \begin{pmatrix} 5 & 4 \\ 4 & 5 \end{pmatrix}$.

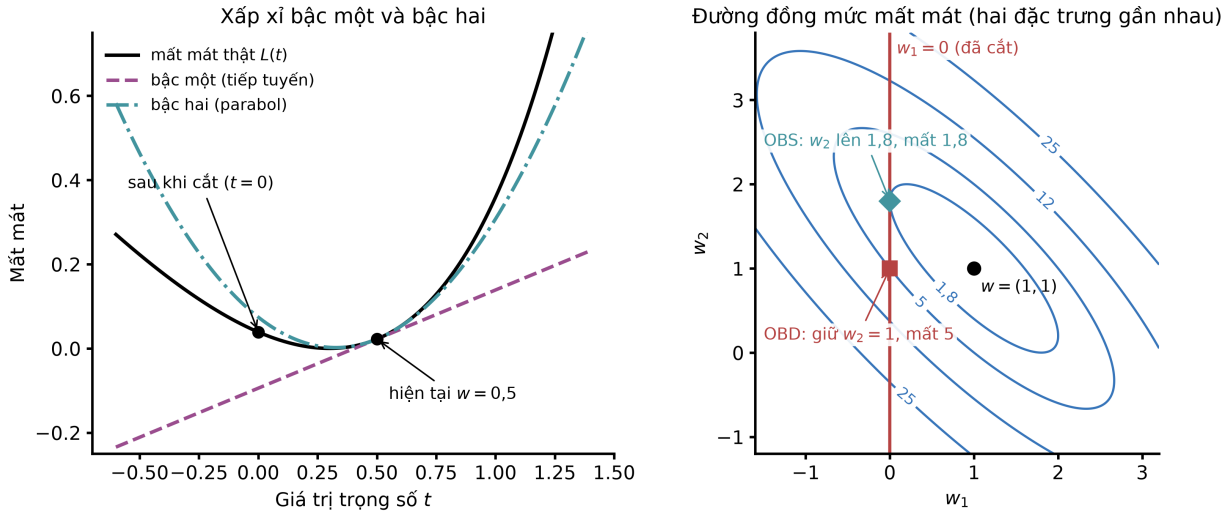
3. Trực giác bằng hình: mỗi tiêu chí nhìn gì

3.1 Độ lớn: nhìn riêng từng trọng số

Tiêu chí độ lớn chấm điểm $|w_{ij}|$ và bỏ những trọng số nhỏ nhất. Trực giác: nếu $w_{ij} \approx 0$ thì $w_{ij}x_j \approx 0$ với đầu vào cỡ bình thường, nên bỏ đi hầu như không đổi đầu ra. Nó không cần dữ liệu, không cần gradient, nên rẻ nhất. Nó giả định mọi đầu vào cùng cỡ, và giả định đó là điểm yếu (mục 3.4).

3.2 Gradient và Taylor bậc một: nhìn độ dốc của loss

Taylor bậc một cho $\Delta L \approx -g_i w_i$, nên điểm quan trọng thường dùng là $|g_i w_i|$ (hoặc bình phương $(g_i w_i)^2$, cùng thứ hạng; theo ghi chú pruning 07 đây là dạng $I^{(1)} = (gw)^2$ của Molchanov và cộng sự). Điểm này lớn khi **vừa** w lớn **vừa** loss nhạy với w.



Hình 9: Trái: xấp xỉ bậc một (tiếp tuyến) và bậc hai (parabol) của mất mát một chiều. Phải: đường đồng mức mất mát của hai trọng số và hai cách xử lý khi cắt w_1 (minh hoạ, tự tính)

Cách đọc hình, panel trái: đường đen là mất mát thật $L(t)$, đường tím nét đứt là tiếp tuyến tại điểm hiện tại $w = 0{,}5$ (chỉ biết độ dốc), đường xanh lơ chấm gạch là parabol (biết cả độ cong). Điểm đen trái là sau khi cắt ($t = 0$). Tiếp tuyến cho mất mát **giảm** (xuống dưới 0) trong khi mất mát thật **tăng** một chút; parabol gần hơn. Hai bài học: bậc một đúng khi bước nhảy nhỏ và loss gần tuyến tính, và nó có thể sai cả dấu khi bước nhảy (ở đây bằng chính w) không nhỏ. Vì vậy người ta lấy giá trị tuyệt đối hoặc bình phương, xem đó là "độ nhạy" thay vì dự đoán chính xác ΔL.

Cảnh báo quan trọng: tại một điểm cực tiểu thật sự, $g = 0$ và mọi điểm bậc một đều bằng 0. Taylor bậc một chỉ có ích khi gradient khác 0, ví dụ trung bình trên một mini-batch, hoặc ở khởi tạo (SNIP, pruning 10).

3.3 Hessian: nhìn độ cong và khả năng bù

Khi loss có độ cong, đường đồng mức là các elip (panel phải của hình trên). Hãy đọc panel phải: điểm đen là $w = (1, 1)$, tâm của các elip; đường dọc đỏ là $w_1 = 0$, tức mặt "đã cắt". Elip nghiêng vì hai đặc trưng gần cùng hướng ($M = \begin{pmatrix} 5 & 4 \\ 4 & 5 \end{pmatrix}$): thay đổi w_1 có thể được bù bằng cách đổi w_2 theo hướng ngược lại.

Ô vuông đỏ là cách **OBD** (Optimal Brain Damage, pruning 01): đặt $w_1 = 0$ và **giữ nguyên** $w_2 = 1$, điểm đó nằm trên elip mức 5, tức thiệt hại 5. Kim cương xanh lơ là cách **OBS** (Optimal Brain Surgeon): đặt $w_1 = 0$ rồi **chỉnh** w_2 lên $1{,}8$, điểm đó nằm trên elip mức $1{,}8$, thiệt hại chỉ $1{,}8$ (tự tính). Điểm kim cương là điểm thấp nhất của mất mát trên đường $w_1 = 0$. Khoảng cách giữa hai đường đồng mức 5 và $1{,}8$ chính là phần thiệt hại mà việc **bù** cứu được; elip càng dẹt (đặc trưng càng gần nhau) thì phần cứu được càng lớn.

3.4 Kích hoạt: nhìn xem trọng số nhân với cái gì

Wanda (Weights AND Activations, pruning 24) chấm điểm $|w_{ij}| \cdot \|X_j\|_2$: độ lớn trọng số nhân với chuẩn của kênh đầu vào mà nó nhân. Hình sau mô phỏng một hàng 16 trọng số với 16 kênh đầu vào, trong đó 3 kênh (số 2, 7 và 12) có kích hoạt lớn gấp khoảng 12 đến 20 lần các kênh còn lại. Đây là mô phỏng numpy (seed 3, $N = 256$ mẫu), mượn hiện tượng "kênh ngoại lai" của LLM (ghi chú pruning 24 nhắc tới) nhưng không phải dữ liệu thật.

Cách đọc hình: ba hàng biểu đồ cột chia sẻ trục kênh. Hàng trên: độ lớn $|w_j|$, tô xanh các trọng số được giữ (8 lớn nhất) và đỏ các trọng số bị cắt. Hàng giữa: chuẩn kích hoạt $\|x_j\|_2$; ba cột vàng là kênh ngoại lai (khoảng 191, 326 và 240 so với khoảng 16 của các kênh còn lại). Hàng dưới: điểm Wanda. Trọng số của kênh 7 chỉ $0{,}015$ nên độ lớn cắt nó, nhưng nhân với 326 nó có điểm khoảng $5{,}4$, vẫn nằm trong nhóm 8 điểm lớn nhất; Wanda giữ cả ba kênh ngoại lai, còn độ lớn chỉ giữ kênh 2. Sai số tương đối của đầu ra là $0{,}20$ với độ lớn và $0{,}07$ với Wanda (tự tính).

4. Định nghĩa chính thức

Cho một lớp tuyến tính $W \in \mathbb{R}^{m \times n}$, ma trận kích hoạt $X \in \mathbb{R}^{n \times N}$ và $M = XX^\top$. Bốn tiêu chí chấm điểm S_{ij} (điểm thấp thì bị cắt trước):

Độ lớn (pruning 02):

$$S_{ij} = |w_{ij}|.$$

Taylor bậc một (pruning 07), với $g_{ij} = \partial L / \partial w_{ij}$ là gradient của hàm mất mát nhiệm vụ:

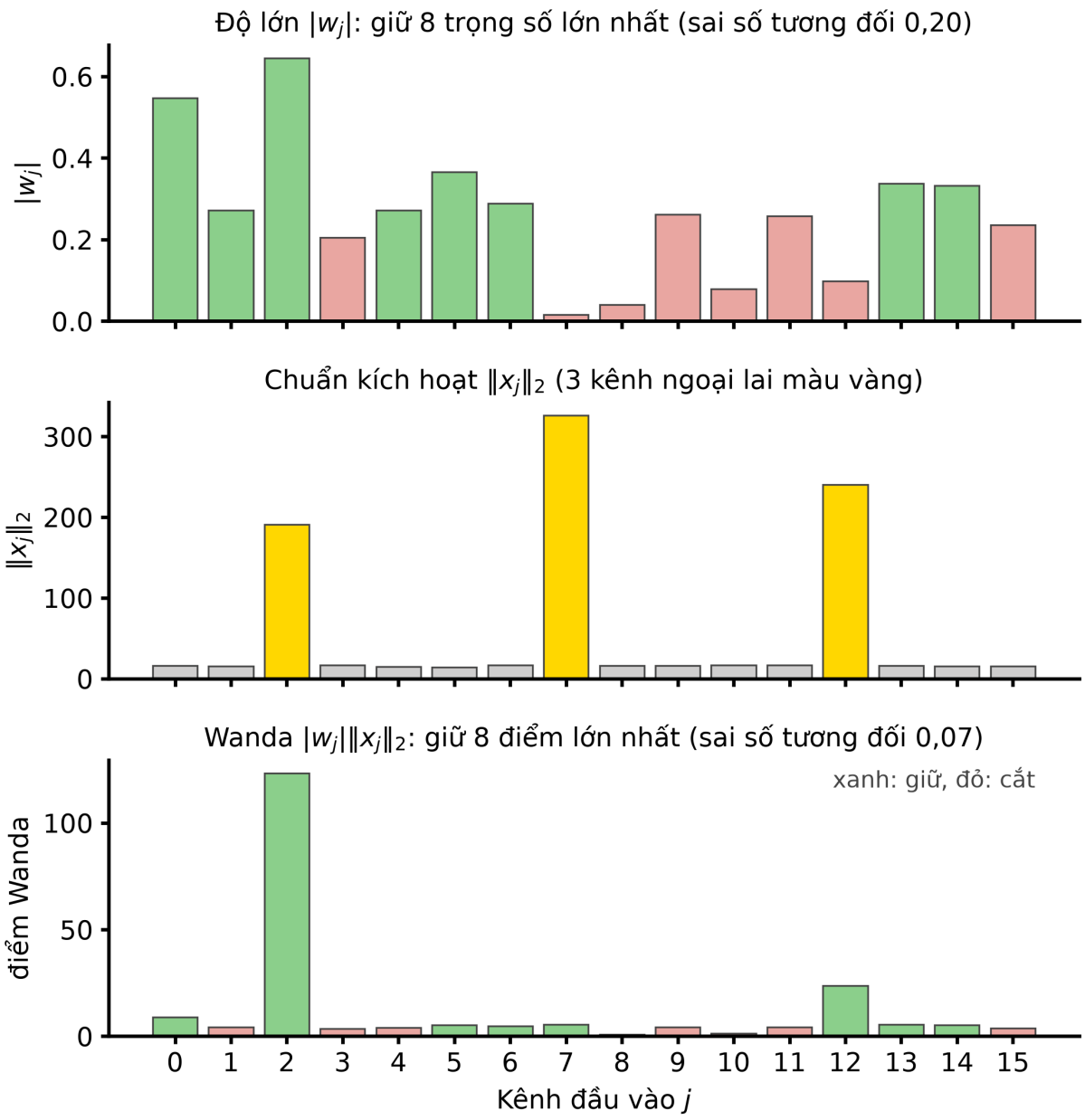
$$S_{ij} = |g_{ij}\, w_{ij}| \quad \text{(hoặc } (g_{ij} w_{ij})^2\text{)}.$$

OBD, bậc hai đường chéo (pruning 01): với H là Hessian của mất mát,

$$S_{ij} = \frac{1}{2} H_{ii}\, w_{ij}^2.$$

OBS, bậc hai đầy đủ có bù (pruning 01, 22): điểm và vector bù cho các trọng số còn lại trong cùng hàng là

$$S_q = \frac{w_q^2}{2\,[H^{-1}]_{qq}}, \qquad \delta = -\frac{w_q}{[H^{-1}]_{qq}}\, H^{-1} e_q,$$



Hình 10: Một hàng 16 trọng số: tiêu chí độ lớn cắt các kênh ngoại lai, Wanda giữ chúng (minh hoạ, tự tính)

với e_q là vector đơn vị thứ q. Với mất mát tái tạo ở mục 2.3 và $H = 2M$, điểm OBS trở thành $w_q^2/[M^{-1}]_{qq}$ và độ tăng mất mát sau khi bù chính xác bằng điểm đó.

Wanda (pruning 24):

$$S_{ij} = |w_{ij}| \cdot \|X_j\|_2, \qquad \|X_j\|_2^2 = M_{jj}.$$

Điều đáng chú ý là quan hệ giữa OBD và Wanda. Với mất mát tái tạo, $H_{jj} = 2M_{jj}$ nên điểm OBD là $\frac{1}{2} \cdot 2M_{jj}\, w_{ij}^2 = M_{jj} w_{ij}^2 = \|X_j\|_2^2\, w_{ij}^2$, tức **bình phương điểm Wanda**. Hai tiêu chí cho cùng thứ hạng. Ghi chú pruning 24 nêu quan hệ này với SparseGPT: bỏ phần ngoài đường chéo của Hessian thì điểm SparseGPT thành bình phương điểm Wanda. Vậy Wanda là OBD áp dụng cho mất mát tái tạo từng hàng.

Ngoài điểm số, còn một lựa chọn quan trọng ngang điểm: **nhóm so sánh**. Wanda so sánh trọng số trong từng hàng (mỗi đầu ra), còn độ lớn cổ điển thường so cả lớp hoặc cả mạng; theo ghi chú pruning 24, trên LLaMA-7B ở độ thưa 50% perplexity là $7{,}26$ khi so theo hàng và $7{,}95$ khi so theo lớp.

5. Ví dụ tay làm chậm từng bước

5.1 Ma trận 2×2: ba tiêu chí, ba mặt nạ

Lớp có $W = \begin{pmatrix} 0{,}5 & 1 \\ 1{,}5 & -2 \end{pmatrix}$ (hàng là đầu ra, cột là kênh vào). Kích hoạt $X = \begin{pmatrix} 1 & 2 & 2 \\ 1 & 0 & 0 \end{pmatrix}$ với 3 mẫu: kênh 1 có $\|X_1\|_2 = \sqrt{1+4+4} = 3$, kênh 2 có $\|X_2\|_2 = 1$. Gradient của mất mát nhiệm vụ (cho sẵn) là $G = \begin{pmatrix} 3 & 2 \\ 2 & 0{,}5 \end{pmatrix}$. Mục tiêu: cắt một trọng số trong mỗi hàng (độ thưa 50%). Mọi số dưới đây là tự tính.

Bước 1: độ lớn. Điểm là $|W| = \begin{pmatrix} 0{,}5 & 1 \\ 1{,}5 & 2 \end{pmatrix}$. Hàng 1 cắt cột 1 ($0{,}5$ nhỏ nhất); hàng 2 cũng cắt cột 1 ($1{,}5$ nhỏ hơn 2). Mặt nạ cắt **cả cột 1**.

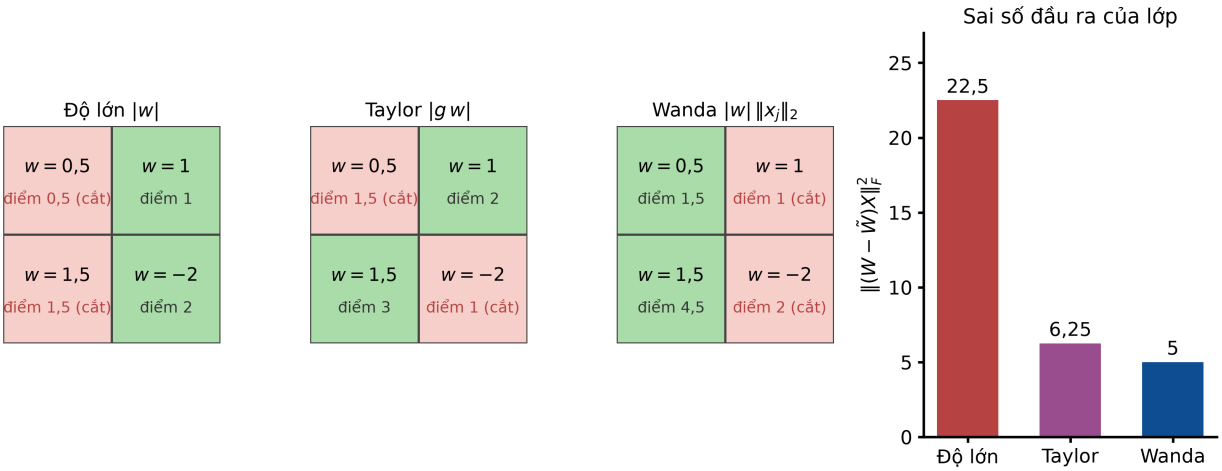
Bước 2: Taylor. Điểm là $|G \odot W| = \begin{pmatrix} 3 \cdot 0{,}5 & 2 \cdot 1 \\ 2 \cdot 1{,}5 & 0{,}5 \cdot 2 \end{pmatrix} = \begin{pmatrix} 1{,}5 & 2 \\ 3 & 1 \end{pmatrix}$. Hàng 1 cắt cột 1 ($1{,}5 < 2$); hàng 2 cắt cột 2 ($1 < 3$). Mặt nạ cắt **đường chéo**.

Bước 3: Wanda. Điểm là $|W|$ nhân từng cột với $(3, 1)$: $\begin{pmatrix} 0{,}5 \cdot 3 & 1 \cdot 1 \\ 1{,}5 \cdot 3 & 2 \cdot 1 \end{pmatrix} = \begin{pmatrix} 1{,}5 & 1 \\ 4{,}5 & 2 \end{pmatrix}$. Hàng 1 cắt cột 2 ($1 < 1{,}5$); hàng 2 cắt cột 2 ($2 < 4{,}5$). Mặt nạ cắt **cả cột 2**.

Cách đọc hình: ba bảng 2×2 là ba mặt nạ; ô xanh được giữ, ô đỏ bị cắt, mỗi ô ghi giá trị w và điểm của tiêu chí đó. Cột bên phải là sai số đầu ra $\|(W - \tilde{W})X\|_F^2$ của từng mặt nạ.

Bước 4: đo sai số đầu ra. Độ lớn cắt $0{,}5$ và $1{,}5$ ở cột 1: sai lệch $(W - \tilde{W})X$ có hàng 1 bằng $0{,}5 \cdot (1, 2, 2)$, hàng 2 bằng $1{,}5 \cdot (1, 2, 2)$; bình phương tổng là $0{,}25 \cdot 9 + 2{,}25 \cdot 9 = 22{,}5$. Wanda cắt 1 và -2 ở cột 2: hàng 1 bằng $1 \cdot (1, 0, 0)$, hàng 2 bằng $-2 \cdot (1, 0, 0)$; tổng $1 + 4 = 5$. Taylor cắt $0{,}5$ ở (1,1) và -2 ở (2,2): $0{,}25 \cdot 9 + 4 \cdot 1 = 6{,}25$.

Độ lớn tệ nhất ($22{,}5$) vì nó xoá toàn bộ kênh 1, kênh có kích hoạt lớn gấp ba. Wanda tốt nhất (5) vì mục tiêu của nó **chính là** sai số này: điểm của từng trọng số là căn của $w^2\|X_j\|^2$, thiệt hại riêng nếu cắt trọng số đó. Taylor ở giữa ($6{,}25$) nhưng đây là so sánh không công bằng: Taylor tối ưu mất mát nhiệm vụ thông qua gradient G, không phải sai số tái tạo. Bảng này chỉ cho thấy **các tiêu chí chọn khác nhau**; tiêu chí nào tốt hơn phụ thuộc vào đại lượng bạn thực sự muốn giữ.



Hình 11: Ba tiêu chí chọn ba mặt nạ khác nhau cho cùng một ma trận 2x2, và sai số đầu ra của lớp (minh hoạ, tự tính)

5.2 Một hàng 3 trọng số: OBD và OBS chọn khác nhau

Ví dụ 2×2 không phân biệt được OBD với OBS: với chỉ hai trọng số trong hàng, thứ hạng của OBS và OBD luôn trùng (tự tính, vì hệ số "bù được" của hai kênh trùng nhau). Cần ít nhất ba trọng số. Xét một hàng $w = (1, 3, -2)$ trên ba kênh với kích hoạt (bốn mẫu)

$$X = \begin{pmatrix} 2 & 1 & 2 & 1 \\ 0 & 0 & 0 & 1 \\ 1 & 0 & 1 & 1 \end{pmatrix}.$$

Bước 1: ma trận $M = XX^\top$. Các tích vô hướng giữa hàng của X: $M_{11} = 4 + 1 + 4 + 1 = 10$, $M_{12} = 1$, $M_{13} = 2 + 0 + 2 + 1 = 5$, $M_{22} = 1$, $M_{23} = 1$, $M_{33} = 1 + 0 + 1 + 1 = 3$. Vậy $M = \begin{pmatrix} 10 & 1 & 5 \\ 1 & 1 & 1 \\ 5 & 1 & 3 \end{pmatrix}$.

Bước 2: định thức. $\det M = 10(1 \cdot 3 - 1 \cdot 1) - 1(1 \cdot 3 - 1 \cdot 5) + 5(1 \cdot 1 - 1 \cdot 5) = 10 \cdot 2 + 2 - 20 = 2$.

Bước 3: đường chéo của M^{-1} bằng đồng thừa số. $[M^{-1}]_{11} = (1 \cdot 3 - 1 \cdot 1)/2 = 1$; $[M^{-1}]_{22} = (10 \cdot 3 - 5 \cdot 5)/2 = 5/2$; $[M^{-1}]_{33} = (10 \cdot 1 - 1 \cdot 1)/2 = 9/2$.

Bước 4: ba điểm.

- Độ lớn: $|w| = (1, 3, 2)$, nhỏ nhất là w_1.
- OBD: $M_{jj} w_j^2 = (10 \cdot 1,\ 1 \cdot 9,\ 3 \cdot 4) = (10, 9, 12)$, nhỏ nhất là w_2.
- OBS: $w_q^2 / [M^{-1}]_{qq} = (1/1,\ 9/2{,}5,\ 4/4{,}5) = (1;\ 3{,}6;\ 0{,}889)$, nhỏ nhất là w_3.

Ba tiêu chí chọn ba trọng số khác nhau.

Bước 5: bù khi cắt w_3. Cột thứ ba của M^{-1} là $(-4, -5, 9)/2 = (-2; -2{,}5; 4{,}5)$. Vector bù $\delta = -\dfrac{w_3}{[M^{-1}]_{33}} M^{-1} e_3 = \dfrac{2}{4{,}5} \cdot (-2; -2{,}5; 4{,}5) = \left(-\dfrac{8}{9}, -\dfrac{10}{9}, 2\right)$. Trọng số mới là $w + \delta = (1/9,\ 17/9,\ 0)$. Thành phần thứ ba bằng $-2 + 2 = 0$ đúng như yêu cầu. Mất mát tái tạo sau khi bù là $\delta M \delta^\top = 8/9 \approx 0{,}889$, đúng bằng điểm OBS.

Bước 6: so với không bù. Nếu cắt w_3 mà không chỉnh gì, thiệt hại là $M_{33} w_3^2 = 12$ (nhiều gấp 13,5 lần). Nếu

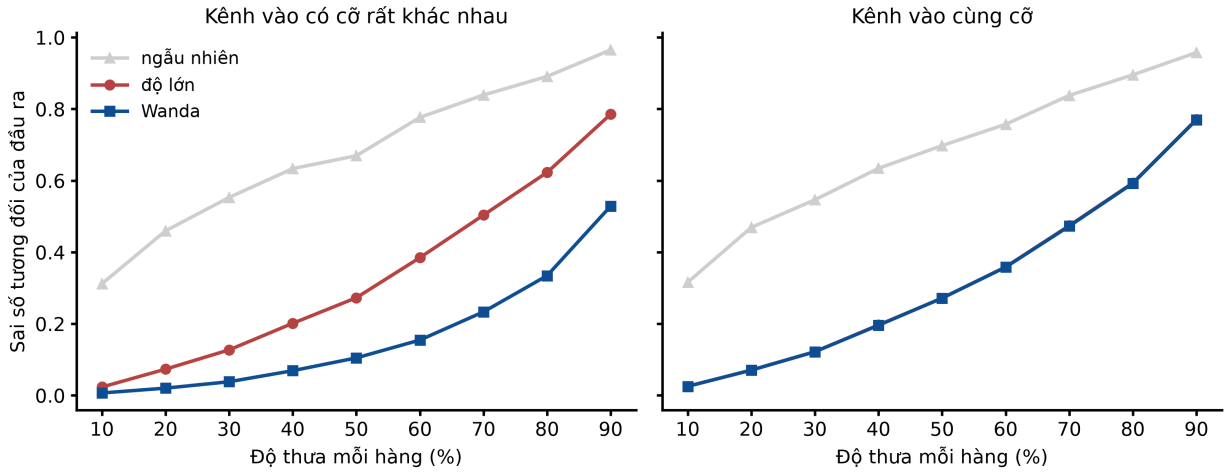
cắt w_2 như OBD chọn rồi bù, thiệt hại là $3{,}6$; nếu cắt w_1 (độ lớn chọn) rồi bù, thiệt hại là $1{,}0$ và trọng số mới là $(0, 2, 0)$. Cả ba đều được kiểm lại bằng bình phương tối thiểu trực tiếp (tự tính).

Điều xảy ra: kênh 1 và kênh 3 gần cùng hướng ($X_1 = (2, 1, 2, 1)$ và $X_3 = (1, 0, 1, 1)$), nên bỏ w_3 rồi tăng w_1 và giảm w_2 bù lại gần như không để lại dấu vết. OBD không nhìn thấy khả năng bù (chỉ đường chéo), nên đánh giá w_3 đắt nhất (12). OBS nhìn thấy và đánh giá rẻ nhất.

6. Biến thể và chi tiết hay bị bỏ qua

6.1 Wanda hơn độ lớn bao nhiêu và khi nào

Hình sau là mô phỏng: lớp 64×64 với trọng số Gaussian, cắt theo từng hàng ở độ thưa từ 10% đến 90%, đo sai số tương đối của đầu ra $\|(\tilde{W} - W)X\| / \|WX\|$ trên 512 mẫu, so ngẫu nhiên, độ lớn và Wanda. Bên trái các kênh vào có cỡ rất khác nhau (độ lệch chuẩn của logarit cỡ kênh khoảng 1, tỉ lệ lớn nhất trên nhỏ nhất khoảng 66 lần); bên phải mọi kênh cùng cỡ. Đây là mô phỏng, không phải mô hình thật.



Hình 12: Sai số tương đối của đầu ra theo độ thưa khi kênh vào khác cỡ (trái) và cùng cỡ (phải) (minh hoạ, tự tính)

Cách đọc hình: đường xám là cắt ngẫu nhiên, đường đỏ là độ lớn, đường xanh là Wanda. Bên trái, Wanda thấp hơn độ lớn ở mọi độ thưa: ở 50%, sai số là $0{,}27$ với độ lớn và $0{,}10$ với Wanda; ở 70% là $0{,}50$ và $0{,}23$; ở 90% là $0{,}79$ và $0{,}53$ (tự tính). Bên phải, kênh cùng cỡ làm $\|X_j\|$ gần như hằng số, nên Wanda và độ lớn gần trùng nhau (hai đường chồng khít, độ lớn nằm dưới đường xanh nên khó thấy). Hiệu quả của Wanda đến từ **sự không đều** của các kênh đầu vào: không có sự không đều thì không có lợi.

6.2 Điểm không chỉ cho từng trọng số

Với cắt có cấu trúc (xoá nguyên kênh, nguyên đầu attention), ta cần điểm cho cả nhóm. Cách phổ biến là gộp điểm của các trọng số trong nhóm. Theo ghi chú pruning 07 có hai cách gộp cho Taylor: bình phương của tổng $\left(\sum_s g_s w_s\right)^2$, hoặc tổng của các bình phương $\sum_s (g_s w_s)^2$. Các bài cắt LLM có cấu trúc (pruning 26, 28) cũng gộp điểm theo gradient hoặc kích hoạt cho nhóm. Điểm nhóm cộng dồn nên **thiên về nhóm lớn**, cần chuẩn hoá khi so sánh nhóm khác kích thước.

6.3 Gradient từ đâu

Gradient g_i trong Taylor phụ thuộc vào dữ liệu và hàm mất mát: gradient của loss huấn luyện trên một batch, trung bình nhiều batch, hay bình phương gradient (xấp xỉ Fisher, theo ghi chú pruning 07 điểm bậc một bằng phương sai gradient nếu giả sử kỳ vọng gradient bằng 0). Một batch cho điểm nhiễu; trung bình nhiều batch ổn định hơn nhưng tốn hơn. Với LLM, gradient trên toàn mô hình rất đắt về bộ nhớ, nên Wanda (chỉ cần một lượt forward) là lựa chọn rẻ.

6.4 Hessian xấp xỉ và hệ số làm mượt

Hessian đầy đủ cỡ $n \times n$ cho mỗi hàng; với $n = 4096$ là hàng triệu phần tử và nghịch đảo tốn n^3. Cách làm thực dụng: $H = 2XX^\top$ dùng chung cho mọi hàng của lớp (ghi chú pruning 22), cộng một hệ số nhỏ λ lên đường chéo để nghịch đảo ổn định khi M gần suy biến, và cập nhật nghịch đảo từng bước khi cắt dần (OBC, SparseGPT). Khi số mẫu N nhỏ hơn số kênh, M có hạng thấp và không khả nghịch nếu thiếu λ.

6.5 Bù và không bù

OBD và Wanda chỉ cho điểm: sau khi cắt, các trọng số còn lại giữ nguyên. OBS và SparseGPT còn cập nhật các trọng số còn lại để bù. Theo ghi chú pruning 24, với Wanda việc thêm cập nhật bậc hai kiểu SparseGPT gần như không giúp ở 50% (perplexity $7{,}26$ so với $7{,}32$ hoặc $7{,}26$), giúp nhẹ ở 2:4 ($11{,}53$ xuống $10{,}89$), nhưng giúp rất nhiều ở 70% ($84{,}50$ xuống $29{,}65$, vẫn kém xa mô hình đầy đủ $5{,}68$). Càng cắt sâu, bù càng quan trọng.

7. Nối sang pruning và LLM

OBD và OBS (pruning 01) là gốc của mọi tiêu chí bậc hai. Chúng định nghĩa saliency là mức tăng mất mát và lần đầu cho công thức $w_q^2/(2[H^{-1}]_{qq})$ cùng vector bù. Khó khăn thực tế là Hessian quá lớn; hầu hết bài sau là cách làm cho nó rẻ.

Han 2015 (pruning 02) chọn độ lớn vì nó rẻ, và dùng huấn luyện lại để bù thiếu chính xác của điểm số. Quy trình đó thành công với CNN cỡ AlexNet và VGG-16.

Molchanov (pruning 07) kiểm chứng tiêu chí bằng "oracle". Theo ghi chú pruning 07, họ đo tương quan giữa điểm số và mức thay đổi loss thật khi tắt từng neuron, và nhận định độ lớn trọng số là chỉ báo kém hơn các điểm dựa trên gradient. Đây là cách tiêu chuẩn để so tiêu chí: không bằng cảm giác mà bằng tương quan với thiệt hại thật.

OBC và SparseGPT (pruning 22, 23) là OBS làm cho đủ rẻ. Chúng khai thác việc $H = 2XX^\top$ giống nhau cho mọi hàng của một lớp và cập nhật nghịch đảo thay vì nghịch đảo lại.

Wanda (pruning 24) là OBD của mất mát tái tạo từng hàng. Như đã chứng minh ở mục 4, điểm Wanda bình phương bằng điểm OBD với Hessian đường chéo. Theo ghi chú pruning 24, trên LLaMA-7B ở 50% Wanda đạt perplexity $7{,}26$ sát SparseGPT $7{,}22$, với chi phí tính điểm $0{,}54$ giây so với $203{,}1$ giây, dù không có bước cập nhật nào. Lý do có thể là **kênh ngoại lai** của LLM: theo ghi chú đó, nhiều chiều của hidden state lớn gấp khoảng 100 lần chiều thường, đúng cảnh mô phỏng ở mục 3.4.

Cùng điểm số, nhóm so sánh khác nhau cho kết quả khác nhau. Theo ghi chú pruning 24, so theo hàng thắng so theo lớp ở LLM, nhưng ở bộ phân loại ảnh (ConvNeXt, DeiT) xu hướng này không xuất hiện: đừng coi nó là quy luật chung.

8. Sai lầm hay gặp

- **"Độ lớn là tiêu chí cơ sở, đủ tốt ở mọi nơi."** Độ lớn bỏ qua kích hoạt. Ở mục 5.1, nó xoá cả một kênh lớn gấp ba; ở LLM với kênh ngoại lai, perplexity bị đẩy từ $5{,}68$ lên $17{,}29$ ở độ thưa 50% (theo ghi chú pruning 24).
- **Dùng điểm Taylor tại một cực tiểu.** Gradient bằng 0 thì điểm bậc một bằng 0 mọi nơi. Phải dùng gradient trên một batch cụ thể hoặc chuyển sang điểm bậc hai.
- **Nhầm OBD với OBS.** OBD chỉ nhìn đường chéo và không bù; OBS dùng H^{-1} đầy đủ và bù. Ví dụ 5.2: OBD chọn w_2 với thiệt hại thật (sau bù) $3{,}6$, trong khi OBS chọn w_3 với $0{,}889$.
- **Quên rằng điểm là dự đoán, không phải thiệt hại.** Điểm OBD và OBS dựa trên xấp xỉ bậc hai quanh một điểm; khi cắt nhiều trọng số cùng lúc, thiệt hại không cộng được vì hai trọng số có thể tương tác (pruning 01 ghi nhận sai lệch sau khoảng 30%).

- **Trộn các đại lượng không so được.** Điểm Wanda, Taylor, OBS có đơn vị khác nhau; chỉ so trong cùng một tiêu chí và cùng nhóm so sánh.
- **Ghi nhầm tên là "cắt theo kích hoạt" cho mọi phương pháp dùng dữ liệu.** Wanda dùng chuẩn kích hoạt; SparseGPT dùng toàn bộ ma trận $XX^\top$; Taylor dùng gradient. Ba loại thông tin khác nhau.

9. Bảng tóm tắt

Tiêu chí	Điểm và ý nghĩa
Độ lớn (pruning 02)	$\lvert w\rvert$: rẻ nhất, bỏ qua dữ liệu
Taylor (pruning 07)	$\lvert gw\rvert$: nhạy với loss nhiệm vụ, vô dụng nếu $g = 0$
OBD (pruning 01)	$\frac{1}{2}H_{ii}w_i^2$: bậc hai đường chéo, không bù
OBS (pruning 01, 22)	$\dfrac{w_q^2}{2[H^{-1}]_{qq}}$: bậc hai đầy đủ, có vector bù δ
Wanda (pruning 24)	$\lvert w\rvert\,\lVert X_j\rVert_2$: bình phương bằng điểm OBD của mất mát tái tạo
Mất mát tái tạo	$\lVert(w' - w)X\rVert_2^2$ với $M = XX^\top$ và $H = 2M$
Nhóm so sánh	Theo hàng, theo lớp hoặc toàn mạng: quan trọng ngang điểm số

10. Bài tập tự tính

Các bài là phép tính độc lập, không phải số liệu paper. Mỗi đáp án đã kiểm bằng numpy.

Bài 1. Độ lớn và Wanda

Một hàng có $w = (0{,}4; -1{,}2; 0{,}9)$ và chuẩn kích hoạt $\lVert X_j\rVert_2 = (5, 1, 1)$. Cắt đúng một trọng số. Độ lớn chọn trọng số nào, Wanda chọn trọng số nào?

Đáp án (tự tính): Độ lớn $(0{,}4; 1{,}2; 0{,}9)$ cắt w_1. Điểm Wanda $(2; 1{,}2; 0{,}9)$ cắt w_3. Hai tiêu chí chọn khác nhau vì kênh 1 có kích hoạt lớn gấp năm.

Bài 2. Điểm Taylor

Cho $w = (2, -1, 0{,}5)$ và gradient $g = (0{,}1, 3, -4)$. Tính $\lvert g_i w_i\rvert$ và cho biết trọng số nào bị cắt trước theo Taylor, theo độ lớn.

Đáp án (tự tính): $\lvert gw\rvert = (0{,}2; 3; 2)$, Taylor cắt w_1. Độ lớn cắt w_3 $(0{,}5)$. Hai tiêu chí lại khác nhau.

Bài 3. Điểm OBD và OBS cho hai trọng số

Cho $M = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}$ và $w = (1; 0{,}5)$. Tính M^{-1}, điểm OBD $M_{jj}w_j^2$ và điểm OBS $w_j^2/[M^{-1}]_{jj}$ của từng trọng số, rồi vector trọng số mới khi cắt w_2 và bù.

Đáp án (tự tính): $M^{-1} = \frac{1}{3}\begin{pmatrix} 2 & -1 \\ -1 & 2 \end{pmatrix}$, nên $[M^{-1}]_{jj} = 2/3$. OBD $= (2; 0{,}5)$. OBS $= (1{,}5; 0{,}375)$. Cả hai cắt w_2. Vector bù $\delta = -\dfrac{w_2}{[M^{-1}]_{22}}M^{-1}e_2 = -\dfrac{0{,}5}{2/3}\cdot\left(-\frac{1}{3}, \frac{2}{3}\right) = (0{,}25; -0{,}5)$, nên trọng số mới là $(1{,}25; 0)$ và mất mát là $0{,}375$.

Bài 4. Wanda bình phương bằng OBD

Một hàng $w = (2, 1)$ trên hai kênh có $\lVert X_1\rVert_2^2 = 1$ và $\lVert X_2\rVert_2^2 = 9$. Tính điểm Wanda và điểm OBD $M_{jj}w_j^2$ rồi kiểm chứng quan hệ bình phương, và cho biết trọng số nào bị cắt.

Đáp án (tự tính): Wanda $= (2 \cdot 1,\ 1 \cdot 3) = (2, 3)$. OBD $= (1 \cdot 4,\ 9 \cdot 1) = (4, 9)$, bằng bình phương Wanda $(4, 9)$. Cả hai cắt w_1.

Bài 5. So theo hàng và theo lớp

Với $W = \begin{pmatrix} 1 & 1 \\ 4 & 4 \end{pmatrix}$ và $X = I$ (ma trận đơn vị 2×2, mọi kênh cùng chuẩn), cắt 50% số trọng số theo độ lớn: (a) so cả lớp, (b) so theo từng hàng. Tính sai số đầu ra $\|(W - \tilde{W})X\|_F^2$ của mỗi cách.

Đáp án (tự tính): (a) so cả lớp cắt hai ô có $|w| = 1$ (toàn hàng 1): sai số $1 + 1 = 2$. (b) so theo hàng cắt một ô mỗi hàng: hàng 1 mất 1, hàng 2 mất 4, sai số $1 + 16 = 17$. Ở ví dụ này so cả lớp tốt hơn rất nhiều. Điều đó cho thấy kết luận “so theo hàng thắng” của ghi chú pruning 24 là kết quả thực nghiệm cho LLM, không phải quy luật toán học.

Tài liệu tham khảo nền: LeCun, Denker và Solla, Optimal Brain Damage; Hassibi và Stork, Second Order Derivatives for Network Pruning: Optimal Brain Surgeon; Molchanov và cộng sự, Importance Estimation for Neural Network Pruning; Frantar và Alistarh, Optimal Brain Compression; Sun và cộng sự, A Simple and Effective Pruning Approach for Large Language Models; các ghi chú pruning 01, 07, 22, 23 và 24 trong sách.

Nền tảng 04

Cắt hạt, cắt khối và mẫu 2:4: quyết định xoá “những gì” sẽ quyết định bạn được gì

Loại: chương nền tảng (không phải ghi chú paper)

Đọc trước: nhân ma trận với vector và ý nghĩa hàng, cột của ma trận trọng số (math nền 02); chuẩn của vector và ma trận (math nền 13); phần phân loại pruning trong preface 00

Dùng ở: pruning 04 (cắt filter), pruning 05 (cắt kênh theo hệ số BN), pruning 17 (CoFi), pruning 18 (mẫu 2:4), pruning 19 (cắt theo độ trễ), pruning 20 (đồ thị phụ thuộc), pruning 26 (LLM-Pruner), cũng như các bài về Wanda và SparseGPT (pruning 23, pruning 24) khi chúng nói đến mẫu N:M

> Chương này giải thích từ đầu, không phải bản dịch một paper. Các phép tính ghi “tự tính” đã kiểm bằng numpy; các hình ghi “mô phỏng” dùng dữ liệu tự sinh, không phải kết quả đo từ mô hình thật. Con số lấy từ paper luôn có chú “theo ghi chú pruning NN”.

1. Vì sao cần phân biệt các kiểu cắt

Cắt tỉa (pruning) là đặt một số trọng số về 0 rồi mong rằng mô hình nhỏ hơn hoặc nhanh hơn. Nhưng “đặt về 0” chưa chắc đã làm máy tính chạy ít việc hơn, vì máy tính chỉ bỏ qua những phần mà nó được biết trước là có thể bỏ qua.

Hãy hình dung bạn dọn một tủ sách có 32 ngăn. Bạn có thể rút ra từng cuốn rải rác khắp tủ (ngăn nào cũng có chỗ trống, nhưng tủ vẫn to bằng cũ), hoặc dỡ hẳn mỗi hàng ngăn thứ hai (tủ nhỏ lại thật), hoặc quy định “mỗi nhóm 4 ngăn liền nhau phải có đúng 2 cuốn” để người thủ thư biết ngay chỗ nào có sách mà không cần dò. Ba cách này đều làm bớt một nửa số sách, nhưng cho ba kết quả rất khác nhau về độ dễ dùng.

Trong AI, câu hỏi này quyết định việc cắt tỉa có ích hay chỉ đẹp trên giấy. Mô hình ngôn ngữ lớn (LLM) có hàng tỉ trọng số, chủ yếu nằm trong các ma trận lớn của attention và MLP. Việc chạy mô hình chủ yếu là nhân ma trận với vector hoặc ma trận. GPU nhân ma trận dày (dense) rất nhanh vì mọi thứ đều đều và thẳng hàng. Cắt rải rác làm mất sự đều đặn đó. Vì vậy các phương pháp cắt tỉa được chia theo **độ chi tiết của mặt nạ**: từng trọng số, từng nhóm nhỏ, từng hàng hoặc cột, từng kênh, từng đầu attention, hoặc cả một lớp. Độ chi tiết càng thô thì càng dễ tăng tốc nhưng càng dễ làm hỏng chất lượng; càng mịn thì càng giữ được chất lượng nhưng càng khó tăng tốc. Chương này đi chậm qua đánh đổi đó, rồi dành riêng một mục cho mẫu 2:4, một dàn xếp mà phần cứng NVIDIA hỗ trợ trực tiếp.

2. Nhắc nền: ma trận trọng số, mặt nạ và việc “bỏ qua số 0”

2.1 Hàng là nơ-ron đầu ra, cột là kênh đầu vào

Một lớp tuyến tính tính $y = Wx$ với $W \in \mathbb{R}^{R \times C}$, $x \in \mathbb{R}^{C}$, $y \in \mathbb{R}^{R}$. Phần tử $y_i = \sum_j W_{ij} x_j$. Ta gọi **hàng** i của W là “nơ-ron đầu ra i” (nó tạo ra thành phần y_i) và **cột** j của W là “kênh đầu vào j” (nó nhân với x_j). Quy ước này sẽ dùng xuyên suốt: xoá một hàng nghĩa là mất một thành phần ở đầu ra; xoá một cột nghĩa là bỏ qua một thành phần ở đầu vào.

Ví dụ nhỏ (tự tính): $W = \begin{pmatrix} 2 & -1 & 0 \\ 3 & 0 & 1 \end{pmatrix}$, $x = (1, 2, 4)$. Khi đó $y_1 = 2 \cdot 1 + (-1) \cdot 2 + 0 \cdot 4 = 0$ và $y_2 = 3 \cdot 1 + 0 \cdot 2 + 1 \cdot 4 = 7$, tức $y = (0, 7)$. Phép nhân tốn $2 \times 3 = 6$ phép nhân-cộng (MAC). Hai phần tử bằng 0 đóng góp 0, nhưng nếu máy cứ thực hiện đủ 6 MAC thì chúng vẫn tốn công.

2.2 Mặt nạ

Mặt nạ (mask) là ma trận nhị phân $M \in \{0,1\}^{R\times C}$ cùng cỡ với W. Mô hình sau cắt dùng $M \odot W$ (nhân từng phần tử). Phần tử có $M_{ij} = 0$ bị "xoá". **Độ thưa** (sparsity) là tỉ lệ phần tử bị xoá: $s = 1 - \|M\|_0/(RC)$, với $\|M\|_0$ là số phần tử khác 0. Độ thưa 50% nghĩa là nửa số phần tử bằng 0.

2.3 Hai loại tiết kiệm khác nhau

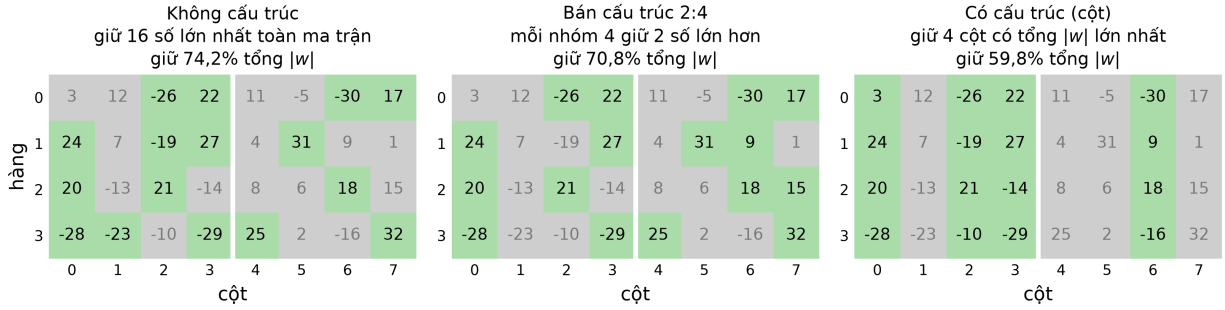
Việc xoá trọng số có thể tiết kiệm hai thứ khác nhau, và hai thứ này không đi cùng nhau:

- **Bộ nhớ:** lưu ít số hơn. Muốn vậy phải có một định dạng lưu trữ không ghi các số 0, kèm thông tin "số còn lại nằm ở đâu" (metadata).
- **Thời gian tính:** làm ít phép nhân hơn. Muốn vậy phần cứng hoặc kernel phải biết bỏ qua các phần tử 0 mà không tốn thêm chi phí tìm kiếm.

Một ma trận đầy đủ 4096 x 4096 nhân với vector trên GPU chạy bằng kernel dày (dense), vốn không hề kiểm tra phần tử nào bằng 0. Nếu bạn chỉ gán 0 cho một nửa số phần tử mà vẫn lưu và nhân như cũ, bộ nhớ và thời gian không đổi. Tiết kiệm chỉ xuất hiện khi **cấu trúc của số 0 khớp với thứ phần cứng biết tận dụng**.

3. Trực giác bằng hình: ba mặt nạ trên cùng một ma trận

Ta dùng một ma trận 4×8 có 32 số nguyên đôi một khác độ lớn (từ 1 đến 32, dấu lẫn lộn), sinh một lần và dùng lại ở mọi ví dụ trong chương. Mục tiêu: xoá đúng 16 phần tử (độ thưa 50%) theo ba kiểu, rồi so sánh phần "độ lớn" còn giữ lại.



Hình 13: Cùng một ma trận 4×8, ba kiểu mặt nạ ở độ thưa 50%: ô xanh là phần tử giữ, ô xám là phần tử bị xoá (minh hoạ, tự tính)

Cách đọc hình: ba bảng là cùng một ma trận với cùng các số; chỉ khác ô nào được giữ. Bảng trái chọn 16 số lớn nhất toàn ma trận nên xanh nằm rải rác, không theo quy luật. Bảng giữa buộc mỗi nhóm 4 cột liên tiếp trong cùng một hàng (hai nửa trái và phải, ngăn bởi vạch trắng) giữ đúng 2 số lớn hơn, nên mỗi nửa hàng có đúng 2 ô xanh. Bảng phải chỉ cho phép giữ hoặc xoá nguyên cả cột, nên xanh thành các dải dọc. Dòng tiêu đề mỗi bảng ghi phần trăm tổng $|w|$ còn giữ lại: 74,2%, 70,8% và 59,8% (tự tính).

Điều nên thấy ngay là càng ràng buộc chặt, càng phải giữ những số nhỏ và xoá nhầm số lớn (ví dụ bảng phải giữ cột 0 vì tổng cột lớn dù chứa số 3, và bỏ cột 5 dù chứa số 31).

4. Thang độ chi tiết và định nghĩa chính thức

4.1 Từ mịn đến thô

Độ chi tiết của mặt nạ	Đơn vị bị xoá	Tên gọi thường gặp	Chạy nhanh trên phần cứng dày?
Từng trọng số	một phần tử W_{ij}	không cấu trúc (unstructured)	không, cần kernel thưa
Từng nhóm cố định nhỏ	trong mỗi nhóm M phần tử liên tiếp, giữ N	bán cấu trúc, N:M (ví dụ 2:4)	có, nếu phần cứng có hỗ trợ
Từng khối	một khối $b \times b$	thưa theo khối	có nếu kernel hỗ trợ khối
Từng hàng hoặc cột	một nơ-ron đầu ra hoặc một kênh đầu vào	có cấu trúc (structured)	có, ma trận nhỏ lại
Từng kênh, filter, nơ-ron MLP	hàng của lớp trước cùng cột của lớp sau	có cấu trúc	có, các ma trận nhỏ lại
Từng đầu attention	cả một đầu (4 ma trận con)	có cấu trúc	có
Từng lớp (khối Transformer)	cả khối	cắt độ sâu	có, mô hình nông hơn

Bảng này chỉ là thang sắp xếp theo thông lệ; ranh giới giữa các dòng không cứng. Chương này tập trung vào ba dòng đầu và hai dòng giữa. Cắt cả khối (độ sâu) được bàn ở các bài về ShortGPT, SLEB và LaCo trong cuốn này.

4.2 Định nghĩa

Gọi $\mathcal{M}$ là **tập các mặt nạ được phép**. Một phương pháp cắt tỉa chọn $M \in \mathcal{M}$ để mất mát thêm vào nhỏ nhất:

$$M^\star = \arg\min_{M \in \mathcal{M},\ \|M\|_0 = (1-s)RC} \mathcal{L}(M \odot W).$$

Chữ "kiểu cắt" thực ra chỉ là cách chọn $\mathcal{M}$:

- **Không cấu trúc:** $\mathcal{M}$ là mọi ma trận nhị phân có đúng $(1-s)RC$ số 1. Tự do cao nhất.
- **N:M:** chia mỗi hàng thành các nhóm M phần tử liên tiếp; $\mathcal{M}$ là các mặt nạ có đúng N số 1 trong mỗi nhóm. Với $N = 2$, $M = 4$: mỗi nhóm giữ 2, xoá 2, độ thưa đúng 50%.
- **Theo cột:** $\mathcal{M}$ là các mặt nạ mà mỗi cột chỉ toàn 0 hoặc toàn 1. Có $\binom{C}{k}$ lựa chọn nếu giữ k cột.

Tập $\mathcal{M}$ càng nhỏ thì tối ưu trong nó càng kém hoặc bằng tối ưu của tập lớn hơn chứa nó: mặt nạ N:M và mặt nạ theo cột đều là trường hợp riêng của mặt nạ không cấu trúc ở cùng độ thưa. Do đó mất mát tối ưu của không cấu trúc luôn không tệ hơn (ở cùng độ thưa, nếu giải đúng). Cái giá của sự tự do đó là phần cứng không biết tận dụng.

5. Ví dụ tay làm chậm: đếm xem thật sự xoá được bao nhiêu

Ma trận của ta (hàng 0 đến 3, cột 0 đến 7), tự tính:

	c0	c1	c2	c3	c4	c5	c6	c7
h0	3	12	-26	22	11	-5	-30	17
h1	24	7	-19	27	4	31	9	1
h2	20	-13	21	-14	8	6	18	15
h3	-28	-23	-10	-29	25	2	-16	32

Tổng độ lớn $\sum |w| = 528$. Mọi kiểu dưới đây nhắm xoá 16 trong 32 phần tử.

Kiểu A: không cấu trúc. Xếp độ lớn tăng dần, xoá 16 số nhỏ nhất, tức mọi số có độ lớn từ 1 đến 16: ngưỡng là 16, giữ số có $|w| \geq 17$. Giữ lại: 16 số, tổng độ lớn 392 (74,2% của 528), năng lượng $\sum w^2$ giữ lại 86,9% (tự tính). Ma trận vẫn 4×8.

Kiểu B: bán cấu trúc 2:4. Mỗi hàng có hai nhóm (cột 0 đến 3, cột 4 đến 7). Trong mỗi nhóm giữ 2 số có độ lớn lớn nhất. Hàng 0, nhóm 1: $(3, 12, -26, 22)$ giữ -26 và 22. Hàng 0, nhóm 2: $(11, -5, -30, 17)$ giữ -30 và 17. Làm tương tự cho 6 nhóm còn lại. Tổng độ lớn giữ lại 374 (70,8%), năng lượng 81,8% (tự tính). Kém kiểu A đúng $392 - 374 = 18$ đơn vị. Cụ thể 2:4 phải xoá hai số mà kiểu A giữ: -19 (hàng 1, nhóm 1, vì nhóm đó đã có 24 và 27 lớn hơn) và -23 (hàng 3, nhóm 1, vì đã có -28 và -29), mất $19 + 23 = 42$. Đổi lại nó phải giữ hai số mà kiểu A xoá: 9 (hàng 1, nhóm 2) và 15 (hàng 2, nhóm 2), vì mỗi nhóm bắt buộc giữ đủ 2, được $9 + 15 = 24$. Hiệu: $42 - 24 = 18$ (tự tính). Nhóm có hai số lớn và hai số nhỏ thì 2:4 không mất gì; nhóm có ba số lớn thì mất một.

Kiểu C: có cấu trúc (cột). Tổng độ lớn các cột: $75, 55, 76, 92, 48, 44, 73, 65$ (tự tính). Giữ 4 cột lớn nhất là cột 3 (92), 2 (76), 0 (75), 6 (73): tổng độ lớn 316 (59,8%) và năng lượng chỉ 62,6% (tự tính). Ma trận co thành 4×4.

5.1 Đếm "xoá thật"

Giả sử phần tử lưu bằng FP16 (16 bit) và vector đầu vào có 8 chiều. Ta xét một token (một vector x).

	Cỡ ma trận cấp phát	Số 0	MAC trên phần cứng dày	MAC có thể tốn nếu phần cứng/kernel tận dụng	Bộ nhớ nén (bit)
Dense	4 x 8 = 32	0	32	32	32 x 16 = 512
A: không cấu trúc	4 x 8 = 32 (vẫn như cũ)	16	32	16 (kernel thưa)	bitmask: 32 + 16 x 16 = 288
B: 2:4	4 x 4 giá trị + chỉ số	16	32	16 (Sparse Tensor Core)	16 x 16 + 16 x 2 = 288
C: theo cột	4 x 4 = 16	16	16	16	16 x 16 = 256

Đây là kết luận của ví dụ, tự tính: cả ba kiểu "xoá 50% phần tử" nhưng số tham số **thật sự biến mất khỏi kích thước ma trận** chỉ có kiểu C: $32 \to 16$ phần tử, không cần metadata. Kiểu A không làm ma trận nhỏ đi; nó chỉ cho phép lưu gọn hơn (288 bit) nếu có định dạng thưa, và chỉ nhanh hơn nếu có kernel thưa đủ hiệu quả. Kiểu B cũng lưu 288 bit nhưng bù lại cấu trúc đều: phần cứng biết mỗi nhóm 4 có đúng 2 số.

Hai con số 288 bit của A và B trùng nhau do tình cờ (bitmask 32 bit và metadata 16 chỉ số x 2 bit đều là 32 bit). Sự khác biệt nằm ở tính đều đặn, không ở bộ nhớ.

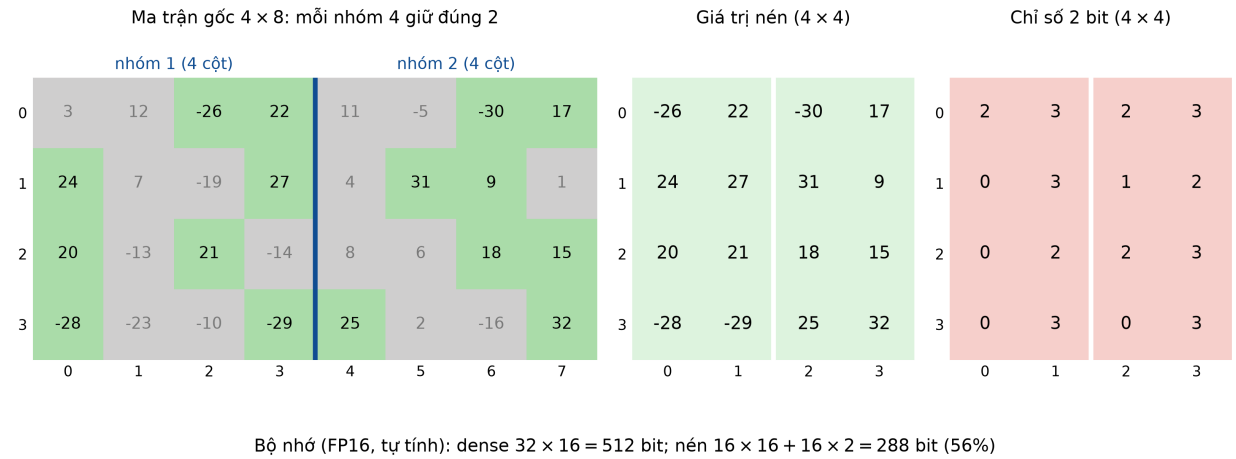
Một chi tiết hay bị bỏ qua ở kiểu C: bỏ 4 cột nghĩa là x chỉ cần 4 thành phần đầu vào. Muốn thế, các thành phần đó phải không còn được tính ở lớp trước. Đó là chuyện của mục 7.

6. Mẫu 2:4 và vì sao phần cứng hỗ trợ

6.1 Cấu trúc

Mẫu 2:4 yêu cầu: trong mỗi nhóm 4 phần tử liên tiếp (thường theo chiều trong của phép nhân, tức dọc theo hàng của W), có không quá 2 phần tử khác 0. Mỗi nhóm có $\binom{4}{2} = 6$ cách chọn 2 vị trí. Định dạng nén lưu hai thứ: ma trận giá trị chỉ còn một nửa số cột, và một chỉ số 2 bit cho mỗi giá trị nói nó nằm ở vị trí 0, 1, 2 hay 3 trong nhóm gốc. Hình sau minh hoạ ngay trên ma trận 4×8 của ví dụ.

Cách đọc hình: lấy hàng 0, nhóm 1 (cột 0 đến 3) chứa $(3, 12, -26, 22)$. Hai số giữ là -26 (vị trí 2) và 22 (vị trí 3), nên ma trận nén ghi $(-26, 22)$ ở hai ô đầu và ma trận chỉ số ghi $(2, 3)$. Nhóm 2 của hàng 0 giữ -30



Hình 14: Mẫu 2:4 trên ma trận 4×8: trái là ma trận gốc với hai nhóm 4 cột, giữa là ma trận giá trị nén 4×4, phải là chỉ số 2 bit của mỗi giá trị trong nhóm (minh hoạ, tự tính)

(vị trí 2) và 17 (vị trí 3), nên hai ô sau là $(-30, 17)$ và $(2, 3)$. Dòng cuối hình ghi tổng bộ nhớ: 512 bit xuống 288 bit, tức 56,25% (tự tính).

6.2 Vì sao phần cứng làm được

Theo ghi chú pruning 18 (bài NVIDIA về Sparse Tensor Core trên GPU Ampere), phép nhân được thực hiện giữa ma trận thưa 2:4 (dạng nén) và ma trận dày. Với mỗi nhóm 4, bộ metadata cho phần cứng biết hai phần tử nào của ma trận dày cần lấy, nên chỉ nhân 2 trong 4 phần tử: số phép nhân-cộng giảm một nửa và lý thuyết cho throughput tối đa 2 lần. Hai điều kiện làm nên điều đó:

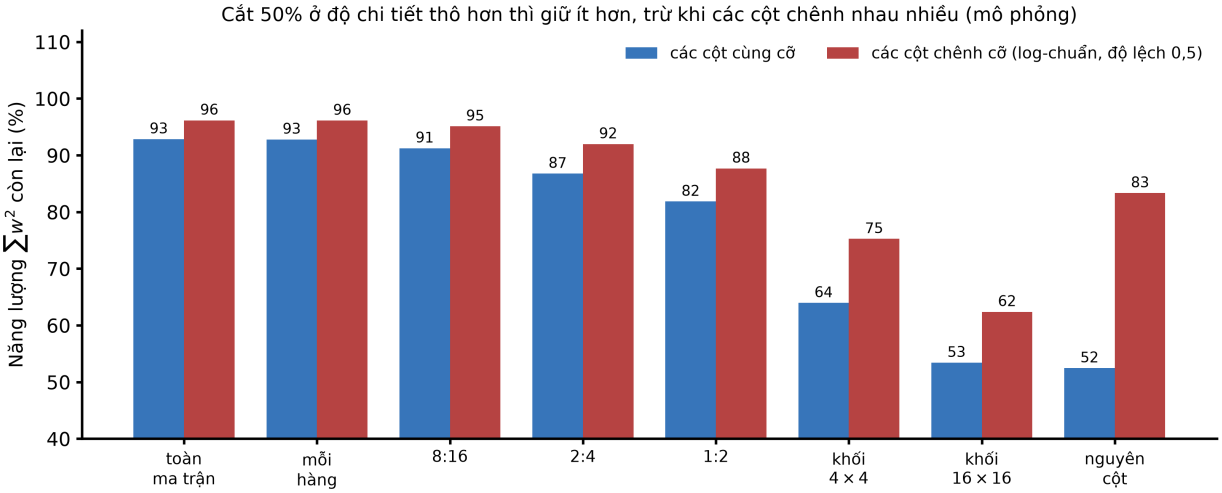
- **Mật độ đều ở mọi nhóm.** Mỗi nhóm đúng 2 số khác 0, nên phần cứng không phải dò, không phải cân bằng tải giữa các hàng.
- **Vị trí tính được từ cấu trúc.** Metadata chỉ cần 2 bit mỗi giá trị, truy cập gọn, không có tra cứu gián tiếp như định dạng CSR của ma trận thưa ngẫu nhiên.

Cũng theo ghi chú pruning 18, ma trận phải có chiều trong chia hết cho 16 (FP16/BF16) hoặc 32 (INT8), nên có lớp nhỏ không áp dụng được. Con số 2 lần là đỉnh lý thuyết của lõi nhân ma trận; trên toàn mô hình luôn thấp hơn, vì còn các phần khác (xem mục 9).

6.3 Vì sao 2:4 không phá chất lượng quá nhiều

Ràng buộc "mỗi nhóm 4 giữ 2" mịn hơn rất nhiều so với "mỗi cột giữ hoặc bỏ". Hình sau mô phỏng điều đó: ma trận Gaussian 512×512, ta cắt đúng 50% ở các độ chi tiết khác nhau và đo phần năng lượng $\sum w^2$ còn lại. Có hai trường hợp: các cột có cỡ như nhau, và các cột được nhân thêm hệ số ngẫu nhiên log-chuẩn (cột to nhỏ chênh nhau).

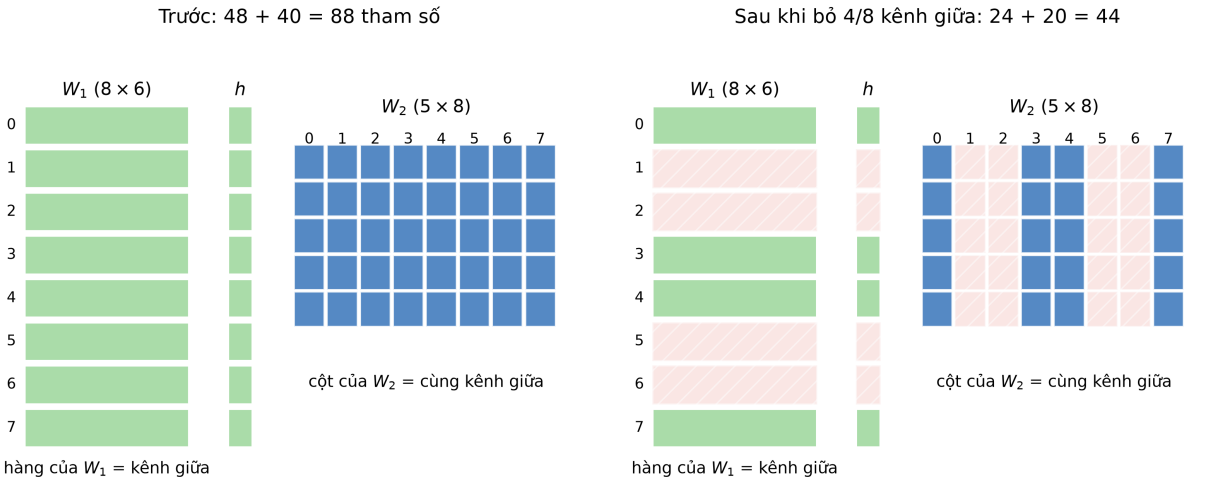
Cách đọc hình: mỗi cặp cột là một độ chi tiết. Với cột cùng cỡ (xanh dương), năng lượng còn lại giảm dần từ 93% (toàn ma trận) xuống 91% (8:16), 87% (2:4), 82% (1:2), 64% (khối 4×4), 53% (khối 16×16) và 52% (nguyên cột), gần mức 50% là mức chọn ngẫu nhiên. Với cột chênh cỡ (đỏ), cắt nguyên cột giữ lại nhiều hơn nhiều (83%) vì có cột đáng xoá thật. Bài học: khi các cột có "độ quan trọng" rất khác nhau, cắt thô hiệu quả; khi chúng gần giống nhau, cắt thô là cắt tuỳ tiện. Đây là số mô phỏng trên dữ liệu tự sinh (seed cố định), không phải số đo trên mô hình thật; nó chỉ cho thấy xu hướng.



Hình 15: Mô phỏng: phần năng lượng giữ lại khi cắt 50% ở các độ chi tiết khác nhau, với cột cùng cỡ và cột chênh cỡ (minh hoạ, tự tính)

7. Cắt kênh và dây chuyền phụ thuộc

Cắt có cấu trúc luôn kéo theo hệ quả ở lớp kề. Xét hai lớp liên tiếp $h = \phi(W_1 x)$ rồi $y = W_2 h$. Nếu bỏ nơ-ron số i của lớp 1, tức hàng i của W_1, thì h_i không còn, nên cột i của W_2 vô dụng và cũng phải bỏ. Hai ma trận co theo cùng một tập chỉ số.



Hình 16: Dây chuyền phụ thuộc: bỏ 4 trong 8 kênh giữa làm hàng của W_1 và cột tương ứng của W_2 cùng biến mất (minh hoạ, tự tính)

Cách đọc hình: bên trái là trạng thái đầu, W_1 cỡ 8×6 (48 tham số) và W_2 cỡ 5×8 (40 tham số), tổng 88. Cùng một chỉ số kênh (số bên cạnh mỗi hàng của W_1 và trên mỗi cột của W_2) trỏ tới cùng một thành phần của h. Bên phải, bỏ các kênh số 1, 2, 5, 6: các hàng gạch chéo của W_1 và các cột gạch chéo tương ứng của W_2 đều biến mất, còn W_1 cỡ 4×6 (24) và W_2 cỡ 5×4 (20), tổng 44, đúng một nửa (tự tính). Chú ý mặt nạ phải đồng bộ giữa hai ma trận: bỏ hàng ở một bên mà quên cột ở bên kia thì kích thước không khớp và mô hình không chạy.

Trong mạng thật, phụ thuộc còn phức tạp hơn một chuỗi hai lớp:

- **Kết nối tắt (residual):** đầu ra của một lớp được cộng với đầu vào, nên hai nhánh cộng nhau phải có

cùng số kênh. Bỏ kênh ở một nhánh buộc bỏ cùng chỉ số ở nhánh kia.
- **Chuẩn hoá (BatchNorm, LayerNorm):** tham số γ, β theo kênh phải bỏ cùng.
- **Attention:** một đầu attention gồm các hàng của W_Q, W_K, W_V ứng với đầu đó và các cột của W_O ứng với đầu đó; bỏ đầu nghĩa là bỏ cả bốn phần cùng lúc.
- **MLP của Transformer:** nơ-ron trung gian gồm hàng của ma trận "up" (và "gate" nếu có) cùng cột của ma trận "down".

Theo ghi chú pruning 20 (DepGraph), thay vì liệt kê tay các phụ thuộc này cho từng kiến trúc, phương pháp xây một đồ thị phụ thuộc rồi nhóm các tham số bị buộc phải cắt cùng nhau. Theo ghi chú pruning 26 (LLM-Pruner), tư tưởng nhóm phụ thuộc được áp dụng cho LLM: cắt nơ-ron MLP, đầu attention theo nhóm. Điểm chung: với cắt có cấu trúc, đơn vị được chấm điểm và cắt không phải một trọng số mà là một nhóm.

8. Đánh đổi: độ thưa, chất lượng, tốc độ thật

Ba đại lượng thường bị trộn lẫn:

1. **Độ thưa đạt được:** tỉ lệ số 0 (hay tỉ lệ tham số bị xoá).
2. **Chất lượng giữ được:** perplexity, độ chính xác trên bộ thử.
3. **Tốc độ thật:** thời gian chạy hoặc băng thông bộ nhớ đo trên phần cứng cụ thể.

Không có kiểu cắt nào tốt nhất trên cả ba. Bảng dưới đây tóm tắt xu hướng thường gặp (xu hướng, không phải định luật):

Kiểu	Chất lượng ở cùng độ thưa	Tốc độ thật trên GPU dày	Chi phí kèm theo
Không cấu trúc	cao nhất	thường không tăng nếu chỉ gán 0	cần kernel thưa hoặc phần cứng chuyên dụng
2:4	khá, thấp hơn không cấu trúc một chút	tăng (lý thuyết gần 2 lần ở lõi nhân ma trận)	phần cứng Ampere trở lên, cấu trúc cố định
Khối	thấp hơn	tăng nếu kernel hỗ trợ	tuỳ kích thước khối
Kênh, đầu, nơ-ron	thấp nhất ở cùng độ thưa	tăng gần tỉ lệ với phần bỏ	phải đồng bộ phụ thuộc, thường cần huấn luyện lại

Ví dụ tay về "tốc độ thật không bằng tốc độ lõi" (tự tính): giả sử trên một mô hình, 70% thời gian chạy là các phép nhân ma trận có thể dùng 2:4 và 30% còn lại (attention softmax, chuẩn hoá, truyền dữ liệu, các lớp không cắt) không đổi. Nếu lõi nhân ma trận nhanh đúng 2 lần, thời gian mới chỉ còn $0{,}3 + 0{,}7/2 = 0{,}65$ so với ban đầu, tức nhanh $1/0{,}65 \approx 1{,}54$ lần chứ không phải 2 lần. Đây là quy tắc Amdahl, và nó là lý do vì sao con số đo trên một ma trận riêng lẻ không được đọc như tốc độ cả mô hình.

Cũng cần nhớ **độ thưa 50% chưa chắc ra mô hình nhỏ hơn 50% tham số**. Cắt khối Transformer chỉ nhắm vào các ma trận lớn; embedding, chuẩn hoá và đầu ra thường giữ nguyên. Ví dụ, theo ghi chú pruning 26, LLaMA-7B cắt 20% theo khối (Block) đi từ 6,74 tỉ xuống 5,42 tỉ tham số, và độ trễ trong bảng của bài đi từ 69,32 giây xuống 58,55 giây, tức giảm khoảng 15% (tự tính từ hai số đó) trong khi tham số giảm khoảng 20%.

9. Biến thể và chi tiết hay bị bỏ qua

- **N:M tổng quát.** Ngoài 2:4 còn 1:2, 4:8, 8:16. Nhóm càng lớn thì mịn hơn (giữ chất lượng tốt hơn, như hình 3 cho thấy) nhưng phần cứng khó hỗ trợ hơn. Phần cứng NVIDIA hỗ trợ 2:4 (và 1:2 cho TF32), theo ghi chú pruning 18.
- **Mặt nạ theo từng hàng đầu ra.** Một số phương pháp (như Wanda trong ghi chú pruning 24) so sánh trọng số trong từng hàng đầu ra thay vì cả lớp; ghi chú đó cho thấy chọn nhóm so sánh quan trọng ngang chọn điểm số. Đây vẫn là cắt không cấu trúc nhưng có tính cân bằng giữa các hàng.

- **Hoán vị kênh trước khi cắt 2:4.** Nếu hai số lớn rơi cùng một nhóm, 2:4 buộc xoá một trong chúng. Hoán vị các cột trước khi cắt, rồi hoán vị ngược ở lớp trước để tổng phép tính không đổi, có thể giữ nhiều độ lớn hơn. Theo ghi chú pruning 18, ví dụ ma trận 4×8 của bài đạt tổng độ lớn sau cắt từ 83,7 lên 102,9 sau khi hoán vị.
- **Không phải lớp nào cũng cắt được.** Theo ghi chú pruning 18, không cắt embedding (là bảng tra), không cắt lớp đầu có số kênh đầu vào quá nhỏ cho GEMM, và các lớp không phải GEMM.
- **Thưa trọng số khác thưa kích hoạt.** Chương này nói về thưa trọng số. Hàm kích hoạt (ví dụ ReLU) cũng tạo ra số 0 trong dữ liệu, nhưng đó là thưa động theo từng đầu vào, không phải mặt nạ cố định.
- **Mặt nạ cố định hay động.** Mặt nạ có thể được tính một lần rồi giữ cố định khi huấn luyện lại (thường gặp ở 2:4), hoặc được cập nhật liên tục (như Movement Pruning, ghi chú pruning 16).
- **Đo độ thưa theo tham số hay theo FLOP.** Cắt filter trong lớp đầu của CNN thường làm giảm FLOP nhiều hơn tham số vì lớp đầu có bản đồ đặc trưng lớn; còn cắt lớp sâu thường giảm tham số nhiều hơn FLOP. Hai con số này không thể đổi cho nhau.

10. Nối sang pruning và LLM

Pruning 04 và 05: cắt kênh cho CNN. Theo ghi chú pruning 04, cắt filter trên VGG-16 (CIFAR-10) bỏ 64,0% tham số và 34,2% FLOP mà lỗi test 6,60% so với 6,75% của mô hình gốc. Theo ghi chú pruning 05, DenseNet-40 cắt 40% kênh trên CIFAR-10 đạt lỗi 5,19% so với 6,11% của baseline. Cả hai đều là cắt có cấu trúc: mô hình co lại thật, chạy được trên thư viện dày thông thường. Tham số giảm nhiều hơn FLOP ở VGG-16, có thể vì phần lớn tham số nằm ở các lớp sâu (đây là cách giải thích của ghi chú, không tự kiểm).

Pruning 17 (CoFi): cấu trúc tự học trên BERT. Theo ghi chú pruning 17, Movement Pruning cho BERT-base còn 9 triệu tham số nhưng tăng tốc đo được là 1,0 lần (không cấu trúc), trong khi CoFi đạt 2,7 lần ở 26 triệu tham số (MNLI 84,9) và 12,1 lần ở 4,4 triệu tham số (MNLI 80,6). Cùng đề tài "thưa" mà tốc độ khác hẳn nhau vì độ chi tiết của mặt nạ khác nhau.

Pruning 18: 2:4 như dàn xếp. Theo ghi chú pruning 18, ma trận 2:4 nén giảm bộ nhớ khoảng 44% với giá trị 16 bit (64 xuống 36 bit mỗi nhóm 4), và độ chính xác ImageNet của ResNet-50 sau cắt 2:4 và huấn luyện lại là 76,2 so với 76,1 của dense. Đó là các số đã đo trong bài; số 2 lần chỉ là đỉnh lý thuyết của lõi.

Pruning 19 (HALP): chọn theo độ trễ. Theo ghi chú pruning 19, thay vì chỉ đếm FLOP, bài dựng bảng độ trễ (LUT) trên GPU thật và cắt kênh theo ngân sách độ trễ; ví dụ ResNet-50 HALP-55% đạt 76,5% top-1 và tăng tốc 1,60 lần trên Titan V. Ý nghĩa: ngân sách cắt nên tính bằng đại lượng bạn quan tâm (độ trễ), không phải bằng số 0.

Pruning 20 và 26: tự động hoá phụ thuộc, rồi đưa lên LLM. Hai bài này giải quyết đúng vấn đề của mục 7: biết nhóm nào phải cắt cùng nhau (DepGraph) và áp dụng cho LLaMA (LLM-Pruner, cắt 20% tham số với điểm bậc hai: zero-shot trung bình 56,82 khi không tinh chỉnh và 60,07 sau LoRA, trong khi mô hình gốc là 63,25; số "gốc" là số tác giả chạy lại, theo ghi chú pruning 26).

Pruning 23 và 24: không cấu trúc và 2:4 trên LLM. Theo ghi chú pruning 23, SparseGPT ở 2:4 trên OPT-175B làm perplexity từ 8,35 lên 8,74, và kernel 2:4 cho khoảng 1,54 đến 1,79 lần ở từng phép nhân ma trận lớn. Con số này nằm giữa 1 lần (không cắt) và 2 lần (lý thuyết) đúng như mục 6 và mục 8 dự đoán.

11. Sai lầm hay gặp

- **Đọc "50% thưa" như "nhanh gấp 2 lần".** Độ thưa là tỉ lệ số 0, không phải tốc độ. Không cấu trúc gần như không tăng tốc trên GPU dày; 2:4 tối đa gần 2 lần ở lõi và thấp hơn ở toàn mô hình.
- **So sánh chất lượng hai kiểu cắt khác độ thưa hoặc khác chi phí huấn luyện lại.** Kiểu thô cần huấn luyện lại nhiều hơn để bù; so sánh khi chỉ một bên được huấn luyện lại là không công bằng.
- **Quên đồng bộ phụ thuộc khi cắt kênh.** Bỏ hàng của W_1 mà quên cột của W_2, hoặc quên nhánh cộng của residual, làm kích thước sai hoặc gây lỗi âm thầm.

- **Đếm tham số giảm mà quên metadata.** Định dạng thưa tốn thêm chỉ số: bitmask thêm 1 bit mỗi phần tử, 2:4 thêm 2 bit mỗi giá trị còn lại. Tiết kiệm thật là hiệu số.
- **Đo tốc độ trên một ma trận rồi suy ra cả mô hình.** Phần không phải nhân ma trận làm tốc độ cả mô hình thấp hơn (ví dụ 1,54 lần thay vì 2 lần ở mục 8).
- **Cho rằng cắt có cấu trúc luôn tệ hơn.** Khi các kênh có độ quan trọng rất khác nhau (cột chênh cỡ ở hình 3), cắt kênh hiệu quả; khi mạng dư thừa đều, cắt kênh phải giữ nhiều hơn để không hỏng.

12. Tóm tắt

Khái niệm	Công thức hoặc con số	Ghi chú
Mặt nạ, độ thưa	$s = 1 - \|M\|_0/(RC)$	độ thưa là tỉ lệ số 0
N:M	trong mỗi nhóm M phần tử giữ N	2:4 có 6 mẫu mỗi nhóm
Bộ nhớ 2:4 (FP16)	$2 \times 16 + 2 \times 2 = 36$ bit mỗi nhóm 4 so với 64	khoảng 56% (ví dụ chương: 288 trên 512 bit)
Cắt cột (giữ k trên C cột)	ma trận co từ R x C còn R x k	kéo theo hàng của lớp trước
Dây chuyền hai lớp	tham số = (r1 + r2) x k nếu giữ k kênh giữa, r1 là số cột của W1, r2 là số hàng của W2	ví dụ chương: 24 + 20 = 44
Amdahl	tăng tốc = 1 / (1 - p + p/g), p là tỉ lệ thời gian được tăng tốc, g là hệ số	p = 0,7 và g = 2 cho 1,54

13. Bài tập tự tính

Bài 1 (dễ). Nhóm 4 phần tử $(-6,\ 2,\ 9,\ -4)$. (a) Mẫu 2:4 giữ phần tử nào, chỉ số trong nhóm là gì? (b) Phần trăm tổng độ lớn giữ lại? (c) Số bit mỗi nhóm khi nén FP16 so với dense?

Đáp án (tự tính): (a) giữ 9 (vị trí 2) và -6 (vị trí 0), chỉ số $(0, 2)$. (b) $(9+6)/(6+2+9+4) = 15/21 \approx 71{,}4\%$. (c) dense $4 \times 16 = 64$ bit, nén $2 \times 16 + 2 \times 2 = 36$ bit.

Bài 2. Ma trận 4096×4096 ở FP16. Bộ nhớ dense là bao nhiêu byte, bộ nhớ nén 2:4 (giá trị cộng chỉ số 2 bit) là bao nhiêu, và tỉ lệ?

Đáp án (tự tính): số phần tử $16\,777\,216$; dense $16\,777\,216 \times 2 = 33\,554\,432$ byte (32 MiB). Nén: $8\,388\,608$ giá trị, mỗi giá trị $16 + 2 = 18$ bit, tổng $150\,994\,944$ bit $= 18\,874\,368$ byte (18 MiB). Tỉ lệ $18/32 = 0{,}5625$.

Bài 3. Dùng ma trận 4×8 của chương, cắt kiểu 1:2 (mỗi nhóm 2 cột liên tiếp giữ 1 số lớn hơn). Tổng độ lớn giữ lại ở hàng 0, và ở cả ma trận? Kiểu này thưa bao nhiêu?

Đáp án (tự tính): hàng 0 gồm các cặp $(3, 12), (-26, 22), (11, -5), (-30, 17)$ nên giữ $12 + 26 + 11 + 30 = 79$ trên tổng hàng 126. Cả ma trận: 351 trên 528, tức $66{,}5\%$. Thưa 50%. Kém 2:4 ($70{,}8\%$) và kém không cấu trúc ($74{,}2\%$), đúng thứ tự mịn đến thô.

Bài 4. Hai lớp: W_1 cỡ 10×6 và W_2 cỡ 4×10 (10 nơ-ron giữa). Bỏ 4 nơ-ron giữa. (a) Số tham số trước và sau? (b) Tỉ lệ giữ lại? (c) Nếu thay vào đó xoá 40% phần tử không cấu trúc ở mỗi ma trận thì số tham số "thật sự" của hai ma trận có đổi kích thước không?

Đáp án (tự tính): (a) trước $60 + 40 = 100$; sau W_1 còn $6 \times 6 = 36$, W_2 còn $4 \times 6 = 24$, tổng 60. (b) 60%. (c) Không: ma trận vẫn 10×6 và 4×10 (100 phần tử cấp phát), chỉ có 40 số 0 mà kích thước không đổi nếu không dùng định dạng thưa.

Bài 5 (khó). Trên một mô hình, 80% thời gian chạy nằm ở các phép nhân ma trận có thể chuyển sang 2:4, 20% không đổi. Lõi 2:4 nhanh đúng $1{,}8$ lần ở các phép nhân đó. (a) Tốc độ cả mô hình tăng bao nhiêu lần? (b) Nếu lõi đạt đúng 2 lần thì sao? (c) Giới hạn trên khi lõi nhanh vô hạn?

Đáp án (tự tính): (a) $1/(0{,}2 + 0{,}8/1{,}8) = 1/0{,}6444 \approx 1{,}55$ lần. (b) $1/(0{,}2 + 0{,}4) = 1{,}67$ lần. (c) $1/0{,}2 = 5$ lần. Không bao giờ vượt qua $1/(1-p)$ dù lõi nhanh đến đâu.

Tài liệu tham khảo nền: ghi chú pruning 18 (NVIDIA, Sparse Tensor Core và mẫu 2:4), ghi chú pruning 04, 05, 17, 19, 20, 26 trong cuốn này; ghi chú pruning 02 và 03 cho các phương pháp không cấu trúc kinh điển; preface 00 phần phân loại pruning.

Nền tảng 05

Cắt rồi huấn luyện lại: sau cú cắt, mô hình cần được chữa lành bằng cách nào

Loại: chương nền tảng (không phải ghi chú paper)

Đọc trước: gradient descent, SGD và learning rate (optimization nền 02, 03, 06); mặt nạ và độ thưa ở chương nền 04 của cuốn này (nen04); phần phân loại trong preface 00

Dùng ở: pruning 02 (cắt, huấn luyện lại, lặp), pruning 03 (lịch bậc ba), pruning 08 (cắt lặp và đặt lại về khởi tạo), pruning 16 (cắt trong lúc tinh chỉnh), pruning 23 và pruning 24 (cắt một lần, không huấn luyện lại), pruning 26 (phục hồi bằng LoRA), pruning 37 (sửa sau khi bỏ lớp), pruning 39 (phục hồi bằng distillation)

> Chương này giải thích từ đầu, không phải bản dịch một paper. Các phép tính ghi "tự tính" đã kiểm bằng numpy; các hình ghi "mô phỏng" dùng dữ liệu tự sinh, không phải kết quả đo từ mô hình thật. Con số lấy từ paper luôn có chú "theo ghi chú pruning NN".

1. Vì sao cần bước "huấn luyện lại"

Mỗi lần xoá trọng số là một lần làm mô hình quên đi một phần những gì nó đã học; bước huấn luyện lại (hoặc một bước bù khác) là để các trọng số còn lại tự điều chỉnh, gánh phần việc của những trọng số đã mất.

Hãy hình dung một đội 10 người làm việc nhịp nhàng. Nếu đột ngột bỏ đi 7 người, đội sẽ làm việc rất tệ ngay lúc đó. Nhưng nếu cho 3 người còn lại thời gian để phân công lại, họ có thể làm được phần lớn việc. Nếu bỏ từng người một, mỗi lần cho thời gian phân công lại, đội thích nghi dễ hơn. Còn nếu mỗi lần bỏ người mà không cho thời gian nào, nhưng lại biết trước chính xác ai làm việc gì (nhờ tính toán), ta vẫn có thể giao sẵn việc cho người ở lại. Ba tình huống đó chính là ba chiến lược của chương này: **cắt một lần rồi huấn luyện lại**, **cắt dần xen kẽ với huấn luyện**, và **cắt một lần rồi bù bằng công thức, không huấn luyện**.

Trong AI, đây không phải chi tiết phụ. Với mô hình cỡ vài triệu tham số (CNN trên ảnh nhỏ, BERT), huấn luyện lại rẻ và gần như luôn được dùng. Với LLM cỡ hàng tỉ tham số, mỗi vòng huấn luyện lại tốn kém về GPU, dữ liệu và thời gian, nên nhiều phương pháp mới cho LLM được thiết kế để tránh nó hoặc rút gọn nó (chỉ vài giờ trên một GPU, hoặc chỉ cập nhật vài bộ trọng số nhỏ). Hiểu bốn trục sau giúp đọc nhanh các bài trong cuốn: cắt **một lần hay lặp**, **lịch** tăng độ thưa, **cách chữa** sau khi cắt (huấn luyện lại, distillation, LoRA), và **khi nào không cần chữa**.

2. Nhắc nền: loss, tinh chỉnh với mặt nạ và vì sao loss tăng ngay sau cú cắt

2.1 Loss và mặt nạ cố định

Mô hình có trọng số w và hàm mất mát $\mathcal{L}(w)$ (loss) đo độ tệ. Cắt tỉa dùng mặt nạ nhị phân m để đặt một phần trọng số về 0: mô hình sau cắt là $m \odot w$. **Tinh chỉnh với mặt nạ cố định** là chạy gradient descent chỉ trên các trọng số còn sống:

$$w \leftarrow m \odot \big(w - \eta \, \nabla \mathcal{L}(w)\big),$$

nhân m sau mỗi bước để các phần tử đã bị cắt giữ nguyên 0. Hệ số η là learning rate. Đây là bản mặc định ở hầu hết các phương pháp trong cuốn (ví dụ bước huấn luyện lại sau cắt 2:4 trong ghi chú pruning 18).

2.2 Một ví dụ số nhỏ về cú sốc khi cắt

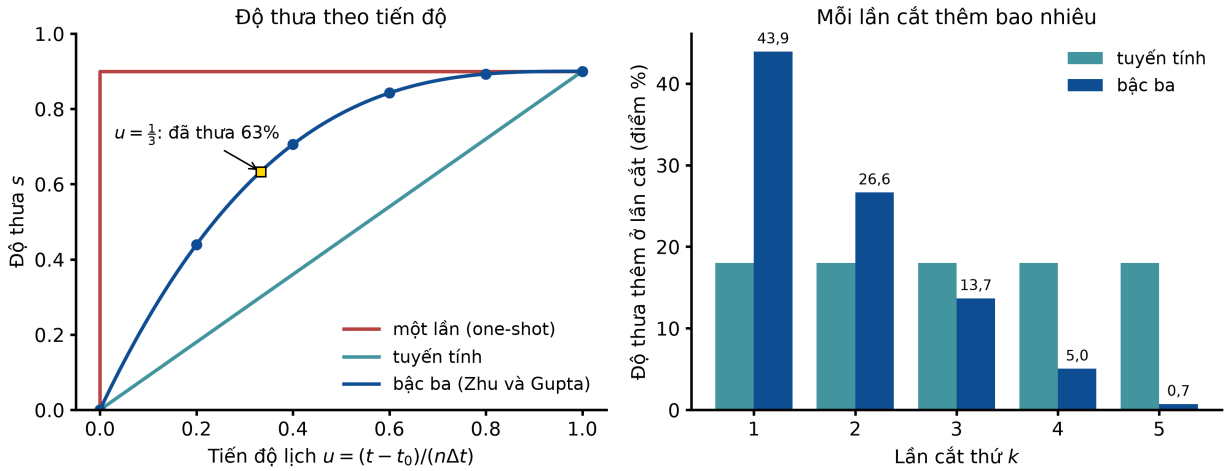
Xét hồi quy tuyến tính với hai trọng số, hàm loss $\mathcal{L}(w) = \frac{1}{2}(w - w^*)^\top H (w - w^*)$ với $w^* = (1;\, 0{,}8)$ là điểm tối ưu và $H = \begin{pmatrix} 1 & 0{,}9 \\ 0{,}9 & 1 \end{pmatrix}$ cho hai trọng số tương quan mạnh. Tại w^*, loss bằng 0 (ta chọn mốc 0 cho dễ tính). Nếu cắt w_2 về 0 mà giữ $w_1 = 1$, thì $d = w - w^* = (0;\, -0{,}8)$ và

$$\mathcal{L} = \tfrac{1}{2}\, d^\top H d = \tfrac{1}{2} \cdot 1 \cdot 0{,}8^2 = 0{,}32 \quad \text{(tự tính)}.$$

Loss nhảy từ 0 lên 0,32 chỉ vì xoá một trọng số có giá trị $0{,}8$. Hai trọng số tương quan nên khi w_2 biến mất, w_1 phải tự đổi để bù. Cú nhảy này là thứ ta gọi là **cú sốc cắt tỉa**, và phần còn lại của chương là về cách làm cho nó nhỏ rồi chữa nó.

3. Trực giác bằng hình: ba cách tăng độ thưa theo thời gian

Cho độ thưa cuối $s_f = 0{,}9$ (xoá 90% trọng số). Có thể đạt nó theo ba cách: cắt hết một lần, cắt đều theo thời gian (tuyến tính), hoặc cắt nhanh lúc đầu và chậm dần (bậc ba). Hình sau vẽ cả ba, cùng lượng thưa thêm ở từng lần cắt khi chia làm 5 lần.



Hình 17: Lịch độ thưa: cắt một lần, tuyến tính và bậc ba (trái), và độ thưa thêm ở từng lần cắt khi chia 5 lần (phải) (minh hoạ, tự tính)

Cách đọc hình: trục ngang bên trái là tiến độ lịch u từ 0 đến 1; trục dọc là độ thưa. Đường đỏ nhảy thẳng lên 0,9 ngay ở $u = 0$. Đường xanh lục tăng đều. Đường xanh dương (bậc ba) dốc lúc đầu rồi phẳng dần, năm chấm tròn là năm lần cắt; chấm vàng ở $u = \frac{1}{3}$ có độ thưa khoảng 63%. Bên phải là lượng thưa thêm ở từng lần: bậc ba cắt thêm 43,9 điểm phần trăm ở lần 1 nhưng chỉ 0,7 điểm ở lần 5, còn tuyến tính cắt đều 18 điểm mỗi lần (tự tính).

Ý chính: cùng đạt 90%, nhưng "cú sốc" mỗi lần khác nhau. Chương sẽ trả lời vì sao nên cắt nhiều lúc đầu rồi chậm dần.

4. Cắt một lần, cắt lặp và cắt dần: định nghĩa

4.1 Ba chiến lược

- **Cắt một lần rồi huấn luyện lại (one-shot):** huấn luyện đến hội tụ; cắt đến độ thưa mục tiêu s_f; tinh chỉnh với mặt nạ cố định. Tốn một lần huấn luyện cộng một lần tinh chỉnh.

- **Cắt lặp (iterative pruning), tiêu biểu là Iterative Magnitude Pruning, IMP:** lặp nhiều vòng: cắt một phần nhỏ, huấn luyện lại, rồi cắt thêm. Độ thưa tăng theo bậc thang.
- **Cắt dần trong lúc huấn luyện (gradual pruning):** không có "vòng" riêng; cứ vài trăm bước cập nhật lại mặt nạ theo một lịch trơn, xen kẽ với bước huấn luyện thường.

Cả ba đều có thể dùng cùng một tiêu chí chọn (ví dụ độ lớn $|w|$). Chúng chỉ khác **thời điểm và nhịp** cắt.

4.2 Cắt lặp: mỗi vòng cắt bao nhiêu

Nếu mỗi vòng giữ lại tỉ lệ q của số trọng số còn sống thì sau n vòng còn lại q^n của ban đầu. Để còn lại một tỉ lệ cho trước r sau n vòng, mỗi vòng giữ $q = r^{1/n}$. Ví dụ (tự tính): muốn còn $r = 10\%$ (thưa 90%) sau $n = 5$ vòng thì $q = 0{,}1^{1/5} \approx 0{,}631$, tức mỗi vòng cắt khoảng 36,9% số trọng số còn sống. Nếu mỗi vòng chỉ cắt 20% ($q = 0{,}8$) thì sau 11 vòng còn $0{,}8^{11} \approx 8{,}6\%$ (thưa 91,4%); 11 vòng là số vòng tối thiểu để vượt 90% (tự tính). Theo ghi chú pruning 08, bài Lottery Ticket cắt lặp theo kiểu này (cắt một tỉ lệ cố định của trọng số còn sống mỗi vòng) và đặt trọng số còn lại về giá trị khởi tạo ban đầu sau mỗi vòng; ghi chú nhắc rằng việc lặp "train 15 lần liên tiếp trở lên" là rất tốn kém.

4.3 Chi phí so sánh

Chiến lược	Số lần huấn luyện (xấp xỉ)	Độ mịn của cú sốc	Hợp với
Một lần	1 lần huấn luyện + 1 lần tinh chỉnh	lớn	mô hình lớn, độ thưa vừa phải
Lặp (IMP)	nhiều vòng, mỗi vòng huấn luyện lại	nhỏ	mô hình nhỏ, cần độ thưa cao
Cắt dần	gần như 1 lần huấn luyện (lịch gắn trong lúc train)	nhỏ	khi được kiểm soát quá trình huấn luyện

Con số "số lần huấn luyện" chỉ nói bậc độ lớn; chi phí thật phụ thuộc số bước mỗi vòng. Ví dụ ở ghi chú pruning 02: huấn luyện AlexNet gốc mất 75 giờ trên Titan X, còn bước cắt rồi huấn luyện lại mất 173 giờ; bài lập luận rằng chi phí đó chấp nhận được vì cắt tỉa là bước cuối trước khi triển khai.

5. Lịch bậc ba của Zhu và Gupta

5.1 Công thức

Theo ghi chú pruning 03, độ thưa được tăng từ s_i (thường 0) đến s_f trong n lần cắt, bắt đầu ở bước huấn luyện t_0, mỗi Δt bước cắt một lần:

$$s_t = s_f + (s_i - s_f)\Big(1 - \frac{t - t_0}{n\,\Delta t}\Big)^3, \qquad t \in \{t_0,\, t_0 + \Delta t, \ldots, t_0 + n\Delta t\}.$$

Đặt $u = \dfrac{t - t_0}{n\,\Delta t} \in [0, 1]$ cho gọn: $s = s_f + (s_i - s_f)(1 - u)^3$. Với $s_i = 0$ thì $s = s_f\big(1 - (1 - u)^3\big)$. Ở mỗi lần cắt, mặt nạ được cập nhật bằng cách xoá các trọng số có độ lớn nhỏ nhất (theo từng lớp) đến khi đạt độ thưa s_t; giữa hai lần cắt, mô hình huấn luyện bình thường trên trọng số còn sống.

5.2 Vì sao bậc ba

Số mũ 3 làm s tăng rất nhanh lúc đầu và gần như phẳng lúc cuối: đạo hàm $\dfrac{ds}{du} = 3s_f(1 - u)^2$ giảm từ $3s_f$ xuống 0. Lý do theo ghi chú pruning 03: lúc đầu mạng còn rất nhiều kết nối dư thừa nên cắt nhanh ít đau; càng về sau càng ít trọng số thừa nên mỗi lần cắt phải nhỏ để mô hình kịp phục hồi.

5.3 Ví dụ tay: năm lần cắt, $s_i = 0$, $s_f = 0{,}9$

Đặt $n = 5$ và $u = k/5$ với $k = 1, \ldots, 5$. Từng bước (tự tính):

- $k = 1$: $u = 0{,}2$, $(1-u)^3 = 0{,}8^3 = 0{,}512$, nên $s = 0{,}9 \times (1 - 0{,}512) = 0{,}9 \times 0{,}488 = 0{,}4392$.
- $k = 2$: $0{,}6^3 = 0{,}216$, $s = 0{,}9 \times 0{,}784 = 0{,}7056$.
- $k = 3$: $0{,}4^3 = 0{,}064$, $s = 0{,}9 \times 0{,}936 = 0{,}8424$.
- $k = 4$: $0{,}2^3 = 0{,}008$, $s = 0{,}9 \times 0{,}992 = 0{,}8928$.
- $k = 5$: $0^3 = 0$, $s = 0{,}9$.

Giả sử lớp có 1000 trọng số. Số còn sống sau từng lần cắt: $561,\ 294,\ 158,\ 107,\ 100$ (làm tròn). Lượng thưa thêm ở mỗi lần: $43{,}92;\ 26{,}64;\ 13{,}68;\ 5{,}04;\ 0{,}72$ điểm phần trăm (tự tính), tổng đúng 90. So sánh lịch tuyến tính $s_k = 0{,}18k$: $0{,}18;\ 0{,}36;\ 0{,}54;\ 0{,}72;\ 0{,}9$, tức mỗi lần cắt thêm 18 điểm đều. Ngay lần cắt đầu, bậc ba đã cắt 43,9% so với 18%, nhưng ở lần cắt thứ 5 chỉ còn 0,72% so với 18%.

Một mốc dễ nhớ: tại $u = \frac{1}{3}$, $s = 0{,}9 \times (1 - (2/3)^3) = 0{,}9 \times 0{,}7037 \approx 0{,}633$; tức đi một phần ba lịch đã thưa 63,3% (tự tính; ghi chú pruning 03 cũng nêu mốc này).

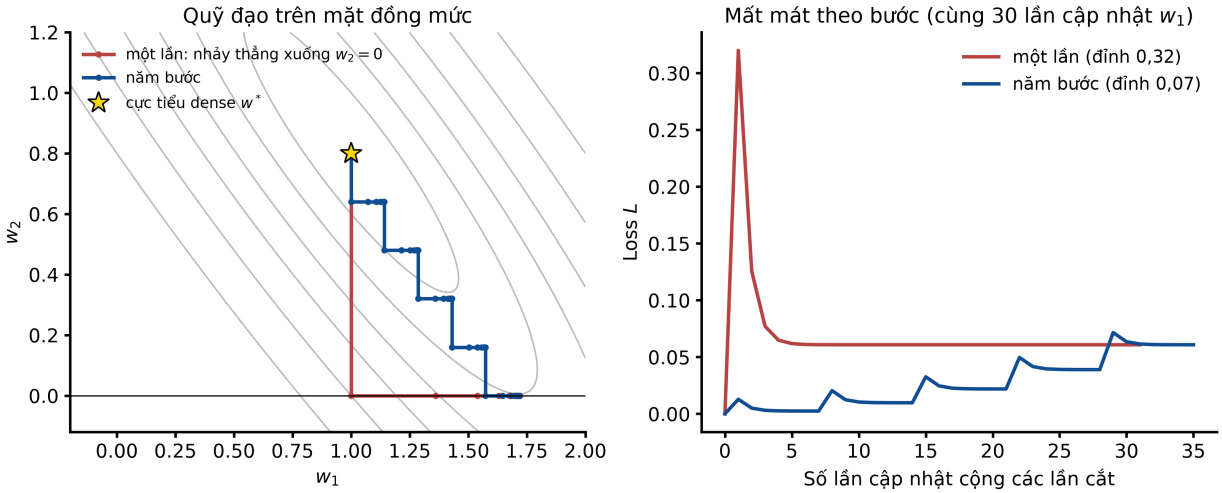
5.4 Chọn siêu tham số

Theo ghi chú pruning 03: Δt từ 100 đến 1000 bước ít ảnh hưởng đến kết quả cuối; t_0 thường sau vài epoch hoặc bắt đầu từ mô hình đã huấn luyện; độ dài lịch phải gắn với learning rate vì nếu learning rate đã quá nhỏ, mô hình khó hồi phục sau cú cắt. Cũng theo ghi chú đó, với InceptionV3 khi độ thưa đạt 87,5% có thời điểm mô hình suy giảm rất mạnh rồi hồi phục nhanh khi tiếp tục huấn luyện. Ví dụ tay (tự tính): với $s_f = 0{,}9$, $n = 10$, $\Delta t = 100$, $t_0 = 1000$, tại bước $t = 1300$ ta có $u = 300/1000 = 0{,}3$ và $s = 0{,}9 \times (1 - 0{,}7^3) = 0{,}9 \times 0{,}657 = 0{,}5913$.

6. Mô phỏng hai biến: một cú nhảy lớn so với nhiều bước nhỏ

Tiếp ví dụ ở mục 2.2. Ta so hai cách đưa w_2 từ $0{,}8$ về 0 mà vẫn cho w_1 tự điều chỉnh bằng gradient descent ($\eta = 0{,}5$):

- **Một lần:** đặt $w_2 = 0$ ngay, rồi 30 bước cập nhật w_1.
- **Năm bước:** hạ w_2 xuống $0{,}64;\ 0{,}48;\ 0{,}32;\ 0{,}16;\ 0$, sau mỗi lần hạ có 6 bước cập nhật w_1 (tổng cũng 30 bước).



Hình 18: Mô phỏng hai biến: quỹ đạo trên mặt đồng mức (trái) và loss theo bước (phải) khi cắt w_2 một lần so với năm bước (minh hoạ, tự tính)

Cách đọc hình: bên trái, ngôi sao vàng là cực tiểu dense $(1;\ 0{,}8)$; các đường xám là đường đồng mức của loss (các hình elip dài hẹp vì hai biến tương quan). Đường đỏ rơi thẳng xuống trục $w_2 = 0$ rồi trượt sang phải;

đường xanh đi bậc thang. Cả hai kết thúc cùng một điểm $w_1 \approx 1{,}72$ trên trục ngang. Bên phải, loss của cách một lần vọt lên 0,32 ngay sau cú cắt rồi giảm dần, còn cách năm bước chỉ có các đỉnh nhỏ (lớn nhất khoảng 0,07) và đều xấp xỉ 0,06 ở cuối.

Các con số tính tay (tự tính): sau cú cắt một lần, $\mathcal{L} = 0{,}32$. Gradient theo w_1 là $H_{11}(w_1 - 1) + H_{12}(w_2 - 0{,}8) = 0 + 0{,}9 \times (-0{,}8) = -0{,}72$, nên $w_1 \leftarrow 1 + 0{,}5 \times 0{,}72 = 1{,}36$ và $\mathcal{L} = 0{,}1256$. Bước kế tiếp cho $w_1 = 1{,}54$ và $\mathcal{L} = 0{,}077$, rồi dần tới điểm cố định. Điểm cố định là nơi gradient theo w_1 bằng 0: $w_1 - 1 + 0{,}9 \times (0 - 0{,}8) = 0$, tức $w_1 = 1{,}72$ và $\mathcal{L} = \frac{1}{2} \times 0{,}64 \times 0{,}19 = 0{,}0608$.

Đọc trung thực kết quả này. Trên hàm lồi hai biến, hai cách về cuối **cho cùng một điểm** (loss $0{,}0608$ trong cả hai trường hợp). Cái khác là *quá trình*: cú sốc lớn nhất của cắt một lần là $0{,}32$ còn của năm bước là khoảng $0{,}07$. Với mạng thật, cú sốc lớn có thể đẩy các tham số khỏi vùng tốt, nên chưa chắc quay lại được, và đó mới là chỗ lặp thật sự có lợi. Mô phỏng này không chứng minh gradual luôn tốt hơn; nó chỉ cho thấy cơ chế "cú sốc nhỏ hơn". Chương còn một lý do khác nằm ở mục 7.

7. Cắt lặp cho phép xem lại quyết định

Lý do thứ hai mà cắt nhiều bước có thể tốt hơn cắt một lần nằm ở chỗ: **sau khi cắt một phần và tinh chỉnh, độ lớn của các trọng số còn lại thay đổi, nên thứ hạng "ai nhỏ nhất" thay đổi theo**. Cắt một lần chốt quyết định dựa trên độ lớn *trước* khi các trọng số bù nhau; cắt lặp quyết định lại sau mỗi vòng.

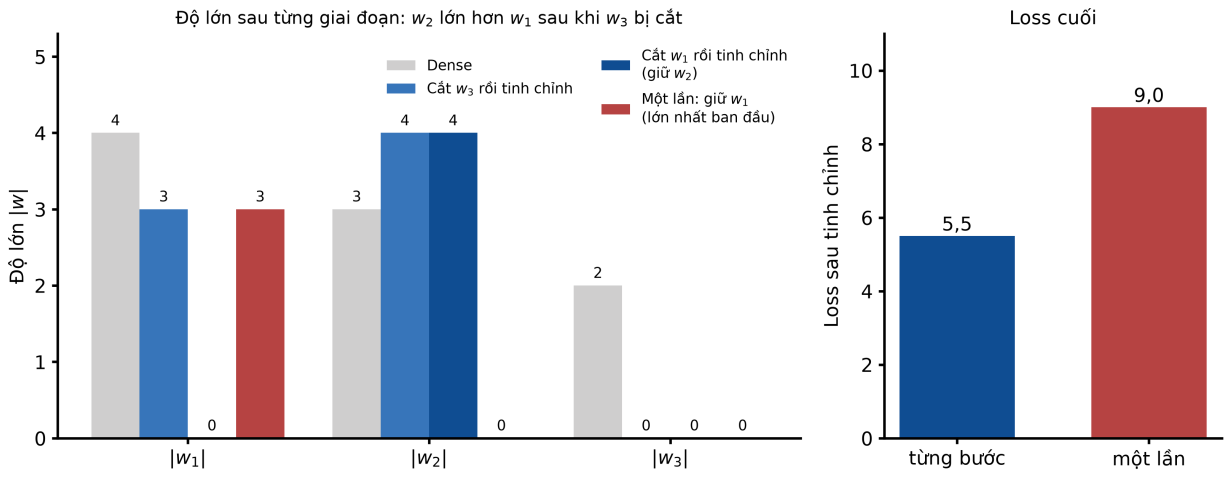
Ví dụ tay ba trọng số, cần giữ đúng 1 trong 3. Loss $\mathcal{L} = \frac{1}{2}(w - w^*)^\top H (w - w^*)$ với $w^* = (4;\ 3;\ 2)$ và

$$H = \begin{pmatrix} 1 & 0 & -0{,}5 \\ 0 & 1 & 0{,}5 \\ -0{,}5 & 0{,}5 & 1 \end{pmatrix}.$$

Khi chỉ giữ tập chỉ số S và tinh chỉnh đến hội tụ, nghiệm giải từ $H_{SS} w_S = (H w^*)_S$.

Cắt một lần theo độ lớn: giữ trọng số lớn nhất là w_1 ($|4|$). Tinh chỉnh: $w_1 = (Hw^*)_1 / H_{11} = 4 + 0 - 0{,}5 \times 2 = 3$. Loss: $d = (-1;\ -3;\ -2)$, $\mathcal{L} = \frac{1}{2}\big(1 + 9 + 4 + 2\,(-0{,}5 \cdot 2 + 0{,}5 \cdot 6)\big) = \frac{1}{2} \times 18 = 9$ (tự tính).

Cắt hai bước: (i) bỏ w_3 (nhỏ nhất, $|2|$), giữ $\{w_1, w_2\}$; vì $H_{12} = 0$ nên $w_1 = 4 - 0{,}5 \times 2 = 3$ và $w_2 = 3 + 0{,}5 \times 2 = 4$. Loss lúc này bằng 1 (tự tính). (ii) Bây giờ độ lớn là $(3;\ 4)$: w_1 trở thành nhỏ hơn. Bỏ w_1, giữ $w_2 = (Hw^*)_2 = 3 + 1 = 4$. Loss $= 5{,}5$ (tự tính).



Hình 19: Ba trọng số tương quan: độ lớn sau từng giai đoạn (trái) và loss cuối của cắt từng bước so với cắt một lần (phải) (minh hoạ, tự tính)

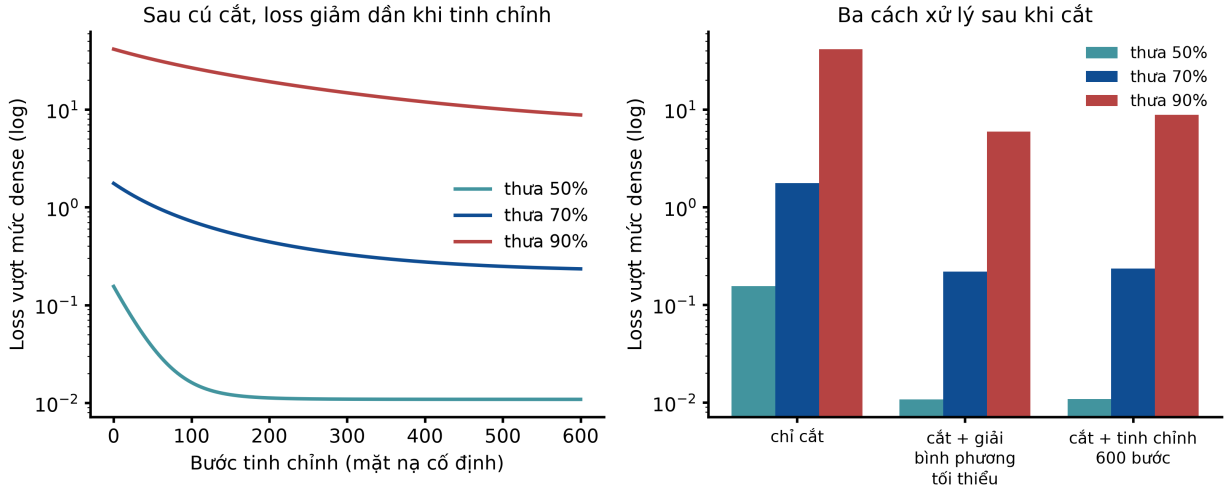
Cách đọc hình: ba nhóm cột là $|w_1|, |w_2|, |w_3|$; mỗi nhóm có bốn cột (dense, sau khi cắt w_3 rồi tinh chỉnh, giữ w_2 cuối cùng, và kết quả cắt một lần giữ w_1). Cột xanh nhạt cho thấy sau khi w_3 bị cắt, $|w_2|$ tăng từ 3 lên 4 và $|w_1|$ giảm từ 4 xuống 3: thứ hạng đã đảo. Bên phải, loss cuối của cách từng bước là $5{,}5$, của cách một lần là 9.

Đây là số trên ví dụ do ta chọn để thấy cơ chế; với mạng thật, không có bảo đảm lặp luôn hơn. Nhưng nó giải thích vì sao các bài về cắt tỉa có xu hướng lặp hoặc cắt dần khi có đủ tài nguyên.

8. Chữa lành sau cú cắt: huấn luyện lại, distillation và LoRA

8.1 Tinh chỉnh với mặt nạ cố định

Cách cơ bản nhất: tiếp tục huấn luyện với loss ban đầu, mặt nạ cố định. Hình sau mô phỏng trên hồi quy tuyến tính 100 chiều có đặc trưng tương quan mạnh: cắt theo độ lớn 50%, 70%, 90% rồi tinh chỉnh bằng gradient descent có mặt nạ. Trục dọc là loss vượt mức dense (đơn vị tuỳ ý).



Hình 20: Mô phỏng hồi quy tuyến tính: loss sau cú cắt giảm dần khi tinh chỉnh (trái), và so sánh chỉ cắt, cắt kèm một lần giải bình phương tối thiểu, cắt kèm tinh chỉnh 600 bước (phải) (minh hoạ, tự tính)

Cách đọc hình: bên trái, ba đường là ba độ thưa, trục dọc ở thang log. Mỗi đường bắt đầu cao ngay sau cú cắt (0,155; 1,75; 41,4 ở 50%, 70%, 90%) rồi giảm dần; độ thưa càng cao thì đường càng cao và hồi phục càng chậm (ở 90% sau 600 bước vẫn còn 8,78). Bên phải, với mỗi độ thưa có ba cột: chỉ cắt; cắt rồi giải một lần bài toán bình phương tối thiểu cho các trọng số còn lại (không dùng gradient); và cắt rồi tinh chỉnh 600 bước. Cột giữa và cột phải gần nhau ở 50% (0,0107 và 0,0108) và 70% (0,218 và 0,234), còn ở 90% cách tinh chỉnh chưa hội tụ (5,91 so với 8,78). Đây là mô phỏng tuyến tính: cách giải một lần chính là đích mà tinh chỉnh dài sẽ tiến tới, nên không phản ánh hết chuyện của mạng phi tuyến.

8.2 Tinh chỉnh bằng distillation

Distillation (chưng cất tri thức) dùng chính mô hình gốc làm **giáo viên**: mô hình đã cắt (học trò) được huấn luyện để đầu ra của nó gần đầu ra của giáo viên, thường bằng độ phân kỳ KL giữa hai phân phối xác suất. Lợi thế: tín hiệu học dày hơn nhãn cứng, vì giáo viên cho cả phân phối trên mọi lớp (hoặc mọi token). Theo ghi chú pruning 39 (Minitron), mô hình sau cắt được huấn luyện lại bằng KL trên logits so với giáo viên là mô hình gốc; Minitron 8B cắt từ Nemotron-4 15B dùng 94 tỉ token, bài ghi khoảng 40 lần ít hơn so với Nemotron-3 8B. Cũng theo ghi chú đó, thứ hạng các lựa chọn cắt có thể đảo sau vài trăm bước huấn luyện lại, nghĩa là đánh giá cấu hình ngay sau cú cắt có thể sai lệch.

8.3 Tinh chỉnh bằng LoRA

Huấn luyện lại toàn bộ trọng số của LLM đắt về bộ nhớ (lưu gradient, trạng thái optimizer). **LoRA** (low-rank adaptation) giữ nguyên trọng số W và chỉ học một số hạng hiệu chỉnh hạng thấp: $W' = W + BA$ với $B \in \mathbb{R}^{R\times r}$, $A \in \mathbb{R}^{r\times C}$, $r \ll \min(R, C)$. Ví dụ đếm tham số (tự tính): một ma trận 4096×4096 có $16\,777\,216$ phần tử; với $r = 8$, LoRA có $2 \times 4096 \times 8 = 65\,536$ tham số học thêm, tức khoảng 0,39%.

Số liệu từ ghi chú pruning 24 (Wanda, LLaMA-7B, mặt nạ cố định, LoRA hạng 8, khoảng 12 giờ trên 1 GPU): với 50% không cấu trúc, điểm zero-shot trung bình từ 54,21 lên 56,53 sau LoRA, và lên 58,15 khi tinh chỉnh toàn bộ (dense là 59,99); với 2:4, từ 48,53 lên 54,46 sau LoRA, và 56,19 khi tinh chỉnh toàn bộ. Tức LoRA lấy lại phần lớn phần mất, đặc biệt ở mẫu 2:4 vốn ràng buộc mạnh hơn. Theo ghi chú pruning 26 (LLM-Pruner), LLaMA-7B cắt 20% tham số theo điểm bậc hai có trung bình 7 tác vụ là 56,82 khi chưa tinh chỉnh và 60,07 sau LoRA (mô hình gốc 63,25, số tác giả chạy lại).

9. Vì sao một số phương pháp LLM không cần huấn luyện lại

Ba lý do thường gặp:

1. **Chi phí và dữ liệu.** Mô hình hàng chục đến hàng trăm tỉ tham số không thể huấn luyện lại thường xuyên; dữ liệu gốc đôi khi không có.
2. **Bù bằng lời giải đóng, theo từng lớp.** Thay vì gradient trên loss toàn mô hình, các phương pháp như SparseGPT giải bài toán tái tạo đầu ra từng lớp: chọn mặt nạ rồi cập nhật các trọng số còn lại để đầu ra lớp gần như cũ trên một ít dữ liệu hiệu chuẩn (ghi chú pruning 23). Trong bài toán tuyến tính ở hình mục 8.1, đó chính là cột "cắt kèm giải bình phương tối thiểu": đạt gần đích mà tinh chỉnh dài mới tới.
3. **Chỉ cần tiêu chí tốt.** Theo ghi chú pruning 24, Wanda không cập nhật trọng số còn lại và không cần gradient; chỉ chấm điểm bằng độ lớn trọng số nhân chuẩn của kích hoạt đầu vào, và ở độ thưa 50% cho kết quả ngang SparseGPT (ví dụ LLaMA-7B perplexity 7,26 so với 7,22).

Với bỏ cả lớp, ghi chú pruning 37 cho thấy một ma trận sửa nhỏ giải bằng bình phương tối thiểu (có chính quy hoá ridge) trên activation của mô hình nguyên có thể thay phần bị bỏ. Phát hiện chính của bài: chọn lớp **dưới đúng sửa chữa sẽ dùng** cho kết quả khác chọn lớp không tính sửa chữa, tối ưu có sửa thường là một khối lớp liền nhau còn không sửa thì tách thành vài đoạn rời; ghi chú ghi rằng các phương pháp bỏ qua sửa chữa có thể bỏ lỡ tối ưu tới 189%. Bài học: bước chữa không chỉ là việc làm sau, nó thay đổi cả câu hỏi "nên cắt gì".

Cần nói rõ: không huấn luyện lại không có nghĩa là không tốn gì. Chất lượng ở độ thưa cao thường kém hơn nếu có huấn luyện lại; và một số bài kết hợp cả hai (cắt một lần bằng tiêu chí tốt, rồi tinh chỉnh nhẹ bằng LoRA, như số liệu Wanda ở mục 8.3).

10. Biến thể và chi tiết hay bị bỏ qua

- **Đặt lại tham số hay giữ nguyên.** Ở Lottery Ticket (ghi chú pruning 08), sau mỗi vòng cắt trọng số còn lại được đặt về giá trị khởi tạo θ_0 rồi huấn luyện lại (Strategy 1); ngược lại, nhiều quy trình khác giữ trọng số đã huấn luyện và chỉ tinh chỉnh. Hai cách trả lời hai câu hỏi khác nhau: "có mạng con nào huấn luyện từ đầu được không" so với "làm sao phục hồi mô hình đã có".
- **Đặt lại optimizer và learning rate.** Ghi chú pruning 18 nhấn mạnh sau khi cắt 2:4, huấn luyện lại dùng lại đúng lịch cũ với siêu tham số và trạng thái optimizer được đặt lại (không dò thêm siêu tham số). Ghi chú pruning 02 dùng learning rate nhỏ hơn: khoảng 1/10 (LeNet) đến 1/100 (AlexNet) so với lúc huấn luyện gốc.
- **Learning rate quá nhỏ làm mất khả năng hồi phục.** Đã nêu ở ghi chú pruning 03: nếu lịch cắt rơi vào lúc learning rate đã rất nhỏ, mô hình bị kẹt ở trạng thái sau cú cắt.
- **Cắt cục bộ theo lớp hay toàn cục.** Cắt cùng tỉ lệ ở mọi lớp (cục bộ) đơn giản nhưng có thể phá lớp mỏng; cắt toàn cục (so sánh mọi trọng số của cả mạng) chọn tự nhiên hơn nhưng có thể gần như xoá sạch

một lớp. Ghi chú pruning 08 dùng cắt cục bộ cho mạng nhỏ và cắt toàn cục cho VGG-19 và ResNet-18, vì lớp đầu rất nhỏ trở thành nút cổ chai nếu cắt cùng tỉ lệ.

- **Cắt trong lúc tinh chỉnh.** Movement Pruning (ghi chú pruning 16) quyết định mặt nạ *trong* quá trình tinh chỉnh theo hướng di chuyển của trọng số, thay vì cắt rồi mới tinh chỉnh. Nó đặt cạnh nhau hai bước mà các mục trên tách rời.
- **Huấn luyện lại không phải lúc nào cũng cần thiết với CNN nhỏ.** Câu hỏi "cắt rồi huấn luyện lại có hơn huấn luyện mô hình nhỏ từ đầu không" có câu trả lời khác nhau tuỳ cấu hình; các ghi chú pruning 11 và 43 bàn riêng.
- **Phục hồi không đều giữa các độ thưa.** Như mô phỏng mục 8.1: độ thưa vừa phải hồi phục nhanh, độ thưa cao hồi phục chậm và chưa chắc hội tụ trong ngân sách cho trước.

11. Nối sang pruning và LLM

Pruning 02. Quy trình huấn luyện, cắt, huấn luyện lại, lặp nhiều vòng. Theo ghi chú, AlexNet giữ 11% trọng số, VGG-16 giữ 7,5% trọng số với 5 vòng cắt và huấn luyện lại; đây là khuôn mẫu cắt lặp cổ điển.

Pruning 03. Lịch bậc ba của chương này, gắn trực tiếp vào quá trình huấn luyện, thay thế các vòng lặp rời. Kèm so sánh mô hình thưa lớn với mô hình dày nhỏ cùng bộ nhớ.

Pruning 08. Cắt lặp theo độ lớn kèm đặt lại về khởi tạo; câu hỏi chuyển từ "chữa mô hình" sang "tìm mạng con có thể huấn luyện được".

Pruning 16. Quyết định mặt nạ trong lúc tinh chỉnh, phù hợp khi nhắm đến độ thưa rất cao.

Pruning 23 và 24. Hai phương pháp cắt một lần cho LLM, không huấn luyện lại; SparseGPT bù bằng cập nhật trọng số theo từng lớp, Wanda chỉ dùng tiêu chí chấm điểm tốt hơn. Cả hai là bằng chứng rằng cú sốc cắt 50% trên LLM lớn có thể nhỏ khi tiêu chí đủ tốt, và LoRA có thể bổ sung (ghi chú pruning 24).

Pruning 26. Cắt có cấu trúc theo nhóm phụ thuộc cho LLaMA, phục hồi bằng LoRA; điểm trung bình lên từ 56,82 đến 60,07 như đã nêu.

Pruning 37. Chọn lớp để bỏ khi biết sẽ có bước sửa; một ví dụ rõ nhất của chuyện "cách chữa quyết định cách cắt".

Pruning 39. Huấn luyện lại bằng distillation và nhận xét thứ hạng cấu hình đảo sau vài trăm bước huấn luyện lại: đánh giá ngay sau cú cắt chưa đủ để chọn cấu hình.

12. Sai lầm hay gặp

- **Đánh giá mô hình ngay sau cú cắt rồi kết luận phương pháp tệ.** Nhiều mô hình chỉ cho thấy năng lực thật sau tinh chỉnh; ghi chú pruning 39 cho thấy thứ hạng cấu hình còn có thể đảo.
- **So sánh một-lần với lặp mà không cân bằng chi phí.** Lặp tốn nhiều lần huấn luyện; so sánh công bằng cần cùng ngân sách số bước hoặc cùng chi phí tính toán.
- **Để lịch cắt rơi vào lúc learning rate đã rất nhỏ.** Mô hình không có sức để hồi phục sau cú cắt, như ghi chú pruning 03 cảnh báo.
- **Quên nhân mặt nạ sau mỗi bước cập nhật.** Trọng số đã bị cắt "sống lại" bằng gradient, độ thưa thực tế thấp hơn tưởng.
- **Tin rằng cắt lặp luôn tốt hơn cắt một lần.** Trên bài toán lồi hai biến ở mục 6 chúng cùng đích; sự khác biệt chỉ rõ khi quyết định cắt thay đổi (mục 7) hoặc khi cú sốc làm hỏng mô hình.
- **Đánh đồng "không huấn luyện lại" với "miễn phí".** Các phương pháp một lần vẫn cần dữ liệu hiệu chuẩn và tính toán; ở độ thưa cao chúng thường vẫn kém hơn phương pháp có huấn luyện lại.

13. Tóm tắt

Khái niệm	Công thức hoặc con số	Ghi chú
Tinh chỉnh với mặt nạ	w ← m nhân (w trừ eta nhân gradient)	mặt nạ cố định, nhân sau mỗi bước
Cắt lặp, giữ q mỗi vòng	còn lại q^n sau n vòng; q = r^(1/n) để còn r	q = 0,8: 11 vòng còn 8,6%
Lịch bậc ba	s = s_f + (s_i - s_f)(1 - u)^3, u từ 0 đến 1	s_f = 0,9, n = 5: 0,4392; 0,7056; 0,8424; 0,8928; 0,9
Tốc độ cắt của lịch	ds/du = 3 s_f (1 - u)^2	nhanh đầu, chậm cuối
LoRA	W’ = W + BA, tham số 2 x r x kích thước	r = 8 trên ma trận 4096 x 4096: 65 536 (0,39%)
Distillation	KL giữa đầu ra học trò và giáo viên	giáo viên là mô hình gốc

14. Bài tập tự tính

Bài 1 (dễ). Lịch bậc ba với $s_i = 0$, $s_f = 0{,}8$, $n = 4$ lần cắt. Tính độ thưa sau từng lần cắt và lượng thưa thêm ở mỗi lần.

Đáp án (tự tính): $u = k/4$: $s_1 = 0{,}8 \times (1 - 0{,}75^3) = 0{,}8 \times 0{,}578125 = 0{,}4625$; $s_2 = 0{,}8 \times (1 - 0{,}125) = 0{,}7$; $s_3 = 0{,}8 \times (1 - 0{,}015625) = 0{,}7875$; $s_4 = 0{,}8$. Lượng thêm: $0{,}4625;\ 0{,}2375;\ 0{,}0875;\ 0{,}0125$ (tổng $0{,}8$).

Bài 2. Cắt lặp, mỗi vòng cắt 20% số trọng số còn sống. Cần bao nhiêu vòng để còn không quá 5% so với ban đầu? Sau 5 vòng còn bao nhiêu?

Đáp án (tự tính): $0{,}8^k \le 0{,}05$ cho $k \ge \ln 0{,}05 / \ln 0{,}8 \approx 13{,}4$, nên $k = 14$ ($0{,}8^{13} \approx 5{,}5\%$ vẫn lớn hơn 5%, $0{,}8^{14} \approx 4{,}4\%$). Sau 5 vòng còn $0{,}8^5 = 0{,}32768$, tức $32{,}8\%$.

Bài 3. Lịch bậc ba với $s_i = 0$, $s_f = 0{,}9$, $n = 10$, $\Delta t = 100$, $t_0 = 1000$. Tính độ thưa ở bước $t = 1500$.

Đáp án (tự tính): $u = (1500 - 1000)/(10 \times 100) = 0{,}5$; $s = 0{,}9 \times (1 - 0{,}5^3) = 0{,}9 \times 0{,}875 = 0{,}7875$.

Bài 4. Hàm hai biến của mục 6 ($H = \begin{pmatrix} 1 & 0{,}9 \\ 0{,}9 & 1 \end{pmatrix}$, $w^* = (1;\ 0{,}8)$). Cắt w_2 từ $0{,}8$ xuống $0{,}64$ (bước đầu của cách năm bước) khi $w_1 = 1$: loss là bao nhiêu? Và loss tại điểm cố định khi $w_2 = 0$?

Đáp án (tự tính): $d = (0;\ -0{,}16)$ nên $\mathcal{L} = \frac{1}{2} \times 0{,}16^2 = 0{,}0128$ (so với $0{,}32$ của cú nhảy một lần: nhỏ hơn 25 lần). Điểm cố định $w_1 = 1 + 0{,}9 \times 0{,}8 = 1{,}72$, $\mathcal{L} = \frac{1}{2} \times 0{,}64 \times (1 - 0{,}9^2) = 0{,}0608$.

Bài 5 (khó). Đổi vai ví dụ mục 7: $w^* = (3;\ 4;\ 2)$ và $H = \begin{pmatrix} 1 & 0 & 0{,}5 \\ 0 & 1 & -0{,}5 \\ 0{,}5 & -0{,}5 & 1 \end{pmatrix}$, cần giữ 1 trong 3 trọng số. Cắt một lần theo độ lớn giữ trọng số nào và loss sau tinh chỉnh bằng bao nhiêu? Cắt hai bước (bỏ nhỏ nhất, tinh chỉnh, bỏ nhỏ nhất, tinh chỉnh) giữ trọng số nào và loss bằng bao nhiêu?

Đáp án (tự tính): một lần giữ w_2 (lớn nhất, 4); tinh chỉnh $w_2 = (Hw^*)_2 = 0 + 4 - 0{,}5 \times 2 = 3$; loss $= 9$. Hai bước: bỏ w_3 (2), giữ $\{w_1, w_2\}$ với $H_{12} = 0$: $w_1 = 3 + 0{,}5 \times 2 = 4$, $w_2 = 4 - 0{,}5 \times 2 = 3$ (loss 1); lúc này $|w_2| = 3 < |w_1| = 4$ nên bỏ w_2, giữ $w_1 = (Hw^*)_1 = 3 + 1 = 4$; loss $= 5{,}5$. Kết luận: cắt hai bước giữ w_1 và tốt hơn ($5{,}5$ so với 9).

Tài liệu tham khảo nền: ghi chú pruning 02 và 03 (khuôn mẫu cắt lặp và lịch bậc ba), 08 (cắt lặp với đặt lại khởi tạo), 16 (cắt trong lúc tinh chỉnh), 23 và 24 (cắt một lần không huấn luyện lại), 26 (LoRA sau cắt), 37 (sửa sau khi bỏ lớp), 39 (distillation sau cắt) trong cuốn này; optimization nền 02 và 06 về gradient descent và lịch learning rate.

Nền tảng 06

Cắt lớp và độ sâu trong LLM: dòng dư làm cho bỏ một lớp ít phá vỡ tính toán, nhưng chỉ khi chọn đúng lớp

Loại: chương nền tảng (không phải ghi chú paper)

Đọc trước: vector, tích vô hướng và cosin ở math nền tảng 03; chain rule và backprop ở math nền tảng 07; cấu trúc khối Transformer (attention rồi MLP, có chuẩn hoá) ở các ghi chú LLM; phần phân loại cắt tỉa trong pruning 00

Dùng ở: pruning 31 (ShortGPT, Block Influence), pruning 32 (cắt khối lớp liền kề, chữa bằng QLoRA), pruning 33 (độ sâu so với chiều rộng, độ trễ), pruning 34 (chọn lặp), pruning 35 (gộp lớp), pruning 36 (thay bằng ánh xạ tuyến tính), pruning 37 (sửa chữa đổi lớp nên cắt)

> Chương này giải thích từ đầu, không phải bản dịch paper. Các phép tính ghi "tự tính" đã kiểm bằng numpy. Các hình mô phỏng dùng mạng ngẫu nhiên tự sinh (seed cố định) để minh hoạ hình dạng chung, không phải số đo từ một mô hình ngôn ngữ thật.

1. Vì sao cần hiểu chuyện "bỏ cả một lớp"

Bỏ một lớp nghĩa là xoá nguyên khối attention và MLP của lớp đó, để mô hình có ít lớp hơn nhưng mọi ma trận còn lại giữ nguyên kích thước. Cách này khác hẳn việc làm thưa từng trọng số hay thu hẹp từng ma trận. Nó hấp dẫn vì rất dễ triển khai: mô hình mới chỉ là cùng kiến trúc với ít lớp hơn, chạy được trên mọi thư viện, và tốc độ giảm gần tỉ lệ với số lớp bỏ.

Câu hỏi đáng sợ là: mô hình được huấn luyện cùng lúc cả 32 lớp, lớp sau "tin" vào đầu ra của lớp trước, vậy xoá một lớp giữa chừng thì phần còn lại có đổ vỡ không? Câu trả lời của các LLM hiện đại thường là: phần lớn lớp ở giữa và sau bỏ được với thiệt hại nhỏ, còn vài lớp đầu và lớp cuối thì không. Lý do cốt lõi nằm ở một chi tiết kiến trúc, **dòng dư** (residual stream). Chương này đi từ chi tiết đó lên đến các phương pháp đo "lớp nào đáng bỏ" trong các bài 31 đến 37.

Trong LLM, hiểu chuyện này cho bạn ba thứ: một công cụ chẩn đoán (lớp nào đang làm việc thật), một cách nén (bỏ lớp để giảm bộ nhớ và độ trễ), và cảnh báo (vì sao chỉ số đơn giản đôi khi chọn sai, vì sao sau khi bỏ nhiều lớp thì chất lượng tụt nhanh).

2. Nhắc nền: lớp, cosin và độ lớn

2.1 Một lớp Transformer là "hàm cộng thêm"

Gọi $x_{l-1} \in \mathbb{R}^d$ là vector ẩn (hidden state) của một token trước lớp l. Một lớp Transformer pre-norm (chuẩn hoá đặt trước attention và MLP, như Llama) tính

$$x_l = x_{l-1} + f_l(x_{l-1}),$$

trong đó f_l gói gọn cả khối attention và khối MLP của lớp, mỗi khối có chuẩn hoá ở đầu vào và được **cộng** vào dòng chính. Để dạy trực giác, chương này coi f_l như một hàm đen: nhận x_{l-1}, trả ra một vector cùng chiều d. Phép cộng $x_{l-1} + \cdots$ là **kết nối dư** (residual connection), còn chuỗi $x_0, x_1, \ldots, x_L$ là **dòng dư**. Ở đây x_0 là embedding của token, còn x_L là thứ được đưa tới đầu ra cuối để dự đoán token kế tiếp.

Mở công thức ra ta được dạng tổng:

$$x_L = x_0 + f_1(x_0) + f_2(x_1) + \cdots + f_L(x_{L-1}).$$

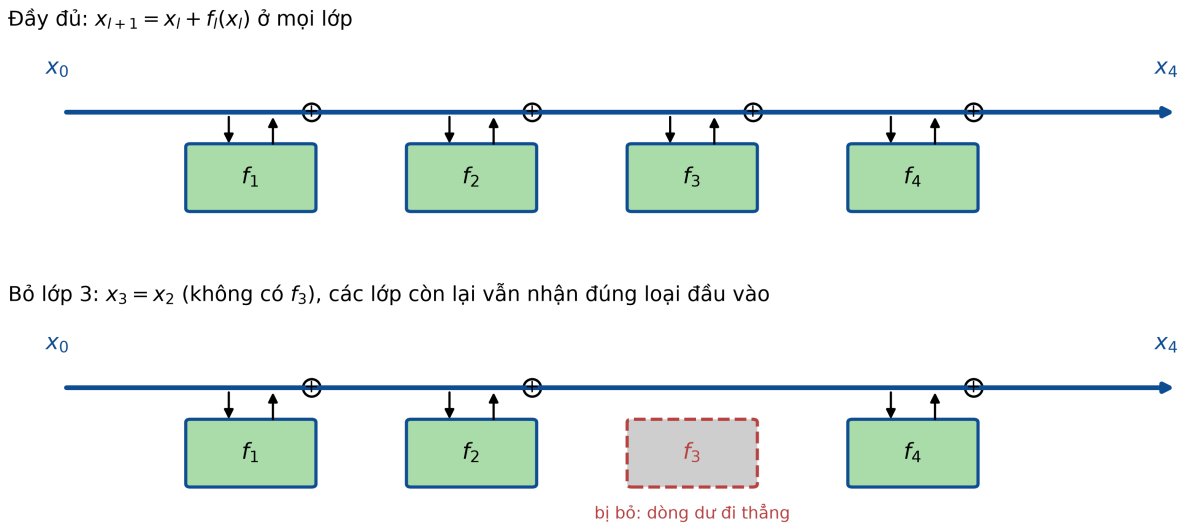
Đây là hình dung quan trọng nhất của chương: đầu ra cuối là **embedding cộng với một chuỗi các đóng góp**, mỗi lớp đóng góp một số hạng. Mỗi lớp không "thay" biểu diễn, nó "chỉnh thêm" biểu diễn.

2.2 Cosin và độ lớn

Hai vector a, b có cosin $\cos(a,b) = \dfrac{a \cdot b}{\|a\|\,\|b\|} \in [-1, 1]$. Cosin chỉ đo **góc** giữa hai vector: 1 nghĩa là cùng hướng, 0 là vuông góc. Nó **không** đo độ dài. Ví dụ (tự tính) $a = (3, 4)$ và $b = (6, 8)$ cùng hướng nên cosin bằng 1 dù b dài gấp đôi.

Một ví dụ nhỏ: $a = (1, 0)$, $b = (1, 1)$ có $a \cdot b = 1$, $\|a\| = 1$, $\|b\| = \sqrt{2}$, nên $\cos = 1/\sqrt{2} \approx 0{,}707$ (tự tính), tức góc 45 độ. Hai vector càng giống hướng thì cosin càng sát 1, và "$1 - \cos$" là một thước đo độ lệch hướng không âm (khi cosin không âm) mà ta sẽ dùng lại.

3. Trực giác bằng hình: bỏ một lớp khi có dòng dư



Hình 21: Sơ đồ dòng dư: khi bỏ lớp 3, dòng chính đi thẳng qua và các lớp sau vẫn nhận đầu vào cùng loại (minh hoạ, tự tính)

Cách đọc hình: đường xanh đậm nằm ngang là dòng dư, đi từ x_0 bên trái đến x_4 bên phải. Mỗi ô xanh lá là một lớp f_l: nó **đọc** từ dòng (mũi tên xuống), tính, rồi **cộng** kết quả trở lại (dấu cộng trong vòng tròn). Hàng dưới, lớp 3 bị bỏ (ô xám nét đứt): dòng đi thẳng, tức là $x_3 = x_2$, còn lớp 4 vẫn đọc được một vector cùng không gian và cùng "kiểu" với trước, chỉ thiếu đi một phần cộng thêm.

Điểm cần nhớ so với mạng không có đường cộng, ví dụ $x_l = f_l(x_{l-1})$. Ở mạng không có đường cộng, xoá một lớp khiến lớp sau nhận x_{l-2} thay cho x_{l-1}, một vector nằm trong "ngôn ngữ biểu diễn" của lớp trước đó, không phải thứ lớp sau được huấn luyện để đọc. Với dòng dư, x_{l-1} và x_{l-2} khác nhau đúng một số hạng, nên chúng giống nhau khi số hạng đó nhỏ so với phần còn lại.

Trực giác này chỉ là **điều kiện đủ để hy vọng**, không phải bảo đảm. Mạng có dòng dư vẫn có lớp không bỏ được, ví dụ lớp đóng góp rất lớn. Phần sau sẽ đo "lớn" như thế nào.

4. Một lớp "đóng góp" bao nhiêu, đo bằng cosin

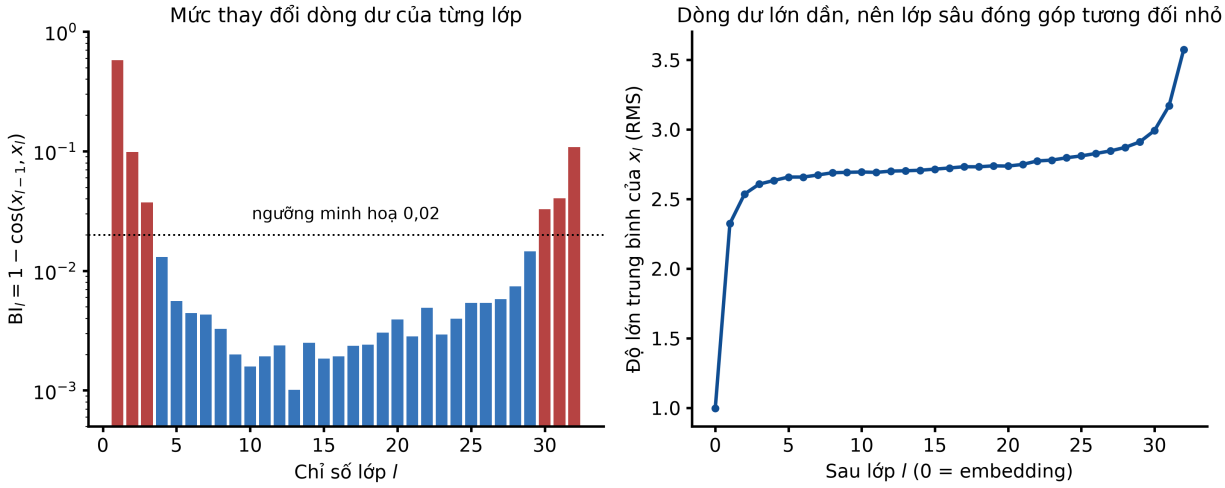
4.1 Block Influence

Nếu lớp l chỉ cộng một vector nhỏ, thì x_l gần cùng hướng với x_{l-1}, cosin của chúng gần 1. Ngược lại, lớp đóng góp mạnh làm hướng của vector đổi nhiều. Từ đó có chỉ số được dùng trong ShortGPT (theo ghi chú pruning 31, gọi là Block Influence, BI): lấy một tập calibration nhỏ không nhãn, tính trung bình trên các token

$$\mathrm{BI}_l = 1 - \mathbb{E}_{\text{token}}\left[\cos\left(x_{l-1}, x_l\right)\right].$$

BI thấp nghĩa là lớp gần như hàm đồng nhất (ứng viên để bỏ), BI cao nghĩa là lớp biến đổi biểu diễn mạnh. Theo ghi chú pruning 31, sau khi sắp xếp BI, bài báo xoá các lớp BI thấp nhất, không dùng gradient và không huấn luyện lại; trên Llama2-13B bỏ khoảng 25% tham số vẫn giữ khoảng 90% hiệu năng trung bình (ghi chú nêu số này; xem bài đó để biết điều kiện). Đây là điểm xuất phát, không phải kết luận phổ quát cho mọi mô hình.

4.2 Hình dạng thường thấy theo độ sâu



Hình 22: BI theo từng lớp và độ lớn của dòng dư trong mạng residual mô phỏng 32 lớp (minh hoạ, tự tính)

Cách đọc hình: bên trái là BI_l theo thang log của từng lớp trong một mạng 32 lớp ngẫu nhiên tự sinh, cột đỏ nếu BI trên $0{,}02$, xanh nếu dưới. Bên phải là độ lớn trung bình (RMS) của dòng dư sau mỗi lớp. Hình dạng chữ U, lớp đầu và lớp cuối cao còn lớp giữa thấp, do **tôi thiết kế cho mạng mô phỏng** (độ lớn đóng góp lớn ở hai đầu, nhỏ ở giữa), để minh hoạ cơ chế; nó không chứng minh điều đó đúng cho từng LLM thật. Điều hình thật sự cho thấy là sự phụ thuộc: khi dòng dư đã lớn và gần phẳng ở giữa mạng (RMS quanh $2{,}7$), cùng một đóng góp tuyệt đối cho BI nhỏ hơn.

Trong mô phỏng này (tự tính): $\mathrm{BI}_1 \approx 0{,}58$, $\mathrm{BI}_{32} \approx 0{,}11$, còn trung bình cosin ở các lớp từ 9 đến 24 xấp xỉ $0{,}997$.

Quan sát thực nghiệm tương ứng được các bài 31 và 32 mô tả: lớp đầu và lớp cuối quan trọng nhất, phần giữa và sâu dư hơn. Giải thích hay gặp (là giả thuyết, không phải định lý): các lớp đầu biến embedding thành biểu diễn có thể dùng, các lớp cuối chuẩn bị cho đầu ra, còn các lớp giữa tinh chỉnh dần một biểu diễn đã khá hoàn chỉnh nên mỗi lớp chỉ đóng góp phần nhỏ.

5. Định nghĩa chính thức

Cho L lớp, $x_l = x_{l-1} + f_l(x_{l-1})$, và đầu ra cuối x_L. Cho một tập chỉ số $S \subseteq \{1, \ldots, L\}$ các lớp bị bỏ, mạng đã cắt chạy

$$x'_l = \begin{cases} x'_{l-1}, & l \in S, \\ x'_{l-1} + f_l(x'_{l-1}), & l \notin S, \end{cases} \qquad x'_0 = x_0.$$

Chú ý f_l ở lớp còn lại vẫn là hàm cũ nhưng **được gọi với** x'_{l-1} **khác** x_{l-1}. Đây là chỗ mọi sai lệch lan truyền. Hai đại lượng đo mức hỏng ta sẽ dùng:

- sai lệch tương đối của đầu ra: $\dfrac{\|x'_L - x_L\|}{\|x_L\|}$,
- méo hướng: $1 - \cos(x'_L, x_L)$.

Với một lớp riêng lẻ, hai trực giác bổ sung:

$$\mathrm{BI}_l = 1 - \cos(x_{l-1}, x_l), \qquad r_l = \frac{\|f_l(x_{l-1})\|}{\|x_{l-1}\|}.$$

BI_l đo thay đổi **hướng**, r_l đo đóng góp **tương đối về độ lớn**. Khi chỉ bỏ lớp cuối L thì sai lệch tuyệt đối của đầu ra đúng bằng $\|f_L(x_{L-1})\|$ (không còn lớp nào truyền tiếp). Với lớp ở giữa, đóng góp bị mất còn đi qua các lớp sau và có thể bị khuếch đại hoặc dập tắt.

6. Ví dụ tay: mạng residual 4 lớp, vector 3 chiều

Chọn một "mạng" nhỏ để tính được bằng tay. Bốn lớp, mỗi lớp là một phép nhân ma trận (tuyến tính, bỏ qua chuẩn hoá cho gọn):

- $f_1(x) = (x_2,\ -x_1,\ x_3)$ (xoay một mặt phẳng và giữ nguyên x_3),
- $f_2(x) = 0{,}1\,x$,
- $f_3(x) = (0,\ 0{,}2\,x_3,\ 0)$,
- $f_4(x) = (0{,}5\,x_1,\ 0{,}5\,x_2,\ -0{,}5\,x_3)$.

Đầu vào là $x_0 = (1, 2, 2)$ với $\|x_0\| = 3$. Mọi số dưới đây là (tự tính).

Chạy đầy đủ.

- Lớp 1: $f_1(x_0) = (2, -1, 2)$, nên $x_1 = (3, 1, 4)$.
- Lớp 2: $f_2(x_1) = (0{,}3,\ 0{,}1,\ 0{,}4)$, nên $x_2 = (3{,}3,\ 1{,}1,\ 4{,}4)$.
- Lớp 3: $f_3(x_2) = (0,\ 0{,}88,\ 0)$, nên $x_3 = (3{,}3,\ 1{,}98,\ 4{,}4)$.
- Lớp 4: $f_4(x_3) = (1{,}65,\ 0{,}99,\ -2{,}2)$, nên $x_4 = (4{,}95,\ 2{,}97,\ 2{,}2)$ và $\|x_4\| \approx 6{,}178$.

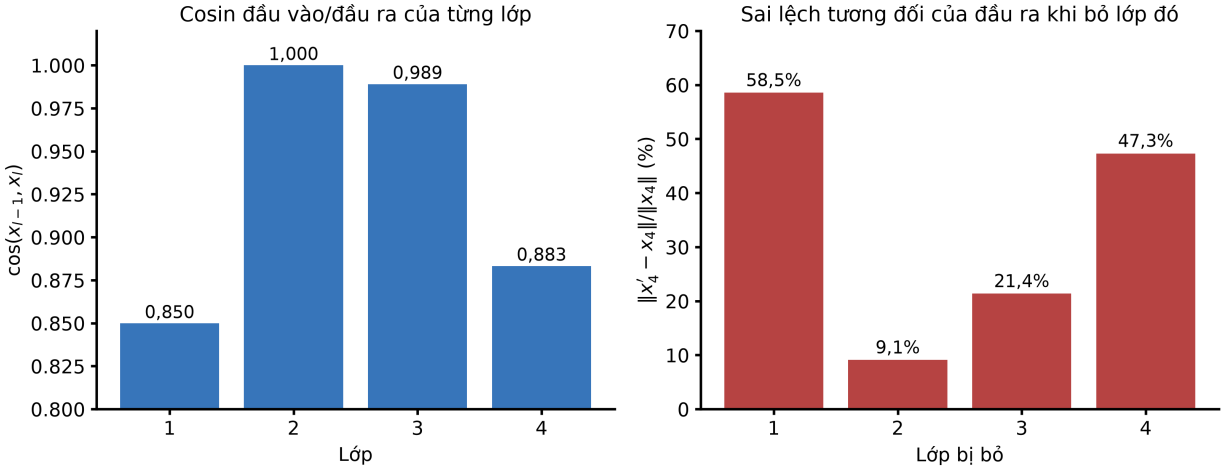
Cosin của từng lớp.

- Lớp 1: $x_0 \cdot x_1 = 3 + 2 + 8 = 13$, $\|x_1\| = \sqrt{26} \approx 5{,}099$, nên $\cos = 13/(3 \cdot 5{,}099) \approx 0{,}850$ và $\mathrm{BI}_1 \approx 0{,}150$.
- Lớp 2: $x_2 = 1{,}1\,x_1$ cùng hướng với x_1, nên $\cos = 1$ và $\mathrm{BI}_2 = 0$.
- Lớp 3: $x_2 \cdot x_3 = 10{,}89 + 2{,}178 + 19{,}36 = 32{,}428$, $\|x_2\| \approx 5{,}609$, $\|x_3\| \approx 5{,}846$, nên $\cos \approx 0{,}989$ và $\mathrm{BI}_3 \approx 0{,}011$.
- Lớp 4: $x_3 \cdot x_4 = 16{,}335 + 5{,}881 + 9{,}68 = 31{,}896$, $\|x_4\| \approx 6{,}178$, nên $\cos \approx 0{,}883$ và $\mathrm{BI}_4 \approx 0{,}117$.

Bỏ từng lớp rồi chạy lại.

- Bỏ lớp 1: $x'_1 = x_0 = (1, 2, 2)$; lớp 2 cho $(1{,}1,\ 2{,}2,\ 2{,}2)$; lớp 3 cho $(1{,}1,\ 2{,}64,\ 2{,}2)$; lớp 4 cho $x'_4 = (1{,}65,\ 3{,}96,\ 1{,}1)$. Sai lệch $\|x'_4 - x_4\| \approx 3{,}617$, tức $58{,}5\%$ so với $\|x_4\|$.
- Bỏ lớp 2: $x'_2 = x_1 = (3, 1, 4)$; lớp 3 cho $(3, 1{,}8, 4)$; lớp 4 cho $x'_4 = (4{,}5,\ 2{,}7,\ 2)$. Sai lệch $\approx 0{,}562$, tức $9{,}1\%$.

- Bỏ lớp 3: $x'_3 = x_2$; lớp 4 cho $x'_4 = (4{,}95,\ 1{,}65,\ 2{,}2)$. Sai lệch $= 1{,}32$ (phần $0{,}88$ bị mất ở trục 2, lớp 4 nhân nó thêm $1{,}5$ lần), tức $21{,}4\%$.
- Bỏ lớp 4: $x'_4 = x_3 = (3{,}3,\ 1{,}98,\ 4{,}4)$. Sai lệch $= \|f_4(x_3)\| \approx 2{,}923$, tức $47{,}3\%$.



Hình 23: Ví dụ tay: cosin từng lớp (trái) và sai lệch tương đối của đầu ra khi bỏ từng lớp (phải) (minh hoạ, tự tính)

Cách đọc hình: trục trái bắt đầu từ $0{,}8$ chứ không phải 0, để thấy khác biệt giữa các cột; cột càng thấp thì lớp càng đổi hướng mạnh. Bên phải là sai lệch khi bỏ riêng lớp đó. Lớp 1 và 4 (cosin thấp) cho sai lệch lớn, lớp 3 vừa phải, lớp 2 nhỏ nhất.

Bài học từ ví dụ.

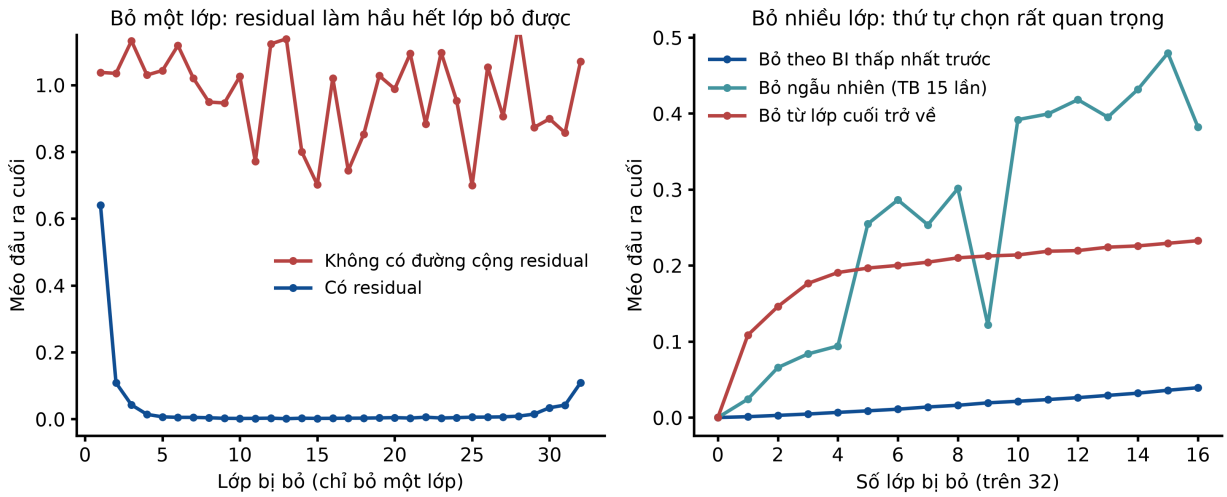
1. Thứ hạng theo cosin (lớp 2 dư nhất, rồi lớp 3, rồi lớp 4, rồi lớp 1) khớp thứ hạng theo sai lệch thật (lớp 2 nhỏ nhất, rồi lớp 3, rồi lớp 4, rồi lớp 1). Đó là lý do BI hoạt động như proxy.
2. Nhưng **lớp 2 có** $\mathrm{BI}_2 = 0$ **mà bỏ vẫn gây sai lệch** $9{,}1\%$. Vì f_2 chỉ phóng to x_1 thêm 10% (đúng hướng cũ), cosin không thấy gì; độ lớn thì đổi. Đo đóng góp theo độ lớn cho $r_2 = \|f_2\|/\|x_1\| = 0{,}5099/5{,}099 = 0{,}1$, tức 10%, phản ánh đúng hơn. Các tỉ lệ r_l khác: $r_1 = 1$, $r_3 \approx 0{,}157$, $r_4 \approx 0{,}5$ (tự tính).
3. **Hiệu ứng không cộng tính.** Bỏ lớp 2 và lớp 3 cùng lúc cho $x'_4 = (4{,}5,\ 1{,}5,\ 2)$ với sai lệch $\approx 1{,}550$ ($25{,}1\%$), trong khi cộng hai sai lệch riêng được $0{,}562 + 1{,}32 = 1{,}882$. Chọn cả tập lớp để bỏ cần nhìn tương tác, không chỉ chấm điểm từng lớp rồi cộng. Đây chính là điều bài 34 và 37 xử lý.

7. Biến thể và chi tiết hay bị bỏ qua

7.1 So mạng có và không có dòng dư

Cách đọc hình: cả hai panel đo "méo đầu ra cuối" $= 1 - \cos(x'_L, x_L)$, trung bình trên 256 token ngẫu nhiên, **không phải perplexity**. Bên trái, mỗi điểm là một lần bỏ một lớp duy nhất. Đường đỏ là mạng cùng kích thước nhưng không có đường cộng (mỗi lớp thay biểu diễn, $x_l = \text{chuẩn hoá}(f_l(x_{l-1}))$): bỏ bất kỳ lớp nào cũng làm méo gần 1, tức đầu ra gần như không còn liên quan. Đường xanh là mạng residual: chỉ lớp 1, lớp 2 và vài lớp cuối cho méo đáng kể, phần giữa gần 0 (tự tính: lớp 1 cho $0{,}64$, lớp 11 cho $0{,}002$, lớp 32 cho $0{,}109$; mạng không residual trung bình $0{,}97$).

Bên phải, bỏ k lớp một lúc. Chọn lớp BI thấp nhất trước (đường xanh) giữ méo ở mức $0{,}016$ khi $k = 8$ và $0{,}039$ khi $k = 16$. Bỏ ngẫu nhiên cho khoảng $0{,}30$ và $0{,}38$ (trung bình 15 lần, đường gập ghềnh vì thỉnh thoảng bốc phải lớp đầu), còn bỏ từ cuối trở về cho $0{,}21$ và $0{,}23$ (tự tính). Hình ngầm nói: có residual không đủ, **cần chọn đúng lớp**.



Hình 24: Bỏ một lớp trong mạng có residual và không có residual (trái); bỏ nhiều lớp theo ba thứ tự khác nhau (phải) (minh hoạ, tự tính)

7.2 Khối liền hay lớp rời

ShortGPT (bài 31) chấm điểm từng lớp rồi bỏ các lớp thấp nhất, có thể rời rạc. Bài 32 chọn một **khối liền** n lớp bằng khoảng cách góc giữa đầu vào lớp ℓ và đầu vào lớp $\ell + n$, vì khoảng cách đó nhằm đúng thứ cần giữ: biểu diễn trước và sau cả khối. Bài 34 (SLEB) nhấn mạnh tầm quan trọng của một lớp thay đổi khi các lớp khác bị bỏ, nên chọn lặp từng lớp trên mô hình đã cắt, và ghi nhận (theo ghi chú) một thử nghiệm dùng cosin cho kết quả tăng perplexity mạnh. Ví dụ tay phần 6 đã thấy lý do: không cộng tính.

7.3 Chữa sau khi cắt

Sau khi bỏ lớp, lớp phía sau nhận đầu vào hơi khác thứ đã quen. Có ba lối chữa: tinh chỉnh nhẹ (bài 32, QLoRA), gộp tham số lớp bị bỏ vào lớp giữ lại (bài 35), hoặc thay khối bị bỏ bằng một ma trận T ước lượng bằng bình phương tối thiểu từ dữ liệu calibration (bài 36). Bài 37 chỉ ra rằng khi biết mình sẽ chữa kiểu nào thì tập lớp tối ưu để bỏ cũng đổi theo.

7.4 Cosin không thấy độ lớn, dòng dư có thể lớn dần

Đã thấy ở lớp 2 trong ví dụ tay. Trong thực tế độ lớn dòng dư thường tăng theo độ sâu (hình mô phỏng: RMS từ $0{,}99$ lên $3{,}57$), nên một đóng góp tuyệt đối cố định là ngày càng nhỏ so với dòng chính. Chuẩn hoá đặt trước mỗi lớp khiến lớp đó đọc dòng dư đã chuẩn hoá chứ không phải độ lớn thô, nên chỉ riêng cosin giữa x_{l-1} và x_l là thước đo chưa đủ.

7.5 Bỏ lớp, bỏ attention hay bỏ MLP

Một lớp gồm hai nhánh cộng vào dòng dư (attention và MLP). Có thể bỏ riêng từng nhánh để cắt mịn hơn. Chương này chỉ xét đơn vị "cả lớp" vì đó là đơn vị của các bài 31 đến 37.

8. Nối sang pruning và LLM

Độ sâu và độ trễ. Theo ghi chú pruning 33 (LLaMA-7B, cắt 20%, batch 1 trên H100, sinh 128 token): mô hình gốc có độ trễ $2{,}4$ giây, các phương pháp cắt chiều rộng được so sánh có $3{,}0$ đến $3{,}2$ giây (chậm hơn dù ít tham số hơn), còn các phương pháp bỏ lớp có $1{,}9$ giây. Nguyên nhân giải thích trong ghi chú: ở batch nhỏ, sinh token bị giới hạn bởi việc đọc trọng số từ bộ nhớ GPU; bỏ cả lớp giảm số lần truy cập bộ nhớ, còn thu hẹp ma trận có thể tạo kích thước không thân thiện GPU (theo ghi chú).

Chiều rộng so với chiều sâu. Cắt chiều rộng (LLM-Pruner, FLAP, SliceGPT ở các bài 26, 28, 29) chia nhỏ mỗi ma trận; cắt chiều sâu bỏ cả lớp. Lợi thế của sâu: tăng tốc thật và đơn giản. Nhược điểm: đơn vị cắt thô, mất mát rơi vào một lớp tại một thời điểm, và ở mức cắt nặng chất lượng tụt nhanh nếu không huấn luyện lại (ghi chú 33 ghi lại điều này khi so LoRA với tiếp tục tiền huấn luyện).

Hai mặt của chất lượng. Theo ghi chú pruning 32, các benchmark hỏi đáp kiểu trắc nghiệm có thể bền khi bỏ nhiều lớp, trong khi loss dự đoán token kế tiếp xấu đi đều theo tỉ lệ cắt. Vì vậy chọn lớp nên nhìn nhiều chỉ số (xem chương nền 07).

Sửa chữa đổi lớp cần bỏ. Theo ghi chú pruning 37, tập lớp tối ưu cho cắt thuần khác tập tối ưu khi đã có sửa chữa tuyến tính, nên chọn lớp theo cosin riêng lẻ không phải lúc nào cũng gần tối ưu. Số liệu chi tiết xem trong bài đó.

Lớp bị bỏ có thể là lớp "giống nhau". Bài 35 (LaCo) và bài 36 (ReplaceMe) tận dụng việc lớp liền kề có biểu diễn rất giống nhau để gộp hoặc thay bằng ánh xạ tuyến tính, thay vì xoá thẳng.

9. Sai lầm hay gặp

- **Nghĩ bỏ lớp nào cũng được vì có dòng dư.** Hình 7.1 cho thấy lớp đầu và vài lớp cuối gây méo lớn; dòng dư chỉ làm cho việc bỏ lớp **có thể** nhẹ.
- **Nhầm cosin cao với "lớp vô dụng".** Lớp 2 trong ví dụ tay đổi độ lớn 10% mà cosin bằng 1. Cosin bỏ sót thay đổi độ lớn, và cả những thay đổi nhỏ nhưng quan trọng về nội dung.
- **Cộng sai lệch từng lớp để dự đoán sai lệch cả tập.** Hiệu ứng không cộng tính: $1{,}55$ chứ không phải $1{,}88$ trong ví dụ.
- **Chọn mô hình giảm tham số rồi tưởng nhanh hơn tương ứng.** Phần chiều rộng của mô hình giữ nguyên khi bỏ lớp; ngược lại, cắt chiều rộng chưa chắc nhanh hơn (xem ghi chú 33 ở batch 1).
- **Đánh giá chỉ bằng một benchmark.** Một số tác vụ bền trong khi loss ngôn ngữ đã tệ đi; ngược lại perplexity tốt có thể che việc suy luận hỏng.
- **Dùng số trong hình mô phỏng như số của LLM thật.** Các hình ở chương này dùng mạng ngẫu nhiên và hình chữ U do thiết kế; chỉ dùng để học cơ chế.

10. Tóm tắt công thức

Khái niệm	Công thức	Ý nghĩa
Lớp residual	$x_l = x_{l-1} + f_l(x_{l-1})$	Mỗi lớp chỉ cộng thêm một đóng góp
Dạng tổng	$x_L = x_0 + \sum_l f_l(x_{l-1})$	Đầu ra là embedding cộng các đóng góp
Block Influence	$\mathrm{BI}_l = 1 - \mathbb{E}[\cos(x_{l-1}, x_l)]$	Thay đổi hướng do lớp đó gây ra
Đóng góp tương đối	$r_l = \|f_l(x_{l-1})\| / \|x_{l-1}\|$	Thay đổi độ lớn do lớp đó gây ra
Bỏ một tập lớp	$x'_l = x'_{l-1}$ nếu $l \in S$	Lớp còn lại nhận đầu vào khác trước
Sai lệch đầu ra	$\|x'_L - x_L\| / \|x_L\|$	Mức hỏng đo trực tiếp

11. Bài tập tự tính

Mọi đáp án đã kiểm bằng numpy.

Bài 1. Cosin của một lớp

Một lớp nhận $x_{in} = (3, 4)$ và cộng thêm $f = (4, -3)$. Tính x_{out}, $\cos(x_{in}, x_{out})$, BI và $r = \|f\| / \|x_{in}\|$.

Đáp án (tự tính): $x_{out} = (7, 1)$. $x_{in} \cdot x_{out} = 21 + 4 = 25$, $\|x_{in}\| = 5$, $\|x_{out}\| = \sqrt{50} \approx 7{,}071$, nên $\cos = 25/(5 \cdot 7{,}071) \approx 0{,}707$ và $\mathrm{BI} \approx 0{,}293$. $\|f\| = 5$ nên $r = 1$.

Bài 2. Cosin không thấy độ lớn

Một lớp biến x_{in} thành $x_{out} = 1{,}5\,x_{in}$ với $x_{in} \neq 0$. Tính BI và r. Lớp này có "vô hại" không?

Đáp án (tự tính): cos $= 1$ nên BI $= 0$. Còn $f = x_{out} - x_{in} = 0{,}5\,x_{in}$ nên $r = 0{,}5$. Đầu ra đổi 50% độ lớn, nên bỏ lớp này vẫn làm thay đổi tín hiệu dù BI bằng 0.

Bài 3. Bỏ một lớp trong mạng ba lớp

Dùng $x_0 = (1, 2, 2)$ và các lớp của phần 6 nhưng chỉ giữ lớp 1, 2, 3 (bỏ lớp 4, dùng x_3 làm đầu ra cuối). Tính sai lệch tương đối của đầu ra khi bỏ lớp 2.

Đáp án (tự tính): Đầy đủ: $x_3 = (3{,}3,\ 1{,}98,\ 4{,}4)$ với $\|x_3\| \approx 5{,}846$. Bỏ lớp 2: $x_2' = x_1 = (3, 1, 4)$, lớp 3 cộng $(0, 0{,}8, 0)$ cho $(3,\ 1{,}8,\ 4)$. Hiệu với x_3 là $(-0{,}3,\ -0{,}18,\ -0{,}4)$ với chuẩn $\sqrt{0{,}09 + 0{,}0324 + 0{,}16} = \sqrt{0{,}2824} \approx 0{,}531$. Tỉ lệ $0{,}531/5{,}846 \approx 0{,}091$, tức $9{,}1\%$ (tình cờ trùng con số của mạng bốn lớp, vì cả hai mạng đều tuyến tính).

Bài 4. Bao nhiêu tham số khi bỏ 8 lớp

Xấp xỉ mỗi lớp có $12d^2$ tham số với $d = 4096$. Mô hình có 32 lớp và embedding cộng đầu ra cuối có $2 \cdot 32000 \cdot 4096$ tham số. Bỏ 8 lớp thì mất bao nhiêu phần trăm tổng tham số?

Đáp án (tự tính): Mỗi lớp $12 \cdot 4096^2 = 201\,326\,592$. Tổng $32 \cdot 201\,326\,592 + 262\,144\,000 = 6\,704\,594\,944$ (xấp xỉ $6{,}7$ tỉ). Bỏ 8 lớp mất $1\,610\,612\,736$, tức $24{,}0\%$, ít hơn 25% vì phần embedding không bị cắt. Đây là xấp xỉ khái niệm cho cấu hình kiểu Llama-7B, không phải đếm chính xác.

Bài 5. Khi mọi cosin bằng 1

Ba lớp có $f_l(x) = c_l\,x$ với $c = (0{,}5,\ 0{,}2,\ -0{,}1)$ và đầu vào $x_0 = (1, 2, 2)$. Tính BI của từng lớp, đầu ra đầy đủ, và sai lệch tương đối khi bỏ từng lớp. Nên bỏ lớp nào?

Đáp án (tự tính): Mọi lớp chỉ nhân x với $1 + c_l$, nên cosin luôn bằng 1 (chú ý $1 + c_3 = 0{,}9 > 0$) và BI $= 0$ cho cả ba, không phân biệt được. Đầu ra đầy đủ là $1{,}5 \cdot 1{,}2 \cdot 0{,}9\,x_0 = 1{,}62\,x_0$. Bỏ lớp 1 cho hệ số $1{,}08$, sai lệch $0{,}54/1{,}62 = 1/3 \approx 0{,}333$. Bỏ lớp 2 cho $1{,}35$, sai lệch $1/6 \approx 0{,}167$. Bỏ lớp 3 cho $1{,}80$, sai lệch $1/9 \approx 0{,}111$. Nên bỏ lớp 3, nhưng chỉ nhờ đo độ lớn, còn BI không giúp được gì.

Tài liệu tham khảo nền: Men và cộng sự (ShortGPT), Gromov và cộng sự (deeper layers), Kim và cộng sự (Shortened LLaMA), Song và cộng sự (SLEB), Yang và cộng sự (LaCo), Shopkhoev và cộng sự (ReplaceMe); các ghi chú pruning 31, 32, 33, 34, 35, 36 và 37 trong sách.

Nền tảng 07

Đo đúng một kết quả pruning: perplexity, benchmark, độ trễ và tỉ lệ nén nói gì và không nói gì

Loại: chương nền tảng (không phải ghi chú paper)

Đọc trước: logarit và hàm mũ ở math nền tảng 12; entropy và cross-entropy ở math nền tảng 11; trung bình và xác suất ở math nền tảng 08; phần "tiêu chí" trong pruning 00

Dùng ở: pruning 12 (so sánh thiếu chuẩn), pruning 21 (các thước đo cho LLM), pruning 23 và pruning 24 (perplexity và zero-shot), pruning 37 (đo trên tập tách rời), pruning 43 (so cùng ngân sách token); cả pruning 18 và pruning 33 khi đọc phần tăng tốc

> Chương này giải thích từ đầu, không phải bản dịch paper. Các phép tính ghi "tự tính" đã kiểm bằng numpy. Các hình dùng đường cong dựng từ công thức giả định hoặc điểm sinh ngẫu nhiên có seed cố định, nên chỉ minh hoạ hình dạng chung, không phải số đo thật.

1. Vì sao cần học cách đo

Đo một kết quả pruning nghĩa là trả lời: mô hình sau cắt tốt đến đâu, nhẹ đến đâu, nhanh đến đâu, và so với cái gì. Mỗi chữ trong câu đó là một cái bẫy. "Tốt" có thể là perplexity thấp hoặc độ chính xác cao trên một bộ câu hỏi, và hai thứ này không luôn đi cùng nhau. "Nhẹ" có thể là ít tham số, ít bộ nhớ hoặc ít phép tính. "Nhanh" có thể là độ trễ một yêu cầu hoặc số token mỗi giây khi phục vụ nhiều người. "So với cái gì" có thể là mô hình gốc, phương pháp khác cùng tỉ lệ cắt, hoặc mô hình nhỏ huấn luyện từ đầu.

Một bài báo viết "cắt 50%, perplexity chỉ tăng 0,1" có thể hoàn toàn đúng nhưng vẫn không cho bạn biết mô hình có chạy nhanh hơn hay không, có còn làm được bài toán suy luận hay không, hay có thắng đối thủ khi cho cùng ngân sách. Chương này dạy từng thước đo, cách tính tay, những giả định ẩn sau chúng, và cách đọc một bảng kết quả để không bị đánh lừa (hoặc không tự đánh lừa mình).

Trong LLM, việc đo đúng đặc biệt quan trọng vì không ai huấn luyện lại đủ để che sai sót: phần lớn phương pháp cắt sau huấn luyện chỉ có một tập calibration nhỏ và một bảng vài số, nên mỗi con số được gánh rất nhiều nghĩa.

2. Nhắc nền: logarit, xác suất token và trung bình

2.1 Mô hình ngôn ngữ cho xác suất của token

Một mô hình ngôn ngữ đọc chuỗi token $t_1, t_2, \ldots, t_N$ và, tại mỗi vị trí i, cho một xác suất p_i mà nó gán cho token **thật sự xuất hiện** tiếp theo (điều kiện trên các token trước). Xác suất cao nghĩa là mô hình "không bất ngờ"; xác suất thấp nghĩa là mô hình bất ngờ nhiều.

Đại lượng đo mức bất ngờ là $-\ln p_i$, đơn vị nat (nếu dùng $\log_2$ thì đơn vị là bit). Ví dụ (tự tính): $p = 1$ cho $-\ln p = 0$ (không bất ngờ), $p = 0{,}5$ cho $0{,}693$, $p = 0{,}125$ cho $2{,}079$, và $p \to 0$ thì bất ngờ tiến ra vô hạn. Đây là lý do mô hình không bao giờ nên gán xác suất 0 cho token thật.

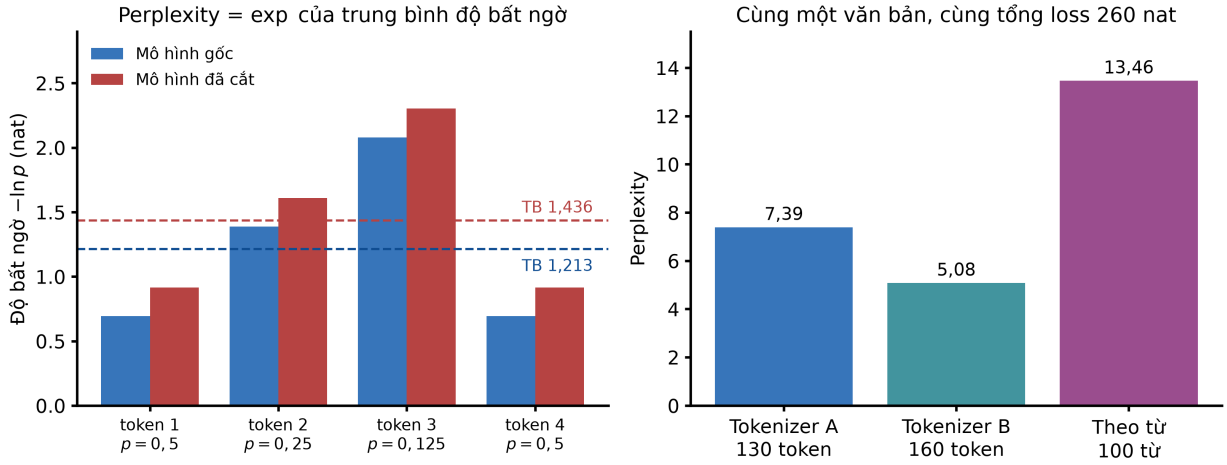
2.2 Loss là bất ngờ trung bình

Loss cross-entropy trên N token là trung bình độ bất ngờ:

$$\mathcal{L} = \frac{1}{N} \sum_{i=1}^{N} \left(-\ln p_i \right).$$

Ví dụ nhỏ (tự tính): $p = (0{,}5,\ 0{,}25,\ 0{,}125,\ 0{,}5)$ cho độ bất ngờ $(0{,}693,\ 1{,}386,\ 2{,}079,\ 0{,}693)$, tổng $4{,}852$, trung bình $\mathcal{L} = 1{,}213$ nat. Loss là thước đo cơ bản mà quá trình huấn luyện cực tiểu hoá; perplexity chỉ là loss được đổi sang một thang dễ nghĩa hơn.

3. Trực giác bằng hình: perplexity là "số lựa chọn tương đương"



Hình 25: Ví dụ tay: độ bất ngờ từng token của mô hình gốc và mô hình đã cắt (trái); cùng một văn bản đo theo ba đơn vị cho ba perplexity khác nhau (phải) (minh hoạ, tự tính)

Cách đọc hình: panel trái có bốn token, mỗi token hai cột, xanh là mô hình gốc và đỏ là mô hình đã cắt; chiều cao cột là $-\ln p$ (độ bất ngờ). Hai đường nét đứt là trung bình, tức loss $1{,}213$ (xanh) và $1{,}436$ (đỏ). Ở ví dụ này mọi xác suất của mô hình đã cắt bằng $0{,}8$ lần xác suất gốc, nên mỗi cột đỏ cao hơn cột xanh đúng $-\ln 0{,}8 = 0{,}223$ nat. Panel phải được giải thích ở phần 5 (bước 4).

Trực giác: nếu mô hình gán xác suất đều $1/V$ cho mọi token trong từ vựng V thì loss là $\ln V$ và perplexity đúng bằng V. Nói cách khác, **perplexity $= k$ có nghĩa mô hình "bối rối" ngang với việc phải chọn đều trong k lựa chọn**. Perplexity 1 là mô hình chắc chắn tuyệt đối, perplexity bằng cỡ từ vựng là mô hình đoán mò.

4. Định nghĩa chính thức

Perplexity.

$$\mathrm{PPL} = \exp(\mathcal{L}) = \exp\Big(-\frac{1}{N}\sum_{i=1}^{N}\ln p_i\Big) = \Big(\prod_{i=1}^{N} p_i\Big)^{-1/N}.$$

Dạng cuối cho thấy PPL là nghịch đảo của **trung bình nhân** các xác suất. Nếu dùng $\log_2$ ở cả hai chỗ thì $2^{\text{loss bit}}$ cho cùng kết quả (tự tính, ví dụ trên: loss $1{,}75$ bit và $2^{1{,}75} = 3{,}364$, bằng $e^{1{,}213}$).

Hai tính chất cần nhớ.

1. Nhân mọi xác suất với c thì PPL nhân $1/c$. Ví dụ: ở phần 2, nhân $0{,}8$ làm PPL từ $3{,}364$ lên $4{,}204 = 3{,}364/0{,}8$ (tự tính).
2. PPL phụ thuộc vào **đơn vị token**. Cùng một văn bản, hai bộ tách token khác nhau cho hai PPL khác nhau (xem phần 6), nên chỉ so PPL khi cùng tokenizer, cùng dữ liệu, cùng độ dài ngữ cảnh.

Tỉ lệ nén và tăng tốc. Với số tham số gốc P và sau cắt P':

$$\text{tỉ lệ nén} = \frac{P}{P'}, \qquad \text{tăng tốc lý thuyết} = \frac{\text{phép tính gốc}}{\text{phép tính sau cắt}}, \qquad \text{tăng tốc thật} = \frac{\text{thời gian gốc}}{\text{thời gian sau cắt}}.$$

Ghi chú pruning 12 yêu cầu báo cả tỉ lệ nén lẫn tăng tốc lý thuyết (nhân-cộng); tăng tốc thật là đại lượng thứ ba, đo trên thiết bị cụ thể, và hai đại lượng kia chỉ là proxy cho nó.

5. Ví dụ tay: tính perplexity từ vài xác suất token

Mọi số dưới đây là (tự tính), đã kiểm bằng numpy.

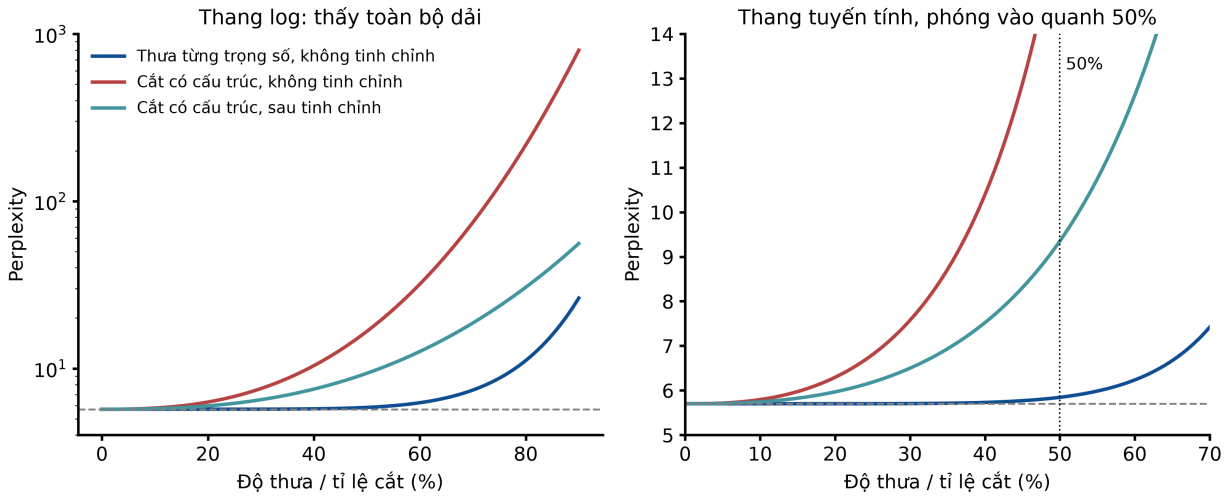
Bước 1. Mô hình gốc. Bốn token có $p = (0{,}5,\ 0{,}25,\ 0{,}125,\ 0{,}5)$. Tích $= 0{,}5 \cdot 0{,}25 \cdot 0{,}125 \cdot 0{,}5 = 2^{-7} = 0{,}0078125$. Lấy mũ $-1/4$: PPL $= (2^{-7})^{-1/4} = 2^{7/4} = 2^{1{,}75} \approx 3{,}364$. Kiểm lại bằng loss: $\mathcal{L} = 1{,}213$, $e^{1{,}213} \approx 3{,}364$.

Bước 2. Mô hình đã cắt. Giả sử sau cắt xác suất của mỗi token chỉ còn $0{,}8$ lần: $q = (0{,}4,\ 0{,}2,\ 0{,}1,\ 0{,}4)$. Tích $= 0{,}0032$. PPL$' = 0{,}0032^{-1/4} \approx 4{,}204$. Đúng bằng $3{,}364/0{,}8$.

Bước 3. Đọc kết quả. PPL tăng $4{,}204 - 3{,}364 = 0{,}84$, tức 25%. Loss tăng $0{,}223$ nat. Hai cách nói nghe rất khác nhau về cùng một thiệt hại. Chuyện này lặp lại ở mọi bài: **PPL tăng "chỉ 0,1" ở PPL 5 và ở PPL 50 không cùng ý nghĩa**, vì khoảng cách tương đối theo thang loss mới có tính so sánh.

Bước 4. Vì sao đơn vị token quan trọng. Giả sử tổng loss của một đoạn văn là 260 nat. Tokenizer A chia thành 130 token: PPL $= e^{260/130} = e^2 = 7{,}389$. Tokenizer B chia thành 160 token: $e^{260/160} = e^{1{,}625} = 5{,}078$. Nếu đếm theo từ (giả sử 100 từ): $e^{2{,}6} = 13{,}46$. Cùng văn bản, cùng tổng thông tin, ba PPL khác nhau. Đây là panel phải của hình ở phần 3. Hệ quả: không so PPL giữa hai họ mô hình dùng tokenizer khác nhau, và khi trích số từ bài báo cần xem họ đo trên tập nào, ngữ cảnh bao nhiêu.

6. Perplexity theo độ thưa: đọc đường cong



Hình 26: Perplexity theo độ thưa, dựng từ công thức giả định cho ba kiểu cắt; hai thang trục để thấy hai điều khác nhau (minh hoạ, tự tính)

Cách đọc hình: đường nét đứt ngang là perplexity mô hình gốc $(5{,}7)$. Ba đường là ba kiểu: thưa từng trọng số không tinh chỉnh (xanh đậm), cắt có cấu trúc không tinh chỉnh (đỏ), cắt có cấu trúc sau tinh chỉnh (xanh lục). **Đây là đường dựng từ công thức** $\text{PPL}(s) = \text{PPL}_0 \exp(c\, s^n)$ **với tham số chọn tay, không phải đo từ mô hình nào.** Hình dạng chung đúng với điều hay gặp: đường thưa từng trọng số phẳng khá lâu rồi dốc lên, còn cắt có cấu trúc bắt đầu tăng sớm hơn; tinh chỉnh làm đường thấp xuống.

Hai panel cùng dữ liệu. Thang log (trái) cho thấy đến 90%, đường đỏ lên hàng trăm; thang tuyến tính phóng quanh 50% (phải) cho thấy ở 50% các giá trị là (tự tính, theo công thức giả định) khoảng 5,84, 16,65 và 9,35. Khi một bài báo chỉ vẽ thang tuyến tính vùng thấp, hai đường tưởng như sát nhau; khi vẽ thang log đến 90%, khoảng cách bung ra. Cả hai đều "đúng"; chúng trả lời hai câu hỏi khác nhau.

Bài học đọc đường cong. Chỉ số quan trọng không phải một điểm mà là điểm gãy: độ thưa mà đường bắt đầu dốc. Ghi chú pruning 21 nhắc rằng ở độ thưa rất cao mô hình hỏng hẳn (theo bảng trong khảo sát, một phương pháp có sai khác perplexity cỡ $2{,}08 \times 10^4$ ở 90%), nên đừng ngoại suy từ vùng phẳng.

7. Biến thể và chi tiết hay bị bỏ qua

7.1 Perplexity không nói hết

Perplexity đo khả năng đoán token kế tiếp trung bình trên một kho văn bản. Nó không trực tiếp đo: suy luận nhiều bước, làm theo chỉ dẫn, độ an toàn, khả năng sinh dài ổn định, hay kiến thức hiếm. Một mô hình có thể giữ PPL gần nguyên mà làm các tác vụ suy luận kém đi, và ngược lại. Theo ghi chú pruning 32, các benchmark hỏi đáp trắc nghiệm có thể bền khi bỏ nhiều lớp trong khi loss ngôn ngữ xấu đi đều theo tỉ lệ cắt; điều đó cho thấy hai thước đo không đồng bộ. Ghi chú pruning 37 đo bằng perplexity trên các chuỗi WikiText-2 tách riêng với tập dùng để khớp ma trận sửa chữa, và có biến thể chọn lớp theo cross-entropy của lựa chọn đúng trên các mẫu tác vụ, kể cả MMLU, tức không chỉ dựa vào perplexity.

7.2 Benchmark zero-shot và cách chấm

Zero-shot nghĩa là hỏi mô hình mà không cho ví dụ mẫu. Với câu hỏi trắc nghiệm, cách phổ biến là so **log-xác suất** mà mô hình gán cho từng phương án trả lời và chọn phương án có điểm cao nhất; độ chính xác là tỉ lệ câu chọn đúng. Ghi chú pruning 24 đo bằng perplexity WikiText và trung bình 7 tác vụ zero-shot (EleutherAI LM Harness), calibration 128 chuỗi từ C4.

Hai điểm tinh tế: (i) độ chính xác chỉ phản ánh **xếp hạng** các phương án, nên PPL tệ đi mà xếp hạng vẫn đúng là hoàn toàn có thể; (ii) có chuẩn hoá theo độ dài hay không có thể đổi phương án được chọn (xem bài tập 5). Vì vậy cùng một mô hình có thể có điểm khác nhau giữa hai bộ công cụ đánh giá; khi so sánh phải dùng cùng công cụ và cùng cách chuẩn hoá.

7.3 Độ trễ, thông lượng, bộ nhớ

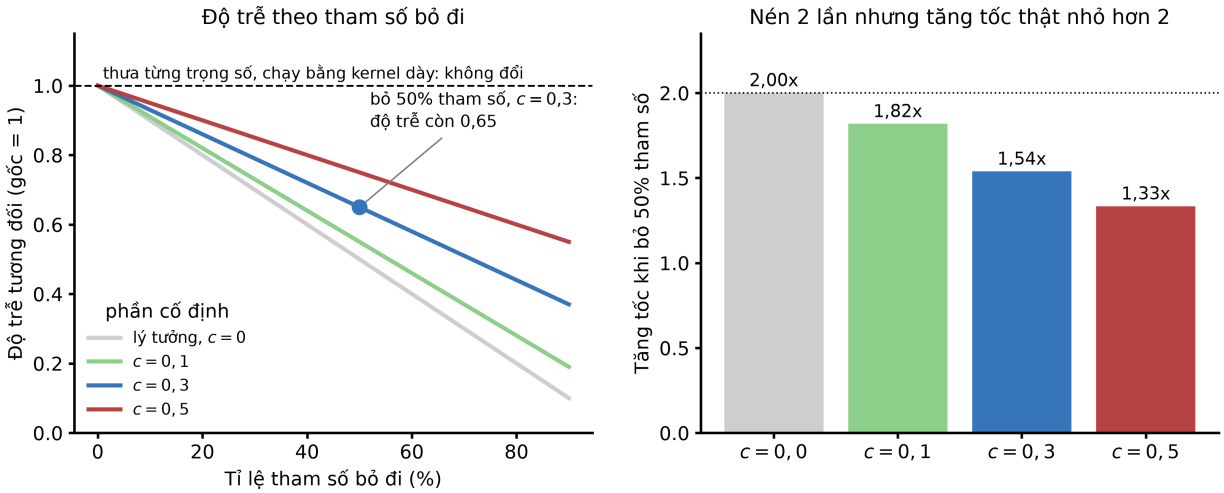
Ba đại lượng "nhanh và nhẹ" khác nhau:

- **Độ trễ** (latency): thời gian trả lời một yêu cầu; quan trọng khi phục vụ một người dùng, batch nhỏ.
- **Thông lượng** (throughput): số token mỗi giây trên toàn hệ thống; quan trọng khi gom nhiều yêu cầu thành batch lớn.
- **Bộ nhớ**: trọng số, bộ nhớ đệm KV, kích hoạt; quyết định mô hình có chạy được trên thiết bị hay không.

Một phương pháp có thể thắng ở đại lượng này và thua ở đại lượng khác. Ví dụ: ở batch nhỏ, sinh từng token bị giới hạn bởi tốc độ đọc trọng số từ bộ nhớ GPU (theo ghi chú pruning 33), nên giảm byte đọc là lợi; ở batch lớn, giới hạn là sức tính toán, nên giảm phép nhân mới là lợi. Theo ghi chú pruning 21, kích thước và FLOPs không bằng độ trễ, và hệ số sử dụng phần cứng (MFU) thấp ở batch 1 là lý do cắt kênh nhỏ ít giúp.

7.4 Tỉ lệ nén tham số so với tăng tốc thật

Cách đọc hình: mô hình khái niệm là độ trễ tương đối $= c + (1-c)(1-p)$, trong đó p là tỉ lệ tham số bỏ đi và c là phần thời gian không giảm theo tham số (gọi kernel, các thao tác cố định, chuẩn hoá, bộ nhớ đệm KV, những phần chưa cắt). **Các giá trị c ở đây là minh hoạ chứ không đo từ thiết bị thật.** Panel trái vẽ bốn giá trị c: $c = 0$ là lý tưởng (bỏ một nửa tham số, độ trễ còn một nửa), còn khi $c = 0{,}3$ thì bỏ 50% tham số chỉ đưa độ trễ còn $0{,}65$. Đường nét đứt ngang ở mức 1 là trường hợp thưa từng trọng số mà phần cứng vẫn chạy



Hình 27: Mô hình khái niệm: độ trễ giảm thế nào khi bỏ tham số, tuỳ theo phần thời gian không phụ thuộc tham số (minh hoạ, tự tính)

bằng kernel dày: số tham số khác 0 giảm nhưng thời gian không đổi. Panel phải cho tăng tốc khi bỏ đúng 50% tham số: $2{,}00$ lần, $1{,}82$, $1{,}54$ và $1{,}33$ lần với $c = 0;\ 0{,}1;\ 0{,}3;\ 0{,}5$ (tự tính).

Ví dụ tay có số kiểu Llama. Dùng cấu hình $d = 4096$, MLP trung gian 11008, 32 lớp, embedding và đầu ra $2 \cdot 32000 \cdot 4096$ tham số. Mỗi lớp có attention $4d^2 = 67\,108\,864$ và MLP ba ma trận $3 \cdot 4096 \cdot 11008 = 135\,266\,304$, tổng $202\,375\,168$ mỗi lớp. Tổng mô hình xấp xỉ $6\,738\,149\,376$ tham số (tự tính, xấp xỉ $6{,}74$ tỉ).

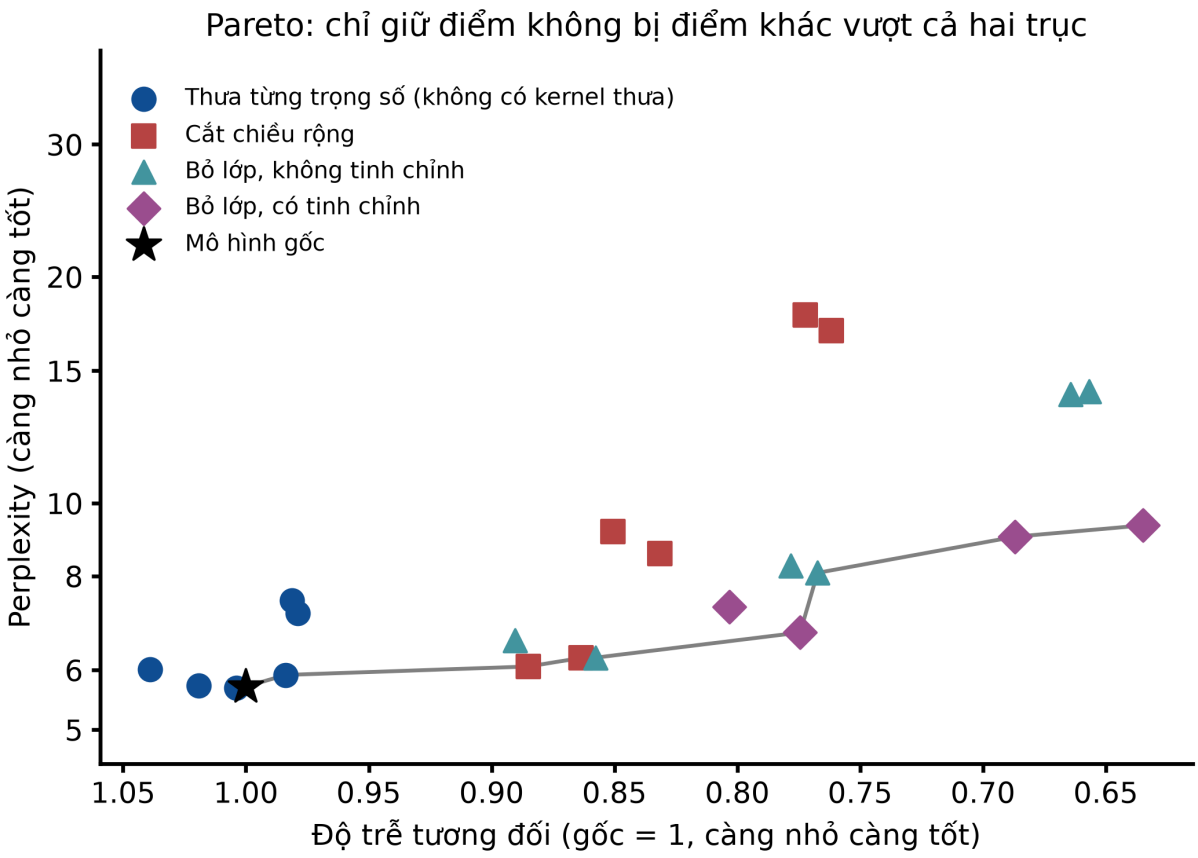
Giả sử cắt 50% chiều trung gian của MLP ở mọi lớp. Số tham số bỏ: $32 \cdot 135\,266\,304/2 = 2\,164\,260\,864$, tức $32{,}1\%$ tổng. Tỉ lệ nén $= 6{,}738/(6{,}738 - 2{,}164) = 1{,}47$ lần. Phép nhân trong các lớp giảm từ $6{,}476$ tỉ xuống $4{,}312$ tỉ tham số, nên tăng tốc lý thuyết của phần nhân ma trận là $1{,}50$ lần (bỏ qua tính điểm attention và bộ nhớ đệm KV). Nếu một phần $c = 0{,}3$ của thời gian thật không co theo tham số, độ trễ còn $0{,}3 + 0{,}7 \cdot (1 - 0{,}321) = 0{,}775$, tức tăng tốc thật chỉ khoảng $1{,}29$ lần (tự tính, theo mô hình khái niệm). Ba con số $1{,}47$, $1{,}50$, $1{,}29$ trả lời ba câu hỏi khác nhau.

Ví dụ này khớp hướng với điều ghi chú pruning 33 đã đo cho LLaMA-7B cắt 20% ở batch 1 trên H100: các phương pháp cắt chiều rộng có độ trễ $3{,}0$ đến $3{,}2$ giây, **chậm hơn** mô hình gốc ($2{,}4$ giây), còn các phương pháp bỏ lớp có $1{,}9$ giây. Chương này không khẳng định cơ chế đằng sau; chỉ nhắc rằng ghi chú đó giải thích phần thu hẹp ma trận có thể làm kích thước không thân thiện với GPU.

7.5 Biểu đồ Pareto

Cách đọc hình: trục ngang là độ trễ tương đối (đảo chiều, bên phải nhanh hơn), trục dọc là perplexity (thấp hơn là tốt hơn). Mỗi điểm là một cấu hình; **các điểm do mô phỏng sinh ngẫu nhiên (seed 0) quanh xu hướng tôi giả định**, nhằm minh hoạ khái niệm chứ không phải kết quả đo. Đường xám nối các điểm không bị điểm nào khác vượt đồng thời ở cả hai trục (biên Pareto). Điểm thưa từng trọng số nằm gần mô hình gốc vì độ trễ không đổi; các điểm bỏ lớp có tinh chỉnh (hình thoi) chiếm biên phía nhanh trong mô phỏng này.

Biên Pareto sửa một lỗi đọc phổ biến: so hai phương pháp ở **một** tỉ lệ cắt. Nếu phương pháp A cắt 50% tham số nhưng không nhanh hơn còn B cắt 30% nhưng nhanh hơn, không thể nói A "mạnh hơn". Cần nói trên trục nào, và cố định trục nào. Theo ghi chú pruning 12 (hình thí nghiệm ResNet-18), cố định kích thước mô hình hay cố định tăng tốc lý thuyết có thể **đảo thứ hạng** giữa cắt toàn cục và cắt từng lớp.



Hình 28: Biểu đồ Pareto chất lượng so với độ trễ: các điểm sinh ngẫu nhiên quanh xu hướng giả định của từng họ phương pháp (minh hoạ, tự tính)

7.6 So cùng ngân sách

Một so sánh công bằng cố định cùng một nguồn lực. Các nguồn lực hay gặp: số tham số, độ trễ, bộ nhớ, số token huấn luyện lại, và tổng chi phí tính toán gồm cả tiền huấn luyện mô hình cha. Ghi chú pruning 43 là ví dụ: nó đặt hai so sánh, một với cùng số token huấn luyện lại (50B token), một với cùng **tổng** số token (baseline từ đầu được cấp 250B token, bằng 200B của cha cộng 50B huấn luyện lại), và thấy kết luận phụ thuộc vào cách đặt ngân sách. Nếu chỉ báo so sánh đầu thì pruning thắng; khi baseline được cấp đủ tổng token, huấn luyện từ đầu bắt kịp hoặc vượt prune có cấu trúc, còn prune mịn vẫn giữ lợi thế nhẹ (theo ghi chú). Cùng một dữ liệu, hai ngân sách, hai kết luận.

7.7 Báo cáo trung thực: danh sách kiểm

Khi đọc hoặc viết kết quả pruning, kiểm các điểm sau:

- Mô hình gốc là gì (tên chính xác, bản nào) và số của nó đo bằng đúng cách đã đo mô hình cắt chưa.
- Tập đánh giá, độ dài ngữ cảnh, tokenizer, cách chấm điểm.
- Tập calibration, kích thước, có trùng với tập đánh giá không.
- Số hạt giống ngẫu nhiên và độ biến thiên; chênh lệch nhỏ hơn nhiễu thì chưa là kết luận.
- "Cắt X%" tính trên cái gì: toàn bộ tham số, chỉ các lớp tuyến tính, hay chỉ phần được cắt.
- Có cả tỉ lệ nén lẫn tăng tốc (lý thuyết và thật, trên thiết bị nào, batch nào).
- Có chữa lại (tinh chỉnh, LoRA) không và tốn bao nhiêu token.

Ghi chú pruning 12 là bản kiểm kê các thiếu sót này trên 81 bài: hơn một phần tư số bài không so sánh với phương pháp pruning nào, và số FLOPs cho cùng một kiến trúc và tập dữ liệu có thể chênh tới 4 lần giữa các bài (ví dụ AlexNet trên ImageNet được báo 371, 724 hoặc 1500 triệu FLOPs ở ba bài). Tác giả đề xuất thư viện ShrinkBench để chuẩn hoá.

8. Nối sang pruning và LLM

Số trong bài khác nhau không đọc cạnh nhau được. Ghi chú pruning 21 trình bày một bảng sai khác perplexity (WikiText-2) và tỉ lệ nén của nhiều phương pháp, và nhắc ngay rằng chúng không cùng mô hình hay điều kiện, nên một sai khác nhỏ ở mô hình 175 tỉ tham số không có nghĩa phương pháp đó "tốt hơn" một phương pháp khác chạy trên mô hình 7 đến 13 tỉ.

Mô hình lớn thưa so với mô hình nhỏ đặc. Theo ghi chú pruning 24, LLaMA-65B cắt 50% không cấu trúc có trung bình zero-shot 66,67 so với 65,38 của LLaMA-30B đặc, tức mô hình lớn thưa đứng trên mô hình nhỏ đặc ở thước đo đó; với cắt theo mẫu $N{:}M$ thì xu hướng đảo ngược khi không tinh chỉnh. Cùng một thước đo, chỉ khác kiểu thưa, cho hai kết luận.

Thưa không cấu trúc và tăng tốc. Theo ghi chú pruning 18, mẫu 2:4 cho phép giảm một nửa phép nhân tối đa 2 lần ở các phép nhân ma trận lớn, nhưng với batch nhỏ hay hidden nhỏ không đạt 2 lần; ở decoder sinh từng token, lợi ích chính có thể là giảm khoảng 38% đến 44% dung lượng và băng thông trọng số chứ không phải giảm tính toán.

Tăng tốc lý thuyết khác tăng tốc thật. Ghi chú pruning 33 và 34 cho thấy bỏ cả lớp cho tăng tốc đo được ở batch nhỏ, còn cắt kênh thì không nhất thiết; ghi chú pruning 21 nhắc MFU thấp ở batch 1.

Đo bằng hai thước. Ghi chú pruning 37 và 32 dùng nhiều hơn một thước (perplexity cộng benchmark), vì một thước có thể che hỏng hóc ở thước kia.

Cùng ngân sách. Ghi chú pruning 43 là mẫu cho cách đặt ngân sách công bằng; ghi chú pruning 12 cho danh sách thực hành.

9. Sai lầm hay gặp

- **Nhầm "tham số giảm 50%" với "nhanh gấp đôi".** Hình ở phần 7.4 cho $1{,}29$ đến $1{,}54$ lần trong mô hình khái niệm, và tăng tốc có thể bằng 1 nếu thưa không cấu trúc chạy trên kernel dày.
- **So perplexity giữa hai tokenizer hoặc hai tập dữ liệu.** Phần 5 cho một văn bản và ba PPL khác nhau.
- **Báo "tăng $0{,}1$ perplexity" mà không nói gốc là bao nhiêu.** Tăng $0{,}1$ trên 5 là 2%, trên 50 là $0{,}2$%.
- **Chọn một tỉ lệ cắt có lợi cho mình.** Cần cả đường cong, đặc biệt điểm gãy (hình phần 6).
- **Đo chất lượng bằng một thước.** Perplexity và độ chính xác trắc nghiệm có thể lệch nhau.
- **Quên nhiễu.** Hai lần chạy khác hạt giống có thể lệch nhau hơn khoảng cách giữa hai phương pháp.
- **Dùng calibration trùng tập đánh giá.** Số đẹp đến mức đáng ngờ thường do đây.

10. Tóm tắt công thức

Khái niệm	Công thức	Ý nghĩa
Độ bất ngờ	$-\ln p_i$	Mức bất ngờ của một token
Loss	$\mathcal{L} = \frac{1}{N}\sum_i(-\ln p_i)$	Bất ngờ trung bình mỗi token
Perplexity	$\text{PPL} = e^{\mathcal{L}}$	Số lựa chọn tương đương
Nhân xác suất với c	PPL nhân với 1/c	Tính chất tỉ lệ
Tỉ lệ nén	P/P'	Giảm kích thước
Tăng tốc lý thuyết	phép tính gốc chia phép tính mới	Proxy, không phải thời gian
Mô hình khái niệm độ trễ	$c + (1-c)(1-p)$	Phần cố định c làm tăng tốc nhỏ hơn mong đợi
Biên Pareto	không bị điểm khác vượt ở cả hai trục	So nhiều trục cùng lúc

11. Bài tập tự tính

Mọi đáp án đã kiểm bằng numpy.

Bài 1. Perplexity từ xác suất token

Mô hình gán xác suất $(0{,}5,\ 0{,}2,\ 0{,}1,\ 0{,}25)$ cho bốn token thật. Tính perplexity.

Đáp án (tự tính): Tích $= 0{,}5 \cdot 0{,}2 \cdot 0{,}1 \cdot 0{,}25 = 0{,}0025$. PPL $= 0{,}0025^{-1/4} = 400^{1/4} = \sqrt{20} \approx 4{,}472$.

Bài 2. Đổi qua lại giữa loss và perplexity

(a) Loss trung bình $2{,}3026$ nat tương ứng perplexity bao nhiêu? (b) Mô hình gốc có PPL $5{,}7$, mô hình cắt có PPL $6{,}2$: loss tăng bao nhiêu nat và PPL tăng bao nhiêu phần trăm?

Đáp án (tự tính): (a) $e^{2{,}3026} \approx 10{,}00$. (b) Loss tăng $\ln(6{,}2/5{,}7) = \ln 1{,}0877 \approx 0{,}084$ nat; PPL tăng $8{,}8$%.

Bài 3. Tăng tốc theo định luật phần cố định

Một mô hình có 80% thời gian nằm trong phần có thể cắt và 20% không đổi. Nếu phần cắt được chạy nhanh gấp đôi, tăng tốc tổng là bao nhiêu? Nếu nhanh gấp bốn? Giới hạn trên khi phần cắt được nhanh vô hạn?

Đáp án (tự tính): Gấp đôi: $1/(0{,}2 + 0{,}8/2) = 1/0{,}6 \approx 1{,}667$. Gấp bốn: $1/(0{,}2 + 0{,}2) = 2{,}5$. Giới hạn: $1/0{,}2 = 5$ lần.

Bài 4. Tỉ lệ nén và tăng tốc lý thuyết

Dùng cấu hình ở phần 7.4 nhưng cắt 50% MLP chỉ ở 16 trong 32 lớp. Tính tỉ lệ nén tham số và tăng tốc lý thuyết của phần nhân ma trận trong các lớp.

Đáp án (tự tính): Tham số bỏ $= 16 \cdot 135\,266\,304/2 = 1\,082\,130\,432$. Tổng $6\,738\,149\,376$, còn lại $5\,656\,018\,944$, tỉ lệ nén $\approx 1{,}191$ lần. Phép nhân trong lớp: $6\,476\,005\,376$ giảm còn $5\,393\,874\,944$, tăng tốc lý thuyết $\approx 1{,}201$ lần.

Bài 5. Xếp hạng không đổi, và chuẩn hoá độ dài

Một câu hỏi có hai phương án trả lời. Phương án A dài 3 token, phương án B dài 5 token. Log-xác suất tổng: mô hình gốc A $= -3{,}0$, B $= -4{,}0$; mô hình đã cắt A $= -4{,}5$, B $= -6{,}0$. (a) Mỗi mô hình chọn gì nếu so tổng log-xác suất? (b) Nếu chia theo số token trước khi so thì sao? (c) Mô hình đã cắt có "tệ đi" không?

Đáp án (tự tính): (a) Mô hình gốc: $-3{,}0 > -4{,}0$ chọn A; mô hình cắt: $-4{,}5 > -6{,}0$ chọn A; cả hai chọn A. (b) Gốc: A $= -3{,}0/3 = -1{,}0$, B $= -4{,}0/5 = -0{,}8$, chọn B; cắt: A $= -1{,}5$, B $= -1{,}2$, chọn B. Cả hai vẫn chọn như nhau, nhưng **phương án chọn đổi** từ A sang B chỉ vì cách chấm. (c) Log-xác suất tuyệt đối của cả hai phương án giảm rõ ($1{,}5$ nat với A) nên mô hình cắt kém tự tin hơn, dù xếp hạng không đổi trong ví dụ này; độ chính xác một mình không thấy được điều đó.

Tài liệu tham khảo nền: Jurafsky và Martin, Speech and Language Processing (chương về mô hình ngôn ngữ và perplexity); Blalock và cộng sự (State of Neural Network Pruning) cho khung báo cáo; các ghi chú pruning 12, 18, 21, 23, 24, 32, 33, 34, 37 và 43 trong sách.

Bài 00

Preface: Pruning cho người mới bắt đầu — toán nền tảng và hướng dẫn đọc

Tác giả: ghi chú tổng hợp (không bám một paper duy nhất), viết từ kiến thức nền về pruning mạng nơ-ron.
Nguồn: không có nguồn đơn lẻ; các paper được nhắc tới nằm trong lộ trình 01–49 của thư mục này.
Chủ đề: Vì sao và cách cắt tỉa (prune) mạng nơ-ron; phân loại; tiêu chí; phục hồi; toán nền; đánh giá; cách đọc paper một cách phê phán.
Vị trí trong lộ trình: Đọc **trước tiên**. Các ghi chú 01–49 giả định bạn đã nắm các ý trong file này.

> Tài liệu học tập bằng tiếng Việt, viết từ kiến thức chung của người soạn, **không phải** bản dịch hay tóm tắt một paper. Mọi con số cụ thể về paper nằm ở các ghi chú riêng.

1. Vì sao phải prune

Pruning (cắt tỉa) là bỏ đi một phần tham số hoặc cấu trúc của một mạng đã (hoặc sắp) được huấn luyện, với hy vọng giữ phần lớn chất lượng. Động lực thực tế có ba loại, và chúng **không đồng nhất**:

Mục tiêu	Đại lượng đo	Pruning giúp thế nào
Bộ nhớ lưu trữ / tải	số byte của checkpoint	bớt tham số; nhưng nếu giữ dạng thưa thì tốn thêm chỉ số (index)
Bộ nhớ chạy (VRAM)	trọng số + activation + KV cache	bớt trọng số; activation chỉ bớt khi cắt channel/width
Độ trễ (latency)	giây/token hoặc giây/ảnh	chỉ bớt khi phần cứng thật sự bỏ qua công việc
Thông lượng (throughput)	token/giây ở batch lớn	phụ thuộc compute-bound hay memory-bound
Năng lượng	Joule	truy cập bộ nhớ thường đắt hơn phép tính nhiều bậc (xem ghi chú 02)

1.1 FLOPs không phải tốc độ

FLOPs (floating-point operations) là số phép tính lý thuyết. Một mạng ít FLOPs hơn có thể chạy *không* nhanh hơn, vì:

- GPU chạy nhanh nhất với phép nhân ma trận **dày đặc, đều đặn** (dense GEMM). Mẫu truy cập thưa, bất quy tắc làm mất lợi thế này.
- Ở **batch-1 decode** của mô hình ngôn ngữ, mỗi token phải đọc toàn bộ trọng số từ bộ nhớ (HBM/DRAM) một lần. Thời gian thường bị chặn bởi băng thông bộ nhớ chứ không phải số phép nhân: gần đúng

$$t_{\text{token}} \approx \max\left(\frac{\text{bytes đọc}}{\text{băng thông}}, \frac{\text{FLOPs}}{\text{đỉnh tính toán}}\right).$$

Với batch nhỏ, số hạng đầu thường lớn hơn → muốn nhanh phải **giảm số byte thật sự phải đọc**, nghĩa là các ma trận phải *nhỏ lại* (hoặc lượng tử hóa), không chỉ "nhiều số 0".

- Các chi phí phụ (kernel launch, overhead Python, attention với KV cache dài, norm, embedding/LM-head) không giảm khi ta chỉ cắt MLP.

Vì vậy một câu hỏi luôn phải đặt ra khi đọc paper: **"giảm gì — tham số, FLOPs hay latency đo trên phần cứng nào?"**

1.2 Pruning trong bức tranh nén mô hình

Pruning chỉ là một trong vài họ kỹ thuật nén, và thường được kết hợp:

- **Quantization**: dùng ít bit hơn cho mỗi số (FP16 → INT8/INT4).
- **Knowledge distillation (KD)**: huấn luyện mô hình nhỏ bắt chước mô hình lớn.
- **Low-rank factorization**: thay $W \approx AB$ với hạng nhỏ.
- **Thiết kế kiến trúc nhỏ ngay từ đầu** (hoặc train từ đầu một mô hình nhỏ).

Một câu hỏi nghiên cứu mở xuyên suốt lộ trình: *prune từ mô hình lớn có thực sự tốt hơn train mô hình nhỏ từ đầu không?* (xem 03, 11, 43).

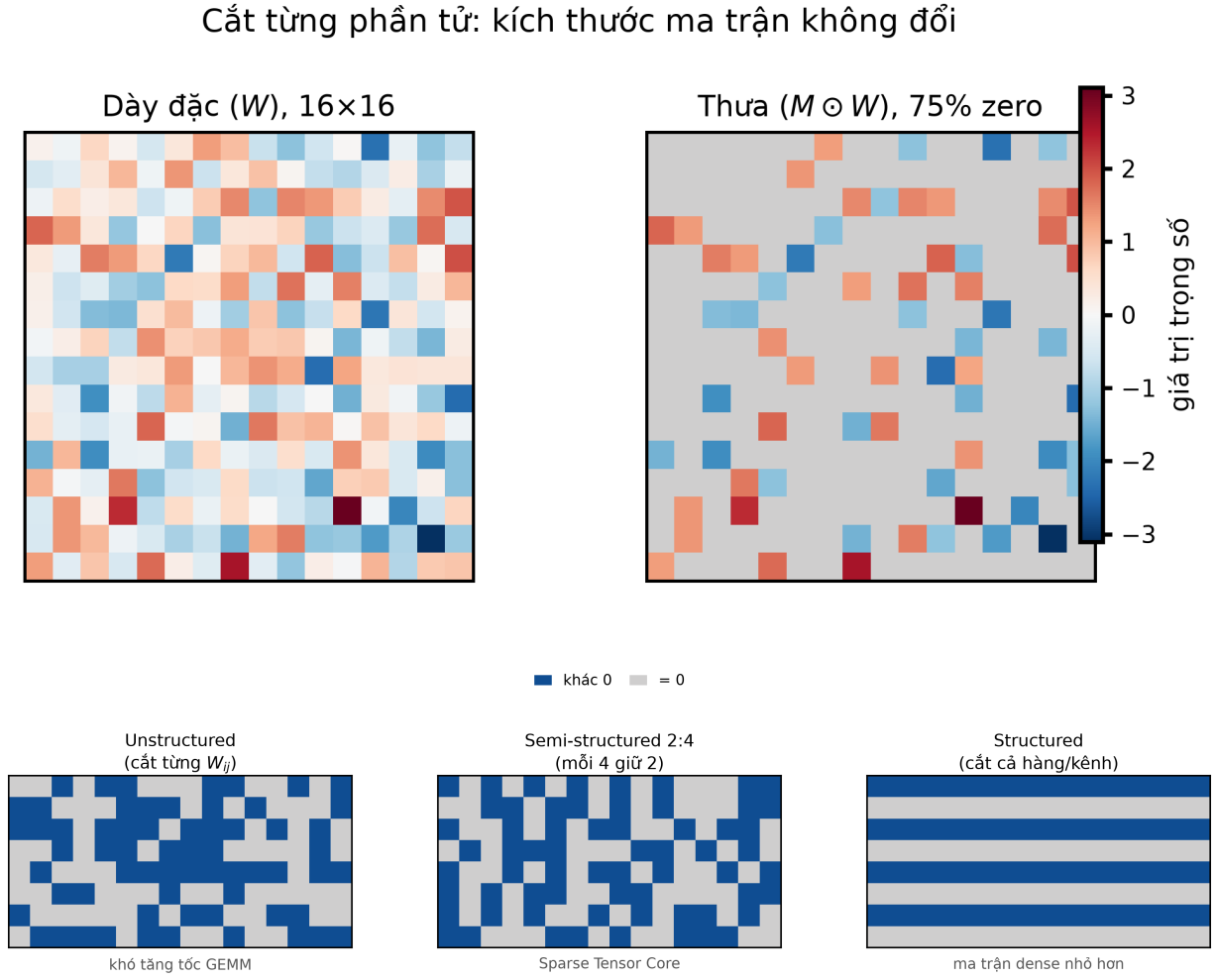
---

2. Phân loại theo "cái gì bị cắt"

Hình dung một tầng tuyến tính $y = Wx$, với $W \in \mathbb{R}^{d_{out} \times d_{in}}$.

2.1 Unstructured (không cấu trúc) — cắt từng trọng số

Đặt từng phần tử $W_{ij} = 0$ theo một mặt nạ (mask) $M \in \{0,1\}^{d_{out} \times d_{in}}$: $W' = M \odot W$. Rất linh hoạt, giữ được chất lượng ở độ thưa cao (50–90%), nhưng ma trận vẫn có kích thước cũ. Các bài tiêu biểu: 01 (OBD/OBS), 02 (Han 2015), 03 (gradual pruning), 08 (Lottery Ticket), 22–24 (OBC, SparseGPT, Wanda).



Ba mẫu thưa: unstructured, semi-structured 2:4, và structured (cắt cả hàng) — khác nhau về tăng tốc phần cứng. (minh hoạ, tự tính)

Ma trận trọng số dày đặc so với bản thưa không cấu trúc sau khi đặt mặt nạ M; kích thước không đổi dù nhiều phần tử bằng 0. (minh hoạ, tự tính)

2.2 Semi-structured, mẫu N:M

Trong mỗi nhóm M phần tử liên tiếp, giữ đúng N phần tử khác 0. Phổ biến nhất là **2:4**: mỗi 4 trọng số liên tiếp có đúng 2 số 0 → độ thưa 50%. Phần cứng NVIDIA (Ampere trở đi) có Sparse Tensor Core nén theo mẫu này và lý thuyết có thể gần gấp đôi thông lượng GEMM. Xem 18 (2:4 sparsity), 30 (MaskLLM). Hạn chế: chỉ cố định 50%, chỉ có lợi trên phần cứng/kernel được hỗ trợ, và lợi ích thực tế thấp hơn lý thuyết.

2.3 Structured (có cấu trúc) — cắt nguyên khối

Cắt cả một đơn vị tính toán, kết quả vẫn là **ma trận dày đặc nhỏ hơn**. Các mức cấu trúc, từ mịn đến thô:

Đơn vị bị cắt	Ví dụ	Ghi chú
Filter / channel (CNN)	04, 05, 06	cắt một filter kéo theo feature map tương ứng và kernel ở tầng sau
Neuron (MLP)	LLM-Pruner (26), FLAP (28)	cắt một hàng của W_{in} và một cột của W_{out}
Attention head	13 (Sixteen heads), 14 (Voita)	cắt các lát W_Q, W_K, W_V theo head và lát W_O tương ứng
Layer / block	31–37 (ShortGPT, SLEB, LaCo, ReplaceMe...)	bỏ cả khối transformer; còn lại nối tiếp qua residual
Embedding width / hidden size	29 (SliceGPT), 38–40 (Sheared LLaMA, Minitron)	giảm d_{model} — đụng đến mọi ma trận, residual stream, norm
Expert (MoE), state-dimension (SSM)	41–46	tùy kiến trúc

Nguyên tắc: càng thô thì càng dễ tăng tốc thật, nhưng mỗi lần cắt càng gây hại nhiều (mất cả "khối chức năng") và càng khó phục hồi bằng fine-tune ngắn.

2.4 Phụ thuộc cấu trúc (dependency)

Trong mạng có residual/skip, các lớp bị ràng buộc: nếu cắt channel ở đầu ra của một lớp cộng vào residual thì mọi lớp cùng cộng vào residual phải cắt cùng chỉ số. Đây là lý do có các công cụ như DepGraph (20) và các luật "nhóm tham số bị ràng buộc" (coupled structures). Ghi chú 04 (Li et al.) mô tả sớm vấn đề này với ResNet.

---

3. Cắt khi nào

3.1 Sau khi train xong rồi fine-tune (prune → fine-tune)

Quy trình kinh điển: train dense → chấm điểm → cắt → fine-tune; có thể lặp (iterative). Han 2015 (02), Li 2016 (04).

3.2 Cắt dần trong lúc train (gradual / during training)

Độ thưa tăng dần theo lịch trong khi vẫn train, để mạng thích nghi từng bước (03 Zhu & Gupta). Các biến thể học mặt nạ cùng lúc với trọng số: Network Slimming (05), Movement Pruning (16), CoFi (17).

3.3 Cắt ngay từ khởi tạo (pruning at initialization, PaI)

Chọn mạng con **trước khi train** dựa trên gradient/độ nhạy tại điểm khởi tạo (SNIP, 10) hoặc dựa trên giả thuyết vé số trúng thưởng (Lottery Ticket, 08–09). Bài học thường thấy: với mô hình lớn hiện đại, kết quả PaI thường không vượt các phương pháp cắt sau train.

3.4 Post-training một lần (one-shot, không train lại hoặc gần như không)

Đây là chế độ chính của thời LLM: không đủ tính toán để huấn luyện lại, nên dùng một lượng nhỏ dữ liệu hiệu chuẩn (calibration data, ví dụ vài trăm đoạn văn) để tính điểm và có thể *sửa trọng số còn lại* bằng đại số: OBC (22), SparseGPT (23), Wanda (24), SliceGPT (29), FLAP (28).

3.5 Cắt rồi huấn luyện lại khối lượng lớn (prune + continued pre-training / KD)

Cắt nặng, sau đó tiếp tục huấn luyện với nhiều token và/hoặc distillation từ mô hình gốc: Sheared LLaMA (38), Minitron (39–40), Puzzle (42), Nemotron-H (44).

Thời điểm	Chi phí tính toán	Mức cắt tối đa điển hình	Ví dụ
Lúc khởi tạo	thấp	thấp–vừa	10
Trong lúc train	cao (train đầy đủ)	cao	03, 05, 16
Sau train + fine-tune	vừa–cao	cao	02, 04
One-shot post-training	rất thấp	vừa (≈ 20–50%)	23, 24, 29
Cắt + train lại lớn	rất cao	rất cao (2–4×)	38–40

---

4. Tiêu chí chọn cái cần cắt

Saliency (độ quan trọng) của một tham số/cấu trúc là điểm số ước lượng "nếu bỏ nó thì tổn hại bao nhiêu". Mọi tiêu chí sau đều là cách xấp xỉ khác nhau của đại lượng này.

4.1 Magnitude (độ lớn)

$s_i = |w_i|$ (hoặc chuẩn ℓ_1/ℓ_2 của một filter/hàng). Không cần dữ liệu, rẻ, mạnh một cách đáng ngạc nhiên. Điểm yếu: bỏ qua độ lớn của đầu vào (một trọng số nhỏ nhân với activation rất lớn vẫn quan trọng) và bỏ qua tương tác. Xem 02, 04.

4.2 Gradient / Taylor bậc nhất

Dùng $s_i = |g_i w_i|$ hay $(g_i w_i)^2$ với $g_i = \partial L/\partial w_i$: khai triển bậc nhất của thay đổi loss khi đặt $w_i = 0$. Với cấu trúc (một nhóm tham số), cộng theo nhóm. Xem 07 (Molchanov 2019), 10 (SNIP), 26 (LLM-Pruner).

4.3 Hessian / bậc hai (OBD, OBS)

Dùng độ cong của loss: $s_i = \frac{1}{2} H_{ii} w_i^2$ (OBD) hoặc $\dfrac{w_i^2}{2[H^{-1}]_{ii}}$ (OBS). Đúng hơn về lý thuyết, nhưng tốn kém; các bản hiện đại làm xấp xỉ theo từng tầng: 22 (OBC), 23 (SparseGPT), 27 (LLM Surgeon). Xem mục 6.

4.4 Dựa trên activation (data-aware, theo từng tầng)

Với tầng $y = Wx$, tác động của việc bỏ W_{ij} lên đầu ra tỉ lệ với $|W_{ij}| \cdot \|x_j\|_2$ (chuẩn của cột j của dữ liệu đầu vào). **Wanda** (24) chấm điểm đúng như vậy: $s_{ij} = |W_{ij}| \cdot \|X_j\|_2$ và cắt theo từng hàng đầu ra. Rất rẻ nhưng bắt được các "outlier feature" của LLM mà magnitude bỏ lỡ.

4.5 Mặt nạ học được (learned masks)

Đưa mặt nạ (hoặc cổng) vào huấn luyện như một biến tối ưu: hệ số γ của BatchNorm với phạt ℓ_1 (05), điểm movement (16), mặt nạ ℓ_0 khả vi / hard-concrete (17 CoFi), mặt nạ N:M học bằng Gumbel-softmax (30 MaskLLM). Chính xác hơn nhưng cần train; xem mục 6.6.

4.6 Dựa trên tái tạo (reconstruction-based)

Chọn và chỉnh phần còn lại sao cho **đầu ra của tầng/khối** gần với đầu ra gốc trên dữ liệu hiệu chuẩn:

$$\min_{W'} \|WX - W'X\|_F^2.$$

Gồm ThiNet (06), OBC/SparseGPT (22, 23), SliceGPT (29), LaCo/ReplaceMe (35, 36) khi thay khối bị cắt bằng một phép biến đổi tuyến tính. Công thức đóng ở mục 6.5.

4.7 Tiêu chí cho cả khối/layer

Với việc bỏ cả layer: độ tương đồng cosine giữa đầu vào và đầu ra khối (ShortGPT 31, 32), thay đổi perplexity khi bỏ khối (34 SLEB, 33), hoặc ước lượng "chi phí còn lại" sau khi thay bằng ánh xạ tuyến tính tối ưu (36, 37).

---

5. Phục hồi sau khi cắt

Cắt gây một "cú sốc"; phục hồi là bước kéo chất lượng trở lại.

1. **Fine-tune toàn bộ trọng số còn lại** với loss gốc (cross-entropy), thường với learning rate nhỏ hơn lúc pretrain (02: 1/10 hoặc 1/100 LR gốc; 04: LR hằng 0.001). Tốt nhất nhưng tốn bộ nhớ và dữ liệu.
2. **LoRA / adapter**: chỉ train các ma trận hạng thấp thêm vào; rẻ, dùng trong LLM-Pruner (26). Hạn chế: dung lượng phục hồi có giới hạn khi cắt nặng.
3. **Knowledge distillation**: dùng mô hình gốc (hoặc một teacher lớn hơn) làm giáo viên; loss thường là

$$\mathcal{L} = \alpha\, \mathrm{KL}\big(p_T(\cdot \mid x) \,\|\, p_S(\cdot \mid x)\big) + (1-\alpha)\, \mathrm{CE}(y, p_S),$$

có thể thêm khớp hidden states / attention. Thường phục hồi tốt hơn CE đơn thuần (xem 38–40, 43).

4. **Sửa trọng số bằng đại số** (không gradient): OBS, SparseGPT, SliceGPT bù bằng nghiệm bình phương tối thiểu. Rất rẻ nhưng chỉ sửa được trong phạm vi một tầng.
5. **Rewinding / retrain từ đầu với mặt nạ**: nền của Lottery Ticket (08, 09) và "Rethinking the value of pruning" (11).

Quy tắc kinh nghiệm: ngân sách phục hồi (số token, số bước) quyết định kết quả nhiều không kém tiêu chí cắt — vì vậy cần so sánh có kiểm soát (mục 10).

---

6. Toán cần biết

Ký hiệu: $w \in \mathbb{R}^n$ là vector mọi tham số, $L(w)$ loss trung bình trên dữ liệu huấn luyện, $g = \nabla L(w)$, $H = \nabla^2 L(w)$.

6.1 Khai triển Taylor bậc hai của loss

Khi nhiễu loạn $w \to w + \delta$:

$$\boxed{L(w+\delta) \;\approx\; L(w) + g^\top \delta + \tfrac{1}{2}\, \delta^\top H\, \delta}$$

Nếu mạng đã train xong và nằm gần một cực tiểu địa phương thì $g \approx 0$, nên

$$\Delta L \approx \tfrac{1}{2}\,\delta^\top H\,\delta.$$

Ba điều cần nhớ:

- H là ma trận $n \times n$ (n có thể là 10^9) → không lưu nổi; phải xấp xỉ.
- $H \succeq 0$ tại cực tiểu: mọi nhiễu loạn chỉ làm loss tăng (hoặc giữ nguyên).
- "Gần cực tiểu" chỉ đúng một phần: mô hình lớn thường được train bằng ít epoch và chưa hội tụ hẳn, nên $g \neq 0$ và số hạng bậc nhất có thể quan trọng (lý do có các tiêu chí Taylor bậc nhất ở 4.2).

Cắt tham số q nghĩa là $\delta_q = -w_q$ (còn các thành phần khác tự do chỉnh nếu ta cho phép).

6.2 Hessian và xấp xỉ Fisher / Gauss–Newton

Với loss là log-likelihood âm $L = -\frac{1}{N}\sum_n \log p(y_n \mid x_n; w)$, **ma trận thông tin Fisher** là

$$F = \mathbb{E}\big[\nabla_w \log p\, \nabla_w \log p^\top\big] \;\approx\; \frac{1}{N}\sum_n g_n g_n^\top, \qquad g_n = \nabla_w \ell_n.$$

Khi mô hình khớp tốt phân phối dữ liệu thì $H \approx F$ (Hessian thật bằng Fisher cộng một số hạng chứa phần dư). Với bình phương sai số $\ell = \frac{1}{2}\|t - F(w,x)\|^2$, bỏ số hạng phần dư $(t-o)$ cho **Gauss–Newton**:

$$H \approx \frac{1}{P}\sum_k \frac{\partial F(w,x_k)}{\partial w}\,\frac{\partial F(w,x_k)}{\partial w}^\top.$$

Lợi ích: nửa dương xác định (PSD), tính được từ gradient (xem bài OBS: đây chính là Eq. 10–12 của Hassibi & Stork, ghi chú 01). Với một tầng tuyến tính $y = Wx$ và loss bậc hai trên đầu ra, Hessian theo mỗi hàng w của W là $H = 2XX^\top$ (với $X \in \mathbb{R}^{d_{in}\times N}$) — dùng trong OBC/SparseGPT.

Diagonal / K-FAC. Chỉ giữ đường chéo H_{ii} (OBD, 01), hoặc xấp xỉ khối theo Kronecker $H_\ell \approx A \otimes G$ (K-FAC; dùng trong LLM Surgeon, 27).

6.3 OBD: Optimal Brain Damage

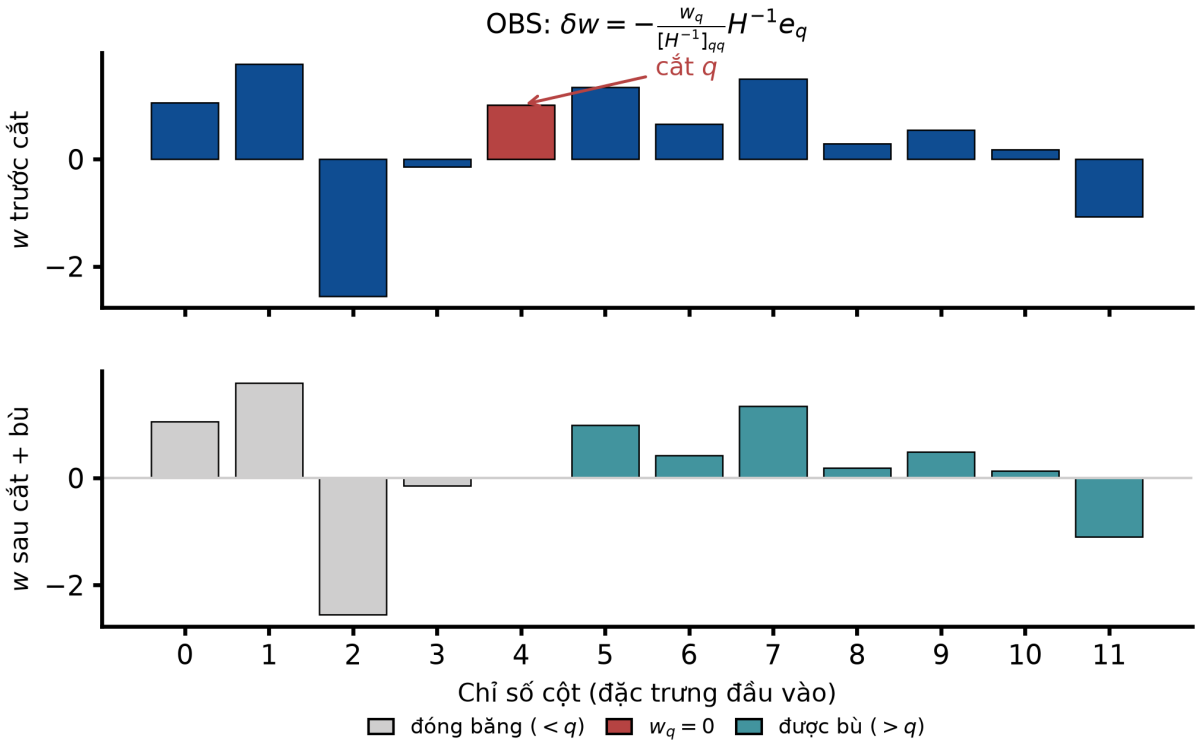
Giả thiết: (i) cực tiểu ($g \approx 0$), (ii) bậc hai, (iii) đường chéo (các hệ số chéo $H_{ij}, i \neq j$ bị bỏ). Khi đó cắt w_k làm loss tăng

$$\boxed{s_k = \tfrac{1}{2}\,H_{kk}\,w_k^2}$$

Cắt các tham số có s_k nhỏ nhất; sau đó train lại. Magnitude pruning là trường hợp $H \propto I$ (cong như nhau theo mọi hướng), khi đó $s_k \propto w_k^2$. Chi tiết ở ghi chú 01.

6.4 OBS: Optimal Brain Surgeon — dẫn xuất tóm tắt

Bỏ giả thiết đường chéo. Muốn đặt w_q về 0 mà làm loss tăng ít nhất, **có quyền chỉnh mọi trọng số khác**:



OBS/SparseGPT: đặt $w_q = 0$ rồi bù vào các cột bên phải (đóng băng cột đã xử lý). (minh hoạ, tự tính)

$$\min_{\delta} \ \tfrac{1}{2}\,\delta^\top H \delta \quad \text{s.t.} \quad e_q^\top \delta + w_q = 0.$$

Lagrangian $\mathcal{L} = \frac{1}{2}\delta^\top H\delta + \lambda\,(e_q^\top \delta + w_q)$. Đạo hàm theo δ bằng 0: $H\delta + \lambda e_q = 0 \Rightarrow \delta = -\lambda H^{-1} e_q$. Thế vào ràng buộc: $-\lambda\,[H^{-1}]_{qq} + w_q = 0 \Rightarrow \lambda = \dfrac{w_q}{[H^{-1}]_{qq}}$. Vậy

$$\boxed{\delta w = -\frac{w_q}{[H^{-1}]_{qq}}\, H^{-1} e_q, \qquad \Delta L_q = \frac{w_q^2}{2\,[H^{-1}]_{qq}}}$$

Hai kết luận: (1) saliency ΔL_q dùng $[H^{-1}]_{qq}$ chứ không phải $1/H_{qq}$ (hai đại lượng bằng nhau khi H chéo); (2) công thức δw **cập nhật mọi trọng số còn lại** để bù, không cần gradient descent. Khó khăn là tính H^{-1}, giải quyết bằng công thức đệ quy (Sherman–Morrison/Woodbury) trong paper gốc. SparseGPT/OBC khai thác lại đúng cấu trúc này theo từng hàng.

6.5 Tái tạo tầng bằng bình phương tối thiểu và nghiệm đóng

Bài toán tầng (layer-wise). Cho $W \in \mathbb{R}^{d_{out}\times d_{in}}$ và dữ liệu hiệu chuẩn $X \in \mathbb{R}^{d_{in}\times N}$ (mỗi cột là một đầu vào). Tìm $\hat{W}$ thỏa ràng buộc thưa/cấu trúc để

$$\min_{\hat{W}} \ \|WX - \hat{W}X\|_F^2 = \sum_{r=1}^{d_{out}} \|(w_r - \hat{w}_r)X\|_2^2.$$

Bài toán tách theo từng hàng r. Gọi $H = XX^\top \in \mathbb{R}^{d_{in}\times d_{in}}$ (ma trận Gram, chính là Hessian đến hằng số).

Mặt nạ cho trước, giữ tập chỉ số S (các cột còn lại): ta chỉ có tự do ở $\hat{w}_S$. Điều kiện cực trị:

$$\boxed{\hat{w}_S = \big(H_{SS}\big)^{-1} H_{S,:}\, w^\top \;=\; w_S + H_{SS}^{-1} H_{S\bar{S}}\, w_{\bar{S}}}$$

(với $\bar{S}$ = tập bị cắt). Dạng thứ hai cho thấy trọng số còn lại *được cộng thêm* một phần bù từ trọng số bị cắt, tỉ lệ với tương quan giữa các đặc trưng. Nếu các cột X_S và $X_{\bar{S}}$ không tương quan ($H_{S\bar{S}} = 0$), không có gì để bù và nghiệm tối ưu giữ nguyên w_S. Thực tế thêm một số hạng ổn định $H \leftarrow H + \lambda I$ (damping).

Bù bằng ánh xạ tuyến tính khi cắt cấu trúc. Giả sử ta bỏ một số kênh của đặc trưng $Z \in \mathbb{R}^{d\times N}$ (đầu ra của khối bị cắt) và muốn tái dựng Z từ phần giữ lại Z_S bằng một ma trận T:

$$\min_T \; \|Z - TZ_S\|_F^2 \;\Rightarrow\; \boxed{T^\star = ZZ_S^\top \big(Z_S Z_S^\top\big)^{-1} = ZZ_S^+}$$

Sai số tái tạo tối thiểu $\|Z - T^\star Z_S\|_F^2$ đo phần thông tin chưa được ánh xạ tuyến tính $T^\star$ khôi phục trên dữ liệu hiệu chuẩn.

6.6 Nới lỏng Lagrangian và ℓ_0 cho mặt nạ

Bài toán gốc: $\min_{w,m}\; L(m \odot w)$ s.t. $\|m\|_0 \le k$, $m \in \{0,1\}^n$. Không khả vi. Các cách xử lý:

- **Phạt** ℓ_1 lên hệ số cổng: $\min L + \lambda \sum_i |\gamma_i|$ (05, Network Slimming). Đạo hàm dưới (subgradient) là $\lambda\,\mathrm{sign}(\gamma_i)$; γ_i nhỏ → cổng gần như đóng → cắt.
- **Phạt** ℓ_0 **khả vi:** biến ngẫu nhiên $z_i \in [0,1]$ ("hard-concrete") có xác suất khác 0 là $\sigma(\log \alpha_i - \beta \log(-l/r))$; kỳ vọng $\mathbb{E}\|z\|_0 = \sum_i \sigma(\cdot)$ là hàm khả vi của tham số $\log \alpha_i$ (CoFi, 17).
- **Lagrangian cho ngân sách:**

$$\min_{w,z}\max_{\lambda_1,\lambda_2}\; L(w,z) + \lambda_1(\hat{s} - t) + \lambda_2(\hat{s} - t)^2,$$

với $\hat{s}$ là độ thưa kỳ vọng và t là mục tiêu; hai nhân tử λ được học để *ép đúng ngân sách* thay vì chỉnh tay hệ số phạt.

- **Straight-through estimator (STE):** lan truyền gradient qua phép làm tròn/top-k như thể nó là hàm đồng nhất (movement pruning 16, MaskLLM 30 dùng biến thể).

6.7 Bỏ một neuron = bỏ một hàng và một cột

Xét MLP một tầng ẩn $h = W_{out}\,\sigma(W_{in}x + b)$ với $W_{in} \in \mathbb{R}^{m\times d}$, $W_{out} \in \mathbb{R}^{d\times m}$, m neuron ẩn. Viết

$$h = \sum_{j=1}^{m} (W_{out})_{:,j}\;\sigma\big((W_{in})_{j,:}x + b_j\big).$$

Mỗi neuron j đóng góp một số hạng độc lập: **hàng** j **của** W_{in} (cộng bias b_j) tạo activation vô hướng, **cột** j **của** W_{out} ghi nó vào residual. Bỏ neuron j là xóa số hạng đó, tức *xóa hàng* j *của* W_{in}, *phần tử* b_j *và cột* j *của* W_{out}. Kết quả là hai ma trận nhỏ hơn, dày đặc. Với MLP có cổng (gated, SwiGLU) $W_{down}(\sigma(W_{gate}x) \odot W_{up}x)$ thì xóa **hàng** j **của** W_{gate} **và** W_{up} **cùng cột** j **của** W_{down}.

Tương tự với CNN: bỏ filter j của tầng i → bỏ feature map thứ j → bỏ kernel ứng với kênh j ở tầng $i{+}1$ (04). Với attention: bỏ head h → bỏ lát đầu ra của W_Q, W_K, W_V và lát cột của W_O.

Số tham số của MLP tỉ lệ tuyến tính với m, nên giữ tỉ lệ $\rho = m'/m$ neuron thì tham số MLP giảm còn ρ — đây là lý do cắt neuron có thể tính trước chính xác lượng nén.

6.8 Điểm hay nhầm: "độ thưa" ≠ "tỷ lệ nén"

Độ thưa 90% (10% khác 0) cho 10× ít tham số khác 0, nhưng lưu trữ thưa cần thêm chỉ số: mặt nạ bit tốn 1 bit/phần tử bất kể độ thưa, CSR tốn một số bit cho mỗi phần tử khác 0 (xem 02, 03). Với trọng số INT4 thì chi phí chỉ số tương đối còn lớn hơn.

---

7. Vì sao structured cho tốc độ thật

Dense GEMM trên GPU. Phép nhân $Y = WX$ được chia ô (tiling) để dùng Tensor Core: các ô 16×16 hoặc tương tự được nạp và nhân liên tục. Hiệu năng đến từ truy cập bộ nhớ liền kề và tái sử dụng dữ liệu.

Unstructured. Đặt 90% phần tử về 0 mà giữ ma trận dạng dày đặc → *cùng số phép tính, cùng số byte*. Muốn tận dụng, phải dùng định dạng thưa (CSR/COO) và kernel SpMM. Khi mật độ còn cao (độ thưa dưới ≈ 90–95%) các kernel này thường chậm hơn dense trên GPU vì truy cập gián tiếp, mất coalescing, không dùng được Tensor Core hiệu quả. Chỉ ở độ thưa cực cao hoặc trên phần cứng chuyên dụng (như bộ tăng tốc EIE được nhắc ở 04 và 03) mới có lợi.

Semi-structured 2:4. Mẫu đều → phần cứng nén 2 giá trị khác 0 + 2 bit chỉ số/ nhóm, Tensor Core bỏ qua phép nhân với số 0. Lý thuyết tối đa ≈ 2× cho GEMM; thực tế thấp hơn vì phần còn lại của mô hình (attention, norm, memory bound) không nhanh lên.

Structured. Ma trận nhỏ hơn thật: ít byte phải đọc (giúp bộ nhớ-bound/batch-1), ít FLOPs (giúp compute-bound/batch lớn), activation nhỏ hơn. Không cần kernel đặc biệt. Điều kiện: kích thước mới nên là bội của 8/16/64/128 để tránh mất hiệu năng do căn chỉnh (alignment); cắt tới số lẻ có thể không nhanh hơn.

Các cạm bẫy.

- Cắt MLP không làm nhanh attention/KV cache; cắt layer làm giảm cả KV cache (với full attention), cắt width chỉ giảm KV nếu cắt số head KV.
- Độ trễ batch-1 còn có phần cố định (launch kernel, norm, embedding, LM-head). Embedding và LM-head có thể chiếm phần đáng kể trong một mô hình nhỏ — cắt thân transformer không ảnh hưởng chúng.
- Giảm *chiều sâu* (số layer) giảm độ trễ gần tuyến tính ở batch-1 vì các layer chạy tuần tự; giảm *chiều rộng* chỉ giảm thời gian đọc trọng số và có thể kém hiệu quả hơn khi kernel quá nhỏ.

---

8. Đánh giá

8.1 Chất lượng

- **Perplexity (PPL)** trên WikiText-2, C4...: $\text{PPL} = \exp(\text{CE trung bình})$. Rẻ, nhạy, nhưng nhiều khi *không* phản ánh khả năng thực hiện tác vụ; mô hình cắt có thể có PPL gần mốc mà làm tác vụ sinh dài tệ hơn.
- **Zero-shot** (ARC, HellaSwag, PIQA, WinoGrande, BoolQ...): độ chính xác trắc nghiệm; nhiễu vài điểm ở mô hình nhỏ, cần cho biết số mẫu/seed.
- **Tác vụ sinh và suy luận dài** (MMLU, GSM8K, code, tác vụ đích): thường nhạy hơn với cắt.

8.2 Hiệu năng

- Báo cáo **latency theo từng điều kiện**: batch-1 vs batch lớn; prefill vs decode; độ dài ngữ cảnh; độ chính xác số (FP16/INT8); GPU cụ thể; framework.
- Thực hiện **warm-up** vài lần, `torch.cuda.synchronize()` trước/sau khi đo, lặp nhiều lần và báo trung vị cùng phân vị cao; khóa xung nhịp nếu có thể.

- **Throughput** (token/giây ở batch lớn) chủ yếu bị chặn bởi compute; **latency batch-1** bị chặn bởi bộ nhớ. Một phương pháp có thể thắng một và thua cái kia.
- Đo **bộ nhớ đỉnh** (trọng số + activation + KV cache), không chỉ kích thước file.
- Đừng suy ra tốc độ từ FLOPs hay số tham số.
- Tính cả thời gian pipeline đầy đủ (tiền xử lý ảnh, ViT encoder, decoder, hậu xử lý) khi mô hình là VLM.

8.3 Chi phí để tạo ra mô hình

Báo cáo số token/giờ GPU dùng cho cắt + phục hồi. Hai phương pháp chỉ so sánh công bằng khi cùng ngân sách tính toán (mục 10).

---

9. Bảng thuật ngữ

Thuật ngữ	Giải thích ngắn
Pruning	Cắt bỏ tham số/cấu trúc khỏi mạng
Sparsity (độ thưa)	Tỷ lệ tham số bằng 0; 90% nghĩa là còn 10% khác 0
Mask (mặt nạ)	Tensor 0/1 chỉ ra phần nào còn giữ
Saliency	Điểm quan trọng; càng nhỏ càng đáng cắt
Unstructured	Cắt từng trọng số riêng lẻ
Structured	Cắt nguyên channel/neuron/head/layer
N:M (2:4)	Mỗi nhóm M có đúng N phần tử khác 0
Global vs layer-wise	Ngưỡng chung toàn mạng vs ngưỡng theo từng tầng
One-shot	Cắt một lần đến mức đích
Iterative / gradual	Cắt nhiều bước nhỏ xen kẽ huấn luyện
Fine-tune / retrain	Huấn luyện tiếp sau khi cắt
Rewinding	Quay trọng số về một thời điểm sớm rồi train lại với mask (08)
Calibration data	Lượng nhỏ dữ liệu dùng để tính thống kê (Hessian, activation)
Hessian	Ma trận đạo hàm bậc hai của loss
Fisher	Kỳ vọng tích ngoại của gradient; xấp xỉ Hessian
OBD / OBS	Optimal Brain Damage / Surgeon (01)
Taylor criterion	Điểm dựa trên khai triển Taylor của loss
Reconstruction error	Sai số giữa đầu ra tầng gốc và tầng đã cắt
ℓ_0 / ℓ_1	Số phần tử khác 0 / tổng trị tuyệt đối
STE	Straight-through estimator: gradient đi qua phép rời rạc
Channel / filter	Đơn vị đầu ra của conv; bỏ filter = bỏ channel đầu ra
Head	Một nhánh attention
Depth vs width pruning	Bớt số layer vs bớt chiều trong mỗi layer
KD	Knowledge distillation: học bắt chước teacher
Teacher / student	Mô hình lớn dạy / mô hình nhỏ học
PPL	Perplexity
Memory-bound / compute-bound	Bị giới hạn bởi băng thông bộ nhớ / bởi tốc độ tính
KV cache	Bộ nhớ khóa/giá trị của attention khi sinh
FLOPs vs MACs	Số phép tính dấu phẩy động; 1 MAC = 2 FLOPs

---

10. Đọc paper pruning một cách phê phán

Lĩnh vực này nổi tiếng khó so sánh. Bài khảo sát thực nghiệm **"What is the State of Neural Network Pruning?" (Blalock et al., 2020; ghi chú 12)** tổng hợp 81 paper và chỉ ra các vấn đề dưới đây. Hãy dùng như danh sách kiểm tra:

1. **Baseline có công bằng không?** Mô hình dense gốc có được train tốt không? Có so với một mô hình nhỏ cùng kích thước train đủ (03, 11, 43)? Có so với một baseline đơn giản (magnitude, random) được chỉnh cẩn thận?
2. **Ngân sách retrain.** Cắt A với 1 epoch và cắt B với 100 epoch thì không so được. Tìm: số bước/epoch/token, learning rate schedule, có dùng distillation hay không.
3. **Công bằng tính toán.** Phương pháp có cần Hessian, nhiều lần forward, huấn luyện mặt nạ? Tính cả giờ GPU của bước cắt.
4. **Metric nén.** Tham số hay FLOPs hay latency? Có tính chi phí lưu chỉ số cho dạng thưa? Cắt embedding có được tính?
5. **Dữ liệu và mô hình.** Chỉ thử trên MNIST/CIFAR hay trên ImageNet/LLM? Kết quả có chuyển được sang kiến trúc khác?
6. **Báo cáo độ biến thiên.** Có nhiều seed và khoảng tin cậy? Chênh lệch 0.1% có ý nghĩa không?
7. **Đường cong, không phải một điểm.** Một điểm (ví dụ "50% mà không giảm") không nói gì về đánh đổi; cần đường cong chất lượng–nén. Blalock et al. nhấn mạnh việc dùng nhiều mức nén và nhiều kiến trúc.
8. **Đo trên phần cứng đích.** Tốc độ lý thuyết ≠ tốc độ đo được; xem mục 7–8.
9. **Dữ liệu hiệu chuẩn và đánh giá có rò rỉ không?** Calibrate trên C4 rồi đánh giá trên C4 sẽ lạc quan.
10. **Tách "cái gì" và "cách phục hồi".** Hai phương pháp tiêu chí khác nhau nhưng khác cả cách retrain thì không biết cái nào tạo ra cải thiện.

Khi viết ghi chú cho từng bài, ta tách rõ **điều paper chứng minh / điều paper giả thuyết / điều ta suy ra**.

---

11. Lịch đọc gợi ý

Mỗi "tuần" là khoảng 3–5 bài; có thể đọc nhanh hoặc bỏ qua các bài mức "tham khảo" nếu thiếu thời gian.

Tuần 1 — Nền móng cổ điển (CNN, bắt buộc). 01 OBD/OBS → 02 Han 2015 → 03 Zhu & Gupta → 04 Pruning filters → 05 Network slimming. Mục tiêu: nắm saliency, magnitude, iterative, gradual, structured vs unstructured, học mặt nạ qua ℓ_1.

Tuần 2 — Tiêu chí và câu hỏi cơ bản. 06 ThiNet (tái tạo) → 07 Molchanov (Taylor) → 08 Lottery Ticket → 09 Linear mode connectivity → 10 SNIP → 11 Rethinking the value of pruning → 12 State of pruning (đọc kỹ cho việc đánh giá phê phán).

Tuần 3 — Transformer và cấu trúc. 13 Sixteen heads → 14 Voita → 15 LTH cho BERT → 16 Movement pruning → 17 CoFi → 18 2:4 sparsity → 19 HALP → 20 DepGraph → 21 khảo sát nén LLM (làm bản đồ).

Tuần 4 — Post-training cho LLM. 22 OBC → 23 SparseGPT → 24 Wanda → 25 ZipLM → 26 LLM-Pruner → 27 LLM Surgeon → 28 FLAP → 29 SliceGPT → 30 MaskLLM.

Tuần 5 — Cắt theo chiều sâu. 31 ShortGPT → 32 Unreasonable Ineffectiveness of Deeper Layers → 33 Shortened LLaMA → 34 SLEB → 35 LaCo → 36 ReplaceMe → 37 Repair Changes Which Layers to Remove.

Tuần 6 — Cắt + phục hồi quy mô lớn, kiến trúc lai. 38 Sheared LLaMA → 39 Minitron → 40 Llama-3.1-Minitron → 41 Flextron → 42 Puzzle → 43 Small LLMs: pruning vs scratch → 44 Nemotron-H → 45 Minitron-SSM → 46 Mamba-Shedder → 47 Jet-Nemotron.

Tuần 7 — Đa phương thức. 48 ECoFLaP → 49 Structural pruning of VLMs.

---

12. Năm câu tự kiểm tra

Câu 1. Một mô hình chạy batch-1 trên GPU bị chặn bởi băng thông bộ nhớ. Bạn chọn giữa (a) độ thưa 50% unstructured và (b) cắt 30% neuron MLP có cấu trúc. Phương án nào chắc chắn giảm latency mà không cần kernel đặc biệt? Vì sao?

Đáp án.

(b). Cắt neuron có cấu trúc làm các ma trận W_{in}, W_{out} nhỏ lại thật sự, nên số byte phải đọc mỗi token giảm và dùng được GEMM dense bình thường. (a) giữ nguyên kích thước ma trận; muốn tận dụng phải có kernel thưa hoặc mẫu 2:4 + Tensor Core, và ở mật độ 50% các kernel thưa thông thường thường chậm hơn dense. Lưu ý (b) chỉ giảm phần MLP, không giảm attention/KV cache/embedding.

Câu 2. Từ khai triển Taylor bậc hai và giả thiết $g \approx 0$, hãy dẫn ra saliency OBD $\frac{1}{2}H_{kk}w_k^2$, và nói OBS khác ở điểm nào.

Đáp án.

$\Delta L \approx \frac{1}{2}\delta^\top H\delta$. Cắt tham số k nghĩa là $\delta = -w_k e_k$ (chỉ thay đổi thành phần k, giả thiết đường chéo bỏ qua tương tác). Khi đó $\Delta L = \frac{1}{2}H_{kk}w_k^2$. OBS không giả thiết chéo và cho phép các trọng số khác tự điều chỉnh: giải $\min \frac{1}{2}\delta^\top H\delta$ s.t. $e_q^\top \delta = -w_q$ bằng Lagrange, được $\Delta L_q = w_q^2/(2[H^{-1}]_{qq})$ và $\delta w = -\frac{w_q}{[H^{-1}]_{qq}}H^{-1}e_q$ (cập nhật cả các trọng số còn lại).

Câu 3. Cho tầng $y = Wx$ và dữ liệu hiệu chuẩn X. Với tập cột giữ lại S cố định, viết nghiệm tối ưu cho một hàng $\hat{w}_S$ và giải thích khi nào "bù trọng số" không cần thiết.

Đáp án.

$\hat{w}_S = H_{SS}^{-1}H_{S,:}w^\top = w_S + H_{SS}^{-1}H_{S\bar{S}}w_{\bar{S}}$ với $H = XX^\top$. Nếu các đặc trưng bị cắt không tương quan với đặc trưng giữ lại ($H_{S\bar{S}} = 0$) thì số hạng bù bằng 0 và $\hat{w}_S = w_S$: chỉ cần cắt, không cần chỉnh. Ngược lại, đặc trưng càng tương quan thì phần bù càng lớn và việc bù càng hữu ích.

Câu 4. Trong MLP SwiGLU, muốn bỏ neuron trung gian thứ j thì phải xóa những phần nào của các ma trận?

Đáp án.

Hàng j của W_{gate}, hàng j của W_{up} và cột j của W_{down} (cùng bias nếu có). Ba lát này cùng tham gia tạo và ghi neuron j; bỏ thiếu một lát sẽ làm kích thước không khớp hoặc để lại phần tử vô nghĩa.

Câu 5. Một paper báo cáo "cắt 50% không mất chất lượng" bằng phương pháp mới và so với một baseline magnitude chỉ fine-tune 1 epoch trong khi phương pháp mới dùng 10 epoch + distillation. Hãy nêu ít nhất ba câu hỏi bạn sẽ đặt ra.

Đáp án.

Ví dụ: (1) Ngân sách retrain/tính toán có bằng nhau không? Baseline được fine-tune cùng số bước và cùng distillation thì kết quả thế nào? (2) Chỉ số là tham số, FLOPs hay latency đo thực, trên phần cứng nào, batch nào? (3) Đây là một điểm hay đường cong nén–chất lượng, có nhiều seed không? (4) Dữ liệu hiệu chuẩn/đánh giá có chồng lấn không? (5) Kết quả có đúng trên mô hình/kiến trúc khác (Blalock et al., 12)?

Kết luận ngắn

Nhớ năm ý: (1) FLOPs/tham số khác tốc độ thật; (2) structured cho ma trận nhỏ lại, unstructured thì không (trừ khi có phần cứng/kernel); (3) mọi tiêu chí đều xấp xỉ ΔL khi bỏ một thành phần, từ magnitude đến Hessian; (4) bù trọng số/phục hồi (fine-tune, KD) quyết định kết quả cuối không kém tiêu chí; (5) so sánh phải cùng ngân sách và đo trên phần cứng đích. Từ đây mở **01_1989_1993_optimal_brain_damage_surgeon.md**.

Bài 01

Optimal Brain Damage (1989/1990) và Second Order Derivatives for Network Pruning: Optimal Brain Surgeon (1992/1993)

Tác giả: (OBD) Yann Le Cun, John S. Denker, Sara A. Solla — AT&T Bell Laboratories. (OBS) Babak Hassibi, David G. Stork — Ricoh California Research Center / Stanford.
Venue / arXiv: Không có trên arXiv. OBD: NeurIPS (NIPS) 1989, in trong volume 2 (trang 598–605, xuất bản 1990). OBS: NeurIPS (NIPS) 1992, trang 164–171 (xuất bản 1993). Hai bài được tải về từ proceedings.neurips.cc và **đọc đầy đủ** để viết ghi chú này.
Nguồn: https://proceedings.neurips.cc/paper_files/paper/1989/hash/6c9882bbac1c7093bd25041881277658-Abstract.html và https://proceedings.neurips.cc/paper_files/paper/1992/hash/303ed4c69846ab36c2904d3ba8573050-Abstract.html
Chủ đề: Pruning dựa trên đạo hàm bậc hai (saliency theo Hessian); gốc rễ của mọi phương pháp "tiêu chí chính xác + bù trọng số".
Vị trí trong lộ trình: Đọc ngay sau `00_preface`. Chưa cần bài nào trước. Bài này đặt nền cho: 02 Han 2015 (magnitude + retrain, phản ứng với "second-order đắt"), 07 Molchanov (Taylor), 22 OBC, 23 SparseGPT, 27 LLM Surgeon (hồi sinh OBS ở quy mô lớn).
Chương nền liên quan: nền 01: Vì sao mạng cắt được, nền 03: Độ quan trọng của trọng số

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ hai bản PDF gốc. Một số chỗ OCR của PDF bị lỗi (công thức); các công thức dưới đây được dựng lại theo logic của paper và đối chiếu với phần mô tả bằng lời.

1. Bối cảnh

Cuối thập niên 1980, dữ liệu ít khiến **quá khớp** (overfitting) trở thành một vấn đề ngay cả với các mạng nơ-ron nhỏ: mạng có quá nhiều trọng số so với số mẫu thì khái quát hóa kém. Giải pháp thống kê cổ điển là **cân bằng sai số huấn luyện với độ phức tạp mô hình** (AIC, MDL, VC-dimension...). Độ phức tạp tự nhiên cho mạng nơ-ron là *số tham số tự do khác 0*.

Vấn đề: nên xóa tham số nào trước? Hai cách làm trước đó mà hai bài đề cập:

- **Magnitude / weight decay không đều**: xóa trọng số nhỏ (Hanson & Pratt, Rumelhart, Chauvin...). OBD gọi đây là giả định "magnitude equals saliency" và nói cần đi xa hơn nó.
- **Gating / skeletonization** (Mozer & Smolensky): thêm hệ số cổng. Cần tinh chỉnh hệ số và làm chậm học.

Saliency (độ quan trọng) ở đây được định nghĩa rõ: *mức tăng của hàm mục tiêu khi xóa tham số đó*. Bài toán là tính nó mà không phải xóa thử từng tham số và đánh giá lại.

OBD (1989) cho đáp án bằng đạo hàm bậc hai đường chéo. OBS (1992) chỉ ra giả định đường chéo là sai trên các bài toán thật và đưa ra lời giải đầy đủ với Hessian nghịch đảo, đồng thời **tự điều chỉnh các trọng số còn lại**.

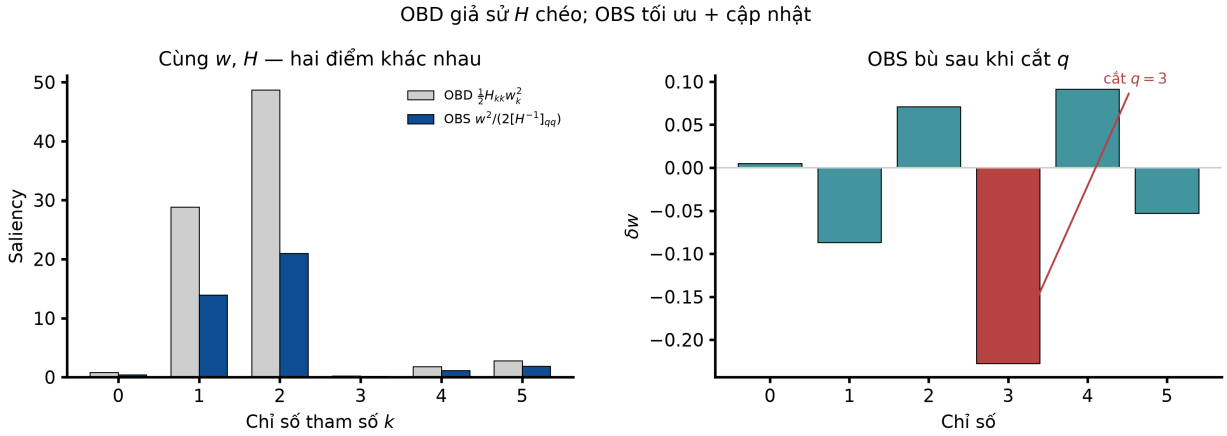
2. Ý tưởng chính

Hai phương pháp xây dựng mô hình bậc hai cục bộ của hàm lỗi quanh nghiệm đã huấn luyện, rồi dùng mô hình đó dự đoán mức tăng lỗi khi xóa một trọng số. OBD giả sử Hessian là ma trận **đường chéo** nên saliency $= \frac{1}{2} h_{kk} u_k^2$, xóa các tham số saliency thấp, rồi huấn luyện lại, lặp. OBS không giả sử gì về dạng

Hessian: giải bài toán tối ưu có ràng buộc "trọng số q bằng 0" bằng nhân tử Lagrange, thu được cả (i) saliency $w_q^2/(2[H^{-1}]_{qq})$ và (ii) công thức cập nhật mọi trọng số còn lại để bù, **không cần huấn luyện lại bằng gradient**. Khó khăn thực tế của OBS là tính H^{-1}, giải quyết bằng một công thức đệ quy duyệt dữ liệu một lần.

---

3. Phương pháp chi tiết: OBD



OBD $\frac{1}{2}H_{kk}w_k^2$ vs OBS $w^2/(2[H^{-1}]_{qq})$ và vector bù δw. (minh hoạ, tự tính)

3.1 Khai triển Taylor và ba xấp xỉ

Nhiễu loạn δU của vector tham số làm hàm mục tiêu E thay đổi:

$$\delta E = \sum_i g_i\,\delta u_i + \tfrac{1}{2}\sum_i h_{ii}\,\delta u_i^2 + \tfrac{1}{2}\sum_{i\neq j} h_{ij}\,\delta u_i \delta u_j + O(\|\delta U\|^3),$$

với $g_i = \partial E/\partial u_i$ và $h_{ij} = \partial^2 E/\partial u_i \partial u_j$. Ma trận H rất lớn (paper nêu 6.5×10^6 phần tử cho mạng 2600 tham số) nên cần ba xấp xỉ mà paper đặt tên:

Xấp xỉ	Nội dung	Hệ quả
Diagonal	Bỏ các số hạng chéo $h_{ij}, i \neq j$	δE khi xóa nhiều tham số = tổng δE khi xóa từng tham số
Extremal	Xóa sau khi đã hội tụ → ở cực tiểu địa phương	Bỏ số hạng bậc nhất $g_i\delta u_i$; mọi $h_{ii} \geq 0$
Quadratic	Hàm mục tiêu gần bậc hai	Bỏ số hạng bậc ba trở lên

Kết quả:

$$\boxed{\delta E \;\approx\; \tfrac{1}{2}\sum_i h_{ii}\,\delta u_i^2} \qquad \Rightarrow \qquad \boxed{s_k = \tfrac{1}{2}\,h_{kk}\,u_k^2}$$

(xóa tham số k tương ứng $\delta u_k = -u_k$).

3.2 Tính đường chéo Hessian bằng lan truyền ngược

Paper dùng hàm MSE; kết quả áp dụng cho mỗi mẫu rồi lấy trung bình. Với mạng có trạng thái $z_i = f(a_i)$, $a_i = \sum_j w_{ij} z_j$, và tham số có thể **chia sẻ** ($w_{ij} = u_k$ cho mọi $(i,j) \in V_k$):

$$h_{kk} = \sum_{(i,j)\in V_k} \frac{\partial^2 E}{\partial w_{ij}^2}, \qquad \frac{\partial^2 E}{\partial w_{ij}^2} = \frac{\partial^2 E}{\partial a_i^2} z_j^2.$$

Đạo hàm bậc hai theo a_i được lan truyền ngược từng tầng, với điều kiện biên ở tầng ra (công thức (8) trong paper):

$$\frac{\partial^2 E}{\partial a_i^2} = 2f'(a_i)^2 - 2(d_i - z_i)f''(a_i).$$

Chi phí tính đường chéo **cùng bậc với tính gradient**. Nếu bỏ số hạng thứ hai (chứa f'') ta được xấp xỉ Levenberg–Marquardt, đảm bảo ước lượng dương.

3.3 Công thức (recipe) của OBD

```
1. Chọn kiến trúc hợp lý
2. Huấn luyện đến khi có nghiệm tốt
3. Tính h_kk cho mọi tham số   (1 lượt lan truyền ngược bậc hai)
4. Tính saliency s_k = h_kk * u_k^2 / 2
5. Sắp xếp theo s_k, xóa (đặt = 0 và đóng băng) các tham số saliency thấp
6. Quay lại bước 2 (huấn luyện lại), lặp
```

Paper nêu các biến thể: giảm dần giá trị thay vì đặt 0, hoặc cho tham số đã xóa được thích nghi lại. "Đơn vị đếm" là *tham số tự do*, không phải kết nối, vì trong mạng ràng buộc một tham số có thể điều khiển nhiều kết nối.

---

4. Phương pháp chi tiết: OBS

4.1 Bài toán có ràng buộc và lời giải

Xuất phát từ cùng khai triển Taylor: ở cực tiểu, bỏ số hạng bậc nhất và bậc ba trở lên, $\delta E = \frac{1}{2}\,\delta w^\top H\,\delta w$. Xóa trọng số w_q là ràng buộc $e_q^\top \delta w + w_q = 0$ (e_q là vector đơn vị). Bài toán (paper dùng nhãn Eq. 3):

$$\min_q \left\{ \min_{\delta w} \ \tfrac{1}{2}\,\delta w^\top H\,\delta w \ \text{ s.t. } \ e_q^\top \delta w + w_q = 0 \right\}.$$

Lagrangian $L = \frac{1}{2}\delta w^\top H \delta w + \lambda(e_q^\top \delta w + w_q)$. Đạo hàm theo δw bằng 0 cho $\delta w = -\lambda H^{-1} e_q$; thế vào ràng buộc ra $\lambda = w_q / [H^{-1}]_{qq}$. Do đó:

$$\boxed{\delta w = -\frac{w_q}{[H^{-1}]_{qq}} H^{-1} e_q, \qquad L_q = \frac{1}{2}\frac{w_q^2}{[H^{-1}]_{qq}}}$$

L_q là **saliency** của trọng số q. Paper nhấn mạnh: không cần H hay H^{-1} chéo; mọi trọng số khác được tính lại bằng δw; và định nghĩa của OBD là trường hợp đặc biệt khi H chéo (khi đó $[H^{-1}]_{qq} = 1/h_{qq}$). Cũng theo paper, magnitude pruning ứng với giả sử $H \propto I$ (đẳng hướng), OBD ứng với H chéo, và FARM (Kung & Hu) giả sử f tuyến tính, chỉ cập nhật trọng số tầng ra-ẩn.

```
OBS procedure
1. Huấn luyện mạng "đủ lớn" đến sai số cực tiểu
2. Tính H^{-1}
3. Tìm q có L_q = w_q^2 / (2 [H^{-1}]_qq) nhỏ nhất.
   Nếu L_q << E: xóa, sang bước 4; ngược lại sang bước 5
```

```
4. Cập nhật mọi trọng số bằng δw (Eq. 5). Quay lại bước 2
5. Không xóa thêm được nữa. (Có thể huấn luyện lại)
```

Hình 1 trong paper mô tả bất đẳng thức bậc hai $E(\text{mag}) \gtrsim E(\text{OBD}) \gtrsim E(\text{OBS})$: sau khi xóa (trước retrain), OBS có sai số thấp nhất vì ngoài việc chọn đúng trọng số còn bù các trọng số khác theo hướng cong thấp.

4.2 Tính H^{-1} bằng đệ quy

Với mạng một đầu ra, $F(w, \text{in})$, sai số bình phương trung bình, bỏ số hạng $(t-o)$ gắn với đạo hàm bậc hai của F (phần dư):

$$H = \frac{1}{P}\sum_{k=1}^{P} X^{[k]} X^{[k]\top}, \qquad X^{[k]} = \frac{\partial F(w, \text{in}^{[k]})}{\partial w}.$$

Tức H là **ma trận hiệp phương sai mẫu** của vector gradient X (Eq. 12). Dựng dần: $H_{m+1} = H_m + \frac{1}{P} X^{[m+1]} X^{[m+1]\top}$, $H_0 = \alpha I$. Áp dụng công thức nghịch đảo ma trận (Woodbury / Sherman–Morrison) cho từng số hạng:

$$H_{m+1}^{-1} = H_m^{-1} - \frac{H_m^{-1} X^{[m+1]} X^{[m+1]\top} H_m^{-1}}{P + X^{[m+1]\top} H_m^{-1} X^{[m+1]}}, \qquad H_0^{-1} = \alpha^{-1} I,\ H_P^{-1} = H^{-1}.$$

Hằng α nhỏ (paper nêu khoảng $10^{-8} \le \alpha \le 10^{-4}$) làm H_0^{-1} có nghĩa; việc này tương đương nghịch đảo của $H + \alpha I$, tức một phạt $\alpha\|\delta w\|^2$ ngăn bước nhảy lớn và giúp bỏ số hạng bậc cao hợp lệ; paper nói phương pháp *không nhạy* với α. Một lượt duyệt dữ liệu, chi phí $O(P\,n^2)$ với n là số tham số. Trường hợp nhiều đầu ra thì có đệ quy theo cả hai chỉ số.

4.3 Vì sao được phép bỏ số hạng $(t-o)$

Paper đưa lý do tính toán và lý do chức năng:

- Tính toán: H đầy đủ thường suy biến, đặc biệt trước khi cắt; xấp xỉ này bảo đảm không suy biến kỳ dị và giữ chi phí $O(Pn^2)$. Trong thống kê đây là cơ sở của *Fisher's method of scoring* (thay Hessian thật bằng kỳ vọng).
- Chức năng: nếu giữ số hạng $(t-o)$, mỗi bước cắt sẽ "đưa nhiễu trở lại" vì phạt cả các thành phần nhiễu. Paper lập luận rằng sau khi cắt ta muốn ở gần giá trị sai số mới, như thể có một tín hiệu dạy hiệu dụng t^* sao cho $(t^* - o) \approx 0$.

4.4 Ví dụ nhỏ (hai trọng số)

Giả sử $H = \begin{pmatrix} 2 & 1.9 \\ 1.9 & 2 \end{pmatrix}$, $w = (1,\ 1)$. Đường chéo bằng nhau nên OBD cho hai saliency bằng nhau ($\frac{1}{2}\cdot 2 \cdot 1 = 1$). Tính $H^{-1} = \frac{1}{2^2 - 1.9^2}\begin{pmatrix} 2 & -1.9 \\ -1.9 & 2 \end{pmatrix} \approx \frac{1}{0.39}\begin{pmatrix} 2 & -1.9 \\ -1.9 & 2 \end{pmatrix}$, nên $[H^{-1}]_{11} \approx 5.13$ và $L_1 = \dfrac{1}{2 \cdot 5.13} \approx 0.097$ — nhỏ hơn rất nhiều so với OBD. Và $\delta w = -\frac{1}{5.13} H^{-1} e_1 = (-1,\ +0.95)$: sau khi xóa w_1, trọng số w_2 tăng gần gấp đôi để bù, vì hai đặc trưng gần như đồng hướng. Đây là hình ảnh trực quan của "off-diagonal quan trọng": OBD đánh giá quá cao cái giá phải trả (nó không thấy khả năng bù). *(Ví dụ số tự dựng cho minh họa, không có trong paper.)*

---

5. Thí nghiệm và kết quả chính

5.1 OBD (NIPS 1989)

- Mạng: nhận dạng chữ số viết tay zip code, ban đầu **kết nối thưa với 10^5 kết nối do 2578 tham số tự do điều khiển**; huấn luyện trên khoảng 9300 mẫu train và 3350 mẫu test (chữ số viết tay và chữ số in).
- Hình 1a: xóa theo saliency làm hàm mục tiêu tăng **ít hơn đáng kể** so với xóa theo độ lớn; xóa ngẫu nhiên tệ đến mức không vẽ cùng thang. Hình 1b: dự đoán (tổng saliency) khớp thực tế **đến khoảng 800 tham số bị xóa (~30%)**; sau đó đường cong tách ra vì các số hạng ngoài chéo và số hạng bậc cao trở nên quan trọng.
- Hình 2: sau khi **huấn luyện lại**, sai số train và test hầu như không đổi **khi xóa đến 1500 tham số (60%)**.
- Dùng như công cụ thiết kế: với biểu đồ saliency của mạng 10 000 tham số, xóa nguyên tầng áp chót giảm số tham số **một nửa**; MSE train tăng 10 lần, MSE tổng quát hóa chỉ tăng 50%, và **sai số phân loại 10 lớp trên test thực sự giảm**.
- Kết luận của bài: dùng OBD tương tác giảm số tham số **hệ số 4**, và xóa tự động thêm **hơn hệ số 2**; tốc độ mạng tăng đáng kể, độ chính xác nhận dạng tăng nhẹ. Tác giả nhấn mạnh xuất phát từ mạng hiện đại tốt (state-of-the-art), không phải mạng dở.

5.2 OBS (NIPS 1992)

Thí nghiệm	Kết quả theo paper
XOR 2-2-1 (9 trọng số)	Chỉ OBS xóa đúng trọng số trong mọi trường hợp huấn luyện từ các khởi tạo khác nhau; magnitude và OBD đôi khi xóa sai và **huấn luyện lại không cứu được** (không đưa sai số về 0). OBS đạt sai số 0 không cần retrain.
MONK's 1	BPWD (backprop + weight decay, Thrun et al.) 58 trọng số, OBS 14 trọng số, cùng 100%/100%
MONK's 2	BPWD 39, OBS 15, cùng 100%/100%
MONK's 3	BPWD 39, OBS 4, cùng 93.4% train / 97.2% test
NETtalk	Khởi đầu 5546 trọng số (test error 5259); OBS xuống 2438, retrain, cắt tiếp còn **1560 trọng số với test error 4701**. Sejnowski & Rosenberg dùng 18 000 trọng số

Paper tóm tắt: OBS cần chỉ **24%, 38% và 10%** số trọng số của mạng BPWD cho MONK's 1, 2, 3 với cùng hiệu năng (bản tóm tắt ghi giảm 90%, 76%, 62%), *không retrain*. Với MONK's 1, mạng sau OBS đơn giản đến mức có thể khôi phục quy tắc logic sinh dữ liệu.

---

6. Phân tích / ablation

- **Hessian không chéo** (OBS): tác giả "ngạc nhiên" nhận thấy Hessian của **mọi bài toán họ thử, kể cả XOR và MONK's, đều rất không chéo**; nhiều phần tử ngoài chéo ngang bằng phần tử chéo. Đây là lời giải thích của họ vì sao OBD đôi khi xóa sai.
- **Magnitude có thể xóa sai trọng số mà retrain không sửa được** (Hình 1, 3, 5): ví dụ XOR, magnitude xóa v_3, OBD xóa u_{22}, chỉ OBS xóa u_{23} — và chỉ lựa chọn đó cho phép mạng giải được XOR. Họ phản bác ý "mạng sẽ tự sắp xếp lại sau retrain", nhất là khi đã cắt nhiều.
- **Số hạng** $(t - o) \to 0$ (OBS): lý do chức năng/tính toán ở mục 4.3.
- **OBD: khớp dự đoán đến ~30%** rồi lệch (mục 5.1), cho thấy phạm vi hiệu lực của xấp xỉ chéo + bậc hai.
- **Phụ thuộc retrain**: OBD luôn kèm retrain; OBS "không đòi hỏi retrain (thường chậm) sau khi cắt một trọng số", dù tác giả vẫn nói có thể muốn retrain ở cuối.

- OBS còn có thể tổng quát: xóa nhiều trọng số cùng lúc, miễn trừ bias, chiếu lên mặt phẳng khác ($w_q =$ hằng số), và dùng cross-entropy/KL → ma trận thông tin Fisher thay Hessian (nêu nhưng chi tiết ở bài đang chuẩn bị của họ).

7. Giới hạn

Do tác giả nêu:

- OBD: tại sao dự đoán lệch sau ~30% (off-diagonal, bậc cao); đếm "số tham số tự do" chỉ là thước đo phức tạp *không hoàn hảo*.
- OBS: tính H^{-1} cho mạng nghìn–triệu tham số trông "bất khả thi" nếu làm trực tiếp; họ dùng đệ quy. OBS không tự có tiêu chí dừng.
- Cả hai đều dựa vào giả thiết đã hội tụ về cực tiểu địa phương.

Nhận xét:

- Quy mô: thử nghiệm trên mạng $10^{3–10}4$ tham số. Chi phí $O(Pn^2)$ bộ nhớ và thời gian không nhân được lên triệu hay tỷ tham số — đó là lý do 22–23 phải làm xấp xỉ theo từng hàng/tầng.
- Chỉ cắt *tham số lẻ* (unstructured); không bàn tốc độ trên phần cứng. Trong thời đại đó việc giảm kết nối còn "cải thiện tốc độ" ở mức nhận định.
- MSE và mạng nhỏ; chưa có BatchNorm, residual, attention; Hessian của mạng sâu hiện đại có tính chất khác (đa cực tiểu, phẳng, không hội tụ hẳn).
- So sánh với magnitude dựa trên một mạng; chưa có nhiều seed.
- Dự đoán sai khi loss không còn gần bậc hai (xóa trọng số lớn).

8. Liên hệ với các bài khác

- **02 Han 2015**: trích dẫn OBD/OBS, nói tiêu chí bậc hai chính xác hơn magnitude *nhưng cần tính toán bổ sung*, rồi chọn magnitude + retrain lặp. Đây là nước đi thực dụng thắng thế suốt nhiều năm.
- **03 Zhu & Gupta**: nhắc lại OBD/OBS trong related work; chọn magnitude + lịch cắt dần.
- **04 Li et al.**: trích OBD/OBS như tiền đề của "đánh giá trọng số theo saliency".
- **07 Molchanov 2019**: Taylor bậc nhất/bậc hai cho cấu trúc, nối trực tiếp với khai triển ở mục 3.1.
- **22 OBC → 23 SparseGPT → 27 LLM Surgeon**: hồi sinh OBS: xấp xỉ $H = XX^\top$ theo từng tầng (đây cũng là dạng Gauss–Newton bên trên) và cập nhật trọng số còn lại bằng công thức δw gần như y hệt; 27 mở rộng bằng xấp xỉ Kronecker (K-FAC) cho cắt có cấu trúc.
- **24 Wanda**: có thể xem là OBD/OBS với Hessian chỉ giữ đường chéo $\propto \|X_j\|^2$ và bỏ bù trọng số (cách diễn giải của người soạn; hãy đọc Wanda để xác nhận).
- **08 Lottery Ticket, 11, 12**: nhìn lại câu hỏi "retrain sau khi xóa sai trọng số có cứu được không?" — OBS nói *không phải lúc nào cũng cứu được*.

9. Gợi ý áp dụng

- **Khuôn mẫu tư duy quan trọng nhất**: chi phí cắt không chỉ là "mất trọng số này", mà là "mất trọng số này *sau khi các phần còn lại đã bù tối ưu*".
- **Nhận xét** có thể được hiểu theo ngôn ngữ bài này: nếu các neuron MLP có Hessian/hiệp phương sai mạnh ngoài chéo (đặc trưng tương quan), tiêu chí chéo/magnitude chọn sai, còn cắt cả layer ít bị lỗi chọn sai nhưng lại mất cả khối. Cần kiểm chứng bằng thí nghiệm.

10. Thí nghiệm gợi ý

2. **So sánh OBD-giống (saliency chéo) và OBS-giống (có bù)** cho việc cắt k neuron MLP của một block: dùng H^{-1} của $XX^\top + \lambda I$; đo lỗi tái tạo đầu ra của block và sau đó loss KD. Mục tiêu: đo xem bù tuyến tính (OBS) lấy lại bao nhiêu phần trăm tổn hại trước khi fine-tune.

3. **Kiểm tra vùng hiệu lực của dự đoán**: vẽ "tổng saliency dự đoán" so với tăng loss thực khi cắt 5%, 10%, 20%, 30% neuron, giống Hình 1b của OBD, để biết ngưỡng mà xấp xỉ bậc hai hỏng.

11. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / ý	Nguồn
Taylor bậc hai	$\delta E \approx g^\top \delta u + \frac{1}{2}\delta u^\top H \delta u$	OBD Eq. 1, OBS Eq. 1
Saliency OBD	$s_k = \frac{1}{2} h_{kk} u_k^2$	OBD
Đường chéo Hessian	$h_{kk} = \sum_{(i,j)\in V_k} \frac{\partial^2 E}{\partial a_i^2} z_j^2$	OBD Eq. 5–6
Điều kiện biên	$\frac{\partial^2 E}{\partial a_i^2} = 2f'^2 - 2(d_i - z_i)f''$	OBD Eq. 8
Saliency OBS	$L_q = \dfrac{w_q^2}{2[H^{-1}]_{qq}}$	OBS Eq. 5
Cập nhật OBS	$\delta w = -\dfrac{w_q}{[H^{-1}]_{qq}} H^{-1} e_q$	OBS Eq. 5
Hessian ≈ hiệp phương sai	$H = \frac{1}{P}\sum_k X^{[k]} X^{[k]\top}$	OBS Eq. 12
Đệ quy nghịch đảo	$H_{m+1}^{-1} = H_m^{-1} - \dfrac{H_m^{-1} X X^\top H_m^{-1}}{P + X^\top H_m^{-1} X}$	OBS Eq. 17
Magnitude / OBD / OBS	giả sử $H \propto I$ / H chéo / H tổng quát	OBS mục 7

12. Kết luận ngắn

OBD đánh giá trọng số bằng tích của độ cong và bình phương độ lớn; OBS dùng Hessian đầy đủ để chọn trọng số cần cắt và điều chỉnh các trọng số còn lại. Hai bài đưa ra đúng hai câu hỏi mà toàn bộ lĩnh vực vẫn lặp lại: *tiêu chí nào*, và *phục hồi bằng cách nào*. Điểm yếu là chi phí tính Hessian — thứ mà các bài sau (02 magnitude, 22–23 xấp xỉ theo tầng) đã khắc phục theo hai hướng khác nhau.

Nguồn gốc:

- OBD: https://proceedings.neurips.cc/paper_files/paper/1989/hash/6c9882bbac1c7093bd25041881277658-Abstract.html
- OBS: https://proceedings.neurips.cc/paper_files/paper/1992/hash/303ed4c69846ab36c2904d3ba8573050-Abstract.html

Bài 02

Learning both Weights and Connections for Efficient Neural Networks (2015)

Tác giả: Song Han, Jeff Pool, John Tran, William J. Dally
Venue / arXiv: arXiv:1506.02626 (bản v3, 30/10/2015). Bản arXiv không ghi venue; bài của Li et al. (04) trích dẫn là NIPS 2015.
Nguồn: https://arxiv.org/abs/1506.02626
Chủ đề: Pruning trọng số theo độ lớn (magnitude), quy trình train → prune → retrain, lặp nhiều vòng; nén AlexNet 9× và VGG-16 13×.
Vị trí trong lộ trình: Đọc sau 01 (OBD/OBS: tiêu chí bậc hai). Bài này là **baseline magnitude pruning kinh điển** mà gần như mọi bài sau so sánh. Tiếp theo: 03 (cắt dần trong lúc train, so sánh với mô hình nhỏ dense), 04 (cắt filter có cấu trúc để có tốc độ thật).
Chương nền liên quan: nền 01: Vì sao mạng cắt được, nền 02: Ma trận thưa và chi phí thật, nền 03: Độ quan trọng của trọng số, nền 05: Cắt rồi huấn luyện lại

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ bản arXiv đã đọc.

1. Bối cảnh

Đến 2012–2015 mạng tích chập lớn rất nhanh: LeNet-5 dưới 1M tham số (1998), AlexNet 60M (2012), Deepface 120M, mạng của Coates và cộng sự 10B. Bài báo lập luận chi phí chính của mạng lớn **không phải phép nhân mà là truy cập bộ nhớ**. Theo bảng năng lượng 45nm CMOS (Horowitz) trong bài báo:

Phép toán	Năng lượng (pJ)	Tương đối
32-bit int ADD	0.1	1
32-bit float ADD	0.9	9
32-bit int MULT	3.1	31
32-bit float MULT	3.7	37
32-bit SRAM (cache)	5	50
32-bit DRAM	640	6400

Ví dụ trong bài báo: mạng 10^9 kết nối chạy 20 Hz cần $(20)(10^9)(640\,\text{pJ}) = 12.8$ W chỉ cho DRAM, vượt khả năng thiết bị di động. Mục tiêu: nhét trọng số vào bộ nhớ trên chip (SRAM) bằng cách bớt kết nối.

Trước đó đã có hướng nén khác (độc lập, có thể kết hợp): lượng tử hóa số nguyên 8 bit (Vanhoucke), xấp xỉ hạng thấp (Denton), vector quantization (Gong), HashedNets. Về pruning, OBD/OBS (01) dùng Hessian nhưng "second-order derivative needs additional computation". Han et al. chọn một tiêu chí **rẻ**: độ lớn trọng số.

Thuật ngữ nhanh

- **Pruning kết nối (connection pruning):** đặt các trọng số về 0 và bỏ chúng; ở đây mỗi trọng số là một "synapse".
- **Sparse (thưa):** tỉ lệ lớn các trọng số bằng 0.
- **Fully connected (FC):** tầng nối đầy đủ, chứa phần lớn tham số của AlexNet/VGG.

2. Ý tưởng chính

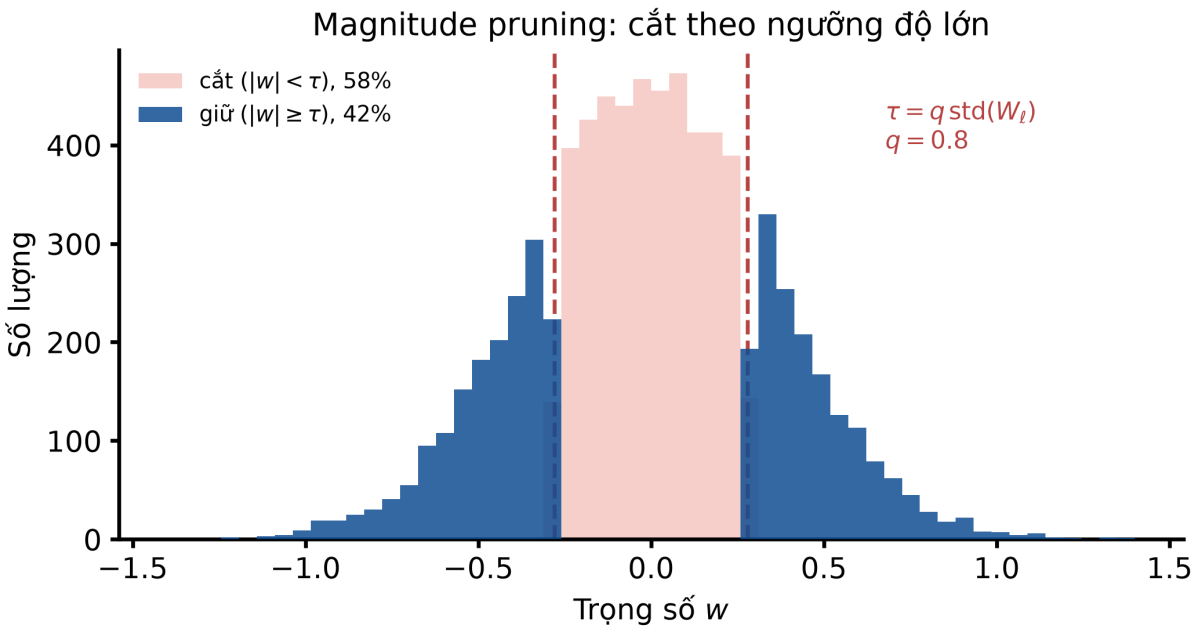
Huấn luyện mạng bình thường để biết *kết nối nào quan trọng*, rồi **bỏ mọi kết nối có |trọng số| dưới một ngưỡng**, rồi **huấn luyện lại phần còn lại** để chúng bù đắp. Có thể lặp "cắt – retrain" nhiều vòng. Quy trình này làm cho việc huấn luyện học **cả cấu trúc kết nối lẫn giá trị trọng số**, tương tự quá trình cắt tỉa synapse ở não trẻ em. Kết quả: AlexNet giảm 9× tham số (61M → 6.7M) và VGG-16 giảm 13× (138M → 10.3M) mà không mất độ chính xác. Bước huấn luyện lại là then chốt: không có nó độ chính xác tụt rõ.

---

3. Phương pháp chi tiết

3.1 Quy trình ba bước

```
1. Train connectivity : huấn luyện mạng đầy đủ bình thường (học xem kết nối nào quan
↪  trọng)
2. Prune connections  : với mỗi tầng, loại mọi trọng số có |w| < ngưỡng (đặt mask = 0)
3. Train weights      : retrain mạng thưa, giữ nguyên các trọng số sống sót làm khởi tạo
(lặp 2–3 nhiều vòng = iterative pruning)
```



Histogram trọng số và ngưỡng magnitude $\tau = q\,\mathrm{std}(W_\ell)$: mọi $|w| < \tau$ bị cắt trước khi retrain. (minh hoạ, tự tính)

Hiện thực trong Caffe: mỗi tensor trọng số có một **mask** nhị phân; trọng số bị mask bị bỏ qua khi forward/backward. **Ngưỡng của mỗi tầng = một "tham số chất lượng" nhân với độ lệch chuẩn của trọng số tầng đó.**

$$\mathrm{mask}_{ij} = \mathbb{1}\big[\,|w_{ij}| \;\geq\; \tau_\ell\,\big], \qquad \tau_\ell = q \cdot \mathrm{std}(W_\ell).$$

(Ký hiệu q, τ_ℓ là cách viết của ghi chú này; paper chỉ mô tả bằng lời.)

3.2 Regularization: L1 hay L2?

- **L1** đẩy nhiều tham số về gần 0 → độ chính xác **ngay sau khi cắt, chưa retrain** tốt hơn. Nhưng các kết nối còn lại *kém hơn* sau retrain.

- **L2** cho kết quả cuối **tốt nhất sau retrain**. Paper kết luận L2 tổng thể tốt nhất. Thử "L1 để cắt, L2 để retrain" không vượt L2 cả hai pha: tham số từ chế độ này không thích nghi tốt với chế độ kia.

3.3 Điều chỉnh dropout khi retrain

Dropout tắt tham số ngẫu nhiên nhưng chúng quay lại lúc suy luận; pruning thì loại vĩnh viễn → dung lượng giảm → **tỉ lệ dropout lúc retrain nên giảm**. Gọi C_i là số kết nối tầng i, N_i số neuron ($C_i = N_i N_{i-1}$), C_{io} và C_{ir} trước / sau cắt:

$$\boxed{D_r = D_o \sqrt{\frac{C_{ir}}{C_{io}}}}$$

Lý do (theo paper): dropout hoạt động trên neuron, C_i tỉ lệ bình phương với N_i, nên tỉ lệ dropout theo căn.

3.4 Giữ trọng số cũ khi retrain (co-adaptation) và retrain cục bộ

Giữ giá trị trọng số sống sót tốt hơn khởi tạo lại: CNN có các đặc trưng "co-adapted mong manh" (Yosinski et al.). Paper còn **khóa các tầng CONV khi retrain FC và ngược lại**, vì mạng sâu dễ gặp vanishing gradient khiến sai sót cắt khó phục hồi, và đỡ tốn chi phí lan truyền ngược qua toàn mạng.

3.5 Iterative pruning

Một vòng cắt + retrain = một lần lặp, mỗi lần là tìm kiếm tham lam. **Nâng tỉ lệ cắt từ 5× lên 9× trên AlexNet không mất độ chính xác** so với cắt mạnh một lần. Cắt theo xác suất dựa trên |w| được thử nhưng cho kết quả tệ hơn.

3.6 Cắt neuron là hệ quả tự nhiên

Sau khi cắt kết nối, neuron không còn kết nối vào hoặc ra có thể bỏ; retrain tự dẫn đến neuron "chết" có cả hai phía bằng 0, nhờ gradient bằng 0 và regularization kéo trọng số còn lại về 0. Đây là cách duy nhất phương pháp này có được cấu trúc, và nó không được điều khiển chủ động (nhận xét của ghi chú, không phải của paper).

3.7 Độ nhạy theo tầng

Paper cắt riêng từng tầng để đo độ nhạy và đặt ngưỡng theo đó: tầng CONV nhạy hơn FC; **tầng conv đầu tiên nhạy nhất** (suy đoán của tác giả: chỉ có 3 kênh vào nên ít dư thừa). Ngưỡng nhỏ nhất áp cho tầng nhạy nhất.

---

4. Thí nghiệm và kết quả chính

Bốn mạng: LeNet-300-100 và LeNet-5 (MNIST), AlexNet và VGG-16 (ImageNet ILSVRC-2012). Mô hình tham chiếu lấy từ Caffe model zoo, độ chính xác đo không augmentation. GPU: Titan X và GTX980.

Bảng 1 (tóm tắt):

Mạng	Top-1 err	Top-5 err	Tham số	Tỉ lệ nén
LeNet-300-100 ref	1.64%	-	267K	
LeNet-300-100 pruned	1.59%	-	22K	12×
LeNet-5 ref	0.80%	-	431K	
LeNet-5 pruned	0.77%	-	36K	12×
AlexNet ref	42.78%	19.73%	61M	

Mạng	Top-1 err	Top-5 err	Tham số	Tỉ lệ nén
AlexNet pruned	42.77%	19.67%	6.7M	9×
VGG-16 ref	31.50%	11.32%	138M	
VGG-16 pruned	31.34%	10.88%	10.3M	13×

LeNet. Sau cắt, retrain với 1/10 learning rate gốc. LeNet-300-100: trọng số còn 8%, FLOP còn 8% (giảm 12×). LeNet-5: trọng số 8%, FLOP thực còn 16% (tính cả activation thưa; giảm 6×). Một quan sát thú vị: sparsity pattern của lớp FC1 LeNet-300-100 (784×300) có **28 dải** tương ứng 28×28 pixel, cho thấy chỉ trọng số ở vùng giữa ảnh còn lại (mạng tự tìm "vùng chú ý").

AlexNet. 61M tham số, 1.5B FLOP; còn **11% trọng số, 30% FLOP** (3× ít tính toán). Retrain với **1/100 learning rate khởi điểm**. Thời gian: huấn luyện gốc 75 giờ trên Titan X, **retrain 173 giờ**. Phần trăm trọng số giữ theo tầng (Bảng 4): conv1 84%, conv2 38%, conv3 35%, conv4 37%, conv5 37%, fc1 9%, fc2 9%, fc3 25%. Paper lập luận retrain lâu ít quan trọng vì pruning là bước cuối trước triển khai.

VGG-16. 138M tham số, 30.9B FLOP; còn **7.5% trọng số, 21% FLOP**, dùng 5 vòng cắt–retrain. Hai tầng FC lớn nhất (fc6 103M, fc7 17M) được cắt còn khoảng **4%** trọng số. Điều này quan trọng cho xử lý ảnh thời gian thực khi FC ít được tái sử dụng giữa các ảnh (không như xử lý theo lô lúc huấn luyện).

So sánh với phương pháp khác trên AlexNet (Bảng 6):

Phương pháp	Top-1 err	Tham số	Nén
Baseline Caffe	42.78%	61.0M	1×
Data-free pruning	44.40%	39.6M	1.5×
Fastfood-32-AD	41.93%	32.8M	2×
Fastfood-16-AD	42.90%	16.4M	3.7×
Collins & Kohli	44.40%	15.2M	4×
Naive cut (giảm kích thước tầng)	47.18%	13.8M	4.4×
SVD (Denton et al.)	44.02%	11.9M	5×
Network pruning	**42.77%**	**6.7M**	**9×**

Lưu trữ thưa: chi phí phụ chỉ khoảng **15.6%**; dùng chỉ số tương đối nên chỉ số FC chỉ cần 5 bit, chỉ số CONV 8 bit.

---

5. Phân tích / ablation

Hình 5 (đường cong: tổn thất top-5 theo % tham số bị cắt, năm cấu hình L1/L2, có/không retrain, lặp):

- **Retrain quan trọng:** không retrain, độ chính xác bắt đầu tụt khi còn khoảng **1/3** số kết nối; có retrain thì mãi đến khi còn khoảng **1/10**. Không retrain vẫn có "bữa trưa miễn phí" **2×** ít kết nối mà không mất độ chính xác; có retrain là 9×.
- **Iterative pruning cho lợi ích lớn nhất:** không mất độ chính xác ở 9×; từ 10× độ chính xác bắt đầu tụt mạnh.
- **L1 vs L2:** L1 tốt hơn lúc chưa retrain, L2 tốt hơn sau retrain (mục 3.2).
- Hai điểm trên đường cong thậm chí **tốt hơn mô hình gốc một chút**; tác giả cho rằng pruning tìm được dung lượng phù hợp và giảm overfitting (đây là giả thuyết của họ, không kiểm chứng riêng).
- **Phân phối trọng số (Hình 7, FC1 của AlexNet):** trước cắt tập trung quanh 0 (hầu hết trong [−0.015, 0.015]); sau cắt và retrain thành **phân phối hai đỉnh** trải rộng hơn ([−0.025, 0.025]).

- **Độ nhạy theo tầng (Hình 6):** CONV nhạy hơn FC, conv1 nhạy nhất.

6. Giới hạn

Do tác giả nêu:

- Phần cứng đa dụng kém trong tính toán thưa; họ nhắm tới **phần cứng chuyên dụng cho DNN thưa** ("fixed-function hardware specialized for sparse DNN").
- Retrain tốn thời gian (173 giờ cho AlexNet), nhưng họ coi là chấp nhận được.
- Sparse storage tốn thêm chỉ số (15.6%).
- Độ giảm FLOP thấp hơn độ giảm tham số (3× vs 9× với AlexNet) vì FC chiếm phần lớn tham số nhưng ít FLOP.

Nhận xét:

- Cắt **không cấu trúc**: kết quả là ma trận cùng kích thước chứa nhiều 0. Trên GPU đa dụng, không tự nhanh hơn (xem preface mục 7; và 04 phản biện đúng điểm này). Paper không báo cáo thời gian suy luận thật.
- Chỉ đo trên CNN phân loại ảnh, 2015; chưa có BatchNorm/residual; kết luận "tầng FC dư thừa nhiều" phản ánh đặc điểm AlexNet/VGG chứ không phải mọi mạng (xem 03 với MobileNet, 04 với ResNet).
- So sánh baseline "naive cut" là một mạng giảm tầng nhưng có thể không được tối ưu hóa siêu tham số cùng mức (về sau 03 và 11, 12 xem lại vấn đề này).
- Cách chọn ngưỡng theo std và độ nhạy từng tầng là thủ công, không có cơ chế tự động.
- Số ngày huấn luyện không so sánh công bằng với các phương pháp khác.

7. Liên hệ với các bài khác

- **01 OBD/OBS:** tiêu chí bậc hai chính xác hơn nhưng đắt; Han chọn magnitude. Bài này kiểm chứng "magnitude + retrain đủ tốt" ở quy mô lớn.
- **03 Zhu & Gupta:** thay lịch "cắt một lần rồi retrain" bằng **lịch cắt dần trong lúc train**, và hỏi câu hỏi mà Han chưa hỏi: mô hình lớn thưa có hơn mô hình nhỏ dense cùng bộ nhớ không?
- **04 Li et al.:** chỉ ra rằng cắt trọng số theo Han giảm tham số ở FC nhưng không giảm tính toán conv và cần thư viện thưa → chuyển sang cắt filter.
- **05 Network Slimming:** cùng bàn về nhược điểm cần phần cứng/thư viện đặc biệt của weight pruning.
- **08 Lottery Ticket, 11, 12:** các bài sau dùng magnitude pruning kiểu này làm nền; 11 nghi ngờ vai trò của trọng số kế thừa; 12 tổng hợp việc so sánh.
- **18 (2:4 sparsity)**: phần cứng NVIDIA hỗ trợ một dạng thưa đều đặn, giải quyết một phần mơ ước "phần cứng cho DNN thưa" ở đây.
- **24 Wanda:** magnitude có điều chỉnh bằng activation; ngược dòng của nó là tiêu chí trong bài này.

8. Gợi ý áp dụng

- **Điều đáng giữ lại:**
  1. Quy trình *train → prune → retrain (lặp)* và nguyên tắc **giữ lại trọng số sống sót** làm khởi tạo thay vì khởi tạo lại.
  2. Quan sát: **learning rate khi retrain nhỏ** (1/10–1/100) so với lúc pretrain.
  3. Tinh thần "đo bộ nhớ ở mức byte" thay vì FLOPs.

9. Thí nghiệm gợi ý

2. **Magnitude vs random vs "activation-aware" cho MLP neuron**: cắt 30% neuron theo chuẩn ℓ_1 của hàng W_{in}, ngẫu nhiên, và bằng $|w| \cdot \|x\|$; so loss trước và sau 500 bước fine-tune. Kiểm chứng xem tiêu chí đơn giản có đủ cho cắt có cấu trúc không.

3. **Iterative vs one-shot ở cùng tổng ngân sách bước**: cắt 40% một lần rồi train N bước vs 4 vòng 10% + $N/4$ bước mỗi vòng. Phản chiếu mục 3.5 và đo xem lợi thế iterative có còn khi cắt cấu trúc.

10. Tóm tắt công thức / khái niệm

Khái niệm	Nội dung	Vị trí
Quy trình	train connectivity → prune → train weights (lặp)	Mục 3
Mask	$M_{ij} = \mathbb{1}[\,\lvert w_{ij}\rvert \geq \tau_\ell\,]$	Mục 4
Ngưỡng tầng	$\tau_\ell = q \cdot \mathrm{std}(W_\ell)$	Mục 4
Điều chỉnh dropout	$D_r = D_o\sqrt{C_{ir}/C_{io}}$	Eq. 2
Số kết nối	$C_i = N_i N_{i-1}$	Eq. 1
Kết quả	AlexNet 61M→6.7M (9×), VGG-16 138M→10.3M (13×)	Bảng 1
FLOP giảm	AlexNet 3×, VGG-16 5×, LeNet-5 6×, LeNet-300-100 12×	Bảng 2–5
Chi phí lưu trữ thưa	15.6% overhead; 5 bit (FC) / 8 bit (CONV) chỉ số	Mục 5

11. Kết luận ngắn

Han et al. cho thấy một công thức đơn giản — huấn luyện, cắt trọng số nhỏ, huấn luyện lại, lặp — đủ nén AlexNet/VGG 9–13× không mất độ chính xác, và đặt ra các chuẩn mực còn dùng đến nay: ngưỡng theo độ nhạy tầng, giữ trọng số cũ, huấn luyện lại với learning rate nhỏ, iterative tốt hơn one-shot. Điểm yếu lớn là kết quả thưa không cấu trúc, cần phần cứng chuyên dụng mới thành tốc độ; các bài 04–05 trả lời điều đó.

Nguồn: https://arxiv.org/abs/1506.02626

Bài 03

To prune, or not to prune: exploring the efficacy of pruning for model compression (2017)

Tác giả: Michael H. Zhu (Stanford; làm trong thời gian thực tập tại Google), Suyog Gupta (Google)
Venue / arXiv: arXiv:1710.01878 (bản v2, 13/11/2017). Bản arXiv không ghi venue.
Nguồn: https://arxiv.org/abs/1710.01878
Chủ đề: Lịch cắt dần (automated gradual pruning) trong lúc train; so sánh "large-sparse" với "small-dense" cùng bộ nhớ trên CNN, LSTM, NMT.
Vị trí trong lộ trình: Đọc sau 02 (Han 2015: train → cắt → retrain). Bài này (a) thay lịch cắt bằng *cắt dần trong lúc train* đơn giản, và (b) đặt câu hỏi nền tảng: *cắt một mô hình to có tốt hơn train thẳng mô hình nhỏ không?* Tiếp theo: 04, 05 (cắt có cấu trúc), 11 (Rethinking the value of pruning) và 43 (pruning vs train from scratch) quay lại câu hỏi này.
Chương nền liên quan: nền 01: Vì sao mạng cắt được, nền 05: Cắt rồi huấn luyện lại

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ bản arXiv đã đọc.

1. Bối cảnh

Han et al. (02) và Narang et al. (2017) cho thấy có thể cắt rất nhiều trọng số với ít mất mát. Điều này gợi ý hai khả năng: mô hình gốc **quá tham số hóa**, và **một lựa chọn khác** là chỉ cần *giảm số hidden unit* (giữ cấu trúc dense) để được đánh đổi kích thước–độ chính xác tương tự. Muốn biết cách nào hơn, phải so sánh ở **cùng dấu chân bộ nhớ**.

Bối cảnh ứng dụng của bài báo: suy luận tiết kiệm năng lượng trên thiết bị biên. Các tác giả lưu ý: tải trọng nhạy độ trễ thường bị giới hạn bởi băng thông bộ nhớ; nén mô hình vừa giảm số lần truy cập bộ nhớ tốn năng lượng vừa tăng băng thông hiệu dụng. Nhưng mô hình thưa cần phần cứng/kernel cho ma trận thưa và tốn thêm bộ nhớ chỉ số.

Hai khái niệm:

- **Large-sparse:** huấn luyện mô hình lớn rồi cắt thưa để còn ít tham số khác 0.
- **Small-dense:** mô hình nhỏ dense (giảm số unit / width multiplier) có kích thước tương đương.

2. Ý tưởng chính

Bài báo có hai đóng góp. Thứ nhất, một **lịch cắt dần tự động** rất đơn giản: tăng độ thưa từ s_i đến s_f theo hàm bậc ba, cứ vài trăm bước cập nhật mặt nạ bằng cách che đi các trọng số nhỏ nhất theo trị tuyệt đối trong từng tầng, không cần chỉnh nhiều siêu tham số và gắn được vào quá trình huấn luyện. Thứ hai, một **phép so sánh có kiểm soát**: với cùng số tham số khác 0, mô hình **lớn–thưa luôn thắng nhỏ–dense** trên MobileNet, LSTM ngôn ngữ (PTB) và NMT, với mức nén tới 10× và mất mát tối thiểu.

---

3. Phương pháp chi tiết

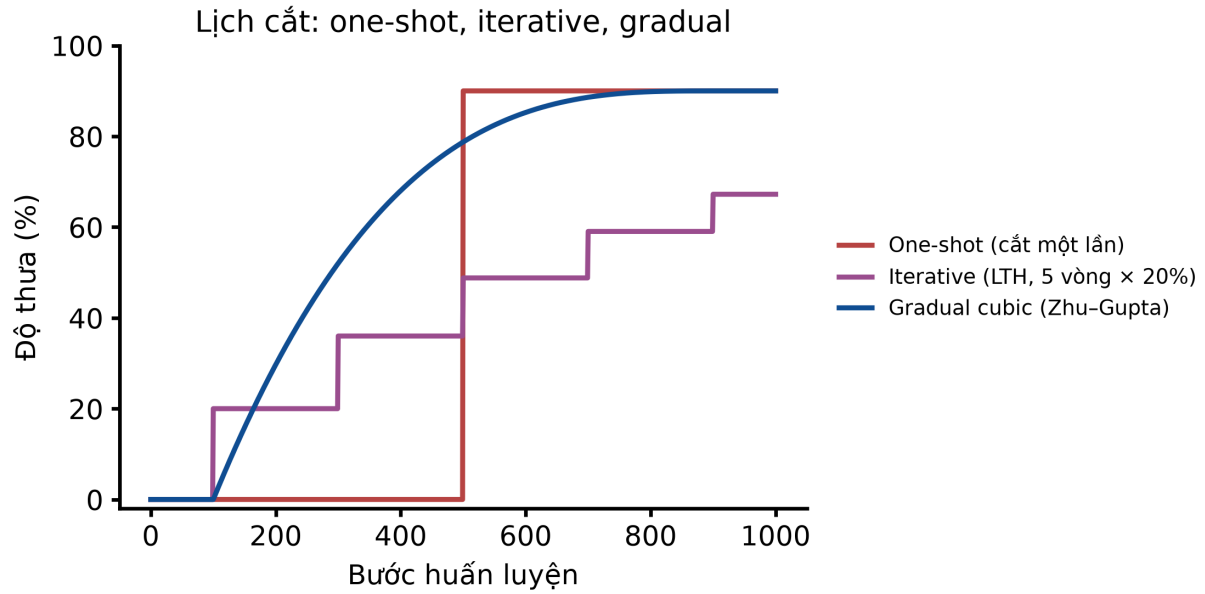
3.1 Cơ chế mask

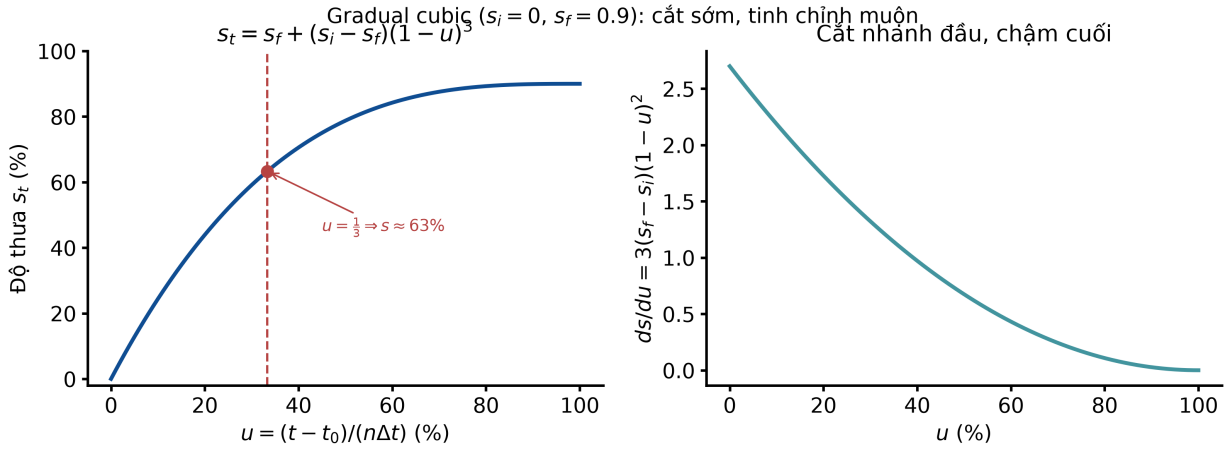
Với mỗi tầng được chọn cắt, thêm một biến mask nhị phân cùng kích thước với tensor trọng số; mask quyết định trọng số nào tham gia forward. Chèn thao tác vào đồ thị TensorFlow để **sắp xếp trọng số theo |w|** rồi

che các trọng số nhỏ nhất đến khi đạt độ thưa s mong muốn. Gradient truyền qua mask nên trọng số bị che không được cập nhật.

3.2 Lịch cắt dần (Eq. 1)

Độ thưa tăng từ s_i (thường 0) đến s_f trong n bước cắt, bắt đầu ở bước huấn luyện t_0, tần suất Δt:





Lịch gradual cubic $s_t = s_f + (s_i - s_f)(1-u)^3$ và ds/du: cắt nhanh đầu, chậm cuối ($u = \frac{1}{3} \Rightarrow s \approx 63\%$). (minh hoạ, tự tính)

Lịch độ thưa theo bước: one-shot, iterative (LTH), và gradual cubic của Zhu–Gupta. (minh hoạ, tự tính)

$$s_t = s_f + (s_i - s_f)\left(1 - \frac{t - t_0}{n\,\Delta t}\right)^3 \quad t \in \{t_0,\, t_0 + \Delta t, \ldots, t_0 + n\Delta t\}$$

Trực giác của paper: **cắt rất nhanh lúc đầu** khi còn nhiều kết nối dư thừa, rồi giảm dần lượng bị cắt mỗi bước khi còn ít trọng số. Số mũ ba làm đạo hàm ds/dt lớn lúc đầu và tiến dần về 0 ở cuối (xem Hình 1 của paper với các mốc 2×, 4×, 8×, 10×). Khi đạt s_f thì mask không cập nhật nữa.

Một ví dụ tính tay (tự dựng): với $s_i = 0$, $s_f = 0.9$, $n = 100$: tại 1/3 quãng đường ($\frac{t-t_0}{n\Delta t} = \frac{1}{3}$), $s = 0.9(1 - (2/3)^3) = 0.9 \times 0.704 \approx 0.63$. Tức đã đạt độ thưa 63% khi mới đi 1/3 lịch; 2/3 lịch còn lại chỉ thêm

27 điểm phần trăm (từ 63% lên 90%), đúng tinh thần "cắt sớm, tinh chỉnh muộn".

```
Pseudo-code
for step in training:
    SGD update (trên trọng số có mask)
    if step in {t0, t0+Δt, ..., t0+nΔt}:
        s = s_f + (s_i - s_f) * (1 - (step - t0)/(n*Δt))**3
        for mỗi tầng được cắt:
            mask = (|W| >= phân vị s của |W| trong tầng đó)
    # sau nΔt bước: mask cố định, tiếp tục train bình thường
```

3.3 Chọn siêu tham số

- Tần suất Δt từ 100 đến 1000 bước: **tác động không đáng kể** đến chất lượng cuối (kinh nghiệm của tác giả).
- Cắt bắt đầu **sau vài epoch** hoặc từ mô hình đã huấn luyện sẵn → xác định t_0.
- Độ dài n phụ thuộc lịch learning rate: nếu learning rate **đã quá nhỏ**, mô hình khó phục hồi sau cú cắt; nếu quá lớn thì cắt khi trọng số chưa hội tụ. Vì vậy phải chọn lịch cắt gắn với lịch learning rate.
- Với InceptionV3 (Hình 2): cắt xảy ra khi learning rate còn tương đối cao để mạng "tự lành". Khi sparsity 87.5%, có thời điểm mô hình suy giảm "gần như thảm họa" rồi hồi phục nhanh khi tiếp tục huấn luyện.

3.4 Biến thể cho NMT

Vì huấn luyện NMT có phương sai lớn, paper thử: (i) **cắt đồng loạt cùng độ thưa mọi tầng** (mặc định); (ii) **"layerwise constant"**: chia khoảng cắt và tăng độ thưa từng tầng một; (iii) **global pruning**: cắt trọng số nhỏ nhất trên toàn mạng không phân biệt tầng (tương tự See et al. 2016). Kết quả: layerwise constant tốt nhất trung bình và được dùng để báo cáo.

3.5 Chi phí lưu trữ thưa

Hai biểu diễn:

- **Bit-mask:** 1 bit mỗi phần tử ma trận + vector các phần tử khác 0 → chi phí cố định bất kể độ thưa.
- **CSR(C):** mỗi phần tử khác 0 kèm số đếm các số 0 đứng trước (4–5 bit) → chi phí tỉ lệ với NNZ; có lợi khi rất thưa.

---

4. Thí nghiệm và kết quả chính

Mô hình: InceptionV3 và MobileNet (ảnh, ImageNet), LSTM ngôn ngữ trên Penn Tree Bank (PTB), seq2seq NMT kiểu Google (GNMT) trên WMT16. Mọi kết quả dùng float 32 bit.

4.1 InceptionV3 (Bảng 1)

Độ thưa	NNZ	Top-1	Top-5
0%	27.1M	78.1%	94.3%
50%	13.6M	78.0%	94.2%
75%	6.8M	76.1%	93.2%
87.5%	3.3M	74.6%	92.5%

Theo paper: 50% thưa ngang baseline; 87.5% (8× ít tham số) chỉ giảm khoảng 2% top-5.

4.2 MobileNet (Bảng 2)

Dense (width multiplier 0.25 / 0.5 / 0.75 / 1.0): NNZ 0.46M / 1.32M / 2.57M / 4.21M, top-1 50.6% / 63.7% / 68.4% / 70.6%, top-5 75.0% / 85.4% / 88.2% / 89.5%.
MobileNet 1.0 được cắt thưa:

Độ thưa	NNZ	Top-1	Top-5
50%	2.13M	69.5%	89.5%
75%	1.09M	67.7%	88.5%
90%	0.46M	61.8%	84.7%
95%	0.25M	53.6%	78.9%

Lưu ý: 99% tham số của MobileNet 1.0 nằm ở pointwise 1×1 (74.6%) và FC (24.3%); conv chuẩn đầu và depthwise (1.1%) **không cắt**. Ví dụ so sánh của paper: sparse 75% (1.09M, 67.7%) hơn dense 0.5 (1.32M, 63.7%) 4 điểm và còn nhỏ hơn; sparse 90% (0.46M, 61.8%) hơn dense 0.25 (0.46M, 50.6%) **10.2 điểm** top-1 với cùng NNZ. Cắt MobileNet dùng learning rate khởi đầu nhỏ hơn 10 lần dense, các siêu tham số khác giữ nguyên.

4.3 LSTM trên PTB (Bảng 3, perplexity — thấp hơn là tốt)

Cỡ	Độ thưa	NNZ	Perplexity
Small	0%	4.6M	115.30
Medium	0%	19.8M	83.37
Medium	80%	4.0M	83.87
Medium	85%	3.0M	85.17
Medium	90%	2.0M	87.86
Medium	95%	1.0M	96.30
Medium	97.5%	0.5M	113.6
Large	0%	66M	78.45
Large	80%	13.2M	77.52
Large	85%	9.9M	78.31
Large	90%	6.6M	80.24
Large	95%	3.3M	87.83
Large	97.5%	1.7M	103.20

Mô hình large: 15M tham số embedding, 18M mỗi LSTM (2 lớp), 15M softmax = 66M. Cắt được cả embedding lẫn LSTM và softmax. **Large 90% (6.6M, PPL 80.24) hơn dense Medium (19.8M, PPL 83.37)** dù ít tham số hơn 3 lần.

4.4 NMT (Bảng 4, BLEU — cao hơn là tốt)

Số unit	Độ thưa	NNZ	EN-DE	DE-EN
256	0%	34M	23.52	26.52
512	0%	81M	26.05	28.88
768	0%	140M	26.63	29.41
1024	0%	211M	26.77	29.47
1024	80%	44M	26.86	29.50

Số unit	Độ thưa	NNZ	EN-DE	DE-EN
1024	85%	33M	26.52	29.24
1024	90%	23M	26.19	28.81

Mô hình 1024 đơn vị có 37.4M tham số cho mỗi embedding encoder, decoder, softmax và 98.6M trong LSTM (211M tổng); không cắt attention. Phương sai lớn: thanh sai số là độ lệch chuẩn của BLEU qua **10 lần huấn luyện** độc lập. Dense giảm nhanh sau khi nhỏ 2×; sparse chỉ bắt đầu giảm sau 5× ít NNZ. Sparse 90% (23M) tương đương hoặc hơn dense 512 đơn vị (81M): 26.19 vs 26.05 EN-DE, 28.81 vs 28.88 DE-EN, dù ít NNZ hơn 3.5 lần.

4.5 Chi phí lưu trữ và kích thước thực (Bảng 5–6)

Bit-mask MobileNet: 0.52 MB cố định; CSR(C): 1.06 / 0.54 / 0.23 / 0.13 MB tại độ thưa 50% / 75% / 90% / 95%. Bảng 6 (kích thước mô hình tính cả overhead, 32 bit mỗi giá trị khác 0) so sánh small-dense với large-sparse cùng cỡ: ví dụ 10.28 MB dense (68.4%) so với 9.04 MB sparse (69.5%); 5.28 MB dense (63.7%) so với 4.88 MB sparse (67.7%). (Bảng trích từ PDF bị xáo cột; chỉ dùng được hai cặp đầu.)

---

5. Phân tích / ablation

- **Large-sparse > small-dense** trên mọi mô hình, và **khoảng cách mở rộng khi mô hình lớn** (PTB, NMT) so với MobileNet nhỏ gọn đã tối ưu (paper nhận xét thêm: PTB có nhiều tham số hơn nên cắt cho kết quả tốt hơn).
- **Có vùng nén tối ưu:** PTB cắt 95% (20×) làm tụt rõ so với 90% (10×). Bình luận của tác giả: để có mô hình thưa tốt nhất ở một cỡ cho trước, nên *huấn luyện dense lớn hơn 5–10× rồi cắt*, đừng lấy mô hình tốt nhất lớn nhất rồi cắt 20× trở lên; ví dụ 85% thưa của medium (3M, PPL 85.17) tốt hơn 95% thưa của large (3.3M, PPL 87.83). (Chú ý: họ dùng cùng siêu tham số với dense, nên có thể có thêm dư địa nếu tinh chỉnh.)
- **Phương sai:** NMT dao động mạnh giữa các lần train → cần nhiều seed (10 lần trong paper).
- **Layerwise constant vs đồng loạt vs global** cho NMT: layerwise constant tốt nhất trung bình.
- **Mối quan hệ lịch cắt–learning rate** (mục 3.3).
- **Overhead lưu trữ thưa** làm giảm tỷ lệ nén thực, nhất là khi lượng tử hóa xuống 8 bit; large-sparse vẫn thắng sau khi tính overhead (Bảng 6).

6. Giới hạn

Do tác giả nêu:

- Kết quả dùng float 32 bit; với suy luận độ chính xác thấp (int8) overhead của biểu diễn thưa chiếm phần lớn hơn; tương tác giữa lượng tử hóa và cắt tỉa **để dành cho công việc sau**.
- Có thể có kết quả tốt hơn ở độ thưa ≥ 95% nếu tinh chỉnh siêu tham số riêng cho từng mô hình.
- Mô hình thưa cần **phần cứng chuyên dụng** hoặc phép toán ma trận thưa để thấy lợi ích tốc độ.

Nhận xét:

- Không đo **tốc độ/độ trễ thực** trên bất kỳ phần cứng nào; kết luận "tốt hơn" chỉ tính số tham số khác 0 (cộng overhead lưu trữ), mà bộ nhớ tại runtime của kernel thưa trên GPU có thể khác.
- "Small-dense" chỉ thu nhỏ bằng width/unit, không thử giảm số tầng, distillation, hay huấn luyện lâu hơn cho mô hình nhỏ. Paper về sau (11, 43) chỉ ra kết quả so sánh phụ thuộc mạnh vào ngân sách huấn luyện mô hình nhỏ.

- Mô hình lớn được cắt **được train/finetune lâu hơn** mô hình nhỏ (cắt xảy ra trong quá trình huấn luyện dài), nên so sánh chưa ngang bằng về tính toán.
- Không có cấu trúc: cắt theo magnitude từng phần tử, không so sánh với cắt có cấu trúc (04, 05) hay cùng ngân sách FLOPs.
- Chỉ cắt các tầng lớn (bỏ depthwise ở MobileNet), kết quả không mang sang cấu trúc hiện đại như attention.

7. Liên hệ với các bài khác

- **02 Han 2015:** cùng magnitude, nhưng 03 *tích hợp cắt vào huấn luyện* thay vì cắt sau rồi retrain; 03 cũng lưu ý So sánh với Narang et al. (hai pha, hai độ dốc, ngưỡng thủ công).
- **01 OBD/OBS:** bài này nhắc lại OBD/OBS trong related work, chọn magnitude vì **mở rộng được**.
- **04, 05:** cắt có cấu trúc; 03 nhận xét chúng "phụ thuộc cấu trúc lớp conv, khó mở rộng sang LSTM", đồng thời lý do cũng là sự đánh đổi mà ta phải chấp nhận.
- **08 Lottery Ticket:** cùng tinh thần "tìm mạng con thưa trong mạng lớn"; 03 dùng cắt dần thay vì rewinding.
- **11 Rethinking the value of pruning, 43 Small LLMs pruning vs scratch, 12 State of pruning:** quay lại câu hỏi "large-sparse hay small-dense"; đáng đọc cùng để thấy kết quả 03 *không phổ quát*.
- **18 2:4 sparsity:** dạng thưa có phần cứng hỗ trợ mà 03 mong chờ ("customize the next generation accelerators").
- Công thức lịch bậc ba về sau xuất hiện trong nhiều thư viện (ví dụ dạng lịch sparsity cubic) và trong các phương pháp cắt dần trong LLM (như 16 movement pruning dùng lịch tăng dần độ thưa tương tự — xin kiểm tra lại ở ghi chú 16).

8. Gợi ý áp dụng

- Nên chạy đối chứng này (xem 43).

9. Thí nghiệm gợi ý

1. **Lịch cắt cấu trúc dần dần**: áp dụng công thức bậc ba để *giảm dần số neuron MLP* (mask mềm: nhân hệ số $m_j(t)$ giảm dần về 0 cho neuron bị chọn) trong 2k bước KD, so với cắt một lần rồi fine-tune cùng tổng số bước. Đo loss KD và CER.
2. **Độ nhạy lịch–LR**: chạy ba cấu hình lịch cắt (sớm / giữa / muộn so với lịch LR) và xem tổn thất phục hồi khớp nhận xét "pruning dưới LR rất nhỏ khó hồi phục".

10. Tóm tắt công thức / khái niệm

Khái niệm	Nội dung	Vị trí
Gradual pruning	$s_t = s_f + (s_i - s_f)(1 - \frac{t-t_0}{n\Delta t})^3$	Eq. 1
Mask	che các	w
Large-sparse vs small-dense	cùng số NNZ/bộ nhớ, sparse thắng trên mọi mô hình thử	Mục 4–5
Bit-mask vs CSR(C)	1 bit/phần tử cố định vs 4–5 bit/phần tử khác 0	Mục 5
Vùng nén tối ưu	cắt 5–10× từ mô hình lớn hơn tốt hơn cắt ≥ 20×	Mục 4.2
Kết quả đỉnh	Inception-V3 87.5% thưa: 74.6/92.5; MobileNet 90%: 61.8% vs dense 0.25: 50.6%; PTB large 90%: PPL 80.24; NMT 1024 @90%: 26.19/28.81 BLEU	Bảng 1–4

11. Kết luận ngắn

Zhu & Gupta đề xuất một lịch cắt dần (bậc ba, mask theo magnitude, vài trăm bước mỗi lần) và một phép so sánh thực nghiệm giữa mạng thưa và mạng dense: khi so cùng bộ nhớ trên nhiều kiến trúc, **mô hình lớn rồi cắt thưa thắng mô hình nhỏ dense**, nhưng chỉ ở mức nén vừa phải (≈ 5–10×) và chưa kể tốc độ thực. Bài học còn nguyên giá trị: lịch cắt gắn với lịch học và luôn so với baseline nhỏ có kiểm soát — nhưng cũng nên đọc cùng 11, 12, 43 vì kết luận này không phải lúc nào cũng đứng vững.

Nguồn: https://arxiv.org/abs/1710.01878

Bài 04

Pruning Filters for Efficient ConvNets (2016)

Tác giả: Hao Li, Asim Kadav, Igor Durdanovic, Hanan Samet, Hans Peter Graf
Venue / arXiv: arXiv:1608.08710 (bản v3, 10/3/2017); ICLR 2017 (bản arXiv ghi "Published as a conference paper at ICLR 2017").
Nguồn: https://arxiv.org/abs/1608.08710
Chủ đề: Cắt filter (channel) có cấu trúc bằng chuẩn ℓ_1, phân tích độ nhạy từng tầng, cắt VGG/ResNet, prune-once-and-retrain.
Vị trí trong lộ trình: Đọc sau 02 (Han: cắt trọng số không cấu trúc) và 03 (cắt dần). Bài này chuyển sang **cắt cấu trúc để giảm tính toán thật mà không cần thư viện thưa**. Tiếp theo: 05 (Network Slimming: học cổng γ thay vì dùng ℓ_1 của trọng số), 06 (ThiNet: chọn filter theo tái tạo đầu ra), 07, 20.
Chương nền liên quan: nền 02: Ma trận thưa và chi phí thật, nền 04: Cắt hạt, cắt khối và mẫu 2

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ bản arXiv đã đọc; một số bảng PDF bị xáo cột nên chỉ trích dẫn khi đọc rõ.

1. Bối cảnh

Han et al. (02) nén AlexNet/VGG bằng cách cắt trọng số theo độ lớn. Li et al. chỉ ra hai vấn đề:

1. Phần lớn tham số bị loại nằm ở tầng FC, nơi tính toán rẻ. Bài báo nêu: FC của VGG-16 chiếm **90% tham số nhưng dưới 1% FLOP**. Vì vậy cắt tham số không nhất thiết giảm tính toán.
2. Cắt trọng số tạo ra **thưa bất quy tắc** trong các tầng conv; muốn tăng tốc cần thư viện/phần cứng cho conv thưa (và cấu trúc dữ liệu thưa tốn thêm bộ nhớ, đáng kể với trọng số độ chính xác thấp).

Trong khi đó, mạng hiện đại (ResNet) đã thay FC bằng average pooling, tính toán nằm ở conv.

Thuật ngữ:

- **Filter:** tensor $F_{i,j} \in \mathbb{R}^{n_i \times k \times k}$ tạo ra *một* feature map; tầng i có n_{i+1} filter.
- **Feature map:** đầu ra 2D của một filter (một channel).
- **BLAS:** thư viện nhân ma trận dense hiệu quả; mục tiêu là vẫn dùng được nó.

2. Ý tưởng chính

Thay vì cắt từng trọng số, **cắt nguyên filter**. Bỏ một filter đồng nghĩa với bỏ feature map nó tạo ra, và bỏ luôn các kernel ở tầng kế tiếp ăn feature map đó, nên ma trận kết quả vẫn dense, chỉ nhỏ hơn. Để chọn filter, chỉ cần **tổng trị tuyệt đối trọng số của filter (ℓ_1-norm)**: filter có chuẩn nhỏ tạo ra feature map yếu nên ít quan trọng. Bài báo còn phân tích độ nhạy từng tầng (cắt riêng từng tầng rồi đo độ chính xác) để chọn tỉ lệ cắt theo giai đoạn (stage), xử lý ràng buộc của ResNet, và huấn luyện lại một lần (prune once and retrain) thay vì lặp từng tầng.

---

3. Phương pháp chi tiết

3.1 Phép tính khi bỏ filter

Tầng i: đầu vào $x_i \in \mathbb{R}^{n_i \times h_i \times w_i}$, n_{i+1} filter $F_{i,j} \in \mathbb{R}^{n_i \times k \times k}$, đầu ra $x_{i+1} \in \mathbb{R}^{n_{i+1} \times h_{i+1} \times w_{i+1}}$. Số phép tính của tầng:

$$n_{i+1}\, n_i\, k^2\, h_{i+1} w_{i+1}.$$

Bỏ một filter $F_{i,j}$ giảm $n_i k^2 h_{i+1} w_{i+1}$ phép tính của tầng i, và các kernel ở tầng $i{+}1$ nhận feature map $x_{i+1,j}$ cũng bị bỏ, giảm thêm $n_{i+2} k^2 h_{i+2} w_{i+2}$. Do đó **cắt m filter của tầng i giảm m/n_{i+1} chi phí tính toán của cả tầng i và $i{+}1$**. Khi tầng bị cắt có BatchNorm theo sau, tham số BN tương ứng cũng bị bỏ; nếu bỏ channel cuối cùng của khối conv, đầu vào tầng linear tiếp theo thay đổi và các kết nối tương ứng cũng bị bỏ.

Đây chính là phiên bản CNN của quy tắc "bỏ một hàng của W_{in} và một cột của W_{out}" (preface mục 6.7).

3.2 Chọn filter trong một tầng

$$\boxed{s_j = \|F_{i,j}\|_1 = \sum_{l=1}^{n_i} \sum_{u,v} |K_l(u,v)|}$$

```
Cắt m filter của tầng i:
1. Với mỗi filter F_{i,j}: s_j = tổng |trọng số|
2. Sắp xếp theo s_j
3. Bỏ m filter có s_j nhỏ nhất cùng feature map tương ứng;
   bỏ các kernel của tầng i+1 ứng với các feature map đó
4. Tạo ma trận kernel mới cho tầng i và i+1, sao chép phần còn lại
```

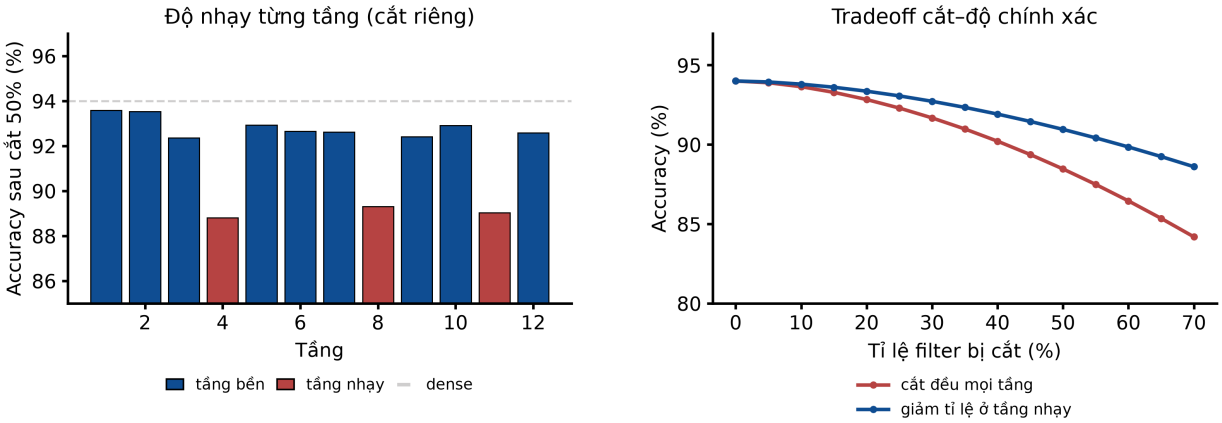
Lý do theo paper: n_i giống nhau ở mọi filter nên s_j cũng tỉ lệ với độ lớn trung bình của kernel; filter nhỏ cho feature map có activation yếu. So sánh thực nghiệm (mục 5): cắt filter nhỏ nhất tốt hơn cắt ngẫu nhiên và hơn nhiều cắt filter lớn nhất.

Quan hệ với các phương pháp khác (theo paper):

- *Cắt trọng số theo magnitude* có thể vô tình cắt cả filter nếu mọi kernel dưới ngưỡng, nhưng khó dự đoán số filter bị cắt, cần chỉnh ngưỡng cẩn thận và tạo kernel thưa khó tăng tốc.
- *Group-sparse regularization* ($\ell_{2,1}$-norm; Zhou 2016, Wen 2016) đẩy cả filter về 0 trong lúc huấn luyện. Li et al. không thấy khác biệt đáng kể giữa ℓ_1 và ℓ_2 khi chọn filter (Phụ lục 6.1: filter quan trọng có cả hai chuẩn lớn).

3.3 Độ nhạy của từng tầng

Cắt **riêng từng tầng** rồi đo accuracy trên tập validation. Hình 2(b): các tầng có accuracy giữ được khi cắt tương ứng với tầng có đường sắp xếp ℓ_1 **dốc** (một số ít filter lớn, nhiều filter nhỏ) trong Hình 2(a); tầng có đường phẳng (filter có chuẩn tương tự nhau) nhạy hơn. Để tránh thêm siêu tham số theo tầng, dùng **cùng tỉ lệ cắt cho mọi tầng trong một stage** (cùng kích thước feature map); với tầng nhạy, cắt ít hơn hoặc bỏ qua.



Độ nhạy từng tầng khi cắt filter và đường cong tradeoff tỉ lệ cắt–accuracy (mô phỏng theo trực giác note). (minh hoạ, tự tính)

3.4 Cắt nhiều tầng cùng lúc

Hai chiến lược chọn filter ở nhiều tầng:

- **Independent:** chọn filter của mỗi tầng độc lập, tính s_j trên toàn bộ kernel gốc, kể cả những kernel ứng với feature map đã bị bỏ ở tầng trước.
- **Greedy:** khi tính s_j, **không tính các kernel ứng với feature map đã bị cắt** ở tầng trước. Không tối ưu toàn cục nhưng "holistic" và cho accuracy cao hơn, nhất là khi cắt nhiều.

3.5 ResNet

- Với khối residual có **projection shortcut** (Hình 4): filter của tầng conv *thứ nhất* của khối cắt tự do (không đổi số channel đầu ra khối). Tầng *thứ hai* bị ràng buộc: số channel ra phải khớp với nhánh tắt (identity), nên chọn channel cần cắt theo kết quả cắt của shortcut (filter 1×1) và cắt cùng chỉ số ở tầng thứ hai.
- Với ResNet CIFAR (shortcut identity + zero padding, không có projection): chỉ cắt tầng thứ nhất của khối.
- Lý do (theo paper): feature map identity quan trọng hơn feature map residual thêm vào.

3.6 Retrain

Hai chiến lược:

1. **Prune once and retrain:** cắt nhiều tầng một lần rồi retrain tới khi phục hồi độ chính xác gốc. Hiệu quả với tầng *bền* (resilient).
2. **Prune and retrain iteratively:** cắt từng tầng/ filter rồi retrain trước khi cắt tiếp; có thể tốt hơn nhưng đòi hỏi **nhiều epoch hơn nhiều**, nhất là mạng rất sâu.

Paper chọn (1) cho đơn giản. Với tầng nhạy hoặc cắt quá nhiều, có thể không phục hồi được. Thiết lập retrain: **learning rate hằng 0.001, 40 epoch với CIFAR-10 và 20 epoch với ImageNet** — khoảng *một phần tư* số epoch huấn luyện gốc (so với việc báo cáo trước đó đến 3× thời gian gốc cho retrain, Han et al.). Cài đặt bằng Torch7: tạo mô hình mới nhỏ hơn rồi sao chép tham số tương ứng.

---

4. Thí nghiệm và kết quả chính

Mạng: VGG-16 (biến thể CIFAR-10 của Zagoruyko có thêm BN, không Dropout), ResNet-56/110 (CIFAR-10), ResNet-34 (ImageNet ILSVRC2012). Baseline huấn luyện từ đầu theo siêu tham số ResNet.

4.1 Bảng tổng kết (Bảng 1; lỗi test/val tốt nhất trong lúc retrain)

Mô hình	Lỗi (%)	FLOP	FLOP cắt	Tham số	Tham số cắt
VGG-16	6.75	3.13×10^8		1.5×10^7	
VGG-16-pruned-A	6.60	2.06×10^8	34.2%	5.4×10^6	64.0%
VGG-16-pruned-A, train từ đầu	6.88				
ResNet-56	6.96	1.25×10^8		8.5×10^5	
ResNet-56-pruned-A	6.90	1.12×10^8	10.4%	7.7×10^5	9.4%
ResNet-56-pruned-B	6.94	9.09×10^7	27.6%	7.3×10^5	13.7%

Mô hình	Lỗi (%)	FLOP	FLOP cắt	Tham số	Tham số cắt
ResNet-56-pruned-B, từ đầu	8.69				
ResNet-110	6.47	2.53×10^8		1.72×10^6	
ResNet-110-pruned-A	6.45	2.13×10^8	15.9%	1.68×10^6	2.3%
ResNet-110-pruned-B	6.70	1.55×10^8	38.6%	1.16×10^6	32.4%
ResNet-110-pruned-B, từ đầu	7.06				
ResNet-34	26.77	3.64×10^9		2.16×10^7	
ResNet-34-pruned-A	27.44	3.08×10^9	15.5%	1.99×10^7	7.6%
ResNet-34-pruned-B	27.83	2.76×10^9	24.2%	1.93×10^7	10.8%
ResNet-34-pruned-C	27.52	3.37×10^9	7.5%	2.01×10^7	7.2%

Chú thích bảng của paper: **huấn luyện mô hình đã cắt từ đầu cho kết quả tệ hơn retrain mô hình đã cắt**, "có thể cho thấy khó khăn khi huấn luyện mạng dung lượng nhỏ". (Điều này mâu thuẫn một phần với kết quả của 11; xem mục 7.)

4.2 VGG-16 trên CIFAR-10 (Bảng 2)

13 tầng conv + 2 tầng linear. Cấu hình pruned-A: conv 1 giảm 64→32 map và conv 2 giữ 64 map (nhưng nhận đầu vào ít hơn), cả hai giảm 50% FLOP; conv 8: 512→256 (-50%); conv 9–13: 512→256 (-75% FLOP mỗi tầng); linear đầu: 512 (-50%); conv 3–7 giữ nguyên. Tổng **34% FLOP**, **64% tham số**. Phát hiện chi tiết:

- Mỗi tầng conv có 512 feature map **có thể bỏ ít nhất 60% filter mà không ảnh hưởng accuracy**; với retrain khoảng **90%** (Hình 2b, 2c). Giải thích đề xuất: các filter này hoạt động trên feature map 4×4 hoặc 2×2 nên có thể không có quan hệ không gian có ý nghĩa; (ResNet CIFAR không có conv dưới 8×8).
- Khác với nhận xét ở các nghiên cứu trước (Zeiler & Fergus; Han), **tầng đầu bền với cắt hơn các tầng kế tiếp** trên CIFAR-10 — tác giả gán cho CIFAR-10 đơn giản, khác ImageNet (Hình 5). Cắt 80% filter của tầng 1 còn 12 filter vẫn nhiều hơn số kênh đầu vào; nhưng cắt 80% filter của tầng 2 là ánh xạ 64→12, làm mất thông tin và hại accuracy.

4.3 ResNet-56/110 trên CIFAR-10

Hình 6: độ nhạy của tầng đầu tiên mỗi khối. Phần lớn bền; một số tầng *không retrain còn cải thiện hiệu năng* ở ResNet-110. Các tầng nhạy (ResNet-56: tầng 20, 38, 54; ResNet-110: 36, 38, 74) nằm ở **các khối gần chỗ số feature map thay đổi** (khối đầu/cuối mỗi stage); tác giả cho rằng cần residual chính xác cho các feature map rỗng mới thêm. Cấu hình:

- ResNet-56-pruned-A: bỏ qua các tầng 16, 20, 38, 54; cắt 10% filter.
- ResNet-56-pruned-B: bỏ qua 16, 18, 20, 34, 38, 54; $p_1 = 60\%$, $p_2 = 30\%$, $p_3 = 10\%$.
- ResNet-110-pruned-A: $p_1 = 50\%$ và bỏ qua tầng 36.
- ResNet-110-pruned-B: bỏ qua 36, 38, 74; $p_1 = 50\%$, $p_2 = 40\%$, $p_3 = 30\%$.

Còn: tầng sâu nhạy hơn tầng nông; khi stage có hơn hai khối, **các khối ở giữa có thể dư thừa**, giải thích vì sao ResNet-110 dễ cắt hơn ResNet-56.

4.4 ResNet-34 trên ImageNet

Bốn stage (56, 28, 14, 7). Cấu hình A: $p_1 = p_2 = p_3 = 30\%$; B: $p_1 = 50\%$, $p_2 = 60\%$, $p_3 = 40\%$; các tầng nhạy bị bỏ qua (tầng 2, 8, 14, 16, 26, 28, 30, 32, tức khối đầu/cuối mỗi stage). B cho **24% giảm FLOP với khoảng 1% mất accuracy**. Cắt cả tầng conv thứ hai và shortcut (C: $p_3 = 20\%$ ở stage 3) chỉ giảm 7.5%

FLOP với mất 0.75% accuracy. Kết luận: **cắt tầng thứ nhất của khối hiệu quả hơn cắt tầng thứ hai**, phù hợp thiết kế bottleneck. Paper cũng dự đoán ResNet-34 khó cắt hơn ResNet sâu hơn.

---

5. Phân tích / ablation

5.1 Nhỏ nhất vs ngẫu nhiên vs lớn nhất (Hình 8, VGG-16 CIFAR-10)

Cắt filter ℓ_1 nhỏ nhất **tốt hơn cắt ngẫu nhiên ở đa số tầng**, ở tỉ lệ cắt 90% tốt hơn trên mọi tầng; cắt filter ℓ_1 lớn nhất làm accuracy rơi nhanh, chứng tỏ các filter lớn là quan trọng.

5.2 So sánh với cắt theo activation (Hình 9)

Dùng thống kê trên $N = 50{,}000$ ảnh train, trước BN và phi tuyến: mean-mean, mean-std, mean-ℓ_1, mean-ℓ_2, var-ℓ_2 (Polyak & Wolf). Kết quả: cắt filter nhỏ nhất theo ℓ_1 của *trọng số* tốt hơn các tiêu chí mean-mean, mean-ℓ_1, mean-ℓ_2 và var-ℓ_2; mean-std tốt hơn hoặc ngang tới tỉ lệ cắt 60% nhưng sau đó giảm nhanh nhất ở conv 1–3. Kết luận: ℓ_1 là "heuristic tốt" vì **không cần dữ liệu**.

5.3 Mỗi cải tiến góp bao nhiêu

- **Độ nhạy theo tầng + bỏ qua tầng nhạy** là đóng góp lớn nhất cho ResNet (pruned-A, B).
- **Retrain:** train mô hình cắt từ đầu kém hơn rõ trên ResNet-56 (8.69 vs 6.94) nhưng gần ngang trên VGG-16 (6.88 vs 6.60).
- **Greedy vs independent:** greedy tốt hơn khi cắt nhiều (nêu bằng lời; paper không có bảng riêng ở bản đã đọc).
- **Prune-once vs iterative:** iterative có thể tốt hơn nhưng đắt (nêu bằng lời; không có thí nghiệm đối chứng đầy đủ trong paper).

6. Giới hạn

Do tác giả nêu:

- Tầng nhạy hoặc cắt quá nhiều có thể không phục hồi được bằng retrain ngắn.
- ResNet-34 khó cắt hơn ResNet sâu; ResNet bị ràng buộc bởi shortcut.
- Một số lựa chọn (tầng bỏ qua, tỉ lệ theo stage) xác định thực nghiệm theo độ nhạy, không tự động.

Nhận xét:

- Paper **không đo thời gian suy luận thực** (chỉ FLOP, tham số) bất chấp mục tiêu tăng tốc thực (thời gian thực được đo ở các bài sau như 05 phụ lục, 19 HALP).
- Tiêu chí ℓ_1 của trọng số **bỏ qua BatchNorm** (hệ số γ, β có thể đảo ý nghĩa của độ lớn filter) và độ lớn đầu vào; các bài sau (05, 06, 07) sửa điều này.
- Kết quả tốt trên CIFAR-10 (mạng nhỏ, tập đơn giản); ResNet-34 trên ImageNet chỉ giảm 24% FLOP và mất ~1% accuracy, tức hiệu quả giảm đáng kể ở bài toán khó.
- Cắt các tầng "độc lập" cùng tỉ lệ trong stage là xấp xỉ thô; không có cơ chế phân bổ ngân sách tối ưu giữa các tầng.
- Không so sánh với baseline "mô hình nhỏ hơn được tối ưu tốt" (khác với 03); nhận định "train from scratch tệ hơn" là trên một số cấu hình và đã bị 11 phản biện.
- Một seed cho mỗi cấu hình trong bảng; chênh lệch 0.1–0.2% ở CIFAR có thể nằm trong nhiễu.

7. Liên hệ với các bài khác

- **02 Han:** tiền thân; 04 chỉ ra vấn đề tăng tốc và đổi sang cắt cấu trúc, đồng thời lưu ý phép so sánh giữa cắt filter và cắt trọng số nhỏ theo ngưỡng.

- **05 Network Slimming:** cùng mục tiêu; thay tiêu chí ℓ_1 của trọng số bằng hệ số γ của BN học bằng ℓ_1 trong lúc train; dùng ngưỡng toàn cục; 05 so sánh trực tiếp với 04 (Bảng 7 của 05: cùng mức nén 88.5% trên CIFAR-10, slimming 6.20% vs 04 6.88%).
- **06 ThiNet, 07 Molchanov:** tiêu chí tốt hơn dựa trên tái tạo đầu ra hoặc Taylor; 04 là baseline "data-free".
- **11 Rethinking the value of pruning:** đặt lại câu hỏi retrain/khởi tạo từ trọng số cũ vs train mô hình cắt từ đầu; kết quả 04 (ở đây "từ đầu" thường kém) cần đối chiếu.
- **13, 14 (cắt head), 26 LLM-Pruner, 28 FLAP:** áp dụng cùng ý "bỏ hàng/cột liên đới" cho attention/MLP.
- **20 DepGraph:** tổng quát hóa việc xử lý phụ thuộc cấu trúc mà 04 làm thủ công với ResNet (mục 3.5).
- **31–36 (cắt layer)**: ở ResNet 04 đã thấy khối giữa dư thừa; 04 không cắt khối nhưng quan sát "các khối giữa stage có thể dư thừa" gợi ý trước hướng bỏ layer.

8. Gợi ý áp dụng

- **Chuyển sang transformer**: filter ↔ neuron MLP (hàng W_{in} + cột W_{out}) hoặc head; ℓ_1-norm của hàng W_{in} là điểm số data-free đơn giản nhất. Nhưng vì transformer có RMSNorm/LayerNorm và residual, độ lớn trọng số kém tin cậy hơn; nên kết hợp tiêu chí dựa activation (xem 24, 28, 29).
- **Greedy**: tính điểm của tầng sau trên *kernel còn lại* của tầng trước là thủ thuật dễ nhập vào quy trình cắt tuần tự (cắt MLP của block b rồi tính điểm block $b+1$ trên activation đã bị thay đổi).

9. Thí nghiệm gợi ý

3. **Greedy vs independent**: cắt đồng thời MLP của nhiều block; so điểm tính trên trọng số gốc (independent) với điểm tính sau khi block trước đã bị cắt (greedy). Đo chênh lệch loss trước fine-tune.

10. Tóm tắt công thức / khái niệm

Khái niệm	Nội dung	Vị trí
FLOP của tầng conv	$n_{i+1} n_i k^2 h_{i+1} w_{i+1}$	Mục 3
Lợi ích cắt m filter	giảm m/n_{i+1} chi phí của cả tầng i và $i{+}1$	Mục 3
Điểm filter	$s_j = \sum_l \sum \lvert K_l \rvert = \lVert F_{i,j} \rVert_1$	Mục 3.1
Độ nhạy tầng	cắt riêng từng tầng rồi đo val accuracy	Mục 3.2
Independent vs greedy	có / không tính kernel của feature map đã cắt	Mục 3.3
ResNet	tầng 1 khối cắt tự do; tầng 2 theo shortcut	Mục 3.3
Retrain	LR 0.001; 40 epoch (CIFAR), 20 epoch (ImageNet) ≈ 1/4 số epoch gốc	Mục 4
Kết quả chính	VGG-16 −34.2% FLOP, −64% tham số, lỗi 6.60 (gốc 6.75); ResNet-110 −38.6% FLOP, lỗi 6.70 (gốc 6.47); ResNet-34 −24.2% FLOP, lỗi 27.83 (gốc 26.77)	Bảng 1

11. Kết luận ngắn

Bài chứng minh rằng một tiêu chí cực đơn giản — tổng trị tuyệt đối trọng số của filter — cộng với phân tích độ nhạy từng tầng và huấn luyện lại ngắn là đủ để giảm khoảng 30–38% FLOP của VGG-16 và ResNet-110 trên CIFAR-10 gần như không mất độ chính xác, bằng những ma trận dense nhỏ hơn dùng được với BLAS thông thường. Hạn chế là mức nén vừa phải, phụ thuộc chọn thủ công các tầng nhạy, tiêu chí data-free thô, và chưa đo tốc độ thật — những điểm mà 05, 06, 07 và 19 cải thiện.

Nguồn: https://arxiv.org/abs/1608.08710

Bài 05

Learning Efficient Convolutional Networks through Network Slimming (2017)

Tác giả: Zhuang Liu, Jianguo Li, Zhiqiang Shen, Gao Huang, Shoumeng Yan, Changshui Zhang
Venue / arXiv: arXiv:1708.06519 (v1, 22/8/2017). Bản arXiv không ghi venue trong phần văn bản đã đọc.
Nguồn: https://arxiv.org/abs/1708.06519
Chủ đề: Cắt channel có cấu trúc bằng phạt ℓ_1 lên hệ số scale γ của BatchNorm, ngưỡng toàn cục, multi-pass.
Vị trí trong lộ trình: Đọc sau 04 (Li et al.: chọn filter theo ℓ_1 của trọng số, sau khi train). Bài này **đưa tiêu chí vào chính hàm mục tiêu huấn luyện**: mạng học luôn "kênh nào cần giữ". Tiếp theo: 06 ThiNet, 07 Molchanov, 16 Movement pruning, 17 CoFi (mặt nạ học được cho transformer; cùng họ "phạt thưa lên cổng").
Chương nền liên quan: nền 04: Cắt hạt, cắt khối và mẫu 2

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ bản arXiv đã đọc (một số bảng bị xáo cột trong PDF, chỉ trích khi đọc rõ).

1. Bối cảnh

Bài báo nêu ba nguồn chi phí khi triển khai CNN lớn:

1. **Kích thước mô hình** (CNN ImageNet điển hình trên 300 MB).
2. **Bộ nhớ lúc chạy**: activation trung gian có thể tốn *hơn cả* trọng số, ngay cả với batch 1.
3. **Số phép tính**: tích chập nặng trên ảnh độ phân giải cao.

Hầu hết các kỹ thuật nén chỉ giải quyết một–hai điều; nhiều kỹ thuật cần phần cứng/thư viện đặc biệt (lượng tử hóa nhị phân, weight pruning thưa). Điểm quan trọng: **cắt trọng số (02) gần như không giảm bộ nhớ runtime** vì phần lớn bộ nhớ là activation (vẫn dense). Muốn giảm cả ba thì phải giảm *số channel*.

Khó khăn của channel pruning (paper gọi là *Challenges*): cắt channel nghĩa là cắt mọi kết nối vào và ra khỏi nó; nên cắt trực tiếp trên mô hình đã huấn luyện rất khó vì hiếm khi toàn bộ trọng số tại hai đầu của một channel đều xấp xỉ 0. Bài báo dẫn rằng ở [Li et al. = bài 04], cắt channel trên ResNet đã huấn luyện chỉ giảm khoảng 10% tham số mà không mất độ chính xác. Hướng khác: Structured Sparsity Learning (SSL, Wen et al.) dùng group LASSO trên trọng số filter, nhưng cần tính gradient của số hạng phạt theo mọi trọng số filter, "không đơn giản".

Thuật ngữ:

- **BatchNorm (BN):** chuẩn hóa activation theo mini-batch rồi biến đổi affine bằng γ, β học được.
- **Hệ số scale γ:** nhân vào mỗi channel sau chuẩn hóa. Nếu $\gamma_c \approx 0$ thì channel c tạo ra gần như hằng số β_c.
- **ℓ_1 regularization:** thêm $\lambda \sum |\gamma|$ vào loss để đẩy nhiều giá trị về đúng 0.

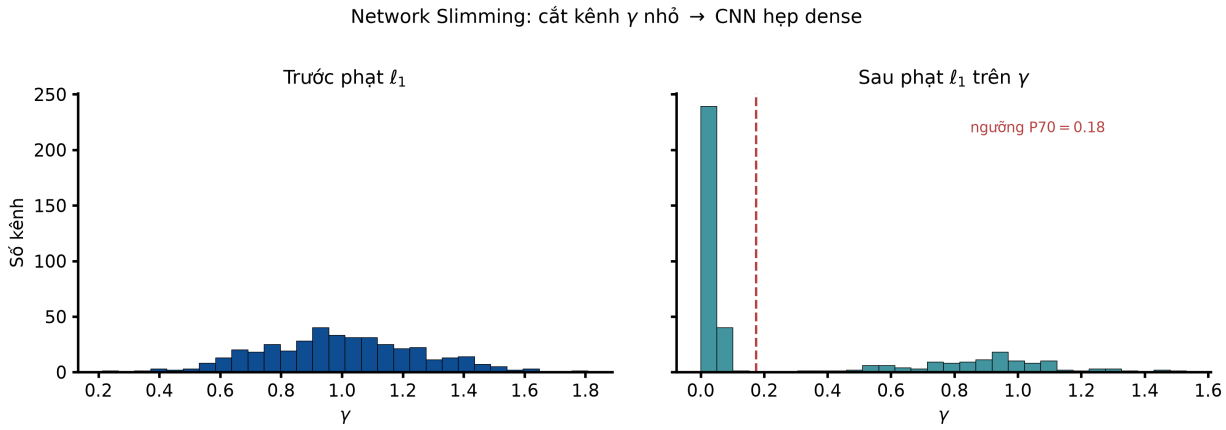
2. Ý tưởng chính

Phương pháp dùng hệ số tỷ lệ γ sẵn có trong BatchNorm để đánh giá từng kênh, không cần thêm tham số. Huấn luyện chung trọng số và γ với phạt ℓ_1 lên γ; sau đó nhiều γ gần 0 — các kênh đó là kênh không quan trọng. Cắt các kênh có γ nhỏ theo **một ngưỡng chung toàn mạng** (ví dụ phân vị 70%), tạo ra mô hình hẹp hơn nhưng dense, rồi fine-tune. Có thể lặp lại cả quy trình (multi-pass) để thu nhỏ thêm. Không cần thư viện thưa hay phần cứng đặc biệt.

3. Phương pháp chi tiết

3.1 Hàm mục tiêu (Eq. 1)

$$L = \sum_{(x,y)} \ell(f(x, W), y) + \lambda \sum_{\gamma \in \Gamma} g(\gamma), \qquad g(s) = |s|$$



Phân phối γ BatchNorm trước/sau phạt ℓ_1; cắt kênh dưới ngưỡng phân vị. (minh hoạ, tự tính)

W là trọng số, Γ là tập mọi hệ số scale; λ cân bằng hai số hạng. Tối ưu bằng **subgradient descent** cho phạt ℓ_1 không trơn; paper nói có thể thay bằng smooth-ℓ_1 (Schmidt et al.) để tránh điểm không khả vi. Gradient của phạt đối với γ: $\lambda \,\mathrm{sign}(\gamma)$.

3.2 Vì sao dùng γ của BN

BN (Eq. 2):

$$\hat{z} = \frac{z_{in} - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}, \qquad z_{out} = \gamma \hat{z} + \beta.$$

Paper lập luận γ trong BN là cách hiệu quả nhất để có hệ số kênh có ý nghĩa, vì thêm một lớp scale riêng thì không ổn:

1. Nếu chèn scale vào CNN không có BN, giá trị scale không có ý nghĩa đánh giá độ quan trọng (conv và scale đều tuyến tính, có thể giảm scale mà tăng trọng số, cùng kết quả).
2. Chèn scale *trước* BN thì tác dụng bị chính BN triệt tiêu.
3. Chèn *sau* BN thì mỗi kênh có hai hệ số scale liên tiếp (thừa).

Dùng γ sẵn có: **không thêm chi phí** nào.

3.3 Cắt channel và fine-tune

- Sau khi huấn luyện với phạt, nhiều γ gần 0 (Hình 1, Hình 4).
- Cắt **kênh có γ nhỏ**, bằng **ngưỡng toàn cục**: lấy phân vị của toàn bộ γ (ví dụ cắt 70% kênh có γ thấp nhất). Khác 04 (tỉ lệ cắt khác nhau theo tầng).
- Cắt = xây mô hình hẹp mới và sao chép các trọng số tương ứng từ mô hình đã train với phạt (không cần thư viện thưa).

- **Fine-tune** mô hình hẹp để phục hồi. Paper nhấn mạnh sau fine-tune mô hình hẹp thường *bằng hoặc tốt hơn* mô hình gốc.

```
Network slimming
1. Khởi tạo gamma = 0.5 (mọi kênh)
2. Huấn luyện với L = Loss + λ Σ|γ|     (subgradient / SGD)
3. Chọn ngưỡng = phân vị p% của mọi |γ| trong toàn mạng
4. Xóa kênh có γ < ngưỡng (xóa filter vào, kernel ra, tham số BN tương ứng)
5. Fine-tune mô hình hẹp
6. (Multi-pass) Lặp 2–5 trên mô hình hẹp
```

3.4 Kết nối chéo tầng và cấu trúc pre-activation (ResNet, DenseNet)

Trong mạng có BN **trước** conv (pre-activation) và nhiều tầng đọc cùng đầu ra, sparsity xuất hiện ở **đầu vào** của tầng: tầng đó chỉ dùng một tập con kênh nhận được. Để thu được lợi ích tham số/tính toán khi suy luận, chèn một **channel selection layer** để che các kênh không quan trọng.

3.5 Multi-pass

Sau một vòng (huấn luyện có phạt → cắt → fine-tune) ta có mạng hẹp, và có thể áp lại toàn bộ quy trình để được mạng gọn hơn nữa (nét đứt trong Hình 2).

---

4. Thí nghiệm và kết quả chính

4.1 Thiết lập

- **Dữ liệu:** CIFAR-10/100 (32×32; validation 5000 ảnh tách khỏi train để tìm λ trên CIFAR-10), SVHN (604 388 ảnh train, 6000 để validation; test 26 032), ImageNet (1.2M train / 50k val), MNIST.
- **Mạng:** VGGNet (biến thể CIFAR), ResNet-164 (pre-activation bottleneck), DenseNet-40 (growth rate 12); ImageNet: VGG-A 11 tầng (8 conv + 3 FC) có BN, bỏ dropout; MNIST: mạng FC 3 tầng như SSL.
- **Huấn luyện:** SGD; CIFAR: batch 64, 160 epoch (SVHN: 20); ImageNet và MNIST: 60 và 30 epoch, batch 256; LR 0.1 giảm 10× tại 50% và 75% (CIFAR/SVHN) hoặc 1/3 và 2/3 (ImageNet/MNIST); weight decay 10^{-4}, Nesterov 0.9. **Khởi tạo mọi γ = 0.5** (cho baseline tốt hơn mặc định 1).
- **λ:** grid search $10^{-3}, 10^{-4}, 10^{-5}$ trên validation CIFAR-10. VGGNet $\lambda = 10^{-4}$; ResNet và DenseNet $\lambda = 10^{-5}$; VGG-A ImageNet $\lambda = 10^{-5}$.
- **Fine-tune:** CIFAR/SVHN/MNIST dùng cùng thiết lập huấn luyện; ImageNet chỉ **5 epoch với LR** 10^{-3} (vì giới hạn thời gian).

4.2 CIFAR và SVHN (Bảng 1)

Dataset	Mô hình	Lỗi test (%)	Tham số	Cắt tham số	FLOPs	Cắt FLOPs
CIFAR-10	VGGNet baseline	6.34	20.04M	-	7.97×10^8	-
	VGGNet 70% pruned	6.20	2.30M	88.5%	3.91×10^8	51.0%
	DenseNet-40 baseline	6.11	1.02M	-	5.33×10^8	-
	DenseNet-40 40%	5.19	0.66M	35.7%	3.81×10^8	28.4%
	DenseNet-40 70%	5.65	0.35M	65.2%	2.40×10^8	55.0%
	ResNet-164 baseline	5.42	1.70M	-	4.99×10^8	-

Dataset	Mô hình	Lỗi test (%)	Tham số	Cắt tham số	FLOPs	Cắt FLOPs
	ResNet-164 40%	5.08	1.44M	14.9%	3.81×10^8	23.7%
	ResNet-164 60%	5.27	1.10M	35.2%	2.75×10^8	44.9%
CIFAR-100	VGGNet baseline	26.74	20.08M	-	7.97×10^8	-
	VGGNet 50%	26.52	5.00M	75.1%	5.01×10^8	37.1%
	DenseNet-40 baseline / 40% / 60%	25.36 / 25.28 / 25.72	1.06M / 0.66M / 0.46M	37.5% / 54.6%		30.3% / 47.1%
	ResNet-164 baseline / 40% / 60%	23.37 / 22.87 / 23.91	1.73M / 1.46M / 1.21M	15.5% / 29.7%		33.3% / 50.6%
SVHN	VGGNet baseline / 60%	2.17 / 2.06	20.04M / 3.04M	84.8%	7.97×10^8 / 3.98×10^8	50.1%
	DenseNet-40 baseline / 40% / 60%	1.89 / 1.79 / 1.81	1.02M / 0.65M / 0.44M	36.3% / 56.6%		30.8% / 49.8%
	ResNet-164 baseline / 40% / 60%	1.78 / 1.85 / 1.81	1.70M / 1.46M / 1.12M	14.5% / 34.3%		31.1% / 54.9%

Nhận xét của paper: tham số giảm tới ~10× (VGGNet), FLOPs giảm thường khoảng 50%; ResNet-164 tiết kiệm ít hơn, tác giả đoán do cấu trúc bottleneck vốn đã "chọn kênh"; CIFAR-100 giảm ít hơn CIFAR-10/SVHN, có thể vì nhiều lớp hơn. Cắt vừa phải (40%) thường làm lỗi test thấp hơn baseline: ví dụ DenseNet-40 cắt 40% đạt **5.19%** (baseline 6.11%, gần 1 điểm), được tác giả giải thích là hiệu ứng điều chuẩn của ℓ_1 lên kênh (một dạng chọn đặc trưng trung gian) — đây là **giả thuyết**, không kiểm chứng riêng.

4.3 ImageNet (Bảng 2) và MNIST (Bảng 3)

ImageNet VGG-A	Baseline	50% pruned
Tham số	132.9M	23.2M (cắt 82.5%)
FLOPs	4.57×10^10	3.18×10^10 (cắt 30.4%)
Lỗi validation (%)	36.69	36.66

FLOPs chỉ giảm 30.4% vì chỉ **378 trên 2752 kênh** ở các tầng conv nặng tính toán bị cắt, trong khi **5094 trên 8192 neuron** của tầng FC nặng tham số bị cắt. Paper nhấn mạnh không mất accuracy trên ImageNet 1000 lớp, trong khi các phương pháp hiệu quả khác thường báo mất.

MNIST (FC 3 tầng)	Lỗi test	Tham số cắt	Neuron
Baseline	1.43%	-	784-500-300-10
SSL (Wen et al.)	1.53%	83.5%	434-174-78-10
Slimming	1.49%	84.4%	784-100-60-10

Với MNIST, ngưỡng toàn cục *đôi khi xóa hẳn một tầng*, nên họ cắt 80% neuron mỗi tầng ẩn.

4.4 Multi-pass (Bảng 4, VGGNet)

CIFAR-10 (baseline lỗi 6.34%): lỗi "trained" (sau huấn luyện có phạt) / "fine-tuned" (sau cắt) theo vòng:

Vòng	Trained	Fine-tuned	Tham số cắt	FLOPs cắt
1	6.38	6.51	66.7%	38.6%
2	6.23	6.11	84.7%	52.7%
3	5.87	6.10	91.4%	63.1%
4	6.19	6.59	95.6%	77.2%
5	5.96	7.73	98.3%	88.7%
6	7.79	9.70	99.4%	95.7%

(Trong bảng gốc, tỉ lệ cắt ở một hàng tương ứng với mô hình fine-tuned của hàng đó và mô hình trained của hàng kế tiếp. Văn bản paper xác nhận: ở vòng 5 mô hình huấn luyện đạt lỗi thấp nhất và đạt **20× giảm tham số và 5× giảm FLOPs** với lỗi thấp hơn baseline.) CIFAR-100 (baseline 26.74%): lỗi bắt đầu tăng sau vòng 3; tuy vậy vẫn cắt gần 90% tham số và gần 70% FLOPs mà không mất accuracy đáng kể. Với multi-pass VGGNet, họ thêm ràng buộc **tối đa 50% kênh mỗi tầng bị cắt mỗi vòng** vì mạng không có skip nên cắt hết một tầng sẽ phá mô hình.

4.5 Thời gian thực và bộ nhớ (Phụ lục B)

VGGNet "70% pruned" trên CIFAR-10, GPU GeForce 1080, Torch, batch 64:

	Thời gian/iter	Bộ nhớ	Lỗi test
Baseline	0.009 s	697 MB	6.34
70% pruned	0.005 s	499 MB	6.20

Thời gian giảm "xấp xỉ khớp" mức giảm FLOP; giảm bộ nhớ thấp hơn vì các tầng sâu (nhiều kênh bị cắt) có activation nhỏ, còn tầng nông (activation lớn) ít bị cắt (Phụ lục A, Bảng 5: tầng sâu bị cắt tới 93.8% kênh).

4.6 So sánh trực tiếp với 04 (Phụ lục C, Bảng 7)

Cùng tiêu chí và chiến lược cắt theo tầng của [Li et al.]:

	Lỗi test	Cắt tham số
CIFAR-10 baseline	6.34	-
Pruned (Li et al.)	6.88	88.5%
Pruned (slimming)	6.20	88.5%
CIFAR-100 baseline	26.74	-
Pruned (Li et al.)	28.36	76.0%
Pruned (slimming)	26.52	75.1%

---

5. Phân tích / ablation

- **Tỉ lệ cắt t (Hình 5, DenseNet-40 CIFAR-10, $\lambda = 10^{-5}$):** lỗi của mô hình đã fine-tune chỉ kém baseline khi tỉ lệ cắt vượt một ngưỡng, **khoảng trên 80%**. Đáng chú ý: *ngay cả mô hình huấn luyện có phạt, chưa fine-tune*, đã tốt hơn mô hình gốc (hiệu ứng điều chuẩn).

- λ **và phân bố** γ **(Hình 4, VGGNet):** $\lambda = 0$ cho phân bố tương đối phẳng; $\lambda = 10^{-5}$ tập trung gần 0 hơn; $\lambda = 10^{-4}$ đẩy gần như toàn bộ γ vào vùng sát 0. Paper coi quá trình này là chọn đặc trưng (feature selection) ở tầng trung gian.
- **Động học (Hình 6):** heatmap γ qua epoch của tầng conv 11 trong VGGNet: các kênh bắt đầu bằng nhau, sau đó một số sáng lên ("được chọn"), số khác tối đi.
- **Cấu trúc nén thu được (Phụ lục A):** tầng sâu bị cắt nhiều hơn. Trong VGGNet nén ở vòng 5: tầng 9–15 chỉ còn 32 kênh trên 512 (cắt 93.8%), tổng 5504 → 1034 kênh (cắt 81.2%), tham số/FLOP cắt 95.6%/77.2%, lỗi 5.96%.
- **Ngưỡng toàn cục vs theo tầng:** dùng ngưỡng toàn cục "cho đơn giản"; trên MNIST gây xóa nguyên tầng nên phải đổi sang cắt cố định 80% mỗi tầng. Đây là điểm yếu được paper ghi nhận.

6. Giới hạn

Do tác giả nêu:

- Ngưỡng toàn cục có thể xóa trọn một tầng (MNIST); với mạng không có skip connection (VGG), multi-pass phải thêm ràng buộc tối đa 50% kênh mỗi tầng mỗi vòng.
- ResNet-164 và CIFAR-100 tiết kiệm kém hơn (giả thuyết: bottleneck; nhiều lớp hơn).
- Cắt quá mạnh thì không phục hồi được (CIFAR-100 multi-pass: lỗi tăng từ vòng 3, Bảng 4; DenseNet-40: trên ngưỡng 80%, Hình 5).
- Trên ImageNet, tiết kiệm FLOPs thấp (30.4%) vì kênh bị cắt chủ yếu ở FC chứ không ở conv.
- Fine-tune ImageNet chỉ 5 epoch do giới hạn thời gian.

Nhận xét:

- Cần **huấn luyện lại từ đầu với phạt** (160 epoch CIFAR, 60 epoch ImageNet): tốn kém khi chỉ có một mô hình pretrain khổng lồ; không phải one-shot post-training.
- γ nhỏ không bảo đảm kênh vô dụng nếu sau đó là conv với trọng số lớn, hoặc β lớn (kênh tạo ra hằng số vẫn mang thông tin offset). Paper không bàn β. Tiêu chí dựa trên γ chỉ phù hợp mạng có BN; transformer dùng LayerNorm/RMSNorm với các quan hệ khác.
- Nhiều siêu tham số (λ, ngưỡng, số vòng); λ chọn bằng grid search nhỏ.
- Kết quả là một seed mỗi cấu hình; chênh lệch 0.1–0.3 điểm có thể nằm trong nhiễu.
- So với baseline "mạng hẹp train từ đầu" thiếu trong bản đã đọc (liên quan 11).
- Chỉ ảnh phân loại; không có kết quả với chuỗi/ngôn ngữ.
- Ghi nhận sự tăng tốc thật dựa trên một thí nghiệm batch 64 trên CIFAR; chưa có batch-1.

7. Liên hệ với các bài khác

- **04 Li et al.:** baseline trực tiếp (Bảng 7). 04 chọn filter bằng ℓ_1 trọng số sau train; 05 học tiêu chí trong lúc train bằng γ.
- **02 Han:** nêu bất lợi của weight pruning (cần phần cứng thưa, không giảm bộ nhớ runtime).
- **06 ThiNet, 07 Molchanov:** các tiêu chí khác cho channel; so sánh với γ.
- **08–11:** Lottery Ticket và "Rethinking": mô hình hẹp thu được từ slimming có thể train từ đầu được không?
- **13/14 (head), 16 Movement pruning, 17 CoFi:** CoFi dùng mặt nạ ℓ_0 cho head, neuron, hidden dimension — cùng tư duy "cổng nhân vào đơn vị + phạt thưa", nâng cấp từ ℓ_1 sang ℓ_0 khả vi (preface 6.6).
- **26 LLM-Pruner, 28 FLAP:** cắt neuron/head cho LLM; không cần train lại với phạt như 05.
- **20 DepGraph:** xử lý tự động các nhóm kênh bị ràng buộc (ở đây 05 xử lý bằng channel selection layer).
- **19 HALP, 12 (Blalock):** nhấn mạnh phải đo latency; Phụ lục B của 05 là ví dụ nhỏ.

8. Gợi ý áp dụng

- **Ý tưởng đáng mượn**: *cổng nhân vô hướng γ_j cho mỗi neuron MLP / head / channel ViT* + phạt ℓ_1 (hoặc ℓ_0 như CoFi) trong **giai đoạn distillation**: vừa học KD vừa để mạng tự chọn neuron cần giữ. Sau đó cắt theo ngưỡng và tinh chỉnh tiếp.
- **Cái cần thay đổi**: Với transformer, "BN γ" không có sẵn; cổng phải được **thêm tay** (một vector scale nhân sau activation của MLP, hoặc dùng scale của RMSNorm nếu thích hợp). Hệ số RMSNorm thuộc residual stream (chung cho mọi neuron) nên không đại diện neuron riêng; vì vậy nên thêm cổng mới khởi tạo bằng 1 (không phải 0.5).
- Giả thuyết cần kiểm chứng; không phải kết quả của paper.

9. Thí nghiệm gợi ý

1. **Cổng ℓ_1 cho neuron MLP khi distill**: thêm vector $g \in \mathbb{R}^m$ (khởi tạo 1) nhân sau activation MLP của một số block; loss = KD + $\lambda\|g\|_1$; so với baseline cắt cùng số neuron theo ℓ_1 trọng số (04) hoặc theo activation. Đo CER và loss KD sau cùng số bước. Quét $\lambda \in \{10^{-5}, 10^{-4}, 10^{-3}\}$.
2. **Phân bố cổng**: vẽ histogram g theo từng block (giống Hình 4) và xem block nào bị cắt nhiều nhất — có trùng với bản đồ độ nhạy từ thí nghiệm của 04 không? Nếu hai kết quả mâu thuẫn, ngưỡng toàn cục không phù hợp.
3. **Ngưỡng toàn cục vs tối thiểu mỗi tầng**: cắt 50% neuron toàn mạng bằng g; so sánh (a) ngưỡng toàn cục thuần túy và (b) có ràng buộc giữ ≥ 30% mỗi block. Đo xem (a) có xóa nguyên một block không.

10. Tóm tắt công thức / khái niệm

Khái niệm	Nội dung	Vị trí
Mục tiêu	$L = \sum \ell(f(x, W), y) + \lambda \sum_\gamma \lvert\gamma\rvert$	Eq. 1
BN	$\hat{z} = (z_{in} - \mu_B)/\sqrt{\sigma_B^2 + \epsilon}$, $z_{out} = \gamma\hat{z} + \beta$	Eq. 2
Scale factor	dùng γ của BN, khởi tạo 0.5	Mục 3–4
Ngưỡng	phân vị toàn cục của $\lvert\gamma\rvert$ (vd 70%)	Mục 3
Multi-pass	lặp train-có-phạt → cắt → fine-tune	Hình 2
Channel selection layer	cho mạng pre-activation (ResNet, DenseNet)	Mục 3
Kết quả đỉnh	VGGNet CIFAR-10: −88.5% tham số, −51.0% FLOPs, lỗi 6.20 (vs 6.34); multi-pass vòng 5: ×20 tham số, ×5 FLOPs; VGG-A ImageNet: −82.5% tham số, −30.4% FLOPs, lỗi 36.66 vs 36.69	Bảng 1, 2, 4

11. Kết luận ngắn

Network Slimming biến việc chọn kênh thành một phần của huấn luyện: chỉ cần phạt ℓ_1 lên γ của BN rồi cắt theo ngưỡng toàn cục, ta được mô hình dense hẹp hơn, nhỏ hơn về tham số, activation và FLOPs, chạy trên phần cứng thường, và thường không mất (đôi khi cải thiện) độ chính xác. Bài này là mẫu của họ phương pháp "cổng học được + phạt thưa" trải dài đến CoFi (17); điểm yếu là phải huấn luyện lại, phụ thuộc BN và ngưỡng toàn cục dễ xóa nguyên tầng.

Nguồn: https://arxiv.org/abs/1708.06519

Bài 06

ThiNet: A Filter Level Pruning Method for Deep Neural Network Compression (2017)

Tác giả: Jian-Hao Luo, Jianxin Wu, Weiyao Lin (Nanjing University; Shanghai Jiao Tong University).
Venue / arXiv: arXiv:1707.06342 (20/07/2017). Bản text không nêu venue; thí nghiệm chạy trên Caffe.
Nguồn: https://arxiv.org/abs/1707.06342
Chủ đề: Structured pruning ở mức filter cho CNN; chọn channel bằng *tái tạo đầu ra của layer kế tiếp* (data-driven), thuật toán tham lam + hồi quy bình phương tối thiểu.
Vị trí trong lộ trình: Đọc sau 04 (Pruning filters, chấm điểm filter bằng tổng |w|) và 05 (Network slimming, chấm điểm bằng hệ số BN). Bài này đổi câu hỏi từ "filter nào có trọng số nhỏ?" sang "bỏ filter này thì *đầu ra của layer sau* sai bao nhiêu?". Đây là tổ tiên trực tiếp của mọi phương pháp "cắt rồi bù tuyến tính". Bài kế: 07 (Molchanov: ước lượng importance bằng Taylor).

> Ghi chú học tập bằng tiếng Việt, không phải bản dịch.

1. Bối cảnh

Vấn đề: VGG-16 có 138.34M tham số (>500MB) và cần 30.94 tỷ FLOPs cho một ảnh; khó chạy trên thiết bị nhỏ.

Các hướng có sẵn trước bài này:

- **Unstructured pruning** (Han et al., bài 02): đặt từng trọng số nhỏ về 0. Mô hình thưa nhưng *ngẫu nhiên*; thư viện thông thường không tăng tốc được, truy cập bộ nhớ kém cục bộ nên đôi khi còn chậm hơn.
- **Group-sparsity / SSL** (Lebedev, Wen et al.): thêm regularizer để cả nhóm trọng số về 0, nhưng cấu trúc mạng bị phá, cần thư viện chuyên dụng.
- **Filter-level pruning**: bỏ nguyên filter. *Filter* = một bộ trọng số $C \times K \times K$ sinh ra một channel đầu ra; bỏ nó thì mạng chỉ "mỏng đi" nhưng vẫn là CNN chuẩn. Các tiêu chí trước đó: tổng |trọng số| (Li et al., bài 04), APoZ (tỷ lệ số 0 sau ReLU, Hu et al.), Taylor (Molchanov et al., bài 07).

Điểm chung của các tiêu chí cũ là chúng chấm điểm filter bằng thống kê **của chính layer đó**. Bài báo cho rằng điều này chưa trực tiếp liên quan đến việc mạng còn đúng hay không.

2. Ý tưởng chính

Một filter ở layer i sinh ra một channel; channel đó là *đầu vào* của lớp $i+1$. Nếu ta có thể xấp xỉ đầu ra của lớp $i+1$ chỉ bằng một tập con các channel đầu vào, thì các channel còn lại (và filter sinh ra chúng ở layer i) bỏ được. Vì vậy **cần dùng layer $i+1$ để quyết định cắt layer i**. Phát biểu thành bài toán tối ưu: chọn tập channel nhỏ nhất sao cho tổng các đóng góp còn lại tái tạo đầu ra layer $i+1$ gần nhất. Giải bằng tham lam, rồi nhân mỗi channel giữ lại với một hệ số học bằng bình phương tối thiểu (khởi tạo tốt cho fine-tune), và tinh chỉnh từng layer một.

3. Phương pháp chi tiết

3.1 Khung tổng thể (cắt từng layer)

Với mạng đã huấn luyện và tỷ lệ nén r định trước:

1. **Chọn filter** bằng thống kê từ layer $i+1$ (mục 3.2).

2. **Cắt** channel yếu ở đầu vào layer $i+1$ và filter tương ứng ở layer i. Cấu trúc mạng không đổi, chỉ ít filter/channel hơn.
3. **Fine-tune**: sau mỗi layer chỉ 1–2 epoch (tiết kiệm thời gian); sau khi cắt hết các layer mới fine-tune thêm nhiều epoch.
4. Lặp lại cho layer kế.

3.2 Chọn channel theo dữ liệu

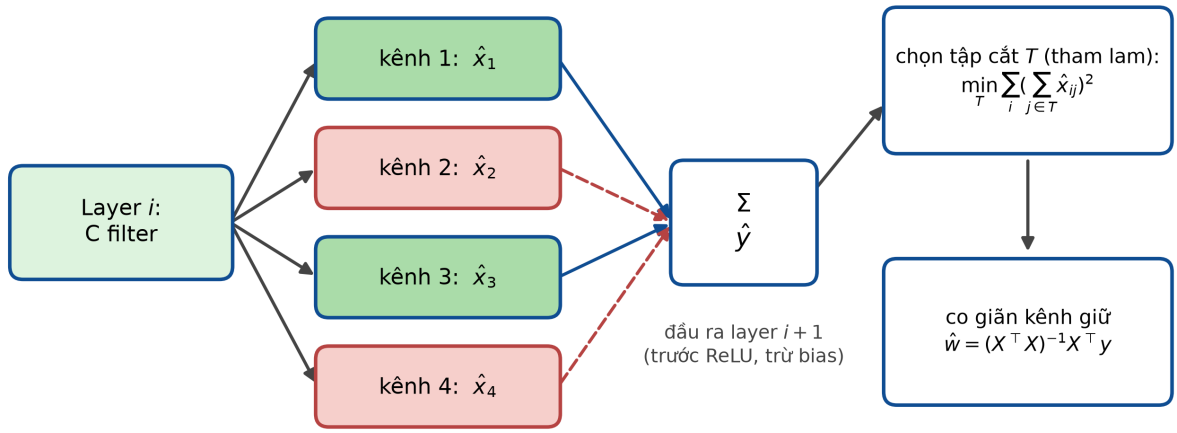
Ký hiệu convolution ở layer i là $\langle I_i, W_i, * \rangle$, với $I_i \in \mathbb{R}^{C\times H\times W}$, $W_i \in \mathbb{R}^{D\times C\times K\times K}$. Lấy một phần tử y của đầu ra layer $i+1$ (trước ReLU) tại một vị trí; nó bằng

$$y = \sum_{c=1}^{C} \sum_{k_1,k_2} \widehat{W}_{c,k_1,k_2}\, x_{c,k_1,k_2} + b.$$

Đặt $\hat{x}_c = \sum_{k_1,k_2} \widehat{W}_{c,k_1,k_2} x_{c,k_1,k_2}$ (đóng góp của channel c) và $\hat{y} = y - b$. Khi đó

$$\hat{y} = \sum_{c=1}^{C} \hat{x}_c.$$

Quan sát then chốt: các $\hat{x}_c$ phụ thuộc **riêng rẽ** từng channel đầu vào, nên đầu ra là *tổng* các đóng góp độc lập. Nếu tìm được tập S sao cho $\hat{y} = \sum_{c\in S} \hat{x}_c$ thì các channel ngoài S bỏ được. Điều đó không đúng chính xác cho mọi mẫu, nhưng có thể làm đúng *xấp xỉ* trên mẫu thu thập.



Xanh: kênh giữ. Đỏ nét đứt: kênh cắt (cùng filter ở layer i). Cắt sao cho tổng đóng góp bị bỏ gần 0.

Hình 29: Đầu ra layer $i+1$ là tổng đóng góp của từng channel đầu vào; chọn tập channel cắt rồi co giãn các channel còn lại (minh hoạ, tự tính)

Cách đọc hình: mỗi kênh ở giữa là một đóng góp $\hat{x}_c$ cộng vào $\hat{y}$. Kênh đỏ nét đứt bị cắt cùng filter sinh ra nó ở layer i, kênh xanh được giữ. Hai khối bên phải là hai bước của thuật toán: chọn tập T sao cho tổng đóng góp bỏ đi gần 0, rồi tính hệ số $\hat{w}$ để bù.

Lấy mẫu. Chạy forward, với mỗi ảnh lấy ngẫu nhiên vị trí/channel đầu ra để được các cặp $(\hat{\mathbf{x}}_i, \hat{y}_i)$, $\hat{\mathbf{x}} \in \mathbb{R}^C$. Trên ImageNet: 10 ảnh mỗi lớp, 10 mẫu mỗi ảnh, tổng 100,000 mẫu.

Bài toán (viết theo tập bị cắt T, nhanh hơn vì $|T| < |S|$):

$$\arg\min_{T} \sum_{i=1}^{m} \Big(\sum_{j \in T} \hat{x}_{i,j} \Big)^2 \quad \text{s.t. } |T| = C(1-r),\ T \subset \{1, \dots, C\}$$

Tương đương với việc giữ tập S làm $\sum_{j \in S} \hat{x}_{i,j}$ gần $\hat{y}_i$ nhất. Bài toán NP-hard, nên dùng tham lam (Algorithm 1): mỗi bước thử thêm từng channel i còn lại vào T, chọn channel làm tăng ít nhất giá trị mục tiêu.

```
Input : mẫu {(x̂_i, ŷ_i)}, tỷ lệ giữ r
T ← ∅ ; I ← {1..C}
while |T| < C·(1−r):
    best ← +∞
    for each j in I:
        v ← Σ_i ( Σ_{k in T∪{j}} x̂_{i,k} )²
        if v < best: best ← v ; j* ← j
    chuyển j* từ I sang T
return T          # các channel bị cắt
```

Paper thừa nhận nghiệm tham lam là dưới tối ưu nhưng "khoảng cách được bù bằng fine-tune", và cho biết đã thử homotopy (sparse coding) nhưng tham lam cho kết quả tốt hơn và nhanh hơn.

3.3 Bù bằng bình phương tối thiểu

Sau khi chọn xong, tái tạo thêm bằng cách gán trọng số w cho các channel giữ lại:

$$\hat{w} = \arg\min_{w} \sum_{i=1}^{m} \left(\hat{y}_i - w^\top \hat{\mathbf{x}}_i^* \right)^2 = (X^\top X)^{-1} X^\top y$$

với $\hat{\mathbf{x}}_i^*$ là mẫu sau khi chọn channel. $\hat{w}_c$ dùng để co giãn filter: $W_{:,c,:,:} \leftarrow \hat{w}_c W_{:,c,:,:}$. Đây là hồi quy tuyến tính cổ điển có nghiệm đóng; đóng vai trò khởi tạo tốt cho fine-tune.

3.4 Chiến lược cho từng loại mạng

- **VGG-16:** hơn 90% FLOPs nằm ở 10 layer đầu (conv1-1 đến conv4-3); các FC chiếm 86.41% tham số. Cắt 10 layer conv đầu để tăng tốc, thay FC bằng global average pooling (GAP).
- **ResNet:** các block trong cùng một nhóm phải cùng số channel để cộng residual, nên **chỉ cắt hai conv đầu của mỗi residual block**, giữ nguyên chiều đầu ra block và shortcut (Figure 3).

3.5 Ví dụ nhỏ (tự đặt, không có trong paper)

3 channel đầu vào cho đóng góp $\hat{x} = (2, -2, 0.1)$ tại một vị trí; cần cắt $|T| = 2$. Mục tiêu $(\sum_{j \in T} \hat{x}_j)^2$: $T = \{1, 2\}$ cho 0 (tối ưu, vì hai đóng góp lớn triệt tiêu nhau, tiêu chí tổng |w| sẽ không bao giờ cắt chúng). Tham lam đi từng bước: bước 1 chọn channel 3 (lỗi 0.01), bước 2 thêm channel 2 (lỗi 3.61) thay vì channel 1 (4.41). Kết quả $T = \{3, 2\}$ kém tối ưu, minh họa vì sao paper nói tham lam "dưới tối ưu" (ví dụ tự đặt).

4. Thí nghiệm và kết quả chính

Thiết lập: Caffe; ImageNet ILSVRC-12 (50k ảnh validation, single-view center crop), CUB-200 (11,788 ảnh, 200 loài chim) và Indoor-67 cho chuyển giao. Tất cả cắt layer-by-layer.

4.1 VGG-16 trên ImageNet (Table 1)

Mô hình	Top-1	Top-5	#Param	#FLOPs
Original	68.34%	88.44%	138.34M	30.94B

Mô hình	Top-1	Top-5	#Param	#FLOPs
ThiNet-Conv (cắt 10 conv, giữ 0.5)	69.80%	89.53%	131.44M	9.58B
Train from scratch (cùng cấu trúc ThiNet-Conv)	67.00%	87.45%	131.44M	9.58B
ThiNet-GAP	67.34%	87.92%	8.32M	9.34B
ThiNet-Tiny	59.34%	81.97%	1.32M	2.01B
SqueezeNet	57.67%	80.39%	1.24M	1.72B

Thời gian f./b. trên một M40, batch 32: Original 189.92/407.56 ms; ThiNet-Conv 76.71/152.05 ms; ThiNet-Tiny 29.51/55.83 ms.
ThiNet-Tiny dùng 5.05MB, đạt mức AlexNet (AlexNet: 57.2% / 80.3%), và tuy cần nhiều FLOPs hơn SqueezeNet, chạy thực tế nhanh hơn do cấu trúc đơn giản.

4.2 So sánh với phương pháp khác trên VGG-16 (Table 2)

Phương pháp	Top-1	Top-5	#Param giảm	#FLOPs giảm
APoZ-1	-2.16%	-0.84%	2.04x	≈1x
APoZ-2	+1.81%	+1.25%	2.70x	≈1x
Taylor-1	–	-1.44%	≈1x	2.68x
Taylor-2	–	-3.94%	≈1x	3.86x
ThiNet-WS	+1.01%	+0.69%	1.05x	3.23x
ThiNet-Conv	+1.46%	+1.09%	1.05x	3.23x
ThiNet-GAP	-1.00%	-0.52%	16.63x	3.31x

Abstract: 3.31x FLOPs, 16.63x nén, chỉ mất 0.52% top-5.

4.3 ResNet-50 (Table 3)

Mô hình	Top-1	Top-5	#Param	#FLOPs
Original	72.88%	91.14%	25.56M	7.72B
ThiNet-70	72.04%	90.67%	16.94M	4.88B
ThiNet-50	71.01%	90.02%	12.38M	3.41B
ThiNet-30	68.42%	88.30%	8.66M	2.20B

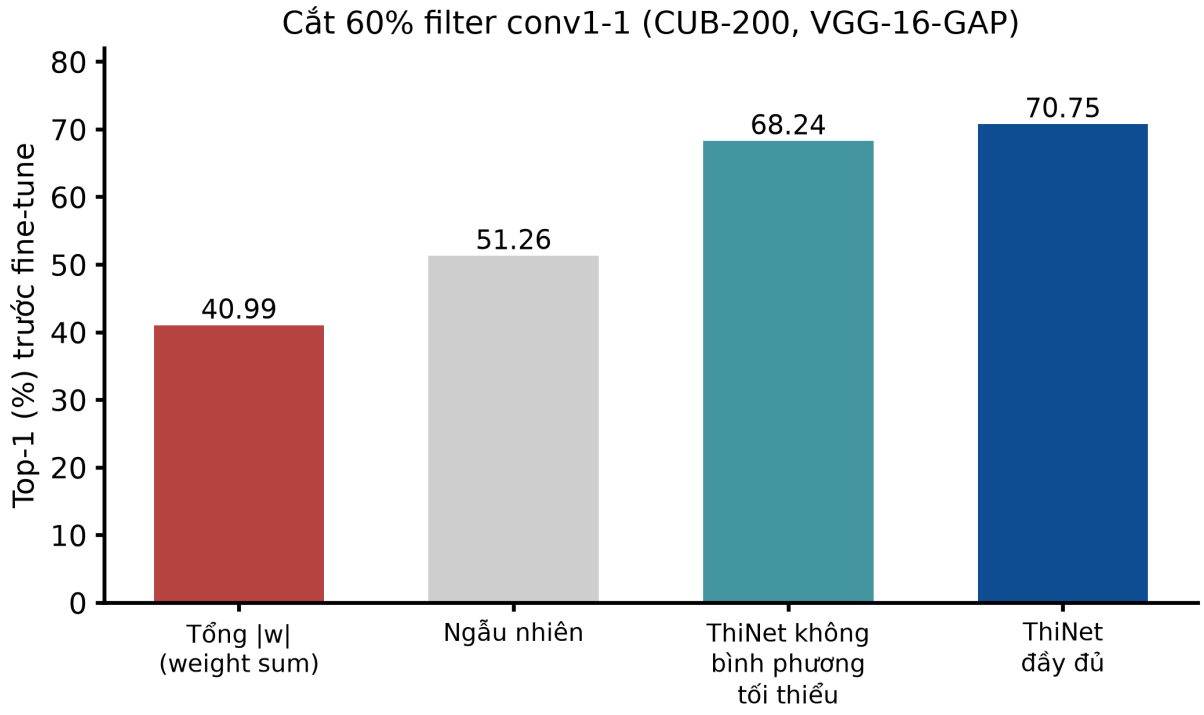
ThiNet-50 bỏ hơn một nửa tham số với khoảng 1% mất top-5. Nhưng FLOPs giảm không đổi thành tăng tốc tương ứng: thời gian f./b. chỉ từ 188.27/269.32 ms xuống 153.60/212.29 ms, vì layer phi-tensor (BN, pooling) chiếm hơn 40% thời gian suy luận GPU.

4.4 Chuyển miền (Table 4)

VGG-16 fine-tune trên CUB-200: 72.30% top-1. “FT & prune” (cắt trên dữ liệu nhỏ): 66.90%; train from scratch cùng cấu trúc: 44.27%; ThiNet-Conv 70.90%; ThiNet-GAP 69.43%; ThiNet-Tiny 65.45%; AlexNet 57.28%. Indoor-67: original 72.46%; FT & prune 64.70%; from scratch 38.81%; ThiNet-Conv 72.31%; ThiNet-GAP 70.22%; ThiNet-Tiny 62.84%; AlexNet 59.55%. Khuyến nghị của tác giả: cắt trên ImageNet *trước*, rồi mới fine-tune sang miền mới.

5. Phân tích / ablation

- **So sánh tiêu chí chọn (Figure 4, VGG-16-GAP trên CUB-200, trung bình 4 lần):** ThiNet cao nhất và ổn định hơn weight sum, APoZ, random. Số đọc từ biểu đồ không đáng tin, nhưng văn bản nêu một con số: cắt 60% filter conv1-1, *trước* fine-tune, weight sum cho 40.99% top-1, random 51.26%, ThiNet không bình phương tối thiểu 68.24%, ThiNet đầy đủ 70.75%.
- **Random khá mạnh nhưng không bền:** đôi khi hơn các heuristic, nhưng có lúc rất tệ; tác giả giải thích bằng distributed representation (khái niệm do nhiều neuron cùng biểu diễn), không dùng được trong thực tế.
- **Bình phương tối thiểu có ích**, nhất là khi tỷ lệ nén cao (ThiNet vs "ThiNet w/o $\hat{w}$").
- **Weight sum** (trọng số nhỏ ≠ ảnh hưởng nhỏ): cắt hàng loạt filter "nhỏ" cùng lúc làm mất độ chính xác mà fine-tune không phục hồi hết.
- **Cắt rồi fine-tune hơn train từ đầu** (67.00% vs 69.80% trên ImageNet; 44.27% vs 66.90% trên CUB-200). Lưu ý: paper chưa chỉnh lịch huấn luyện cho from-scratch; bài 11 sẽ xem lại điểm này.
- **Layer cuối mỗi nhóm** (conv1-2, conv2-2, conv3-3) nhạy hơn nên được fine-tune thêm với lr 10^{-4}.



Hình 30: Top-1 sau khi cắt 60% filter conv1-1 và trước fine-tune, theo bốn cách chọn filter (CUB-200, VGG-16-GAP) (vẽ lại từ số liệu trong ghi chú)

Cách đọc hình: cột càng cao thì độ chính xác càng ít bị mất ngay sau khi cắt. Tiêu chí tổng $|w|$ thấp hơn cả chọn ngẫu nhiên, còn ThiNet có và không có bước bình phương tối thiểu cách nhau khoảng 2,5 điểm. Số lấy từ câu văn của paper nêu trong mục này, không phải đọc từ biểu đồ.

6. Giới hạn

Do tác giả nêu:

- ResNet chỉ cắt hai conv đầu mỗi block; shortcut và conv cuối block chưa cắt ("khó, nếu không phải là không thể").
- Layer phi-tensor trên ResNet chiếm >40% thời gian, cần được tăng tốc.
- Tỷ lệ nén mỗi layer đặt sẵn, tham lam là dưới tối ưu; tương lai cần cách chọn channel tốt hơn.

Nhận xét:

- Chỉ đo trên CNN phân loại ảnh (VGG-16, ResNet-50); không có transformer, attention, norm có trạng thái.
- Mục tiêu tái tạo là *cục bộ một layer* và dựa vào ReLU đứng sau sum; với activation không đơn điệu hoặc có gating (GLU) thì phép tách tổng độc lập còn đúng nhưng phi tuyến phía sau làm sai lệch ánh xạ giữa lỗi tái tạo và lỗi nhiệm vụ.
- Tỷ lệ nén giống nhau cho mọi layer (0.5) là thô; paper không tối ưu phân bổ ngân sách giữa các layer.
- Lỗi tái tạo trong 100,000 mẫu từ ImageNet; với dữ liệu miền hẹp (OCR), phân phối mẫu quan trọng.
- Tăng tốc đo trong Caffe trên M40 với batch 32; batch-1 có thể khác.

7. Liên hệ với các bài khác

- **04 Pruning filters:** ThiNet-WS (dùng weight sum trong khung ThiNet) là đường cơ sở trực tiếp; Table 2 cho ThiNet-Conv hơn ThiNet-WS về độ chính xác.
- **05 Network slimming:** cùng cắt channel, nhưng slimming học điểm số khi huấn luyện (L1 lên hệ số BN); ThiNet là post-hoc, không cần huấn luyện lại từ đầu.
- **07 Molchanov 2019:** tiêu chí dựa trên Taylor của loss; ThiNet so với Taylor 2017 trong Table 2.
- **08/11 LTH và Rethinking:** bài 11 sẽ phản biện kết luận "cắt+fine-tune hơn train from scratch" của mục 4.4 bằng cách train from scratch đủ lâu.
- **22–23 OBC, SparseGPT; 29 SliceGPT:** các phương pháp LLM sau này cũng cắt xong rồi giải *bình phương tối thiểu* để bù lỗi lớp; ThiNet là một trường hợp đơn giản và sớm của ý tưởng đó.
- **26 LLM-Pruner, 28 FLAP, 38 Sheared LLaMA:** cũng cắt nhóm kênh/neuron có cấu trúc.

8. Gợi ý áp dụng

- Có thể dùng ThiNet làm *công cụ đo*: tỉ lệ (lỗi trước hồi quy − lỗi sau)/lỗi trước.

9. Thí nghiệm gợi ý

3. **Ablation hồi quy:** cùng tập bị cắt, có và không có bước $\hat{w}$, rồi distill cùng số bước; xem bước khởi tạo này còn giúp sau distillation không.

10. Tóm tắt công thức / khái niệm

Khái niệm	Ý nghĩa
Filter	Bộ trọng số $C \times K \times K$ tạo một channel đầu ra
Filter-level pruning	Bỏ nguyên filter, mạng giữ cấu trúc chuẩn
$\hat{y} = \sum_c \hat{x}_c$	Đầu ra layer $i+1$ (trừ bias) là tổng đóng góp từng channel
$\arg\min_T \sum_i (\sum_{j \in T} \hat{x}_{ij})^2$	Chọn tập bị cắt sao cho tổng đóng góp còn bỏ đi gần 0
Greedy (Alg. 1)	Thêm từng channel làm tăng lỗi ít nhất vào T
$\hat{w} = (X^\top X)^{-1} X^\top y$	Hệ số co giãn kênh còn lại (OLS)
GAP	Global average pooling, thay FC để giảm 86.41% tham số của VGG-16
APoZ	Tỷ lệ số 0 sau ReLU của một channel
ThiNet-Conv / GAP / Tiny	10 conv cắt giữ 0.5 / + bỏ FC / nén mạnh hơn (giữ 0.25, 5.05MB)

11. Kết luận ngắn

ThiNet cho thấy tiêu chí cắt tốt nên đo *tác động lên đầu ra lớp kế tiếp*, không phải độ lớn trọng số của chính layer; kết hợp chọn tham lam, bù OLS và tinh chỉnh từng layer cho 3.31x ít FLOPs và 16.63x nén trên VGG-16 với mất 0.52% top-5.

Link gốc: https://arxiv.org/abs/1707.06342

Bài 07

Importance Estimation for Neural Network Pruning (2019)

Tác giả: Pavlo Molchanov, Arun Mallya, Stephen Tyree, Iuri Frosio, Jan Kautz (NVIDIA).
Venue / arXiv: arXiv:1906.10771 (25/06/2019). Bản text không nêu venue; mã nguồn: github.com/NVlabs/Taylor_pruning.
Nguồn: https://arxiv.org/abs/1906.10771
Chủ đề: Structured pruning (filter/neuron) dựa trên khai triển Taylor bậc 1 và bậc 2 của *bình phương thay đổi loss*; so khớp với "oracle"; đặt cổng (gate) sau BatchNorm để chấm điểm mọi loại layer kể cả skip connection.
Vị trí trong lộ trình: Đọc sau 01 (OBD/OBS, bản gốc dùng đạo hàm bậc 2 làm điểm số), 04–06 (các tiêu chí "chủ nghĩa độ lớn", tái tạo của ThiNet). Bài này nâng tiêu chí dựa trên gradient lên mức dùng được ở quy mô ImageNet và chứng minh bằng *tương quan với oracle* rằng độ lớn trọng số là chỉ báo kém. Bài kế: 08 (Lottery Ticket) rồi 10 (SNIP, cũng dùng gradient×trọng số nhưng lúc khởi tạo).
Chương nền liên quan: nền 03: Độ quan trọng của trọng số

> Ghi chú học tập bằng tiếng Việt, không phải bản dịch. Số liệu lấy từ paper (một số bảng PDF bị lệch dòng, chỉ ghi những số đọc được rõ).

1. Bối cảnh

Muốn cắt một neuron/filter, ta cần biết **nó quan trọng đến đâu**. Các tiêu chí đã có:

- **Độ lớn** (|w|, ℓ2 norm của filter, hệ số scale γ của BatchNorm): rẻ, nhưng dựa trên niềm tin "trọng số nhỏ = ít quan trọng". Bài báo nghi ngờ niềm tin này (Mozer 1988 và LeCun 1990 cũng từng nghi ngờ).
- **OBD** (bài 01): điểm = đạo hàm bậc hai (đường chéo Hessian) × w^2. Giả định Hessian dương xác định; với mạng sâu huấn luyện bằng minibatch, đường chéo Hessian có phần tử âm, và Hessian tốn bộ nhớ.
- **ThiNet** (bài 06): tái tạo đầu ra layer sau theo từng lớp, tốn kém tính toán.
- **Taylor-output** (Molchanov 2017): khai triển theo *activation* đầu ra; cần lưu activation (tốn bộ nhớ), phải chuẩn hóa heuristic giữa các lớp, hỏng với skip connection.
- Nhiều phương pháp cần **sensitivity analysis** (thử cắt từng layer để chọn tỷ lệ riêng cho layer).

Thuật ngữ nhanh:

- **Oracle**: tiêu chuẩn "vàng": thật sự đặt từng neuron về 0 và đo loss tăng bao nhiêu (đắt vì phải chạy M lần).
- **Hệ số tương quan hạng Spearman**: đo hai bảng xếp hạng giống nhau thế nào (1 = giống hệt).
- **Gate**: hệ số nhân z_m khởi tạo 1 đặt sau neuron; tắt neuron tức đặt $z_m = 0$.

2. Ý tưởng chính

Định nghĩa độ quan trọng của một neuron là **bình phương thay đổi loss** nếu bỏ nó đi. Tính trực tiếp thì quá đắt, nên khai triển Taylor quanh trọng số hiện tại: ở bậc 1 chỉ cần gradient g (sẵn có sau backprop) và trọng số w, điểm = $(g \cdot w)^2$. Đặt "cổng" sau mỗi lớp BatchNorm để cùng một công thức dùng được cho mọi loại layer, kể cả skip connection. Cắt dần từng đợt nhỏ (khoảng 2% filter mỗi lượt) và tính lại điểm sau mỗi lượt. Kết quả: điểm số tương quan >93% (Spearman) với oracle, thang đo giống nhau giữa các lớp nên không cần phân tích độ nhạy theo lớp.

3. Phương pháp chi tiết

3.1 Độ quan trọng và hai bậc xấp xỉ

Mạng có tham số $W = \{w_1, \ldots, w_M\}$, lỗi $E(D, W)$. Độ quan trọng của tham số w_m (giả định độc lập, i.i.d.):

$$I_m = \left(E(D, W) - E(D, W \mid w_m = 0)\right)^2 \qquad (3)$$

Khai triển Taylor bậc 2 quanh W:

$$I_m^{(2)}(W) = \left(g_m w_m - \tfrac{1}{2} w_m H_m W\right)^2 \qquad (4)$$

với $g_m = \partial E / \partial w_m$, H_m là hàng thứ m của Hessian. Chỉ giữ bậc 1:

$$\boxed{I_m^{(1)}(W) = (g_m w_m)^2} \qquad (5)$$

Vì sao dùng *bình phương*: không giả định mọi neuron đều làm loss tăng (loại bỏ có thể làm loss giảm); paper cho rằng bình phương tương quan với oracle tốt hơn dấu (signed) của OBD và cho kết quả cắt tốt hơn.

3.2 Cho nhóm tham số (một filter)

Với tập S (các trọng số của một filter), có hai cách gộp:

$$I_S^{(1)}(W) = \Big(\sum_{s \in S} g_s w_s\Big)^2 \text{ (group contribution, Eq. 7)}, \qquad \hat{I}_S^{(1)}(W) = \sum_{s \in S} (g_s w_s)^2 \text{ (tổng riêng lẻ, Eq. 8)}$$

Cổng. Thêm $z_m = 1$ sau neuron. Thì

$$I_m^{(1)}(z) = \Big(\frac{\partial E}{\partial z_m}\Big)^2 = \Big(\sum_{s \in S_m} g_s w_s\Big)^2 = I_{S_m}^{(1)}(W)$$

tức là điểm của cổng chính là "group contribution" của các tham số trước đó; cổng cũng gộp luôn weight và bias, không tham gia tối ưu, tiện tính.

Cổng đặt **ngay sau BatchNorm** (mặc định): với feature map m có γ_m, β_m của BN,

$$I_m^{(1)}(\gamma_m, \beta_m) = \Big(\gamma_m \frac{\partial E}{\partial \gamma_m} + \beta_m \frac{\partial E}{\partial \beta_m}\Big)^2.$$

Chỉ cần $\partial E / \partial \gamma$ và $\partial E / \partial \beta$, có sẵn trong backprop thông thường, nên rất rẻ.

3.3 Diễn giải thông tin

Đặt $h_m = \partial E / \partial z_m$. Giả sử tại hội tụ $E[h_m] = 0$, thì $\mathrm{Var}(h_m) = E[h_m^2] = I^{(1)}(z)$: điểm số chính là **phương sai của gradient**, và nếu E là log-likelihood thì đó là đường chéo của ma trận thông tin Fisher kỳ vọng. Tức là tiêu chí này đồng thời đo mức độ "nhạy" của log-likelihood với neuron đó.

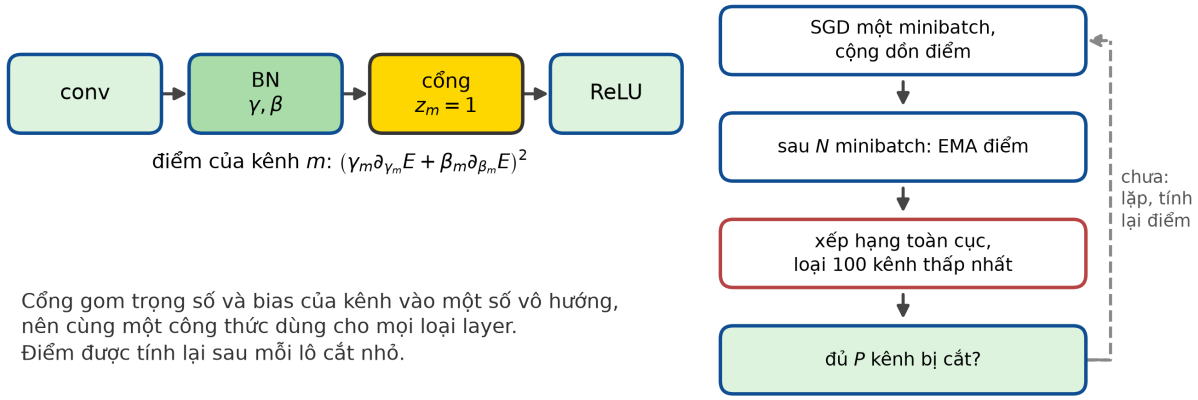
3.4 Thuật toán (Section 3.1)

```
Input: mạng đã huấn luyện, số neuron cần cắt tổng cộng P, lr nhỏ
repeat (trong fine-tune):
    for mỗi minibatch:
        tính gradient, cập nhật trọng số (SGD)
```

```
        cộng dồn điểm quan trọng  I_m  (từ gradient trung bình minibatch)
    sau N minibatch (ImageNet: 30; CIFAR: 10):
        trung bình điểm theo thời gian (EMA, momentum 0.9)
        loại N neuron điểm thấp nhất (ImageNet: 100 neuron mỗi lượt; ≈2% filter ban đầu)
until đạt P neuron đã cắt (hoặc loss vượt ngưỡng chịu được)
reset momentum, fine-tune tiếp
```

Chi tiết quan trọng:

- Hessian dùng xấp xỉ đường chéo; trên ImageNet không tính được (hết bộ nhớ), nên chỉ dùng bậc 1 ("Taylor FO").
- **Group contribution (Eq. 7)** tương quan rất thấp với oracle khi tập S lớn vì kỳ vọng gradient về 0 khi hội tụ, nên với filter không dùng cổng thì cộng điểm riêng lẻ (Eq. 8).
- Điểm tính trên toàn bộ mạng với cùng một thang, nên **xếp hạng toàn cục** và không cần chọn tỷ lệ cắt từng layer.
- Cắt theo lô cố định và fine-tune hơn là "cắt liên tục chừng nào loss còn dưới ngưỡng" (cách sau rơi vào cực tiểu địa phương, kết quả hơi tệ hơn).



Hình 31: Cổng đặt sau BatchNorm và vòng lặp cắt dần: cộng dồn điểm, xếp hạng toàn cục, loại một lô nhỏ rồi tính lại (minh hoạ, tự tính)

Cách đọc hình: bên trái, cổng z_m nhận gradient của cả γ và β nên điểm của một kênh chỉ cần các đạo hàm có sẵn trong backprop. Bên phải, mỗi lượt chỉ loại một lô nhỏ (khoảng 2% filter) rồi tiếp tục huấn luyện, vì vậy điểm luôn được tính lại sau khi mạng đã thích nghi.

3.5 Ví dụ nhỏ (tự đặt)

Hai filter A, B, mỗi filter 2 trọng số. A: $g = (0.1, -0.1)$, $w = (1, 1)$ nên tổng $g{\cdot}w = 0$; Eq.7 cho điểm 0 (nhóm triệt tiêu nhau), Eq.8 cho $0.01 + 0.01 = 0.02$. B: $g = (0.2, 0.2)$, $w = (0.1, 0.1)$ cho Eq.7 $= 0.04^2 = 0.0016$, Eq.8 $= 0.0004 + 0.0004$. Tiêu chí |w| sẽ chọn cắt B (trọng số nhỏ), Eq.7 chọn cắt A, hai ra kết luận khác nhau; đây là điều khiến chọn công thức quan trọng.

4. Thí nghiệm và kết quả chính

4.1 CIFAR-10: so với oracle (không fine-tune)

- **LeNet3** (73% test acc): oracle tổ hợp (thử mọi tổ hợp k neuron) không tốt hơn đáng kể oracle tham lam khi cắt ít neuron, nên các thí nghiệm sau dùng oracle tham lam. Kết quả: oracle > Taylor SO $\gtrsim$ Taylor FO; **weight-based không hơn cắt ngẫu nhiên**; OBD ngang oracle.

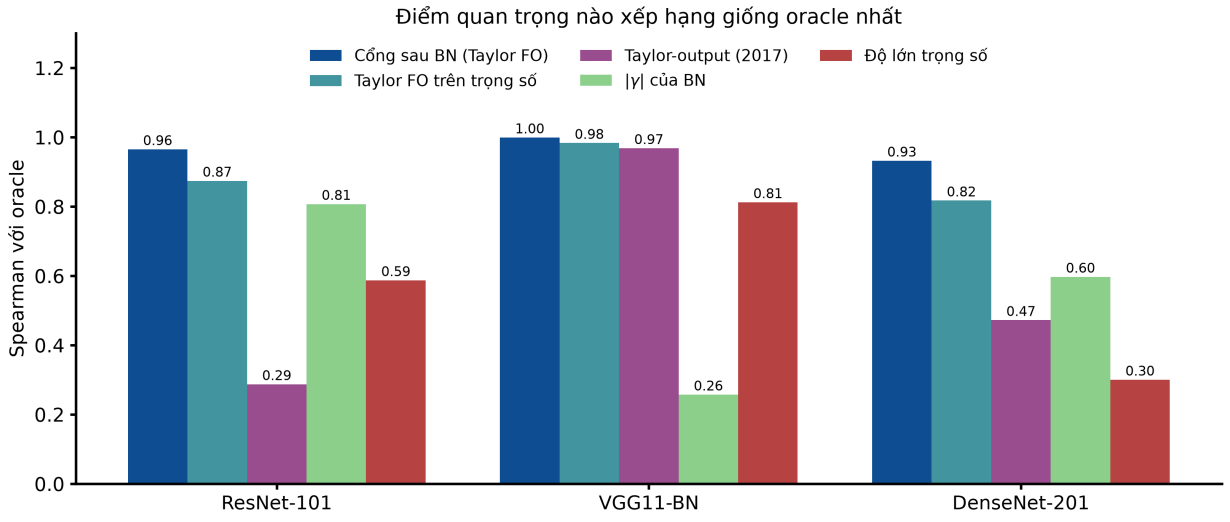
- **ResNet-18** pre-activation (94.79% acc), Spearman với oracle (Table 1): đặt cổng sau BN2: Taylor FO 0.955 (conv1), 0.811 (conv2), 0.924 (all layers); Taylor SO 0.968 / 0.849 / 0.957; trọng số conv Taylor FO 0.726 / 0.766 / 0.892; weight magnitude 0.660 / 0.677 / 0.861. Cổng đặt *trước* BN2 làm tương quan sụt mạnh (Taylor FO conv1 từ 0.955 xuống 0.275), nghĩa là BatchNorm ảnh hưởng lớn tới đóng góp của filter.
- OBD tương quan tốt nhưng cắt tệ: lúc đo tương quan, điểm OBD được bình phương; lúc cắt dùng giá trị có dấu, trong mạng sâu đường chéo Hessian có phần tử âm nên mất ổn định khi không fine-tune.
- Tính lại điểm sau mỗi lượt cắt quan trọng: biến thể "fixed" (không tính lại) kém rõ rệt.

4.2 ImageNet: tương quan (Table 2, trích)

Spearman tương quan với oracle, cột "All layers":

Mạng	Gate after BN (Taylor FO)	Taylor FO trên trọng số conv/linear	BN scale (	γ	, cách cũ)
ResNet-101	0.965	0.874	0.807	0.587	0.287
VGG11-BN	0.999	0.983	0.257	0.812	0.968
DenseNet-201	0.932	0.817	0.597	0.300	0.472

(Bảng PDF bị lệch dòng và còn các biến thể khác như "full gradient", "BN scale" có áp dụng phương pháp của tác giả; tôi chỉ giữ các cột đọc được chắc chắn, coi là gần đúng.) Văn bản: phương pháp đạt >93% Spearman trên cả ba mạng; Pearson >92% trên ResNet và DenseNet. Taylor-output tốt trên VGG nhưng thất bại với ResNet/DenseNet. Thêm cổng cho skip connection vẫn tương quan cao (ResNet-101: 0.946).



Hình 32: Tương quan hạng Spearman giữa các điểm quan trọng và oracle trên ImageNet, cột All layers của Table 2 (vẽ lại từ số liệu trong ghi chú)

Cách đọc hình: cột càng cao thì thứ tự kênh càng giống thứ tự của oracle. Cổng sau BN đứng đầu ở cả ba mạng, còn độ lớn trọng số thấp hơn nhiều ở ResNet-101 và DenseNet-201. Taylor-output tốt ở VGG11-BN nhưng tụt xuống 0,29 và 0,47 ở hai mạng có skip connection hoặc nối dày.

4.3 ImageNet: cắt và fine-tune (Table 3, 1-crop top-1 error)

Thiết lập: PyTorch, model pretrained, 4 GPU, batch 256, SGD lr 0.01 (hoặc 0.001), giảm 10 lần mỗi 10 epoch, tổng 25 epoch; mỗi 30 minibatch cắt 100 neuron.

Mạng	Mô hình	GFLOPs	Params (1e7)	Error %
ResNet-101	Không cắt	7.80	4.47	22.63
	Taylor-FO-BN-75%	4.70	3.12	22.65
	Taylor-FO-BN-55%	2.85	2.07	24.05
	Taylor-FO-BN-50%	2.47	1.78	24.62
	Taylor-FO-BN-40%	1.76	1.36	25.84
	BN-ISTA v1 / v2	4.47 / 3.69	2.36 / 1.73	24.73 / 25.44
ResNet-50	Không cắt	4.09	2.56	23.82
	Taylor-FO-BN-91%	3.27	2.26	23.57
	Taylor-FO-BN-81%	2.66	1.79	24.52
	Taylor-FO-BN-72%	2.25	1.42	25.50
	Taylor-FO-BN-56%	1.34	0.79	28.31
	ThiNet-70 / ThiNet-30	≈2.58 / ≈1.17	1.69 / 0.87	27.96 / 31.58
ResNet-34	Không cắt	3.64	2.18	26.69
	Taylor-FO-BN-82%	2.83	1.72	27.17
	Li et al.	2.76	1.93	27.80
VGG11-BN	Không cắt	7.61	13.29	29.16
	Taylor-FO-BN-50%	6.93	3.18	29.35
DenseNet-201	Không cắt	4.29	2.20	23.20
	Taylor-FO-BN-60%	3.02	1.25	23.49

Điểm nhấn: ResNet-101 giảm 40% FLOPs và 30% tham số chỉ mất 0.02% top-1. VGG11-BN cắt 76% tham số chỉ mất 0.19%, hơn "From scratch" (30.00%) và Slimming (31.38%) trên cùng cấu hình. DenseNet-201 cắt đến FLOPs bằng DenseNet-121 (Taylor-FO-BN-52%) cho lỗi thấp hơn 1.79%. Bảng PDF có nhãn "No pruning" lệch dòng trong ResNet-101; số "75%" tôi đối chiếu với Table 5.

4.4 Tốc độ suy luận (Table 5, ResNet-101, Tesla V100, ms)

Mô hình	GFLOPs	Error %	B16	B256
Không cắt	7.80	22.63	29.0	379.8
Taylor-FO-BN-50%	2.47	24.62	20.9	251.4
Taylor-FO-BN-40%	1.76	25.84	21.0	223.4
chỉ cắt skip, 22%	5.19	24.77	19.5	239.3

Chỉ cắt skip connection (bỏ 33% FLOPs) cho tăng tốc 1.59x, hơn cắt toàn bộ bỏ 68% FLOPs chỉ 1.51x: FLOPs không phải là độ trễ.

5. Phân tích / ablation

- **Cổng sau BN** > áp lên trọng số conv > BN scale > |w| về tương quan với oracle (Table 2).
- **Trung bình gradient minibatch** (group) hơn "full gradient" từng mẫu, dù paper chưa giải thích chắc chắn.
- **Cắt lặp > cắt một bước**: cắt 10,000 neuron một lần sau 3000 minibatch cho top-1 error 25.3%, kém Taylor-FO-BN-50% 0.68 điểm.
- **Lịch cắt:** iterative tốt hơn "single step" và "continuous" ở mọi epoch (supplement).
- **Bỏ cắt skip connection** (No skip): ResNet-50 56% tăng lỗi từ 28.31% lên 30.74%.

- **ResNet20/CIFAR-10 (Table 4):** Random và magnitude là tệ nhất; Taylor FO/SO và OBD khác nhau trong 1 độ lệch chuẩn (≈91.5–91.6% khi giữ ~30% neuron của "Prune A"), cao hơn BN-ISTA 90.9%. Với "train from scratch" cùng cấu trúc thì thấp hơn (89.6%). Tác giả nói phương pháp không bị "pruning paradox" của bài 11.
- Sau khi cắt, thứ hạng điểm giữa các layer trở nên "đồng đều hơn" (Figure 5).

6. Giới hạn

Do tác giả nêu:

- Không tính được Hessian trên ImageNet, chỉ dùng bậc 1.
- Chất lượng điểm số phụ thuộc vị trí cổng (sau BN tốt hơn nhiều so với trước BN).
- Cần số minibatch đủ giữa các lượt cắt để thống kê ổn định; số neuron cắt mỗi lượt phụ thuộc vào mức tương quan giữa chúng.

Nhận xét:

- Giả định độc lập giữa các neuron (i.i.d.); các neuron cùng cắt có thể tương tác (hiệu ứng nhóm), được xử lý phần nào bằng cách tính lại sau mỗi lượt.
- Toàn bộ thí nghiệm là CNN phân loại ảnh có BatchNorm; công thức cổng "sau BN" không áp dụng nguyên si cho transformer dùng LayerNorm/RMSNorm.
- Bậc 1 tại điểm đã hội tụ có gradient kỳ vọng ≈ 0; paper dựa vào bình phương của gradient từng minibatch (biến thiên), tức là điểm gần như đo phương sai của gradient chứ không đo đúng "thay đổi loss"; với dữ liệu hiệu chỉnh nhỏ, điểm số có thể nhiễu.
- Đánh giá chủ yếu bằng độ chính xác sau khi fine-tune 25 epoch bằng dữ liệu huấn luyện đầy đủ; chưa xét trường hợp ít dữ liệu hoặc chưng cất.

7. Liên hệ với các bài khác

- **01 OBD/OBS:** Taylor SO chính là phiên bản OBD có bình phương; paper cho thấy dấu của OBD gây bất ổn ở mạng sâu.
- **04, 05:** Li et al. (tổng |w|) và Slimming (BN γ) là các đường cơ sở dựa trên độ lớn; ở đây cả hai đều thua tiêu chí Taylor về tương quan oracle.
- **06 ThiNet:** thua ở Table 3 (ThiNet-70 27.96% vs Taylor-FO-BN-72% 25.50% với FLOPs và tham số gần nhau).
- **10 SNIP:** cùng ý "độ nhạy = gradient × trọng số", nhưng SNIP dùng ngay khi khởi tạo, ở mức từng kết nối.
- **11 Rethinking:** bài 11 đưa ra "pruning paradox" mà bài này tuyên bố không bị.
- **16 Movement pruning, 26 LLM-Pruner, 27 LLM Surgeon, 28 FLAP, 30 MaskLLM:** nhiều bài LLM dùng điểm quan trọng dựa trên gradient/Fisher (LLM-Pruner dùng Taylor trên các nhóm phụ thuộc); bài này là tổ tiên trực tiếp.
- **20 DepGraph:** cần nhóm các tham số phụ thuộc để áp "điểm theo nhóm" (như S trong Eq. 7/8).

8. Gợi ý áp dụng

- **Chọn neuron MLP/head để cắt bằng gradient** là hướng hợp lý thay vì |w|: bài này là bằng chứng (trên CNN) rằng độ lớn gần như không dự báo được tác động.
- **Cắt dần theo lượt + fine-tune** gần với kịch bản "cắt từng đợt rồi distill"; có thể kết hợp mỗi đợt cắt ~2% rồi vài trăm bước chưng cất.
- **Hạn chế thực dụng:** bài này không xét độ trễ batch-1; Table 5 nhắc rằng cắt kênh sâu hơn không nhất thiết nhanh hơn, cần đo thực.

9. Thí nghiệm gợi ý

1. **Oracle nhỏ:** trên một khối MLP của decoder, tắt từng neuron (hoặc từng nhóm 64 neuron) và đo ΔKL so với teacher; tính Spearman với (a) |w| (b) $(g \cdot w)^2$ trên cổng (c) Fisher đường chéo. Kiểm tra liệu |w| có tương quan thấp như trên CNN không.
2. **Điểm theo mục tiêu chưng cất:** so sánh điểm tính từ CE nhiệm vụ với điểm tính từ KL-teacher; xem chúng chọn cùng neuron đến mức nào.

10. Tóm tắt công thức / khái niệm

Khái niệm	Ý nghĩa
$I_m = (E(W) - E(W\|w_m = 0))^2$	Oracle: bình phương thay đổi loss khi bỏ w_m
$I_m^{(1)} = (g_m w_m)^2$	Taylor bậc 1: chỉ cần gradient và trọng số
$I_m^{(2)} = (g_m w_m - \frac{1}{2} w_m H_m W)^2$	Taylor bậc 2 (cần Hessian, dùng đường chéo)
Group contribution	$(\sum_{s \in S} g_s w_s)^2$, tương đương điểm của cổng
Tổng riêng lẻ	$\sum_{s \in S} (g_s w_s)^2$
Gate sau BN	$(\gamma\, \partial E/\partial \gamma + \beta\, \partial E/\partial \beta)^2$
Var của gradient	Điểm bậc 1 = phương sai gradient = đường chéo Fisher (giả sử $E[h] = 0$)
Taylor-FO-BN-X%	Phiên bản cắt đến khi còn lại X% neuron
Spearman	Tương quan hạng giữa điểm tiêu chí và oracle

11. Kết luận ngắn

Thí nghiệm cho thấy điểm quan trọng dựa trên gradient nhân trọng số, đặt tại cổng sau BatchNorm và tính lại sau mỗi đợt cắt nhỏ, bám sát oracle (Spearman >93%) và hơn rõ ràng các tiêu chí độ lớn, ThiNet, BN-ISTA trên ImageNet (ví dụ ResNet-101 mất 0.02% khi bỏ 40% FLOPs). Điều cần mang theo: dùng thước đo tác động lên loss thay cho độ lớn, xếp hạng toàn cục, cắt lặp, và luôn đo tốc độ thật.

Link gốc: https://arxiv.org/abs/1906.10771

Bài 08

The Lottery Ticket Hypothesis: Finding Sparse, Trainable Neural Networks (2018)

Tác giả: Jonathan Frankle, Michael Carbin (MIT CSAIL).
Venue / arXiv: arXiv:1803.03635 (v1 09/03/2018; bản dùng ở đây là v5, 04/03/2019); xuất bản tại ICLR 2019.
Nguồn: https://arxiv.org/abs/1803.03635
Chủ đề: Giả thuyết "vé số": mạng dày khởi tạo ngẫu nhiên chứa mạng con thưa (winning ticket) mà nếu huấn luyện riêng từ *khởi tạo gốc* thì đạt độ chính xác tương đương với số bước tương đương; thuật toán tìm vé bằng cắt độ lớn lặp + reset về khởi tạo.
Vị trí trong lộ trình: Đọc sau 02 (Han 2015, cắt độ lớn rồi fine-tune), 03 (cắt dần), 07. Bài này đặt câu hỏi khác: không phải "cắt thế nào để nén mô hình đã train" mà "mạng thưa có train được từ đầu không". Bài kế: 09 (linear mode connectivity, sửa lại phương pháp cho mạng lớn), 10 (SNIP, tìm mạng con *trước* khi train), 11 (Rethinking: phản biện).
Chương nền liên quan: nền 01: Vì sao mạng cắt được, nền 05: Cắt rồi huấn luyện lại

Ghi chú học tập bằng tiếng Việt, không phải bản dịch. Số liệu lấy từ paper (phần chính, một vài chi tiết phụ lục).

1. Bối cảnh

Pruning (LeCun 1990, Han 2015...) giảm hơn 90% tham số mà không hại độ chính xác. Câu hỏi đặt ra là: nếu mạng nhỏ đủ tốt, sao không *train mạng nhỏ đó từ đầu* cho rẻ? Kinh nghiệm thời đó: kiến trúc thu được từ cắt thì **khó huấn luyện từ đầu**, đạt độ chính xác thấp hơn mạng gốc. Bài báo trích Li et al. 2016 và Han et al. 2015 nói rằng cắt xong phải giữ lại trọng số đã học, không nên khởi tạo lại.

Hình 1 của bài báo minh họa: lấy mẫu ngẫu nhiên mạng con thưa của Lenet (MNIST) và Conv-2/4/6 (CIFAR-10); càng thưa thì học càng chậm và độ chính xác cuối càng thấp.

Khái niệm nền:

- **Unstructured pruning**: đặt từng trọng số về 0 bằng mặt nạ nhị phân $m \in \{0,1\}^{|\theta|}$; mạng thưa nhưng không nhanh hơn trên phần cứng thông thường.
- **Early-stopping iteration**: số bước tại đó loss validation nhỏ nhất; paper dùng làm đại diện cho "tốc độ học".
- **Khởi tạo Glorot**: cách khởi tạo Gaussian chuẩn hóa theo kích thước layer.

2. Ý tưởng chính

Một mạng dày khởi tạo ngẫu nhiên giống như mua rất nhiều vé số: trong đó có một số mạng con (mặt nạ cộng với các giá trị khởi tạo của chúng) "trúng" tức là huấn luyện riêng ra đã học nhanh và chính xác như mạng đầy đủ. Cách tìm vé: huấn luyện mạng, cắt các trọng số có độ lớn nhỏ nhất, rồi **đặt các trọng số còn lại về giá trị lúc khởi tạo ban đầu** (không phải giá trị đã huấn luyện, cũng không phải giá trị khởi tạo mới), và huấn luyện lại. Cùng mặt nạ nhưng khởi tạo ngẫu nhiên mới thì thua rõ rệt, nghĩa là *cấu trúc thưa một mình chưa đủ, giá trị khởi tạo cũng quan trọng*. Bài báo còn đưa ra một phỏng đoán chưa kiểm chứng: SGD thực chất tìm và huấn luyện một mạng con được khởi tạo tốt, và mạng dày dễ huấn luyện vì chứa nhiều ứng viên vé hơn.

3. Phương pháp chi tiết

3.1 Phát biểu giả thuyết

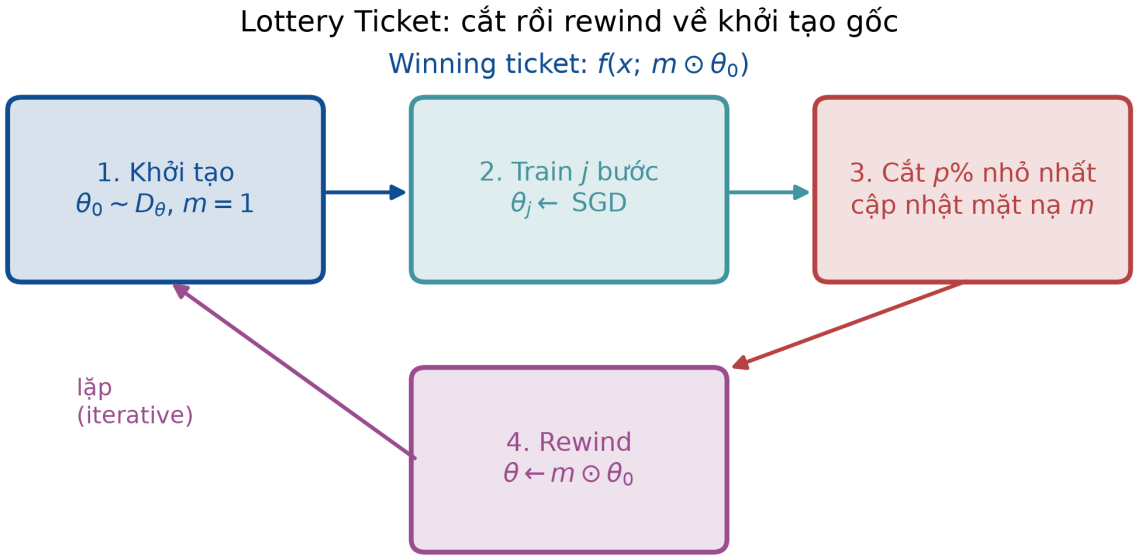
Cho mạng $f(x;\theta)$ với $\theta = \theta_0 \sim D_\theta$. Train bằng SGD đạt loss validation nhỏ nhất l tại bước j với độ chính xác test a. Train $f(x; m \odot \theta_0)$ với mặt nạ cố định m đạt l' tại bước j' với độ chính xác a'.

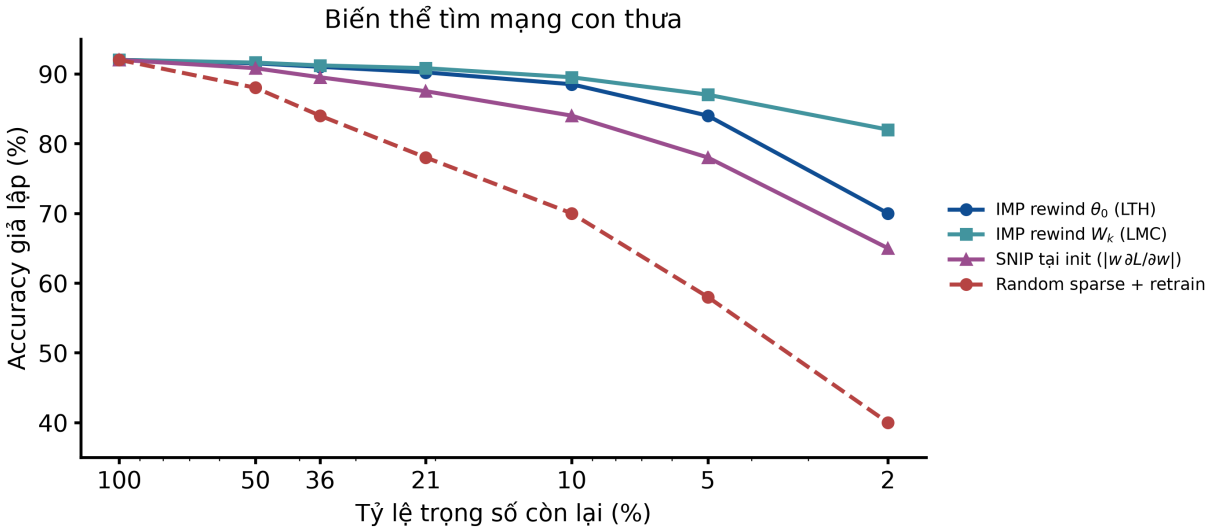
$$\exists\, m: \quad j' \le j \ \text{(thời gian tương đương)}, \quad a' \ge a \ \text{(độ chính xác tương đương)}, \quad \|m\|_0 \ll |\theta|$$

$f(x; m \odot \theta_0)$ gọi là **winning ticket**. Nếu thay θ_0 bằng khởi tạo mới $\theta_0'' \sim D_\theta$ thì thường mất tính chất này.

3.2 Thuật toán tìm vé (one-shot rồi lặp)

```
1. Khởi tạo ngẫu nhiên θ0 ~ Dθ;  mặt nạ m = 1.
2. Train f(x; m ⊙ θ) trong j bước  ->  θj
3. Cắt p% trọng số có |θj| nhỏ nhất (trong từng layer, hoặc toàn cục), cập nhật m
4. Reset trọng số còn lại về giá trị θ0  ->  vé f(x; m ⊙ θ0)
(iterative pruning: lặp bước 2-4 trong n vòng, mỗi vòng cắt p^(1/n)% trọng số còn sống,
 sau mỗi vòng reset về θ0)
```





Biến thể tìm mạng con: IMP rewind θ_0, rewind W_k, SNIP tại init, vs random sparse. (minh hoạ, tự tính)

Chu kỳ Lottery Ticket: train, cắt theo độ lớn, rewind về khởi tạo θ_0 rồi lặp để tìm winning ticket. (minh hoạ, tự tính)

- **One-shot**: train một lần, cắt p%, reset. Đơn giản nhưng ra vé lớn hơn.
- **Iterative**: lặp nhiều vòng; tìm được vé nhỏ hơn mà vẫn khớp mạng gốc. Tốn kém (train 15 lần liên tiếp trở lên).
- Phụ lục B so sánh hai biến thể lặp: *Strategy 1* (reset về θ_0 sau mỗi vòng cắt, rồi train lại) và *Strategy 2* (train tiếp với trọng số đã train); Strategy 1 cho độ chính xác validation cao hơn và dừng sớm nhanh hơn ở kích thước nhỏ hơn trên Lenet và Conv-2/4/6. Paper dùng Strategy 1.
- Cắt cục bộ theo layer cho Lenet, Conv-2/4/6 (Lenet: fc 20%, output cắt một nửa tốc độ, tức 10%; Conv-2: fc 20%, conv 10%...). Với VGG-19 và ResNet-18 dùng cắt toàn cục: lấy trọng số nhỏ nhất gộp mọi layer conv, vì layer đầu rất nhỏ (1728 và 36864 tham số so với 2.35M ở layer cuối) trở thành nút cổ chai khi cắt cùng tỷ lệ.

Ký hiệu: $P_m = \|m\|_0/|\theta|$ là tỷ lệ trọng số còn lại (ví dụ $P_m = 25\%$ nghĩa là đã cắt 75%).

3.3 Kiến trúc thí nghiệm (Figure 2)

Mạng	Dữ liệu	Số trọng số	Bước / batch	Optimizer	Tỷ lệ cắt mỗi vòng
Lenet 300-100-10	MNIST	266K	50K / 60	Adam 1.2e-3	fc 20%
Conv-2	CIFAR-10	4.3M / 38K conv	20K / 60	Adam 2e-4	fc 20%, conv 10%
Conv-4	CIFAR-10	2.4M / 260K	25K / 60	Adam 3e-4	fc 20%, conv 10%
Conv-6	CIFAR-10	1.7M / 1.1M	30K / 60	Adam 3e-4	fc 20%, conv 15%
ResNet-18	CIFAR-10	274K / 270K	30K / 128	SGD momentum, 0.1-0.01-0.001	conv 20% (toàn cục), fc 0%
VGG-19	CIFAR-10	20.0M	112K / 64	SGD momentum	conv 20% (toàn cục), fc 0%

(Bố cục bảng PDF lệch; cột "tỷ lệ cắt" tôi đọc theo đoạn "Pruning Rate" của Figure 2 và có thể sai ở Conv-6/ResNet-18/VGG-19; các số học tốc độ 0.1 hoặc 0.01 cho VGG/ResNet được nêu rõ ở Mục 4 của paper.)

4. Thí nghiệm và kết quả chính

4.1 Lenet / MNIST (Mục 2)

- Cắt lặp 20% mỗi vòng. Học nhanh dần khi P_m giảm từ 100% xuống 21%: lúc đó dừng sớm **sớm hơn 38%** so với mạng gốc. Độ chính xác test tăng hơn 0.3 điểm phần trăm ở $P_m = 13.5\%$, rồi giảm, về bằng mạng gốc ở $P_m = 3.6\%$.
- Tại iteration 50,000, huấn luyện đã đạt ~100% train accuracy nhưng vé vẫn hơn tới 0.35 điểm test, tức khoảng cách train–test nhỏ hơn: vé khái quát tốt hơn.
- **Reinit ngẫu nhiên** (giữ mặt nạ, đổi khởi tạo; 3 lần/vé): học ngày càng chậm hơn và độ chính xác rơi ngay sau khi cắt ít. Khi $P_m = 21\%$ vé dừng sớm nhanh hơn 2.51 lần so với bản reinit và chính xác hơn nửa điểm phần trăm. Độ chính xác của bản reinit bắt đầu tụt từ $P_m = 21.1\%$, còn vé giữ được đến $P_m = 2.9\%$.
- **One-shot**: khi $67.5\% \geq P_m \geq 17.6\%$ vé dừng sớm hơn mạng gốc; khi $95.0\% \geq P_m \geq 5.17\%$ độ chính xác test cao hơn mạng gốc; nhưng cắt lặp cho vé nhỏ hơn nữa.

4.2 Conv-2/4/6 / CIFAR-10 (Mục 3)

- Mô hình nhanh nhất so với mạng gốc: Conv-2 nhanh gấp 3.5 lần (ở $P_m = 8.8\%$), Conv-4 gấp 3.5 lần (9.2%), Conv-6 gấp 2.5 lần (15.1%).
- Độ chính xác test tăng tối đa: Conv-2 +3.4 điểm ($P_m = 4.6\%$), Conv-4 +3.5 điểm (11.1%), Conv-6 +3.3 điểm (26.4%). Cả ba vẫn trên độ chính xác gốc khi $P_m > 2\%$.
- Reinit: độ chính xác lúc dừng sớm ban đầu ổn định hoặc còn tăng cho Conv-2 và Conv-4, gợi ý ở mức cắt vừa phải thì chỉ riêng *cấu trúc* đã đủ giúp; rồi tụt nhanh hơn vé.
- **Dropout 0.5**: dropout tăng độ chính xác ban đầu 2.1, 3.0, 2.4 điểm (Conv-2/4/6); cắt lặp tăng thêm tới 2.3, 4.6, 4.7 điểm. Tìm vé vẫn được khi có dropout.

4.3 VGG-19 và ResNet-18 (Mục 4)

- **Cần learning-rate warmup** (tăng tuyến tính lr từ 0 tới lr gốc trong k bước) để tìm vé ở lr cao.
- **VGG-19** (20M tham số, 160 epoch = 112,480 bước): ở lr 0.1 cắt lặp *không* tìm ra vé, không hơn reinit. Ở lr 0.01 mạng con nằm trong 1 điểm của độ chính xác gốc khi $P_m \geq 3.5\%$ (nhưng chưa phải vé vì chưa khớp độ chính xác gốc). Với warmup $k = 10000$ ở lr 0.1: vé vượt độ chính xác ban đầu khi $P_m \geq 1.5\%$ (độ chính xác mạng gốc tăng khoảng 1 điểm nhờ warmup).
- **ResNet-18** (271K tham số, 30K bước): lr 0.01 tìm được vé tốt nhất 89.5% ($41.7\% \geq P_m \geq 21.9\%$), thấp hơn mạng gốc ở lr 0.1 (90.5%). Warmup ($k = 20000$, lr 0.03) đạt 90.5% ở $P_m = 27.1\%$; vẫn tìm được vé khi $P_m \geq 11.8\%$. Không tìm được hyperparameter nào cho vé ở lr gốc 0.1.

5. Phân tích / ablation

- **Khởi tạo quan trọng:** reinit ngẫu nhiên làm mất hiệu quả (mọi mạng). Thêm nhiễu Gaussian vào khởi tạo của vé (phụ lục F.7): nhiễu 0.5σ gần như không đổi; ngay cả 3σ vẫn tốt hơn reinit hoàn toàn.
- **Cấu trúc cũng quan trọng:** ở Lenet reinit tốt hơn mạng thưa ngẫu nhiên cùng số tham số, còn ở các mạng conv không khác biệt đáng kể (phụ lục E). Giải thích của tác giả: MNIST chỉ có vùng ảnh nhất định chứa thông tin, nên một số kết nối có giá trị hơn.
- **Cắt tại iteration 0 (phụ lục F.4):** cắt theo độ lớn khởi tạo trước khi train còn tệ hơn tìm bằng cắt lặp, tức không thể chỉ nhìn khởi tạo là chọn được vé.
- **Vé không "đã được train sẵn":** trọng số thuộc vé thay đổi nhiều hơn trọng số khác giữa đầu và cuối train (phụ lục F.5), phản bác giả thuyết rằng chúng sẵn gần nghiệm. Tác giả cho rằng lợi ích gắn với thuật toán tối ưu, dữ liệu, mô hình.
- **Phân bố khởi tạo của vé:** các layer ẩn sâu hơn có phân bố hai đỉnh (bimodal), trọng số ban đầu có độ lớn lớn có khả năng sống sót cao hơn (F.1, F.2). Layer đầu giữ phân bố gốc với Adam.
- **Kết nối:** mỗi neuron giữ số kết nối vào tỷ lệ với mức cắt của layer (F.6), không có neuron nào bị "cắt cụt" bất thường.
- **Global vs layerwise pruning** cho mạng sâu: global cho vé nhỏ hơn (I.1).
- **Dropout** bổ sung cho cắt lặp.

6. Giới hạn

Do tác giả nêu:

- Chỉ phân loại ảnh trên MNIST/CIFAR-10; không có ImageNet vì cắt lặp cần train >15 lần liên tiếp.
- Chỉ cắt thưa không cấu trúc (unstructured), nên mạng thu được không tối ưu cho thư viện/phần cứng hiện hành; cắt structured và tiêu chí không dựa độ lớn để dành cho tương lai.
- Với mạng sâu (ResNet-18, VGG-19) cần warmup mới tìm được vé ở lr cao; chưa rõ vì sao.
- "Lottery ticket conjecture" (SGD tìm vé) chưa được kiểm chứng.
- Không giải thích được tại sao khởi tạo của vé lại tốt.

- Liu et al. 2019 (bài 11) cho thấy mạng cắt rồi reinit vẫn bằng mạng gốc ở mức cắt đến ~80% trên VGG-19; paper thừa nhận điều này khớp với kết quả mình ở mức thưa đó, nhưng tuyên bố ở mức thưa hơn (cắt tới 98.5%) thì khởi tạo mới quan trọng.

Nhận xét:

- Tìm vé cần *train mạng đầy đủ nhiều lần*, nên không phải phương pháp tiết kiệm chi phí huấn luyện trong thực tế; ý nghĩa chính là khoa học về khả năng huấn luyện mạng thưa.
- Mạng thưa không cấu trúc khó chuyển thành tăng tốc suy luận GPU batch-1.
- Tất cả mạng dưới ~20M tham số; không có transformer, cũng không có ngôn ngữ.
- Số liệu "học nhanh hơn" dựa vào proxy *early-stopping iteration* trên validation, nhạy với lịch lr (nên phải đổi sang đo độ chính xác tại vài mốc cho VGG/ResNet).

7. Liên hệ với các bài khác

- **02 Han 2015, 03 Zhu & Gupta:** dùng cùng tiêu chí độ lớn; LTH thêm bước *reset về khởi tạo* và đổi mục tiêu từ nén sang train mạng thưa.
- **09 Linear mode connectivity & LTH:** chứng minh vé ở mạng lớn (ResNet-50, Inception-v3) chỉ tìm được nếu reset về *một thời điểm sớm* k sau khởi tạo (rewinding), không phải iteration 0 (đây chính là vấn đề warmup ở Mục 4).
- **10 SNIP:** tìm mạng con ngay tại khởi tạo, không cần train trước, trái với bước "train rồi cắt" của LTH.
- **11 Rethinking the value of pruning:** nghi ngờ vai trò của khởi tạo trong cắt structured/ lr chuẩn; LTH đã đề cập sự khác biệt (mục 5 Discussion).
- **15 LTH for BERT:** mở rộng LTH sang transformer.
- **43 Pruning vs scratch:** vấn đề tương tự, cắt+recover so với train từ đầu, ở quy mô LLM.

8. Gợi ý áp dụng

Giá trị gián tiếp:

- **Phỏng đoán (giả thuyết, chưa kiểm chứng):** nếu ý tưởng "reset" tương đương bằng "rewinding" (bài 09) áp dụng cho recovery bằng chưng cất thì việc quay lại trọng số sớm là một nguồn khởi tạo cho mạng cắt; cần thí nghiệm riêng.
- **Cảnh báo về warmup:** kết quả Mục 4 nhắc rằng recovery nên dùng lr nhỏ hoặc warmup để không phá cấu trúc đã học.

9. Thí nghiệm gợi ý

1. **Lặp tái hiện nhỏ** (cho bản thân hiểu): Lenet-300-100 trên MNIST, cắt lặp 20% mỗi vòng, so vé (reset θ0) với reinit và mạng thưa ngẫu nhiên; vẽ độ chính xác theo P_m (khoảng vài chục phút trên CPU).
2. **Cắt neuron MLP của decoder theo bước nhỏ so với một bước** với cùng ngân sách chưng cất (ý tưởng "iterative > one-shot" của bài).

10. Tóm tắt công thức / khái niệm

Khái niệm	Ý nghĩa
Lottery ticket hypothesis	Mạng dày chứa mạng con (mặt nạ m + khởi tạo θ_0) train riêng đạt độ chính xác tương đương trong số bước tương đương
Winning ticket	$f(x; m \odot \theta_0)$ thỏa $j' \leq j,\ a' \geq a,\ \|m\|_0 \ll \|\theta\|$
$P_m = \|m\|_0 / \|\theta\|$	Tỷ lệ trọng số còn lại
One-shot vs iterative pruning	Cắt p% một lần / cắt dần qua n vòng, reset về θ_0 mỗi vòng

Khái niệm	Ý nghĩa
Random reinit	Giữ mặt nạ, lấy lại $\theta_0'' \sim D_\theta$; thường kém hơn nhiều
Early-stopping iteration	Bước có loss validation nhỏ nhất, đại diện tốc độ học
Global pruning	Cắt trọng số nhỏ nhất gộp mọi layer (cần cho VGG-19, ResNet-18)
Warmup	Tăng lr tuyến tính từ 0; cần để tìm vé ở lr cao
Lottery ticket conjecture	SGD tìm và train một mạng con được khởi tạo tốt (chưa kiểm chứng)
Occam's Hill	Độ chính xác tăng rồi giảm khi cắt: mạng gốc quá phức tạp, mạng cắt quá nhiều quá đơn giản

11. Kết luận ngắn

Mạng dày chứa mạng con thưa mà nếu giữ nguyên khởi tạo gốc sẽ huấn luyện được nhanh và chính xác bằng hoặc hơn mạng gốc, với kích thước 10–20% (hoặc nhỏ hơn) trên MNIST và CIFAR-10; cùng cấu trúc nhưng khởi tạo mới thì thua.

Link gốc: https://arxiv.org/abs/1803.03635

Bài 09

Linear Mode Connectivity and the Lottery Ticket Hypothesis (2019)

Tác giả: Jonathan Frankle, Gintare Karolina Dziugaite, Daniel M. Roy, Michael Carbin.
Venue / arXiv: arXiv:1912.05671 (v1 11/12/2019; bản dùng ở đây v4, 18/07/2020); Proceedings of ICML 2020 (PMLR 119).
Nguồn: https://arxiv.org/abs/1912.05671
Chủ đề: Instability analysis (phân tích độ bất ổn trước nhiễu SGD) qua nội suy tuyến tính; vì sao LTH hoạt động ở MNIST nhưng không ở ImageNet; *rewinding* (quay về bước k sớm thay vì bước 0) để mở rộng LTH ra ResNet-50, Inception-v3.
Vị trí trong lộ trình: Đọc ngay sau 08 (Lottery Ticket). Bài 08 để lại một vết nứt: LTH không tìm được vé ở mạng lớn/lr cao (cần warmup) và Liu et al. (bài 11) phản biện. Bài này giải thích vết nứt đó bằng động lực học tối ưu hóa và đưa ra bản sửa "rewind". Bài kế: 10 (SNIP), 11 (Rethinking).
Chương nền liên quan: nền 01: Vì sao mạng cắt được

> Ghi chú học tập bằng tiếng Việt, không phải bản dịch. Số liệu lấy từ phần chính của paper.

1. Bối cảnh

Bài 08 cho thấy iterative magnitude pruning (IMP) tìm được "vé" ở mạng nhỏ (LeNet/MNIST, các CNN nhỏ CIFAR-10) khi reset về khởi tạo. Nhưng các nghiên cứu sau (Liu et al. 2019, Gale et al. 2019) và chính bài báo này xác nhận: với VGG/ResNet chuẩn trên CIFAR-10 và ImageNet, IMP **không** cho mạng con khớp độ chính xác ở độ thưa không tầm thường, tức không hơn cắt ngẫu nhiên. Chưa ai giải thích được vì sao.

Khái niệm nền:

- **SGD noise**: khác biệt giữa các lần huấn luyện do thứ tự mini-batch và data augmentation ngẫu nhiên, dù khởi tạo và siêu tham số giống nhau.
- **Mode (cực tiểu)**: một vùng loss thấp mà huấn luyện hội tụ về. Hai mode là *kết nối tuyến tính* nếu trên đoạn thẳng nối hai bộ trọng số, lỗi không có "rào cản" tăng lên.
- **Mode connectivity** (Draxler 2018, Garipov 2018): các mạng huấn luyện từ các khởi tạo *khác nhau* nối được bằng đường *phi tuyến* có lỗi hằng, nhưng không nối được bằng đường thẳng.
- **Matching subnetwork**: mạng con huấn luyện đến độ chính xác của mạng đầy đủ (trong phạm vi sai số); từ khi quay về bước $k > 0$ gọi là *matching* chứ không gọi winning ticket (vì không còn là khởi tạo ngẫu nhiên).

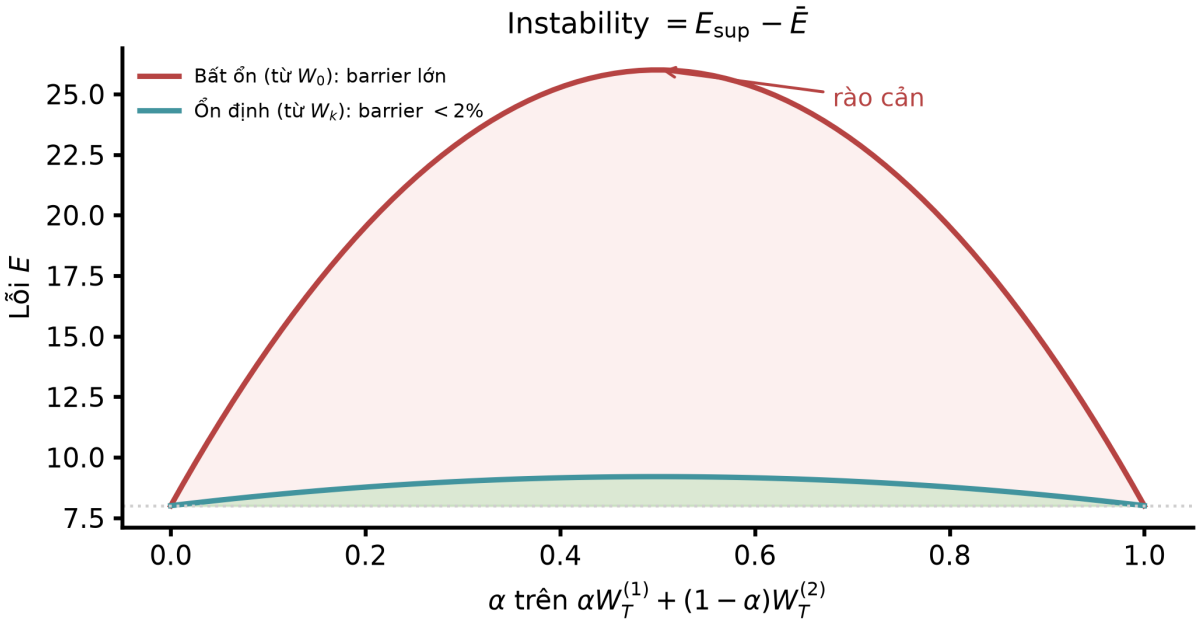
2. Ý tưởng chính

Hỏi: nếu huấn luyện cùng một mạng (W_0 hoặc trạng thái sau k bước) hai lần với nhiễu SGD khác nhau, hai kết quả có nằm trong cùng một "thung lũng" nối bằng đường thẳng không? Nếu có thì mạng **ổn định** với nhiễu SGD. Phát hiện: mạng đầy đủ chỉ ổn định *sau một đoạn đầu huấn luyện* (3% với ResNet-20, 20% với ResNet-50), và mạng con từ IMP **chỉ khớp độ chính xác khi nó ổn định**. Do đó với mạng lớn, thay vì reset về khởi tạo ($k = 0$), reset về trạng thái sớm k (rewinding) làm IMP tìm được mạng con vừa ổn định vừa khớp, mở rộng LTH lên ResNet-50 và Inception-v3 trên ImageNet.

3. Phương pháp chi tiết

3.1 Instability analysis

Mô hình SGD là hàm $A_{s\to t} : \mathbb{R}^D \times U \to \mathbb{R}^D$ nhận trọng số bước s và nhiễu $u \sim U$, trả trọng số bước t.



Rào cản lỗi trên đoạn thẳng giữa hai nghiệm: bất ổn từ W_0 vs ổn định từ W_k. (minh hoạ, tự tính)

```
Algorithm 1: instability của W_k
  W_T^1 = A_{k->T}(W_k, u1)      # nhiễu SGD 1
  W_T^2 = A_{k->T}(W_k, u2)      # nhiễu SGD 2
  return f(W_T^1, W_T^2)
```

Hàm so sánh f là **chiều cao rào cản lỗi** trên đoạn thẳng. Gọi $E(W)$ là lỗi (train hoặc test), $E_\alpha = E(\alpha W_1 + (1-\alpha)W_2)$, $E_{\text{sup}} = \sup_\alpha E_\alpha$, $\bar{E} = \text{mean}(E(W_1), E(W_2))$:

$$\boxed{\text{instability}(W_k) = E_{\text{sup}}(W_T^1, W_T^2) - \bar{E}(W_T^1, W_T^2)}$$

Quy ước: instability $< 2\%$ là *stable* (hai mạng linearly mode connected); lấy 30 giá trị α cách đều; trung bình 9 mẫu (3 khởi tạo x 3 lần chạy mỗi khởi tạo). Ngưỡng 2% khớp với mức tăng lỗi của các đường nối trong Draxler/Garipov.

Phụ: khi làm từ bước k, hai bản sao chỉ train $T-k$ bước; để loại trừ khả năng "ổn định chỉ do train ngắn", tác giả huấn luyện lại T bước với lịch lr reset; kết quả không khác (Figure 4).

3.2 IMP có rewinding (Algorithm 2)

```
1. W0 ngẫu nhiên; m = 1
2. Train W0 -> W_k  (k bước, nhiễu u)
3. for n = 1..N:
       train m ⊙ W_k -> W_T  (T-k bước, nhiễu u')
       cắt các trọng số còn sống có |W_T| nhỏ nhất (toàn cục); đặt m[i]=0
4. return (W_k, m)          # mạng con = m ⊙ W_k
```

Iterative: cắt 20% mỗi vòng cho MNIST/CIFAR-10; one-shot đến 30% còn lại cho ImageNet (Table 1). So với bài 08, điểm mới là quay về W_k chứ không phải W_0 ($k=0$ là LTH gốc).

3.3 Thiết lập (Table 1, trích)

Mạng	Dữ liệu	Tham số	Độ chính xác	lr	Warmup	Mật độ sau cắt	Kiểu cắt
LeNet	MNIST	266K	98.3 ± 0.1%	Adam 1.2e-3	0	3.5%	Iterative
ResNet-20 (Standard / Low / Warmup)	CIFAR-10	274K	91.7 / 88.8 / 89.7%	0.1 / 0.01 / 0.03	0 / 0 / 30K	16.8 / 8.6 / 8.6%	Iterative
VGG-16 (Standard / Low / Warmup)	CIFAR-10	14.7M	93.7 / 91.7 / 93.4%	0.1 / 0.01 / 0.1	0 / 0 / 30K	1.5 / 5.5 / 1.5%	Iterative
ResNet-50	ImageNet	25.5M	76.1 ± 0.1%	0.4	5 epoch	30%	One-shot
Inception-v3	ImageNet	27.1M	78.1 ± 0.1%	0.03 (giảm tuyến tính đến 0.005)	0	30%	One-shot

("Low" và "Warmup" là các biến thể của bài 08: lr thấp hoặc warmup để IMP tìm được vé.) Độ thưa trong Table 1 là mức cực đoan nhất mà IMP vẫn khớp ở một bước quay lại nào đó.

4. Thí nghiệm và kết quả chính

4.1 Mạng đầy đủ: bất ổn lúc khởi tạo, ổn định sớm (Mục 3)

- Tại $k = 0$ chỉ LeNet ổn định; các mạng khác khi nội suy tuyến tính có lỗi tăng lên mức đoán ngẫu nhiên.
- Càng bắt đầu từ k lớn thì instability càng thấp. **ResNet-20 ổn định ở iteration 2000 (3% huấn luyện), VGG-16 ở 1000 (1.5%)**; **ResNet-50 ở epoch 18 (20%), Inception-v3 ở epoch 28 (16%)** (đo trên test). Với ResNet-50, độ ổn định trên train xuất hiện muộn hơn và Inception-v3 không đạt trong khoảng khảo sát.
- Từ điểm đó, kết quả tối ưu hóa được quyết định tới một vùng cực tiểu nối tuyến tính, bất kể nhiễu SGD.

4.2 Mạng con IMP tại $k = 0$ (Table 2, độ chính xác)

Mạng	Đầy đủ	IMP	Random Prune	Random Reinit	Δ IMP	Matching?
LeNet	98.3	98.2	96.7	97.5	0.1	Có
ResNet-20	91.7	88.5	88.6	88.8	3.2	Không
ResNet-20 Low	88.8	89.0	85.7	84.7	-0.2	Có
ResNet-20 Warmup	89.7	89.6	85.7	85.6	0.1	Có
VGG-16	93.7	90.9	89.4	91.0	2.8	Không
VGG-16 Low	91.7	91.6	90.1	90.2	0.1	Có
VGG-16 Warmup	93.4	93.2	90.1	90.7	0.2	Có
ResNet-50	76.1	73.7	73.1	73.4	2.4	Không
Inception-v3	78.1	75.7	75.2	75.5	2.4	Không

(Sai số chuẩn < 0.2.) Chỉ các trường hợp "matching" mới là các trường hợp mạng con ổn định ở $k = 0$ (Figure 5): LeNet, ResNet-20 low/warmup, VGG-16 low/warmup. Còn lại (ResNet-20/VGG-16 chuẩn, ResNet-50,

Inception-v3) vừa không ổn định vừa không matching; mạng con IMP không hơn mạng con cắt ngẫu nhiên hay reinit.

4.3 Rewinding (Figures 6–7)

- Mạng con IMP của ResNet-20 và VGG-16 chuẩn **trở nên ổn định ở iteration 500 (0.8% huấn luyện) và 1000 (1.6%)**; của ResNet-50 và Inception-v3 ở **epoch 5 (5.5%) và 6 (3.5%)**. Trong mọi trường hợp, mạng con ổn định *sớm hơn* mạng đầy đủ (ResNet-50: epoch 5 so với 18; Inception-v3: epoch 6 so với 28).
- Các mạng con vốn không matching ở $k = 0$ trở thành matching khi quay về k sau đó, và điểm quay lại **trùng khớp** với điểm mạng con trở nên ổn định. Kết luận (tại độ thưa cực đoan): *mạng con IMP khớp độ chính xác khi và chỉ khi nó ổn định*.
- Train error cũng giảm khi rewind (ResNet-20, VGG-16 đạt 0% train error), gợi ý mạng con ổn định tối ưu tốt hơn.
- Mạng con cắt ngẫu nhiên hay reinit **không ổn định và không matching ở mọi bước quay lại** (LeNet là ngoại lệ về độ ổn định).

4.4 Các mức thưa khác (Figure 9, ResNet-20 và VGG-16)

- **Dải I** (còn >80.0% trọng số với ResNet-20, >16.8% với VGG-16): đủ quá tham số hóa nên ngay cả cắt ngẫu nhiên cũng khớp ("thưa tầm thường").
- **Dải II** (80.0%–13.4% ResNet-20; 16.8%–1.2% VGG-16): chỉ mạng con IMP khớp. Trong một phần dải (51.2%–13.4% và 6.9%–1.5%) bước mà mạng con thành stable và thành matching xấp xỉ trùng nhau.
- **Dải III** (<13.4% và <1.2%): cực thưa đến mức ngay cả IMP không khớp ở bước quay lại nào; lỗi vẫn giảm khi mạng con trở nên ổn định.
- Khi độ thưa tăng, bước mà IMP trở nên stable giảm, đi ngang rồi tăng; bước của cắt ngẫu nhiên chỉ tăng đến khi không bao giờ stable.

5. Phân tích / ablation

- **Loại trừ giải thích "chỉ do train ngắn":** instability không đổi khi train lại T bước với lịch lr reset (Figure 4).
- **Hai giai đoạn huấn luyện:** pha bất ổn (SGD noise đẩy vào các cực tiểu không nối tuyến tính) rồi pha ổn định; phù hợp với việc Hessian sớm tách thành vài trị riêng lớn và "bulk" (Gur-Ari 2018) và với lợi ích của warmup lr (Goyal 2017).
- **Hyperparameter "low"/"warmup" của bài 08** vừa làm IMP matching vừa làm mạng con ổn định ở $k = 0$: bằng chứng thêm cho mối liên hệ giữa ổn định và độ chính xác.
- **Rewinding dùng trong thực tế:** theo Discussion, Morcos et al. 2019 cho thấy vé có rewinding chuyển được giữa các tác vụ ảnh; Renda et al. 2020 cho thấy IMP + rewinding đạt sparsity SOTA, bằng hoặc hơn fine-tune với lr thấp.
- Phụ lục G (không đọc kỹ ở đây) so sánh các cách đo khác ngoài nội suy tuyến tính.

6. Giới hạn

Do tác giả nêu:

- Kết quả là *thực nghiệm*, không có chứng minh lý thuyết; mối quan hệ nhân quả giữa stable và matching chỉ là tương quan ("matching khi stable").
- Matching ở độ thưa cực đoan chỉ trong điều kiện bước $k > 0$; không còn là khởi tạo ngẫu nhiên, nên không còn "winning ticket" theo nghĩa gốc, và không tiết kiệm được chi phí train nếu cần mạng đầy đủ đến bước k.
- Dải III: ngay cả khi stable, IMP không matching, tức stable không phải điều kiện đủ.

- Chi phí tính toán cao: mức thưa chỉ được khảo sát toàn diện với ResNet-20 và VGG-16.

Nhận xét:

- Chỉ phân loại ảnh; không có mô hình ngôn ngữ, transformer, hay mô hình tạo sinh.
- Cắt là thưa không cấu trúc; kết luận về stability không đảm bảo chuyển sang cắt có cấu trúc.
- Đo stability dựa trên *lỗi* phân loại; với bài toán sinh chuỗi (OCR/VLM) chưa có định nghĩa tương đương (nội suy trọng số tuyến tính vẫn áp dụng được nhưng metric cần chọn).
- Ngưỡng 2% và việc dùng 9 mẫu là quy ước thực nghiệm.

7. Liên hệ với các bài khác

- **08 Lottery Ticket:** bài này giải thích vì sao LTH gốc thất bại ở mạng sâu/lr cao (mạng chưa ổn định ở $k = 0$) và sửa bằng rewind.
- **11 Rethinking:** Liu et al. cho rằng reinit và train từ đầu cho kết quả ngang cắt; bài 09 giải thích mâu thuẫn bề ngoài: ở độ thưa tầm thường (dải I), đúng như vậy; khác biệt chỉ xuất hiện ở dải II.
- **10 SNIP:** cắt ngay tại khởi tạo: nếu các mạng thưa chưa ổn định lúc $k = 0$ thì chọn mặt nạ tại $k = 0$ có thể thiệt thòi; SNIP đối lập trực tiếp với kết luận "cần train một chút trước khi chọn mặt nạ".
- **15 LTH for BERT:** dùng rewinding cho BERT khi vé ở khởi tạo không hoạt động.
- **16 Movement pruning, 03 Zhu & Gupta:** cắt trong lúc train, khác hướng.
- **43 Pruning vs scratch:** cùng chủ đề "cắt + phục hồi so với train lại".

8. Gợi ý áp dụng

- Nhưng ý tưởng "phục hồi từ điểm quay về" có thể thay bằng: sau khi cắt, khởi tạo tham số bị đụng tới từ checkpoint gốc (đó là "rewind về trạng thái cuối").
- **Công cụ đo mới: nội suy tuyến tính giữa hai lần phục hồi.** Chạy hai lần chưng cất với thứ tự dữ liệu khác nhau từ cùng một mô hình đã cắt, nội suy tuyến tính trọng số và đo loss: nếu có rào cản, quá trình phục hồi nhạy với nhiễu (instability cao); nếu không, có thể trung bình trọng số (model soup), và quan trọng hơn, ta biết kết quả ổn định. Có thể đối chiếu: cắt neuron MLP có instability cao hơn cắt layer không? Đó có thể là một cách (giả thuyết) giải thích phục hồi kém của cắt neuron.
- Cần thí nghiệm dưới.
- **Warmup/lr khi phục hồi:** bài này tiếp tục củng cố nguyên tắc dùng lr nhỏ/warmup ở giai đoạn đầu sau khi đổi kiến trúc (pha bất ổn xảy ra đầu huấn luyện).
- **Ít liên quan:** thuật toán IMP và việc chọn mặt nạ thưa không cấu trúc.

9. Thí nghiệm gợi ý

1. **Instability của phục hồi sau cắt:** với mô hình đã cắt (cắt layer vs cắt neuron MLP cùng số tham số), chạy 2 phục hồi bằng chưng cất với 2 seed dữ liệu, nội suy tuyến tính trọng số ở 11 điểm α, vẽ loss KL/CER theo α; so rào cản hai kiểu cắt.
2. **Điểm bắt đầu phục hồi:** thay vì phục hồi ngay từ trạng thái sau cắt, thử "warm start" bằng vài trăm bước lr thấp rồi mới lr đầy đủ, đo instability theo số bước warmup (mô phỏng ý "stable sau k bước").
3. **Model soup sau cắt:** trung bình trọng số của 2–3 lần phục hồi ổn định; so với mô hình đơn (chỉ làm khi thí nghiệm 1 cho rào cản ≈ 0).

10. Tóm tắt công thức / khái niệm

Khái niệm	Ý nghĩa
Instability analysis	Train hai bản sao với nhiễu SGD khác nhau và so sánh kết quả
$E_\alpha = E(\alpha W_1 + (1-\alpha)W_2)$	Lỗi tại điểm nội suy tuyến tính

Khái niệm	Ý nghĩa
instability $= E_{\text{sup}} - \bar{E}$	Chiều cao rào cản lỗi
Stable	instability $< 2\%$ (linearly mode connected)
Linear mode connectivity	Hai cực tiểu nối bằng đoạn thẳng không có rào cản
IMP với rewinding	Train đến W_T, cắt trọng số nhỏ nhất, quay về W_k (không phải W_0)
Matching	Mạng con đạt độ chính xác của mạng đầy đủ (sai số < 0.2 điểm trong Figure 9)
Sparsity range I/II/III	Cắt ngẫu nhiên khớp / chỉ IMP khớp / không ai khớp
Kết luận chính	IMP subnetwork matching ⇔ stable (ở độ thưa cực đoan khảo sát)

11. Kết luận ngắn

Bài báo chuyển câu hỏi “LTH đúng không” thành câu hỏi có thể đo: mạng con có ổn định với nhiễu SGD không. Mạng con IMP chỉ khớp độ chính xác khi chúng ổn định; ở mạng lớn điều đó xảy ra sau 0.8–5.5% đầu huấn luyện, nên phải *rewind* về bước sớm thay vì khởi tạo.

Link gốc: https://arxiv.org/abs/1912.05671

Bài 10

SNIP: Single-shot Network Pruning based on Connection Sensitivity (2018)

Tác giả: Namhoon Lee, Thalaiyasingam Ajanthan, Philip H. S. Torr (University of Oxford).
Venue / arXiv: arXiv:1810.02340 (v1 04/10/2018; bản dùng ở đây v2, 23/02/2019); ICLR 2019.
Nguồn: https://arxiv.org/abs/1810.02340
Chủ đề: Pruning tại khởi tạo (pruning at initialization, PaI): cắt một lần, trước khi huấn luyện, bằng điểm "độ nhạy kết nối" tính trên một mini-batch; không cần pretrain, không có lịch cắt.
Vị trí trong lộ trình: Đọc sau 07 (Taylor importance, cùng họ gradient x trọng số nhưng cho mạng đã huấn luyện) và 08–09 (LTH: cần train nhiều lần để tìm vé). SNIP trả lời câu hỏi "có tìm được mạng con tốt mà không cần train mạng đầy đủ không?". Bài kế: 11 (Rethinking the value of pruning, cho thấy cách nhìn "mạng con tìm được = kiến trúc" gần với SNIP).
Chương nền liên quan: nền 01: Vì sao mạng cắt được, nền 03: Độ quan trọng của trọng số

> Ghi chú học tập bằng tiếng Việt, không phải bản dịch. Số liệu lấy từ paper.

1. Bối cảnh

Các phương pháp cắt trước SNIP, dù dùng độ lớn (Han 2015), Hessian (OBD/OBS, bài 01) hay phạt thưa (L0/L1), đều nằm *trong* vòng lặp huấn luyện: huấn luyện, cắt, huấn luyện lại, nhiều lần, với nhiều siêu tham số (tần suất cắt, lịch giảm...). Hậu quả: đắt, khó mở rộng sang kiến trúc/tác vụ mới. Hơn nữa độ lớn trọng số và Hessian phụ thuộc vào *thang đo trọng số*, mà thang này thay đổi theo lớp chuẩn hóa, nên phải có pretrain.

Mục tiêu bài toán cắt (Eq. 1): với độ thưa mong muốn κ (số trọng số khác 0),

$$\min_{w} L(w; D) = \min_{w} \frac{1}{n} \sum_{i} \ell(w; (x_i, y_i)) \quad \text{s.t. } w \in \mathbb{R}^m,\ \|w\|_0 \leq \kappa.$$

Khái niệm nền:

- **Pruning at initialization**: chọn mặt nạ thưa trước khi huấn luyện; sau đó chỉ huấn luyện các kết nối còn lại, nên mọi bước huấn luyện sau đó rẻ hơn (nếu có hỗ trợ ma trận thưa).
- **Variance scaling initialization** (Glorot, He): chọn phương sai trọng số khởi tạo để tín hiệu qua các lớp giữ phương sai ổn định.
- **Độ thưa** $\bar{\kappa} = (m - \kappa)/m \cdot 100\%$: phần trăm tham số bị cắt.

2. Ý tưởng chính

Tách "có kết nối hay không" ra khỏi "giá trị trọng số" bằng cách đặt hệ số chỉ báo $c_j \in \{0, 1\}$ cho mỗi kết nối: trọng số hiệu dụng là $c_j w_j$. Rồi hỏi: nếu hơi giảm c_j khỏi 1, loss thay đổi bao nhiêu? Đạo hàm $\partial L/\partial c_j$ đo đúng điều đó và tính được cho mọi kết nối cùng lúc bằng một lượt forward-backward. Kết nối nào có độ lớn đạo hàm cao (dù dương hay âm) là nhạy cảm với loss nên phải giữ. Tính điểm này tại khởi tạo (variance scaling) trên một mini-batch, giữ top-κ, vứt phần còn lại, rồi huấn luyện mạng thưa theo cách thông thường. Không có pretrain, không có lịch cắt, không có siêu tham số cắt.

3. Phương pháp chi tiết

3.1 Độ nhạy kết nối

Bài toán sau khi thêm chỉ báo (Eq. 3):

$$\min_{c,w} L(c \odot w; D) \quad \text{s.t. } w \in \mathbb{R}^m,\ c \in \{0,1\}^m,\ \|c\|_0 \le \kappa.$$

Ảnh hưởng của việc bỏ kết nối j (Eq. 4): $\Delta L_j(w; D) = L(\mathbf{1} \odot w; D) - L((\mathbf{1} - e_j) \odot w; D)$. Tính cho mọi j cần $m + 1$ lượt forward: quá đắt. Vì c nhị phân nên L không khả vi theo c; nới lỏng c thành thực và xấp xỉ bằng đạo hàm (Eq. 5):

$$\Delta L_j \approx g_j(w; D) = \frac{\partial L(c \odot w; D)}{\partial c_j}\Big|_{c=\mathbf{1}} = \lim_{\delta \to 0} \frac{L(c \odot w) - L((c - \delta e_j) \odot w)}{\delta}\Big|_{c=1}.$$

Đây khác $\partial L / \partial w_j$ (thay đổi *cộng* trọng số): ở đây là thay đổi *nhân* trọng số. Điểm độ nhạy chuẩn hóa (Eq. 6):

$$\boxed{s_j = \frac{|g_j(w; D)|}{\sum_{k=1}^{m} |g_k(w; D)|}}$$

Giữ top-κ (Eq. 7): $c_j = \mathbb{1}[s_j - \tilde{s}_\kappa \ge 0]$, với $\tilde{s}_\kappa$ là phần tử lớn thứ κ của s.

Vì trọng số hiệu dụng là $c \odot w$, quy tắc chuỗi cho $\partial L / \partial c_j|_{c=1} = w_j\, \partial L / \partial w_j$, tức $|g_j| = |w_j\, \partial L / \partial w_j|$. Đây chính là điểm bậc 1 của bài 07 (khi bỏ phép bình phương), nhưng tính ở khởi tạo, trên từng trọng số, và so xếp hạng toàn cục. Mẫu số chuẩn hóa $\sum |g_k|$ không đổi thứ hạng; chỉ để làm điểm cùng thang.

Khác với Mozer & Smolensky / Karnin (cổ điển): các tiêu chí đó xét phần tử nào làm hỏng hiệu năng ít nhất khi bỏ ($-\partial L / \partial w$...), phụ thuộc giá trị loss trước khi cắt, nên cần mạng đã pretrain. SNIP dùng *độ lớn* của đạo hàm, không quan tâm dấu và không cần loss thấp.

3.2 Vì sao cắt được ở khởi tạo

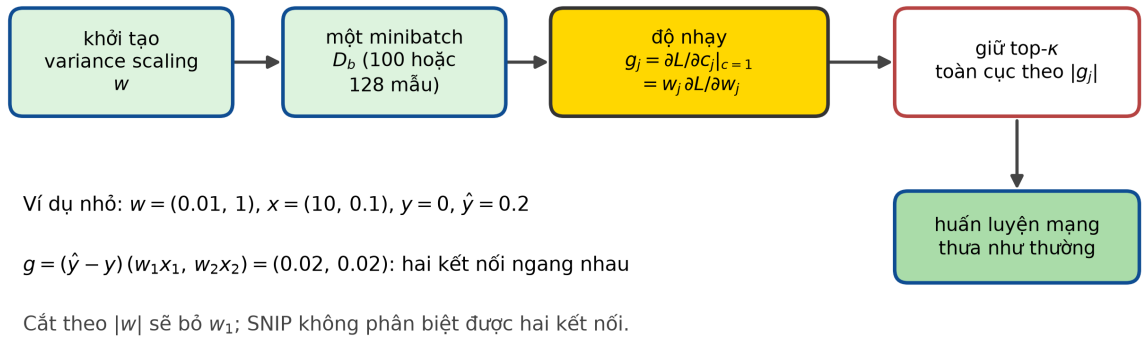
Điểm s_j phụ thuộc vào w, D và L:

- w**:** chọn khởi tạo variance-scaling (Glorot/He) để phương sai tín hiệu giống nhau ở mọi layer, giúp điểm không phụ thuộc kiến trúc. Nếu trọng số quá lớn, kích hoạt bão hòa và gradient mất thông tin.
- D **và** L**:** định nghĩa tác vụ nên các kết nối được chọn phụ thuộc tác vụ ("data-dependent"). Chỉ cần một mini-batch (100 mẫu MNIST, 128 mẫu CIFAR); có thể cộng dồn qua nhiều batch hoặc EMA nếu bộ nhớ hạn chế.

3.3 Thuật toán (Algorithm 1)

```
Input : hàm loss L, dữ liệu train D, độ thưa κ
1  w  <- VarianceScalingInitialization
2  D_b <- lấy một mini-batch từ D
3  s_j <- |g_j(w; D_b)| / Σ_k |g_k(w; D_b)|     với g_j = ∂L/∂c_j tại c = 1
4  sắp xếp s giảm dần -> s~
5  c_j <- 1[ s_j >= s~_κ ]                       # giữ top-κ, toàn cục
6  w*  <- argmin_w L(c ⊙ w; D)                   # train bình thường
7  w*  <- c ⊙ w*                                 # mạng thưa cuối cùng
```

Chỉ một lượt forward-backward để cắt; không có siêu tham số cắt nào ngoài κ. Paper nhấn mạnh (lý thuyết) vì sao có thể nhanh hơn train chuẩn "một bậc độ lớn" nếu dùng thư viện ma trận thưa.



Hình 33: Quy trình SNIP: một minibatch tại khởi tạo, độ nhạy kết nối, giữ top-κ toàn cục rồi huấn luyện mạng thưa, kèm ví dụ nhỏ ở mục 3.4 (minh hoạ, tự tính)

Cách đọc hình: hàng trên là toàn bộ bước cắt, chỉ cần một lượt forward-backward. Ví dụ hai trọng số bên dưới cho thấy $g_j = w_j\, \partial L/\partial w_j$ có thể bằng nhau dù $|w_1| \ll |w_2|$, nên SNIP và tiêu chí độ lớn chọn khác nhau.

3.4 Ví dụ nhỏ (tự đặt)

Mạng 1 lớp: $\hat{y} = w_1 x_1 + w_2 x_2$, loss bình phương, $w = (0.01,\ 1)$, một mẫu $x = (10,\ 0.1)$, nhãn $y = 0$. Có $\hat{y} = 0.1 + 0.1 = 0.2$. Với c: $\hat{y} = c_1 w_1 x_1 + c_2 w_2 x_2$, $g_j = \partial L/\partial c_j = (\hat{y} - y)\, w_j x_j$ nên $g = (0.2 \cdot 0.1,\ 0.2 \cdot 0.1) = (0.02, 0.02)$: độ nhạy bằng nhau dù $|w_1| \ll |w_2|$. Cắt theo |w| bỏ w_1; SNIP coi hai kết nối ngang nhau. Đây là kiểu khác biệt giữa độ lớn và độ nhạy mà bài 07 cũng nêu.

4. Thí nghiệm và kết quả chính

Thiết lập: điểm tính bằng 100 mẫu (MNIST) hoặc 128 mẫu (CIFAR); sau cắt train bằng SGD momentum 0.9, batch 100/128, weight decay 0.0005, lr ban đầu 0.1 giảm 0.1 mỗi 25k/30k bước; 10% dữ liệu train làm validation; augmentation chuẩn cho CIFAR.

4.1 LeNet (Mục 5.1–5.2, Table 1)

LeNet-300-100 (267k tham số), LeNet-5-Caffe (431k tham số); 20 seed cho mỗi mức thưa. Lỗi tham chiếu: 1.7% và 0.9%.

Phương pháp	Tiêu chí	LeNet-300-100: thưa % / lỗi %	LeNet-5-Caffe: thưa % / lỗi %
Ref.	–	– / 1.7	– / 0.9
LWC (Han)	Magnitude	91.7 / 1.6	91.7 / 0.8
DNS	Magnitude	98.2 / 2.0	99.1 / 0.9
LC	Magnitude	99.0 / 3.2	99.0 / 1.1
SWS	Bayesian	95.6 / 1.9	99.5 / 1.0
SVD	Bayesian	98.5 / 1.9	99.6 / 0.8
OBD	Hessian	92.0 / 2.0	92.0 / 2.7
L-OBS	Hessian	98.5 / 2.0	99.0 / 2.1
SNIP	Connection sensitivity	95.0 / 1.6; 98.0 / 2.4	98.0 / 0.8; 99.0 / 1.1

Văn bản: ở thưa 98% (LeNet-300-100) và 99% (LeNet-5-Caffe), lỗi chỉ tăng khoảng 0.7% và 0.3%. Ở 95% và 98%, mạng thưa còn tốt hơn mạng dày. LeNet-300-100 mất không đáng kể ở 90%. Điểm khác biệt chính SNIP nêu: các phương pháp khác cần pretrain, siêu tham số cắt và/hoặc mục tiêu huấn luyện sửa đổi, SNIP thì không.

4.2 Các kiến trúc hiện đại (Table 2, lỗi trước → sau)

CIFAR-10 (AlexNet, VGG, WRN) và sequential MNIST (LSTM/GRU, xử lý từng hàng):

Mô hình	Thưa %	Tham số	Lỗi %	Δ
AlexNet-s	90	5.1m → 507k	14.12 → 14.99	+0.87
AlexNet-b	90	8.5m → 849k	13.92 → 14.50	+0.58
VGG-C	95	10.5m → 526k	6.82 → 7.27	+0.45
VGG-D	95	15.2m → 762k	6.76 → 7.09	+0.33
VGG-like	97	15.0m → 449k	8.26 → 8.00	−0.26
WRN-16-8	95	10.0m → 548k	6.21 → 6.63	+0.42
WRN-16-10	95	17.1m → 856k	5.91 → 6.43	+0.52
WRN-22-8	95	17.2m → 858k	6.14 → 5.85	−0.29
LSTM-s	95	137k → 6.8k	1.88 → 1.57	−0.31
LSTM-b	95	535k → 26.8k	1.15 → 1.35	+0.20
GRU-s	95	104k → 5.2k	1.87 → 2.41	+0.54
GRU-b	95	404k → 20.2k	1.71 → 1.52	−0.19

Tác giả: mất <1% ở mọi mô hình. Không phải sửa thuật toán cho kiến trúc residual hay recurrent. So sánh gián tiếp: SVD cắt 97.9% VGG-like không mất độ chính xác (SNIP 97%); SWS đạt 93.4% thưa trên WRN-16-4 nhưng mất 2%.

4.3 Tiny-ImageNet (Phụ lục C, Table 4)

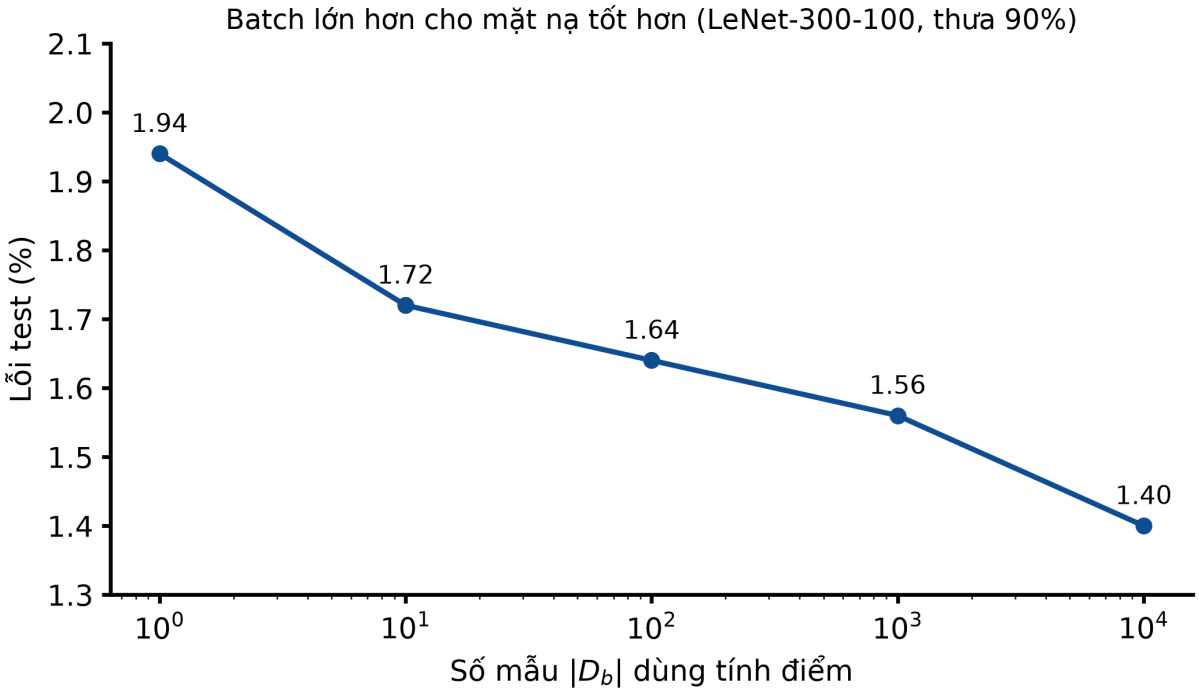
200 lớp, ảnh 64x64, lỗi trước → sau: AlexNet-s (90%) 62.52 → 65.27 (+2.75); AlexNet-b (90%) 62.76 → 65.54 (+2.78); VGG-C (95%) 56.49 → 57.48 (+0.99); VGG-D (95%) 56.85 → 57.00 (+0.15); VGG-like (95%) 54.86 → 55.73 (+0.87). AlexNet mất nhiều hơn; tác giả đoán do stride 4 ở conv đầu làm mất thông tin khi bị cắt.

5. Phân tích / ablation

- **Kết nối giữ lại có ý nghĩa (Mục 5.4, Figure 2):** dùng batch gồm chỉ một lớp (một chữ số), hình hóa mặt nạ lớp đầu của LeNet-300-100 (784x300 trung bình theo cột thành ảnh 28x28): kết nối giữ lại tái dựng hình chữ số (MNIST) hoặc bóng đồ vật (Fashion-MNIST); vùng nền bị cắt. Trên dữ liệu đảo màu (Phụ lục A) kết quả giống hệt; trong khi dùng $\partial L/\partial w$ thay cho $\partial L/\partial c$ thì mẫu cắt không nhất quán giữa các mức thưa và thay đổi khi đảo màu.
- **Ảnh hưởng dữ liệu (Figure 3, LeNet-300-100, thưa 90%):** kích thước batch tính điểm $|D_b|$ = 1, 10, 100, 1000, 10000 cho lỗi 1.94%, 1.72%, 1.64%, 1.56%, 1.40%. Batch càng lớn, mặt nạ càng giống trung bình của toàn tập train và lỗi càng thấp; ngay cả 1 mẫu đã cho mạng huấn luyện được.

Cách đọc hình: trục ngang để thang log. Lỗi giảm đều khi dùng nhiều mẫu hơn, từ 1,94% với một mẫu xuống 1,40% với 10000 mẫu. Ngay cả một mẫu đã cho mạng huấn luyện được, nhưng mặt nạ tốt hơn khi tiến gần trung bình của cả tập train.

- **Ảnh hưởng khởi tạo (Table 3, thưa 90%, 20 seed, lỗi %):**



Hình 34: Lỗi test của LeNet-300-100 ở độ thưa 90% theo số mẫu dùng để tính điểm (Figure 3) (vẽ lại từ số liệu trong ghi chú)

Init	LeNet-300-100	LeNet-5-Caffe	LSTM-s	GRU-s
RN (random normal)	1.90 ± 0.09	0.89 ± 0.04	2.93 ± 0.20	47.61 ± 20.49
TN (truncated normal)	1.96 ± 0.11	0.87 ± 0.05	3.03 ± 0.17	46.48 ± 22.25
VS-X (Glorot)	1.91 ± 0.10	0.88 ± 0.07	1.48 ± 0.09	1.80 ± 0.10
VS-H (He)	1.88 ± 0.10	0.85 ± 0.05	1.47 ± 0.08	1.80 ± 0.14

Variance scaling tốt nhất (VS-H); khác biệt nhỏ ở LeNet nhưng *thiết yếu* cho RNN, đặc biệt GRU (không có variance scaling mạng cắt không đạt độ chính xác tốt).

- **Khớp nhãn ngẫu nhiên (Mục 5.6, Figure 4, kiểu Zhang et al. 2017):** mạng đầy đủ khớp được nhãn ngẫu nhiên (loss gần 0, kể cả với L2); mạng cắt 99% bởi SNIP (điểm tính với nhãn thật) *không* khớp nhãn ngẫu nhiên nhưng học nhãn thật. Phụ lục B: càng ít bị cắt, càng dễ khớp nhãn ngẫu nhiên. Giải thích của tác giả: mạng cắt không đủ dung lượng để ghi nhớ ngẫu nhiên nhưng đủ để học tác vụ thật.

6. Giới hạn

Do tác giả nêu / ngầm thừa nhận:

- Chỉ thí nghiệm trên MNIST, CIFAR-10, Tiny-ImageNet; mạng vừa và nhỏ.
- So sánh với phương pháp khác khó chính xác vì khác kiến trúc, lịch huấn luyện, tập dữ liệu và tác vụ (“challenges in directly comparing”).
- Phải chọn bằng tay: dùng toàn bộ train set, mini-batch hay validation để tính điểm; lựa chọn khởi tạo (variance scaling) quyết định chất lượng.
- Mục tiêu của tác giả là khái quát hóa của mạng thưa, chưa khảo sát (để tương lai).

Nhận xét:

- Cắt *thưa không cấu trúc*; lợi ích tốc độ/bộ nhớ phụ thuộc thư viện ma trận thưa, không đạt được trên GPU thông thường (paper chỉ nói "in theory").
- Chọn mặt nạ toàn cục từ **một** batch tại khởi tạo; về sau (ngoài phạm vi paper) người ta thấy PaI có thể làm sập layer hoặc không hơn mặt nạ ngẫu nhiên có cùng tỷ lệ theo layer (điều này không được kiểm chứng trong bài).
- Không so với "train rồi cắt" công bằng về chi phí tổng; so sánh chủ yếu với phương pháp cũ.
- Điểm xấp xỉ bậc 1 tại khởi tạo ngẫu nhiên; bài 09 gợi ý mạng chưa ổn định lúc $k = 0$ nên mặt nạ chọn lúc đó có thể kém (suy luận).

7. Liên hệ với các bài khác

- **01 OBD/OBS, 02 Han 2015:** hai tiêu chí (Hessian, độ lớn) mà SNIP phê phán là phụ thuộc thang trọng số; SNIP có hàng tương ứng trong Table 1 (OBD, L-OBS, LWC).
- **07 Molchanov 2019:** cùng họ gradient×trọng số (xem nhận xét ở 3.1); bài 07 tính trên mạng đã train và cho filter, SNIP tính tại khởi tạo và cho từng trọng số.
- **08 LTH và 09 Linear mode connectivity:** LTH tìm mặt nạ bằng train đầy đủ nhiều lần; SNIP tìm bằng một lượt gradient. Bài 09 cho thấy ở mạng lớn mặt nạ tốt chỉ xuất hiện sau vài % đầu huấn luyện, gợi ý hạn chế của PaI.
- **11 Rethinking:** Liu et al. coi cắt như tìm kiến trúc; SNIP là ví dụ "tìm kiến trúc thưa trước khi train".
- **16 Movement pruning:** cũng dùng gradient × trọng số (điểm di chuyển) nhưng cắt trong lúc fine-tune trên mô hình ngôn ngữ.
- **22–24 OBC, SparseGPT, Wanda:** cắt LLM sau huấn luyện dùng thống kê trên calibration nhỏ, thoải mái hơn nhiều so với SNIP; ý "một mini-batch dữ liệu là đủ" cũng xuất hiện ở đó.

8. Gợi ý áp dụng

Điểm có thể dùng:

9. Thí nghiệm gợi ý

2. **Độ phụ thuộc phân phối:** lặp lại với hai miền (tài liệu in vs. biểu mẫu viết tay); đo phần neuron được chọn trùng nhau (kiểu Figure 2/3).
3. **Chạy lại SNIP nhỏ cho hiểu:** LeNet-300-100 trên MNIST, thưa 90–98%, so mặt nạ SNIP với mặt nạ ngẫu nhiên cùng tỷ lệ theo layer (một giờ trên CPU), để kiểm tra bằng thực nghiệm giới hạn ở mục 6.

10. Tóm tắt công thức / khái niệm

Khái niệm	Ý nghĩa
$c \odot w$, $c \in \{0,1\}^m$	Tách kết nối có/không khỏi giá trị trọng số
$g_j = \partial L/\partial c_j\big\vert_{c=1}$	Độ nhạy của loss khi nhân kết nối j với $1-\delta$ (một lượt backward cho mọi j)
$s_j = \vert g_j\vert / \sum_k \vert g_k\vert$	Độ nhạy kết nối chuẩn hóa
$c_j = \mathbb{1}[s_j \geq \tilde{s}_\kappa]$	Giữ top-κ toàn cục
$\bar{\kappa} = (m-\kappa)/m \cdot 100\%$	Độ thưa (% bị cắt)
Variance scaling init	Glorot/He: giữ phương sai tín hiệu qua các layer; thiết yếu cho RNN
Single-shot / PaI	Cắt một lần, trước khi train
$\vert\partial L/\partial c\vert$ vs $\partial L/\partial w$	Thay đổi nhân vs. cộng; chỉ cái đầu cho mẫu cắt nhất quán
Random-label test	Mạng cắt không khớp nhãn ngẫu nhiên nhưng học nhãn thật

11. Kết luận ngắn

SNIP cho thấy một lượt gradient trên một mini-batch tại khởi tạo đủ để chọn mạng thưa 95–99% mà vẫn huấn luyện tới độ chính xác gần mạng đầy đủ trên các tác vụ nhỏ và vừa, với CNN, ResNet và RNN, không cần pretrain hay lịch cắt.

Link gốc: https://arxiv.org/abs/1810.02340

Bài 11

Rethinking the Value of Network Pruning (2018)

Tác giả: Zhuang Liu, Mingjie Sun, Tinghui Zhou, Gao Huang, Trevor Darrell (UC Berkeley; Tsinghua University).
Venue / arXiv: arXiv:1810.05270 (v1 11/10/2018; bản dùng ở đây v2, 05/03/2019); ICLR 2019.
Nguồn: https://arxiv.org/abs/1810.05270
Chủ đề: Với structured pruning, mô hình cắt rồi fine-tune từ trọng số kế thừa chỉ ngang hoặc kém hơn *train mô hình cùng cấu trúc từ khởi tạo ngẫu nhiên* (train from scratch); giá trị thực của cắt có thể là *tìm kiến trúc* (số kênh mỗi layer) chứ không phải chọn "trọng số quan trọng". Có so sánh với Lottery Ticket.
Vị trí trong lộ trình: Đọc sau 04–06 (filter pruning, slimming, ThiNet: đối tượng bị phản biện), 08–10 (LTH, SNIP).

> Ghi chú học tập bằng tiếng Việt, không phải bản dịch. Số liệu lấy từ paper (một vài bảng PDF bị lệch dòng, tôi ghi rõ khi nghi ngờ).

1. Bối cảnh

Quy trình cắt tỉa điển hình có **ba giai đoạn**: (1) huấn luyện mô hình lớn quá tham số hóa; (2) cắt theo một tiêu chí; (3) tinh chỉnh mô hình đã cắt để lấy lại độ chính xác. Đằng sau là hai niềm tin phổ biến:

1. **Phải bắt đầu từ mô hình lớn.** Mô hình lớn mạnh hơn (về biểu diễn và tối ưu hóa), nên cắt từ nó tốt hơn là huấn luyện mô hình nhỏ từ đầu; nhiều bài (Li 2017, Luo 2017 tức ThiNet bài 06, He 2017) báo cáo train-from-scratch là baseline yếu hơn.
2. **Cả kiến trúc lẫn trọng số kế thừa đều quan trọng.** Do đó cả lĩnh vực tập trung vào việc chọn đúng "trọng số quan trọng" (bài 04, 05, 06, 07...).

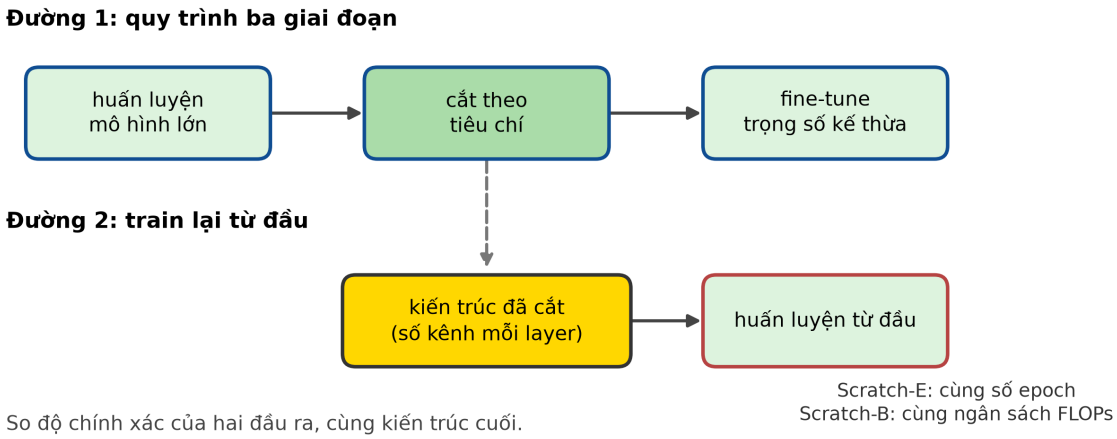
Khái niệm nền:

- **Structured pruning**: cắt nguyên kênh/filter/khối; mô hình nhỏ vẫn dày đặc nên chạy nhanh với thư viện thường.
- **Predefined target architecture**: người dùng định tỷ lệ cắt mỗi lớp (ví dụ cắt 50% kênh mỗi lớp); thuật toán chỉ chọn *kênh nào*, kiến trúc cuối biết trước.
- **Automatic target architecture**: tiêu chí so sánh toàn cục giữa các lớp (ví dụ Network Slimming, bài 05), nên thuật toán tự quyết định số kênh mỗi lớp; cắt thưa từng trọng số cũng thuộc loại này (Figure 2).
- **Train from scratch**: khởi tạo ngẫu nhiên (He init) mô hình nhỏ và huấn luyện từ đầu.

2. Ý tưởng chính

Tác giả chạy lại các phương pháp cắt có cấu trúc tiên tiến theo đúng kiến trúc và dữ liệu của bài gốc, rồi thêm một đường cơ sở công bằng: lấy kiến trúc đã cắt, **khởi tạo ngẫu nhiên và huấn luyện từ đầu** với ngân sách tính toán tương đương. Kết quả cho thấy: trong hầu hết trường hợp, huấn luyện từ đầu bằng hoặc tốt hơn tinh chỉnh từ trọng số kế thừa. Hai hệ quả: (a) với cắt có kiến trúc định sẵn, bỏ cả quy trình, huấn luyện thẳng mô hình đích; (b) với cắt tự động, điều có giá trị nằm *ở kiến trúc* được tìm ra (số kênh mỗi lớp) chứ không ở các trọng số "quan trọng". Đây là cách nhìn "cắt = tìm kiến trúc". Ngoại lệ: cắt thưa không cấu trúc ở ImageNet.

Cách đọc hình: đường 1 là quy trình cắt quen thuộc, đường 2 chỉ lấy lại kiến trúc (số kênh mỗi layer) từ bước cắt. Scratch-E và Scratch-B khác nhau ở ngân sách huấn luyện. Nếu đường 2 ngang hoặc hơn đường 1 thì giá trị của cắt nằm ở kiến trúc tìm được.



Hình 35: Hai đường để có cùng một kiến trúc đã cắt: kế thừa trọng số rồi fine-tune, hoặc khởi tạo ngẫu nhiên và huấn luyện từ đầu (minh hoạ, tự tính)

3. Phương pháp chi tiết

Paper không có công thức mới; đóng góp là phương pháp đánh giá.

3.1 Ngân sách huấn luyện từ đầu

Train mô hình nhỏ cùng số epoch với mô hình lớn có thể thiệt thòi (mỗi epoch rẻ hơn). Hai biến thể:

- **Scratch-E**: cùng *số epoch* với mô hình lớn.
- **Scratch-B**: cùng *ngân sách tính toán* (FLOPs) với mô hình lớn; trên ImageNet, nếu mô hình cắt tiết kiệm >2x FLOPs thì chỉ nhân đôi số epoch (nên vẫn ít tính toán hơn mô hình lớn). Khi tăng epoch, lịch giảm lr được co giãn tương ứng.

Tính toán của bước fine-tune **không** được cộng vào ngân sách (vì ngắn), điều này theo tác giả vẫn là có lợi cho quy trình cắt+fine-tune.

3.2 Thiết lập

- Dữ liệu: CIFAR-10, CIFAR-100, ImageNet; mạng VGG, ResNet, PreResNet, DenseNet.
- Phương pháp được kiểm tra: 4 *predefined* (Li 2017 L1-norm filter pruning, Luo 2017 ThiNet, He 2017b Regression-based feature reconstruction, He 2018a Soft Filter Pruning), 2 *automatic* (Liu 2017 Network Slimming, Huang & Wang 2018 Sparse Structure Selection), 1 không cấu trúc (Han 2015). Với 6 phương pháp đầu dùng đúng cặp (mô hình, dữ liệu) của bài gốc; CIFAR chạy 5 seed, báo trung bình và độ lệch chuẩn.
- Triển khai: dùng mã gốc nếu có; nếu không, tự cài lại quy trình 3 giai đoạn (kết quả tương tự); với ThiNet và He 2017b chỉ có mô hình đã cắt công khai, nên train lại cả mô hình lớn và nhỏ (mô hình lớn của họ chính xác hơn bản gốc nhờ PyTorch so với Caffe, vì vậy báo cáo **độ giảm tương đối** so với mô hình không cắt).
- Huấn luyện: SGD Nesterov momentum, lr giảm bậc thang, He init. CIFAR: train/fine-tune 160/40 epoch; ImageNet: 90/20 epoch. Fine-tune dùng lr nhỏ nhất của lúc train mô hình lớn.

4. Thí nghiệm và kết quả chính

Tất cả là độ chính xác (%), cao hơn tốt hơn.

4.1 Cắt có cấu trúc định sẵn

L1-norm filter pruning (Li 2017), Table 1

Dữ liệu	Mô hình (không cắt)	Mô hình cắt	Fine-tuned	Scratch-E	Scratch-B
CIFAR-10	VGG-16 93.63	VGG-16-A	93.41	93.62	93.78
	ResNet-56 93.14	ResNet-56-A	92.97	92.96	93.09
		ResNet-56-B	92.67	92.54	93.05
	ResNet-110 93.14	ResNet-110-A	93.14	93.25	93.22
		ResNet-110-B	92.69	92.89	93.60
ImageNet	ResNet-34 73.31	ResNet-34-A	72.56	72.77	73.03
		ResNet-34-B	72.29	72.55	72.91

Trên ImageNet cả hai Scratch-B hơn fine-tuned rõ rệt.

ThiNet (Luo 2017), Table 2 (độ giảm tương đối so với mô hình không cắt; không cắt VGG-16: 71.03 theo bài gốc, 71.51 trong cài đặt này; ResNet-50: 75.15 và 76.13):

	VGG-Conv	VGG-GAP	VGG-Tiny
Fine-tuned	−1.23	−3.67	−11.61
Scratch-E	−2.75	−4.66	−14.36
Scratch-B	+0.21	−2.85	−11.58

	ResNet50-30%	ResNet50-50%	ResNet50-70%
Fine-tuned	−6.72	−4.13	−3.10
Scratch-E	−5.21	−2.82	−1.71
Scratch-B	−4.56	−2.23	−1.01

Ngoại lệ duy nhất: Scratch-E cho VGG-Tiny (cắt rất mạnh, giảm 15x FLOPs, nên ngân sách của Scratch-E bị rút xuống rất nhiều). Scratch-B của VGG-Tiny cũng chỉ có ngân sách nhỏ hơn 7 lần mô hình gốc nhưng vẫn ngang fine-tuned.

Regression-based feature reconstruction (He 2017b), Table 3 (không cắt VGG-16 71.03/71.51; ResNet-50 75.51/76.13):

	VGG-16-5x	ResNet-50-2x
Fine-tuned	−2.67	−3.25
Scratch-E	−3.46	−1.55
Scratch-B	−0.51	−1.07

(Với VGG-16-5x, Scratch-E kém hơn fine-tuned; Scratch-B tốt hơn rõ rệt.)

Soft Filter Pruning (Phụ lục A): Scratch-E hơn mô hình cắt phần lớn thời gian, Scratch-B hơn trong gần như mọi trường hợp.

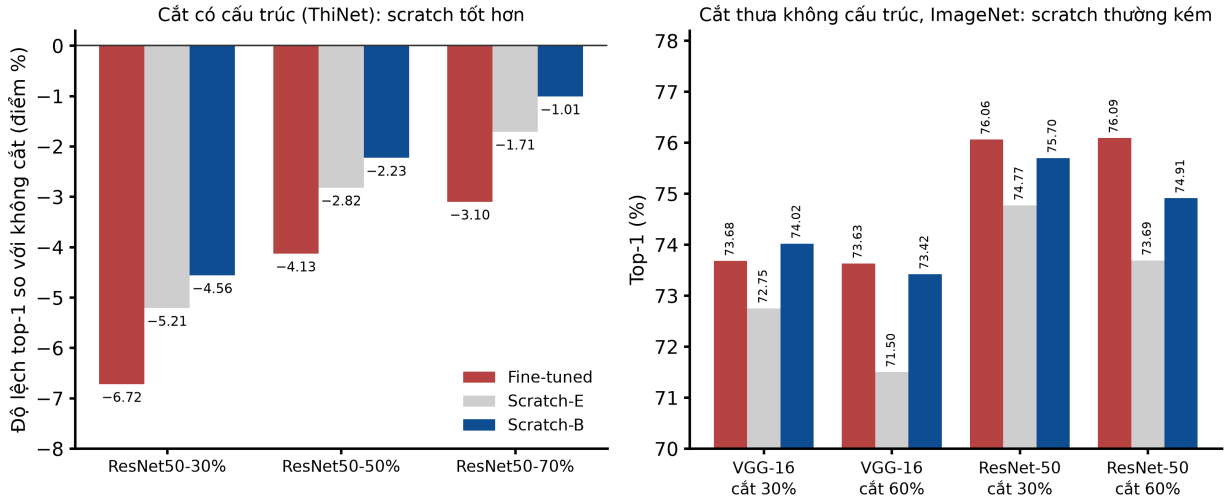
4.2 Cắt tự động có cấu trúc

Network Slimming (Liu 2017), Table 4: train from scratch ngang fine-tuned ở mọi mạng; Scratch-B hơn trong 8/10 thí nghiệm. Ví dụ CIFAR-10 VGG-19 (không cắt 93.53, cắt 70%): fine-tuned 93.60, Scratch-E 93.30, Scratch-B 93.81; CIFAR-100 VGG-19 (72.63, cắt 50%): 72.32 / 71.94 / 73.08; ImageNet VGG-11 (70.84, cắt 50%): 68.62 / 70.00 / 71.18.

Sparse Structure Selection (Huang & Wang), cắt khối residual, Table 5: ImageNet ResNet-50 (76.12) cắt thành ResNet-41/32/26; "pruned" (không fine-tune) / Scratch-E / Scratch-B: ResNet-41 75.44 / 75.61 / 76.17; ResNet-32 74.18 / 73.77 / 74.67; ResNet-26 71.82 / 72.55 / 73.41.

4.3 Cắt thưa không cấu trúc (Han 2015), Table 6

- CIFAR: ở tỷ lệ cắt ≤80%, Scratch-E đôi khi kém fine-tuned nhưng Scratch-B ít nhất ngang. Ở 95%, fine-tune đôi khi hơn train-from-scratch (ví dụ CIFAR-10 DenseNet-BC-100 95%: 94.19 fine-tuned vs 92.91 Scratch-E vs 93.44 Scratch-B).
- **ImageNet: Scratch-B thường kém fine-tuned đáng kể**: VGG-16 (73.37) cắt 60%: 73.63 vs 71.50 vs 73.42; cắt 30%: 73.68 / 72.75 / 74.02. ResNet-50 (76.15): cắt 30%: 76.06 / 74.77 / 75.70; cắt 60%: 76.09 / 73.69 / 74.91.
- Tác giả cho nguyên nhân có thể là mạng rất thưa khó train trực tiếp, độ phức tạp của ImageNet, và cắt không cấu trúc làm đổi phân bố trọng số mạnh hơn (Phụ lục F). Đây là khác biệt quan trọng giữa hai loại cắt.



Hình 36: Fine-tuned, Scratch-E và Scratch-B: trái là ThiNet trên ResNet-50 (Table 2), phải là cắt thưa không cấu trúc trên ImageNet (Table 6) (vẽ lại từ số liệu trong ghi chú)

Cách đọc hình: ở panel trái, giá trị là độ giảm top-1 so với mô hình không cắt, nên cột càng gần 0 càng tốt; Scratch-B tốt nhất ở cả ba mức. Ở panel phải, cắt thưa trên ImageNet đảo ngược: fine-tuned thường cao hơn Scratch-E, và Scratch-B chỉ hơn fine-tuned ở một trong bốn cấu hình (VGG-16 cắt 30%).

4.4 Mô hình bị cắt rất mạnh (Phụ lục C)

Train from scratch càng hơn nhiều khi tỷ lệ cắt cao. Network Slimming (Table 13): PreResNet-164/CIFAR-10 (95.04) cắt 80%: fine-tuned 91.76, Scratch-E 93.21, Scratch-B 93.49; cắt 90%: 82.06 / 87.55 / 88.44. L1-norm ResNet-56 (93.14) cắt 90% (Table 14): 89.17 / 91.02 / 91.93.

4.5 Chuyển sang phát hiện đối tượng (Phụ lục B)

PASCAL VOC 07, Faster-RCNN, backbone ResNet-34-A/B từ Li 2017: mô hình train from scratch trên phân loại rồi chuyển sang detection vượt mô hình fine-tuned. Ví dụ ResNet34-B (mAP): Prune-C 70.84, Prune-D 69.62, Scratch-E 71.68, Scratch-B 71.26 (không cắt ResNet-34: 71.69). Prune-C là cắt trên backbone phân loại, Prune-D là cắt sau khi chuyển sang detection; bất ngờ là Prune-C hơn Prune-D, tác giả đoán cắt sớm giúp tránh kẹt vào cực tiểu xấu do kế thừa trọng số.

5. Phân tích / ablation

- **Fine-tune thêm epoch không giúp** (Phụ lục D, Table 16): Fine-tune-40/80/160 cho độ chính xác gần như nhau (ví dụ VGG-16-A: 93.40/93.45/93.45 so với Scratch-E 93.62). Nên Scratch không thắng chỉ do fine-tune quá ít.
- **Kéo dài lịch huấn luyện** (Phụ lục E, 160 → 300 epoch CIFAR, Table 17): kết luận vẫn giữ, scratch ít nhất ngang fine-tuned.
- **Cắt = tìm kiến trúc (Mục 5):** so kiến trúc do cắt tự động tìm ra với kiến trúc cắt đều (cùng tỷ lệ mọi layer), cả hai train from scratch cùng số epoch.
  - VGG-16 trên CIFAR-10 (Figure 3 trái): kiến trúc Network Slimming đạt cùng độ chính xác với **ít hơn 5 lần tham số** so với cắt đều. Cắt thưa so với thưa đều cũng cho kết quả tương tự (Figure 3 phải).
  - Mẫu cắt của VGG khá ổn định giữa các lần chạy (Table 7: cắt 60% kênh, tổng chiều rộng 4224 → 1689.2; layer 1–7 gần như giữ nguyên, layer sâu 9–13 bị cắt mạnh, ví dụ layer 10 từ 512 còn 63.0±1.9).
  - **Không phải lúc nào cũng thắng cắt đều:** với PreResNet-164 (CIFAR-10), PreResNet-110 và DenseNet-40 (CIFAR-100), kiến trúc cắt không hiệu quả hơn đáng kể so với cắt đều (Figure 5). Các mẫu cắt này gần đều giữa các stage (Phụ lục G). VGG có độ dư thừa mất cân bằng giữa các stage nên cắt giúp ích.
  - **Nguyên tắc thiết kế chuyển giao được:** lấy số kênh trung bình theo stage từ kiến trúc đã cắt để thiết kế ("Guided Pruning/Sparsification") cho hiệu quả ngang kiến trúc cắt trực tiếp; mẫu rút từ VGG-16/CIFAR-10 áp sang VGG-19/CIFAR-100 ("Transferred Guided") vẫn tốt hơn cắt đều đáng kể (Figure 6).
  - So với NAS (RL, tiến hóa): cắt chỉ cần một lần huấn luyện, nhưng không gian tìm kiếm chỉ gồm các mạng con của mạng lớn.
- **So với Lottery Ticket (Mục 6):** tác giả liệt kê bốn khác biệt cài đặt so với Frankle & Carbin: (a) họ chỉ cắt thưa không cấu trúc; (b) họ dùng mạng nhỏ nông (<6 layer); (c) họ dùng Adam/lr rất thấp, còn bài này dùng SGD momentum lr đầu 0.1 chuẩn; (d) họ không có ImageNet. Thí nghiệm (Figure 7, Table 8, 9): với lr 0.1, "winning ticket" (khởi tạo gốc) **không** hơn khởi tạo ngẫu nhiên, cả với cắt có cấu trúc và không cấu trúc; chỉ khi lr nhỏ (0.01) cho cắt thưa trên CIFAR thì winning ticket mới hơn, nhưng lr nhỏ cho độ chính xác thấp hơn lr chuẩn. Ví dụ cắt cấu trúc L1-norm, lr 0.1: VGG-16-A winning ticket 93.62 vs random 93.60; ResNet-56-B 92.78 vs 92.90. Cắt thưa one-shot VGG-16, lr 0.1, cắt 95%: 93.18 vs 93.05.

6. Giới hạn

Do tác giả nêu:

- Kết luận chủ yếu cho structured pruning; với cắt thưa không cấu trúc trên ImageNet, train from scratch kém hơn (mục 4.3).
- Cắt+fine-tune vẫn đáng giá trong hai trường hợp: (a) đã có mô hình lớn sẵn và ít hoặc không có ngân sách huấn luyện (và phương pháp không đòi sửa quá trình train mô hình lớn); (b) cần nhiều mô hình ở nhiều kích cỡ, hoặc chưa biết kích cỡ mong muốn, khi đó train một mô hình lớn rồi cắt theo nhiều tỷ lệ.

- Không phải mọi kiến trúc đều hưởng lợi từ kiến trúc cắt (ResNet/DenseNet gần đều).
- Một số mâu thuẫn với tài liệu gốc có thể do siêu tham số/augmentation kém được chọn kỹ và ngân sách tính toán không công bằng ở các baseline.

Nhận xét:

- Toàn bộ thí nghiệm là CNN phân loại ảnh (và một thí nghiệm detection); không có mô hình ngôn ngữ, VLM, hay quy mô tỷ tham số, nơi train-from-scratch tốn hàng nghìn lần tính toán.
- Scratch-B dùng ngân sách lớn (bằng mô hình lớn), nên "train từ đầu" ở đây không rẻ; phù hợp nếu train là rẻ (CIFAR), không phù hợp khi tiền huấn luyện là phần chi phí chính.
- Kết luận phụ thuộc vào recipe huấn luyện chuẩn (SGD lr 0.1, 160 epoch...); các bài sau (Blalock 12) lưu ý rằng so sánh cắt phụ thuộc mạnh vào cài đặt.
- Dữ liệu huấn luyện cho mô hình nhỏ ở đây giống mô hình lớn; trong chưng cất, mô hình nhỏ còn có tín hiệu từ giáo viên, mà paper không xét.

7. Liên hệ với các bài khác

- **04 Pruning filters, 05 Network slimming, 06 ThiNet:** ba trong các đối tượng được kiểm chứng (Table 1, 4, 2). Với ThiNet, mục 4.4 của bài 06 ("cắt+fine-tune hơn train from scratch") bị phản biện bằng ngân sách đủ lớn (Scratch-B).
- **07 Molchanov:** bài 07 tuyên bố không dính "pruning paradox" của bài này (ResNet20/CIFAR-10); xem lại để thấy hai bên đồng ý rằng khác biệt nằm trong độ lệch chuẩn.
- **08, 09 LTH:** Mục 6 của bài này là bên phản biện LTH; bài 09 sau đó giải thích mâu thuẫn bằng độ ổn định trước nhiễu SGD: ở mức thưa tầm thường train lại từ khởi tạo ngẫu nhiên là đủ, điều này khớp phần "thưa vừa phải" của bài 11.
- **10 SNIP:** cũng chọn cấu trúc thưa trước huấn luyện, ủng hộ cách nhìn "cấu trúc quan trọng hơn trọng số".
- **12 State of pruning (Blalock):** hệ thống hóa vấn đề baseline và thiếu chuẩn so sánh mà bài này gợi ra.
- **38 Sheared LLaMA, 39–40 Minitron, 43 Pruning vs scratch:** bản LLM của câu hỏi này; kết quả ở đó cho rằng cắt + tiếp tục huấn luyện thường tiết kiệm token rất lớn so với train từ đầu, khác bối cảnh CNN ở đây.
- **42 Puzzle, 41 Flextron:** tìm kiến trúc từ mô hình lớn, đúng tinh thần "cắt = tìm kiến trúc".

8. Gợi ý áp dụng

Liên quan cao về mặt phương pháp luận, thấp về kết quả cụ thể.

- **Cắt = tìm kiến trúc** gợi ý dùng điểm quan trọng để quyết định *độ rộng/độ sâu mỗi khối* (kết quả: VGG có mẫu cắt ổn định, nhất quán giữa các lần chạy), rồi có thể khởi tạo lại.
- **Bài học thực nghiệm:** báo cáo độ chính xác theo ngân sách tính toán đồng nhất (Scratch-B so với Scratch-E); cẩn thận với recipe lr/epoch khi so sánh.

9. Thí nghiệm gợi ý

2. **Cắt đều vs cắt do điểm quyết định:** cùng ngân sách tham số, so kiến trúc có độ rộng MLP đều giữa các block với kiến trúc theo điểm quan trọng (ít cắt ở block đầu, nhiều ở block sâu hoặc ngược lại), cả hai khởi tạo từ trọng số đã cắt rồi chưng cất.
3. **Ngân sách đồng nhất:** cho mọi baseline cùng số token chưng cất; báo cáo thêm đường "cùng FLOPs huấn luyện" như Scratch-B.

10. Tóm tắt công thức / khái niệm

Khái niệm	Ý nghĩa
Pipeline 3 giai đoạn	Train mô hình lớn, cắt, fine-tune
Predefined vs automatic	Kiến trúc đích do người định vs do thuật toán cắt toàn cục quyết định
Scratch-E	Train từ đầu cùng số epoch với mô hình lớn
Scratch-B	Train từ đầu cùng ngân sách FLOPs (ImageNet: nhân đôi epoch nếu tiết kiệm >2x)
Độ giảm tương đối	Độ chính xác trừ độ chính xác mô hình không cắt (cùng framework)
"Cắt = tìm kiến trúc"	Giá trị của cắt tự động là số kênh mỗi layer, không phải trọng số kế thừa
Guided / Transferred Guided	Dùng mẫu kênh trung bình theo stage làm quy tắc thiết kế, có thể chuyển sang VGG/dữ liệu khác
Kết luận LTH	Với lr 0.1 (chuẩn), khởi tạo winning ticket không hơn khởi tạo ngẫu nhiên
Ngoại lệ	Cắt thưa không cấu trúc trên ImageNet: fine-tune thường hơn scratch

11. Kết luận ngắn

Với cắt có cấu trúc trên CNN, mô hình cắt rồi tinh chỉnh thường không hơn (và hay kém hơn) cùng kiến trúc huấn luyện từ đầu với ngân sách tương đương; nên giá trị của cắt nằm ở việc tìm kiến trúc hiệu quả (nhất là trên VGG), và ở việc tiết kiệm khi đã có sẵn mô hình lớn hoặc cần nhiều kích cỡ.

Link gốc: https://arxiv.org/abs/1810.05270

Bài 12

What is the State of Neural Network Pruning? (2020)

Tác giả: Davis Blalock, *Jose Javier Gonzalez Ortiz*, Jonathan Frankle, John Guttag (MIT CSAIL; * đóng góp ngang nhau).
Venue / arXiv: arXiv:2003.03033 (6/3/2020); bản trong file ghi là Proceedings of the 3rd MLSys Conference, Austin, TX, 2020.
Nguồn: https://arxiv.org/abs/2003.03033
Chủ đề: Meta-analysis 81 bài pruning; danh mục lỗi trong cách báo cáo/so sánh; khuyến nghị thực hành; thư viện ShrinkBench. Đây là bài **"cách đọc paper pruning một cách phê phán"**.
Vị trí trong lộ trình: Nên đọc sau các bài nền tảng 01-11 (OBD/OBS, Han 2015, Zhu & Gupta, các bài structured pruning, Lottery Ticket, SNIP, Rethinking the value of pruning). Bài này không đề xuất thuật toán mới; nó cho bạn "bộ lọc nghi ngờ" để đọc mọi bài sau (13-49). Bài kế tiếp: 13 (Sixteen Heads), mở đầu nhánh pruning cho Transformer.
Chương nền liên quan: nền 01: Vì sao mạng cắt được, nền 02: Ma trận thưa và chi phí thật, nền 07: Đo đúng một kết quả pruning

> Ghi chú học tập bằng tiếng Việt, không phải bản dịch.

1. Bối cảnh

Pruning (cắt tỉa) = bỏ bớt tham số của một mạng đã có để mạng nhỏ/nhanh hơn mà độ chính xác gần như giữ nguyên. Ý tưởng có từ cuối thập niên 1980 (Janowsky 1989, Mozer & Smolensky 1989, Karnin 1990), bùng nổ sau 2015 nhờ deep learning.

Đến 2020 có hàng chục phương pháp, và mỗi bài đều tuyên bố "state of the art". Các tác giả đặt những câu hỏi tưởng đơn giản:

- Kỹ thuật nào cho đánh đổi độ chính xác/hiệu quả tốt nhất?
- Có chiến lược nào hợp riêng với kiến trúc/dataset nào không?
- Lựa chọn thiết kế cấp cao nào hiệu quả nhất?

Kết luận trung tâm (nguyên văn ý): **các câu hỏi này không thể trả lời được** vì văn liệu quá phân mảnh. Bài này là bản "kiểm toán" của lĩnh vực.

Thuật ngữ nền:

- **Unstructured pruning**: bỏ từng trọng số riêng lẻ → ma trận thưa, khó tăng tốc trên phần cứng thông thường.
- **Structured pruning**: bỏ cả neuron/filter/channel (và với Transformer là head, hàng/cột của MLP, cả layer) → mạng nhỏ dạng dense, tăng tốc thật.
- **Mask** $M \in \{0,1\}^{|W|}$: vector nhị phân nói tham số nào còn sống.
- **Fine-tuning**: huấn luyện tiếp sau khi cắt để phục hồi.
- **FLOPs** (ở đây là multiply-adds): số phép nhân-cộng khi suy luận.

2. Ý tưởng chính

Tác giả tổng hợp kết quả từ **81 bài** (79 bài từ 2010 trở đi + 2 bài kinh điển LeCun 1990 và Hassibi 1993) và tự cắt tỉa hơn **800 mạng** trong điều kiện kiểm soát. Phân tích đưa ra các kết luận sau: (1) vài kết luận nhất quán (pruning hiệu quả, magnitude pruning là baseline mạnh, ...); (2) kết luận lớn nhất là *việc so sánh gần như bất khả thi* vì thiếu benchmark, metric và thực hành chuẩn; (3) họ đề xuất danh sách thực hành tốt và thư

viện **ShrinkBench** để so sánh công bằng; (4) vài thí nghiệm nhỏ cho thấy các "bẫy" thường gặp (chọn metric, chọn mô hình gốc, chọn điểm hoạt động) có thể đảo ngược thứ hạng giữa các phương pháp.

3. Phương pháp chi tiết

Bài là meta-analysis nên "phương pháp" gồm khung phân loại, quy trình chuẩn hóa dữ liệu và ShrinkBench.

3.1 Khung chung: Algorithm 1 (prune + fine-tune)

Gần như mọi chiến lược trong corpus là biến thể của thuật toán Han et al. (2015):

```
Input: N = số vòng prune; X = dữ liệu train/fine-tune
W <- initialize()
W <- trainToConvergence(f(X; W))
M <- 1^{|W|}                        # mask toàn 1
for i in 1..N:
    M <- prune(M, score(W))          # bỏ tham số điểm thấp
    W <- fineTune(f(X; M ⊙ W))       # train tiếp để hồi phục
return M, W
```

Mạng sau cắt: $f(x; M \odot W')$, trong đó W' có thể khác W (nếu có rewinding/re-init), $\odot$ là nhân từng phần tử.

3.2 Bốn trục phân biệt các phương pháp

Trục	Lựa chọn trong paper
Structure	unstructured (từng trọng số) vs structured (neuron, filter, channel)
Scoring	theo $\|w\|$, theo hệ số "importance" học được, theo đóng góp vào activation hoặc gradient; so sánh **local** (trong từng layer, vd Han 2015) hay **global** (toàn mạng, vd Lee 2019b, Frankle & Carbin 2019)
Scheduling	cắt một phát (one-shot, Liu 2019), cắt lặp cố định tỉ lệ mỗi vòng (Han 2015), hoặc lịch phức tạp (Gale 2019)
Fine-tuning	tiếp tục từ trọng số đã train (phổ biến nhất), **rewinding** về trạng thái sớm hơn (Frankle 2019), hoặc khởi tạo lại hoàn toàn (Liu 2019)

Mẹo đọc paper: khi gặp bài mới, hãy điền bốn ô này trước. Hai bài khác nhau ở cả bốn ô thì kết quả của chúng gần như không so sánh được.

Cách đọc hình: ba khối phía trên là thuật toán chung của hầu hết bài trong corpus. Mỗi khối chứa một hoặc hai lựa chọn nằm ở bốn ô phía dưới. Hai bài chỉ so sánh được khi các ô này giống nhau ngoài trục đang được khảo sát.

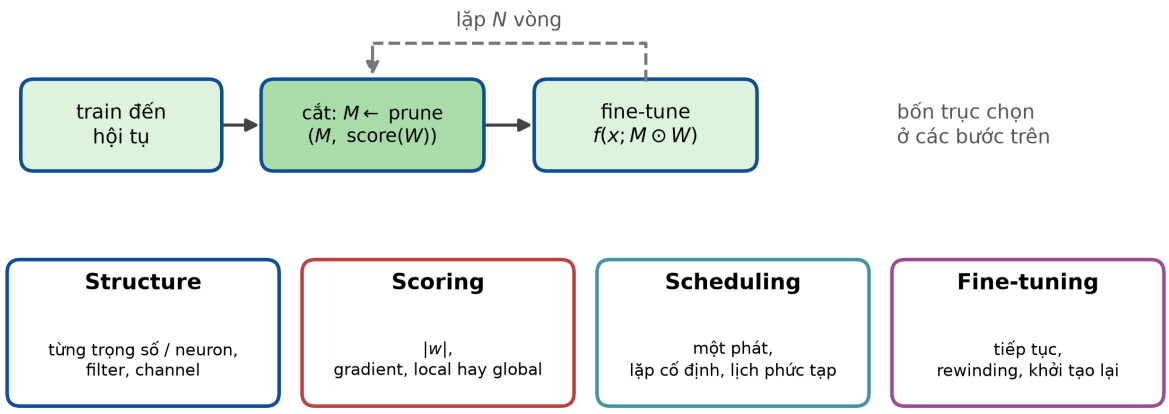
3.3 Hai metric hiệu quả, và định nghĩa nên dùng

$$\boxed{\text{compression ratio} = \frac{\text{original size}}{\text{new size}}}, \qquad \boxed{\text{theoretical speedup} = \frac{\text{original multiply-adds}}{\text{new multiply-adds}}}$$

Tác giả nhấn mạnh: không có lý do gì chỉ báo cáo một trong hai. Nhiều bài lại định nghĩa "compression ratio" là $1 - \frac{\text{compressed}}{\text{original}}$, trái với truyền thống ngành nén dữ liệu.

3.4 Chuẩn hóa để vẽ Figure 1

Vì nhiều bài chỉ báo "thay đổi accuracy" hoặc "% pruned" mà không cho baseline, tác giả **nhân tỉ lệ đã báo cáo với giá trị khởi đầu chuẩn** (median kích thước/FLOPs được báo cáo cho kiến trúc đó trên toàn bộ các



Hình 37: Vòng lặp prune và fine-tune chung của văn liệu cùng bốn trục phân biệt các phương pháp (minh hoạ, tự tính)

bài). Họ tự nhận cách này "không hoàn hảo" nhưng giúp kiểm soát phần nào việc các phương pháp bắt đầu từ baseline khác nhau.

3.5 ShrinkBench

Thư viện PyTorch mã nguồn mở. Người dùng chỉ cần viết một **callback** tính mask cho các tensor tham số tại một iteration; ShrinkBench tự áp mask, cập nhật mạng theo quy trình train/fine-tune chuẩn, tính metric, chạy nhiều mô hình/dataset/seed/mức cắt và vẽ biểu đồ. Baselines đi kèm (chỉ "lấy cảm hứng", không phải tái hiện chính xác bài gốc):

- Global Magnitude: bỏ trọng số $|w|$ nhỏ nhất trên toàn mạng.
- Layerwise Magnitude: bỏ $|w|$ nhỏ nhất trong từng layer.
- Global / Layerwise Gradient Magnitude: dùng $|w \cdot \nabla w|$ tính trên một batch.
- Random: mỗi trọng số bị bỏ độc lập với xác suất bằng tỉ lệ cần cắt; hữu ích làm công cụ debug.

4. Thí nghiệm và kết quả chính

4.1 Những điều nhất quán trong văn liệu (Mục 3)

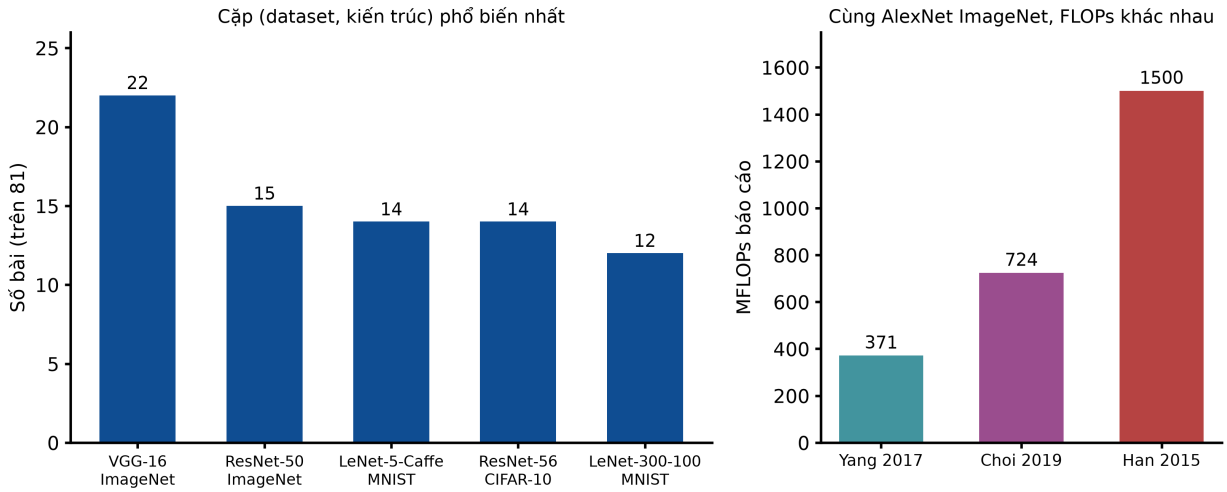
1. **Pruning hiệu quả**: nhiều phương pháp nén đáng kể mà mất rất ít hoặc không mất accuracy; với nén nhẹ thỉnh thoảng accuracy còn tăng (Han 2015, Suzuki 2018).
2. **Nhiều phương pháp thắng random pruning** (ít nhất ở mức cắt lớn); ở mức cắt nhẹ điều này không phải lúc nào cũng đúng (Morcos 2019).
3. **Phân bổ tham số thông minh giữa các layer** hoặc cắt global tốt hơn cắt đều mọi layer.
4. Với số bước fine-tune bằng nhau, nhiều phương pháp cho mô hình tốt hơn **train từ đầu với cùng pattern thưa** (với mức cắt đủ lớn).
5. **Sparse > dense khi cố định số tham số**: ví dụ tăng kích thước danh nghĩa của ResNet-20 trên CIFAR-10 rồi làm thưa để giữ nguyên số tham số thì lỗi giảm (Lee 2019a).
6. Figure 1 (ImageNet, so mô hình pruned với họ kiến trúc không pruned): pruning cải thiện đánh đổi size/speed vs accuracy của một kiến trúc; **nhưng thường không bằng việc đổi sang kiến trúc tốt hơn**; và pruning hiệu quả hơn với kiến trúc vốn kém hiệu quả.

4.2 Những vấn đề phân mảnh (Mục 4-5), các con số quan trọng

Vấn đề	Số liệu từ paper
Thiếu so sánh	hơn 1/4 số bài không so sánh với phương pháp pruning nào; thêm 1/4 chỉ so sánh 1 phương pháp; hầu hết so sánh ≤ 3
Bỏ qua phương pháp cũ	cả thập niên 1990 gần như bị bỏ ngoài OBD và OBS; Han 2015 "tái phát hiện" magnitude pruning của Janowsky 1989; Lee 2019b tái phát hiện saliency của Mozer & Smolensky 1989a; He 2018a tái phát hiện "reviving" của Tresp 1997
Phân mảnh dataset/kiến trúc	81 bài dùng **49 dataset, 132 kiến trúc, 195 cặp (dataset, architecture)**
Cặp phổ biến nhất	VGG-16 trên ImageNet: 22/81 bài; ResNet-50 ImageNet: 15; LeNet-5-Caffe MNIST: 14; ResNet-56 CIFAR-10: 14; LeNet-300-100 MNIST: 12 (Table 1). Ba trong sáu cặp phổ biến nhất dùng MNIST
Chỉ 37/81 bài có kết quả trên bất kỳ cấu hình chung nào trong Figure 3	(bốn cặp phổ biến nhất không tính MNIST)
Điểm hoạt động	đa số dùng ≤ 3 cặp (dataset, arch), và thường **chỉ 1 điểm** để mô tả cả đường đánh đổi
Độ lệch chuẩn	trong Figure 3 chỉ He 2018 (CIFAR-10) báo độ lệch chuẩn
FLOPs không nhất quán	chênh tới 4 lần cho cùng kiến trúc/dataset: AlexNet ImageNet được báo 371 MFLOPs (Yang 2017), 724 MFLOPs (Choi 2019), 1500 MFLOPs (Han 2015)

Ngoài ra: "ResNet-50" mơ hồ vì có hai họ ResNet (He 2016a,b); "VGG-16" thường là biến thể tự chế (bỏ FC, đổi kích thước FC, thêm dropout/batchnorm); "Pruned%" có thể là phần còn lại hoặc phần bị bỏ; "speedup" có khi là wall time, có khi là tỉ lệ FLOPs.

Figure 5 cho thấy: cùng là **magnitude pruning không cấu trúc trên ResNet-50 ImageNet**, các bài khác nhau về cài đặt/lịch/fine-tuning cho kết quả phân tán gần bằng độ phân tán giữa các *phương pháp* khác nhau. Nghĩa là sai khác do "chi tiết cài đặt" ngang với sai khác do "ý tưởng thuật toán".



Hình 38: Số bài dùng mỗi cặp (dataset, kiến trúc) phổ biến nhất và FLOPs khác nhau báo cho cùng AlexNet ImageNet (vẽ lại từ số liệu trong ghi chú)

Cách đọc hình: panel trái đếm số bài trong 81 bài dùng từng cặp; chỉ cặp đứng đầu cũng chưa tới một phần ba tổng số. Panel phải cho thấy cùng một mô hình bị báo FLOPs chênh hơn 4 lần giữa các bài, nên so sánh

theo con số tự báo cáo là rủi ro.

4.3 Thí nghiệm ShrinkBench (>800 mạng; thanh sai số = 1 độ lệch chuẩn qua 3 lần chạy trên CIFAR-10)

- **Metric không thể thay thế nhau** (Figure 6, ResNet-18 ImageNet): Global thắng Layerwise khi cố định *kích thước mô hình*, nhưng **ngược lại** khi cố định *theoretical speedup*. Lý do hợp lý (diễn giải): Global thường cắt các layer có nhiều tham số nhưng ít FLOPs.
- **Kết quả thay đổi theo mô hình, dataset, mức cắt** (Figure 7, CIFAR-VGG và ResNet-56 trên CIFAR-10): nói chung Global > Layerwise, Magnitude > Gradient, Random tệ nhất. Nhưng nếu chỉ nhìn CIFAR-VGG ở compression < 10 thì có thể kết luận nhầm là Global Gradient tốt nhất; và Global Gradient luôn hơn Layerwise Magnitude trên CIFAR-VGG nhưng luôn kém hơn trên ResNet-56.
- **Seed**: gần điểm "sụp" (Global Gradient, compression 16) các seed khác nhau cho accuracy **0.88 so với 0.61** do ngẫu nhiên khi chọn minibatch.
- **Mô hình gốc phải giống nhau** (Figure 8): hai ResNet-56 huấn luyện bằng Adam với $\eta = 10^{-3}$ (Weights A) và $\eta = 10^{-4}$ (Weights B). Với A hai phương pháp xấp xỉ nhau; với B Global tốt hơn ở compression cao. Báo cáo *thay đổi* accuracy so với gốc **không đủ** để khử nhiễu này: Layerwise+Weights B có thể trông tốt hơn Global+Weights A dù khi giữ mô hình gốc cố định nó chưa bao giờ hơn.

5. Phân tích / ablation

Paper không có ablation theo nghĩa thuật toán; phần tương đương là các "kiểm soát biến nhiễu". Biến nhiễu được liệt kê: độ chính xác và hiệu quả của mô hình ban đầu; data augmentation/tiền xử lý; ngẫu nhiên khi khởi tạo/train/fine-tune (optimizer, siêu tham số, learning-rate schedule); lịch prune và fine-tune; thư viện deep learning; khác biệt nhỏ trong code/môi trường. Tác giả cho rằng gần như không bài nào kiểm soát hết, trừ khi tác giả tự chạy lại tất cả với cùng code (hiếm; họ chỉ nêu Gale 2019 và Liu 2019 và hai bài này là các phân tích hơn là đề xuất phương pháp mới).

Một lập luận quan trọng: các thay đổi accuracy ở điểm hoạt động được báo cáo thường **< 1%**, nên một biến nhiễu nhỏ cũng đủ đổi thứ hạng.

Khuyến nghị thực hành của tác giả (Mục 6), đây là phần nên chép thành checklist:

1. Nêu chính xác kiến trúc, dataset, metric, ở dạng có cấu trúc.
2. Dùng ít nhất **3 cặp (dataset, architecture)** hiện đại, quy mô lớn; MNIST/mô hình đồ chơi không tính.
3. Báo cáo cả compression ratio **và** theoretical speedup.
4. Với ImageNet và dataset nhiều lớp: báo cả Top-1 và Top-5.
5. Báo metric của mô hình đối chứng (thường là mô hình gốc chưa cắt).
6. Vẽ đường đánh đổi cùng các phương pháp cạnh tranh.
7. Dùng **≥ 5 điểm hoạt động**, gợi ý compression $\{2, 4, 8, 16, 32\}$.
8. Báo trung bình và độ lệch chuẩn mẫu thay vì một lần chạy.
9. Cùng thư viện, data loading, code cho mọi phương pháp được so sánh.
10. Reviewer nên đòi mức nghiêm ngặt cao hơn với các bài tuyên bố phương pháp tốt hơn.

6. Giới hạn

Do tác giả nêu:

- Phân tích thiên về phân loại ảnh (Top-1/Top-5) và hai metric FLOPs/số tham số vì "đại đa số bài trong corpus" làm vậy; tác giả thừa nhận hai metric này chỉ là proxy lỏng cho latency, throughput, bộ nhớ, năng lượng thực tế.
- Cách chuẩn hóa để vẽ Figure 1 "không hoàn hảo".
- Các baseline của ShrinkBench chỉ lấy cảm hứng từ heuristic, không tái hiện bài gốc.

Nhận xét:

- Corpus dừng ở đầu 2020: không có Transformer/NLP/LLM (trừ ngụ ý chung), không có structured pruning trên mô hình ngôn ngữ, không có distillation kết hợp. Mọi con số cụ thể ("magnitude là baseline mạnh") cần kiểm lại cho hệ Transformer; các bài 13-17 và 22-30 cho thấy bức tranh khác.
- Thí nghiệm ShrinkBench dùng CIFAR-10 là chính (xem Fig. 6-8 là CIFAR-10/ImageNet nhỏ); tức chính bài này cũng chỉ có vài cặp (dataset, arch), và các khuyến nghị "≥ 3 cặp lớn" không được tự áp dụng triệt để.
- Phát hiện "pruning kém hơn đối kiến trúc" dựa trên các đường cong tự báo cáo đã chuẩn hóa, nên bản thân chịu đúng những nhiễu mà bài phê phán.

7. Liên hệ với các bài khác

- **01 OBD/OBS**: được nhắc là hai bài kinh điển hiếm hoi còn được so sánh; phần lớn văn liệu 1990s bị bỏ quên.
- **02 Han 2015**: nguồn của Algorithm 1 (train → prune → fine-tune lặp) và magnitude pruning.
- **03 Zhu & Gupta**: một ví dụ của "scheduling" khác (lịch cắt dần).
- **04-06 (Pruning filters, Network slimming, ThiNet)** và **07 Molchanov**: các biến thể structured/scoring; hãy dùng 4 trục ở mục 3.2 để so sánh chúng.
- **08-09 Lottery Ticket, Linear mode connectivity**: "rewinding" xuất hiện trong trục fine-tuning; Frankle là đồng tác giả bài này.
- **10 SNIP**: ví dụ cắt tại khởi tạo bằng gradient (nhóm "gradient-based").
- **11 Rethinking the value of pruning**: liên quan trực tiếp đến phát hiện "pruned vs train-from-scratch"; bài 12 cho thấy kết luận này phụ thuộc vào cài đặt.
- **13-17** (bài sau trong lộ trình): đều có thể được "chấm" bằng checklist ở mục 5 (xem 4 câu hỏi ở phần Thí nghiệm gợi ý).
- **43 Pruning vs scratch** và **37 Repair changes which layers**: các bài hiện đại giữ tinh thần "so sánh có kiểm soát".

8. Gợi ý áp dụng

Figure 8 cho thấy sự khác biệt về mô hình gốc có thể đủ làm đảo thứ hạng.

5. **Baseline random và magnitude** luôn nên có làm chuẩn debug.
6. Đối chiếu với bài 12: thứ bạn so là *cấu trúc cắt* (trục Structure) nên phải giữ nguyên ba trục còn lại (scoring, scheduling, fine-tuning).

Mức liên quan: **cao về phương pháp luận, thấp về kỹ thuật**.

9. Thí nghiệm gợi ý

1. **Bảng "4 trục" cho mọi paper trong lộ trình**: lập một bảng (Structure, Scoring, Scheduling, Fine-tuning) + (dataset/arch, metric hiệu quả, số điểm hoạt động, có độ lệch chuẩn không, đối chứng có không) cho bài 13-17. Chỉ ra ô nào "không báo cáo". Kiểm tra thứ hạng có đảo như Global vs Layerwise trong Figure 6 không.
2. **Độ nhạy theo seed/checkpoint**: lặp một cấu hình cắt với 3 seed và 2 checkpoint gốc khác nhau (ví dụ trước/sau một epoch fine-tune), ghi biên độ dao động để biết mức chênh nào là đáng tin.

10. Tóm tắt công thức / khái niệm

Khái niệm	Ý nghĩa
$f(x; M \odot W')$	mạng đã cắt: mask M nhân từng phần tử với trọng số
Compression ratio	$\frac{\text{original size}}{\text{new size}}$ (theo paper; không phải $1 - \ldots$)
Theoretical speedup	$\frac{\text{original multiply-adds}}{\text{new multiply-adds}}$
Structure / Scoring / Scheduling / Fine-tuning	4 trục phân loại phương pháp
Global vs layerwise	so điểm toàn mạng hay trong từng layer
Rewinding	quay trọng số về trạng thái sớm hơn rồi mới train tiếp
Operating point	một điểm (mức nén, accuracy) trên đường đánh đổi
ShrinkBench	thư viện đánh giá pruning chuẩn hóa (PyTorch)
Nhiễu "initial model"	khác mô hình gốc có thể đảo thứ hạng phương pháp, kể cả khi báo thay đổi accuracy
Dải nén gợi ý	$\{2, 4, 8, 16, 32\}$, ≥ 5 điểm

11. Kết luận ngắn

Pruning "hoạt động", nhưng phần lớn con số trong văn liệu 2010-2019 không so sánh được với nhau: ít so sánh chéo, 195 cặp (dataset, arch) rời rạc, metric mơ hồ, một điểm hoạt động, không có độ lệch chuẩn, mô hình gốc khác nhau. Mang theo bài này như một checklist: với mỗi paper tiếp theo, hỏi "cùng mô hình gốc chưa, cùng metric chưa, bao nhiêu điểm, bao nhiêu seed, đối chứng là gì".

Link gốc: https://arxiv.org/abs/2003.03033

Bài 13

Are Sixteen Heads Really Better than One? (2019)

Tác giả: Paul Michel, Omer Levy, Graham Neubig (CMU; Facebook AI Research).
Venue / arXiv: arXiv:1905.10650 (v3, 4/11/2019); NeurIPS 2019 (33rd Conference on Neural Information Processing Systems).
Nguồn: https://arxiv.org/abs/1905.10650
Chủ đề: Cắt tỉa attention head của Transformer/BERT sau huấn luyện; điểm quan trọng của head dựa trên gradient; head quan trọng thay đổi thế nào trong quá trình train.
Vị trí trong lộ trình: Đọc sau 07 (Molchanov importance 2019, vì điểm Taylor ở đây chính là phương pháp đó áp cho head) và 12 (cách đọc pruning phê phán). Bài này là bài **đầu tiên** của nhánh "pruning cho Transformer", cùng thời với 14 (Voita), cho thấy đơn vị cắt có thể là *head*. Bài kế tiếp: 14, rồi 15-17 (BERT, movement pruning, CoFi vốn cũng cắt head).

> Ghi chú học tập bằng tiếng Việt, không phải bản dịch.

1. Bối cảnh

Transformer dùng **multi-head attention (MHA)**: thay vì một phép attention, chạy N_h phép attention song song ("head"), mỗi head có ma trận W_q, W_k, W_v, W_o riêng với chiều nhỏ hơn $d_h = d/N_h$, rồi cộng kết quả. Lý do thường được nêu: mỗi head có thể chú ý vào một kiểu quan hệ khác nhau (ví dụ chủ-vị, phụ thuộc cú pháp).

Câu hỏi của bài báo: *các head đó thực sự mua được gì?* Nếu một mô hình huấn luyện với 12-16 head mà tại test chỉ cần vài head thì phần lớn tham số attention là thừa. Đây cũng là động cơ cho **structured pruning** (cắt cả khối) chứ không chỉ cắt từng trọng số: bỏ một head bỏ đúng một khối ma trận dense, nên tăng tốc thật.

Thuật ngữ:

- **Ablation**: tắt một thành phần rồi đo hiệu năng thay đổi thế nào.
- **BLEU**: điểm đánh giá dịch máy (càng cao càng tốt). **MNLI**: tác vụ suy luận ngôn ngữ (3 lớp), đo độ chính xác.
- **Pruning "tại test time"**: cắt trên mô hình đã huấn luyện xong, *không* tinh chỉnh lại (điểm quan trọng của bài).

2. Ý tưởng chính

Phương pháp gắn một biến mask nhị phân ξ_h cho mỗi head, đặt $\xi_h = 0$ để tắt head. Thí nghiệm cho thấy: tắt riêng lẻ phần lớn head không làm hiệu năng giảm có ý nghĩa thống kê, và nhiều layer có thể chỉ giữ **một** head mà hầu như không mất gì. Để cắt nhiều head cùng lúc, paper xếp hạng mọi head theo điểm quan trọng I_h (độ nhạy của loss với mask, tức Taylor bậc nhất theo Molchanov) rồi cắt lần lượt từ thấp lên cao; làm vậy cắt được 20% (dịch máy) và 40% (BERT) số head mà không tổn hại đáng kể. Phần phân tích cho thấy attention encoder-decoder trong dịch máy cần nhiều head hơn self-attention, và các head quan trọng được xác định sớm trong lúc train.

3. Phương pháp chi tiết

3.1 Attention và MHA

Một lớp attention (scaled bilinear) với W_k, W_q, W_v, W_o:

$$\mathrm{Att}(x,q) = W_o \sum_{i=1}^{n} \alpha_i W_v x_i, \qquad \alpha_i = \mathrm{softmax}\Big(\frac{q^\top W_q^\top W_k x_i}{\sqrt{d}}\Big)$$

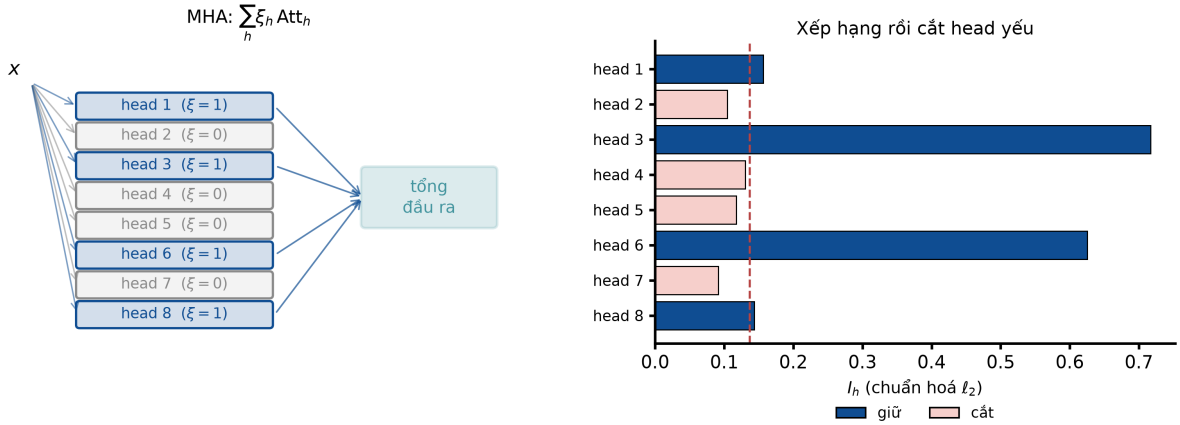
MHA (cách viết "concatenation" tương đương, tổng các head):

$$\mathrm{MHAtt}(x,q) = \sum_{h=1}^{N_h} \mathrm{Att}_{W_k^h, W_q^h, W_v^h, W_o^h}(x,q)$$

với $W_k^h, W_q^h, W_v^h \in \mathbb{R}^{d_h \times d}$, $W_o^h \in \mathbb{R}^{d \times d_h}$. Để giữ số tham số không đổi, $d_h = d/N_h$, và MHA có thể xem như ensemble của các attention hạng thấp.

3.2 Mask head

$$\boxed{\mathrm{MHAtt}(x,q) = \sum_{h=1}^{N_h} \xi_h \, \mathrm{Att}_h(x,q), \qquad \xi_h \in \{0,1\}}$$



Cắt head trong MHA bằng mask ξ_h; xếp hạng theo importance I_h rồi tắt các head yếu. (minh hoạ, tự tính)

Tắt head $h \Leftrightarrow \xi_h = 0$ (thay output của head bằng 0).

3.3 Điểm quan trọng của head

$$\boxed{I_h = \mathbb{E}_{x\sim X}\left|\frac{\partial \mathcal{L}(x)}{\partial \xi_h}\right| = \mathbb{E}_{x\sim X}\left|\mathrm{Att}_h(x)^\top \frac{\partial \mathcal{L}(x)}{\partial \mathrm{Att}_h(x)}\right|}$$

Ý nghĩa: nếu thay đổi ξ_h làm loss đổi nhiều thì head quan trọng. Dấu **trị tuyệt đối** theo từng mẫu được tác giả gọi là "crucial" để các mẫu dương/âm không triệt tiêu nhau. Paper nói công thức này **tương đương phương pháp Taylor của Molchanov et al. (2017)**; chỉ cần một lượt forward + backward, không chậm hơn huấn luyện. Điểm được **chuẩn hóa theo layer bằng chuẩn** ℓ_2 (theo khuyến nghị của Molchanov).

3.4 Thuật toán cắt lặp

```
Input: mô hình đã train, dữ liệu để ước lượng I (train hoặc tập con)
1. Tính I_h cho mọi head (forward+backward, trung bình trên dữ liệu), chuẩn hóa l2 theo
↪  layer
2. Sắp xếp head theo I_h tăng dần
3. for step in 1..:
```

```
đặt xi_h = 0 cho thêm 10% tổng số head thấp nhất (tích lũy)
đo BLEU/accuracy trên tập đánh giá
```

Không fine-tune sau khi cắt. Chọn thứ tự theo I_h nhanh hơn và tốt hơn thứ tự theo "độ chênh điểm khi ablate từng head" (oracle đơn lẻ).

Ví dụ nhỏ: BERT-base có 12 layer × 12 head = 144 head. Mỗi bước cắt khoảng 10% (≈ 14 head) theo thứ hạng toàn mạng (sau chuẩn hóa theo layer); ở bước 4 (≈ 40%) còn khoảng 86 head mà accuracy chưa giảm đáng kể.

4. Thí nghiệm và kết quả chính

Hai mô hình (đã train sẵn):

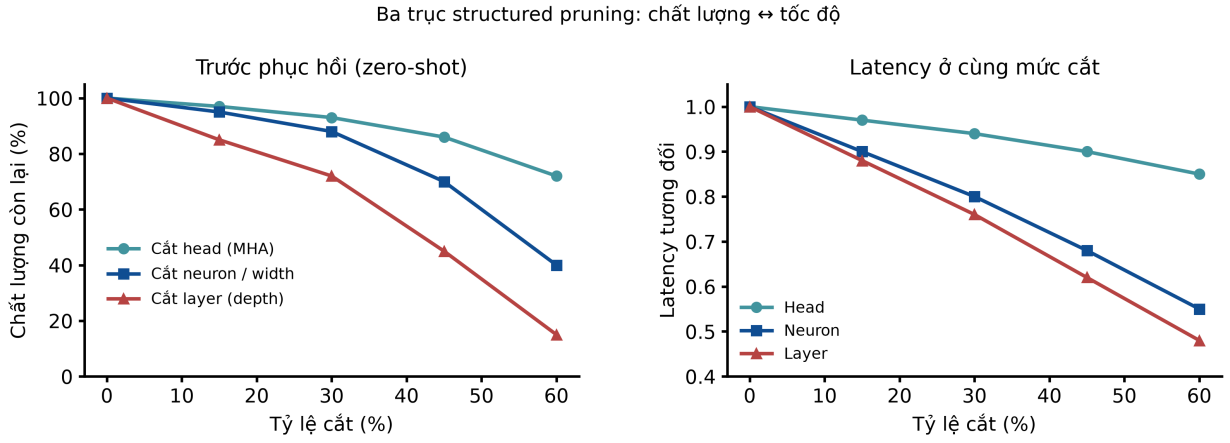
- **WMT**: Transformer "large" của Vaswani (6 layer, 16 head/layer), EN→FR WMT2014, bản pretrained của Ott et al. 2018; BLEU trên newstest2013; có 3 loại attention (Enc-Enc, Enc-Dec, Dec-Dec); kiểm định ý nghĩa bằng paired bootstrap (1000 mẫu).
- **BERT**: base-uncased (12 layer, 12 head) fine-tune trên MultiNLI; accuracy trên tập "matched"; t-test.

4.1 Ablate một head (mục 3.2 của paper)

- Phần lớn head có thể bỏ riêng lẻ mà điểm gần như không đổi; có trường hợp bỏ head còn làm điểm *tăng*.
- Trên encoder self-attention của WMT (Table 1, BLEU gốc 36.05): chỉ **8 trên 96 head** gây thay đổi có ý nghĩa thống kê ($p < 0.01$), **một nửa trong 8 đó thực ra làm BLEU tăng**.

4.2 Chỉ giữ một head mỗi layer (mục 3.3)

Mô hình	Kết quả
WMT (Table 2)	Phần lớn layer giữ 1 head ổn; ngoại lệ lớn: **layer 6 Enc-Dec: -13.56 BLEU**; layer 5 Enc-Dec: -1.55; layer 1 Enc-Enc: -1.31
BERT (Table 3)	Chênh accuracy tốt nhất mỗi layer từ -0.96% (layer 9) đến +0.10% (layer 2); **không** thay đổi nào có ý nghĩa thống kê ở $p < 0.01$



So sánh trục cắt head / neuron / layer: tradeoff chất lượng zero-shot và latency. (minh hoạ, tự tính)

Một layer còn 1 head chỉ còn 1/16 (WMT) hoặc 1/12 (BERT) tham số attention. Khi chọn "head tốt nhất" bằng tập validation riêng (newstest2013 cho WMT; 5000 mẫu train MNLI cho BERT) rồi đánh giá trên test, kết luận giữ nguyên: không thay đổi có ý nghĩa ở 50% layer của WMT và **100%** layer của BERT (chi tiết trong Appendix A của paper, không tái hiện ở đây).

4.3 Head quan trọng có giống nhau giữa các miền?

Dùng tập ngoài miền: MTNT (EN→FR) cho WMT, MNLI "mismatched" cho BERT. Ảnh hưởng của việc bỏ từng head tương quan dương **> 0.5** giữa hai miền ($p < 0.001$; hình ghi Pearson $r = 0.56$ cho WMT, $r = 0.68$ cho BERT). Kết luận của tác giả: các head quan trọng nhất là quan trọng "phổ quát" (ở mức độ phép thử này).

4.4 Cắt lặp theo I_h (mục 4)

- Cắt tới **20% head của WMT** và **40% head của BERT** mà không có tác động tiêu cực đáng chú ý.
- Cắt thêm thì hiệu năng rơi mạnh: cả hai mô hình **không** thể giảm xuống toàn single-head nếu không retrain.
- Appendix B có thêm 4 dataset (không ghi lại ở đây).

4.5 Hiệu quả suy luận (Table 4, BERT, MNLI-matched, mẫu/giây, GTX 1080Ti, 3 lần lặp trên mỗi 2 máy)

Batch size	1	4	16	64
Original	17.0 ± 0.3	67.3 ± 1.3	114.0 ± 3.6	124.7 ± 2.9
Pruned (50% head)	17.3 ± 0.6	69.1 ± 1.3	134.0 ± 3.6	146.6 ± 3.4
Tăng tốc	+1.9%	+2.7%	+17.5%	+17.5%

Mỗi head chiếm 6.25% (WMT) hoặc ≈ 8.34% (BERT) tham số của một lớp attention; MHA chiếm khoảng 1/3 tổng tham số của cả hai mô hình. Lưu ý: **với batch nhỏ, tăng tốc gần như biến mất** (+1.9% ở batch 1). Tác giả cảm ơn Thomas Wolf (Hugging Face) vì tái hiện độc lập giúp sửa một lỗi trong thí nghiệm tốc độ, nên đây là ví dụ tốt về việc nên kiểm chứng số đo latency.

5. Phân tích / ablation

Loại attention nào cần nhiều head? (mục 5, Figure 4, WMT). Cắt lặp riêng từng loại attention: Enc-Dec nhạy nhất; cắt hơn **60%** head Enc-Dec là sụp đổ hiệu năng, trong khi self-attention (Enc-Enc, Dec-Dec) vẫn dịch ra BLEU khoảng 30 khi chỉ giữ **20%** số head ban đầu. Kết luận: attention encoder-decoder phụ thuộc multi-headedness nhiều hơn nhiều.

Động học huấn luyện (mục 6, Figure 5). Mô hình nhỏ IWSLT (6 layer, 8 head/layer, DE→EN, IWSLT 2014). Lặp thí nghiệm cắt mỗi epoch:

- Epoch rất sớm (1-2): điểm giảm tuyến tính theo % cắt, tức mọi head quan trọng gần như ngang nhau.
- Từ epoch 10 trở đi: có "cụm head không quan trọng"; có thể cắt tới 40% tổng head mà vẫn giữ 85-90% BLEU gốc.
- Diễn giải của tác giả: head quan trọng được xác định sớm (nhưng không tức thì) trong train. Họ liên hệ với hai pha của Shwartz-Ziv & Tishby (ERM rồi nén) nhưng nói rõ cần nghiên cứu sâu hơn; đây là bằng chứng sơ bộ ("precursory evidence"), không phải kết luận chắc.

So sánh điểm: thứ tự theo I_h tốt hơn và nhanh hơn thứ tự theo hiệu năng khi ablate từng head.

6. Giới hạn

Do tác giả nêu / ngụ ý:

- Chỉ hai mô hình chính (WMT Transformer, BERT-MNLI) và vài dataset phụ; các kết quả ablate dựa trên tập test nhỏ (tác giả tự đặt nghi vấn về tính tổng quát, rồi kiểm bằng tập ngoài miền).
- Không retrain/fine-tune sau cắt; cắt hơn 20-40% làm hiệu năng rơi mạnh.
- Phần động học train chỉ trên mô hình nhỏ IWSLT, giải thích còn sơ bộ.

- Thứ tự greedy theo I_h tránh tìm kiếm tổ hợp, nên không đảm bảo tối ưu.

Nhận xét:

- Mô hình 2019, tác vụ NLU/MT, kích thước nhỏ so với VLM/LLM hiện nay; chưa rõ nhận xét "một head là đủ" còn đúng cho mô hình decoder-only lớn, đặc biệt không có Enc-Dec attention.
- Tăng tốc chỉ đo ở batch lớn trên GPU cũ; ở batch 1 gần như bằng 0 (bản thân bảng đã cho thấy).
- Chuẩn hóa ℓ_2 theo layer làm thứ hạng so sánh *giữa* layer khó diễn giải (mỗi layer đều bị cắt phần nào).
- Số liệu mỗi dòng bảng chỉ là trung bình ít lần chạy; không đo được ảnh hưởng của sự phụ thuộc giữa các head (hai head thừa riêng lẻ nhưng cùng bỏ thì mất thông tin).

7. Liên hệ với các bài khác

- **07 Molchanov importance 2019**: I_h chính là điểm Taylor bậc nhất. Bài này là một ứng dụng sớm trong NLP.
- **12 State of pruning**: dùng checklist ở bài 12: bài này chỉ có 2 mô hình chính và rất ít điểm hoạt động (bước 10%), có độ lệch chuẩn cho tốc độ nhưng không cho điểm số cắt.
- **14 Voita et al.**: nghiên cứu song song cùng thời; Voita học mask bằng L_0 trong khi bài này dùng điểm gradient một lần (paper tự so sánh ở mục Related Work: hai cách "bổ sung cho nhau").
- **15 LTH for BERT, 16 Movement pruning**: tiếp tục cắt BERT, nhưng cắt trọng số, học mask trong lúc fine-tune.
- **17 CoFi**: cắt *đồng thời* head, hidden unit và layer bằng mask học được, kế thừa trực tiếp ý "mask ξ cho head".
- **05 Network slimming, 04 Pruning filters**: tương tự ở CNN (đơn vị là channel/filter).
- **26 LLM-Pruner, 28 FLAP, 29 SliceGPT**: cắt head/channel ở LLM.

8. Gợi ý áp dụng

3. **Latency batch-1**: Table 4 gợi ý cắt head có thể không giúp batch 1 (+1.9%). Hãy đo thật.
4. **Nhận xét:** ở đây head ablate riêng lẻ gần như vô hại, song cắt tập thể bị rơi mạnh sau 20-40%. Đây là ví dụ của "redundancy riêng lẻ không bằng redundancy tập thể", điều cần kiểm tra khi cắt các nhóm neuron.
5. Phát hiện "Enc-Dec attention cần nhiều head" gợi ý: trong VLM, **cross-modal path** (nơi phần văn bản đọc đặc trưng ảnh) có thể nhạy hơn self-attention thuần; nên cẩn thận khi cắt ở chỗ nối vision-language (giả thuyết, không có trong paper).

Mức liên quan: **trung bình** (ý tưởng và công cụ chấm điểm hữu ích; kỹ thuật cắt trực tiếp có thể ít lợi cho batch-1).

9. Thí nghiệm gợi ý

2. **Cá thể vs tập thể**: ablate từng head (như Table 1) và so với cắt theo thứ hạng I_h; kiểm tra điểm nào bắt đầu có sự "mất đột ngột" (đường cong gập).

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / ý nghĩa
MHA với mask	$\sum_h \xi_h \, \mathrm{Att}_h(x, q)$, $\xi_h \in \{0, 1\}$
Điểm head	$I_h = \mathbb{E}_x \left\lvert \frac{\partial \mathcal{L}}{\partial \xi_h} \right\rvert = \mathbb{E}_x \left\lvert \mathrm{Att}_h^\top \frac{\partial \mathcal{L}}{\partial \mathrm{Att}_h} \right\rvert$
Chuẩn hóa	ℓ_2 theo từng layer
Thuật toán	xếp I_h tăng dần, cắt dần 10% mỗi bước, không fine-tune

Khái niệm	Công thức / ý nghĩa
Ablate-all-but-one	giữ 1 head mỗi layer; BERT: không đổi có ý nghĩa; WMT layer 6 Enc-Dec: -13.56 BLEU
Giới hạn cắt	khoảng 20% head (WMT), 40% (BERT) không hại đáng kể
Tốc độ	+17.5% ở batch 16/64, +1.9% ở batch 1 (BERT, cắt 50% head)
Động học	head quan trọng định hình sớm; epoch 1-2 chưa phân biệt

11. Kết luận ngắn

Multi-head attention thừa đáng kể sau khi train: nhiều head có thể bỏ riêng lẻ, nhiều layer chỉ cần một head, và có thể cắt 20-40% head toàn mạng bằng một điểm gradient rẻ mà không huấn luyện lại. Nhưng không cắt hết được, attention encoder-decoder nhạy hơn, và lợi ích tốc độ phụ thuộc mạnh vào batch size.

Link gốc: https://arxiv.org/abs/1905.10650

Bài 14

Analyzing Multi-Head Self-Attention: Specialized Heads Do the Heavy Lifting, the Rest Can Be Pruned (2019)

Tác giả: Elena Voita, David Talbot, Fedor Moiseev, Rico Sennrich, Ivan Titov (Yandex; ĐH Amsterdam; ĐH Edinburgh; ĐH Zurich; MIPT).
Venue / arXiv: arXiv:1905.09418 (v2, 7/6/2019). Paper trong file không ghi tên hội nghị.
Nguồn: https://arxiv.org/abs/1905.09418
Chủ đề: Phân tích vai trò từng attention head ở encoder của Transformer dịch máy; phương pháp cắt head bằng cổng ngẫu nhiên và nới lỏng khả vi của chuẩn L_0 (Hard Concrete).
Vị trí trong lộ trình: Đọc ngay sau 13 (Sixteen Heads): cùng một phát hiện (nhiều head thừa) nhưng bài này đi từ góc "head nào làm việc gì" và đề xuất cách cắt có **học** (learnable gates). Nền cần có: 05 Network slimming (hệ số học được để cắt) và khái niệm regularization. Bài kế tiếp: 15, 16; ý tưởng cổng L_0 quay lại ở 17 (CoFi) và Sheared LLaMA (38).

> Ghi chú học tập bằng tiếng Việt, không phải bản dịch.

1. Bối cảnh

Trong Transformer dịch máy (NMT), cấu hình attention tám head được báo cáo là tốt hơn cấu hình một head khoảng 1 BLEU ở cùng kích thước mô hình. Nhưng các phân tích trước đó chỉ nhìn **trung bình** hoặc **giá trị lớn nhất** của trọng số attention trên các head, nên che mất việc các head đóng góp rất khác nhau.

Bài hỏi bốn câu: dịch phụ thuộc bao nhiêu vào từng head của encoder? Head có vai trò nhất quán, diễn giải được không? Loại attention nào (encoder self-attn, decoder self-attn, decoder-encoder attn) và layer nào nhạy với số head? Có giảm mạnh số head mà giữ chất lượng được không?

Thuật ngữ:

- **Encoder-decoder (seq2seq)**: encoder đọc câu nguồn, decoder sinh câu đích; có 3 loại attention như nói trên.
- **Dependency relation (quan hệ phụ thuộc cú pháp)**: ví dụ `nsubj` (chủ ngữ danh từ), `dobj` (tân ngữ trực tiếp), `amod` (tính từ bổ nghĩa), `advmod` (trạng từ bổ nghĩa).
- **LRP (layer-wise relevance propagation)**: phương pháp lan truyền ngược "độ liên quan" để biết nơron/đơn vị nào đóng góp vào một đầu ra.
- **Chuẩn L_0**: số thành phần khác 0; phạt L_0 ép nhiều thành phần bằng đúng 0 (tắt hẳn), nhưng không khả vi.

2. Ý tưởng chính

Hai giai đoạn. (1) Dùng LRP đo mức đóng góp của từng head vào logit top-1, kèm "confidence" (trọng số attention lớn nhất trung bình) của head; chỉ một ít head mỗi lớp là quan trọng, và chúng có vai trò rõ: chú ý token kề bên (positional), chú ý token có quan hệ cú pháp nhất định (syntactic), hoặc trỏ tới từ hiếm nhất câu (rare words). (2) Để kiểm tra các head còn lại có thừa không, gắn mỗi head một cổng vô hướng g_i độc lập với đầu vào, lấy từ phân phối Hard Concrete, và huấn luyện tiếp từ mô hình đã hội tụ với phạt L_0 khả vi. Kết quả: phần lớn head bị cắt, những head chuyên biệt là những head **bị cắt sau cùng**, xác nhận độc lập kết quả của LRP.

3. Phương pháp chi tiết

3.1 Xác định head quan trọng

- **Confidence** của head: trung bình (trên các token của tập dev) của trọng số attention cực đại, không tính token EOS (vì không phải token thật).
- **LRP**: dùng để ước lượng mức đóng góp của đầu ra từng head ở mỗi layer vào logit top-1 mà mô hình dự đoán. Chi tiết LRP nằm ở Appendix A (không đọc ở đây).
- Quan sát: LRP chỉ ra rất ít head quan trọng hơn hẳn các head khác trong mỗi layer; độ liên quan LRP khớp khá tốt với confidence (ngoại lệ: head quan trọng nhất ở layer 1, có confidence thấp: chính là "rare words head").

3.2 Ba vai trò được định nghĩa

Vai trò	Tiêu chí của paper
Positional	≥ 90% thời gian, trọng số cực đại rơi vào một vị trí tương đối cố định (thực tế là -1 hoặc +1, token kề bên). Trọng số cực đại trung bình > 0.8 cho mọi head positional ở mọi cặp ngôn ngữ
Syntactic	"accuracy" (tần suất head trỏ tới token có quan hệ đó, theo parser CoreNLP, hai chiều tính riêng) cao hơn ít nhất 10 điểm tuyệt đối so với baseline "vị trí tương đối phổ biến nhất của quan hệ đó"
Rare words	head ở layer 1 trỏ tới token hiếm nhất câu

3.3 Cắt head bằng cổng L_0 (Hard Concrete)

Mỗi head i được nhân với cổng vô hướng g_i (tham số riêng của head, **không** phụ thuộc đầu vào):

$$\text{MultiHead}(Q, K, V) = \text{Concat}_i(g_i \cdot \text{head}_i)\, W^O$$

Ta muốn phạt số cổng khác 0:

$$L_0(g_1, \ldots, g_h) = \sum_{i=1}^{h} (1 - [[g_i = 0]])$$

Không khả vi, nên dùng **nới lỏng ngẫu nhiên** (Louizos et al., 2018): g_i là biến ngẫu nhiên từ phân phối **Hard Concrete** trên $[0, 1]$ với tham số ϕ_i: lấy phân phối Concrete (Gumbel-softmax) có support $(0, 1)$, kéo giãn ra $(-\epsilon, 1+\epsilon)$ rồi dồn phần khối lượng ngoài $[0, 1]$ về hai điểm 0 và 1. Kết quả là phân phối hỗn hợp rời rạc-liên tục có khối lượng dương tại 0 và 1. Khi đó kỳ vọng số cổng khác 0 có dạng đóng:

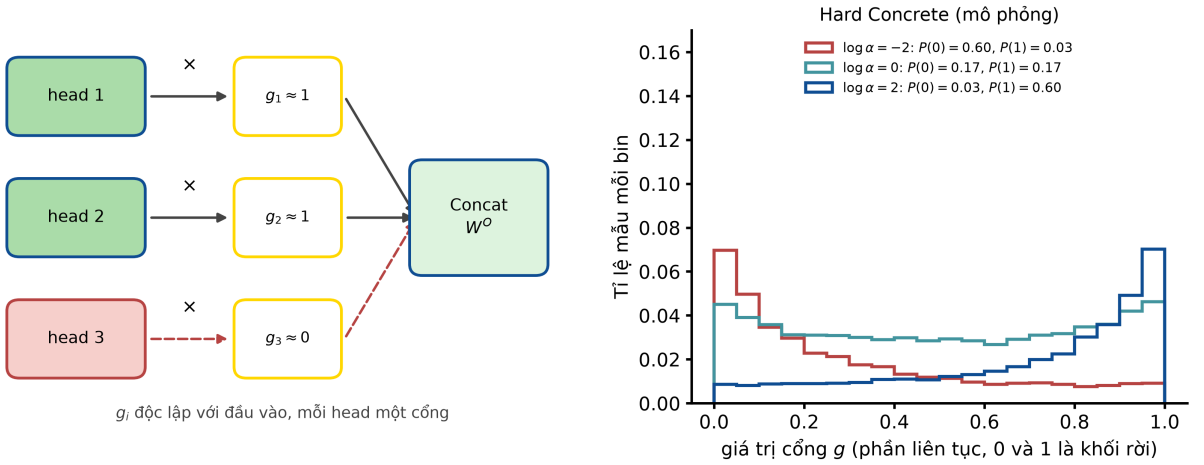
$$\boxed{L_C(\phi) = \sum_{i=1}^{h} (1 - P(g_i = 0 \mid \phi_i))}, \qquad \boxed{\mathcal{L}(\theta, \phi) = \mathcal{L}_{xent}(\theta, \phi) + \lambda\, L_C(\phi)}$$

- θ: tham số Transformer gốc; $\mathcal{L}_{xent}$: cross-entropy dịch máy.
- Gradient qua việc lấy mẫu g_i dùng **reparameterization trick**; L_C và gradient của nó có dạng đóng.
- Mỗi batch lấy mẫu lại g_i. Khi test, g_i là 0 hoặc 1 tùy $P(g_i = 0)$ hay $P(g_i = 1)$ lớn hơn.
- **Hiện tượng đẹp**: mô hình hội tụ về cổng hoặc gần như đóng ($P(g_i = 0) \approx 1$) hoặc hoàn toàn mở ($P(g_i = 1) \approx 1$) dù không có gì ép buộc cổng mở hẳn; nhiễu ngẫu nhiên đẩy mô hình tránh giá trị giữa. Nên khi suy luận mô hình là Transformer chuẩn với tập con head.

```
Input: mô hình Transformer đã hội tụ (theta*), hệ số lambda
1. Khởi tạo theta = theta*; thêm cổng g_i ~ HardConcrete(phi_i) cho mỗi head
2. Khi cắt encoder: đóng băng decoder, chỉ fine-tune encoder + phi
```

```
3. Lặp train với  L = L_xent + lambda * sum_i (1 - P(g_i = 0))
4. Test: g_i = 1 nếu P(g_i=1) > P(g_i=0), ngược lại 0 (loại hẳn head)
5. Thay đổi lambda để được các mô hình với số head còn lại khác nhau
```

Việc đóng băng decoder khi chỉ cắt encoder là chủ ý: để chức năng của head encoder bị cắt không "di cư" sang decoder.



Hình 39: Cổng g_i nhân với từng head (trái) và phân phối Hard Concrete mô phỏng với ba giá trị $\log \alpha$ (phải) (minh hoạ, tự tính)

Cách đọc hình: bên trái, head có cổng gần 0 bị loại khỏi phép nối W^O. Bên phải là mô phỏng với $\beta = 2/3$ và khoảng giãn $(-0{,}1;\ 1{,}1)$, các giá trị này chọn tuỳ ý để minh hoạ. Khối rời tại 0 và 1 chiếm phần lớn khi $|\log \alpha|$ lớn, vì vậy khi suy luận cổng gần như chỉ là mở hoặc đóng.

4. Thí nghiệm và kết quả chính

Dữ liệu và mô hình. Ngôn ngữ nguồn tiếng Anh, đích Nga/Đức/Pháp; mỗi cặp dùng cùng 2.5 triệu cặp câu WMT (bằng lượng dữ liệu EN-RU, không gồm UN và Paracrawl); EN-RU còn thêm OpenSubtitles2018 (có chạy 6 triệu cặp) để xem ảnh hưởng miền. Tập held-out dùng cho phân tích: 50k câu tiếng Anh lấy từ WMT EN-FR. Encoder 48 head (8 head × 6 layer, theo tổng nêu trong Table 2: 48/48/48).

4.1 Vai trò các head quan trọng

- Head positional trùng nhiều với head confident nhất và head LRP cao nhất.
- Syntactic (Table 1, EN-RU, độ chính xác của head tốt nhất / baseline vị trí):

Quan hệ	WMT	OpenSubtitles
nsubj v → s	45 / 35	77 / 45
nsubj s → v	52 / 35	70 / 45
dobj v → o	78 / 41	61 / 46
dobj o → v	73 / 41	84 / 46
amod noun → adj.m.	74 / 72	81 / 80
amod adj.m. → noun	82 / 72	81 / 80
advmod v → adv.m.	48 / 46	38 / 33
advmod adv.m. → v	52 / 46	42 / 33

(Với amod và advmod khoảng cách tới baseline nhỏ; paper kết luận "certain heads" học được quan hệ cú pháp

rõ rệt, nổi bật là nsubj và dobj.) Nhiều head cùng đảm nhiệm một quan hệ. Paper nêu giả thuyết nhưng nói chưa rút được kết luận chắc về ảnh hưởng hình thái học ngôn ngữ đích.

- Rare-words head: ở layer 1. Trên OpenSubtitles, với các câu mà token hiếm nhất không nằm trong top-500 token phổ biến, head này trỏ tới token hiếm nhất ở **66%** trường hợp và tới một trong hai token hiếm nhất ở **83%**; trên WMT, trỏ tới một trong hai token hiếm nhất ở hơn 50% trường hợp.

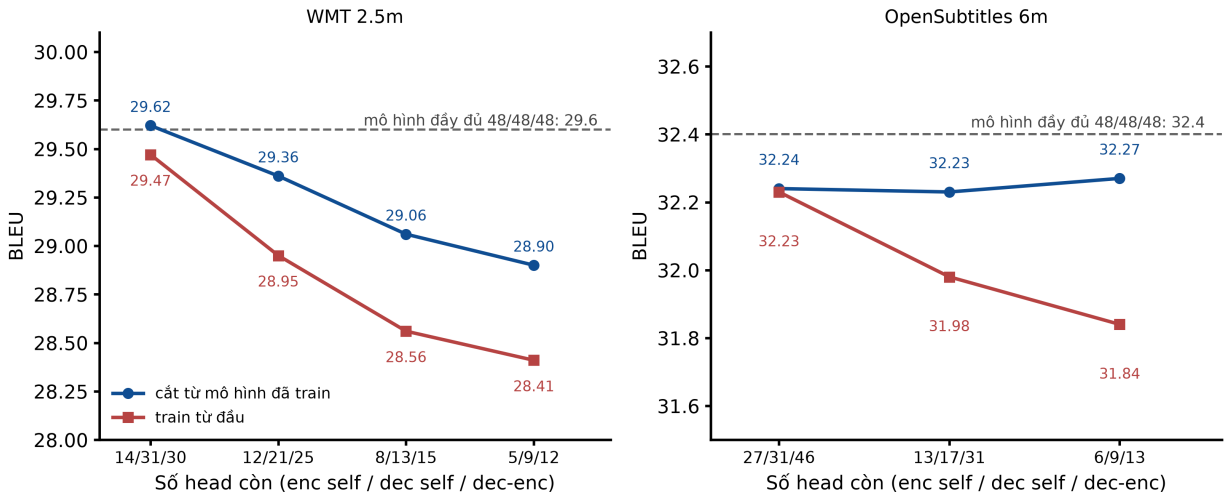
4.2 Cắt head encoder (Figure 7)

- OpenSubtitles: **chỉ mất 0.25 BLEU khi cắt tất cả trừ 4 head trên 48**.
- WMT (EN-RU): **10 head** encoder là đủ để nằm trong 0.15 BLEU của mô hình đầy đủ. Khớp với abstract: cắt 38/48 head, giảm chỉ 0.15 BLEU.

4.3 Cắt tất cả loại attention (Table 2, EN-RU; số head theo thứ tự enc self / dec self / dec-enc)

Dữ liệu	Số head còn	BLEU (cắt từ mô hình đã train)	BLEU (train từ đầu cùng cấu hình)
WMT 2.5m	baseline 48/48/48	29.6	n/a
	14/31/30	29.62	29.47
	12/21/25	29.36	28.95
	8/13/15	29.06	28.56
	5/9/12	28.90	28.41
OpenSubtitles 6m	baseline 48/48/48	32.4	n/a
	27/31/46	32.24	32.23
	13/17/31	32.23	31.98
	6/9/13	32.27	31.84

Kết luận của paper: trên WMT cắt gần 3/4 head encoder và hơn 1/3 head của decoder self-attn và decoder-encoder attn mà không mất chất lượng đáng kể; có thể cắt hơn một nửa tổng số head với mất tối đa 0.25 BLEU.



Hình 40: BLEU EN-RU của mô hình cắt từ mô hình đã train và mô hình train từ đầu cùng cấu hình head (Table 2) (vẽ lại từ số liệu trong ghi chú)

Cách đọc hình: nhãn trục ngang là số head còn lại ở encoder self, decoder self và decoder-encoder. Đường

xanh luôn nằm trên đường đỏ, khoảng cách tăng khi còn ít head hơn. Trên OpenSubtitles mô hình cắt gần như giữ nguyên BLEU ngay cả khi chỉ còn 6/9/13 head.

5. Phân tích / ablation

- **Chức năng của các head còn lại (Figure 8, EN-RU Subtitles).** Mô hình còn 17 head (đã cắt 2/3) vẫn giữ **đủ mọi chức năng** đã nhận diện. Khi còn ít head hơn nữa, một số chức năng "trôi" sang head khác (ví dụ head positional bắt đầu theo dõi phụ thuộc cú pháp).
- **Cắt rồi fine-tune tốt hơn train từ đầu** với cùng cấu hình head: chênh khoảng 0.2-0.6 BLEU (chú thích 8 trong paper) và Table 2 cột phải thấp hơn cột giữa, ví dụ WMT 5/9/12: 28.90 so với 28.41. Giải thích của tác giả: head ít có khả năng học vai trò quan trọng khi train từ đầu với số head nhỏ; tương hợp với các báo cáo của Zhu & Gupta 2017 và Gale et al. 2019 (bài 03) rằng kiến trúc thưa học qua pruning không train từ đầu được tới cùng chất lượng.
- **Loại attention nào quan trọng nhất (Figure 9).** Mô hình cắt head encoder self-attn trước; decoder-encoder attn là quan trọng nhất ở cả hai dataset ("without decoder-encoder attention no translation can happen"). Decoder self-attn (đóng vai trò mô hình ngôn ngữ phía đích) quan trọng gần ngang decoder-encoder attn trên WMT (câu dài, trung bình 24 token), và quan trọng hơn encoder self-attn trên OpenSubtitles (câu ngắn, 8 token).
- **Theo layer của decoder (Figure 10).** Self-attn head được giữ lại nhiều ở layer thấp; decoder-encoder attn head ở layer cao. Diễn giải của tác giả: layer thấp của decoder lo mô hình hóa ngôn ngữ, layer cao lo điều kiện hóa theo câu nguồn.

6. Giới hạn

Do tác giả nêu:

- Tập trung vào encoder self-attention; cắt các loại khác chỉ ở phần thứ hai.
- Tác giả viết rõ họ chưa so sánh phương pháp cắt này với các phương pháp nén NMT khác (hướng tương lai).
- Chưa rút được kết luận chắc về ảnh hưởng hình thái của ngôn ngữ đích lên head cú pháp.
- Không rõ liệu mô hình có thể train từ đầu với ít head như vậy (kết quả train từ đầu kém hơn).

Nhận xét:

- Chỉ dịch máy EN→{RU,DE,FR}; chưa chứng minh cho encoder-only, decoder-only hay đa phương thức.
- Cổng g_i độc lập với đầu vào, nên đây là **cắt tĩnh**; không tìm được head "quan trọng cho một số câu".
- Chỉ báo cáo BLEU, không báo latency/bộ nhớ thực; giảm số head không tự nhiên cho tăng tốc nếu cài đặt giữ nguyên.
- Số liệu BLEU trong Table 2 không kèm phương sai theo seed, nên chênh 0.1-0.2 BLEU khó kết luận (ứng dụng checklist bài 12).
- Vai trò "diễn giải được" dựa trên tiêu chí ngưỡng (90%, +10%) do tác giả đặt; các head còn lại có thể có vai trò khác.

7. Liên hệ với các bài khác

- **13 Sixteen Heads**: cùng thời, cùng kết luận "head thừa". Michel et al. cắt bằng điểm gradient một lần (không học); Voita học cổng bằng L_0. Michel là phân tích mô hình đã train, Voita có bước fine-tune với phạt. Michel tự ghi nhận Voita là "complementary" trong Related Work.
- **05 Network slimming**: cũng học hệ số vô hướng cho mỗi channel với phạt thưa (L_1); bài này dùng L_0 nới lỏng thay cho L_1.

- **03 Zhu & Gupta, 11 Rethinking the value of pruning**: kết quả "cắt rồi fine-tune > train từ đầu" ở đây ủng hộ nhóm quan sát của Zhu & Gupta, trái với thông điệp chính của bài 11 (nên xem lại cả hai trong cùng bối cảnh: phụ thuộc cài đặt).
- **15 LTH for BERT, 16 Movement pruning**: cắt trọng số với mask học được.
- **17 CoFi**: dùng mask L_0 Hard Concrete để cắt head, hidden unit, layer, phát triển trực tiếp từ ý tưởng của bài này (và Louizos 2018, Wang et al. 2020).
- **38 Sheared LLaMA**: cũng dùng mask kiểu L_0 nhưng cho LLM.
- **12 State of pruning**: kiểm tra bài này với checklist (ít điểm hoạt động, một dataset chính, chưa so sánh phương pháp khác).

8. Gợi ý áp dụng

1. **Phương pháp học cổng L_0 cho đơn vị cấu trúc** là công cụ cắt tự động rất phù hợp: gắn cổng cho head ViT, cho neuron MLP, thậm chí cho cả block, và đặt ngân sách bằng λ. Đây gần với cách CoFi (17) làm.
2. **Giữ decoder cố định khi chỉ cắt encoder** (tác giả làm để tránh chức năng "di cư").
3. **Hiện tượng chức năng trôi (function drift)** gợi ý rằng sau khi cắt, các head còn lại *gánh thêm vai trò*; đây là lý do phục hồi bằng distillation hiệu quả (giả thuyết, cần kiểm).

Mức liên quan: **trung bình-cao** về phương pháp (cổng L_0), thấp về kết luận ngôn ngữ học.

9. Thí nghiệm gợi ý

2. **Positional heads trong ViT**: tính tỉ lệ thời gian trọng số attention cực đại rơi vào patch kề (±1 theo hàng/cột) cho mỗi head; kiểm tra các head này có nằm trong tập được giữ lại sau cắt không (tái hiện logic "head chuyên biệt cắt sau cùng").
3. **Train từ đầu vs cắt**: lấy cấu hình ViT sau cắt, khởi tạo ngẫu nhiên và train cùng ngân sách (nhỏ) để thấy khoảng cách cắt-vs-scratch như Table 2.

10. Tóm tắt công thức / khái niệm

Khái niệm	Nội dung
Confidence	trung bình trọng số attention cực đại (bỏ EOS)
LRP	đo đóng góp của từng head vào logit top-1
Head positional	≥ 90% attention cực đại vào vị trí tương đối cố định (±1)
Head syntactic	accuracy cao hơn baseline vị trí ≥ 10 điểm
Cổng head	$\text{Concat}_i(g_i\,\text{head}_i)W^O$, g_i độc lập đầu vào
L_0	$\sum_i(1-[[g_i=0]])$
Nới lỏng	$L_C(\phi)=\sum_i(1-P(g_i=0\mid\phi_i))$ với Hard Concrete
Mục tiêu	$\mathcal{L}_{xent}+\lambda L_C$
Quy trình	bắt đầu từ mô hình hội tụ, thêm cổng, fine-tune (decoder đóng băng khi cắt encoder)
Kết quả chính	EN-RU WMT: 38/48 encoder head bị cắt, -0.15 BLEU; Subtitles: còn 4/48 head, -0.25 BLEU

11. Kết luận ngắn

Chỉ một số ít head encoder quan trọng và chúng đảm nhiệm vai trò rõ (kề bên, quan hệ cú pháp, từ hiếm). Cổng ngẫu nhiên Hard Concrete với phạt L_0 nới lỏng cho phép cắt hầu hết head mà giảm rất ít BLEU, và các

head chuyên biệt bị cắt sau cùng, một bằng chứng độc lập về tầm quan trọng của chúng. Cắt-rồi-tinh-chỉnh tốt hơn huấn luyện lại từ đầu với cùng cấu hình thưa.

Link gốc: https://arxiv.org/abs/1905.09418

Bài 15

The Lottery Ticket Hypothesis for Pre-trained BERT Networks (2020)

Tác giả: Tianlong Chen, Jonathan Frankle, Shiyu Chang, Sijia Liu, Yang Zhang, Zhangyang Wang, Michael Carbin (UT Austin, MIT CSAIL, MIT-IBM Watson AI Lab).
Venue / arXiv: arXiv:2007.12223 (v2, 18/10/2020); NeurIPS 2020 (34th Conference on Neural Information Processing Systems).
Nguồn: https://arxiv.org/abs/2007.12223
Chủ đề: Lottery Ticket Hypothesis (LTH) áp cho BERT đã pretrain: tìm subnetwork "matching" bằng iterative magnitude pruning (không cấu trúc), và kiểm tra chúng có chuyển (transfer) giữa các tác vụ không.
Vị trí trong lộ trình: Cần đọc trước: 08 (Lottery Ticket), 09 (Linear mode connectivity & LTH, khái niệm rewinding), 12 (Blalock, vì cùng nhóm tác giả Frankle). Bài này là cầu nối giữa LTH (nghiên cứu khoa học trên CNN nhỏ) và pruning cho mô hình pretrain-rồi-fine-tune. Kế tiếp: 16 (Movement pruning), cùng bối cảnh BERT nhưng hướng ứng dụng.

> Ghi chú học tập bằng tiếng Việt, không phải bản dịch.

1. Bối cảnh

Lottery Ticket Hypothesis (Frankle & Carbin 2019, bài 08): một mạng lớn chứa mạng con thưa ("winning ticket") mà nếu huấn luyện riêng từ khởi tạo ban đầu thì đạt độ chính xác ngang mạng đầy đủ. Quy trình tìm vé: huấn luyện mạng đầy đủ, cắt trọng số nhỏ, **hồi trọng số còn lại về giá trị lúc khởi tạo** (hoặc lúc sớm trong train: **rewinding**), lặp lại.

Hai chủ đề đã nổi lên trong văn liệu LTH:

1. **Initialization via pre-training**: ở quy mô lớn (CV, NLP), vé chỉ tìm được sau một đoạn huấn luyện sớm chứ không phải từ khởi tạo ngẫu nhiên; đoạn đầu đó giống như pretrain dày để tạo điểm khởi đầu dễ làm thưa.
2. **Transfer learning**: tìm vé tốn nhiều tài nguyên kém (train cả mạng, cắt, lặp nhiều lần); chỉ đáng nếu vé dùng lại được cho nhiều tác vụ.

BERT hội tụ cả hai: khởi tạo = trọng số pretrain, và pretrain được khấu hao qua nhiều tác vụ downstream. Câu hỏi: **BERT pretrained có chứa vé? Vé có chuyển tác vụ? Có vé "phổ quát"?**

Thuật ngữ:

- **IMP (iterative magnitude pruning)**: lặp huấn luyện → cắt phần trăm nhỏ trọng số có $|w|$ nhỏ nhất → hồi lại → ...
- **Sparsity (độ thưa)**: tỉ lệ trọng số bằng 0.
- **GLUE, SQuAD**: bộ benchmark NLP (phân loại cặp câu / câu đơn; hỏi đáp).
- **MLM (masked language modeling)**: tác vụ pretrain của BERT, che token rồi đoán.
- **Unstructured pruning**: cắt từng trọng số riêng, tạo ma trận thưa; không tự tăng tốc trên GPU phổ thông.

2. Ý tưởng chính

Nghiên cứu áp dụng quy trình Lottery Ticket Hypothesis cho BERT-base. Họ lấy **trọng số pretrain θ_0 làm "khởi tạo"**, chạy IMP trên từng tác vụ downstream, rồi huấn luyện lại subnetwork bắt đầu từ θ_0. Kết quả: có vé "winning" ở độ thưa 40% đến 90% tùy tác vụ, **ngay tại** θ_0 (không cần rewind sớm như các nghiên cứu

NLP trước). Vé tìm trên tác vụ nào thì phần lớn chỉ tốt cho tác vụ đó, **trừ vé tìm trên MLM** (chính tác vụ pretrain), ở độ thưa 70% chuyển tốt sang mọi tác vụ downstream, tức là một mạng con "phổ quát" có thể thay BERT đầy đủ.

3. Phương pháp chi tiết

3.1 Cài đặt

- Mạng: BERT-base (12 block, hidden 768, 12 head, 110M tham số) + lớp phân loại riêng cho tác vụ (< 3% tham số). Ký hiệu: $f(x;\theta,\gamma)$, θ là tham số BERT, γ là tham số lớp phân loại.
- Tác vụ: MLM + 9 tác vụ GLUE (MNLI, QQP, STS-B, WNLI, QNLI, MRPC, RTE, SST-2, CoLA) + SQuAD v1.1; mọi số đo lấy trên tập validation/dev. Siêu tham số chuẩn (AdamW, learning rate giảm tuyến tính về 0; ví dụ 3 epoch, batch 32, lr 2×10^{-5} cho hầu hết GLUE; SQuAD: 2 epoch, batch 16, lr 3×10^{-5}; MLM: 0.1 epoch, batch 16, lr 5×10^{-5}).
- Subnetwork: $f(x; m\odot\theta,\cdot)$ với mask $m\in\{0,1\}^{d_1}$.

3.2 Các định nghĩa

Gọi A_t^T là thuật toán huấn luyện t bước cho tác vụ T, $\mathcal{E}^T$ là metric đánh giá, θ_0 là trọng số pretrain:

- **Matching subnetwork**: $\mathcal{E}^T\big(A_t^T(f(x;m\odot\theta,\gamma))\big)\ \geq\ \mathcal{E}^T\big(A_t^T(f(x;\theta_0,\gamma))\big)$
- **Winning ticket**: matching subnetwork với $\theta=\theta_0$ (tức khởi tạo ở trọng số pretrain).
- **Universal subnetwork**: matching trên mọi tác vụ trong tập $\{T_i\}$ (với lớp phân loại γ_{T_i} riêng).

Trong thực nghiệm, để chịu nhiễu, subnetwork được coi là winning nếu điểm của nó **nằm trong một độ lệch chuẩn** so với BERT đầy đủ (trung bình năm lần chạy).

3.3 Thuật toán IMP với bước rewind i (Algorithm 1)

```
1. Train BERT pretrain f(x; theta_0, gamma_0) tới bước i  ->  theta_i
2. m <- 1^{d1}
3. repeat
4.     train f(x; m ⊙ theta_i, gamma_i) tới bước t  ->  theta_t        # train trên tác
↪  vụ T
5.     cắt 10% trọng số còn lại có |theta_t| nhỏ nhất (toàn cục), cập nhật m
6. until sparsity(m) = s
7. return f(x; m ⊙ theta_i)                                           # rewind về
↪  theta_i
```

Cắt **toàn cục** (global) theo độ lớn trên cả mạng. Với $i=0$, subnetwork trả về bắt đầu từ θ_0 (winning ticket). **Standard pruning** (đối chứng): train xong, cắt 10% nhỏ nhất, train tiếp mà *không* rewind, lặp tới độ thưa mong muốn; các trọng số giữ lại là trọng số cuối quá trình train.

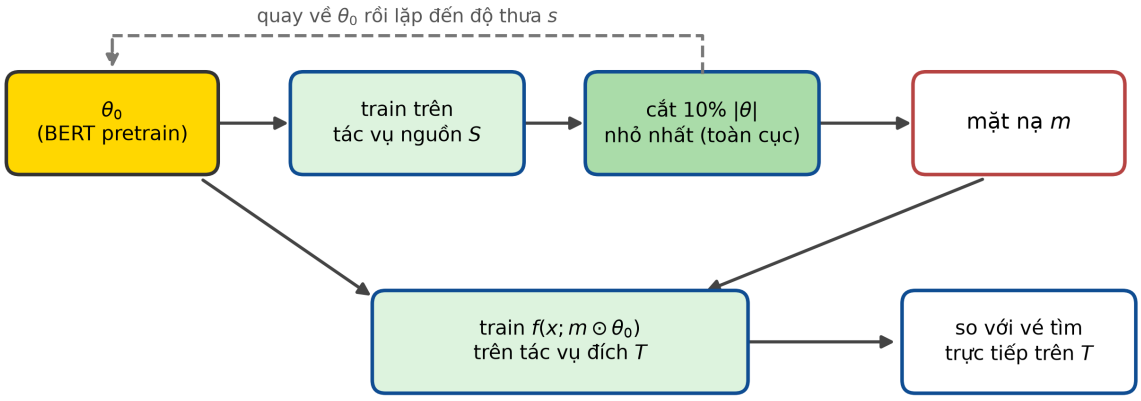
Cách đọc hình: vòng nét đứt là bước rewind về θ_0; với BERT không cần quay về bước sớm hơn. Mặt nạ m tìm trên tác vụ nguồn được áp lên chính θ_0 để huấn luyện trên tác vụ đích. Trường hợp $S=T$ là vé thường, còn S là MLM ở độ thưa 70% cho vé dùng chung.

3.4 Bốn "claim" của LTH được kiểm lại

1. IMP tìm được winning ticket $f(x; m_{IMP}\odot\theta_0)$.
2. Các vé đó thưa hơn mức mà random pruning hoặc random init còn matching.
3. Ở mạng khác, vé chỉ có ở bước sớm i (rewinding); hoặc rewind giúp hơn θ_0.
4. Vé đạt cùng accuracy ở cùng độ thưa với standard pruning.

4. Thí nghiệm và kết quả chính

4.1 Tồn tại winning ticket (Table 2; trung bình 5 lần chạy ± độ lệch chuẩn)

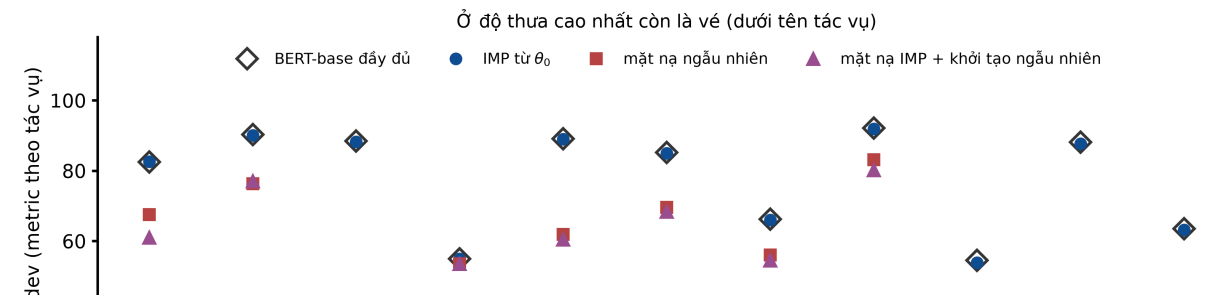


Nếu S là MLM và $s = 70\%$: một mặt nạ dùng được cho mọi tác vụ GLUE và SQuAD (ngang vé tìm trực tiếp trên T).

Hình 41: IMP trên tác vụ nguồn, quay về trọng số pretrain rồi huấn luyện mạng con trên tác vụ đích (minh hoạ, tự tính)

Tác vụ	MNLI	QQP	STS-B	WNLI	QNLI	MRPC	RTE	SST-2	CoLA	SQuAD	MLM
Độ thưa cao nhất còn là vé	70%	90%	50%	90%	70%	50%	60%	60%	50%	40%	70%
BERT-base đầy đủ	82.4 ± 0.5	90.2 ± 0.5	88.4 ± 0.3	54.9 ± 1.2	89.1 ± 1.0	85.2 ± 0.1	66.2 ± 3.6	92.1 ± 0.1	54.5 ± 0.4	88.1 ± 0.6	63.5 ± 0.1
IMP từ θ_0	82.6 ± 0.2	90.0 ± 0.2	88.2 ± 0.2	54.9 ± 1.2	88.9 ± 0.4	84.9 ± 0.4	66.0 ± 2.4	91.9 ± 0.5	53.8 ± 0.9	87.7 ± 0.5	63.2 ± 0.3
Random pruning (mặt nạ ngẫu nhiên)	67.5	76.3	21.0	53.5	61.9	69.6	56.0	83.1	9.6	31.8	32.3
IMP mask + random init θ'_0	61.0	77.0	9.2	53.5	60.5	68.4	54.5	80.2	0.0	18.6	14.4
IMP mask + trọng số pretrain xáo trộn trong từng layer θ'''_0	70.1	79.2	19.6	53.3	62.0	69.6	52.7	82.6	4.0	24.2	42.3

(Metric: accuracy cho hầu hết; STS-B dùng Pearson, CoLA dùng Matthew's correlation, SQuAD dùng F1, MRPC/QQP có thêm số khác trong appendix.) Độ thưa vé cao nhất: từ 40% (SQuAD), 50% (MRPC, CoLA) đến 90% (QQP, WNLI). Paper nhận xét **không thấy mối quan hệ rõ** giữa độ thưa này và tính chất tác vụ (như cỡ tập train).



(theo paper). Xáo trộn trọng số trong layer tốt hơn random init nhưng vẫn kém xa vé, cho thấy phân phối trọng số theo layer của BERT pretrain là điểm khởi đầu "hơi có thông tin". Figure 1 (CoLA, SST-2, SQuAD): random pruning chỉ gần IMP ở độ thưa thấp (10-20%).

4.2 Rewinding không giúp (Table 3)

Rewind đến 5%, 10%, 20%, 50% quá trình train; ví dụ SQuAD: bước 1000, 2000, 4000, 10000. Rewind **không cải thiện đáng kể** ở tác vụ nào; ở STS-B và RTE kết quả giảm đến mức không còn matching (biên 2 điểm phần trăm). Ví dụ RTE từ 66.06 (rewind 0%) xuống 62.09 (rewind 5%/10%). Giải thích khả dĩ của tác giả: tập train nhỏ nên overfit.

4.3 IMP vs standard pruning (Table 3, dòng cuối)

Tùy tác vụ: QQP, QNLI, MRPC, MLM standard pruning hơn vé tới ~2 điểm; STS-B, WNLI, RTE, SST-2 kém hơn tới ~3 điểm. Ví dụ SST-2: 90.02 (standard) so với 91.74 (rewind 0%); QNLI: 89.88 so với 88.05. Chênh lớn nhất ở tác vụ có tập train nhỏ, nơi standard pruning có thể overfit.

4.4 Transfer (mục 5; độ thưa cố định 70% để loại yếu tố kích thước)

- Quy trình: IMP trên tác vụ nguồn S → train subnetwork (từ θ_0, lớp phân loại mới) trên tác vụ đích T; Transfer(S, T).
- **Phần lớn vé không chuyển**: trong 11 tác vụ, chỉ 3 tác vụ nguồn cho vé chuyển sang hơn hai tác vụ khác (để chuyển bằng same-task). Nếu cho phép giảm 2.5 điểm thì 7 tác vụ nguồn chuyển được ít nhất một nửa tác vụ khác.
- **Vé từ MLM là phổ quát**: transfer ngang same-task trên mọi tác vụ (Figure 3). Cắt trực tiếp trọng số pretrain θ_0 (không IMP) kém hơn: chỉ ngang same-task ở 4 trường hợp, nhưng vẫn tốt hơn mọi tác vụ nguồn khác ngoài MLM.
- Phụ lục: ở độ thưa s% cao nhất mà tác vụ T có vé, vé MLM cùng độ thưa còn là winning ticket với 5 tác vụ; 3 tác vụ nữa chênh dưới nửa điểm; hai tác vụ còn lại chênh 1.6 và 2.6 điểm.
- **Quy luật**: khả năng chuyển tương quan với **số mẫu train** của tác vụ nguồn (MNLI chuyển được 4 tác vụ, SQuAD 3, MRPC 1, WNLI 0, MLM tốt nhất), **không** liên quan loại tác vụ.
- **Chuyển từ θ_0 có tốt hơn?** Với SQuAD (nguồn tốt thứ hai sau MLM), rewind hoặc standard pruning cho kết quả chuyển bằng hoặc tốt hơn, tức trọng số đã học trên tác vụ nguồn không hại, thậm chí giúp.

Paper kết luận có thể xem đây là "giai đoạn thứ hai của pretrain BERT": sau pretrain, chạy IMP với MLM để được một subnetwork 70% thưa tương đương về năng lực.

5. Phân tích / ablation

- **Mask quan trọng, không chỉ độ thưa**: random pruning (mask ngẫu nhiên) thua IMP xa ở độ thưa cao.
- **Khởi tạo quan trọng**: random init và xáo trộn đều thua; trọng số pretrain mang thông tin.
- **Rewinding vô ích ở BERT**: khác với các bối cảnh khác (nơi vé chỉ tồn tại ở bước sớm), vé có sẵn tại θ_0; coi pretrain là giai đoạn "dense pretraining" làm điểm khởi đầu dễ thưa hóa.
- **Độ thưa vé không liên quan thuộc tính tác vụ**: không có quy luật theo kích thước tập train.
- **Chồng lấn mask**: (Phụ lục D) mask của các tác vụ downstream chia sẻ nhiều trọng số bị cắt chung; mask của MLM khác nhiều hơn.

6. Giới hạn

Do tác giả nêu:

- Họ dùng pruning **không cấu trúc** có chủ đích (nghiên cứu khoa học, không phải nỗ lực tăng tốc), nên không đo speedup phần cứng; ghi chú rằng ở độ thưa 70-90% XNNPACK đã cho tăng tốc trên CPU điện thoại, và để tương lai.

- Chi phí tìm vé: IMP cần train đầy đủ nhiều lần, đắt hơn nhiều so với train thường.
- Tác giả so sánh với Prasanna et al. (structured, cắt head/MLP): kết quả của họ khác; vì vé structured ít bằng chứng trong CV (Liu 2019, bài 11).

Nhận xét:

- Chỉ BERT-base (110M) và các tác vụ nhỏ của GLUE/SQuAD; không có mô hình sinh, không có LLM, không có đa phương thức. Không biết có áp được cho VLM/OCR không.
- "Matching" được định nghĩa trong một độ lệch chuẩn; ở tác vụ tập nhỏ có phương sai cao (RTE: ±3.6, WNLI toàn ngẫu nhiên ~54.9) tiêu chí này lỏng; WNLI (accuracy 54.9) hầu như không học được nên vé 90% thưa ở WNLI không có nhiều ý nghĩa.
- Cho thấy tồn tại mạng con thưa, không cho thuật toán rẻ tìm chúng; không bằng movement pruning (16) về hiệu quả đối với mức nén tổng thể.
- Theo checklist bài 12: ít kiến trúc (1), một cỡ mô hình, nhưng có 5 lần chạy cho Table 2 và nhiều tác vụ, tốt hơn trung bình văn liệu.

7. Liên hệ với các bài khác

- **08 Lottery Ticket**: định nghĩa winning ticket, IMP gốc; ở bài 08, vé có ở khởi tạo ngẫu nhiên với mạng nhỏ.
- **09 Linear mode connectivity & LTH**: giải thích vì sao vé ở mạng lớn cần rewinding (ổn định với nhiễu SGD); bài 15 cho thấy BERT pretrain đã "ổn định" nên $i = 0$ là đủ.
- **11 Rethinking the value of pruning**: gần như đối lập: ở CV với structured pruning, mạng thưa train từ đầu cũng tốt. Bài 15 dùng unstructured và khởi tạo pretrain nên kết luận khác; cần đọc cả hai để thấy kết luận phụ thuộc cài đặt.
- **12 State of pruning**: đồng tác giả Frankle; bài 15 là ví dụ có kiểm soát (random baselines, 5 lần chạy).
- **13-14 (cắt head)**, **16 Movement pruning**, **17 CoFi**: các cách khác nén BERT; 16 chọn trọng số theo "di chuyển" khi fine-tune thay vì độ lớn.
- **22-24 (OBC, SparseGPT, Wanda)**: pruning không cấu trúc cho LLM sau train, không cần IMP.

8. Gợi ý áp dụng

Mức liên quan: **thấp** trực tiếp, **trung bình** về ý tưởng.

2. **Bài học khái niệm quan trọng**: tại điểm khởi đầu pretrain, mask quan trọng chứ không chỉ kích thước; mask ngẫu nhiên kém xa mask theo độ lớn.
3. **Giả thuyết cảnh báo**: LTH nói "mạng con tồn tại", không nói "cắt xong phục hồi được bằng ít dữ liệu".

9. Thí nghiệm gợi ý

1. **Baseline ngẫu nhiên có kiểm soát**: với cấu hình MLP-neuron-cắt (cùng số tham số), so mask theo độ lớn / theo Taylor / ngẫu nhiên, sau cùng một lịch distillation ngắn; xem khoảng cách giống Table 2 (random thua xa) hay nhỏ hơn.
2. **Chồng lấn mask**: tính Jaccard giữa các tập đơn vị được giữ khi dùng các tập dữ liệu hiệu chuẩn khác nhau; nếu chồng lấn cao thì dữ liệu hiệu chuẩn không quan trọng.

10. Tóm tắt công thức / khái niệm

Khái niệm	Nội dung
Subnetwork	$f(x; m \odot \theta, \cdot),\ m \in \{0,1\}^{d_1}$
Matching	$\mathcal{E}^T(A_t^T(f(x; m \odot \theta, \gamma))) \geq \mathcal{E}^T(A_t^T(f(x; \theta_0, \gamma)))$

Khái niệm	Nội dung
Winning ticket	matching với $\theta = \theta_0$
Universal	matching trên mọi tác vụ trong tập
IMP	train → cắt 10% $\|w\|$ nhỏ nhất (toàn cục) → rewind → lặp
Standard pruning	cắt lặp, không rewind
Độ thưa vé	40% (SQuAD) … 90% (QQP, WNLI)
Vé MLM, 70%	phổ quát cho các tác vụ downstream
Transfer	tương quan với kích thước tập train nguồn, không với loại tác vụ

11. Kết luận ngắn

Với BERT, các quan sát LTH vẫn đúng: có vé thưa 40-90% tại chính trọng số pretrain, mask và khởi tạo đều quan trọng, rewinding không cần, và vé tìm bằng MLM ở 70% thưa là phổ quát cho GLUE/SQuAD. Đây là kết quả khoa học đẹp về cấu trúc mạng pretrain nhưng không cung cấp phương pháp tăng tốc thực tế (không cấu trúc, tốn kém).

Link gốc: https://arxiv.org/abs/2007.12223

Bài 16

Movement Pruning: Adaptive Sparsity by Fine-Tuning (2020)

Tác giả: Victor Sanh, Thomas Wolf, Alexander M. Rush (Hugging Face; Cornell).
Venue / arXiv: arXiv:2005.07683 (v2, 23/10/2020); NeurIPS 2020 (34th Conference on Neural Information Processing Systems).
Nguồn: https://arxiv.org/abs/2005.07683
Chủ đề: Pruning trọng số (không cấu trúc) của mô hình pretrain trong lúc fine-tune, dùng điểm quan trọng bậc nhất học bằng straight-through estimator ("di chuyển ra xa 0" thay vì "độ lớn").
Vị trí trong lộ trình: Cần đọc trước: 02 (Han 2015, magnitude pruning), 03 (Zhu & Gupta, lịch cắt cubic, dùng ở đây), 07 (Molchanov, điểm bậc nhất), 14 (Voita, L_0 Hard Concrete, là baseline ở đây), 15 (LTH BERT). Bài này thêm: một tiêu chí cắt mới phù hợp "pretrain rồi fine-tune". Kế tiếp: 17 (CoFi, cắt cấu trúc kết hợp distillation).
Chương nền liên quan: nền 05: Cắt rồi huấn luyện lại

> Ghi chú học tập bằng tiếng Việt, không phải bản dịch.

1. Bối cảnh

Magnitude pruning (giữ trọng số có $|w|$ lớn, Han 2015) là phương pháp cắt phổ biến nhất. Nó hợp lý khi học có giám sát từ đầu: giá trị trọng số do dữ liệu tác vụ quyết định. Nhưng trong **transfer learning** (pretrain trên dữ liệu khổng lồ, rồi tinh chỉnh trên dữ liệu nhỏ có nhãn), giá trị trọng số phần lớn đã được quyết định bởi mô hình pretrain và chỉ nhích nhẹ khi fine-tune. Magnitude pruning khi đó gần như "biết trước" ngay từ trước tinh chỉnh cái gì sẽ bị cắt (trọng số nhỏ lúc pretrain sẽ vẫn nhỏ) nên không thể thích nghi với tác vụ. Tác giả gọi việc cắt dựa trên tinh chỉnh là **"fine-pruning"**.

Thuật ngữ:

- **Zeroth-order / first-order**: tiêu chí chỉ dùng giá trị trọng số (bậc 0) so với dùng cả gradient (bậc 1).
- **Straight-through estimator (STE)**: khi một phép toán (như top-v%) có gradient bằng 0 hầu khắp nơi, ở bước backward giả vờ nó là hàm đồng nhất để gradient "đi thẳng qua".
- **Hadamard product** $\odot$: nhân từng phần tử.
- **Distillation**: huấn luyện mô hình nhỏ bắt chước phân phối đầu ra của mô hình lớn (teacher).

2. Ý tưởng chính

Giữ lại các kết nối **đang di chuyển ra xa 0** trong khi tinh chỉnh, thay vì các kết nối vốn đã lớn. Cụ thể, thêm một ma trận điểm quan trọng S cùng kích thước với trọng số W, và để S tích lũy $-\frac{\partial \mathcal{L}}{\partial W} \cdot W$ theo thời gian: trọng số nào cứ dương lên (gradient âm khi $W > 0$) hoặc âm xuống thì điểm tăng; trọng số nào co về 0 thì điểm giảm. Mask = top-v% của S. Khác với magnitude pruning, cả trọng số nhỏ lẫn lớn đều có thể bị cắt hay giữ. Ở độ thưa cao (còn < 15% trọng số) cách này vượt rõ magnitude pruning, và vượt L_0 regularization; phiên bản "soft" còn tốt hơn.

3. Phương pháp chi tiết

3.1 Khung score-based pruning

$W \in \mathbb{R}^{n \times n}$ là một ma trận trọng số; $S \in \mathbb{R}^{n \times n}$ là ma trận điểm; mask $M \in \{0,1\}^{n \times n}$. Suy luận: $a = (W \odot M)x$. Hàm chọn:

$$\text{Top}_v(S)_{i,j} = \begin{cases} 1, & S_{i,j} \text{ nằm trong top } v\% \\ 0, & \text{ngược lại} \end{cases}$$

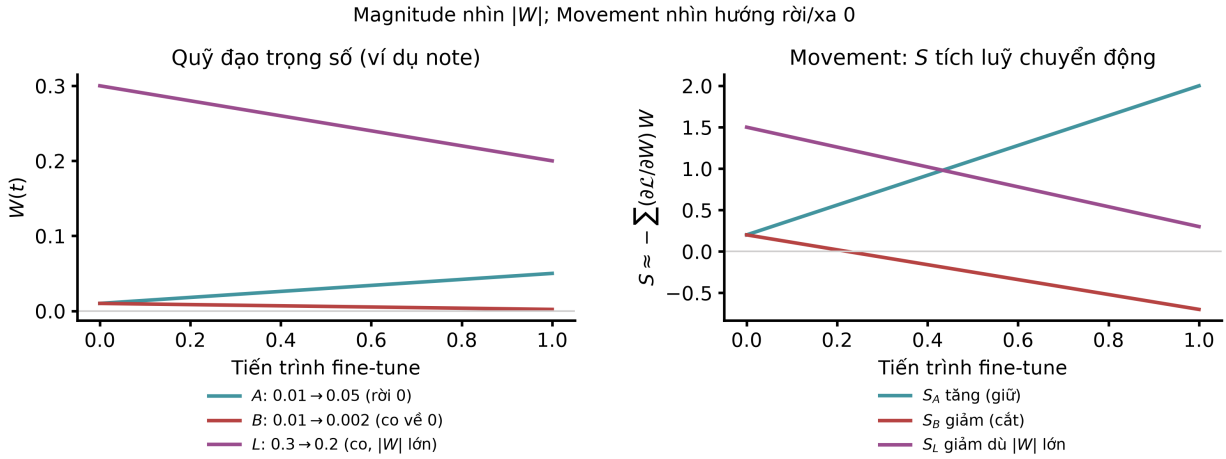
- **Magnitude pruning (MaP)**: $S_{i,j} = |W_{i,j}|$, $M = \text{Top}_v(S)$.
- Họ dùng **automated gradual pruning** (Zhu & Gupta, bài 03): trọng số bị mask vẫn được cập nhật nên có thể "sống lại", và độ thưa tăng dần theo lịch cubic. Với v_f mức giữ lại cuối, v_i khởi đầu, t_f bước cool-down, N bước cắt, khoảng cách Δt:

$$v^{(t)} = \begin{cases} v_i & 0 \le t < t_i \\ v_f + (v_i - v_f)\left(1 - \frac{t - t_i - t_f}{N \Delta t}\right)^3 & t_i \le t < T - t_f \\ v_f & \text{ngược lại} \end{cases}$$

(Paper viết bằng "độ giữ lại" v, và nói thêm vài bước cool-down cuối "empirically improves performance especially in high sparsity".)

3.2 Movement pruning (hard)

Mask $M = \text{Top}_v(S)$; **cả** W **và** S **cùng được học**. Forward: $a_i = \sum_k W_{i,k} M_{i,k} x_k$. Vì gradient của Top bằng 0 hầu khắp nơi, dùng STE: ở backward bỏ qua Top, gradient đi thẳng sang S:



Quỹ đạo $W(t)$ và điểm movement S: trọng số rời xa 0 được giữ dù $|W|$ nhỏ; trọng số lớn đang co về 0 có thể bị cắt. (minh hoạ, tự tính)

$$\boxed{\frac{\partial \mathcal{L}}{\partial S_{i,j}} = \frac{\partial \mathcal{L}}{\partial a_i} W_{i,j}\, x_j = \frac{\partial \mathcal{L}}{\partial W_{i,j}} W_{i,j}}$$

(bỏ số hạng mask cho gọn). Hệ quả: điểm của trọng số **bị mask vẫn được cập nhật**, nên trọng số đã bị cắt có thể quay lại nếu điểm tăng. Sau T bước cập nhật, với learning rate α_S:

$$\boxed{S_{i,j}^{(T)} = -\alpha_S \sum_{t<T} \left(\frac{\partial \mathcal{L}}{\partial W_{i,j}}\right)^{(t)} W_{i,j}^{(t)}}$$

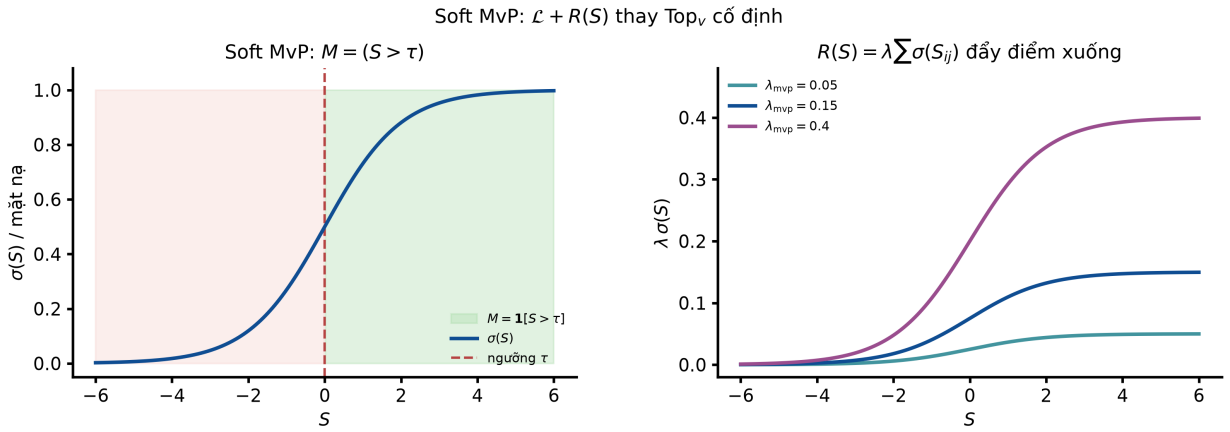
Nên S là **bộ tích lũy chuyển động**. Ý nghĩa dấu: $S_{i,j}$ tăng khi $\partial \mathcal{L}/\partial S_{i,j} < 0$, tức (a) $\partial \mathcal{L}/\partial W < 0$ và $W > 0$ (đang tăng lên khi dương), hoặc (b) $\partial \mathcal{L}/\partial W > 0$ và $W < 0$ (đang giảm khi âm); cả hai là W đang rời xa 0.

Ngược lại S giảm khi W co về 0.

Paper chứng minh (Phụ lục A.1) rằng bài toán tối ưu của movement pruning hội tụ (đảm bảo giảm loss với learning rate nhỏ); chi tiết chứng minh không đọc ở đây.

3.3 Soft movement pruning

Thay tỉ lệ cố định v bằng **ngưỡng toàn cục** τ: $M = (S > \tau)$. Để điều khiển độ thưa, thêm regularizer đẩy điểm giảm dần:



Soft MvP: mặt nạ $M = (S > \tau)$ và regularizer $R(S) = \lambda \sum \sigma(S)$ đẩy điểm xuống để tăng độ thưa. (minh hoạ, tự tính)

$$\boxed{R(S) = \lambda_{mvp} \sum_{i,j} \sigma(S_{i,j})}$$

với σ là sigmoid; λ_{mvp} càng lớn càng thưa. (Tác giả cũng thử $\sum |S_{i,j}|$ nhưng khó chỉnh hơn với kết quả tương tự.) Mục tiêu: $\mathcal{L} + R(S)$.

3.4 So sánh với L_0 regularization (Louizos 2017)

L_0 dùng Hard Concrete (như bài 14): mask lấy mẫu $u \sim U(0,1)$, $\bar{S} = \sigma((\log u - \log(1-u) + S)/b)$, $Z = (r-l)\bar{S} + l$, $M = \min(1, \text{ReLU}(Z))$, với $b > 0, l < 0, r > 1$; chuẩn L_0 kỳ vọng có dạng đóng. Gradient lên S có cùng dạng $\frac{\partial \mathcal{L}}{\partial a_i} W_{i,j} x_j \, f(\bar{S}_{i,j})$ nhân một hàm f (Eq. 4). Tác giả chỉ ra **movement pruning và soft movement pruning cho cập nhật tương tự** L_0, một phương pháp "movement" khác, nhưng đơn giản và tất định (không lấy mẫu).

Tóm tắt (Table 1 của paper):

	Magnitude	L_0 reg	Movement	Soft movement
Bậc quyết định	0	1	1	1
Hàm mask	Top$_v$	Hard-Concrete liên tục	Top$_v$	ngưỡng
Cấu trúc	local hoặc global	global	local hoặc global	global
Mục tiêu	$\mathcal{L}$	$\mathcal{L} + \lambda_{l0}\mathbb{E}(L_0)$	$\mathcal{L}$	$\mathcal{L} + \lambda_{mvp} R(S)$

3.5 Pseudo-code

```
Khởi tạo: W <- BERT pretrain; S <- khởi tạo (điểm ban đầu); schedule v(t) (cubic)
for t = 1..T:
```

```
    M <- Top_{v(t)}(S)            # hard;   hoặc  M <- 1[S > tau]  # soft
    a <- (W ⊙ M) x                 # forward
    L <- loss(a, nhãn) [+ distillation] [+ λ_mvp * Σ σ(S) nếu soft]
    # backward: Top bị bỏ qua (STE)
    W <- W - α_W * ∂L/∂W          # W được cập nhật cả khi M = 0
    S <- S - α_S * (∂L/∂W ⊙ W)    # tích lũy chuyển động
Kết thúc: cố định M, bỏ trọng số mask bằng 0
```

Ví dụ nhỏ: hai trọng số cùng $|W| = 0.01$ ở thời điểm đầu. Trọng số A tăng đều lên 0.05 trong fine-tune; B co về 0.002. Magnitude pruning sau đó xếp B thấp hơn A, nhưng cả hai đã bị coi là "nhỏ" trước fine-tune. Movement pruning cho A điểm tăng, B điểm giảm bất kể độ lớn ban đầu; một trọng số lớn $|W| = 0.3$ đang co về 0.2 cũng có thể bị cắt trước A.

4. Thí nghiệm và kết quả chính

Thiết lập. BERT-base-uncased (khoảng 84M tham số trong encoder và head; embedding bị **đóng băng**); fine-tune lớp transformer và head tác vụ. Mọi con số "độ thưa/remaining weights" tính trên BERT-base và bằng đúng kích thước mô hình, kể cả với baseline. Ba tác vụ: SQuAD v1.1 (hỏi đáp; paper ghi 8K ví dụ train, có thể là lỗi in vì bài 15 và 17 ghi ~88K), MNLI (393K), QQP (364K). Mỗi phương pháp cùng số bước cập nhật (6 đến 10 epoch). Baseline: MaP, L_0 regularization (Hard Concrete, unstructured), RPP (Guo 2019), LayerDrop (Fan 2020a, chỉ số báo từ tác giả gốc), mini-BERT (Turc 2019).

4.1 Kết quả chính (Table 2, không distillation; 10% = 8.5M tham số encoder, 3% = 2.6M)

Tác vụ (metric), BERT-base fine-tune	Còn lại	MaP	L_0	MvP	soft MvP
SQuAD dev EM/F1 (80.4/88.1)	10%	67.7/78.5	69.9/80.0	71.9/81.7	71.3/81.5
	3%	40.1/54.5	61.2/73.3	65.2/76.3	69.5/79.9
MNLI dev acc/MM acc (84.5/84.9)	10%	77.8/79.0	77.9/78.5	79.3/79.5	80.7/81.1
	3%	68.9/69.8	75.1/75.4	76.1/76.7	79.0/79.6
QQP dev acc/F1 (91.4/88.4)	10%	78.8/75.1	87.5/81.9	89.1/85.5	90.5/87.1
	3%	72.1/58.4	86.5/81.0	85.6/81.0	89.3/85.6

Đọc nhanh: SQuAD 3%: MaP 54.5 F1; L_0 73.3; MvP 76.3; soft MvP **79.9**.

- Mức thưa thấp (còn hơn 70% trọng số): **magnitude pruning thắng**, gần như không mất; còn movement pruning giảm nhanh ngay cả ở độ thưa thấp.
- Mức thưa cao (< 15% còn lại): các phương pháp bậc nhất vượt rõ; soft MvP luôn tốt nhất hoặc ngang tốt nhất (QQP: RPP ngang soft MvP). Hard MvP ngang L_0 (hơn rõ ở QQP).
- Movement pruning vượt mini-BERT (nhỏ, train từ đầu), nhất quán với Li et al. 2020 (huấn luyện mô hình lớn rồi nén hiệu quả hơn train mô hình nhỏ). Tác giả cảnh báo: phần cứng hiện tại chưa tối ưu suy luận cho mô hình thưa, nên mô hình dense nhỏ như mini-BERT vẫn có thể hợp lý hơn về tốc độ.

4.2 Kết hợp distillation (Table 3)

Teacher là chính BERT-base đã fine-tune; mục tiêu là tổ hợp lồi (convex combination) giữa loss huấn luyện và loss distillation trên phân phối đầu ra. Distillation tăng điểm mọi phương pháp và mọi mức, **thứ hạng giữ nguyên**:

Tác vụ	Còn lại	MaP	L_0	MvP	soft MvP
SQuAD EM/F1	10%	70.2/80.1	72.4/81.9	75.6/84.3	76.6/84.9
	3%	45.5/59.6	64.3/75.8	67.5/78.0	72.7/82.3
MNLI acc/MM	10%	78.3/79.3	78.7/79.7	80.1/80.4	81.2/81.8
	3%	69.4/70.6	76.0/76.2	76.5/77.4	79.5/80.1
QQP acc/F1	10%	79.8/65.0	88.1/82.8	89.7/86.2	90.2/86.8
	3%	72.4/57.8	87.0/81.9	86.1/81.5	89.1/85.5

Ví dụ nêu trong paper: SQuAD, MvP 10%: 81.7 F1 → 84.3 F1. Soft MvP + distillation đạt **95% hiệu năng BERT-base chỉ với ~5% trọng số encoder** (SQuAD và MNLI), như abstract ghi "down to only 3% of the model parameters" với mất mát tối thiểu.

5. Phân tích / ablation

- **Phân phối trọng số còn lại (Figure 4a):** magnitude pruning loại sạch trọng số gần 0, còn lại hai cụm; movement pruning cho phân phối mượt phủ toàn khoảng, chỉ trừ vùng sát 0.
- **Điểm S vs trọng số (Figure 4b):** không có quan hệ đơn giản; trọng số lớn hay nhỏ đều có thể quan trọng, nhưng điểm cao luôn đi cùng trọng số khác 0 (hình chữ V), khớp diễn giải "chuyển động ra xa 0". Figure 1: trong fine-tuning, trọng số ở sát giá trị pretrain; vì vậy magnitude pruning gần như quyết định từ trước.
- **Local và global:** hiệu năng tương đương ở hầu hết mức thưa; chỉ ở 3% còn lại, global hơn local 2.3 F1 (movement pruning). Phân bổ theo layer (Figure 6, còn 10%): các phương pháp bậc nhất toàn cục **giữ nhiều trọng số ở layer thấp và cắt nặng layer cao**; global magnitude pruning cắt gần đều các layer như local.
- **Soft vs hard:** soft thường tốt hơn hard và L_0, dù chỉ có thêm một regularizer đơn giản.
- **Số bước cắt:** ghi chú trong bài rằng tăng số bước cắt thường cải thiện kết quả cuối.

6. Giới hạn

Do tác giả nêu:

- Thưa không cấu trúc: phần cứng chưa tối ưu suy luận cho mô hình thưa; mô hình dense nhỏ có thể hợp lý hơn về tốc độ.
- Ở độ thưa thấp, magnitude pruning tốt hơn.
- Gợi ý hướng tương lai: dùng group sparsity (xóa cả cột/hàng) để cắt có cấu trúc, ví dụ loại cả cột của ma trận embedding.
- Phải có dữ liệu và bước fine-tune cho từng tác vụ (cắt cụ thể theo tác vụ).

Nhận xét:

- Mỗi trọng số cần thêm một điểm S và cập nhật STE: chi phí bộ nhớ train gần gấp đôi tham số (thêm S và trạng thái optimizer), đáng kể khi áp cho mô hình 1B+ tham số.
- Chỉ BERT-base và 3 tác vụ; chỉ đo accuracy, không đo tốc độ; embedding bị đóng băng nên "độ thưa" không tính chúng.
- Tính chất "dựa vào chuyển động" giả định fine-tune đủ dài/đủ tín hiệu; với dữ liệu hiệu chuẩn nhỏ (vài trăm mẫu) điểm có thể nhiễu (chưa được thử).
- Checklist bài 12: có 3 tác vụ, nhiều điểm hoạt động (đường cong), nhưng không báo độ lệch chuẩn qua nhiều seed trong Table 2-3.

7. Liên hệ với các bài khác

- **02 Han 2015 / 03 Zhu & Gupta**: dùng lại magnitude pruning và lịch cubic; movement pruning chỉ thay tiêu chí chọn.
- **07 Molchanov 2019**: điểm Taylor bậc nhất $\propto \frac{\partial \mathcal{L}}{\partial W} W$ có cùng dạng; khác ở chỗ ở đây **tích lũy qua nhiều bước kèm dấu** thay vì tính một lần và bình phương/tuyệt đối. Paper nói họ giữ hướng chuyển động, khác các bài dùng |gradient|.
- **13 Sixteen Heads, 14 Voita**: cũng phương pháp bậc nhất/mask học được nhưng cho head; 14 dùng Hard Concrete đúng là baseline L_0 ở đây.
- **15 LTH BERT**: cùng bối cảnh BERT, cùng IMP cho ra mạng thưa, nhưng 16 tập trung độ thưa cực cao (3-10%) khi fine-tune.
- **17 CoFi**: lấy lại ý "cắt trong lúc fine-tune + distillation", nhưng chuyển sang cắt cấu trúc.
- **22-24 (OBC, SparseGPT, Wanda)**: pruning LLM sau train không có fine-tune; ngược với "fine-pruning" ở đây.
- **12 State of pruning**: ví dụ về vấn đề so sánh phụ thuộc mức nén: bài này cho thấy thứ hạng MaP vs MvP **đảo** theo độ thưa (MaP thắng ở thưa thấp).

8. Gợi ý áp dụng

Mức liên quan: **trung bình về ý tưởng, thấp về kỹ thuật trực tiếp**.

Chỉ giảm dung lượng lưu trữ nếu có định dạng thưa.

2. **Ý tưởng cốt lõi dùng được cho cấu trúc**: tính điểm cho *nhóm* trọng số (một neuron MLP = một hàng của W_{up} + cột tương ứng của W_{down}; một head; một block) bằng tích lũy $-\sum \frac{\partial \mathcal{L}}{\partial W} \odot W$ nhóm lại. Paper tự gợi ý group sparsity là hướng tương lai.
3. **Distillation hầu như luôn cộng thêm** (Table 3: tăng cho mọi phương pháp, giữ thứ hạng) nên không cần chọn giữa "cắt thông minh" và "distill".
4. **Điểm S là một đại lượng chẩn đoán**: nhìn phân bố remaining theo layer (Figure 6: giữ lớp thấp, cắt lớp cao) cho giả thuyết về block nào ít quan trọng.

9. Thí nghiệm gợi ý

Đo CER.

2. **Mức thưa thấp vs cao**: lặp (1) ở mức cắt 10%, 30%, 60% neuron của một block; kiểm chứng có điểm giao cắt giống paper (tiêu chí tĩnh thắng ở mức nhẹ, động thắng ở mức nặng) không.
3. **Phân bổ theo layer**: dùng điểm s_g toàn cục để xem layer nào được giữ nhiều/ít neuron; so với phân bổ đều.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / ý nghĩa
Suy luận với mask	$a = (W \odot M)x$
Top-v	$M = \text{Top}_v(S)$
Magnitude	$S = \lvert W \rvert$
Movement (hard)	$\frac{\partial \mathcal{L}}{\partial S_{i,j}} = \frac{\partial \mathcal{L}}{\partial W_{i,j}} W_{i,j}$ (STE), $S^{(T)} = -\alpha_S \sum_{t<T} \frac{\partial \mathcal{L}}{\partial W} W$
Soft movement	$M = (S > \tau)$, $R(S) = \lambda_{mvp} \sum \sigma(S_{i,j})$
L_0 reg	mask Hard Concrete, $\mathbb{E}(L_0) = \sum \sigma(\log S_{i,j} - b \log(-l/r))$
Lịch cắt	cubic, có bước cool-down t_f
Kết quả	SQuAD 3% còn: MaP 54.5, L_0 73.3, MvP 76.3, soft MvP 79.9 F1

Khái niệm	Công thức / ý nghĩa
Với distillation	soft MvP đạt 95% BERT-base với ~5% trọng số encoder (SQuAD, MNLI)

11. Kết luận ngắn

Khi mô hình đã pretrain và chỉ tinh chỉnh nhẹ, độ lớn trọng số là một tiêu chí gần như "đóng băng". Chọn giữ trọng số đang di chuyển ra xa 0 (điểm S tích lũy bằng STE) thích nghi với tác vụ và vượt rõ magnitude pruning, L_0 khi còn < 15% trọng số; soft MvP kết hợp distillation đạt 95% BERT-base với ~5% trọng số encoder. Hạn chế lớn là tính không cấu trúc.

Link gốc: https://arxiv.org/abs/2005.07683

Bài 17

Structured Pruning Learns Compact and Accurate Models (2022)

Tác giả: Mengzhou Xia, Zexuan Zhong, Danqi Chen (Princeton University).
Venue / arXiv: arXiv:2204.00408 (v3, 2/5/2022). Paper trong file không ghi tên hội nghị. Phương pháp thường được gọi là **CoFi Pruning** (Coarse- and Fine-grained Pruning).
Nguồn: https://arxiv.org/abs/2204.00408
Chủ đề: Structured pruning task-specific cho BERT: mask nhiều mức hạt (layer, head, neuron FFN, chiều ẩn) cùng học bằng L_0 Hard Concrete + distillation theo layer với ánh xạ layer động.
Vị trí trong lộ trình: Cần đọc trước: 14 (Voita, cổng L_0 Hard Concrete), 16 (Movement pruning, baseline không cấu trúc), 13 (cắt head), 12 (cách đọc thí nghiệm). Bài này là **điểm hội tụ** của nhánh "cắt cấu trúc cho Transformer": mask nhiều tầng + distillation trong lúc cắt. Kế tiếp: 18 (2:4 sparsity, phần cứng), 25 ZipLM, 26 LLM-Pruner, 38 Sheared LLaMA (kế thừa trực tiếp ý mask L_0 đa tầng cho LLM).
Chương nền liên quan: nền 04: Cắt hạt, cắt khối và mẫu 2

> Ghi chú học tập bằng tiếng Việt, không phải bản dịch.

1. Bối cảnh

Hai họ nén mô hình ngôn ngữ cho tới 2022:

- **Pruning**: bắt đầu từ mô hình lớn đã huấn luyện, loại dần tham số, thường ra mô hình **task-specific** (riêng cho từng tác vụ). Unstructured pruning (như bài 16) đạt độ thưa rất cao (tới 97%) nhưng khó tăng tốc thật. Structured pruning (cắt layer, head, chiều trung gian FFN, block ma trận) phù hợp phần cứng hơn nhưng trước CoFi **hiếm khi tăng tốc quá 2-3 lần**.
- **Distillation**: chọn trước kiến trúc student nhỏ (DistilBERT, TinyBERT, MobileBERT...), khởi tạo ngẫu nhiên, chưng cất từ teacher trên **corpus không nhãn khổng lồ** ("general distillation") rồi mới chưng cất/fine-tune theo tác vụ. Đạt tăng tốc lớn nhưng cực đắt: TinyBERT cần 3.5 ngày trên 4 GPU cho 2,500M token × 3 epoch trước khi làm bước theo tác vụ (Figure 1).

Mục tiêu của CoFi: đạt **tăng tốc >10× với độ chính xác tương đương distillation** mà **không cần dữ liệu không nhãn** và với chi phí huấn luyện nhỏ.

Thuật ngữ:

- **Task-specific vs task-agnostic**: mô hình dành cho một tác vụ (đã biết nhãn) hay mô hình đa dụng.
- **Mask variable** z: biến (0 hay 1, hoặc [0,1] khi huấn luyện) nhân vào một đơn vị để bật/tắt đơn vị đó.
- **FFN intermediate dimension**: số neuron của lớp ẩn trong khối MLP ($d_f = 4d$ thường).
- **Hidden dimension**: chiều của residual stream ($d = 768$).
- **Speedup**: ở paper, thời gian suy luận của BERT-base chia thời gian của mô hình cắt, đo trên một GPU V100, batch 128.

2. Ý tưởng chính

Thay vì cắt một loại đơn vị (chỉ head, hoặc chỉ neuron FFN), CoFi gắn **nhiều mask ở các mức hạt khác nhau** lên cùng một tham số: mask cho cả lớp MHA (z_{MHA}), cả lớp FFN (z_{FFN}), từng head (z_{head}), từng neuron trung gian (z_{int}) và từng chiều ẩn (z_{hidn}, dùng chung mọi layer). Một trọng số bị cắt nếu *bất kỳ* mask nào bao nó bằng 0. Mask mịn cho tính linh hoạt, mask thô cho phép bỏ cả layer, điều quan trọng để tăng tốc thật ở độ thưa cao. Các mask được học bằng L_0 Hard Concrete (như bài 14) với ràng buộc độ thưa chính xác

bằng Lagrangian. Bên cạnh đó, vì kiến trúc student chưa biết trước, họ dùng **layerwise distillation với ánh xạ layer động**: mỗi lớp của teacher được ghép với layer student gần nhất (theo MSE) ở thời điểm hiện tại.

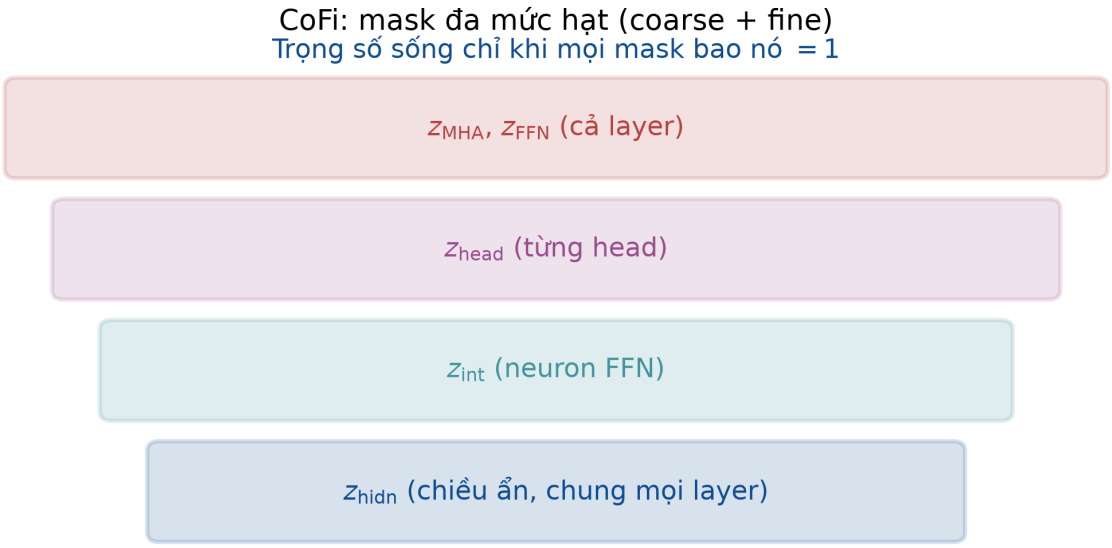
3. Phương pháp chi tiết

3.1 Kiến trúc Transformer và các đơn vị có thể cắt

Block $= $ MHA + FFN (có residual và LayerNorm). MHA với N_h head, $W_Q^{(i)}, W_K^{(i)}, W_V^{(i)}, W_O^{(i)} \in \mathbb{R}^{d \times d_h}$, $d_h = d/N_h$. FFN: $\text{FFN}(X) = \text{gelu}(XW_U)\,W_D$ với $W_U \in \mathbb{R}^{d \times d_f}$, $W_D \in \mathbb{R}^{d_f \times d}$. MHA chiếm khoảng 1/3 tham số, FFN 2/3 (không tính embedding). Theo Ganesh et al. (2021) trích trong paper: MHA và FFN tốn thời gian xấp xỉ nhau trên GPU, còn FFN là nút cổ chai trên CPU.

3.2 Năm loại mask

$$\boxed{\text{MHA}(X) = z_{MHA} \sum_{i=1}^{N_h} z_{head}^{(i)}\, \text{Att}\big(W_Q^{(i)}, W_K^{(i)}, W_V^{(i)}, W_O^{(i)}, X\big)}$$



CoFi chồng năm mask ($z_{MHA/FFN}$, head, neuron, hidn); trọng số sống khi mọi mask $= 1$. (minh hoạ, tự tính)

$$\boxed{\text{FFN}(X) = z_{FFN} \cdot \text{gelu}(XW_U) \cdot \text{diag}(z_{int}) \cdot W_D}$$

- $z_{head}^{(i)} \in \{0,1\}$: từng head. $z_{int} \in \{0,1\}^{d_f}$: từng neuron trung gian của FFN.
- $z_{MHA}^{(l)}, z_{FFN}^{(l)} \in \{0,1\}$: từng lớp MHA / FFN, **tách riêng nhau** (khác Fan 2020, Sajjad 2020 cắt cả block).
- $z_{hidn} \in \{0,1\}^{d}$: chiều ẩn, **dùng chung mọi layer** vì mỗi chiều nối với cùng chiều đó ở layer sau qua residual; áp lên mọi ma trận (ví dụ $\text{diag}(z_{hidn})W_Q$). Thực nghiệm chỉ một số ít chiều bị cắt (ví dụ 768 → 760) nhưng vẫn cải thiện đáng kể.

Điểm khác biệt chính: **nhiều mask cùng điều khiển một tham số**. Ví dụ một trọng số của FFN bị cắt nếu cả lớp FFN, hoặc neuron trung gian tương ứng, hoặc chiều ẩn tương ứng bị cắt. Block Pruning (Lagunas 2021) so sánh chỉ dùng một chiến lược mỗi loại lớp.

Vì sao cần mask thô? Cắt hết $N_h = 12$ head tương đương cắt cả lớp MHA, nhưng tác giả quan sát rằng điều này **hiếm khi tự xảy ra** nếu chỉ có mask mịn, gây khó tối ưu ở độ thưa cao. Mask thô tạo đường tắt.

3.3 Học mask: L_0 Hard Concrete + Lagrangian

Mỗi mask là biến Hard Concrete (Louizos 2018; xem bài 14, 16): với $u \sim U(0,1)$,

$$s = \text{sigmoid}\Big(\tfrac{1}{\beta}\big(\log \tfrac{u}{1-u} + \log \alpha\big)\Big), \quad \tilde{s} = s\,(r-l) + l, \quad z = \min(1, \max(0, \tilde{s}))$$

với $l < 0$, $r > 0$ kéo giãn đầu ra của sigmoid; $\log \alpha$ là tham số học được chính; β điều chỉnh độ dốc. Khi huấn luyện mask nhận giá trị thực trong $[0,1]$; lúc suy luận, mask nhỏ hơn ngưỡng đặt về 0 (ngưỡng xác định theo độ thưa kỳ vọng của từng ma trận; chi tiết ở Appendix B).

Độ thưa kỳ vọng $\hat{s}$ tính trên kích thước mô hình đầy đủ M (nhân các mask liên quan, hệ số 4 cho Q,K,V,O và 2 cho W_U, W_D):

$$\hat{s} = \frac{1}{M}\sum_l \sum_i \sum_k z^{(l)}_{MHA} z^{(l,i)}_{head} z^{(k)}_{hidn} \cdot 4d_h + \frac{1}{M}\sum_l \sum_j \sum_k z^{(l)}_{FFN} z^{(l,j)}_{int} z^{(k)}_{hidn} \cdot 2$$

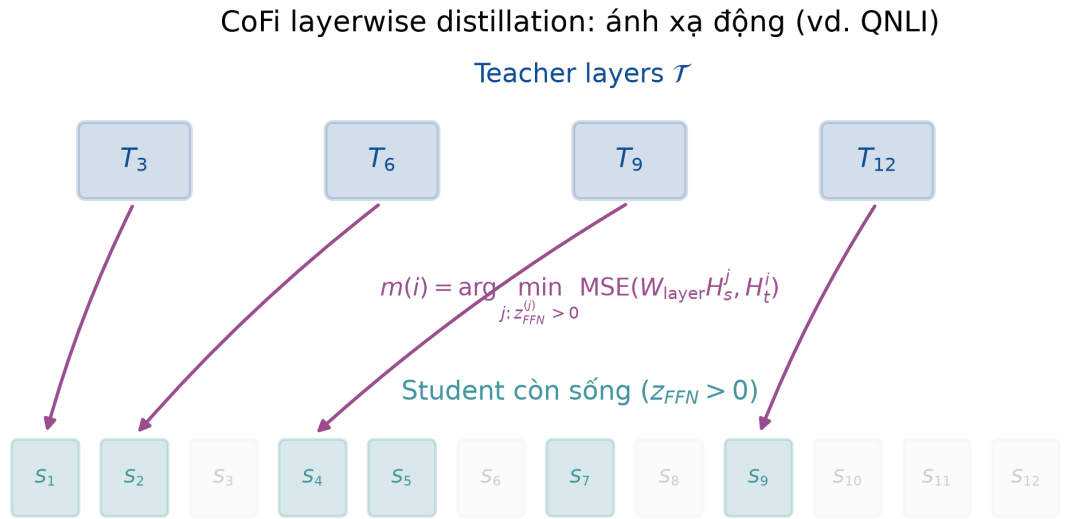
(Công thức trong paper là dạng này; đây là cách viết đã được làm gọn: xem Mục 3.1 của paper cho dấu hiệu chính xác.) Vì chỉ phạt $\lambda\|z\|_0$ thì kích thước cuối phụ thuộc lịch và learning rate, nên dùng **Lagrangian** (theo Wang 2020b) để ép $\hat{s} = t$ (độ thưa mục tiêu):

$$\boxed{\mathcal{L}_c = \lambda_1(\hat{s} - t) + \lambda_2(\hat{s} - t)^2}$$

Tác giả cũng thử straight-through estimator (như bài 16) và thấy hiệu năng ngang; chọn L_0 vì dễ điều khiển độ thưa chính xác.

3.4 Layerwise distillation với ánh xạ layer động

Mục tiêu dự đoán (KL giữa phân phối student p_s và teacher p_t):



Ánh xạ layer động teacher → student theo MSE trên layer FFN còn sống (vd. QNLI). (minh hoạ, tự tính)

$$\mathcal{L}_{pred} = D_{KL}(p_s \,\|\, p_t)$$

Với student chưa biết kiến trúc, định nghĩa trước ánh xạ layer cố định như TinyBERT là không thể. Gọi $\mathcal{T}$ là tập layer teacher dùng để chưng cất, $m(i)$ là layer student ghép với layer teacher i:

$$\boxed{\mathcal{L}_{layer} = \sum_{i \in \mathcal{T}} \text{MSE}\big(W_{layer} H_s^{m(i)},\, H_t^i\big), \qquad m(i) = \arg \min_{j:\, z_{FFN}^{(j)} > 0} \text{MSE}\big(W_{layer} H_s^j, H_t^i\big)}$$

- H: biểu diễn ẩn **ở các layer FFN** (đầu ra FFN); $W_{layer} \in \mathbb{R}^{d \times d}$ là ma trận tuyến tính học được, **khởi tạo là ma trận đơn vị**.
- Ánh xạ chọn layer student còn sống gần nhất (theo MSE) với mỗi layer teacher; tính khoảng cách song song nên chi phí nhỏ.
- **Ràng buộc chống "layer mismatch"** (xảy ra với dataset nhỏ như RTE, MRPC): chỉ cho ghép layer teacher tới layer student **cao hơn hoặc bằng layer đã ghép trước đó** (đơn điệu). Với dataset lớn, mismatch hiếm.

Tổng hợp:

$$\boxed{\mathcal{L}_{distil} = \lambda\, \mathcal{L}_{pred} + (1 - \lambda)\, \mathcal{L}_{layer}}$$

Toàn bộ mục tiêu huấn luyện: $\mathcal{L}_{distil}$ + $\mathcal{L}_c$ (độ thưa). Teacher là BERT-base đã fine-tune trên tác vụ; student khởi đầu từ chính nó.

3.5 Quy trình và pseudo-code

```
Input: BERT-base đã fine-tune (teacher = student ban đầu), độ thưa mục tiêu t,
       dữ liệu tác vụ (GLUE/SQuAD)
Gắn mask Hard Concrete: z_MHA, z_FFN (mỗi layer), z_head, z_int, z_hidn
Embedding bị đóng băng
Pha A:  fine-tune 1 epoch với L_distil                    (chưa cắt)
Pha B:  tăng độ thưa tuyến tính tới t trong 2 epoch;       (cắt)
        mỗi bước tối ưu: L_distil + L_c (Lagrangian) cập nhật
        trọng số W, tham số mask log_alpha, lambda_1, lambda_2
        mỗi bước: m(i) = argmin_j MSE(W_layer H_s^j, H_t^i)  # ánh xạ layer động
        (tiếp tục huấn luyện/tìm cấu trúc đến hết 20 epoch)
Pha C:  loại hẳn đơn vị có mask = 0 (cấu trúc cố định),
        fine-tune mạng con 20 epoch với L_distil           (không mask nữa)
Output: mạng con dense, nhỏ
```

Chi tiết (Appendix A): bốn tập lớn (MNLI, QNLI, SST-2, QQP) và SQuAD: 20 epoch tổng cộng + 20 epoch fine-tune mạng con; tập GLUE nhỏ: 100 epoch tổng + 20 epoch fine-tune; với tập nhỏ có 4 epoch distill rồi cắt tới target trong 20 epoch kế. Siêu tham số: $\lambda \in \{0.1, 0.3, 0.5\}$, nhiệt độ distill $t = 2$, lr fine-tune $\{1, 2, 3\} \times 10^{-5}$, lr huấn luyện 2×10^{-5} (GLUE), 3×10^{-5} (SQuAD), batch 32 (GLUE), 16 (SQuAD). Tác giả ghi rằng "finetuning the final subnetwork is essential for high sparsity models" và sau khi đạt độ thưa mục tiêu quá trình vẫn tiếp tục tìm cấu trúc tốt hơn.

4. Thí nghiệm và kết quả chính

Thiết lập. BERT-base (và RoBERTa, ở Appendix I); 8 tác vụ GLUE (SST-2, MNLI, QQP, QNLI, MRPC, CoLA, STS-B, RTE) + SQuAD v1.1; kết quả trên dev set. Độ thưa mục tiêu $\{60, 70, 75, 80, 85, 90, 95\}$% (tính theo tham số, **không tính embedding**; embedding bị đóng băng). Baseline: DistilBERT$_6$, TinyBERT$_6$/TinyBERT$_4$, DynaBERT, Block Pruning (Lagunas 2021, bản "Hybrid Filled"), thêm FLOP,

LayerDrop, Movement Pruning, MobileBERT, AutoTinyBERT (không trực tiếp so sánh được; ở Appendix F). Tăng tốc đo trên **một GPU NVIDIA V100**, input 128 token (GLUE) hoặc 384 (SQuAD), batch 128, so với BERT-base chưa cắt.

4.1 Bảng so sánh tổng (Table 1, MNLI; U = dùng dữ liệu không nhãn, T = task-specific)

Phương pháp	U	T	Tăng tốc	Tham số	MNLI
BERT-base (teacher)	không	không	1.0×	85M	84.8
DistilBERT$_6$	có	không	2.0×	43M	82.2
TinyBERT$_6$	có	có	2.0×	43M	84.0
MobileBERT‡	có	không	2.3×	20M	83.9
DynaBERT	không	có	6.3×	11M	76.3
AutoTinyBERT‡	có	có	9.1×	3.3M	78.2
TinyBERT$_4$	có	có	11.4×	4.7M	78.8
Movement Pruning	không	có	1.0×	9M	81.2
Block Pruning	không	có	2.7×	25M	83.7
CoFi (ours)	không	có	**2.7×**	26M	**84.9**
CoFi (ours)	không	có	**12.1×**	4.4M	**80.6**

‡ dùng teacher khác, không so sánh trực tiếp. Lưu ý: Movement Pruning có 9M tham số **nhưng tăng tốc 1.0×** (thưa không cấu trúc) đúng như đã dự đoán ở bài 16.

4.2 CoFi vs TinyBERT$_4$ (Table 2, ~10× tăng tốc, ~5M tham số, ~95% thưa; không dùng data augmentation)

	SST-2	QNLI	MNLI	QQP	CoLA	RTE	STS-B	MRPC	SQuAD	Thời gian train (GPU-giờ)
BERT-base (teacher)	93.1	91.5	84.8	91.2	61.2	70.0	88.7	85.0	88.4	n/a
TinyBERT$_4$ w/o GD	87.7	81.8	78.7	89.5	16.6	47.3	17.8	68.9	n/a	≤ 10
TinyBERT$_4$	89.7	86.7	78.8	90.0	32.5	63.2	85.0	81.4	82.1	~350
CoFi	90.6	86.1	80.6	90.1	35.6	64.7	83.1	82.6	82.6	≤ 20

Tăng tốc: TinyBERT$_4$ 11.4× (SQuAD 8.7×); CoFi từ 11.0× đến 12.9× (SQuAD 8.7×). GD = general distillation. CoFi thắng hoặc ngang ở 6/9 cột (hơn ở SST-2, MNLI, QQP, CoLA, RTE, MRPC, SQuAD; kém ở QNLI và STS-B), với chi phí ≤ 20 GPU-giờ so với ~350.

Với data augmentation theo TinyBERT (Table 3; dữ liệu tăng cường lớn hơn 20×, nên chỉ chạy 4 tập; số trước → là không augment): TinyBERT$_4$: SST-2 89.7→91.6, QNLI 86.7→87.6, RTE 63.2→62.5, MRPC 81.4→83.6; CoFi: SST-2 90.6→92.4, QNLI 86.1→86.8, RTE 64.7→67.5, MRPC 82.6→84.6. (Tác giả tự chạy lại augment vì tập gốc không được công bố; kết quả tái hiện của TinyBERT$_4$ thấp hơn số công bố: ví dụ SST-2 91.6 so với 92.7.)

4.3 Đường cong (Figure 2)

CoFi cho mô hình chính xác hơn distillation và pruning baseline **ở mọi mức tăng tốc và kích thước** trên SST-2, QNLI, MNLI, SQuAD. Block Pruning không đạt tăng tốc như TinyBERT$_4$; DynaBERT chậm hơn vì

chỉ cắt tối đa nửa số lớp MHA/FFN. Các đường cong này đọc từ hình; bảng số chi tiết không nằm trong text.

5. Phân tích / ablation

5.1 Đơn vị cắt (Table 4: % = tăng tốc; 60% = 34M, 95% = 5M tham số)

	QNLI 60%	QNLI 95%	MNLI 60%	MNLI 95%	SQuAD 60%	SQuAD 95%
CoFi	2.1× / 91.8	12.1× / 86.1	2.1× / 85.1	12.1× / 80.6	2.0× / 89.1 F1	8.7× / 82.6 F1
−hidden	2.2× / 91.3	13.3× / 85.6	2.1× / 85.2	13.7× / 79.8	2.0× / 88.7	9.7× / 80.8
−layer & hidden	2.2× / 91.3	7.2× / 84.6	2.1× / 84.8	7.0× / 78.4	2.1× / 88.5	6.4× / 74.1
−layer	2.1× / 91.5	8.3× / 86.7	2.1× / 85.4	8.4× / 80.6	2.0× / 89.1	7.9× / 80.5

- **Bỏ mask layer** (z_{MHA}, z_{FFN}): ở độ thưa 95% tăng tốc tụt mạnh (12.1× → 8.3× QNLI; 12.1× → 8.4× MNLI) mà độ chính xác gần như không đổi/hơi cao hơn ở QNLI; ở 60% không ảnh hưởng. Tức **cùng số tham số, cấu hình khác nhau cho tốc độ khác hẳn**.
- **Bỏ mask hidden** (z_{hidn}): mô hình nhanh hơn chút (cắt thêm layer) nhưng độ chính xác giảm trên mọi tập.
- Kết luận: đặt mask ở nhiều mức khuyến khích tối ưu cắt đúng loại đơn vị để tối đa hóa hiệu năng dưới ràng buộc độ thưa.

5.2 Mục tiêu distillation (Table 5, độ thưa 95%)

	SST-2	QNLI	MNLI	SQuAD
CoFi	90.6	86.1	80.6	82.6
$-\mathcal{L}_{layer}$	91.1	85.1	79.7	82.5
$-\mathcal{L}_{pred}, -\mathcal{L}_{layer}$ (không distill)	86.6	84.2	78.2	75.8
Fixed Hidden Distillation	90.0	85.8	80.5	80.9

- Bỏ distillation hoàn toàn: giảm 1.9 đến 6.8 điểm trên các tập.
- Ánh xạ layer **động** hơn ánh xạ cố định; bỏ $\mathcal{L}_{layer}$ hại mọi tập trừ SST-2. Ánh xạ động hội tụ về một liên kết ổn định, ví dụ QNLI: layer teacher 3, 6, 9, 12 khớp layer student 1, 2, 4, 9.
- Lợi ích của ánh xạ động rõ hơn ở độ thưa thấp (Appendix G.2).

5.3 Cấu trúc mô hình được cắt (chạy 3 lần mỗi cấu hình; 5 tập; độ thưa 60-95%)

1. **FFN bị cắt nặng hơn MHA** ở mọi độ thưa. Ở 60%: số neuron trung gian trung bình giảm **71% (3072 → 884)**, số head trung bình giảm **39% (12 → 7.3)**. Gợi ý FFN dư thừa hơn MHA.
2. **Cắt nhiều hơn ở layer trên**: layer MHA trên còn ít head hơn layer dưới.
3. Ở 95%: các mô hình còn **nhiều lớp MHA hơn FFN** (Table 6), gợi ý MHA quan trọng hơn cho tác vụ. Layer MHA đầu và thứ hai thường được giữ, layer giữa thường bị bỏ.
4. Cấu trúc **khác nhau theo dataset** (ví dụ SST-2, QNLI giữ MHA đầu, QQP và SQuAD bỏ), tức tồn tại mạng con tối ưu riêng cho mỗi tác vụ. Mô hình cắt có thể có FFN/MHA liền kề (như Press et al. 2020 gợi ý có thể có lợi).

5.4 So sánh với Movement Pruning (Figure 4, Appendix)

CoFi (cấu trúc) vẫn vượt movement pruning (không cấu trúc) trên MNLI và SQuAD ở nhiều kích thước; "CoFi Logit Distill" (chỉ distill logits như movement pruning) cũng được vẽ để tách ảnh hưởng của loss layerwise (các giá trị chi tiết nằm ở hình, không đọc được từ text).

6. Giới hạn

Do tác giả nêu:

- Phạm vi giới hạn ở **task-specific** pruning; áp cho mô hình task-agnostic (upstream pruning) là "hướng hứa hẹn" nhưng phức tạp ở các lựa chọn thiết kế.
- Chi phí huấn luyện của CoFi hơi dài hơn task-specific distillation của TinyBERT vì vừa tìm cấu trúc vừa học tham số.
- Phép so sánh với MobileBERT, AutoTinyBERT không trực tiếp (teacher khác).
- Tăng tốc phụ thuộc nền tảng (chỉ một V100); số có thể khác bản gốc của các baseline.

Nhận xét:

- Toàn bộ thí nghiệm là BERT-base trên GLUE/SQuAD: encoder-only, tác vụ phân loại/trích đoạn, đầu ra ngắn. Không có mô hình sinh (decoder), không có KV cache, không có đa phương thức.
- Mỗi tác vụ cần train riêng (20 GPU-giờ cho mỗi cặp tác vụ/độ thưa); chi phí nhân lên theo số cấu hình.
- Số liệu là một lần chạy cho Table 2 (cấu trúc thì chạy 3 lần), không có thanh sai số trên điểm số; chênh nhỏ như 86.1 vs 86.7 (QNLI) nên đọc thận trọng (checklist bài 12).
- $\mathcal{L}_{layer}$ chỉ tính ở các layer FFN và dùng W_{layer} cho mọi ánh xạ; chưa rõ đây có là lựa chọn tối ưu.

7. Liên hệ với các bài khác

- **13 Sixteen Heads, 14 Voita**: mask head; CoFi dùng đúng L_0 Hard Concrete của Voita nhưng mở rộng sang 5 loại đơn vị.
- **16 Movement pruning**: baseline không cấu trúc; CoFi chứng minh cấu trúc + distillation đạt tăng tốc thật (12.1× vs 1.0× ở số tham số tương đương).
- **15 LTH BERT**: nhắc lại như một nhánh unstructured khác.
- **04-05 Pruning filters, Network slimming**: tiền thân cho CNN (mask/scale theo channel).
- **11 Rethinking the value of pruning**, **03 Zhu & Gupta**: bài này tiếp tục "cắt từ mô hình lớn rồi tinh chỉnh" thay vì train mô hình nhỏ từ đầu.
- **18 2:4 sparsity**: hướng thưa có cấu trúc phần cứng, khác với cắt cấu trúc dense của CoFi.
- **25 ZipLM, 26 LLM-Pruner, 28 FLAP**: pruning cấu trúc cho LLM, cùng triết lý chọn đơn vị theo ngân sách latency.
- **38 Sheared LLaMA**: áp mask L_0 đa tầng (layer, head, FFN, hidden) lên LLaMA kèm dynamic batch loading; kế thừa trực tiếp CoFi.
- **39-42 (Minitron, Flextron, Puzzle)**: cắt cấu trúc rồi distillation ở quy mô LLM; cùng công thức "mask/ngân sách + distill".
- **12 State of pruning**: đánh giá CoFi theo checklist: nhiều điểm hoạt động (7 mức thưa), nhiều tác vụ, đo speedup thật (tốt), nhưng một kiến trúc và một nền tảng GPU.

8. Gợi ý áp dụng

- z_{int} cho neuron MLP;
- z_{hidn} cho chiều residual stream (chung mọi layer; cắt chiều ẩn đụng đến cả embedding, norm, và projector VLM nên khó hơn).

2. **Dùng ngân sách latency thay cho "độ thưa tham số"**: Table 4 là bằng chứng mạnh rằng *cùng số tham số, tốc độ khác nhau 7.2× đến 13.3×*. Có thể dùng $\mathcal{L}_{layer}$ với W_{layer} (khởi tạo đơn vị) để căn chỉnh

khác chiều (teacher d lớn hơn student: W_{layer} cần là ma trận chữ nhật, một điều chỉnh không có trong paper).

3. **Giai đoạn: distill trước → cắt dần tuyến tính → fine-tune mạng cố định** là công thức quen thuộc, và "finetuning the final subnetwork is essential for high sparsity" hợp với kế hoạch phục hồi bằng distillation.
4. **Quan sát cấu trúc**: FFN dư thừa nhiều hơn MHA, layer trên bị cắt nhiều hơn, mô hình cuối có nhiều MHA hơn FFN.

9. Thí nghiệm gợi ý

2. **Latency thay vì độ thưa**: đo bảng latency cho khối decoder theo (số head, số neuron MLP) trên GPU đích; thay $\hat{s}$ bằng latency kỳ vọng và xem cấu trúc cuối có đổi không (nhiều layer bị bỏ hơn?).

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / ý nghĩa
MHA có mask	$z_{MHA} \sum_i z_{head}^{(i)} \text{Att}_i$
FFN có mask	$z_{FFN}\ \text{gelu}(XW_U)\ \text{diag}(z_{int})\ W_D$
Mask hidden	$z_{hidn} \in \{0,1\}^d$, chung mọi layer, áp lên mọi ma trận
Hard Concrete	$s = \text{sigmoid}((\log \frac{u}{1-u} + \log \alpha)/\beta)$, $z = \min(1, \max(0, s(r-l)+l))$
Ràng buộc độ thưa	$\mathcal{L}_c = \lambda_1(\hat{s}-t) + \lambda_2(\hat{s}-t)^2$
Pred distillation	$D_{KL}(p_s \| p_t)$
Layer distillation	$\sum_{i\in\mathcal{T}} \text{MSE}(W_{layer} H_s^{m(i)}, H_t^i)$, $m(i) = \arg\min_{j:z_{FFN}^{(j)}>0} \text{MSE}(W_{layer} H_s^j, H_t^i)$
Tổng	$\lambda \mathcal{L}_{pred} + (1-\lambda)\mathcal{L}_{layer}$
Kết quả chính	MNLI: 2.7× / 84.9 (26M), 12.1× / 80.6 (4.4M); vs TinyBERT$_4$ 11.4× / 78.8
Chi phí	≤ 20 GPU-giờ (CoFi) vs ~350 (TinyBERT$_4$)

11. Kết luận ngắn

CoFi cho thấy cắt cấu trúc kết hợp mask ở nhiều mức hạt (layer, head, neuron FFN, chiều ẩn) cùng ràng buộc độ thưa Lagrangian và distillation layerwise động đạt >10× tăng tốc với độ chính xác ngang hoặc hơn distillation (TinyBERT$_4$), mà không cần dữ liệu không nhãn và chỉ ≤ 20 GPU-giờ. Cần thận trọng khi chuyển từ BERT-base/GLUE sang VLM sinh văn bản autoregressive.

Link gốc: https://arxiv.org/abs/2204.00408

Bài 18

Accelerating Sparse Deep Neural Networks (2021)

Tác giả: Asit Mishra, Jorge Albericio Latorre, Jeff Pool, Darko Stosic, Dusan Stosic, Ganesh Venkatesh, Chong Yu, Paulius Micikevicius (NVIDIA)
Venue / arXiv: arXiv:2104.08378 (v1, 16/04/2021); bài không nêu venue hội nghị.
Nguồn: https://arxiv.org/abs/2104.08378
Chủ đề: Sparsity có cấu trúc mịn 2:4, Sparse Tensor Core trên GPU Ampere, định dạng nén, quy trình "train - prune - retrain", hoán vị kênh (channel permutation).
Vị trí trong lộ trình: Đọc sau 02 (Han 2015: magnitude pruning + retrain), 03 (Zhu & Gupta: pruning dần) và 12 (Blalock: "State of pruning"). Bài này thêm góc nhìn **phần cứng**: sparsity chỉ có giá trị khi GPU thật sự chạy nhanh hơn. Tiếp theo: 19 (HALP, tối ưu latency thực), 22-23 (OBC/SparseGPT, nơi 2:4 được dùng cho LLM), 30 (MaskLLM, học mask 2:4).
Chương nền liên quan: nền 02: Ma trận thưa và chi phí thật, nền 04: Cắt hạt, cắt khối và mẫu 2, nền 07: Đo đúng một kết quả pruning

> Đây là ghi chú học tập bằng tiếng Việt, không phải bản dịch.

1. Bối cảnh

Hai trường phái pruning trước bài này đều có điểm yếu (theo bài báo):

- **Unstructured (không cấu trúc):** xoá từng trọng số riêng lẻ (Han 2015). Giữ được độ chính xác ở sparsity cao (cỡ 80-99%) nhưng ma trận thưa ngẫu nhiên khiến truy cập bộ nhớ không đều, cần chỉ số CSR/COO. Bài báo nói lợi ích tốc độ "negligible and at times negative", đặc biệt trên GPU/TPU có đơn vị nhân ma trận.
- **Structured (có cấu trúc, thô):** xoá cả filter/channel. Chạy nhanh trên phần cứng dense, nhưng mất độ chính xác khi sparsity cao; ví dụ paper trích: ResNet-50 channel pruning 2x speedup mất gần 1.5% độ chính xác.

Thêm nữa, các quy trình tinh chỉnh trước đó không thống nhất (số epoch, learning rate khác nhau từng mô hình). Câu hỏi của bài: có thể chọn **một mẫu thưa** vừa đủ "mịn" để giữ độ chính xác, vừa đủ "đều" để phần cứng tăng tốc, kèm **một quy trình** không cần dò hyper-parameter?

Giải thích thuật ngữ:

- **Sparsity:** tỉ lệ trọng số bằng 0.
- **GEMM** (General Matrix Multiply): phép nhân ma trận, lõi tính toán của conv, linear, attention projection.
- **Tensor Core:** đơn vị phần cứng của GPU NVIDIA chuyên làm nhân-cộng ma trận nhỏ (MMA) rất nhanh.
- **CSR/COO:** định dạng lưu ma trận thưa kèm chỉ số cột/hàng; tốn metadata và truy cập gián tiếp.

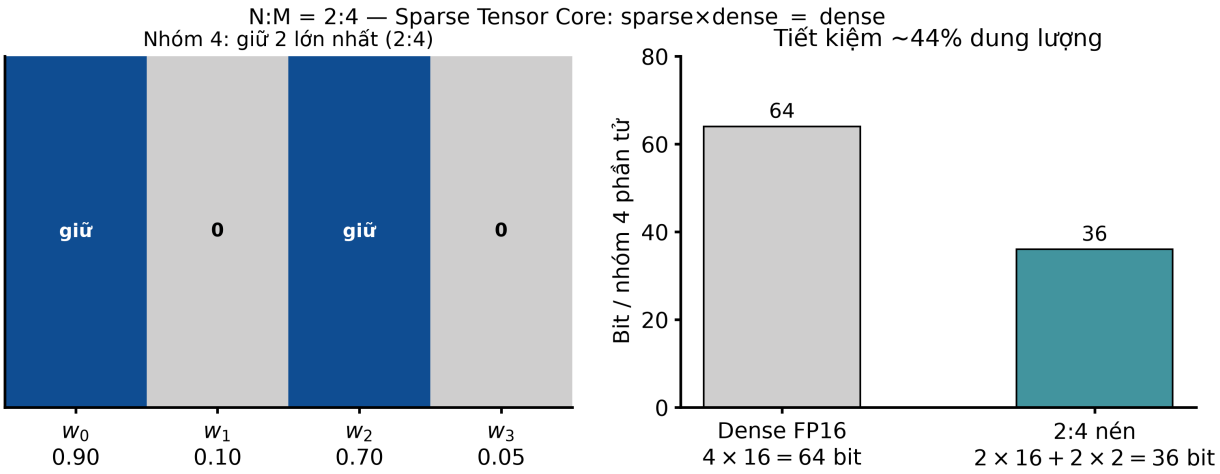
2. Ý tưởng chính

Phương pháp áp đặt ràng buộc **mỗi nhóm 4 giá trị liên tiếp có ít nhất 2 số 0** (mẫu "2:4", sparsity đúng 50%). Vì mẫu cố định theo khối, chỉ cần lưu 2 giá trị khác 0 cùng 2 bit chỉ số vị trí mỗi giá trị, và Sparse Tensor Core của Ampere chỉ chọn đúng các phần tử tương ứng của ma trận kia để nhân, nên số phép nhân-cộng giảm một nửa (tối đa 2x throughput). Quy trình rất đơn giản: huấn luyện dense bình thường, cắt 2 trọng số nhỏ nhất trong mỗi nhóm 4, rồi **lặp lại đúng lịch huấn luyện cũ** với mask cố định. Với các mạng rất nhỏ (MobileNet, EfficientNet...) thì hoán vị cột trước khi cắt để giữ được nhiều độ lớn trọng số hơn.

3. Phương pháp chi tiết

3.1 Mẫu 2:4 và định dạng nén

Với ma trận trọng số $W \in \mathbb{R}^{R \times C}$ (dùng làm toán hạng đầu của GEMM), chia mỗi hàng thành các nhóm 4 phần tử liên tiếp; ràng buộc



Mẫu 2:4 giữ 2/4 và định dạng nén FP16: 64 bit dense xuống 36 bit ($\approx 44\%$). (minh hoạ, tự tính)

$$\boxed{\forall \text{ nhóm } g = (w_{4k}, \ldots, w_{4k+3}) : \ \|g\|_0 \leq 2}$$

Ma trận nén có kích thước $R \times C/2$ cho giá trị, cộng $R \times C/2$ metadata 2 bit (vị trí 0-3 của mỗi giá trị trong nhóm). Nếu một nhóm có hơn 2 số 0, vẫn lưu đủ 2 giá trị để định dạng đồng nhất.

Số liệu lưu trữ (paper):

- Overhead metadata: 12.5% với giá trị 16-bit, 25% với 8-bit (so với tới 200% của CSR với giá trị 8-bit).
- 16-bit: dense 4 phần tử = 64 bit; 2:4 = $2 \cdot 16 + 2 \cdot 2 = 36$ bit, tiết kiệm khoảng 44%.
- 8-bit: tiết kiệm khoảng 38% dung lượng và băng thông.

Vì mật độ thưa đồng nhất ở mọi khối, đọc bộ nhớ dùng trọn cache line, và vị trí phần tử được suy ra trực tiếp từ tỉ lệ nén, không cần truy cập gián tiếp.

3.2 Sparse Tensor Core

Phép tính là **sparse × dense = dense**: toán hạng A (M×K) ở dạng 2:4 nén (M×K/2); phần cứng, nhờ metadata, chọn K/2 phần tử khớp của B dense (K×N) rồi nhân. B và C luôn dense. Bảng throughput đỉnh A100 trong paper (TOPS):

Đầu vào	Accumulator	Dense	Sparse
TF32	FP32	156	312
FP16 / BF16	FP32	312	624
FP16	FP16	312	624
INT8	INT32	624	1248

(FP32 thuần: 19.5 dense, không có sparse. Với TF32, mẫu thành 1:2.) Thực tế tốc độ phụ thuộc kích thước GEMM: GEMM lớn (arithmetic intensity cao) tiến gần 2x; Hình 3 đo INT8 cuSPARSELt so với cuBLAS với

M=N=10240, K thay đổi. Yêu cầu thư viện: chiều trong của ma trận thưa là bội số của 16 (FP16/BF16) hoặc 32 (INT8). Từ TensorRT 8.0 trọng số 2:4 được xử lý tự động.

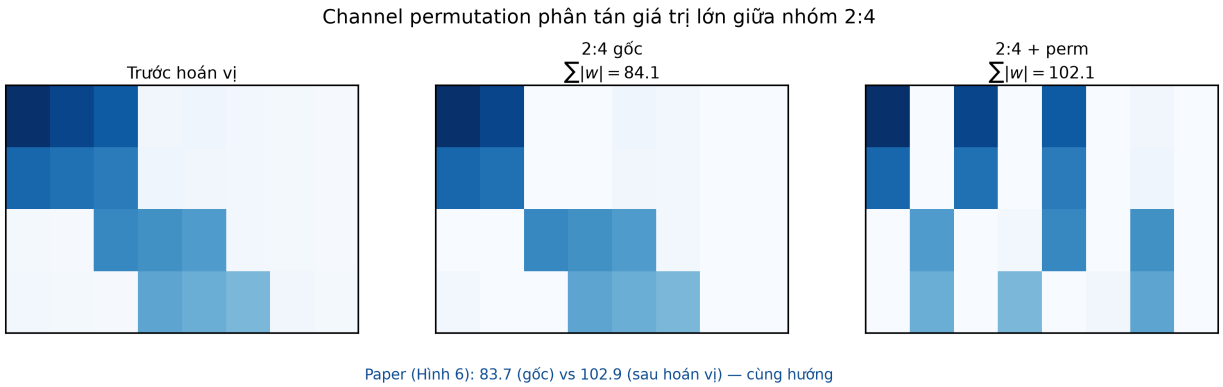
3.3 Quy trình 3 bước (ASP)

```
# Quy trình trong paper (thư viện PyTorch ASP)
1. Huấn luyện dense với lịch (optimizer, lr schedule, số epoch) S
2. Cắt: với mỗi tensor trọng số GEMM-like, trong mỗi nhóm 4,
       đặt 0 cho 2 trọng số có |w| nhỏ nhất  -> mask M cố định
       (tuỳ chọn: hoán vị kênh trước khi cắt)
3. Retrain: khởi tạo = trọng số sau bước 2,
       RESET hyper-parameter và optimizer state (momentum...),
       dùng đúng lịch S như bước 1,
       sau mỗi update: w <- w * M   # giữ nguyên số 0
4. (Nếu dùng INT8) hiệu chuẩn/fine-tune quantization SAU khi đã prune+retrain
```

Điểm chủ chốt: không dò hyper-parameter; đánh đổi chi phí huấn luyện gấp đôi để lấy tính tổng quát (chi phí được "trả dần" qua thời gian triển khai). Paper chỉ thấy tiêu chí magnitude là đủ, và nhắc có thể thử độ tương đồng activation hoặc gradient tức thời.

3.4 Hoán vị kênh (channel permutation)

Cắt 2:4 theo khối cố định có thể xoá nhầm giá trị lớn nếu hai giá trị lớn rơi cùng nhóm. Nếu hoán vị các **cột** của W trước khi cắt để phân bố đều giá trị lớn, tổng độ lớn giữ lại tăng. Ví dụ ma trận 4×8 trong Hình 6: tổng độ lớn sau cắt 83.7 (gốc) so với 102.9 (sau hoán vị).



Hoán vị cột trước cắt 2:4 tăng tổng $|w|$ giữ lại (cùng hướng paper $83.7 \rightarrow 102.9$). (minh hoạ, tự tính)

Để kết quả GEMM không đổi, hàng tương ứng của activation B cũng phải hoán vị cùng; điều này đạt được bằng cách hoán vị **hàng của trọng số tạo ra activation đó** (thường là layer trước). Với conv, hoán vị trên chiều kênh đầu vào. Không đổi phép tính, không overhead runtime. Paper nói thuật toán tìm hoán vị tốt "sẽ trình bày riêng", nên bài này không cho chi tiết.

3.5 Lớp nào được cắt, mô hình nhiều giai đoạn

- Chỉ cắt lớp có tham số học được và dẫn đến GEMM (conv, fully-connected, recurrent), với chiều trong là bội số của 16/32. Không cắt embedding (là bảng tra), không cắt các lớp chỉ dùng khi train (LM head của BERT, auxiliary classifier của Inception, discriminator của GAN). Conv đầu tiên 3 kênh (7×7) không cắt vì GEMM-K không phù hợp.
- **Mô hình huấn luyện nhiều giai đoạn:** (a) nếu giai đoạn sau dùng dataset nhỏ (BERT: pre-train lớn rồi fine-tune nhỏ), phải cắt và retrain ngay **sau pre-training** rồi mới fine-tune với mask giữ nguyên (Hình 7); (b) nếu giai đoạn sau đủ lớn (detection/segmentation trên COCO), chỉ cần cắt sau giai đoạn 2 và lặp lại giai đoạn 2 (Hình 8); head phải được fine-tune xong trước khi cắt.

- **Kết hợp quantization:** khuyến nghị hiệu chuẩn quantization sau khi đã prune và retrain.

4. Thí nghiệm và kết quả chính

Hyper-parameter lấy từ paper gốc/repo công khai của từng mạng; so sánh Dense FP16, Sparse FP16, Sparse INT8.

ImageNet top-1 (Bảng 2, trích):

Mạng	Dense FP16	Sparse FP16	Sparse INT8
ResNet-34	73.7	73.9	73.7
ResNet-50	76.1	76.2	76.2
ResNet-101	77.7	78.0	77.9
ResNeXt-101-32x16 (WSL)	84.2	84.0	84.2
DenseNet-121	75.5	75.3	75.3
Wide ResNet-50	78.5	78.6	78.5
Inception v3	77.1	77.1	77.1
VGG-16	74.0	74.1	74.1

(Bảng đầy đủ có 20 mạng; chênh lệch nằm trong biến thiên giữa các lần chạy theo nhận xét của tác giả.)

Mạng nhỏ, tham số hiệu quả (Bảng 3, FP16): mặc định vs hoán vị

Mạng	Dense	2:4 mặc định	2:4 hoán vị
MobileNet v2	71.55	69.56	71.56
SqueezeNet v1.0	58.09	54.08	58.38
SqueezeNet v1.1	58.21	56.96	58.23
MNASNet 1.0	73.24	71.99	73.27
ShuffleNet v2	68.32	66.97	68.42
EfficientNet B0	77.25	75.98	77.29
EfficientNet-WideSE B0	77.63	76.64	77.63

Detection / segmentation (COCO 2017, Bảng 4, trích): MaskRCNN-RN50 37.9 / 37.9 / 37.8; RetinaNet-RN50 34.8 / 35.2 / 35.1; FasterRCNN-RN50-FPN-3x 39.8 / 39.9 / 39.4; MaskRCNN-RN101-FPN-3x 42.9 / 43.2 / 42.8 (bbox AP, Dense FP16 / Sparse FP16 / Sparse INT8).

GAN (FID, thấp hơn là tốt, Bảng 5, dense vs 2:4): DCGAN-MNIST 50.39 vs 50.54; Pix2Pix 17.64 vs 17.89; CycleGAN Monet→Photo 63.15 vs 63.00; StarGAN-CelebA 6.11 vs 6.93; SRGAN-DIV2K 14.65 vs 16.60 (suy giảm lớn nhất trong bảng).

NLP:

Mô hình	Metric	Dense FP16	Sparse FP16	Dense INT8	Sparse INT8
GNMT (WMT'14 En-De)	BLEU	24.6	24.9	-	24.9 (INT8)
FairSeq Transformer	BLEU	28.2	28.5	-	28.3 (INT8)
Transformer-XL 12 lớp (enwik8)	BPC	1.06	1.06	-	-

Mô hình	Metric	Dense FP16	Sparse FP16	Dense INT8	Sparse INT8
BERT-large (SQuAD v1.1)	F1	91.9	91.9	90.9	90.8

(Với bảng 6, các cột là Dense FP16 / Sparse FP16 / Sparse INT8.) Với BERT và Transformer-XL: chỉ cắt các GEMM chứa trọng số trong khối transformer; lớp attention (QK^T, AV) không có trọng số nên không cắt.

Tốc độ: paper chỉ báo cáo throughput đỉnh (bảng trên) và một benchmark GEMM (Hình 3), không đo latency end-to-end của các mạng ở bảng accuracy.

5. Phân tích / ablation

- **Magnitude là đủ** cho đa số mạng; không cần tiêu chí phức tạp (theo paper, có thử các lựa chọn khác như gợi ý, nhưng không có bảng so sánh).
- **Hoán vị kênh** là yếu tố then chốt cho mạng nhỏ: mất 1-4 điểm accuracy ở bảng 3 được lấy lại gần như hoàn toàn.
- **Cắt ở giai đoạn nào** quan trọng: với BERT, chỉ cắt lúc fine-tune SQuAD không đủ số update để phục hồi nên phải lặp lại pre-training thưa.
- **Retrain schedule ngắn hơn:** thí nghiệm sơ bộ tìm được tập hyper-parameter ngắn hơn vẫn giữ accuracy, nhưng phụ thuộc mạnh vào mạng/tác vụ, không có công thức chung (không có số liệu trong bài).
- Phần cứng: lợi ích tốc độ phụ thuộc kích thước GEMM, nên mô hình hidden nhỏ hoặc batch nhỏ sẽ không đạt 2x.

6. Giới hạn

Tác giả nêu:

- Chi phí huấn luyện gấp đôi (train dense + retrain thưa); tìm lịch ngắn hơn là việc tương lai.
- Chỉ tăng tốc inference (forward). Để tăng tốc cả backward, ma trận chuyển vị cũng phải thoả 2:4, cần mask “transposable”.
- Chưa kiểm tra mô hình đã huấn luyện đến giới hạn dung lượng (dataset lớn/lịch dài).
- Attention (không có trọng số) chưa được cắt; thử nghiệm sơ bộ cắt activation 2:4 chưa có phương pháp tổng quát.
- Thuật toán tìm hoán vị chưa trình bày chi tiết.

Nhận xét:

- Sparsity cố định ở 50%: không cho phép chọn tùy ý mức độ thưa.
- Không giảm số tham số “thực” theo nghĩa kiến trúc: tensor vẫn nguyên hình dạng, chỉ nén lưu trữ; bộ nhớ activation và KV/state không đổi.
- Không có số đo end-to-end, nhất là với **batch 1** (lúc đó phép nhân là GEMV, bị giới hạn băng thông chứ không phải tính toán).
- Kết quả “không mất accuracy” trên CNN/BERT lớn không đảm bảo cho mô hình nhỏ đã được distill mạnh.

7. Liên hệ với các bài khác

- **02, 03:** cùng khung “train - prune - retrain”; bài này cố định mask một lần (one-shot) và dùng magnitude, khác pruning dần của 03.
- **12 (Blalock):** bài nêu rõ vấn đề so sánh công bằng; 2:4 là ví dụ về việc phải có lợi ích phần cứng đo được.

- **19 (HALP):** cùng coi tốc độ thật là mục tiêu, nhưng HALP làm structured pruning dựa trên bảng latency, còn 2:4 dựa vào hạ tầng phần cứng.
- **22-23 (OBC, SparseGPT):** dùng chính mẫu 2:4 (n:m) cho LLM và thay magnitude bằng cập nhật dựa trên Hessian.
- **24 (Wanda), 30 (MaskLLM):** tiêu chí khác / học mask cho 2:4.
- **44-47 (Nemotron, Minitron...):** các bài này thiên về structured pruning thật, 2:4 là hướng bổ sung độc lập.

8. Gợi ý áp dụng

- **Cảnh báo batch-1:** decoder sinh token là GEMV, chủ yếu bị giới hạn băng thông bộ nhớ; 2:4 giảm ~38-44% dung lượng/băng thông trọng số, đó có thể là lợi ích chính, chứ không phải 2x tính toán. Prefill của ViT/ảnh lớn mới hưởng 2x tính toán.

9. Thí nghiệm gợi ý

1. Lấy một block MLP của decoder (hoặc ViT), ép 2:4 bằng magnitude, đo lỗi đầu ra (MSE so với dense) có và không có hoán vị kênh; so với cắt 50% neuron trung gian để xem lỗi nào lớn hơn, rồi distill ngắn xem cái nào hồi phục tốt hơn.
2. Benchmark latency batch-1 thực trên GPU đích: Linear dense vs 2:4 (cuSPARSELt/TensorRT) cho kích thước hidden của mô hình; xác nhận lợi ích có đến từ băng thông hay không. ## 10. Tóm tắt công thức / khái niệm

Khái niệm	Nội dung
2:4 sparsity	Mỗi nhóm 4 giá trị liên tiếp có ≥ 2 số 0, sparsity 50%
Định dạng nén	$R \times C/2$ giá trị + 2 bit chỉ số/giá trị; overhead 12.5% (16b), 25% (8b)
Tiết kiệm lưu trữ	~44% (16-bit: 36 vs 64 bit/nhóm), ~38% (8-bit)
Tăng tốc phần cứng	Tối đa 2x throughput GEMM (A100: FP16 312 → 624 TOPS)
Quy trình	Train dense → cắt 2 nhỏ nhất/nhóm → retrain cùng lịch, mask cố định
Reset khi retrain	Hyper-parameter và optimizer state được reset
Channel permutation	Hoán vị cột W (và hàng của layer trước) để giữ tổng độ lớn; 83.7 → 102.9 trong ví dụ
Không cắt	Embedding, layer chỉ dùng khi train, conv 3 kênh đầu, chiều không là bội 16/32
Quantization	Hiệu chuẩn sau khi prune + retrain

11. Kết luận ngắn

Đóng góp chính của bài là kỹ thuật + hệ thống: một mẫu thưa vừa đủ mịn để giữ độ chính xác trên rất nhiều mạng, vừa đủ đều để Ampere nhân nhanh gấp đôi, cùng quy trình train-prune-retrain "không dò tham số".

Link gốc: https://arxiv.org/abs/2104.08378

Bài 19

HALP: Hardware-Aware Latency Pruning (2021)

Tác giả: Maying Shen, Hongxu Yin, Pavlo Molchanov, Lei Mao, Jianna Liu, Jose M. Alvarez (NVIDIA)
Venue / arXiv: arXiv:2110.10811 (v1, 20/10/2021); bản text không nêu venue.
Nguồn: https://arxiv.org/abs/2110.10811
Chủ đề: Structured pruning (filter/channel) với ràng buộc **latency** đo thật trên GPU, bảng tra latency (lookup table), bài toán knapsack, nhóm neuron theo bậc thang latency.
Vị trí trong lộ trình: Đọc sau 04 (Pruning filters), 05 (Network slimming), 07 (Molchanov importance 2019, tiêu chí Taylor mà HALP dùng) và 18 (2:4, tốc độ phần cứng). Bài này thêm: biến "mục tiêu nén" từ số tham số / FLOPs thành **latency đo trên đúng GPU đích**, và coi chọn số kênh mỗi lớp là bài toán phân bổ tài nguyên toàn cục. Tiếp theo: 20 (DepGraph, cách nhóm các tham số phụ thuộc), 25 (ZipLM, latency-aware cho LLM), 42 (Puzzle, chọn kiến trúc theo ràng buộc phần cứng).
Chương nền liên quan: nền 02: Ma trận thưa và chi phí thật, nền 04: Cắt hạt, cắt khối và mẫu 2

> Đây là ghi chú học tập bằng tiếng Việt, không phải bản dịch. Paper chỉ thử trên CNN (ResNet, MobileNet, VGG, SSD), không có transformer/LLM.

1. Bối cảnh

Các phương pháp structured pruning trước đó thường nhắm vào **số tham số** hoặc **FLOPs** (số phép nhân-cộng). Bài báo nhắc rằng FLOPs là proxy kém: hai mạng có FLOPs gần nhau có thể có latency khác nhau rất nhiều. Lý do phần cứng: trên GPU, latency của một lớp conv theo số kênh có dạng **bậc thang** (staircase): giảm kênh trong một bậc không làm nhanh hơn, chỉ khi vượt qua một "vách" (latency cliff) mới giảm. Hệ quả: nếu cắt từng kênh lẻ, bạn mất độ chính xác mà không được lợi tốc độ; phải cắt **theo nhóm** khớp với bậc.

Cách đọc hình: đường xanh là latency mô phỏng của một lớp, chỉ giảm khi số kênh vượt qua một vách. Cắt 20 kênh từ 128 vẫn nằm trong bậc cũ nên không nhanh hơn dù mất độ chính xác. Bậc rộng 32 là giả định của hình, không phải số đo của paper.

Thêm nữa, mỗi lớp có "giá" khác nhau: bỏ kênh ở lớp đầu (độ phân giải cao) thường tốn độ chính xác hơn và cũng giảm latency nhiều hơn lớp sau. Nên "cắt nhiều ở lớp sau hay ít ở lớp đầu để cùng một độ chính xác-latency" là bài toán tổ hợp cần lời giải toàn cục.

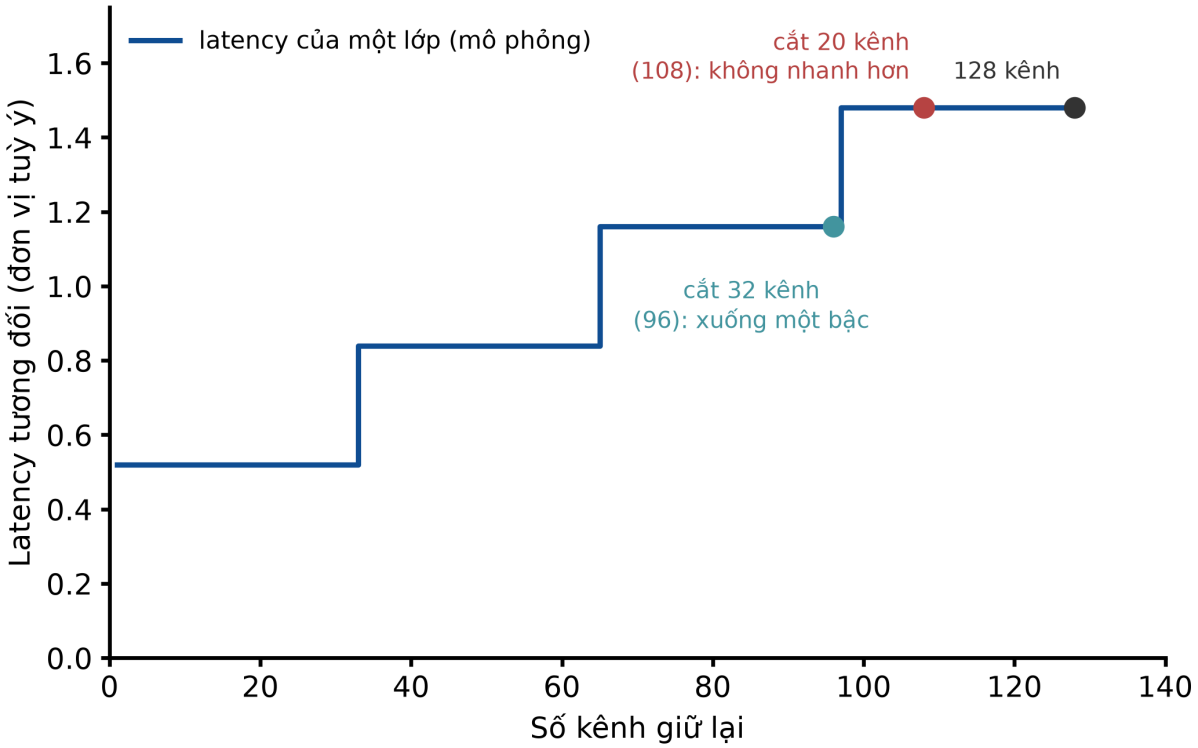
Các hướng latency-aware trước đó (theo bài báo): NAS (đắt), NetAdapt, AutoSlim, EagleEye (sinh nhiều ứng viên rồi đánh giá, tốn nhiều giờ GPU), MetaPruning/AMC (phải huấn luyện mạng phụ trợ).

Giải thích thuật ngữ:

- **Latency lookup table (LUT):** bảng đo sẵn thời gian chạy của từng lớp ứng với từng cấu hình số kênh vào/ra trên GPU đích.
- **Knapsack (bài toán cái túi):** chọn tập vật phẩm có giá trị tổng lớn nhất mà tổng "trọng lượng" không vượt sức chứa; ở đây giá trị = importance, trọng lượng = latency, sức chứa = ngân sách latency.
- **Taylor importance:** ước lượng loss tăng bao nhiêu nếu bỏ một neuron bằng khai triển Taylor bậc 1 (xem 07).
- **Neuron / filter:** một kênh đầu ra của lớp conv.

2. Ý tưởng chính

Xem việc cắt tỉa toàn mạng là bài toán "xếp đồ vào cái túi": mỗi nhóm neuron có một **giá trị** (độ quan trọng với độ chính xác, từ gradient) và một **giá** (latency nó đóng góp, tra từ bảng đo trên GPU thật). Cần chọn tập



Hình 43: Latency theo số kênh có dạng bậc thang; ví dụ đồ chơi với bậc rộng 32 kênh (đơn vị latency tuỳ ý) (minh hoạ, tự tính)

nhóm giữ lại sao cho tổng giá trị lớn nhất mà tổng latency ≤ ngân sách. Điểm phải xử lý: giá của một neuron chỉ đúng khi các neuron quan trọng hơn trong cùng lớp đều được giữ (giữ theo thứ tự quan trọng), nên cần một knapsack "tăng cường" có ràng buộc thứ tự. Và để khớp với bậc thang latency, các neuron được gom thành nhóm có kích thước bằng độ rộng bậc.

3. Phương pháp chi tiết

3.1 Bài toán tối ưu

Mạng có L lớp, lớp l có N_l neuron. Bài toán chung:

$$\arg\min_{\hat{W}} \ \mathcal{L}(\hat{W}, D) \quad \text{s.t. } \Phi(f(\hat{W}, x_i)) \le C$$

với Φ là ràng buộc (latency, FLOPs, bộ nhớ); paper tập trung vào latency. Dạng được giải:

$$\max_{p_1,\ldots,p_L} \sum_{l=1}^{L} I_l(p_l) \quad \text{s.t.} \sum_{l=1}^{L} T_l(p_{l-1}, p_l) \le C, \ \ 0 \le p_l \le N_l$$

p_l: số neuron giữ ở lớp l (p_0 cố định, vd. 3 kênh RGB); $I_l(p_l)$: độ quan trọng tối đa với p_l neuron; $T_l(p_{l-1}, p_l)$: latency lớp l khi có p_{l-1} kênh vào, p_l kênh ra (tra bảng).

3.2 Importance

Prune trên lớp BatchNorm; importance của neuron n ở lớp l (Phương trình 3):

$$I_l^n = \left| g_{\gamma_l^n} \gamma_l^n + g_{\beta_l^n} \beta_l^n \right|$$

(g là gradient của tham số BN γ, β.) Paper nói khác với Molchanov 2019 ở chỗ dùng **giá trị tuyệt đối** thay vì bình phương, vì "slight improvements". Sắp xếp giảm dần trong từng lớp và cộng dồn:

$$I_l(p_l) = \sum_{j=1}^{p_l} I_l^j, \quad I_l^1 \geq \cdots \geq I_l^{N_l}.$$

Điểm trung bình qua nhiều minibatch được dùng khi xếp hạng.

3.3 Đóng góp latency của từng neuron

Latency lớp là tổng đóng góp neuron:

$$T_l(p_{l-1}, p_l) = \sum_{j=1}^{p_l} c_l^j, \qquad c_l^j = T_l(p_{l-1}, j) - T_l(p_{l-1}, j-1).$$

Vì số kênh vào p_{l-1} thay đổi khi lớp trước bị cắt, đóng góp được **cập nhật động** trong quá trình prune. Một số c_l^j có thể âm (do bảng đo nhiễu / phi đơn điệu) và được xử lý riêng trong phần cài đặt (Appendix F); chi tiết xử lý không nằm trong đoạn đã đọc kỹ ở đây.

3.4 Knapsack tăng cường (augmented)

Bài toán thành 0/1 knapsack với giá trị I, trọng lượng c, sức chứa C, nhưng neuron hạng j chỉ có latency c_l^j nếu mọi neuron hạng $1..j-1$ trong lớp được giữ. Do đó thêm ràng buộc "tiền nhiệm" (preceding): chỉ cho giữ neuron n nếu neuron đứng ngay trước nó trong cùng lớp cũng được giữ.

```
# Augmented Knapsack Solver (Algorithm 1, rút gọn)
Input: I[1..N] (importance, sắp giảm dần trong mỗi lớp),
       c[1..N] (latency integer), C (ngân sách), r[n] = chỉ số neuron tiền nhiệm
dp = zeros(C+1);  keep = zeros(N, C+1)
for n in 1..N:
    new = copy(dp)
    for cap in c[n]..C:
        v_keep  = I[n] + dp[cap - c[n]]
        v_prune = dp[cap]
        if v_keep > v_prune and keep[r[n], cap - c[n]]:   # tiền nhiệm phải được giữ
            keep[n, cap] = True;  new[cap] = v_keep
        else:
            keep[n, cap] = False; new[cap] = v_prune
    dp = new
# truy vết ngược từ n = N về 1: nếu keep[n, C] thì chọn n, C -= c[n]
```

Latency phải đổi sang số nguyên (nhân 1000 rồi làm tròn) để làm chỉ số mảng (Appendix F).

3.5 Gom nhóm neuron theo bậc latency

Độ rộng bậc thang (khoảng cách số kênh giữa hai "vách") gọi là **latency step size**; mỗi lớp gom s kênh liền nhau (theo thứ tự quan trọng) thành một nhóm, s = step size. Lợi ích kép: (i) chỉ cắt theo bậc nên cắt nào cũng có lợi tốc độ; (ii) giảm số "vật phẩm" N cho knapsack, ví dụ ResNet50 từ 26,560 neuron xuống 215 nhóm. Với skip connection (ResNet) và group conv (MobileNet) thì các lớp ràng buộc nhau cùng chỉ số kênh được gom chung, dùng group size lớn nhất trong số đó.

3.6 Quy trình HALP đầy đủ

```
Input: mạng pretrained, ngân sách latency C, LUT
Lịch: k = 30 mốc, C^1 > C^2 > ... > C^k = C   (giảm theo hàm mũ)
Mỗi r = 320 minibatch:
  1. train bình thường 1 minibatch, tích luỹ importance (Eq.3)
  2. trung bình importance, xếp hạng trong từng lớp,
     cập nhật latency contribution c theo số kênh đang còn (Eq.6)
  3. gom nhóm theo step size, tính I và c của nhóm, tìm nhóm tiền nhiệm
  4. chạy augmented knapsack với C^t -> chọn nhóm giữ
Sau khi đạt C: fine-tune (90 epoch, batch 128/GPU, warmup 8 epoch rồi cosine)
```

Cài đặt: 8 GPU V100 để prune/fine-tune; LUT đo trên NVIDIA Titan V (batch 256) hoặc Jetson TX2 (batch 32), mỗi phép đo trung bình 100 lần; cuDNN 7.6.5.

4. Thí nghiệm và kết quả chính

Mặc định: Titan V, đo latency ở **batch 256**; baseline các phương pháp khác được dựng lại từ cấu trúc đã công bố rồi đo trên cùng GPU. “HALP-X%” nghĩa là giữ X% latency.

ResNet50 / ImageNet (Bảng 1, trích; FPS trên Titan V):

Phương pháp	FLOPs (G)	Top-1 (%)	FPS	Speedup
Không prune	4.1	76.2	1019	1x
AutoSlim	3.0	76.0	1215	1.14x
GReg-1	2.7	76.3	1171	1.15x
HALP-80%	3.1	77.2	1256	1.23x
AutoSlim	2.0	75.6	1592	1.56x
MetaPruning	2.0	75.4	1604	1.58x
HALP-55%	2.0	76.5	1630	1.60x
AutoSlim	1.0	74.0	2390	2.45x
MetaPruning	1.0	73.4	2381	2.34x
HALP-30%	1.0	74.3	2755	2.70x

Với baseline của EagleEye (dense 77.2): EagleEye-2G 76.4 / 1.44x vs HALP-55% 76.6 / 1.64x; EagleEye-1G 74.2 / 2.38x vs HALP-30% 74.5 / 2.55x.

Cách đọc hình: điểm càng cao và càng sang phải càng tốt. Ba điểm HALP tạo thành đường nằm trên các điểm của AutoSlim và MetaPruning ở tốc độ tương đương. Điểm không prune (1019 FPS, 76,2%) làm mốc.

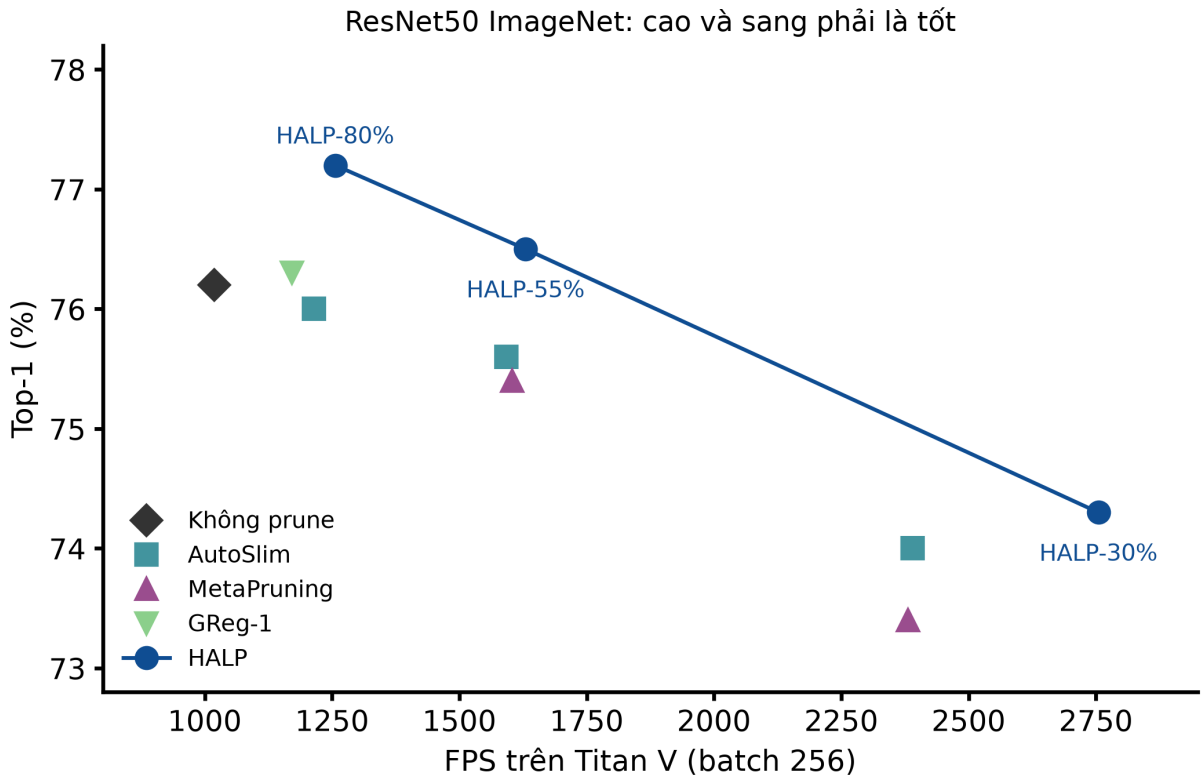
ResNet101: không prune 77.4% / 620 FPS; HALP-60% 78.3% (1.37x); HALP-50% 77.8% (1.60x); HALP-40% 77.2% (1.90x); HALP-30% 76.5% (2.45x). So sánh Taylor-75% 77.4% (1.21x) và Taylor-55% 76.0% (1.47x).

MobileNetV1: không prune 72.6% / 3415 FPS; HALP-42% 68.3% (2.32x) so với AutoSlim 67.9% (2.27x); HALP-60% 71.3% (1.68x) so với EagleEye 70.9% (1.47x).

VGG16: không prune 71.59% / 766 FPS; HALP-30% 72.33% (2.42x); HALP-20% 70.76% (5.49x).

Tóm tắt abstract: ResNet-50/-101 throughput x1.60 / x1.90 với thay đổi top-1 +0.3% / -0.2%; SSD trên VOC x1.94 với giảm 0.56 mAP.

Kết hợp distillation: dùng teacher RegNetY-16GF (top-1 82.9%), mô hình ResNet50 đã prune nhanh 2.70x với giảm 1% accuracy (Hình 2).



Hình 44: Top-1 theo FPS của ResNet50 trên ImageNet (Titan V, batch 256) cho HALP và các phương pháp so sánh (Bảng 1) (vẽ lại từ số liệu trong ghi chú)

TensorRT (Bảng 2, tăng tốc so với dense cùng định dạng, Titan V / RTX3080): HALP-55%: FP32 1.24x, FP16 4.70x (Titan V); trên RTX3080 FP32 1.18x, FP16 3.32x, INT8 6.40x (acc drop 1.02%). HALP-30%: FP32 2.91x, FP16 9.61x; RTX3080 FP32 2.56x, FP16 6.44x, INT8 14.12x (acc drop 2.55%). (Bảng trong text bị lệch dòng; các số trên đọc theo cặp EagleEye/HALP nhưng cột accuracy drop có thể lẫn, nên đáng kiểm lại bản gốc nếu cần dùng.)

Chi phí thiết kế (Bảng 3): chọn sub-network của EagleEye ~30 giờ GPU, NetAdapt 864 giờ GPU, ThiNet ~8000 giờ GPU; HALP ~30 phút **CPU** (30 bước prune, mỗi bước ~1 phút giải knapsack) và dưới 1 phút cho MobileNetV1. Không cần train mạng phụ trợ, không cần đánh giá nhiều ứng viên. Chi phí dựng LUT chỉ làm một lần cho mọi tỉ lệ nén.

Detection (SSD512-ResNet50, VOC07+12): base 77.98 mAP, 65.56 GFLOPs, 68.24 FPS; HALP: 77.42 mAP, 15.38 GFLOPs, 132.57 FPS (khoảng 77% FLOPs giảm, 1.94x). Chỉ prune backbone.

Batch size 1 (Appendix E, Bảng 5, Titan V): không prune 76.2% / 181 FPS. Các phương pháp khác chỉ nhanh 1.00-1.14x (GReg-2 ở 0.50x: 73.9%, 1.14x). HALP-90%: 76.4%, 220 FPS (1.22x); HALP-80%: 75.3%, 247 FPS (1.37x).

5. Phân tích / ablation

- **Kích thước nhóm (Hình 4, ResNet50):** nhóm quá nhỏ (vd. 2) kém nhất độc lập với ngân sách vì giữ/bỏ những nhóm không đem lại lợi latency mà vẫn gây hại; nhóm quá lớn (256) cũng kém vì importance bị trung bình hoá, kém phân biệt và dễ xoá hẳn một lớp trong một bước. Latency-aware grouping (LG) tốt nhất. Với ResNet50, 23 lớp dùng nhóm 32, 20 lớp nhóm 64, 10 lớp nhóm 128; lớp gần đầu vào nhóm nhỏ hơn.

- **FLOPs không tương đương latency:** HALP đôi khi để lại nhiều FLOPs hơn đối thủ nhưng chạy nhanh hơn (vd. batch 1: HALP-80% còn 2.3 GFLOPs nhưng nhanh hơn nhiều so với các mạng 1.0-1.3 GFLOPs).
- **Cấu trúc sau prune** (Hình 8 và phụ lục) được phân tích phụ lục, không có số liệu trong phần đọc chính.

6. Giới hạn

Tác giả nêu/ngầm nêu:

- Latency table phải đo lại cho mỗi GPU/batch size/độ chính xác số học; INT8 TensorRT LUT là "future work".
- Ở batch 1, bậc thang gần như biến mất, nên phải dùng nhóm nhỏ (kích thước 2) và lợi ích giảm (nhưng vẫn hơn các baseline).
- Code "sẽ release khi được chấp nhận" (phiên bản này).

Nhận xét:

- Chỉ CNN và detector; chưa có transformer, attention, MLP gated, linear attention hay mô hình sinh. Importance dựa vào tham số BatchNorm (không có trong LLM/transformer dùng LayerNorm/RMSNorm).
- Latency được cộng độc lập theo lớp (tổng của T_l), bỏ qua tương tác giữa lớp (fusion, bộ nhớ, kernel launch); với mô hình nhỏ chạy batch 1, overhead launch có thể áp đảo.
- Importance bậc nhất (gradient) ước lượng trên minibatch, có thể nhiễu và nhạy với dữ liệu.
- Không có so sánh với quá trình train-từ-đầu cấu trúc đã prune (xem 11).

Trong Phụ lục E, ở batch 1, các phương pháp giảm FLOPs chỉ đạt mức tăng tốc khoảng 1.00–1.14 lần; kết quả này nhấn mạnh sự cần thiết của phép đo độ trễ trực tiếp. Bài báo cũng báo cáo HALP kết hợp chưng cất với teacher RegNetY-16GF đạt tăng tốc 2.70 lần với mức giảm độ chính xác 1%.

7. Liên hệ với các bài khác

- **07 (Molchanov 2019):** HALP kế thừa tiêu chí Taylor và lịch prune dần; thêm latency và knapsack.
- **04, 05 (Pruning filters, Slimming):** những phương pháp prune cục bộ theo tỉ lệ cố định mỗi lớp; HALP phân bổ toàn cục.
- **18 (2:4):** cùng quan tâm tốc độ phần cứng thật; 2:4 dùng mẫu thưa vi cấu trúc, HALP đổi kích thước lớp.
- **20 (DepGraph):** giải bài toán "cắt cái này thì phải cắt cái kia" cho kiến trúc tuỳ ý; HALP xử lý skip connection bằng cách gom nhóm cùng chỉ số kênh (cách đơn giản).
- **25 (ZipLM), 42 (Puzzle), 41 (Flextron):** các bài sau cho LLM cũng chọn cấu hình theo ràng buộc latency/phần cứng.
- **43 (Pruning vs scratch), 11 (Rethinking):** câu hỏi liệu cấu trúc tìm được có giá trị hơn trọng số.

8. Gợi ý áp dụng

- **Nhóm theo bậc phần cứng:** khi cắt MLP width, làm tròn về bội số của 64/128 (bậc thang GEMM và Tensor Core), thay vì kênh lẻ.
- Không áp dụng trực tiếp: importance dựa trên BN, quy trình 90 epoch fine-tune ImageNet.

9. Thí nghiệm gợi ý

1. Đo latency batch-1 trên GPU đích của một khối MLP và của một block đầy đủ khi width giảm dần (ví dụ 100% → 10%, bước 64): có bậc thang không? Giảm width 50% có giảm latency khoảng 50% không? Trả lời câu hỏi "tại sao cắt neuron kém". ## 10. Tóm tắt công thức / khái niệm

Khái niệm	Nội dung
Bài toán	max $\sum_l I_l(p_l)$ s.t. $\sum_l T_l(p_{l-1}, p_l) \leq C$
Importance	$I_l^n = \lvert g_\gamma \gamma + g_\beta \beta \rvert$ (Taylor bậc 1 trên BN), cộng dồn theo thứ hạng
Latency neuron	$c_l^j = T_l(p_{l-1}, j) - T_l(p_{l-1}, j-1)$, cập nhật khi mạng thay đổi
Knapsack tăng cường	0/1 knapsack + ràng buộc: giữ neuron n chỉ khi tiền nhiệm trong lớp được giữ
Latency step size	Độ rộng bậc thang latency; kích thước nhóm s bằng nó
Lợi ích gom nhóm	ResNet50: 26,560 neuron → 215 nhóm; giải knapsack ~1 phút/bước
Lịch	30 bước, mỗi 320 minibatch, ngân sách giảm hàm mũ; sau đó fine-tune 90 epoch
Chi phí	~30 phút CPU (so với 30 giờ GPU của EagleEye)
Kết quả nổi bật	ResNet50 HALP-30%: 74.3% top-1, 2.70x; ResNet101 HALP-50%: 77.8%, 1.60x; batch 1 HALP-80%: 75.3%, 1.37x

11. Kết luận ngắn

HALP biến structured pruning thành bài toán chọn cấu hình dưới ngân sách **latency thực** bằng bảng tra và knapsack, kèm nhóm neuron khớp bậc thang GPU.

Link gốc: https://arxiv.org/abs/2110.10811

Bài 20

DepGraph: Towards Any Structural Pruning (2023)

Tác giả: Gongfan Fang, Xinyin Ma, Mingli Song, Michael Bi Mi, Xinchao Wang (National University of Singapore; Zhejiang University; Huawei)
Venue / arXiv: arXiv:2301.12900 (v2, 23/03/2023); bản text không nêu venue. Code: Torch-Pruning (github.com/VainF/Torch-Pruning).
Nguồn: https://arxiv.org/abs/2301.12900
Chủ đề: Structured pruning cho kiến trúc bất kỳ; mô hình hoá **phụ thuộc giữa các lớp** (dependency graph); gom tham số thành nhóm; sparse training ở mức nhóm.
Vị trí trong lộ trình: Đọc sau 04 (Pruning filters, nơi đã có ví dụ "cắt filter lớp này thì phải cắt kernel lớp sau"), 05 (Network slimming), 19 (HALP). Bài này thêm: một **thuật toán tự động** tìm "cắt cái này thì phải cắt cái nào khác" cho mọi kiến trúc, nền tảng kỹ thuật cho các bài structured pruning LLM sau này. Tiếp theo: 26 (LLM-Pruner dùng ý tưởng dependency group cho LLM), 29 (SliceGPT), 38 (Sheared LLaMA).
Chương nền liên quan: nền 04: Cắt hạt, cắt khối và mẫu 2

> Đây là ghi chú học tập bằng tiếng Việt, không phải bản dịch. Paper tự nói mục tiêu **không** phải đạt SOTA mà là chứng minh tính tổng quát; tiêu chí importance dùng ở đây rất đơn giản (norm).

1. Bối cảnh

Structural pruning (cắt có cấu trúc) xoá hẳn một nhóm tham số (kênh, neuron, head...) để mạng thật sự nhỏ đi và chạy nhanh trên phần cứng thường, trái với **unstructured pruning** chỉ đặt từng trọng số về 0 (cần accelerator đặc biệt để hưởng lợi).

Khó khăn: các lớp trong mạng bị **ràng buộc** với nhau. Ví dụ trong CNN (Hình 1a của bài báo): muốn xoá một kênh của Conv2 trong khối residual, bạn phải xoá đồng thời BN2, kernel tương ứng ở lớp sau, và vì phép cộng residual yêu cầu hai nhánh cùng số kênh, cả Conv1, BN1 cũng phải cắt cùng chỉ số kênh. Tương tự trong Transformer (head của attention, LayerNorm, MLP), RNN (cổng LSTM dùng phép nhân phần tử), GNN.

Trước bài này, các pruner phải **thiết kế tay** quy tắc nhóm cho từng kiến trúc; không chuyển được sang kiến trúc mới, thậm chí kiến trúc khác trong cùng họ. Vấn đề thứ hai: các tiêu chí importance cũ (norm, Taylor) tính **trong một lớp**; khi nhiều lớp bị cắt cùng nhau, điểm số từng lớp không cộng được vì phân phối và độ lớn khác nhau.

Giải thích thuật ngữ:

- **Coupled / dependent parameters:** các tham số phải cắt cùng nhau, nếu không mạng "gãy" (shape không khớp).
- **Transitive reduction:** rút gọn đồ thị có hướng, giữ ít cạnh nhất mà vẫn bảo toàn quan hệ "tới được".
- **Connected component:** tập đỉnh nối được với nhau trong đồ thị.
- **Group sparsity / sparse training:** thêm phạt regularization khiến cả nhóm tham số về gần 0 cùng lúc.

2. Ý tưởng chính

Phương pháp xây dựng một **đồ thị phụ thuộc** thay cho việc liệt kê thủ công các quan hệ (Dependency Graph) chỉ ghi các phụ thuộc **cục bộ** giữa lớp kề nhau (đầu ra lớp này nối đầu vào lớp kia, hoặc lớp có đầu vào và đầu ra chung số kênh như BatchNorm). Sau đó, **nhóm** gồm các thành phần liên thông của đồ thị này, tìm bằng duyệt đồ thị trong thời gian tuyến tính. Mỗi nhóm được cắt cùng nhau. Vì các phần tử trong nhóm phải đều "không quan trọng", paper huấn luyện thưa ở mức nhóm (phạt cùng chỉ số kênh trên mọi lớp trong nhóm) để norm của cả nhóm tách bạch, rồi cắt nhóm có norm nhỏ.

3. Phương pháp chi tiết

3.1 Ví dụ phụ thuộc cơ bản

Xét ba lớp fully-connected liên tiếp w^l, w^{l+1}, w^{l+2}. Để cắt neuron thứ k nối w^l và w^{l+1} phải xoá cả hàng $w^l[k,:]$ và cột $w^{l+1}[:,k]$. Paper phân ra bốn kiểu phụ thuộc (Hình 2): cơ bản (a), residual (b: hai lớp cộng với nhau cùng chỉ số kênh), concatenation (c: kênh nối nhau, chỉ số bị dịch), reduction (d: phép gộp/tổng). Các kiểu này có thể lồng nhau nên không thể phân tích tay.

3.2 Ma trận nhóm G và đồ thị phụ thuộc D

Muốn tìm ma trận nhóm $G \in \mathbb{R}^{L\times L}$ với $G_{ij} = 1$ nếu lớp i, j phụ thuộc nhau; nhóm của lớp i:

$$g(i) = \{j \mid G_{ij} = 1\}.$$

Nhưng G khó ước lượng trực tiếp vì quan hệ giữa i, j bị ảnh hưởng bởi các lớp ở giữa. Nên paper xây D là **transitive reduction** của G: chỉ ghi phụ thuộc giữa các lớp kề trực tiếp; $G_{ij} = 1$ khi và chỉ khi có đường đi từ i đến j trong D. (Ví dụ $w_1 \Leftrightarrow w_2$, $w_2 \Leftrightarrow w_3$ suy ra $w_1 \Leftrightarrow w_3$ nên không cần lưu cạnh thứ ba.)

3.3 Phân rã mạng thành đầu vào và đầu ra

Vì cùng một lớp có thể có hai kiểu cắt khác nhau (FC có thể cắt hàng $w[k,:]$ hay cột $w[:,k]$) và có phép toán không tham số (cộng skip), paper phân rã mạng thành các thành phần f_i (lớp có tham số hoặc phép không tham số), mỗi thành phần tách thành đầu vào f_i^- và đầu ra f_i^+: $F = \{f_1^-, f_1^+, \ldots, f_L^-, f_L^+\}$.

Hai loại phụ thuộc:

- **Inter-layer:** $f_i^- \Leftrightarrow f_j^+$ nếu hai lớp nối nhau ($f_i^- \leftrightarrow f_j^+$), vì đó chính là cùng một feature trung gian.
- **Intra-layer:** $f_i^- \Leftrightarrow f_i^+$ nếu đầu vào và đầu ra có cùng "scheme" cắt, $sch(f_i^-) = sch(f_i^+)$. BatchNorm thoả (kênh vào = kênh ra); conv thì không ($w[:,k,:,:] \neq w[k,:,:,:]$) nên không có phụ thuộc nội lớp.

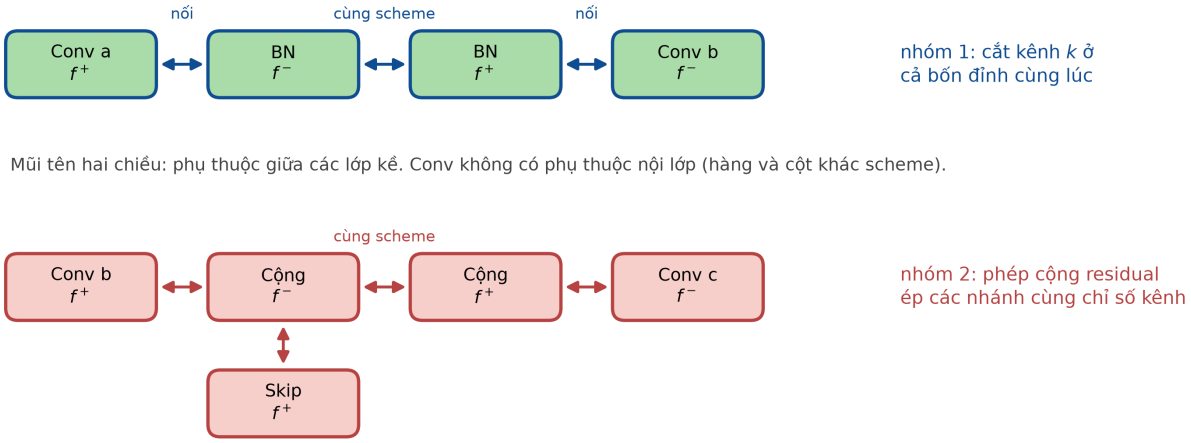
$$\boxed{D(f_i^-, f_j^+) = \mathbb{1}[f_i^- \leftrightarrow f_j^+] \ \vee \ \mathbb{1}[i = j \ \wedge \ sch(f_i^-) = sch(f_j^+)]}$$

D đối xứng.

```
# Algorithm 1: xây DepGraph
phân rã mạng thành f^-_1..f^-_L, f^+_1..f^+_L
D = zeros(2L, 2L)
for i, j:
    D[f_i^-, f_j^+] = D[f_j^+, f_i^-] =
        (f_i^- nối với f_j^+)  OR  (i == j AND sch(f_i^-) == sch(f_j^+))

# Algorithm 2: gom nhóm (thành phần liên thông)
G = {}
for mỗi đỉnh i:
    g = {i}
    repeat:
        g' = {j chưa thấy | tồn tại k in g, D[k,j] = 1}
        g = g U g'
    until g' rỗng
    G = G U {g}
```

Độ phức tạp tuyến tính theo số đỉnh (duyệt đồ thị). Ví dụ: Hình 3 trình bày lan truyền cắt từ f_4^+ trong khối CNN có residual.



Hình 45: Đồ thị phụ thuộc giữa đầu vào f^- và đầu ra f^+ của các thành phần, và hai nhóm hình thành từ các thành phần liên thông (minh hoạ, tự tính)

Cách đọc hình: mỗi ô là một đỉnh, mũi tên hai chiều là cạnh phụ thuộc giữa hai đỉnh kề. Nhóm là tập đỉnh nối được với nhau, nên cắt một kênh ở đỉnh nào cũng kéo theo cùng chỉ số ở mọi đỉnh trong nhóm. Nhóm 2 cho thấy phép cộng residual ép các nhánh cùng chỉ số kênh.

3.4 Group-level pruning (cắt ở mức nhóm)

Với nhóm $g = \{w_1, \ldots, w_{|g|}\}$, tiêu chí norm cho từng tham số $I(w) = \|w\|_2$ (kiểu Li et al., 04) có thể cộng thành $I(g) = \sum_{w \in g} I(w)$. Nhưng điểm của các lớp độc lập không cộng được vì lệch phân phối. Giải pháp: **sparse training nhất quán theo nhóm**. Với mỗi chiều có thể cắt k (K chiều) của nhóm g:

$$I_{g,k} = \sum_{w \in g} \|w[k]\|_2^2, \qquad \boxed{R(g,k) = \sum_{k=1}^{K} \gamma_k \, I_{g,k}}$$

hệ số co γ_k tăng theo độ "không quan trọng" tương đối:

$$\gamma_k = 2^{\alpha \, (I_g^{\max} - I_{g,k}) / (I_g^{\max} - I_g^{\min})}, \qquad \alpha = 4$$

nên $\gamma_k \in [2^0, 2^\alpha]$ (phạt mạnh nhất cho chiều ít quan trọng nhất). Sau sparse training, dùng điểm tương đối chuẩn hoá $\hat{I}_{g,k} = N \cdot I_{g,k} / \sum \text{TopN}(I_g)$ (trong văn bản gốc công thức này bị mangled; ý là chuẩn hoá bởi tổng N giá trị lớn nhất trong nhóm) để so sánh giữa các nhóm và cắt phần nhỏ nhất. Sau đó fine-tune bằng quy trình giống pretrain, với learning rate nhỏ hơn và ít iteration hơn.

Paper cho ba chiến lược sparse (Hình 4): (a) unstructured, (b) cấu trúc nhưng **không nhất quán** (mỗi lớp độc lập), (c) **nhất quán theo nhóm** (cái họ đề xuất): ép mọi tham số cùng chỉ số trong nhóm về 0 cùng nhau.

Hai kiểu độ thưa theo lớp: **uniform** (cùng tỉ lệ mỗi lớp) và **learned** (học từ ngưỡng toàn cục dựa trên importance chuẩn hoá).

4. Thí nghiệm và kết quả chính

Dữ liệu: CIFAR-10/100, ImageNet-1k, PPI (đồ thị), ModelNet40 (point cloud), AGNews (văn bản). Mô hình: ResNe(X)t, VGG, DenseNet, MobileNet, GoogleNet, ViT, LSTM, DGCNN, GAT. Speed-up đo bằng tỉ

lệ MACs (theo lý thuyết), không phải latency.

CIFAR (Bảng 1, trích):

Mô hình / dữ liệu	Phương pháp	Base	Pruned	ΔAcc	Speed-up
ResNet56 / CIFAR-10	HRank	93.26	92.17	-0.09	2.00x
	ResRep	93.71	93.71	+0.00	2.12x
	Ours w/o SL	93.53	93.46	-0.07	2.11x
	Ours	93.53	93.77	+0.24	2.11x
	GReg-2	93.36	93.36	-0.00	2.55x
	Ours w/o SL	93.53	93.36	-0.17	2.51x
	Ours	93.53	93.64	+0.11	2.57x
VGG19 / CIFAR-100	GReg-1	74.02	67.55	-6.67	8.84x
	GReg-2	74.02	67.75	-6.27	8.84x
	Ours w/o SL	73.50	67.60	-5.44	8.87x
	Ours	73.50	70.39	-3.11	8.92x

(“SL” = sparse learning.) ResNet-56 đạt 2.57x với 93.64% (cao hơn mô hình gốc 93.53%).

ImageNet (Bảng 3, trích; cột MACs theo G, chỉ lấy các số đọc được rõ):

Mạng	Phương pháp	Base	Pruned	ΔAcc	MACs (G)
ResNet-50	FPGM	76.15	75.50	-0.65	2.38
	Taylor	76.18	74.50	-1.68	2.25
	GFP	76.79	76.42	-0.37	2.04
	GReg-2	76.13	75.36	-0.77	2.77
	Ours	76.15	75.83	-0.32	1.99
DenseNet-121	PSP-1.38G	74.35	74.05	-0.30	1.38
	Ours-1.38G	74.44	73.98	-0.46	1.37
	Ours-0.58G	74.44	70.13	-4.31	0.57
MobileNet-v2	Ours	71.87	68.46	-3.41	0.15
ResNeXt-50	GFP	77.97	77.53	-0.44	2.11
	Ours	77.62	76.48	-1.14	2.09
ViT-B/16	CP-ViT	77.91	77.36	-0.55	11.7
	Ours	81.07	79.17	-1.90	10.4
	Ours+EMA	81.07	79.58	-1.39	10.4

(ViT-B/16 gốc 17.6 GMACs. Bảng này lưu ý: DepGraph không luôn tốt nhất; ví dụ GFP trên ResNeXt-50 mất ít hơn; ResNet-50 là điểm sáng “hơn 2x speed-up, mất 0.32%”.)

Dữ liệu phi ảnh (Bảng 4, trích):

Mạng / dữ liệu	Phương pháp	Base	Pruned	Δ	Speed-up
LSTM / AGNews	Ours	92.10	91.75	-0.35	16.28x
DGCNN / ModelNet40	Ours	92.10	92.02	-0.08	11.98x

Mạng / dữ liệu	Phương pháp	Base	Pruned	Δ	Speed-up
GAT / PPI (micro-F1)	Ours	0.986	0.961	-0.025	8.43x

Các baseline "DepGraph+Random / +CP / +Slim" cho thấy ngay cả pruner khác chạy trên nhóm của DepGraph vẫn chạy được (ví dụ LSTM: Random 91.23, CP 91.50).

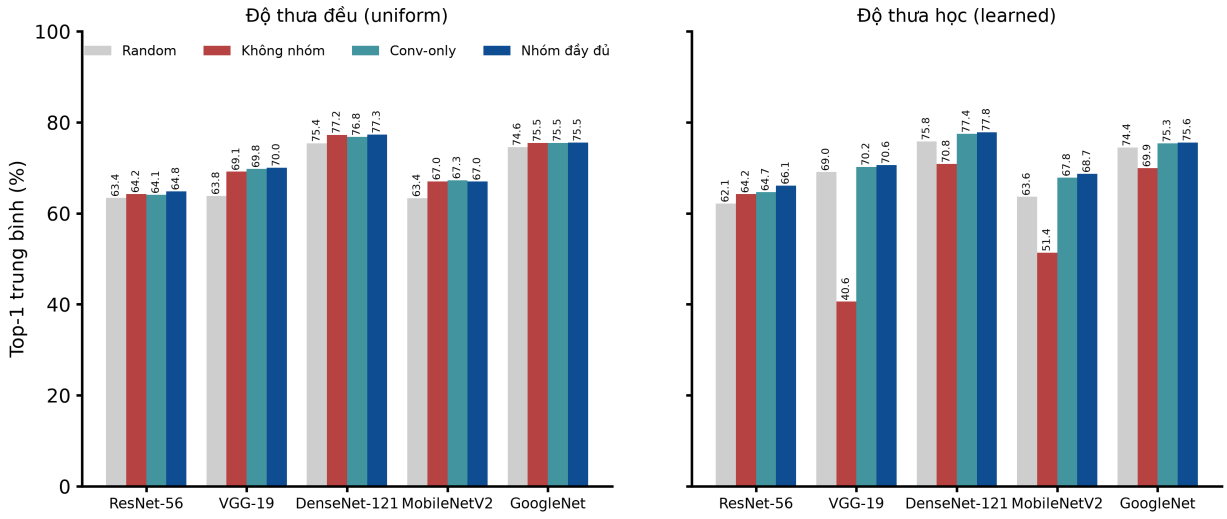
5. Phân tích / ablation

Chiến lược nhóm (Bảng 2, CIFAR-100, acc trung bình sau prune ở 1.5x/3x/6x/12x; trích cột Avg. "uniform / learned"):

Mạng (acc gốc)	Random	No grouping	Conv-only	Full grouping
ResNet-56 (72.58)	63.39 / 62.15	64.24 / 64.20	64.07 / 64.65	64.83 / 66.09
VGG-19 (73.50)	63.83 / 69.05	69.14 / 40.58	69.76 / 70.21	70.03 / 70.58
DenseNet-121 (78.73)	75.41 / 75.80	77.24 / 70.82	76.77 / 77.44	77.29 / 77.77
MobileNetV2 (70.80)	63.35 / 63.62	67.03 / 51.37	67.27 / 67.85	66.99 / 68.67
GoogleNet (77.56)	74.57 / 74.44	75.45 / 69.92	75.49 / 75.35	75.53 / 75.57

Kết luận paper rút ra:

- **Không nhóm (sparsify từng lớp riêng)** là chiến lược dễ sụp đổ với learned sparsity: VGG-19 ở 12x xuống 3.87% vì "over-prune một chiều xuống 1" (ghi chú dấu † trong bảng); trung bình 40.58.
- **Conv-only** tốt hơn, **Full grouping** (cả BN và FC cùng nhất quán) thường tốt nhất.
- **Learned sparsity** thường hơn uniform, nhưng đôi khi cắt quá tay một lớp.
- **Hình 5:** sparse training nhất quán cho phân bố norm nhóm tách bạch (nhiều nhóm gần 0), còn không nhất quán thì norm nhóm không thưa rõ.
- "Ours w/o SL" (không sparse learning) ở Bảng 1 thấp hơn "Ours" ở mọi dòng, ví dụ VGG19: 67.60 vs 70.39, cho thấy sparse training có ích thực sự.



Hình 46: Độ chính xác trung bình sau prune theo chiến lược nhóm, với độ thưa đều và độ thưa học (Bảng 2, CIFAR-100) (vẽ lại từ số liệu trong ghi chú)

Cách đọc hình: mỗi cụm là một mạng, bốn cột là bốn chiến lược nhóm. Nhóm đầy đủ thường cao nhất hoặc gần cao nhất. Không nhóm với độ thưa học sụp đổ ở VGG-19 (40,58) và MobileNetV2 (51,37) do cắt quá tay một chiều.

6. Giới hạn

Tác giả nêu:

- Mục tiêu là tính tổng quát, **không phải SOTA**; chỉ dùng tiêu chí norm đơn giản.
- Bài nhấn mạnh phân loại; chưa phải generation/detection.

Nhận xét:

- Speed-up báo cáo là tỉ lệ MACs lý thuyết, không đo latency; HALP (19) cho thấy hai thứ này có thể lệch nhiều.
- Phần ViT: baseline và mức MACs khác nhau giữa các phương pháp nên so sánh với CP-ViT không hoàn toàn cùng điều kiện; ma trận phụ thuộc cho LLM hiện đại (RMSNorm, RoPE, GQA, gated MLP, linear attention) không được kiểm tra.
- Cần model trace được thành đồ thị tính toán; các mô-đun có luồng điều khiển phức tạp (điều kiện, vòng lặp, cache) có thể khó.
- Sparse training thêm chi phí huấn luyện; đã thử trên dataset nhỏ-vừa, chưa trên pretrain quy mô lớn.
- Bảng nhiều dòng bị mangled trong text trích xuất; một số cột (ví dụ cột tham số/MACs) tôi chỉ chép khi đọc rõ.

7. Liên hệ với các bài khác

- **04 (Pruning filters):** DepGraph dùng tiêu chí norm của Li et al. và tổng quát hoá phần "cắt filter ở lớp trước thì cắt kernel lớp sau".
- **05 (Network slimming):** cùng ý "sparse training trên scale rồi cắt"; DepGraph áp dụng nhất quán qua nhóm.
- **19 (HALP):** HALP xử lý skip/group conv bằng cách gom chỉ số kênh; DepGraph là cách tổng quát và tự động. Hai bài bổ sung nhau: DepGraph nói **cái gì phải cắt cùng**, HALP nói **cắt bao nhiêu để đạt latency**.
- **26 (LLM-Pruner):** áp dụng trực tiếp ý tưởng dependency group lên LLM (cắt couple structure trong Llama).
- **13, 14 (Sixteen heads, Voita):** cắt head attention, một dạng nhóm.
- **29 (SliceGPT), 38 (Sheared LLaMA), 39 (Minitron):** các cách cắt hidden dimension / head / MLP đều phải giải quyết phụ thuộc kiểu residual mà DepGraph hình thức hoá.

8. Gợi ý áp dụng

- Việc cắt "hidden size" hoặc số head/kênh bị ràng buộc bởi residual stream: cắt một chiều của residual phải cắt đồng bộ ở mọi lớp đọc/ghi nó (embed, norm, mọi projection). Một đồ thị phụ thuộc giúp **kiểm tra tính nhất quán** và tự động hoá việc này.
- **Cắt MLP neuron (nhóm đơn giản: hàng của up/gate và cột của down)** là trường hợp không cần DepGraph; còn cắt **layer** hoàn toàn cũng không cần. Giá trị lớn của DepGraph nằm ở các cắt ngang residual (hidden dim) và head/state.
- **Ý tưởng group-level sparse training:** nếu tín hiệu importance cho từng neuron nhiễu, ép sparsity nhất quán trước khi cắt (thay vì cắt tức thời bằng xếp hạng) có thể giảm sốc. Chưa có bằng chứng trong paper cho LLM/VLM.

9. Thí nghiệm gợi ý

2. Trên một ViT hoặc MLP nhỏ: so sánh cắt neuron 30% bằng (a) xếp hạng norm tức thời và (b) sparse training nhất quán theo nhóm ngắn rồi cắt, đo loss trước/sau distill ngắn. ## 10. Tóm tắt công thức / khái niệm

Khái niệm	Nội dung
Nhóm phụ thuộc	$g(i) = \{j \mid G_{ij} = 1\}$, các lớp phải cắt cùng nhau
DepGraph D	Transitive reduction của G; chỉ ghi phụ thuộc giữa lớp kề
Phân rã	Mỗi thành phần f_i tách thành đầu vào f_i^- và đầu ra f_i^+
Inter-layer dep	$f_i^- \Leftrightarrow f_j^+$ nếu nối nhau
Intra-layer dep	$f_i^- \Leftrightarrow f_i^+$ nếu cùng scheme cắt (BN có, conv không)
Gom nhóm	Thành phần liên thông trên D, độ phức tạp tuyến tính
Phạt nhóm	$R(g,k) = \sum_k \gamma_k \sum_{w \in g} \|w[k]\|_2^2$
Hệ số co	$\gamma_k = 2^{\alpha(I^{\max} - I_{g,k})/(I^{\max} - I^{\min})}$, $\alpha = 4$
Kết quả nổi bật	ResNet-56 CIFAR-10: 2.57x, 93.64%; ResNet-50 ImageNet: 75.83% (-0.32), ~1.99 GMACs; ViT-B/16: 79.17% (-1.90)

11. Kết luận ngắn

DepGraph biến “cắt kênh nào cùng kênh nào” từ công việc thủ công theo từng kiến trúc thành một bài toán đồ thị tự động, kèm sparse training nhất quán ở mức nhóm.

Link gốc: https://arxiv.org/abs/2301.12900

Bài 21

A Survey on Model Compression for Large Language Models (2023)

Tác giả: Xunyu Zhu, Jian Li, Yong Liu, Can Ma, Weiping Wang (Institute of Information Engineering, CAS; UCAS; Renmin University of China)
Venue / arXiv: arXiv:2308.07633 (bản v4, 30/07/2024, định dạng ACL-style). Bản text không ghi tên venue.
Nguồn: https://arxiv.org/abs/2308.07633
Chủ đề: Survey nén LLM: quantization, pruning (unstructured / structured / semi-structured), knowledge distillation (black-box / white-box), low-rank factorization; metric, benchmark, hướng phát triển. **Ghi chú đọc:** bài này dùng làm **bản đồ lĩnh vực**; ghi chú tập trung vào pruning, còn quantization / distillation / low-rank chỉ đặt vào vị trí của chúng.
Vị trí trong lộ trình: Đọc sau 12 (Blalock, State of pruning) và 18 (2:4). Bài này không có ý tưởng mới mà là **mục lục có chú thích** cho giai đoạn LLM: nó nhắc đến SparseGPT (23), Wanda (24), LLM-Pruner (26), FLAP (28), SliceGPT (29), Shortened LLaMA (33), Sheared LLaMA (38) trước khi bạn đọc từng bài. Đọc nhanh bây giờ, quay lại tra khi cần.
Chương nền liên quan: nền 01: Vì sao mạng cắt được, nền 07: Đo đúng một kết quả pruning

> Đây là ghi chú học tập bằng tiếng Việt, không phải bản dịch. Quan trọng: survey này ra từ 2023-2024 nên **không** phủ lớp phương pháp 2024-2025 mà lộ trình này nhắm tới (layer pruning như ShortGPT/SLEB, Minitron, Puzzle, mô hình lai Mamba/linear attention, VLM). Các số trong bảng của survey được lấy "solely from the original papers", chạy trên các mô hình và điều kiện khác nhau nên **không so sánh trực tiếp** được.

1. Bối cảnh

LLM quá lớn để triển khai: ví dụ trong survey, GPT-175B cần tối thiểu 350 GB bộ nhớ ở FP16 và ít nhất 5 GPU A100 80 GB để chạy suy luận. Nén mô hình (model compression) là hướng chính. Survey tự nhận là survey đầu tiên chuyên về nén **LLM**; trước đó có survey về nén mạng nơ-ron nói chung hoặc chỉ nhắc nhẹ trong survey LM.

Điểm khác biệt khi nén LLM so với CNN (nền tảng của các bài 02-12): (i) không thể huấn luyện lại đầy đủ vì hàng chục đến hàng trăm tỉ tham số, nên hầu hết phương pháp là **post-training** (một lần, với một ít dữ liệu hiệu chuẩn); (ii) có hiện tượng outlier trong activation làm quantization khó; (iii) đánh giá bằng perplexity, zero-shot, benchmark suy luận.

Giải thích thuật ngữ:

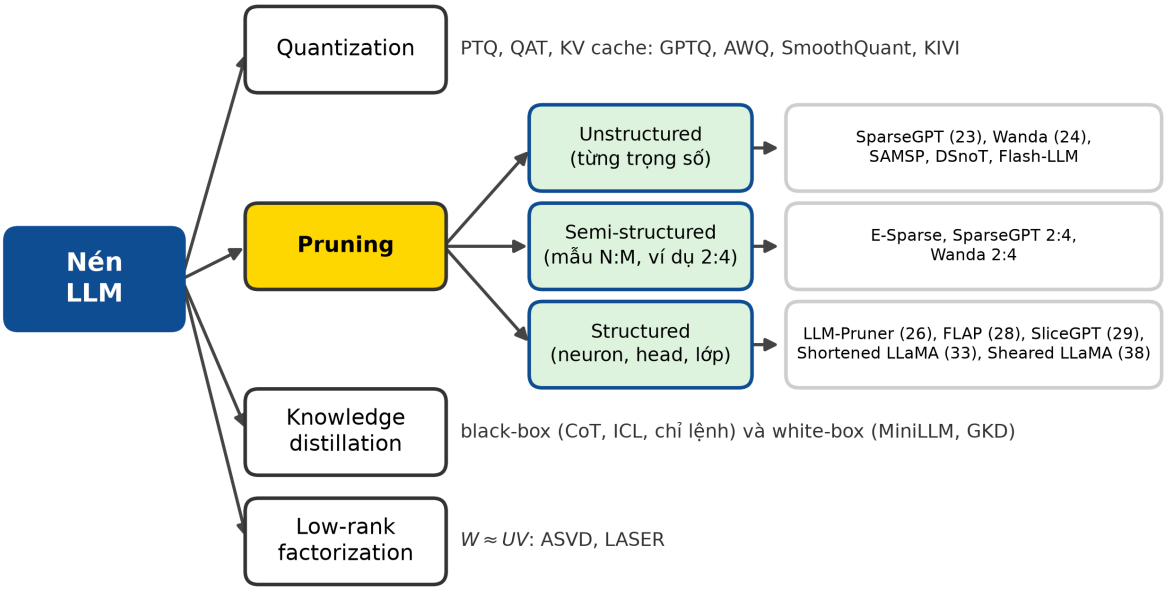
- **PTQ (post-training quantization):** lượng tử hoá sau khi huấn luyện, không huấn luyện lại; **QAT** huấn luyện lại có mô phỏng lượng tử.
- **Calibration data (dữ liệu hiệu chuẩn):** một ít mẫu chạy qua mô hình để đo activation/Hessian dùng cho nén.
- **KV cache:** bộ nhớ lưu key/value của attention cho các token đã sinh.
- **Black-box / white-box KD:** teacher chỉ cho đầu ra (API) hay cho cả logit/tham số.

2. Ý tưởng chính (của survey)

Survey chia nén LLM thành bốn họ (Hình 1): **quantization** (giảm số bit), **pruning** (bỏ tham số/cấu trúc), **knowledge distillation** (học từ teacher lớn), **low-rank factorization** (phân tích $W \approx UV$). Phần pruning lại chia theo **độ mịn cấu trúc**: unstructured, structured, semi-structured (N:M). Tác giả nhấn mạnh quantization

và KD có kiểu "task-based" (QAT, KD cần dữ liệu tác vụ) còn lại là "task-agnostic" (Hình 2), và gợi ý nên **kết hợp** (đặc biệt pruning cấu trúc + KD).

3. Bản đồ phương pháp



Hình 47: Bản đồ nén LLM theo survey: bốn họ kỹ thuật, ba mức độ mịn của pruning và các phương pháp được nhắc trong ghi chú (minh hoạ, tự tính)

Cách đọc hình: đi từ ô "Nén LLM" sang bốn họ, rồi từ ô pruning (tô vàng) sang ba mức độ mịn cấu trúc. Cột phải liệt kê phương pháp tương ứng và số trong ngoặc là bài của lộ trình. Cần nhớ rằng trục phân loại này là độ mịn, chưa nói gì về chuyện có phải huấn luyện lại hay không.

3.1 Metric và benchmark (Mục 2)

- Metric kích thước/tính toán: model size (số tham số), FLOPs, **MFU** (Mean FLOPS Utilization: FLOPS thực dùng / FLOPS lý thuyết của thiết bị), inference time (latency), speedup ratio (thời gian gốc / thời gian đã nén), compression ratio (kích thước gốc / sau nén).
- Benchmark: perplexity trên WikiText-2, C4, PTB; zero-shot LAMBADA, PIQA, OpenBookQA; suy luận GSM8K, CommonsenseQA, StrategyQA; BIG-Bench (BBH, hơn 200 tác vụ); bộ chỉ lệnh chưa thấy (Vicuna-Instructions, User-Oriented-Instructions); khung EleutherAI LM Harness (hơn 60 benchmark học thuật).

3.2 Pruning (Mục 4), phần trọng tâm

Survey định nghĩa: **structured** xoá cả neuron/head/layer, giữ nguyên hình dạng chung; **unstructured** xoá từng tham số, tạo cấu trúc thưa bất quy tắc; **semi-structured** xoá theo mẫu cố định (N:M) rồi gọi là "fine-grained structured".

Unstructured (4.1): thường không cần retrain vì giữ chất lượng tốt, nhưng cần xử lý đặc biệt để tăng tốc.

- **SparseGPT** (→ 23): prune một lần không retrain; coi pruning như bài toán hồi quy thưa lớn giải bằng solver xấp xỉ; đạt trên 50% sparsity ở OPT-175B, BLOOM-176B với perplexity tăng ít.

- **Wanda** (→ 24): giữ trọng số theo |w| nhân norm của activation đầu vào; không cần cập nhật trọng số hay retrain.
- **SAMSP:** dùng Hessian để đo độ nhạy từng ma trận và phân bổ sparsity động.
- **DSnoT:** lặp "prune-and-grow" để giảm sai số tái tạo giữa dense và sparse, nhất là sparsity cao.
- **Flash-LLM:** hỗ trợ phần cứng: nạp ma trận ở dạng thưa từ global memory, dựng lại dạng dense trong bộ đệm trên chip để dùng tensor core.

Structured (4.2): không phụ thuộc phần cứng đặc biệt nhưng dễ mất chất lượng hơn, thường phải fine-tune hiệu quả (LoRA). Survey chia theo tiêu chí chọn:

- **Loss-based** (Molchanov 2019, → 07): **LLM-Pruner** (→ 26) phát hiện cấu trúc phụ thuộc rồi chọn nhóm bằng gradient, task-agnostic; **Shortened LLaMA** (→ 33) cắt cả block Transformer (depth pruning) dựa trên loss và đạo hàm bậc hai; cả hai dùng LoRA phục hồi.
- **Magnitude-based: FLAP** (→ 28) dùng "fluctuation metric" đo độ biến thiên của từng đặc trưng đầu vào để chọn cột bỏ, tìm cấu trúc thích nghi toàn cục, bù bias nên không cần fine-tune; **SliceGPT** (→ 29) dùng tính bất biến tính toán của Transformer và PCA ở mỗi lớp để chiếu rồi xoá hàng/cột.
- **Regularization-based: Sheared LLaMA** (→ 38) dùng cặp nhân tử Lagrange trên mask để ràng buộc trực tiếp hình dạng mô hình, kèm dynamic batch loading theo tốc độ giảm loss của từng miền dữ liệu.

Semi-structured (4.3): N:M (mỗi M phần tử liên tiếp giữ N khác 0). **E-Sparse** dùng entropy thông tin làm tiêu chí và shuffle để giảm ảnh hưởng. SparseGPT chia khối M trọng số, chọn N để cắt theo sai số tái tạo (Hessian); Wanda chia nhóm M, giữ N trọng số điểm cao nhất. Survey nhắc A100 chỉ hỗ trợ tăng tốc tỉ lệ 2:4 (→ 18).

3.3 Quantization (Mục 3) — chỉ đặt vị trí

- **QAT:** LLM-QAT (distill từ dữ liệu do chính LLM sinh), BitDistiller (QAT + self-distillation), OneBit (biểu diễn 1-bit); kết hợp PEFT: QLoRA, PEQA, LoftQ, L4Q.
- **PTQ weight-only:** GPTQ (dựa trên OBQ của bài OBC, → 22; cập nhật trọng số bằng nghịch đảo Hessian), AWQ (giữ 1% trọng số quan trọng + scale theo kênh), QuIP (LDL + ma trận trực giao ngẫu nhiên, 2-bit), SpQR, SqueezeLLM (k-means), OWQ, LUT-GEMM.
- **PTQ weight-activation:** ZeroQuant, LLM.int8() (giữ chiều outlier ở độ chính xác cao), SmoothQuant (scale theo kênh để làm mịn outlier), RPTQ, OliVe, OS+, LLM-FP4, OmniQuant.
- **KV cache:** KVQuant, KIVI (key theo kênh, value theo token, 2-bit không fine-tune), WKVQuant.
- Nhận xét trong survey: weight-only đạt bit thấp hơn nhưng phải dequantize trước khi nhân nên không hưởng tăng tốc thấp-bit của phần cứng; activation nhạy hơn weight.

3.4 Knowledge distillation (Mục 5) — chỉ đặt vị trí

- **Black-box KD:** teacher (ChatGPT, GPT-4) chỉ cho văn bản; chia theo năng lực nổi lên cần chuyển: Chain-of-Thought (Distilling Step-by-Step, MT-CoT, Fine-tune-CoT, SCOTT, PaD, Socratic CoT...), In-Context Learning (ICL distillation, AICD), Instruction Following (Lion, LaMini-LM, Self-Instruct, Selective Reflection-Tuning).
- **White-box KD:** có phân phối/tham số teacher. MiniLLM (reverse KL, tránh student ước lượng quá cao vùng xác suất thấp của teacher), GKD (student học trên đầu ra tự sinh có phản hồi của teacher, loss linh hoạt), TED (lọc theo tác vụ để căn chỉnh biểu diễn ẩn từng lớp). Survey ghi nhận white-box tới nay bị hạn chế vì LLM mã nguồn mở yếu hơn đóng.

3.5 Low-rank factorization (Mục 6) — chỉ đặt vị trí

$W \approx UV$ với $U \in \mathbb{R}^{m\times k}$, $V \in \mathbb{R}^{k\times n}$, $k \ll m, n$. LPLR (phân rã hạng thấp + độ chính xác thấp ngẫu nhiên), ASVD (scale trọng số theo phân phối activation, phân bổ hạng khác nhau theo lớp), LASER/LSAER (giảm có chọn lọc hạng của thành phần bậc cao ở một số lớp; paper trích dẫn cho thấy cải thiện xử lý dữ liệu hiếm và độ bền với diễn đạt lại).

4. Số liệu trong survey

Survey nhắc thẳng: kết quả "chỉ lấy từ paper gốc". Sai khác perplexity = (perplexity mô hình nén) - (perplexity gốc), WikiText-2, nên **không** cùng mô hình, không cùng điều kiện.

Bảng 2 (pruning):

Loại	Phương pháp	Mô hình	Δ perplexity (WikiText-2)	Tỉ lệ nén	Tăng tốc
Unstructured	SparseGPT	OPT-175B	-0.14	50%	-
	Wanda	LLaMA-65B	1.01	50%	-
	SAMSP	LLaMA2-13B	0.63	50%	-
	DSnoT	LLaMA-65B	2.08e4	90%	-
Structured	LLM-Pruner	LLaMA-13B	3.6	20%	-
	Shortened LLaMA	LLaMA-7B	10.5	35%	-
	FLAP	LLaMA-65B	7.09	50%	-
	SliceGPT	LLaMA2-70B	1.73	30%	1.87x
Semi-structured	E-Sparse	LLaMA-65B	2.13	2:4	1.53x
	SparseGPT	OPT-175B	0.39	2:4	2x
	Wanda	LLaMA-65B	2.69	2:4	1.24x

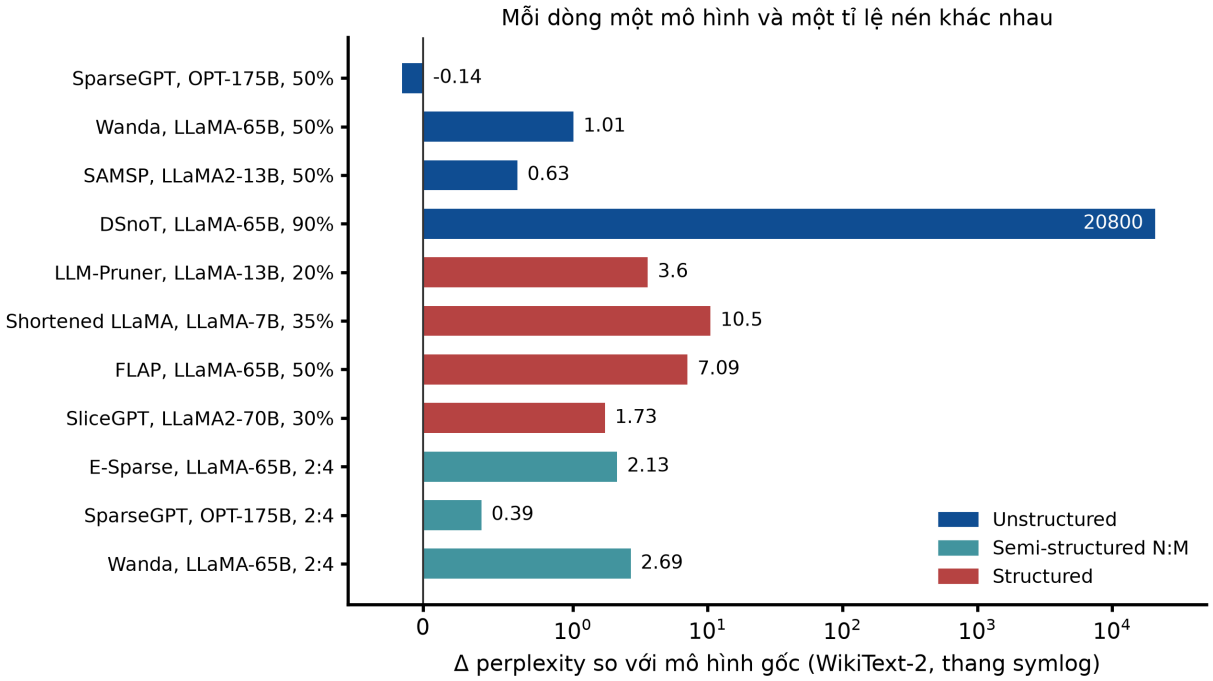
Đọc đúng bảng: Δ perplexity âm hoặc nhỏ ở OPT-175B không có nghĩa SparseGPT "tốt hơn" LLM-Pruner; mô hình khác nhau (175B so với 7-13B), tỉ lệ nén khác nhau, và structured luôn khó hơn unstructured ở cùng tỉ lệ. Con số DSnoT 2.08e4 ở 90% cho thấy sparsity cực cao làm mô hình hỏng (theo bảng).

Cách đọc hình: mỗi thanh là một dòng của bảng, màu theo loại pruning; thanh âm nghĩa là perplexity thấp hơn mô hình gốc. Mỗi dòng dùng mô hình và tỉ lệ nén khác nhau, nên không so độ cao các thanh giữa các dòng để kết luận phương pháp nào tốt hơn. Chỉ nên đọc ra hai điều: DSnoT ở 90% hỏng hẳn, còn các dòng còn lại nằm trong khoảng từ gần 0 đến 10,5.

Bảng 1 (quantization, trích để cảm nhận thang): GPTQ OPT-175B 3-bit, Δ perplexity 0.34 (WikiText-2), tăng tốc 3.24x; AWQ LLaMA2-70B 3-bit, 0.42, 3.2x; QuIP LLaMA2-70B 2-bit, 3.007; OneBit LLaMA-13B 1-bit, 4.09; SmoothQuant OPT-175B W8A8, 0.18, 1.56x; KVQuant LLaMA-65B KV 2-bit, 0.19, 1.4x.

5. Remarks và nhận định của tác giả

- **Remark 1:** QAT tốn công; weight-only lượng tử hoá được bit thấp hơn nhưng có overhead dequantize.
- **Remark 3:** dynamic pruning (cắt khác nhau theo tác vụ/đầu vào) để cân bằng hiệu năng và hiệu quả.
- **Remark 4:** chất lượng **dữ liệu hiệu chuẩn** rất quan trọng cho PTQ và pruning; nhắc nghiên cứu của Williams & Aletras (2023) cho thấy kết quả downstream thay đổi đáng kể theo dữ liệu hiệu chuẩn chọn.
- **Remark 5-7:** hạn chế của black-box KD (đầu ra teacher không phủ hết tình huống), white-box hạn chế vì LLM mở yếu hơn, và white-box cần hiểu cấu trúc nội bộ.
- **Hướng tương lai (Mục 7):** phương pháp tiên tiến hơn (còn khoảng cách với mô hình chưa nén), đưa kỹ thuật nén cổ điển (lottery ticket, parameter sharing) sang LLM, triển khai suy luận (arithmetic intensity, bộ nhớ, throughput, mô hình Roofline), tác động của scaling law (đánh đổi kích thước-hiệu năng), AutoML/NAS cho nén, khả diễn giải.



Hình 48: Δ perplexity WikiText-2 của các phương pháp pruning trong Bảng 2 của survey, theo thang symlog (vẽ lại từ số liệu trong ghi chú)

6. Giới hạn

Tác giả nêu (ngầm/nêu): các phương pháp nén còn khoảng cách hiệu năng so với mô hình gốc; thiết kế còn phụ thuộc thủ công; thiếu khả diễn giải.

Nhận xét:

- Tập trung LLM văn bản dạng Transformer thuần; **không** có multimodal/VLM, **không** có kiến trúc lai (linear attention, SSM), **không** có pruning layer/depth hiện đại (ShortGPT, SLEB, LaCo...) và không có pruning + distillation quy mô lớn kiểu Minitron/Puzzle.
- Phân loại pruning là "structured/unstructured/semi" theo **độ mịn**, còn trục quan trọng hơn cho kỹ sư triển khai (có cần retrain không, có cần dữ liệu không, tốc độ thật trên GPU) bị trộn lẫn.
- Gần như không có số đo latency thật ngoài vài chỗ tăng tốc ghi trong bảng.

7. Liên hệ với các bài khác

Phần của survey	Bài tương ứng trong lộ trình
Magnitude pruning, retrain	02, 03
Lottery ticket	08, 09
Loss-based importance (Taylor)	07
N:M / 2:4, Ampere	18
GPTQ ← OBQ	22 (OBC)
SparseGPT	23
Wanda	24
LLM-Pruner	26
FLAP	28
SliceGPT	29

Phần của survey	Bài tương ứng trong lộ trình
Shortened LLaMA	33
Sheared LLaMA (regularization + dynamic batch loading)	38
Pruning + KD (Remark 2)	39-45 (Minitron, Llama-3.1-Minitron, Puzzle...)
White-box KD (MiniLLM, GKD)	43 và các bài distillation khác

8. Gợi ý áp dụng

- Survey xác nhận rằng đây là hướng được tác giả gợi ý nhưng không cung cấp công thức cụ thể.
- **Khung đánh giá:** metric “Inference time / Speedup ratio / MFU” trong Mục 2.1 nhắc rằng kích thước và FLOPs không bằng latency; MFU thấp ở batch 1 là lý do cắt neuron nhỏ ít giúp (suy luận; xem thêm 19).
- **Thứ tự nén:** survey không nói rõ; bài 18 khuyên prune rồi mới quantize.
- Chưa có trong survey; ý này sẽ gặp lại ở 36 (ReplaceMe).

9. Thí nghiệm gợi ý

1. Điền bằng đo thực nghiệm nhỏ thay vì số của paper. ## 10. Tóm tắt khái niệm

Khái niệm	Nội dung
Bốn họ nén	Quantization, Pruning, Knowledge Distillation, Low-rank factorization
Pruning theo độ mịn	Unstructured (từng trọng số), Structured (neuron/head/layer), Semi-structured (N:M)
Pruning cấu trúc theo tiêu chí	Loss-based, Magnitude-based, Regularization-based
PTQ vs QAT	Không train lại vs train lại
Black-box vs white-box KD	Chỉ đầu ra teacher vs có phân phối/tham số teacher
MFU	FLOPS thực dùng / FLOPS đỉnh lý thuyết của thiết bị
Speedup ratio	Thời gian chạy chưa nén / đã nén
Δ perplexity (trong bảng)	Perplexity nén trừ perplexity gốc, WikiText-2
Remark 2	Kết hợp KD với structured pruning
Remark 4	Dữ liệu hiệu chuẩn ảnh hưởng mạnh đến chất lượng nén

11. Kết luận ngắn

Bài khảo sát hệ thống hóa các hướng nén LLM đến khoảng giữa năm 2024: bốn họ, ba mức độ mịn của pruning, các tên phương pháp chủ chốt và hướng đi (đáng nhớ nhất: kết hợp structured pruning với distillation, và vai trò của dữ liệu hiệu chuẩn). Nó là điểm xuất phát để tra cứu, không phải nguồn công thức hay so sánh công bằng; các bài cụ thể trong lộ trình mới là nơi cần đọc kỹ.

Link gốc: https://arxiv.org/abs/2308.07633

Bài 22

Optimal Brain Compression: A Framework for Accurate Post-Training Quantization and Pruning (2022)

Tác giả: Elias Frantar, Sidak Pal Singh, Dan Alistarh (IST Austria; ETH Zurich; Neural Magic)
Venue / arXiv: arXiv:2208.11580 (v2, 8/1/2023); NeurIPS 2022 (paper ghi "36th Conference on NeurIPS 2022"). Code: github.com/IST-DASLab/OBC.
Nguồn: https://arxiv.org/abs/2208.11580
Chủ đề: Nén **post-training** (một lần, không retrain) theo từng lớp: ExactOBS (pruning) và OBQ (quantization), trên nền Optimal Brain Surgeon; cập nhật bù trừ theo từng hàng của ma trận trọng số.
Vị trí trong lộ trình: Đọc sau 01 (OBD/OBS: nguồn gốc công thức), 18 (N:M) và 21 (survey). **Đây là nền toán của 23 (SparseGPT)** và của GPTQ (survey 21 nói GPTQ dựa trên OBQ). Bài này thêm: (1) đưa OBS về bài toán **hồi quy bình phương tối thiểu theo từng lớp** nên công thức OBS là chính xác, không còn là xấp xỉ; (2) tính exact greedy một cách hiệu quả bằng cập nhật nghịch đảo Hessian; (3) dùng cùng khung cho quantization. Tiếp theo: 23 (SparseGPT, mở rộng lên LLM bằng cách ép thứ tự cột chung), 27 (LLM Surgeon).
Chương nền liên quan: nền 03: Độ quan trọng của trọng số

> Đây là ghi chú học tập bằng tiếng Việt, không phải bản dịch. Phần dẫn xuất công thức dưới đây được viết lại từ các phương trình của paper (paper dẫn đến Hassibi & Stork cho chứng minh), kèm một ví dụ số tự tính. Thí nghiệm của paper là CNN (ResNet, YOLOv5) và BERT, **chưa có LLM** (paper nói "plan to investigate ... very large-scale language models").

1. Bối cảnh

Hai cách nén mạng: pruning (đặt trọng số về 0) và quantization (giảm số bit). Phần lớn phương pháp cần huấn luyện lại một phần hoặc toàn bộ sau khi nén.

Bối cảnh post-training: cho một mô hình đã huấn luyện và một ít dữ liệu hiệu chuẩn (calibration; ở bài này 1024 mẫu), phải nén **trong một bước, không huấn luyện lại**. Cách chuẩn: chia nhỏ thành bài toán **theo từng lớp** (layer-wise): với lớp có trọng số W_ℓ và đầu vào X_ℓ, tìm $\hat{W}_\ell$ thoả ràng buộc nén sao cho đầu ra gần nhất. Quantization post-training đã có AdaRound, AdaQuant, BRECQ; pruning post-training có AdaPrune (cắt theo magnitude rồi tối ưu lại phần còn lại).

Optimal Brain Surgeon (OBS) (Hassibi & Stork; xem 01) là khung cổ điển: dùng Hessian của loss để chọn trọng số cần xoá và cập nhật các trọng số còn lại để bù. Vấn đề: Hessian $d \times d$ quá lớn với d cỡ 10^5-10^6, và các biến thể hiện đại (WoodFisher, M-FAC) cần pruning dần cộng huấn luyện lại.

Câu hỏi của bài: có thể áp dụng OBS **chính xác** (xoá từng trọng số, cập nhật lại Hessian sau mỗi bước) ở quy mô mạng hiện đại, trong bối cảnh post-training, và dùng chung cho quantization không?

Giải thích thuật ngữ:

- **Hessian H:** ma trận đạo hàm bậc hai của loss theo trọng số; cho biết loss cong thế nào quanh điểm hiện tại.
- **Layer-wise reconstruction:** thay vì nhìn loss toàn mạng, đo sai khác đầu ra của từng lớp $\|WX - \hat{W}X\|_2^2$ trên dữ liệu hiệu chuẩn.
- **Greedy (tham lam):** mỗi bước chọn quyết định tốt nhất cục bộ, không đảm bảo tối ưu toàn cục.
- **Dampening:** cộng một số nhỏ vào đường chéo H để nghịch đảo ổn định.

2. Ý tưởng chính

Với một lớp tuyến tính, hàm mất mát tái tạo $\|WX - \hat{W}X\|_2^2$ là **hàm bậc hai chính xác** theo trọng số, và điểm xuất phát (trọng số dense) có loss bằng 0 nên gradient bằng 0. Khi đó, hai giả định của OBS (Taylor bậc hai, gradient ≈ 0) đều được thỏa mãn, nên công thức OBS cho "trọng số xoá tiếp theo tối ưu" và "cập nhật bù tối ưu" là chính xác. Hơn nữa loss tách theo **hàng** của W (mỗi hàng là một đầu ra độc lập), nên chỉ cần Hessian $d_{col} \times d_{col}$ cho từng hàng, và Hessian đó là $H = 2XX^\top$**, giống nhau cho mọi hàng**. Khó khăn còn lại là tính nghịch đảo sau mỗi lần xoá; paper dùng một bước khử Gauss để cập nhật nghịch đảo trong $\Theta(d_{col}^2)$ thay vì nghịch đảo lại $\Theta(d_{col}^3)$. Thay "xoá về 0" bằng "làm tròn vào lưới lượng tử" thì có thuật toán quantization (OBQ) với cùng chi phí.

3. Phương pháp chi tiết (dẫn xuất)

3.1 Bài toán layer-wise và tách theo hàng

Lớp có $W \in \mathbb{R}^{d_{row} \times d_{col}}$ và đầu vào hiệu chuẩn $X \in \mathbb{R}^{d_{col} \times N}$ (với conv, d_{col} là số trọng số trong một filter). Bài toán (Phương trình 2 của paper):

$$\arg\min_{\hat{W}} \|WX - \hat{W}X\|_2^2 \quad \text{s.t. } \mathcal{C}(\hat{W}) > C.$$

Vì $\|WX - \hat{W}X\|_F^2 = \sum_{i=1}^{d_{row}} \|W_{i,:}X - \hat{W}_{i,:}X\|_2^2$, mỗi hàng độc lập: xoá $[W]_{ij}$ chỉ ảnh hưởng đầu ra hàng i. Do đó **không có tương tác Hessian giữa các hàng**.

3.2 Hessian của một hàng

Gọi $w \in \mathbb{R}^{d_{col}}$ là một hàng và $\hat{w} = w + \delta$. Sai số đầu ra:

$$L(\delta) = \|(w+\delta)^\top X - w^\top X\|_2^2 = \|\delta^\top X\|_2^2 = \delta^\top XX^\top \delta = \tfrac{1}{2}\,\delta^\top H\,\delta, \qquad \boxed{H = 2XX^\top}$$

Loss bậc hai chính xác, tại $\delta = 0$ gradient bằng 0, Hessian H **không phụ thuộc vào** w nên giống nhau cho mọi hàng. (Chỉ phụ thuộc dữ liệu hiệu chuẩn X.)

3.3 Công thức OBS: xoá một trọng số p

Muốn đặt $\hat{w}_p = 0$, tức ràng buộc $e_p^\top \delta = -w_p$ (e_p là vector chuẩn thứ p). Chọn δ nhỏ nhất loss:

$$\min_{\delta} \tfrac{1}{2}\delta^\top H\delta \quad \text{s.t. } e_p^\top \delta + w_p = 0.$$

Lagrangian $\mathcal{L}(\delta, \lambda) = \frac{1}{2}\delta^\top H\delta + \lambda\,(e_p^\top \delta + w_p)$ (paper viết $\delta^\top H\delta$ không có hệ số 1/2; chỉ khác hệ số, không đổi argmin). Đạo hàm theo δ bằng 0:

$$H\delta + \lambda e_p = 0 \;\Rightarrow\; \delta = -\lambda H^{-1} e_p = -\lambda H^{-1}_{:,p}.$$

Thế vào ràng buộc: $-\lambda\, e_p^\top H^{-1} e_p + w_p = 0$, suy ra $\lambda = w_p / [H^{-1}]_{pp}$. Vậy:

$$\boxed{\delta_p = -\frac{w_p}{[H^{-1}]_{pp}}\, H^{-1}_{:,p}}$$

Loss tăng lên:

$$\Delta L_p = \tfrac{1}{2}\delta_p^\top H \delta_p = \tfrac{1}{2}\lambda^2\, e_p^\top H^{-1} e_p = \tfrac{1}{2}\frac{w_p^2}{[H^{-1}]_{pp}}.$$

Do đó trọng số chọn xoá (Phương trình 3 của paper, bỏ hệ số 1/2):

$$\boxed{p^\star = \arg\min_p \frac{w_p^2}{[H^{-1}]_{pp}}}$$

Diễn giải: bỏ trọng số w_p "rẻ" nếu $|w_p|$ nhỏ **hoặc** nếu $[H^{-1}]_{pp}$ lớn (hướng p có độ cong H thấp, tức các trọng số khác dễ bù). Khi H chéo, $[H^{-1}]_{pp} = 1/H_{pp}$ và tiêu chí thành $H_{pp} w_p^2$ giống OBD (01); phần cập nhật δ là điều OBD không có. Cập nhật δ sửa các trọng số còn lại để bù đắp (hàng $w \leftarrow w + \delta_p$, thành phần p trở thành đúng 0).

3.4 Ví dụ số nhỏ (tự tính)

Một hàng 2 trọng số, $H = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}$, $H^{-1} = \frac{1}{3}\begin{pmatrix} 2 & -1 \\ -1 & 2 \end{pmatrix}$, $w = (1,\, 0.5)$.

- Điểm: $p = 1$: $1^2/(2/3) = 1.5$; $p = 2$: $0.5^2/(2/3) = 0.375$. Chọn xoá $p = 2$.
- Cập nhật: $\delta = -\frac{0.5}{2/3} H^{-1}_{:,2} = -0.75 \cdot (-\frac{1}{3}, \frac{2}{3}) = (0.25,\, -0.5)$. Trọng số mới $(1.25,\, 0)$.
- Kiểm tra: $H\delta = (0.5 - 0.5,\; 0.25 - 1) = (0, -0.75)$, $\delta^\top H \delta = 0.375$, nên $\Delta L = 0.1875 = \frac{1}{2} \cdot 0.375$. Khớp công thức. Trọng số còn lại tăng từ 1 lên 1.25 để bù cho trọng số bị xoá, vì hai đầu vào tương quan (độ lệch ngoài đường chéo của H bằng 1).

Cách đọc hình: các elip là đường mức của $L = \frac{1}{2}\delta^\top H \delta$ quanh trọng số dense (chấm xanh), đường ngang là ràng buộc $w_2 = 0$. Điểm vuông đỏ chỉ đặt $w_2 = 0$ và đứng trên elip loss 0,25; điểm thoi xanh dịch w_1 từ 1 lên 1,25 và rơi xuống elip loss 0,1875, thấp nhất có thể trên đường ràng buộc. Hai trọng số tương quan nên trọng số còn lại phải tăng để bù.

3.5 Lặp lại theo từng bước (ExactOBS) và cập nhật nghịch đảo

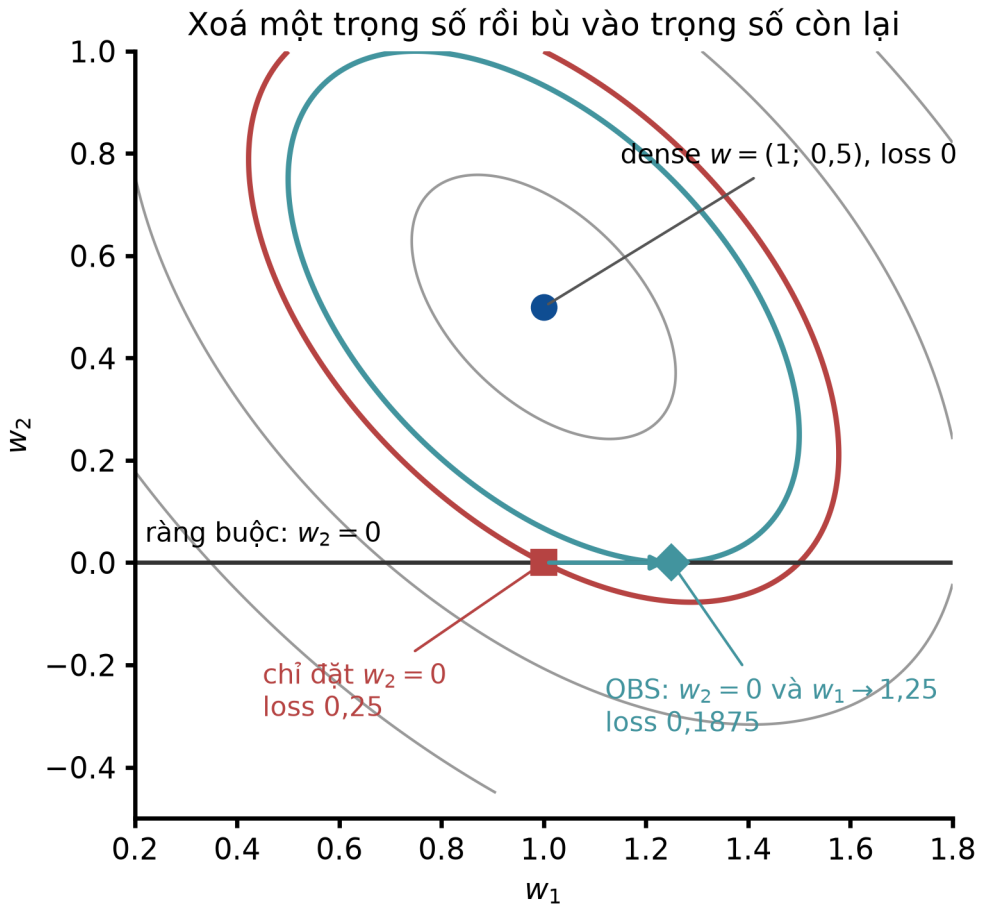
Lặp: xoá một trọng số, cập nhật phần còn lại, rồi lại tính tiêu chí trên **Hessian của tập trọng số còn lại**. Gọi H_M là Hessian của các trọng số còn trong mask M: nó chỉ là các hàng/cột tương ứng của H dense (vì $H = 2XX^\top$ không phụ thuộc w). **Nhưng** $(H_M)^{-1} \neq (H^{-1})_M$, nên không thể chỉ cắt nghịch đảo ban đầu. Nghịch đảo lại $\Theta(d_{col}^3)$ mỗi bước là quá đắt.

Lemma 1 (xoá hàng/cột p khỏi H, cập nhật nghịch đảo):

$$\boxed{H_{-p}^{-1} = \left(H^{-1} - \frac{1}{[H^{-1}]_{pp}} H^{-1}_{:,p} H^{-1}_{p,:}\right)_{-p}}$$

Tức là khử Gauss hàng/cột p trong H^{-1} rồi bỏ chúng đi; chi phí $\Theta(d_{col}^2)$. (Đây là công thức Schur-complement quen thuộc cho nghịch đảo ma trận con: phần tử (i, j) của kết quả là $[H^{-1}]_{ij} - [H^{-1}]_{ip}[H^{-1}]_{pj}/[H^{-1}]_{pp}$.) Trong cài đặt, không cần co ma trận: sau khi khử, hàng và cột p bằng 0 và không bao giờ được truy cập lại.

```
# Algorithm 1: ExactOBS cho MỘT hàng w (d_col trọng số), cần H^{-1} = (2XX^T)^{-1}
M = {1..d_col}
for i in 1..k:                                  # k = số trọng số cần xoá
    p  = argmin_{p in M}  w[p]^2 / Hinv[p,p]    # chọn trọng số rẻ nhất
    w  = w - Hinv[:,p] * (w[p] / Hinv[p,p])     # cập nhật bù; w[p] về đúng 0
    Hinv = Hinv - outer(Hinv[:,p], Hinv[p,:]) / Hinv[p,p]   # Lemma 1, O(d_col^2)
```



Hình 49: Ví dụ hai trọng số: chỉ đặt w2 về 0 so với xoá kèm cập nhật bù OBS, trên các đường mức loss của hàm bậc hai (minh hoạ, tự tính)

```
    M  = M \ {p}
# chi phí: O(k * d_col^2) cho một hàng
```

3.6 Xét cả ma trận: chọn mask toàn cục

Muốn xoá trọng số có điểm thấp nhất **trên toàn ma trận** ở mỗi bước, tưởng như cần giữ d_{row} ma trận nghịch đảo $d_{col} \times d_{col}$ (tốn bộ nhớ $O(d_{row} d_{col}^2)$). Quan sát: không có tương tác giữa hàng, nên **thứ tự xoá trong mỗi hàng là cố định**, và độ tăng loss khi xoá một trọng số chỉ phụ thuộc những trọng số đã xoá trong **cùng hàng**. Do đó:

1. Chạy ExactOBS cho từng hàng, xoá hết (hoặc đến mức cần), ghi lại $\delta L_p = \frac{1}{2} w_p^2 / [H^{-1}]_{pp}$ tại thời điểm xoá mỗi trọng số ($\Theta(d)$ bộ nhớ thêm).
2. Mask toàn cục = các trọng số có δL thấp nhất (xếp theo thứ tự đã định của từng hàng).
3. Với mask mỗi hàng M_i đã biết, tính trọng số cuối bằng công thức OBS nhóm: $\delta_M = -H^{-1}_{:,M}((H^{-1})_M)^{-1} w_M$ (nhanh hơn chạy lại), hoặc đọc lại vết (trace) từ bộ nhớ CPU nếu đủ ($O(d_{row} d_{col}^2)$ CPU).

Độ phức tạp tổng: thời gian $O(d_{row} \cdot d_{col}^3)$ (tương đương $O(d \cdot d_{col}^2)$ với $d = d_{row} d_{col}$), bộ nhớ $\Theta(d_{col}^2)$, **chính xác** (không xấp xỉ), so với $O(d^4)$ cho OBS ngây thơ. Ví dụ: ResNet50 toàn mạng trong hơi hơn 1 giờ trên một RTX 3090.

3.7 Chi tiết cài đặt và các mẫu thưa

- **Dampening:** H có thể không khả nghịch (ít mẫu, đầu vào chết hoặc phụ thuộc tuyến tính): cộng một số nhỏ vào đường chéo, và/hoặc tăng dữ liệu bằng augmentation (rẻ vì chỉ cộng dồn vào H một lần).
- **N:M (ví dụ 2:4):** chỉ đổi một điều: chọn trọng số có δL nhỏ nhất **trong khối còn chưa đủ N trọng số bị xoá**; mọi hàng có cùng độ thưa nên dừng khi đạt $1 - N/M$, không cần bước mask toàn cục.
- **Block sparsity (khối c):** công thức nhóm (Phương trình 5): $w_P = \arg\min_P \; w_P^\top ((H^{-1})_P)^{-1} w_P$, $\delta_P = -H^{-1}_{:,P}((H^{-1})_P)^{-1} w_P$, áp dụng Lemma 1 lần lượt cho từng $p \in P$.

3.8 Optimal Brain Quantizer (OBQ)

Cùng Lagrangian (Phương trình 6), nhưng ràng buộc mới là $e_p^\top \delta = \text{quant}(w_p) - w_p$ (sau cập nhật $\hat{w}_p = \text{quant}(w_p)$). Kết quả (Phương trình 7):

$$p^\star = \arg\min_p \frac{(\text{quant}(w_p) - w_p)^2}{[H^{-1}]_{pp}}, \qquad \delta_p = -\frac{w_p - \text{quant}(w_p)}{[H^{-1}]_{pp}} H^{-1}_{:,p}.$$

Nếu $\text{quant}(\cdot) \equiv 0$ ta quay lại OBS cho pruning, nên đây là khái quát hoá. Với quantization cả lớp: lần lượt lượng tử hoá trọng số "dễ nhất" (tăng loss ít nhất), rồi **cập nhật các trọng số chưa lượng tử hoá** để bù. Điểm khác với làm tròn thẳng: thứ tự và bù trừ làm các trọng số về sau bị đẩy, nên kết quả khác phép làm tròn ban đầu và tốt hơn. Vấn đề **outlier** (trọng số nằm ngoài lưới có lỗi lớn): nếu để cuối thì không còn trọng số nào để bù; heuristic: lượng tử hoá ngay những trọng số có lỗi $> \Delta/2$ (Δ là khoảng cách giữa hai mức lưới) khi chúng xuất hiện.

(GPTQ, ghi trong survey 21, bắt nguồn từ OBQ.)

3.9 Phân bổ độ thưa/bit theo từng lớp

Với ràng buộc FLOPs/thời gian, OBC dùng **model database**: nén độc lập từng lớp ở mọi mức (độ thưa, bit) và đo loss hiệu chuẩn của mô hình chỉ nén một lớp; rồi giải bài toán chọn mức cho từng lớp bằng quy hoạch động (thuật toán SPDY, công thức kiểu AdaQuant), cuối cùng "ghép" các lớp lại và hiệu chỉnh thống kê (batchnorm reset, hoặc mean/variance correction cho YOLO và BERT).

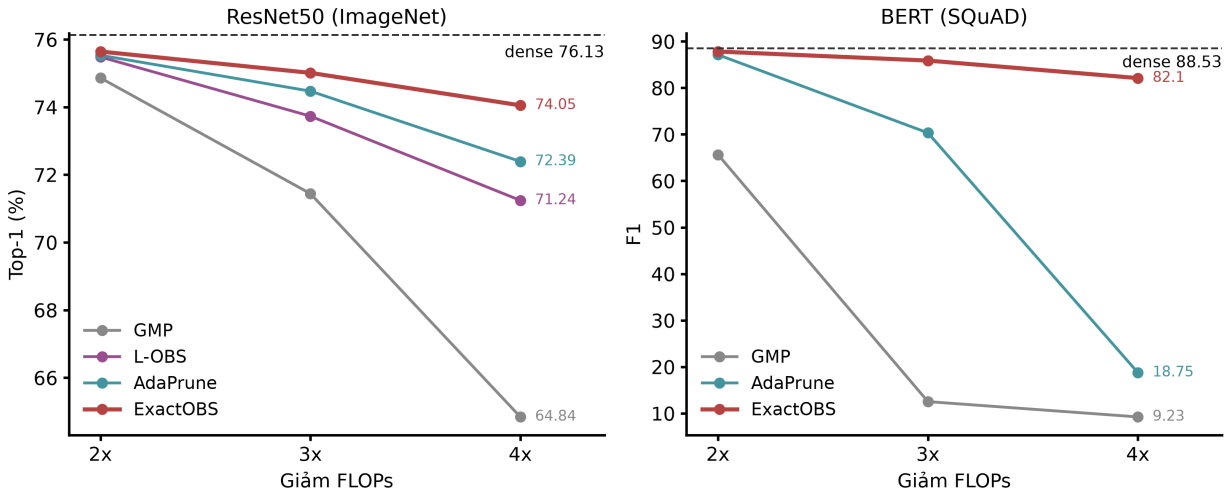
4. Thí nghiệm và kết quả chính

Mô hình: ResNet18/34/50 (ImageNet), YOLOv5 (COCO), BERT, BERT3, BERT6 (SQuAD, F1). Dữ liệu hiệu chuẩn: 1024 mẫu train ngẫu nhiên; ImageNet thêm flip/crop để tăng 10x. So sánh: GMP (global magnitude pruning), L-OBS (xấp xỉ OBS theo lớp), AdaPrune, ExactOBS.

Pruning không cấu trúc theo FLOPs giảm (Bảng 1):

Mô hình (dense)	Phương pháp	2x	3x	4x
ResNet50 (76.13)	GMP	74.86	71.44	64.84
	L-OBS	75.48	73.73	71.24
	AdaPrune	75.53	74.47	72.39
	ExactOBS	75.64	75.01	74.05
YOLOv5l (66.97, mAP@0.5)	AdaPrune	66.00	64.88	62.71
	ExactOBS	66.14	65.35	64.05
BERT (88.53, F1)	GMP	65.64	12.52	09.23
	AdaPrune	87.12	70.32	18.75
	ExactOBS	87.81	85.87	82.10

(YOLOv5l 2x: tất cả gần nhau; GMP 65.83, L-OBS 66.21, tức L-OBS nhỉnh hơn ExactOBS.) BERT là mô hình khó cắt: ExactOBS là phương pháp duy nhất còn kết quả hợp lý ở 3x, 4x. Với BERT 3x/4x (sụt >2-3%), thêm bước global AdaPrune (gAP) sau cùng: gAP+AdaPrune 86.99 / 84.10; gAP+ExactOBS 87.57 / 86.42 (Bảng 5).



Hình 50: Độ chính xác khi giảm FLOPs 2x, 3x, 4x với ResNet50 và BERT, theo Bảng 1 của ghi chú (vẽ lại từ số liệu trong ghi chú)

Cách đọc hình: trục ngang là mức giảm FLOPs, đường nét đứt là độ chính xác của mô hình dense. Ở 2x các phương pháp còn gần nhau, khoảng cách mở ra khi cắt mạnh hơn. Trên BERT ở 4x, ExactOBS còn 82,10 F1 trong khi AdaPrune chỉ còn 18,75.

N:M (Bảng 2, 3):

Mô hình	Dense	AdaPrune 4:8	ExactOBS 2:4	ExactOBS 4:8
ResNet18	69.76	68.63	68.81	69.18
ResNet34	73.31	72.36	72.66	72.95
ResNet50	76.13	74.75	74.71	75.20

Mô hình (2:4, F1)	Dense	AdaPrune	ExactOBS
BERT3	84.66	82.75	83.54
BERT6	88.33	85.02	86.97
BERT	88.53	85.24	86.77

ExactOBS 2:4 ngang hoặc hơn AdaPrune 4:8 (mẫu khắt khe hơn), và hơn AdaPrune 2:4 khoảng 1-2% F1 trên BERT.

Quantization trọng số, per-channel bất đối xứng (Bảng 4, ResNet; ImageNet top-1; dense ResNet18 69.76, ResNet50 76.13):

Phương pháp	R18 4b	R18 3b	R18 2b	R50 4b	R50 3b	R50 2b
AdaRound	69.34	68.37	63.37	75.84	75.14	71.58
AdaQuant	68.12	59.21	00.10	74.68	64.98	00.10
BRECQ	69.37	68.47	64.70	75.88	75.32	72.41
OBQ	69.56	68.69	64.04	75.72	75.24	70.71

OBQ nén **độc lập từng lớp** nhưng ngang các phương pháp tuần tự (chỉ thua ở 2-bit).

Nén kết hợp:

- **GPU (BOPs = số bit × FLOPs):** mỗi lớp chọn trong 8w8a, 4w4a, 8w8a+2:4, 4w4a+2:4. Với ≈2.5% sụt tương đối: giảm 12-14x BOPs cho ResNet, 7-8x cho YOLO và BERT. 4w4a + 2:4 post-training trên ResNet18 đạt 67.20%, so với 67.33% của QAT-thưa 90 epoch (không hoàn toàn so sánh được).
- **CPU (DeepSparse, block size 4 + INT8, ResNet50, batch 64):** tăng tốc thật 4x với sụt 1%, 5x với sụt 2% (tăng tốc dense INT8 cơ sở ≈2.7x, thưa nhân lên xấp xỉ).

5. Phân tích / ablation

- **Sai số tái tạo theo lớp:** ở lớp đầu của ResNet18 (Hình 2), ExactOBS có squared error thấp nhất "by a wide margin", rồi AdaPrune, rồi L-OBS và GMP. Kiểm chứng rằng việc "chính xác từng bước" giúp ở cấp lớp, rồi chuyển được thành độ chính xác end-to-end.
- **Cập nhật tần suất cao hơn tốt hơn:** AdaPrune chỉ cắt theo magnitude rồi tối ưu lại; ExactOBS tái tối ưu **sau mỗi trọng số** và chọn bằng thông tin bậc hai (paper nói đây là đẩy ý tưởng của AdaPrune đến cực hạn).
- **Độc lập vs tuần tự:** nén từng lớp độc lập cho phép ghép linh hoạt (database) và nhanh đổi ràng buộc; cái giá phải trả là không bù lỗi tích luỹ giữa các lớp (paper nói ExactOBS vẫn có thể chạy tuần tự, ở phụ lục).
- **Global AdaPrune cộng thêm:** ở BERT 3x/4x, bước tối ưu toàn cục vẫn để lại lợi thế ExactOBS (>0.5% và >2% F1 theo văn bản).
- **Mặt chi phí:** phần dẫn xuất ở mục 3 cho thấy chi phí/bộ nhớ; phần runtime chi tiết nằm ở phụ lục, tôi không chép số cụ thể vì chưa đọc kỹ.

6. Giới hạn

Tác giả nêu:

- Greedy, không đảm bảo tối ưu toàn cục (nói rõ trong phần OBS layer-wise).
- Chỉ xét lớp tuyến tính/conv; structured pruning "nên mở rộng được" nhưng chưa làm.
- Chưa thử mô hình rất lớn; kế hoạch tương lai là LLM.
- H có thể không khả nghịch, cần dampening.

Nhận xét:

- Thời gian $O(d_{row}d_{col}^3)$ và việc mỗi hàng có thứ tự riêng với H^{-1} riêng: với d_{col} cỡ 4k-12k (LLM), chi phí lớn. Đây chính là điều SparseGPT (23) giải quyết bằng cách dùng **chung một thứ tự cột** cho mọi hàng để tái dùng một nghịch đảo (nội dung bài 23).
- Loss layer-wise dùng X **của mô hình dense** (đầu vào lớp tính từ mô hình gốc): lỗi tích luỹ của các lớp trước không được tính.
- Mọi kết quả là CNN/BERT cỡ nhỏ; chưa có kiểm chứng trên decoder lớn, mô hình lai hoặc VLM.
- Mục tiêu là sparsity/bit của **trọng số**, không phải cắt cả kênh/lớp.

7. Liên hệ với các bài khác

- **01 (OBD/OBS):** OBC dùng đúng công thức OBS; điểm mới là chỉ ra rằng ở bài toán tái tạo lớp tuyến tính thì giả định bậc hai là chính xác và làm nó nhanh.
- **02, 03:** pruning cổ điển cần retrain dài; OBC thuộc nhánh post-training.
- **18 (2:4):** OBC chọn mask 2:4 bằng δL thay cho magnitude, kèm cập nhật bù; kết quả tốt hơn AdaPrune ở N:M.
- **23 (SparseGPT):** cùng bài toán $\|WX - \hat{W}X\|^2$ và công thức OBS, nhưng ép chung thứ tự cột + khử nghịch đảo qua phân rã Cholesky để mở rộng lên LLM tỉ tham số; **đọc 22 kỹ trước khi đọc 23**.
- **21 (survey):** GPTQ bắt nguồn từ OBQ.
- **27 (LLM Surgeon), 24 (Wanda):** 27 mở rộng OBS ra nhiều mức và cấu trúc; Wanda đơn giản hoá bằng cách bỏ cập nhật.
- **29 (SliceGPT), 36 (ReplaceMe):** cũng có ý "sửa phần còn lại sau khi cắt" bằng ánh xạ tuyến tính, cùng tinh thần với bù trừ δ.

8. Gợi ý áp dụng

- Ý tưởng: sau khi cắt neuron MLP, thay vì chỉ distill, hãy **giải thẳng bình phương tối thiểu** cho trọng số của lớp kế tiếp (hồi quy W_{down} để khớp đầu ra dense trên mẫu hiệu chuẩn).
- **Cắt neuron MLP có cấu trúc (xoá các thành phần tương ứng trong W_{up}/W_{gate} và W_{down}):** OBS cho trọng số lẻ, nhưng khung mở rộng được cho nhóm (công thức block δ_P ở Mục 3.7 là cùng một họ). Cần kiểm tra thực nghiệm; paper chỉ nói "structured pruning nên mở rộng được".
- **Chọn mask 2:4 hoặc quantization:** nếu sau cùng muốn thêm 2:4 (bài 18) hay INT4, OBC/SparseGPT cho phương án tốt hơn magnitude khi không retrain.

9. Thí nghiệm gợi ý

2. Cắt 30% neuron của một khối MLP: (a) xoá theo norm rồi để nguyên; (b) xoá rồi giải bình phương tối thiểu lại W_{down} trên dữ liệu hiệu chuẩn; (c) (b) + distill ngắn.
3. Kiểm tra độ nhạy với kích thước dữ liệu hiệu chuẩn (128, 1024, 4096 mẫu) và hệ số dampening, vì H thiếu hạng khi N nhỏ.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / nội dung
Bài toán lớp	$\min_{\hat{W}} \|WX - \hat{W}X\|_2^2$ s.t. ràng buộc nén
Tách hàng	Tổng sai số các hàng, không tương tác giữa hàng
Hessian một hàng	$H = 2XX^\top$ ($d_{col} \times d_{col}$), chung cho mọi hàng
Điểm OBS	$\delta L_p = \frac{1}{2} w_p^2 / [H^{-1}]_{pp}$ (paper bỏ 1/2)
Cập nhật bù	$\delta_p = -\frac{w_p}{[H^{-1}]_{pp}} H^{-1}_{:,p}$
Cập nhật nghịch đảo	$H^{-1}_{-p} = (H^{-1} - \frac{1}{[H^{-1}]_{pp}} H^{-1}_{:,p} H^{-1}_{p,:})_{-p}$, $\Theta(d_{col}^2)$
Mask toàn cục	Ghi δL khi xoá trong từng hàng, chọn δL nhỏ nhất, rồi dùng $\delta_M = -H^{-1}_{:,M}((H^{-1})_M)^{-1} w_M$
Độ phức tạp	$O(d_{row} d_{col}^3)$ thời gian, $\Theta(d_{col}^2)$ bộ nhớ (so với $O(d^4)$)
N:M	Chỉ xét khối còn < N trọng số bị xoá; dừng khi đạt $1 - N/M$
OBQ	Thay đích 0 bằng quant(w_p); xử lý outlier ngay khi lỗi $> \Delta/2$
Kết quả nổi bật	ResNet50 4x FLOPs: 74.05 (ExactOBS) vs 72.39 (AdaPrune); BERT 4x: 82.10 vs 18.75; 4w4a+2:4 ResNet18: 67.20; CPU 4x thực với sụt 1%

11. Kết luận ngắn

OBC chứng minh rằng khi nhìn nén từng lớp như bài toán bình phương tối thiểu, OBS là công thức **chính xác** và có thể chạy chính xác với chi phí $O(d_{row} d_{col}^3)$ nhờ cập nhật nghịch đảo từng bước và tận dụng việc Hessian chung cho mọi hàng. Hai thứ cần mang theo sang bài 23: (1) công thức chọn và bù δ_p, $w_p^2/[H^{-1}]_{pp}$; (2) ý tưởng "xoá một trọng số, bù ngay vào các trọng số còn lại theo hướng H^{-1}".

Link gốc: https://arxiv.org/abs/2208.11580

Bài 23

SparseGPT: Massive Language Models Can Be Accurately Pruned in One-Shot (2023)

Tác giả: Elias Frantar, Dan Alistarh (IST Austria; Neural Magic)
Venue / arXiv: arXiv:2301.00774 (v3, 22/3/2023); bản text không ghi tên venue (bài trình bày theo định dạng ICML). Code: github.com/IST-DASLab/sparsegpt.
Nguồn: https://arxiv.org/abs/2301.00774
Chủ đề: Pruning một lần (one-shot, không fine-tune) cho LLM cỡ 10-175B tham số; bài toán hồi quy thưa theo lớp; cập nhật kiểu OBS với **chia sẻ Hessian giữa các hàng**; chọn mask thích nghi theo khối; N:M; kết hợp lượng tử hoá.
Vị trí trong lộ trình: Đọc ngay sau **22 (OBC)**, vì SparseGPT là "OBC mở rộng lên LLM bằng một xấp xỉ có kiểm soát". Cũng nên đọc cùng 18 (2:4) và 21 (survey). Tiếp theo: 24 (Wanda: bỏ hẳn bước cập nhật trọng số, chỉ dùng |w|·‖x‖), 27 (LLM Surgeon), 30 (MaskLLM).
Chương nền liên quan: nền 02: Ma trận thưa và chi phí thật, nền 03: Độ quan trọng của trọng số, nền 04: Cắt hạt, cắt khối và mẫu 2, nền 05: Cắt rồi huấn luyện lại, nền 07: Đo đúng một kết quả pruning

> Đây là ghi chú học tập bằng tiếng Việt, không phải bản dịch.

1. Bối cảnh

Mô hình GPT 175B cần ít nhất 320 GB ở FP16 và ít nhất 5 GPU A100 80 GB để suy luận. Trước bài này, nén LLM gần như chỉ là **quantization** (LLM.int8, ZeroQuant, SmoothQuant, GPTQ). Pruning chính xác thì cần huấn luyện lại dài, không khả thi ở quy mô này. Các phương pháp pruning post-training chính xác (AdaPrune, OBC, bài 22) chỉ chạy được trên mô hình cỡ vài trăm triệu tham số: AdaPrune đã mất vài giờ cho mô hình 1.3B, ngoại suy tuyến tính lên 175B sẽ cỡ vài tuần; OBC có độ phức tạp tăng như luỹ thừa 4 của kích thước ẩn. Magnitude pruning là baseline duy nhất chạy được ở quy mô lớn, nhưng theo bài báo mất chính xác rất nhanh trên LLM.

Nền tảng cần nhớ từ bài 22 (OBC): bài toán theo từng lớp

$$\arg\min_{M_\ell, \hat{W}_\ell} \|W_\ell X_\ell - (M_\ell \odot \hat{W}_\ell) X_\ell\|_2^2 \tag{1}$$

với M_ℓ là mask nhị phân. Bài toán này (chọn mask cùng cập nhật trọng số) là NP-hard, nên cần xấp xỉ.

Giải thích thuật ngữ:

- **One-shot / post-training pruning:** nén một lần trên mô hình đã huấn luyện với ít dữ liệu hiệu chuẩn, không huấn luyện lại.
- **Sparse regression (hồi quy thưa):** tìm vector trọng số ít thành phần khác 0 sao cho dự đoán gần nhất.
- **Cholesky decomposition:** phân tích $A = LL^\top$ với ma trận đối xứng xác định dương, ổn định số học.
- **Semi-structured N:M:** trong mỗi M trọng số liên tiếp có đúng N số 0 (xem 18).

2. Ý tưởng chính

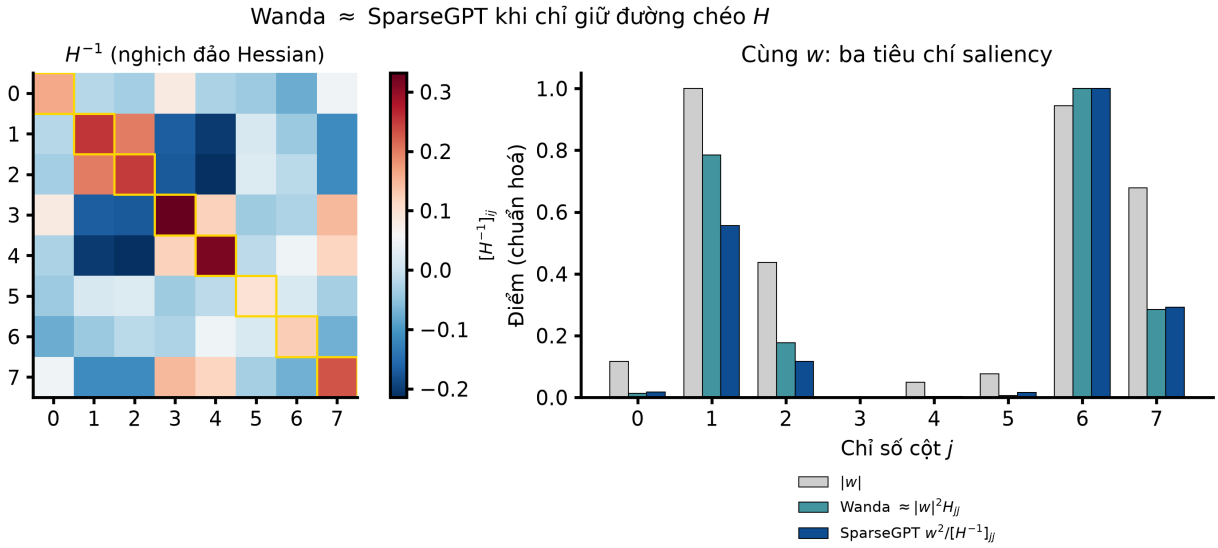
Với mặt nạ cố định, cập nhật tối ưu trọng số của mỗi hàng là một bài toán hồi quy bình phương tối thiểu, nhưng mỗi hàng có mask khác nhau nên cần nghịch đảo một Hessian khác nhau cho mỗi hàng: quá đắt. SparseGPT **ép mọi hàng dùng chung một chuỗi các Hessian nghịch đảo** bằng cách xử lý các cột theo một thứ tự cố định (từ trái sang phải): khi đến cột j, mọi hàng đều chỉ cho phép cập nhật các trọng số ở bên phải j (chưa

xử lý), nên tập chỉ số "còn cập nhật được" giống nhau cho mọi hàng và chỉ phải tính một chuỗi nghịch đảo. Phần giá phải trả: mỗi lần xoá chỉ bù vào các trọng số ở bên phải thay vì tất cả trọng số còn lại (cập nhật "một phần", tối ưu trong tập con). Để mask không bị ép phải đều theo cột, mask được chọn thích nghi theo khối 128 cột dựa trên sai số OBS. Kết quả: độ phức tạp từ $O(d_{row}d_{col}^3)$ xuống $O(d_{col}^3 + d_{row}d_{col}^2)$, cho phép cắt OPT-175B trong khoảng 4 giờ trên một A100.

3. Phương pháp chi tiết

3.1 Với mask cố định: nghiệm đóng và cách nhìn lặp

Cho mask hàng M_i, các trọng số giữ lại tối ưu (Phương trình 2):



So sánh saliency magnitude / Wanda (đường chéo H) / SparseGPT ($w^2/[H^{-1}]_{jj}$) trên cùng w. (minh hoạ, tự tính)

$$w_{M_i} = (X_{M_i}X_{M_i}^\top)^{-1}X_{M_i}(w_{M_i}X_{M_i})^\top$$

(tức bình phương tối thiểu trên các đặc trưng đầu vào còn giữ). Cần $(H_{M_i})^{-1}$ riêng cho mỗi hàng i: $O(d_{row}d_{col}^3)$; với Transformer là luỹ thừa 4 của d_{hidden} (cần nhanh hơn d_{hidden} lần).

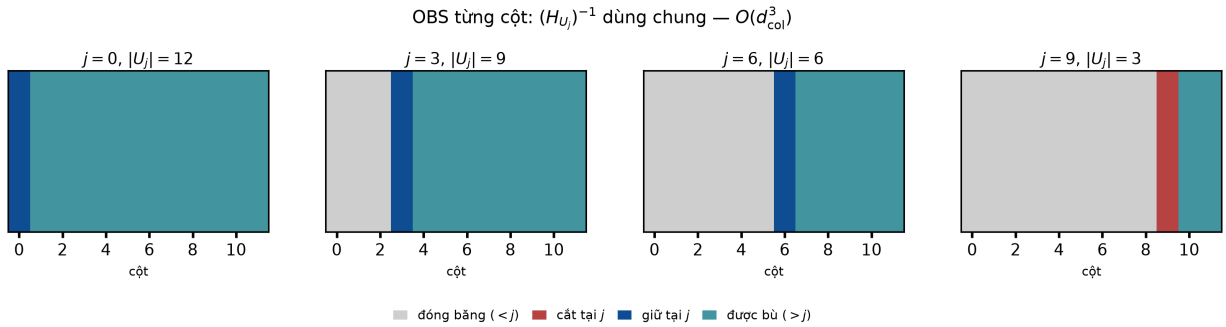
Cách nhìn lặp (OBS): cập nhật OBS khi xoá trọng số m (như dẫn xuất ở bài 22) là

$$\delta_m = -\frac{w_m}{[H^{-1}]_{mm}}H^{-1}_{:,m}, \qquad \varepsilon_m = \frac{w_m^2}{[H^{-1}]_{mm}} \tag{3}$$

(ở đây $H = XX^\top$; paper SparseGPT dùng H không nhân 2, hệ số nhân không ảnh hưởng w đã cập nhật). Vì loss một hàng chính xác bậc hai, OBS đúng tuyệt đối; xoá lần lượt $m_1, \ldots, m_p$ bằng OBS cho cùng nghiệm với giải đóng với mask đầy đủ.

3.2 Cập nhật một phần và đồng bộ Hessian

Nếu thay vì cập nhật mọi trọng số còn lại M, chỉ cập nhật tập con $U \subseteq M$ (dùng H_U thay H_M), OBS vẫn tối ưu **trong tập con đó**, không thêm sai số xấp xỉ riêng; chỉ phần bù có thể kém hơn vì ít trọng số để điều chỉnh.



SparseGPT xử lý theo cột: đóng băng $< j$, cắt/giữ tại j, bù vào $> j$ với $(H_{U_j})^{-1}$ dùng chung. (minh hoạ, tự tính)

Cố định thứ tự cột $j = 1, \ldots, d_{col}$ và định nghĩa

$$U_1 = \{1, \ldots, d_{col}\}, \qquad U_{j+1} = U_j - \{j\}.$$

Chuỗi nghịch đảo dùng chung cho mọi hàng: $(H_{U_j})^{-1} = ((XX^\top)_{U_j})^{-1}$, và mỗi bước cập nhật được tính bằng một bước khử Gauss (như Lemma 1 của bài 22):

$$\boxed{(H_{U_{j+1}})^{-1} = \left(B - \frac{1}{[B]_{11}} B_{:,1} B_{1,:}\right)_{2:,2:}}, \quad B = (H_{U_j})^{-1}$$

Mỗi bước $O(d_{col}^2)$; cả chuỗi $O(d_{col}^3)$ (bằng chi phí một phép nghịch đảo thêm).

Quy tắc xử lý: lần lượt qua các cột j; ở cột j, với **mọi hàng** i **có** $j \notin M_i$ (trọng số w_{ij} bị cắt) áp dụng OBS dùng $(H_{U_j})^{-1}$ để xoá w_{ij} và bù vào các trọng số ở cột $> j$ của hàng đó; các trọng số đã xử lý (cột $< j$) bị "đóng băng" và không còn được cập nhật. Mỗi nghịch đảo $(H_{U_j})^{-1}$ chỉ tính một lần và dùng cho mọi hàng.

Độ phức tạp: (a) tính Hessian ban đầu: $\Theta(n\, d_{col}^2)$, với n số mẫu hiệu chuẩn (paper thấy n cỡ một bội nhỏ của d_{col} là đủ); (b) chuỗi nghịch đảo: $O(d_{col}^3)$; (c) áp dụng cập nhật cho mọi hàng, mọi cột: $O(d_{col}\, d_{row}\, d_{col})$. Tổng:

$$\boxed{O(d_{col}^3 + d_{row}\, d_{col}^2)}$$

Với Transformer là $O(d_{hidden}^3)$, tức nhanh hơn tái tạo chính xác đúng d_{hidden} lần.

Cách diễn giải "đóng băng" (weight freezing): nén một trọng số nghĩa là cố định nó tại một giá trị (0 nếu cắt, giá trị hiện tại nếu giữ) rồi không bao giờ đổi nữa. Định nghĩa compress$(w_j)_i = 0$ nếu $j \notin M_i$, ngược lại giữ w_{ij}. SparseGPT khi đó là một lược đồ tham lam đúng theo cột (giống GPTQ). Cách nhìn này cho phép **gộp cắt tỉa và lượng tử hoá** trong một lượt.

3.3 Chọn mask thích nghi theo khối (iterative blocking)

Nếu dùng magnitude để chọn mask trước (như AdaPrune), bỏ qua việc các cập nhật làm thay đổi đáng kể trọng số. Cách đơn giản là chọn p% trọng số dễ cắt nhất trong **mỗi cột** khi tới lượt, nhưng như vậy ép mật độ đều theo cột; không hợp với LLM, vốn có vài đặc trưng (cột) outlier rất nhạy. Giải pháp:

- Chọn mask cho $B_s = 128$ **cột một lần**, theo sai số OBS $\varepsilon = w^2/[H^{-1}]_{cc}^2$ lấy từ đường chéo chuỗi Hessian (trong thuật toán dùng đường chéo của nhân tử Cholesky), giữ $(1-p)$% trọng số có ε lớn nhất trong khối.

- Rồi thực hiện cập nhật cho khối đó, sau đó chọn mask cho khối kế tiếp (đã tính các cập nhật trước).

Như vậy cột nào cũng có thể thưa khác nhau trong khối nhưng sparsity đều ở mức khối. (Với một cột đơn lẻ, tiêu chí thu về magnitude vì $[H^{-1}]_{jj}$ hằng với mọi hàng.)

3.4 Mở rộng N:M

Chọn $B_s = m$ và trong mỗi nhóm m cột liên tiếp của mỗi hàng, ép đúng n số 0 bằng cách chọn n trọng số có sai số (3) nhỏ nhất. Khối lớn hơn m không có ích với N:M vì các số 0 không được phân bố bất đều giữa các nhóm cột kích thước m.

3.5 Pseudo-code đầy đủ (Algorithm 1, tóm tắt)

```
# Input: W (d_row x d_col), Hinv = (X X^T + lambda*I)^{-1}, sparsity p,
#        lazy-update block B, mask block Bs (=128)
M = ones(d_row, d_col)                 # mask nhị phân
E = zeros(d_row, B)                    # sai số của khối
Hinv = Cholesky(Hinv)^T                # nhân tử trên; hàng j chứa thông tin chuỗi
↪  (H_Uj)^-1
for i in 0, B, 2B, ...:                # khối lazy-batch
    for j in i .. i+B-1:
        if j mod Bs == 0:
            # chọn mask cho Bs cột kế: giữ (1-p)% trọng số có w^2 / Hinv[c,c]^2 lớn nhất
            M[:, j:j+Bs] = top_(1-p)% theo  W[:, j:j+Bs]^2 / diag(Hinv)^2
        E[:, j-i] = W[:, j] / Hinv[j, j]          # sai số cắt (chuẩn hoá)
        E[:, j-i] = (1 - M[:, j]) * E[:, j-i]     # chỉ giữ ở vị trí bị cắt (đóng băng
        ↪  phần còn lại)
        W[:, j:i+B] -= E[:, j-i] * Hinv[j, j:i+B] # bù vào các cột bên phải trong khối
    W[:, i+B:] -= E @ Hinv[i:i+B, i+B:]           # lazy: bù vào phần còn lại, một lần
W = W * M                                         # đặt trọng số bị cắt về 0
```

Chi tiết cài đặt kế thừa từ GPTQ: phân rã **Cholesky** của H^{-1} cho phép chuẩn bị thông tin cần thiết cho chuỗi nghịch đảo một cách ổn định về số; cập nhật lazy theo khối B giúp tăng tỷ lệ tính toán so với truy cập bộ nhớ. Quan hệ giữa hàng j của nhân tử Cholesky và $(H_{U_j})^{-1}$ dựa trên tính chất đại số được GPTQ sử dụng; bài SparseGPT nêu lợi ích nhưng không trình bày lại phép dẫn xuất. Hessian được ổn định bằng cách cộng λI, với λ bằng 1% trung bình đường chéo.

3.6 Kết hợp lượng tử hoá

Vì cùng khung đóng băng theo cột như GPTQ, tất cả trọng số bị đóng băng được **lượng tử hoá luôn**, và sai số cần bù là (Phương trình 7):

$$E_{:,j-i} \leftarrow \frac{W_{:,j} - M_{:,j} \odot \text{quant}(W_{:,j})}{[H^{-1}]_{jj}}$$

(trong đó $M = 0$ tại vị trí cắt nên sai số là chính giá trị trọng số; $M = 1$ tại vị trí giữ nên sai số là sai số làm tròn.) Nhờ vậy các quyết định cắt và làm tròn ảnh hưởng lẫn nhau trong cùng một lượt, khác với OBC (cắt xong mới lượng tử hoá).

4. Thí nghiệm và kết quả chính

Thiết lập: OPT (125M đến 175B) và BLOOM-176B; mọi lớp linear được cắt đồng đều trừ embedding và head; một A100 80 GB; xử lý các lớp Transformer tuần tự; calibration = **128 đoạn 2048 token** lấy ngẫu nhiên từ shard đầu của C4 (zero-shot thực sự vì không thấy dữ liệu tác vụ); đánh giá perplexity trên raw-WikiText2, PTB, tập con C4, cộng zero-shot (LAMBADA, PIQA, ARC-easy/challenge, StoryCloze). Baseline: magnitude theo lớp; AdaPrune (chỉ đến ~1B).

Perplexity raw-WikiText2 trên OPT (Bảng 1):

OPT, 50%	125M	350M	1.3B
Dense	27.66	22.00	14.62
Magnitude	193.	97.80	1.7e4
AdaPrune	58.66	48.46	32.52
SparseGPT	36.85	31.58	17.46

OPT	Sparsity	2.7B	6.7B	13B	30B	66B	175B
Dense	0%	12.47	10.86	10.13	9.56	9.34	8.35
Magnitude	50%	265.	969.	1.2e4	168.	4.2e3	4.3e4
SparseGPT	50%	13.48	11.55	11.17	9.79	9.32	8.21
SparseGPT	4:8	14.98	12.56	11.77	10.30	9.65	8.45
SparseGPT	2:4	17.18	14.20	12.96	10.90	10.09	8.74

Ở 175B: perplexity tăng chỉ 0.11 (4:8) và 0.39 (2:4) so với dense (8.35 → 8.45, 8.74). Cắt 50% còn **thấp hơn dense một chút** (8.21 vs 8.35) nhưng paper nói đây là hiện tượng đặc thù tập dữ liệu: trên PTB và C4 thì không (PTB OPT-175B: dense 12.01, SparseGPT 50% 12.37; C4: 10.13 vs 10.36).

Quy mô lớn: OPT-175B, magnitude giữ chất lượng chỉ đến khoảng 10% sparsity rồi suy sụp; SparseGPT đạt 60% với tăng perplexity tương đương. BLOOM-176B dễ cho magnitude hơn (đến 30%). Ở 80% sparsity SparseGPT vẫn cho perplexity "hợp lý", magnitude sụp đổ (>100 perplexity) ở 40% (OPT) / 60% (BLOOM). Paper nhấn mạnh hơn 100 tỉ trọng số có thể bỏ. Thời gian: 175B trong khoảng 4 giờ (abstract: dưới 4.5 giờ), trong khi AdaPrune cần ≈1.3 giờ cho 350M và ≈4.3 giờ cho 1.3B.

Zero-shot OPT-175B (Bảng 2):

Phương pháp	Lamb.	PIQA	ARC-e	ARC-c	Story.	Avg.
Dense	75.59	81.07	71.04	43.94	79.82	70.29
Magnitude 50%	00.02	54.73	28.03	25.60	47.10	31.10
SparseGPT 50%	78.47	80.63	70.45	43.94	79.12	70.52
SparseGPT 4:8	80.30	79.54	68.85	41.30	78.10	69.62
SparseGPT 2:4	80.92	79.54	68.77	39.25	77.08	69.11

(LAMBADA cao hơn dense là nhiễu của đánh giá zero-shot, theo lời tác giả.)

Cắt + lượng tử hoá (Hình 6): 50% thưa + 4-bit có cùng bộ nhớ với 3-bit (lưu giá trị khác 0 + bitmask). OPT-175B: 8.29 (50% + 4-bit) vs 8.68 (GPTQ 3-bit); 2:4 + 4-bit: 8.55; 4:8 + 4-bit: 8.85; từ 2.7B trở lên cấu hình 50%+4bit chính xác hơn 3-bit. Phụ lục: 50% + 3-bit đạt 8.60 trên OPT-175B so với GPTQ 2.5-bit 8.94; 4:8 + 3-bit đạt 8.93.

Tăng tốc (Phụ lục E): CPU DeepSparse, OPT-2.7B, 1 batch 400 token: 40%/50%/60% thưa cho tăng tốc 1.57x / 1.82x / 2.16x. GPU 2:4 (CUTLASS so với cuBLAS dense, 2048 token, kích thước ma trận của OPT-175B): Q/K/V/Out 2.84 ms → 1.59 ms (1.79x); FC1 10.26 ms → 6.15 ms (1.67x); FC2 10.23 ms → 6.64 ms (1.54x).

5. Phân tích / ablation

- **Mô hình lớn dễ nén hơn** (hiện tượng chính): ở sparsity cố định, độ sụt giảm tương đối thu hẹp theo kích thước; tác giả suy đoán do dư tham số (overparametrization), chưa kiểm chứng.
- **Magnitude sụp đổ trên LLM** dù hiệu quả với CNN nhỏ, nhấn mạnh vai trò của bước bù trọng số.
- **Nhạy cảm theo tầng và theo loại lớp (partial 2:4, Hình 7):** nếu chỉ cắt 2/3 số lớp, **các lớp cuối nhạy hơn lớp đầu**; bỏ qua 1/3 lớp cuối cho độ chính xác tốt nhất. Hệ quả thực tế: một lượt SparseGPT cho phép tạo chuỗi mô hình thưa 2:4 một phần (1/2, 2/3, 3/4...) bằng cách ghép x lớp đầu đã cắt với các lớp sau còn dense. Ví dụ OPT-175B raw-WikiText2: dense 8.34, 1/2 8.22, 2/3 8.38, 3/4 8.49, 4/5 8.52, full 8.74 (Bảng 5).
- **Số mẫu hiệu chuẩn:** làm việc khá tốt với chỉ vài đoạn 2048 token, tốt hơn khi tăng nhưng đường cong phẳng nhanh; chọn 128 mẫu.
- **Dampening:** 0.001 đến 0.1 gần như tương đương; chỉ khi rất lớn mới xấu đi; chọn 1%.
- **Block size chọn mask:** blocksize 1 (từng cột) và rất lớn (4096-8192) đều kém rõ rệt; nhiều giá trị ở mức vài trăm hơi tốt hơn; chọn 128.
- **Độ nhạy theo seed lấy mẫu dữ liệu:** 13.52 ± 0.075 (5 lần), nên khá ổn định (OPT-2.7B, 50%).
- **Chất lượng xấp xỉ so với tái tạo chính xác (A.1, OPT-2.7B 50%):** sai số bình phương theo lớp của SparseGPT chỉ khoảng 20% tệ hơn tái tạo chính xác trung bình (một số ngoại lệ ở out-projection của attention các lớp đầu; ở các lớp FC2 cuối chỉ ~10%).

6. Giới hạn

Tác giả nêu hoặc ngầm nêu:

- Chỉ cắt trọng số (unstructured / N:M), mô hình lớn nhất vẫn có khoảng cách so với dense ở N:M với mô hình nhỏ (“accuracy loss relative to the dense baseline for smaller models”).
- Tăng tốc thực chỉ “preliminary”: CPU có DeepSparse, GPU 2:4 chỉ benchmark từng lớp; tăng tốc end-to-end chưa đo (“will likely be slightly lower”).
- Giải thích “mô hình lớn dễ nén hơn” chỉ là suy đoán.
- Perplexity là chỉ số nhạy; zero-shot nhiễu.

Nhận xét:

- Chỉ OPT và BLOOM (kiến trúc GPT cũ), chưa có LLaMA, mô hình lai hay VLM trong paper này.
- Kết quả tốt nhất ở các mô hình rất lớn (66B–175B). Với mô hình nhỏ hơn, mức mất chất lượng có thể đáng kể: ở độ thưa 50%, perplexity của OPT-125M tăng từ 27.66 lên 36.85 và của OPT-1.3B tăng từ 14.62 lên 17.46. Điều này gợi ý mô hình nhỏ có thể ít dư thừa hơn.
- Thứ tự cột cố định (giữ thứ tự gốc, không hoán vị) là lựa chọn tiện lợi; paper nói bất kỳ hoán vị nào cũng có thể dùng nhưng không khám phá.
- Mỗi lớp được cắt với đầu vào của mô hình đang được cắt tuần tự, nhưng mục tiêu tái tạo vẫn so với đầu ra dense của chính lớp đó, nên sai số lớp trước có thể tích luỹ.
- Calibration chỉ dùng C4 văn bản chung; chưa thử miền hẹp (ví dụ OCR/ảnh).

7. Liên hệ với các bài khác

- **01 (OBD/OBS) và 22 (OBC):** SparseGPT = OBS theo hàng (22) + chia sẻ Hessian qua thứ tự cột cố định + chọn mask khối + Cholesky lazy-batch của GPTQ. Công thức (3) giống hệt dẫn xuất ở 22.
- **18 (2:4):** SparseGPT cung cấp cách chọn mask 2:4 tốt hơn magnitude cho LLM không cần retrain; số tăng tốc 1.54-1.79x theo lớp khớp với chặn trên ~2x của bài 18.
- **21 (survey):** phân loại SparseGPT là unstructured và semi-structured pruning; Δ perplexity OPT-175B 50% = -0.14, 2:4 = +0.39 (bảng của survey, khớp với Bảng 1 ở đây).

- **24 (Wanda):** bỏ hẳn bước cập nhật trọng số và Hessian, chỉ dùng |w|·‖x‖; nhanh hơn nhưng không bù. So sánh trực tiếp nên có trong bài 24.
- **27 (LLM Surgeon), 25 (ZipLM):** mở rộng OBS sang nhiều dạng cấu trúc hơn (cắt cấu trúc, không chỉ trọng số lẻ).
- **29 (SliceGPT), 36 (ReplaceMe):** cũng tìm một ánh xạ tuyến tính tối ưu theo bình phương tối thiểu trên dữ liệu hiệu chuẩn để bù cho phần bị cắt.

8. Gợi ý áp dụng

- Gợi ý: khi cắt neuron MLP, thay vì chỉ cắt rồi distill, tính trước **hồi quy bình phương tối thiểu** để cập nhật W_{down} (cùng tinh thần SparseGPT/OBC, áp cho cắt nhóm); đây là suy diễn chưa kiểm.
- **Vai trò của bù trọng số:** SparseGPT cho thấy trên LLM, bước bù trọng số quan trọng hơn nhiều so với tiêu chí chọn (magnitude sụp đổ, SparseGPT giữ được).
- **Phần nén cuối (tuỳ chọn):** sau khi đã cắt cấu trúc và distill, SparseGPT 2:4 (+ lượng tử hoá cùng lượt) có thể là bước nén phụ, nếu phần cứng hỗ trợ.

9. Thí nghiệm gợi ý

2. Cắt 30% neuron của MLP ở 3 chế độ: (a) norm, không bù; (b) norm + bù bằng bình phương tối thiểu cho W_{down}; (c) (b) + distill ngắn.
3. Quét độ nhạy từng khối: áp 2:4 chỉ cho khối k (cả decoder lẫn ViT) và đo CER, để vẽ đường nhạy cảm giống Hình 7 và dùng làm cơ sở phân bổ ngân sách cắt.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / nội dung
Bài toán theo lớp	$\min \|WX - (M \odot \hat{W})X\|_2^2$ (NP-hard)
Nghiệm với mask cố định	$w_{M_i} = (X_{M_i} X_{M_i}^\top)^{-1} X_{M_i} (w_{M_i} X_{M_i})^\top$
OBS một trọng số	$\delta_m = -\frac{w_m}{[H^{-1}]_{mm}} H^{-1}_{:,m}$, $\varepsilon_m = \frac{w_m^2}{[H^{-1}]_{mm}}$
Chuỗi tập chỉ số	$U_1 = \{1..d_{col}\}$, $U_{j+1} = U_j - \{j\}$
Cập nhật nghịch đảo	$(H_{U_{j+1}})^{-1} = (B - \frac{1}{[B]_{11}} B_{:,1} B_{1,:})_{2:,2:}$
Chi phí	$O(d_{col}^3 + d_{row} d_{col}^2)$ so với $O(d_{row} d_{col}^3)$ của OBC
Chọn mask	Theo khối $B_s = 128$ cột, tiêu chí $w^2/[H^{-1}]_{cc}^2$
N:M	$B_s = m$, chọn n trọng số sai số nhỏ nhất mỗi nhóm
Kết hợp lượng tử hoá	Sai số $= (W - M \odot \text{quant}(W))/[H^{-1}]_{jj}$
Siêu tham số	128 mẫu × 2048 token (C4), dampening 1%, $B_s = 128$
Kết quả nổi bật	OPT-175B: 50% → 8.21 (dense 8.35), 2:4 → 8.74, 4:8 → 8.45; thời gian ~4 giờ trên A100; 50%+4bit 8.29 vs GPTQ 3-bit 8.68

11. Kết luận ngắn

SparseGPT biến phương pháp OBS chính xác nhưng đắt của OBC thành thuật toán chạy được trên 175B tham số, bằng cách cho mọi hàng dùng chung một chuỗi Hessian nghịch đảo (cập nhật một phần về bên phải), chọn mask theo khối và dùng Cholesky/lazy-batch của GPTQ. Kết quả: 50-60% thưa với ít mất chất lượng ở mô hình rất lớn, và kết hợp tự nhiên với lượng tử hoá.

Link gốc: https://arxiv.org/abs/2301.00774

Bài 24

A Simple and Effective Pruning Approach for Large Language Models (2023)

Tác giả: Mingjie Sun, Zhuang Liu, Anna Bair, J. Zico Kolter (CMU; Meta AI Research; Bosch Center for AI).
Venue / arXiv: arXiv:2306.11695 (20/06/2023, bản v3 05/2024). Published as a conference paper at ICLR 2024.
Nguồn: https://arxiv.org/abs/2306.11695
Chủ đề: Unstructured và N:M pruning sau huấn luyện cho LLM; điểm quan trọng của trọng số = |trọng số| × chuẩn L2 của activation đầu vào; so sánh theo từng output; không cần cập nhật trọng số, không cần gradient.
Vị trí trong lộ trình: Đọc sau 02 (Han 2015, magnitude pruning), 01 (OBD/OBS), 22 (OBC) và 23 (SparseGPT). Bài này cho thấy phần "đắt" của SparseGPT (cập nhật trọng số bằng nghịch đảo Hessian) phần lớn không cần thiết ở sparsity 50%: chỉ cần một điểm số rẻ hơn nhiều. Bài kế tiếp: 25 (ZipLM), 26 (LLM-Pruner), 28 (FLAP, mở rộng ý tưởng Wanda sang structured).
Chương nền liên quan: nền 02: Ma trận thưa và chi phí thật, nền 03: Độ quan trọng của trọng số, nền 04: Cắt hạt, cắt khối và mẫu 2, nền 05: Cắt rồi huấn luyện lại, nền 07: Đo đúng một kết quả pruning

> Ghi chú học tập bằng tiếng Việt, không phải bản dịch.

1. Bối cảnh

Vấn đề: LLM có hàng tỷ tham số. Quantization (lượng tử hoá, lưu trọng số bằng ít bit hơn) đã phát triển rất nhanh, còn pruning (xoá bớt trọng số) thì ít được chú ý cho LLM. Lý do theo tác giả: các phương pháp pruning cũ thường cần huấn luyện lại, hoặc huấn luyện từ đầu, hoặc lặp nhiều vòng (iterative), mà với LLM thì không đủ tài nguyên.

Các điểm cần biết trước:

- **Magnitude pruning** (Han 2015, bài 02): xoá trọng số có $|W_{ij}|$ nhỏ nhất, so sánh trong cả layer hoặc cả mạng. Trước LLM rất mạnh, nhưng SparseGPT (bài 23) báo rằng nó **sụp đổ ngay ở sparsity thấp** trên LLM.
- **SparseGPT** (bài 23): coi pruning là bài toán tái tạo đầu ra từng layer (layer-wise reconstruction), dùng thông tin bậc hai kiểu OBS (bài 01) và *cập nhật các trọng số còn lại* để bù. Tốt nhưng tốn tính toán (có nghịch đảo Hessian).
- **Calibration data** (dữ liệu hiệu chỉnh): một ít chuỗi văn bản (ở đây 128 chuỗi từ C4) chạy qua mô hình để đo thống kê activation. Không dùng để huấn luyện.
- **Emergent large-magnitude features** (Dettmers 2022): khi LLM đủ lớn (khoảng 6B), một vài chiều của hidden state có độ lớn gấp khoảng 100 lần bình thường, thưa theo chiều, và nếu zero chúng thì mô hình hỏng. Những "outlier channel" này là lý do quantization khó trên LLM.

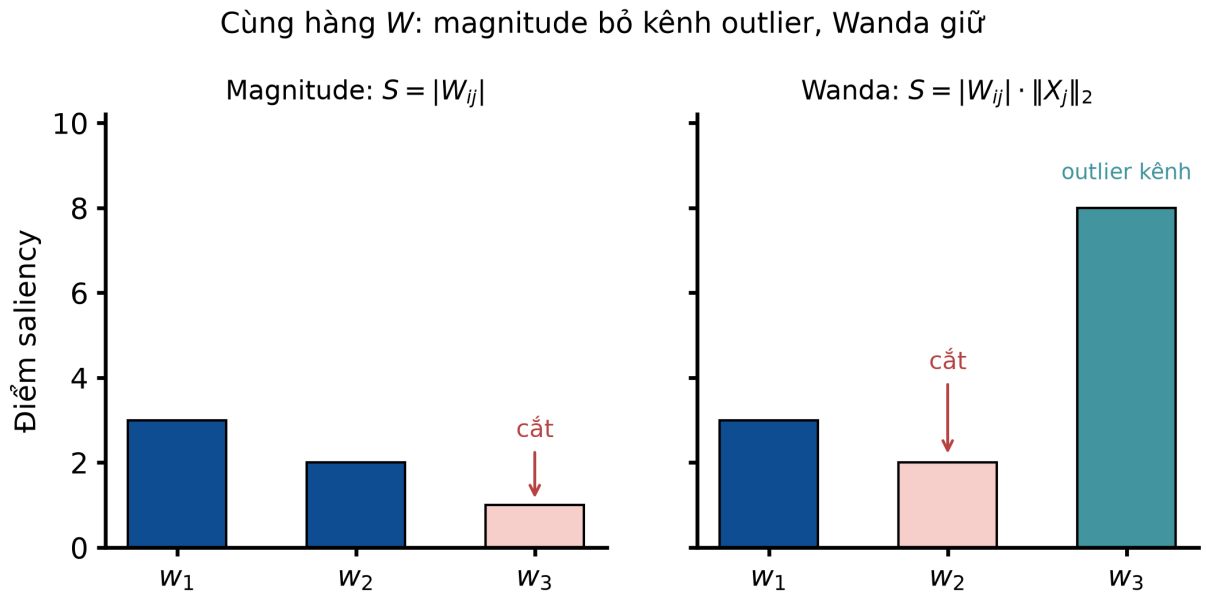
2. Ý tưởng chính

Xét một neuron $y = w_1x_1 + w_2x_2$ với $|w_1| \leq |w_2|$. Magnitude pruning luôn bỏ w_1. Nhưng nếu đầu vào x_1 lớn gấp trăm lần x_2 (giống outlier channel của LLM) thì w_1x_1 mới là số hạng quan trọng, và nên bỏ w_2. Wanda (Pruning by **W**eights **and A**ctivations) chấm điểm mỗi trọng số bằng **|trọng số| × chuẩn L2 của kênh đầu vào tương ứng** (đo trên calibration data), rồi **trong từng hàng của ma trận (mỗi output neuron)** bỏ đi s% trọng số có điểm thấp nhất. Không cập nhật trọng số còn lại, không backprop, không nghịch đảo ma trận; chỉ một lượt forward. Kết quả ngang SparseGPT ở 50% sparsity với chi phí tính điểm thấp hơn rất nhiều.

3. Phương pháp chi tiết

3.1 Điểm quan trọng

Layer tuyến tính có $W \in \mathbb{R}^{C_{out} \times C_{in}}$, đầu vào $X \in \mathbb{R}^{(N \cdot L) \times C_{in}}$ (N = batch, L = độ dài chuỗi). Điểm của trọng số W_{ij}:



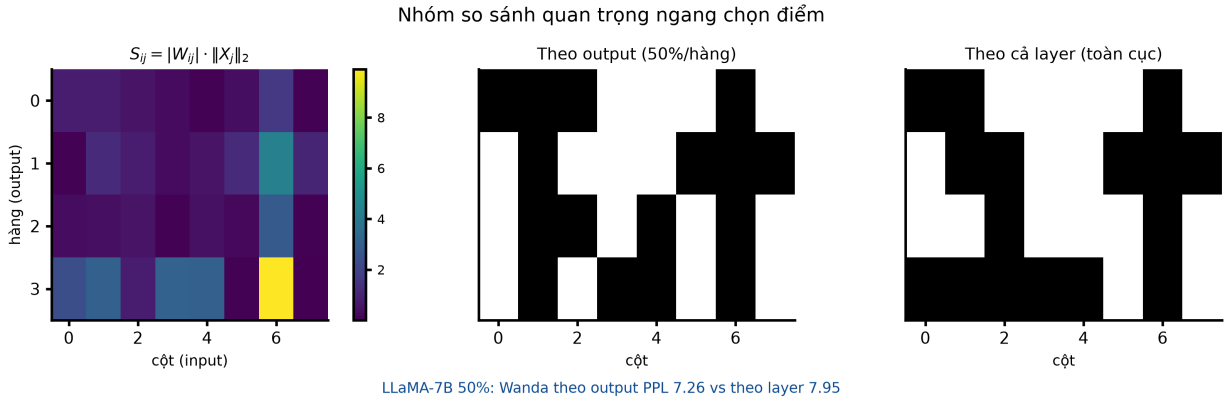
Cùng một hàng W: magnitude cắt kênh outlier còn Wanda ($|W| \cdot \|X\|_2$) giữ kênh có activation lớn. (minh hoạ, tự tính)

$$\boxed{S_{ij} = |W_{ij}| \cdot \|X_j\|_2}$$

trong đó $\|X_j\|_2$ là chuẩn L2 của cột j (kênh đầu vào j) gộp trên mọi token. Paper thử chuẩn ℓ_1, ℓ_∞ và thấy ℓ_2 tốt hơn (tác giả đoán vì ℓ_2 "mượt" hơn).

3.2 Nhóm so sánh (comparison group)

Sau khi có điểm, phải quyết định "so với ai thì bị coi là thấp". Các lựa chọn: cả mạng, từng layer, hoặc **từng output** (từng hàng i của W):



Wanda so sánh theo từng hàng output vs theo cả layer; note: PPL 7.26 vs 7.95 trên LLaMA-7B 50%. (minh hoạ, tự tính)

$$G_{ij} = \{W_{uv} \mid u = i\}$$

Wanda bỏ đúng s% trọng số trong *mỗi hàng*. Nghe như ràng buộc cứng hơn (pattern thưa chặt hơn), nhưng paper thấy nó **luôn tốt hơn so sánh theo cả layer** cho LLM, kể cả với điểm magnitude thuần. Trên bộ phân loại ảnh (ConvNeXt, DeiT) thì xu hướng này **không** xuất hiện, nên có thể là đặc thù của LLM (tác giả nói vậy).

3.3 Pseudo-code

```
Wanda(W, X, s):                          # một layer tuyến tính
    norm   = sqrt(sum_over_tokens(X^2), axis=tokens)   # (C_in,)  ||X_j||_2
    metric = |W| * norm[None, :]                       # (C_out, C_in)
    for each row i:
        idx = argsort(metric[i, :])[: int(C_in * s)]   # s% điểm thấp nhất trong hàng
        W[i, idx] = 0
    return W

Toàn mô hình: duyệt layer từ đầu đến cuối; sau khi cắt layer l,
layer l+1 nhận activation tính trên mạng đã cắt. Bỏ embedding và đầu phân loại.
```

(Paper cho đúng 5 dòng PyTorch ở Algorithm 1.) Mở rộng **N:M** (ví dụ 2:4): trong mỗi hàng, so sánh điểm trong từng nhóm M trọng số liên tiếp và giữ N trọng số cao nhất.

Ví dụ số tự dựng (không phải của paper): một hàng $W_i = [3, -2, 1]$, chuẩn kênh $\|X_j\|_2 = [1, 1, 8]$. Magnitude: điểm $[3, 2, 1]$, bỏ 1/3 hàng thì bỏ trọng số thứ 3. Wanda: điểm $[3, 2, 8]$, bỏ trọng số thứ 2. Kênh 3 là outlier channel nên Wanda giữ nó.

3.4 Liên hệ với SparseGPT và OBD

Điểm của SparseGPT: $S_{ij} = |W_{ij}|^2 / [(X^\top X + \lambda I)^{-1}]_{jj}$. Nếu đặt $\lambda = 0$ và chỉ giữ đường chéo của $H = X^\top X$ (bỏ các phần tử ngoài đường chéo):

$$S_{ij} = \frac{|W_{ij}|^2}{[(X^\top X)^{-1}]_{jj}} \xrightarrow{\text{diag}} |W_{ij}|^2 \, \|X_j\|_2^2 = (|W_{ij}| \cdot \|X_j\|_2)^2$$

tức bình phương điểm Wanda. Cho nên Wanda là phiên bản "chỉ dùng đường chéo Hessian, không nghịch đảo, không cập nhật" của SparseGPT, giống như OBD (chỉ đường chéo, bài 01) là bản đơn giản của OBS. Tác giả viết: Wanda là "renaissance" của OBD, áp dụng cho từng neuron với mục tiêu tái tạo đầu ra cục bộ, tương tự quan hệ giữa SparseGPT và OBS.

Phương pháp	Cập nhật trọng số	Cần calibration	Điểm	Độ phức tạp (paper)
Magnitude	không	không	$\|W_{ij}\|$	$O(1)$
SparseGPT	có	có	$\|W\|^2 / \text{diag}(H^{-1})$	$O(d^3_{hidden})$
Wanda	không	có	$\|W_{ij}\| \cdot \|X_j\|_2$	$O(d^2_{hidden})$

4. Thí nghiệm và kết quả chính

Mô hình: LLaMA 7B/13B/30B/65B và LLaMA-2 7B/13B/70B (kết quả OPT, BLOOM, Pythia ở phụ lục). **Calibration:** 128 chuỗi (độ dài context) từ tập train C4, giống SparseGPT. **Cắt:** mọi linear layer (bỏ embedding và đầu cuối; khoảng 99% tham số), sparsity đồng đều. **Đánh giá:** perplexity (PPL) WikiText và trung bình 7 tác vụ zero-shot (EleutherAI LM Harness). Không fine-tune.

Zero-shot, độ chính xác trung bình (%) (Table 2, trích):

Phương pháp	Sparsity	LLaMA-7B	LLaMA-13B	LLaMA-65B	LLaMA-2-7B	LLaMA-2-70B
Dense	0%	59.99	62.59	66.97	59.71	67.08
Magnitude	50%	46.94	47.61	62.74	51.14	60.93
SparseGPT	50%	54.94	58.61	66.30	56.24	67.28
Wanda	50%	54.21	59.33	66.67	56.24	67.03
SparseGPT	2:4	50.60	53.22	62.57	50.94	63.89
Wanda	2:4	48.53	52.30	62.84	48.75	64.14

WikiText PPL (Table 3, trích):

Phương pháp	Sparsity	LLaMA-7B	LLaMA-13B	LLaMA-65B	LLaMA-2-7B	LLaMA-2-70B
Dense	0%	5.68	5.09	3.56	5.12	3.12
Magnitude	50%	17.29	20.21	5.90	14.89	4.98
SparseGPT	50%	7.22	6.21	4.57	6.51	3.98
Wanda	50%	7.26	6.15	4.57	6.42	3.98
Magnitude	2:4	42.13	18.37	7.11	54.59	6.33
SparseGPT	2:4	11.00	9.11	6.28	10.17	5.40
Wanda	2:4	11.53	9.58	6.25	11.02	5.16

Nhận xét của paper: ở sparsity 50% Wanda ngang SparseGPT; 2:4 thì lẫn lộn (mô hình nhỏ 7B SparseGPT hơn, mô hình lớn như LLaMA-30B và LLaMA-2-70B Wanda hơn); LLaMA-65B và LLaMA-2-70B ở 50% unstructured **khớp zero-shot của bản dense**; khoảng cách dense/sparse nhỏ dần khi mô hình lớn. "Large sparse vs small dense": LLaMA-65B 50% (66.67) hơn LLaMA-30B dense (65.38) với unstructured; với structured (N:M) xu hướng đảo ngược, mô hình lớn thưa không fine-tune kém mô hình nhỏ dense.

Tốc độ: thời gian tính điểm (giây, không tính forward chung, A6000): SparseGPT 203.1 / 339.0 / 810.3 / 1353.4 cho 7B/13B/30B/65B so với Wanda 0.54 / 0.91 / 2.9 / 5.6 (paper nói "có thể nhanh hơn 300 lần"). **Suy luận (inference) với 2:4**: nhân ma trận LLaMA-65B (batch 1) nhanh khoảng 1.6x cho các linear layer (q/k/v/o: 3.49 ms đến 2.14 ms, 1.63x; up/gate: 1.61x; down: 1.54x); end-to-end LLaMA-7B 251 ms so với 312 ms (1.24x). Tác giả nhấn mạnh tăng tốc này không riêng của Wanda mà của 2:4.

Sparsity cao hơn: phụ lục có 60% và 80%; ở 60% Wanda vẫn cạnh tranh; ở 80% SparseGPT tốt hơn rõ, nhưng cả hai (best 25.86 PPL) còn xa dense LLaMA-7B (5.68).

5. Phân tích / ablation

- **Fine-tuning sau khi cắt (Table 6, LLaMA-7B, mask cố định):** zero-shot: 50% unstructured 54.21, 4:8 52.76, 2:4 48.53 (không tune); LoRA (rank 8, 1 GPU 12 giờ, khoảng 0.06% tham số thêm) lên 56.53 / 54.87 / 54.46; full fine-tune dense (4 GPU 3 ngày) lên 58.15 / 56.65 / 56.19 (dense 59.99). PPL: 7.26 / 8.57 / 11.53 xuống 6.84 / 7.29 / 8.24 (LoRA) và 5.98 / 6.63 / 7.02 (full). LoRA đưa 2:4 (54.46) vượt cả 50% unstructured không tune (54.21).
- **Cấu hình cắt (Table 7, PPL LLaMA-7B 50%):** Wanda với nhóm "theo output" 7.26 (tốt nhất); nhóm "theo layer" 7.95. Với magnitude: theo layer 17.29, nhóm (input, 1) 8.86, (output, 1) 16.82. Tức là **chọn nhóm so sánh quan trọng ngang chọn điểm**. Lưu ý (input,1) buộc so sánh các trọng số cùng kênh đầu vào, khi đó ba phương pháp cho kết quả như nhau vì $\|X_j\|$ là hằng.

- **Cập nhật trọng số (Table 8):** với Wanda, update bậc hai kiểu SparseGPT gần như không giúp ở 50% (7.26 sang 7.32 hoặc 7.26) và 4:8; ở 2:4 giúp nhẹ (11.53 sang 10.89, tốt hơn SparseGPT 11.00). Với magnitude, update giúp rất nhiều (17.59 xuống 12.56). Ở sparsity 70%, update đưa Wanda từ 84.50 xuống 29.65, nhưng vẫn kém xa dense 5.68.
- **Số mẫu calibration (Fig. 2):** Wanda ổn định hơn SparseGPT khi ít dữ liệu; chỉ 1 mẫu vẫn cho PPL 7.66. Giải thích của tác giả: chuẩn $\|X_j\|$ dễ ước lượng hơn nghịch đảo Hessian đầy đủ.
- **Các họ khác (phụ lục):** nhóm theo output cũng tốt hơn theo layer trên OPT và BLOOM; với OPT SparseGPT đôi khi nhỉnh hơn Wanda.

6. Giới hạn

Do tác giả nêu:

- Không fine-tune thì vẫn có khoảng cách rõ so với dense (nhất là mô hình nhỏ và ở N:M).
- Ở sparsity rất cao (70%, 80%) thiếu cập nhật trọng số làm Wanda kém xa; update giúp nhiều ở đó.
- Quan sát "nhóm theo output tốt hơn" chưa giải thích được và không đúng với bộ phân loại ảnh.

Do ta nhận xét:

- Đây là **unstructured/N:M**, không giảm kích thước ma trận. Muốn tăng tốc thật cần kernel sparse (2:4 trên GPU NVIDIA đời mới) và lợi ích bộ nhớ chỉ có khi lưu định dạng nén.
- Chỉ dùng đường chéo của $X^\top X$: bỏ qua mối tương quan giữa các kênh (hai kênh đầu vào giống nhau cùng được giữ). Đây là chỗ OBS/SparseGPT/ZipLM hơn.
- Điểm theo từng layer cục bộ, không nhìn thấy gradient/loss cuối, và không phân bổ sparsity khác nhau giữa các layer.
- Thí nghiệm chỉ trên LM text thuần; không có mô hình thị giác-ngôn ngữ hay kiến trúc lai.

7. Liên hệ với các bài khác

- **01 (OBD/OBS) và 23 (SparseGPT):** Wanda là "OBD-hoá" SparseGPT, mục 3.4.
- **18 (2:4 sparsity):** Wanda là cách chọn mask 2:4 rẻ không cần retrain. **30 (MaskLLM)** so sánh trực tiếp và thắng nhờ học mask bằng gradient.
- **28 (FLAP):** lấy khung "metric dựa trên activation + trọng số" nhưng chuyển sang structured và thêm bù bias; FLAP dùng "Wanda-sp" làm baseline (tổng $|W_{ij}| \cdot \|X_j\|_2$ theo cột) và chỉ ra nó sụp ở 50%.
- **25 (ZipLM), 27 (LLM Surgeon), 29 (SliceGPT):** các phương pháp dùng nhiều thông tin hơn đường chéo (tương quan, gradient, PCA) để làm được structured.
- **07 (Molchanov 2019):** cũng là importance dựa trên đóng góp vào đầu ra, nhưng bằng Taylor bậc nhất.

8. Gợi ý áp dụng

- Sparsity không cấu trúc không rút gọn ma trận; 2:4 chỉ cho khoảng 1.2 đến 1.6x và vẫn giữ kích thước tham số gần như gấp đôi nếu lưu dense.
- Bảng 8 của paper (update giúp lớn khi sparsity cao) ủng hộ giả thuyết rằng *bù tuyến tính càng quan trọng khi cắt càng mạnh*.
- Wanda dùng được như **công cụ chẩn đoán rẻ**: tính $\|X_j\|$ cho từng linear layer trong 1 forward pass để xem kênh nào "nóng".

9. Thí nghiệm gợi ý

3. **Nhóm so sánh:** thử xếp hạng neuron theo từng layer so với xếp hạng toàn cục (đã chuẩn hoá) để thấy ảnh hưởng của "nhóm so sánh" ở dạng structured.

10. Tóm tắt công thức / khái niệm

Khái niệm	Nội dung
Điểm Wanda	$S_{ij} = \|W_{ij}\| \cdot \|X_j\|_2$
Nhóm so sánh	Từng hàng (output neuron); bỏ s% điểm thấp nhất mỗi hàng
Quan hệ SparseGPT	Wanda² $= \|W\|^2/\text{diag}(X^\top X)^{-1}$ với $\lambda = 0$ và Hessian chỉ đường chéo, bỏ cập nhật trọng số
N:M	Trong mỗi nhóm M trọng số liên tiếp giữ N điểm cao nhất
Outlier / emergent features	Vài kênh hidden có độ lớn khoảng 100x, quan trọng cho mô hình
Calibration data	128 chuỗi C4; 1 mẫu cũng cho PPL 7.66 trên LLaMA-7B
Con số chính	LLaMA-7B 50%: PPL 7.26 (magnitude 17.29, SparseGPT 7.22); tính điểm LLaMA-7B 0.54 s so với 203.1 s

11. Kết luận ngắn

Wanda cho thấy ở sparsity vừa phải, phần lớn chất lượng của pruning LLM đến từ hai lựa chọn đơn giản: nhân độ lớn trọng số với độ lớn activation, và so sánh theo từng output. Không cần Hessian nghịch đảo hay cập nhật trọng số.

Link gốc: https://arxiv.org/abs/2306.11695

Bài 25

ZipLM: Inference-Aware Structured Pruning of Language Models (2023)

Tác giả: Eldar Kurtic, Elias Frantar, Dan Alistarh (IST Austria; Alistarh cũng ở Neural Magic).
Venue / arXiv: arXiv:2302.04089 (07/02/2023, bản v2 10/2023). NeurIPS 2023 (ghi ở trang đầu bản text).
Nguồn: https://arxiv.org/abs/2302.04089
Chủ đề: Structured pruning (attention head, chiều trung gian của FFN, cả module) cho BERT và GPT-2; chấm điểm bằng sai số tái tạo kiểu OBS **có bù trọng số**, rồi chọn cấu hình theo từng layer sao cho **đạt đúng mục tiêu tốc độ đo trên thiết bị thật** (latency table); thêm distillation token-level.
Vị trí trong lộ trình: Đọc sau 01 (OBD/OBS), 22 (OBC), 23 (SparseGPT), 17 (CoFi, đối thủ chính) và 24 (Wanda). Bài này là phiên bản *structured, inference-aware* của dòng OBS/SparseGPT: đóng góp then chốt là (i) công thức OBS cho cả nhóm cột cùng lúc, và (ii) biến "sparsity mục tiêu" thành "latency mục tiêu". Bài kế: 26 (LLM-Pruner), 27 (LLM Surgeon), 38 (Sheared LLaMA).

Ghi chú học tập bằng tiếng Việt, không phải bản dịch. Số liệu lấy từ paper (một số chỗ chỉ có trong hình, ghi rõ).

1. Bối cảnh

Khi cắt các cấu trúc lớn như attention head hoặc cột ma trận, mô hình có thể rất nhạy; trước bài này các phương pháp tốt nhất (Block Movement Pruning, CoFi bài 17) phải **tích hợp pruning vào quá trình huấn luyện** (mask học được, xấp xỉ khả vi) chứ không làm kiểu "cắt từng bước rời rạc rồi fine-tune" (gradual pruning, bài 03). Ba vấn đề cụ thể mà paper nêu:

1. **Tiêu chí chọn cấu trúc thô:** thường chỉ dựa vào một trong ba giả định: trọng số nhỏ (magnitude), activation đầu vào nhỏ, hoặc cấu trúc gần như là tổ hợp tuyến tính của cấu trúc khác (dư thừa tuyến tính).
2. **Không đảm bảo tốc độ:** các phương pháp cắt đến một mức sparsity/FLOPs cố định rồi mới đo. Ví dụ paper: BERT thưa 95% của CoFi nhanh 12x trên V100 nhưng chỉ 5x trên A100. Nếu không đạt yêu cầu phải làm lại từ đầu.
3. **Distillation khó với mô hình bị cắt:** cần ghép layer thầy-trò thủ công hoặc động, và ma trận chiếu để khớp kích thước.

Khái niệm cần biết: **OBS** (Optimal Brain Surgeon, bài 01) chọn trọng số để xoá và *cập nhật các trọng số còn lại* để bù, dùng nghịch đảo Hessian; **latency table** là bảng thời gian chạy đo thật của từng module ở từng cấu hình (số head, kích thước FFN) trên thiết bị đích.

2. Ý tưởng chính

ZipLM kết hợp hai bước. Thứ nhất, với mỗi lớp, nó xoá từng cấu trúc một (từng head, từng cột FFN) theo thứ tự "ít hại nhất", **mỗi lần xoá đều cập nhật các cột còn lại** để bù, rồi cập nhật nghịch đảo Hessian; nhờ vậy các cấu trúc dư thừa lẫn nhau không bị xoá cùng lúc. Thứ hai, nó đo trước thời gian chạy của từng module ở mọi mức cắt (latency table) và dùng thuật toán tìm kiếm (SPDY) để chọn "mức cắt của từng layer" sao cho tổng thời gian đúng bằng mục tiêu (ví dụ nhanh gấp 5x), thay vì nhắm vào một tỷ lệ sparsity. Cắt, tinh chỉnh (kèm distillation theo token ở mọi layer), rồi lặp để ra cả một họ mô hình 2x, 3x, ... 15x trong một lần chạy.

3. Phương pháp chi tiết

3.1 Sai số tái tạo cục bộ và điểm cho một cấu trúc

Với một layer tuyến tính có trọng số $W \in \mathbb{R}^{d_{row} \times d_{col}}$ và đầu vào calibration X, ta muốn trọng số nén $\widehat{W}$ (thoả ràng buộc cấu trúc) sao cho

$$\min_{\widehat{W}} \|WX - \widehat{W}X\|_2^2.$$

Với unstructured, bài toán tách theo từng hàng. Với **structured, cùng một tập chỉ số cột phải bị xoá ở mọi hàng**, nên các hàng phụ thuộc nhau. Gọi $H = XX^\top$ (Hessian của bài toán bình phương tối thiểu, không phụ thuộc W), S là một cấu trúc (ví dụ $d_{head} = 64$ cột liên tiếp) và M_S là tập chỉ số cột của nó. Mở rộng công thức OBS cho mọi hàng cùng lúc, ZipLM chọn cấu trúc bằng:

$$\boxed{\text{err}(S) = \sum_{i=0}^{d_{row}} W_{i,M_S} \Big([(H^{-1})_{M_S,M_S}]^{-1} \Big) W_{i,M_S}^\top} \qquad (2)$$

và cập nhật bù cho **tất cả cột còn lại**:

$$\boxed{\delta_S = -W_{:,M_S}[(H^{-1})_{M_S,M_S}]^{-1}(H^{-1})_{M_S,:}} \qquad (3)$$

Ma trận nhỏ $[(H^{-1})_{M_S,M_S}]^{-1}$ dùng chung cho mọi hàng nên chi phí thêm chỉ $O(d_{col} \cdot |M_S|^2)$. Phương trình (2) gộp ba tiêu chí cũ: độ lớn trọng số (W), tác động activation (qua H), và dư thừa tuyến tính (qua phần ngoài đường chéo của H^{-1}).

3.2 Xoá từng cấu trúc một (tương quan trong cùng layer)

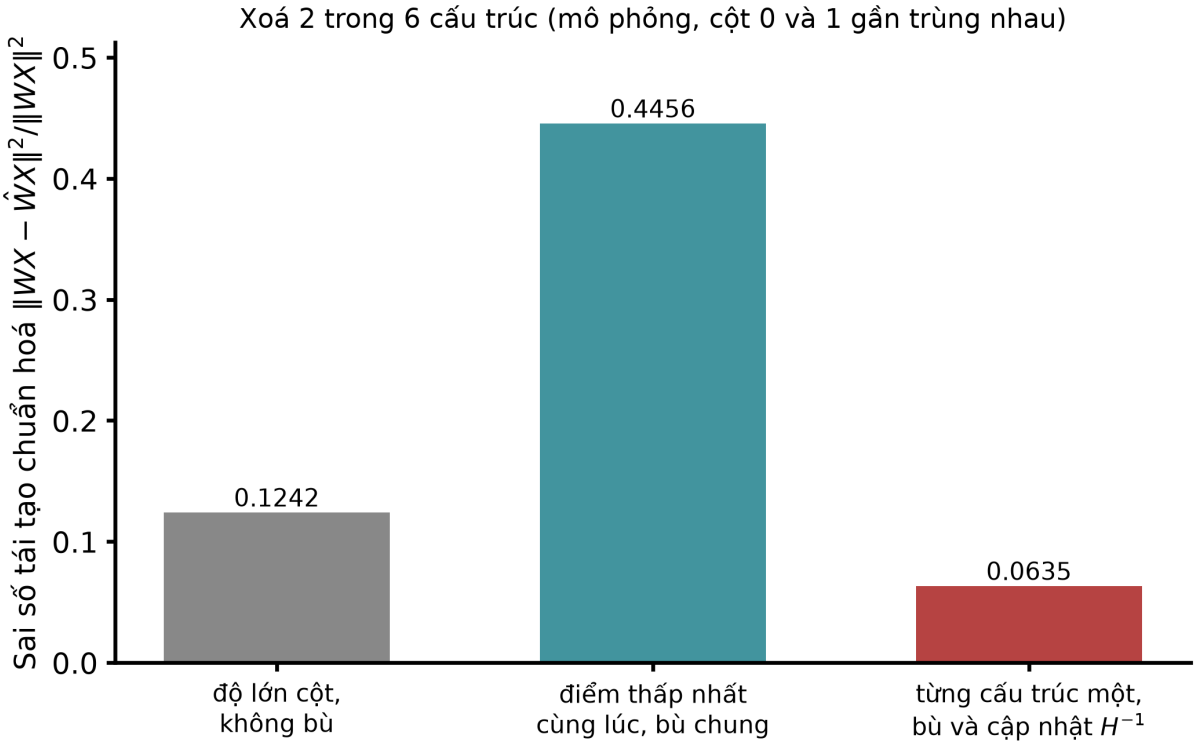
Chọn theo (2) cho mọi cấu trúc cùng lúc vẫn bỏ qua tương quan giữa các cấu trúc: hai cấu trúc S_1, S_2 trùng nhau đều có điểm thấp và cùng bị xoá. Vì vậy ZipLM **xoá từng cấu trúc một**: xoá S_1, áp dụng δ_{S_1}, rồi tính lại điểm; lúc đó S_2 đã gánh phần của S_1 nên không còn dễ xoá. Tính lại nghịch đảo từ đầu tốn $O(d_{col}^3)$, nên dùng một bước khử Gauss theo khối:

$$H^{-1} \leftarrow H^{-1} - H^{-1}_{:,M_S}[(H^{-1})_{M_S,M_S}]^{-1} H^{-1}_{M_S,:} \qquad (4)$$

chỉ tốn $O(|M_S|\, d_{col}^2)$.

```
ZipLM_layer(W, H_inv, k):              # xoá đúng k cấu trúc
    R = tập mọi cấu trúc (head hoặc cột FFN)
    repeat k times:
        S = argmin_{S in R}  sum_i W[i,M_S] @ inv(H_inv[M_S,M_S]) @ W[i,M_S]^T
        delta = - W[:,M_S] @ inv(H_inv[M_S,M_S]) @ H_inv[M_S,:]
        W  += delta                                  # bù vào cột còn lại
        H_inv -= H_inv[:,M_S] @ inv(H_inv[M_S,M_S]) @ H_inv[M_S,:]
        R -= {S}
    W *= mask(R)                                      # đưa cột đã xoá về đúng 0
```

Các cấu trúc bị cắt (Transformer): (i) bỏ head attention (xoá d_{head} cột liền nhau của ma trận out-projection của attention), (ii) thu nhỏ chiều trung gian của FFN (xoá cột lẻ ở ma trận FC2), (iii) bỏ nguyên module attention hoặc FFN (phần residual). Sau khi cột của ma trận *sau* bị đưa về 0, hàng tương ứng của ma trận *trước* xoá được mà không đổi đầu ra. Tác giả cố ý cắt cột của FC2 chứ không phải hàng của FC1, vì như vậy mới tận dụng được tương quan đầu vào qua Hessian.



Hình 51: Mô phỏng xoá 2 trong 6 cấu trúc (cột 0 và 1 gần trùng nhau): độ lớn cột, chọn đồng thời rồi bù chung, và xoá từng cấu trúc một (minh hoạ, tự tính)

Cách đọc hình: số mô phỏng (seed cố định), không phải kết quả của paper. Cột 0 và 1 gần trùng nhau nên mỗi cột riêng lẻ có điểm thấp vì cột kia bù được. Chọn đồng thời xoá cả hai và sai số nhảy lên 0,4456; xoá từng cái một thì sau khi xoá cột đầu, cột kia không còn rẻ nữa nên được giữ, sai số chỉ còn 0,0635.

3.3 Chọn cấu trúc dưới ràng buộc latency (phần quan trọng)

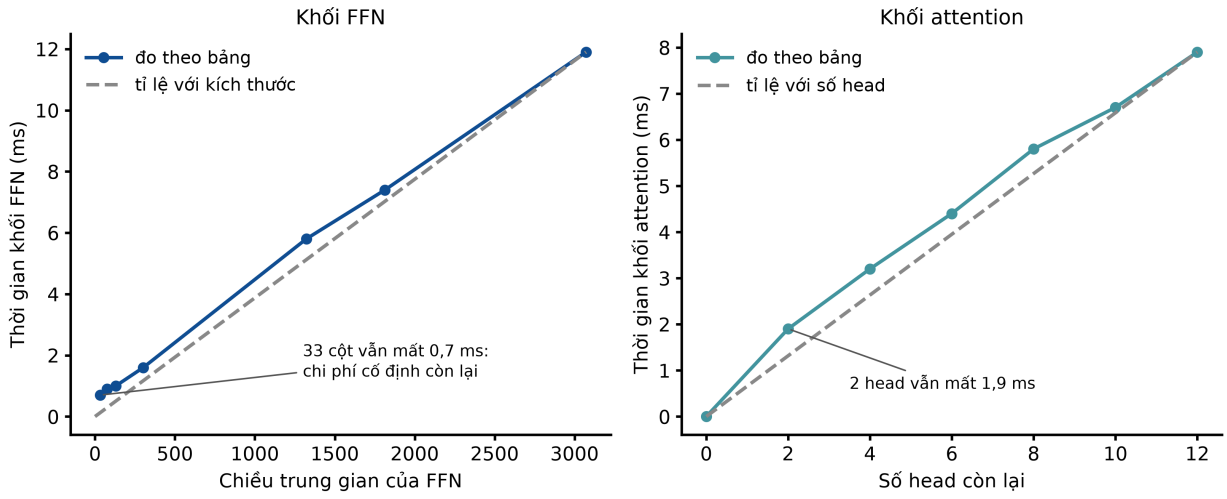
Đây là phần liên quan nhất đến câu hỏi "chọn cấu trúc bằng sai số OBS-có-bù dưới ràng buộc độ trễ". Quy trình, theo đúng paper:

1. **Latency table.** Với thiết bị, batch và độ dài chuỗi đích, đo thời gian chạy:
   - khối attention (gồm mọi overhead) khi bỏ $0, 1, \ldots, N_{heads} - 1$ head (bỏ hết = 0 ms cũng được tính);
   - khối FFN khi chiều trung gian thu nhỏ theo hệ số 0.9^i, $i = 0, \ldots, 42$ (bước 10% đến khoảng 99% sparsity). Ví dụ trong phụ lục (Table 7, paper không nêu thiết bị cho bảng này; kích thước là của BERT-base): FFN 3072 → 11.9 ms, 1814 → 7.4, 1322 → 5.8, 302 → 1.6, 130 → 1.0, 76 → 0.9, 33 → 0.7; attention 12 head → 7.9 ms, 10 → 6.7, 8 → 5.8, 6 → 4.4, 4 → 3.2, 2 → 1.9, 0 → 0. Bảng đo ở mức module (không phải từng toán tử), nên độc lập với tối ưu hoá của engine (fuse toán tử, v.v.). Ta hiểu thời gian cả mô hình là tổng thời gian các module (paper nói bảng cho phép "ước lượng nhanh" cho từng cấu hình per-layer; đo thật lệch tối đa 5.28% so với dự kiến, Table 8).
2. **Cơ sở dữ liệu các phiên bản đã cắt.** Vì ZipLM xử lý mỗi layer độc lập và xoá từng cấu trúc một, **một lần chạy** cho ra, với mỗi layer, mọi mức thưa (đã bù trọng số). Mỗi lựa chọn (layer ℓ, mức s) có hai thuộc tính: độ trễ (tra bảng) và sai số.
3. **Tìm cấu hình per-layer.** Cần chọn một mức s_ℓ cho từng layer sao cho tổng độ trễ đạt mục tiêu và độ chính xác cao nhất. Paper dùng **SPDY** (Frantar & Alistarh 2022) với hai sửa đổi cho structured:
   - SPDY gốc gán *prior bậc hai* cho độ nhạy theo mức thưa, không hợp với structured (nó ngụ ý bỏ

cả layer chỉ khó hơn một chút so với thưa 99%). ZipLM thay prior bằng **sai số tương đối của layer**: $p_s = \|\widehat{W}_s X - WX\|^2 / \|WX\|^2$, bằng 1 khi bỏ hẳn layer. Đây chính là **sai số sau khi đã bù OBS**.

- SPDY gốc dùng "shrinking neighborhood search" (phương sai lớn với structured); ZipLM chạy **cố định 1000 bước**, mỗi bước đột biến ngẫu nhiên trung bình 10% hệ số độ nhạy theo layer. Mọi ứng viên đều **đã đạt đúng mục tiêu tốc độ** (vì thoả ràng buộc ngay khi dựng), nên thời gian tìm kiếm giảm mạnh.
- Ứng viên được chấm bằng thông tin loss đầu cuối trên calibration data (so với heuristic Fisher đường chéo của Kwon et al.). Chi tiết thuật toán quy hoạch bên trong SPDY **không được nhắc lại trong paper**; phần mô tả cho biết thuật toán nhận prior p_s nhân hệ số độ nhạy của layer và độ trễ, và chọn mức per-layer thoả ràng buộc.

4. **Cắt dần, fine-tune xen kẽ.** Với dãy mục tiêu 2x, 3x, ..., lặp: cắt đến mục tiêu kế tiếp, fine-tune, cắt tiếp. Mỗi mô hình trong họ được đảm bảo tốc độ kỳ vọng.



Hình 52: Bảng latency theo module: thời gian khối FFN theo chiều trung gian và khối attention theo số head, theo ví dụ trong ghi chú (vẽ lại từ số liệu trong ghi chú)

Cách đọc hình: đường liền là số đo trong bảng ví dụ, đường nét đứt là mốc thời gian tỉ lệ với kích thước. Đường đo nằm trên mốc tỉ lệ ở mọi điểm: FFN 33 cột vẫn mất 0,7 ms và 2 head vẫn mất 1,9 ms. Vì vậy cắt thật nhỏ không đổi lấy được thời gian tương ứng, và bảng này mới là thứ quyết định nên cắt tới đâu.

Ví dụ minh hoạ bằng số (do ta dựng từ bảng ví dụ ở trên, không phải kết quả của paper): cắt FFN từ 3072 xuống 1814 tiết kiệm $11.9 - 7.4 = 4.5$ ms, còn 1814 xuống 1322 chỉ tiết kiệm $7.4 - 5.8 = 1.6$ ms; bỏ head từ 12 xuống 10 tiết kiệm $7.9 - 6.7 = 1.2$ ms, từ 10 xuống 8 tiết kiệm 0.9 ms. Thuật toán nhìn vào tỷ lệ "sai số tăng thêm / ms tiết kiệm" (ngầm qua tìm kiếm) và tránh những cắt không đổi lấy được ms. Điểm quan trọng: thời gian không tuyến tính theo kích thước, và đường cong khác nhau theo GPU (Table 3: thu FFN 3072 xuống 302 cho 6.9x trên V100 nhưng chỉ 3.1x trên A100).

Chú ý về diễn đạt của paper: phần tóm tắt nói ZipLM "lặp đi lặp lại tìm và xoá thành phần có đánh đổi loss-runtime tệ nhất". Trong thân bài, đánh đổi ấy được hiện thực qua *cơ sở dữ liệu theo layer + tìm kiếm SPDY có ràng buộc latency*, **không** phải một thứ tự tham lam toàn cục theo tỷ số sai số/ms. Điều này đáng ghi nhớ nếu bạn muốn tái hiện.

3.4 Distillation theo token ở mọi layer

Vì ZipLM giữ nguyên hidden size, hidden tensor $H \in \mathbb{R}^{B \times seq \times H}$ giữa các layer có cùng shape ở thầy và trò. Loss:

$$\mathcal{L} = \lambda_1 \mathcal{L}_{task} + \lambda_2 \mathcal{L}_{logit} + \lambda_3 \mathcal{L}_{token}$$

với $\mathcal{L}_{logit}$ là KL giữa logits (kiểu Hinton) và $\mathcal{L}_{token}$ là khoảng cách Euclid giữa vector ẩn của từng token (bỏ qua padding), trung bình trên mọi layer chưa bị cắt hoàn toàn. Nhờ vậy **không cần ghép layer thủ công** và không cần ma trận chiếu học được.

4. Thí nghiệm và kết quả chính

Thiết lập: BERT-base và BERT-large trên SQuAD v1.1 và GLUE (SST-2, QNLI, MNLI, QQP; thêm CoLA, MRPC, STS-B, RTE ở phụ lục); GPT-2 (124M) trên OpenWebText, đánh giá zero-shot trên WikiText. Môi trường BERT: một V100 16GB, batch 128, độ dài 384 (SQuAD) và 128 (GLUE). GPT-2 hai chế độ: throughput (batch 16, độ dài 1024) và latency (batch 1, nhiều độ dài prompt). Đối thủ: CoFi, Block Movement Pruning, TinyBERT, MiniLM, DistilBERT, MobileBERT, DynaBERT, DistilGPT2, v.v.

BERT:

- SQuAD, BERT-base: hơn CoFi và TinyBERT khoảng 3 điểm F1 ở cùng tốc độ; cùng F1 thì nhanh hơn ít nhất 60%.
- Mức "hồi phục 99% độ chính xác" (chuẩn MLPerf): BERT-base nhanh 5x (SQuAD), 6x (QNLI, MNLI), 13x (SST-2), 15x (QQP); BERT-large SQuAD nhanh 6x. BERT-large cắt xuống khớp MobileBERT (kiến trúc tối ưu bằng NAS).
- So CoFi ở tốc độ tương đương (dev set, Table 6): QNLI 88.6 so với 86.1; SST-2 91.7 so với 90.4; MNLI 81.7 so với 80.6; SQuAD F1 85.7 so với 82.6.

GPT-2 (Table 1):

Chế độ	Mô hình	Tốc độ	Decoder	PPL WikiText-103
Throughput	GPT2	1.0x	85.0M	28.5
Throughput	DistilGPT2	1.6x	42.5M	43.0
Throughput	ZipGPT2	1.5x	47.3M	35.4
Throughput	ZipGPT2	2.1x	26.5M	41.5
Throughput	ZipGPT2	3.3x	5.7M	72.1
Latency	DistilGPT2	1.9x	42.5M	43.0
Latency	ZipGPT2	1.6x	48.7M	37.8
Latency	ZipGPT2	2.0x	39.2M	41.2
Latency	ZipGPT2	2.5x	20.7M	55.0

(Paper nhận xét: cùng chất lượng, ZipGPT2 nhỏ hơn DistilGPT2 60% và nhanh hơn 30%. Chú ý: GPT2 gốc được huấn luyện trên dữ liệu lớn kín nên không so trực tiếp.)

One-shot (không retrain), Table 2, BERT-base (ZipLM so với Kwon et al.): SQuAD F1 87.1 so với 86.2 (1.5x) và 84.1 so với 76.5 (2.0x); QQP 89.7 so với 89.5 và 84.8 so với 83.9; MNLI 83.0 so với 82.8 và 78.2 so với 78.1.

Chi phí: cả họ BERT-base 2x đến 15x cần 115 epoch tổng, CoFi cần 560 (4.87 lần ít hơn); khoảng 35 giờ (MNLI) hoặc 10 giờ (SST-2) trên một RTX A6000, một bộ siêu tham số duy nhất.

Kéo dài tới cực đoan: đến 75x (BERT-base) và 55x (BERT-large) mà không sụp. Fit tuyến tính: $F1_{large} \approx 92.1 - 0.3 \times speedup$, $F1_{base} \approx 90.3 - 0.6 \times speedup$.

5. Phân tích / ablation

- **Pruning for speedup so với pruning for sparsity (Fig. 4):** nhắm tốc độ thay vì sparsity cho hơn tới 10 điểm F1 (BERT-base SQuAD), nhất là ở tốc độ cao, vì thuật toán không cắt thứ không làm nhanh thêm.
- **Chiều sâu so với chiều rộng (GPT-2, quan trọng):** cùng mục tiêu tốc độ nhưng kiến trúc kết quả khác hẳn. Khi tối ưu **throughput** (đầu vào lớn), ZipLM giữ nguyên số layer và thu hẹp ma trận (khoảng 10 lần). Khi tối ưu **latency** (batch 1, đầu vào nhỏ), ma trận không còn là nút cổ chai nên giữ nguyên độ rộng và giảm số module (độ sâu giảm gần 4 lần). Các phương pháp chỉ nhắm sparsity không thấy được khác biệt này.
- **Khả năng của thiết bị (Table 3):** thu MLP 3072 → 302 (khoảng 90% sparsity) được 6.9x trên V100 nhưng 3.1x trên A100; A100 mạnh hơn nên ma trận nhỏ dùng kém hiệu quả. MLP 33 chỉ 14.8x (V100) và 4.4x (A100).
- **Distillation token (Table 5):** ZipBERT có Ltoken so với không: SST-2 91.7 so với 89.2; QNLI 88.6 so với 86.5; MNLI 81.7 so với 81.2; SQuAD 85.7 so với 85.7. Lợi ích rõ nhất ở tập nhỏ (tới 2 điểm).
- **Dữ liệu calibration (Table 4):** one-shot SQuAD 1.5x: 4 mẫu 82.3, 32 mẫu 86.8, 128 mẫu 86.8, 4096 mẫu 87.6 (Kwon: 86.2 với 2048 mẫu). Ở 2.0x: 4 mẫu 48.4, 32 mẫu 82.6, 4096 mẫu 84.7 (Kwon 76.5). Vượt SOTA từ 32 mẫu.
- **Cấu trúc mô hình đã cắt (phụ lục G):** FFN bị cắt mạnh hơn head; để 2x, cắt khoảng 60% chiều trung gian và 40% head. Mô hình 15x giữ trung bình 2% chiều trung gian và 6% head (2.9M tham số encoder) mà còn hơn 95% độ chính xác.
- **Ước lượng tốc độ (Table 8):** lệch tối đa 5.28% so với đo thật (mục tiêu 14x BERT-large, đạt 14.74x).

6. Giới hạn

Do tác giả nêu: phần Broader impact/Limitations chỉ bàn tác động xã hội (rẻ hơn thì dễ bị lạm dụng, ví dụ sinh spam); không có mục giới hạn kỹ thuật riêng. Ghi chú trong thân bài: bảng latency phải đo lại cho mỗi môi trường đích; GPT-2 throughput tối đa khoảng 3.5x do từ vựng lớn.

Do ta nhận xét:

- Thí nghiệm chính là BERT và GPT-2 cỡ nhỏ, có fine-tune/distillation nhiều bước; chưa kiểm tra trên LLM hàng tỷ tham số hay mô hình lai tuyến tính-attention. Tính nghịch đảo Hessian theo d_{col} (3072 cho FFN của BERT-base) khả thi ở đây nhưng nặng hơn ở các layer rất rộng.
- Bù OBS chỉ dùng **trọng số của chính layer đó** (cột còn lại của cùng ma trận) và sai số **cục bộ** (đầu ra layer); không chứa tín hiệu loss cuối. Phần "đầu cuối" chỉ vào ở bước tìm kiếm.
- Bảng latency dùng ước lượng cộng tính theo module; không mô hình hoá tương tác giữa các module (bộ nhớ đệm, kernel fusion) ngoài phần đo thật 5.28%.
- Cắt cột FC2 chỉ giải quyết FFN kiểu 2 ma trận; FFN có gate (SwiGLU) và các block khác (linear-attention) cần định nghĩa lại "cột".

7. Liên hệ với các bài khác

- **01, 22, 23 (OBS, OBC, SparseGPT):** ZipLM chính là OBS layer-wise nhưng cho nhóm cột (structured), có tính lại nghịch đảo từng bước, khác SparseGPT (xử lý unstructured, cập nhật theo khối) ở chỗ đồng bộ chung cho mọi hàng. Paper nói rõ rằng đây không phải mở rộng tầm thường.
- **17 (CoFi) và 38 (Sheared LLaMA):** đối thủ "học mask trong lúc huấn luyện"; ZipLM chứng minh cắt rời rạc rồi fine-tune có thể thắng nếu thuật toán cắt đủ chính xác.
- **19 (HALP):** cùng tinh thần "chọn theo ngân sách latency, dùng latency table" nhưng cho CNN; ZipLM dùng SPDY (HALP là bài toán chọn theo ngân sách latency, ta nhớ là dạng knapsack, chưa kiểm lại ở bài 19).

- **27 (LLM Surgeon) và 28 (FLAP):** cùng tìm cách bù sau khi cắt; LLM Surgeon thêm gradient; FLAP chỉ bù bằng bias.
- **29 (SliceGPT):** thay vì chọn cấu trúc theo OBS, tự do xoay không gian để cắt các thành phần chính.
- **42 (Puzzle) và 41 (Flextron):** đẩy ý tưởng tối ưu kiến trúc theo ràng buộc phần cứng đến cấp "chọn biến thể block".

8. Gợi ý áp dụng

- Cập nhật δ_S là **ánh xạ tuyến tính tối ưu theo nghĩa bình phương tối thiểu** từ các cột (neuron) còn lại đến đầu ra layer, ràng buộc ở chỗ chỉ được sửa *chính các cột còn lại của cùng ma trận*.
- **Nhận xét:** kết quả GPT-2 chế độ latency (batch 1) của ZipLM là chỗ duy nhất trong bài hỗ trợ gián tiếp: ở batch 1 thì giảm độ sâu cho tốc độ hiệu quả hơn thu hẹp ma trận (độ sâu giảm gần 4 lần, độ rộng gần như giữ). Nó nói về *tốc độ*, chưa nói về *độ hồi phục*; hai thứ này cần tách khi diễn giải.
- Distillation theo token ở mọi layer (3.4) dùng được nếu giữ nguyên hidden size (cắt neuron MLP, cắt head) nhưng không dùng nguyên được nếu ta cắt cả chiều residual (cần ma trận chiếu).

9. Thí nghiệm gợi ý

Vẽ "ms tiết kiệm so với % cắt". Nếu đường cong phẳng thì cắt neuron không đáng.

10. Tóm tắt công thức / khái niệm

Khái niệm	Nội dung
Mục tiêu layer	$\min \|WX - \widehat{W}X\|_2^2$, cắt cùng chỉ số cột ở mọi hàng
Điểm cấu trúc	$\sum_i W_{i,M_S}\left[(H^{-1})_{M_S,M_S}\right]^{-1} W_{i,M_S}^\top$, $H = XX^\top$
Cập nhật bù	$\delta_S = -W_{:,M_S}\left[(H^{-1})_{M_S,M_S}\right]^{-1}(H^{-1})_{M_S,:}$
Cập nhật nghịch đảo	$H^{-1} \leftarrow H^{-1} - H^{-1}_{:,M_S}[\cdot]^{-1}H^{-1}_{M_S,:}$, $O(\|M_S\|d_{col}^2)$
Latency table	Thời gian module theo #head (0..N-1) và kích thước FFN (0.9^i, $i \leq 42$), đo trên thiết bị đích
Prior tìm kiếm	$p_s = \|\widehat{W}_s X - WX\|^2 / \|WX\|^2$ (bằng 1 khi bỏ hết layer)
Tìm kiếm	SPDY chỉnh sửa, 1000 bước, đột biến 10% hệ số độ nhạy
Loss distill	$\lambda_1\mathcal{L}_{task} + \lambda_2\text{KL}_{logit} + \lambda_3\mathcal{L}_{token}$
Con số chính	BERT-base 99% accuracy: 5x (SQuAD), 6x (QNLI/MNLI), 13x (SST-2), 15x (QQP); ước lượng tốc độ sai tối đa 5.28%

11. Kết luận ngắn

ZipLM kết hợp (1) OBS có bù cho nhóm cột, xoá từng cấu trúc một và cập nhật nghịch đảo Hessian, với (2) latency table đo thật và tìm kiếm per-layer để đạt đúng mục tiêu tốc độ.

Link gốc: https://arxiv.org/abs/2302.04089

Bài 26

LLM-Pruner: On the Structural Pruning of Large Language Models (2023)

Tác giả: Xinyin Ma, Gongfan Fang, Xinchao Wang (National University of Singapore).
Venue / arXiv: arXiv:2305.11627 (19/05/2023, bản v3 09/2023). Bản text không ghi tên hội nghị.
Nguồn: https://arxiv.org/abs/2305.11627
Chủ đề: Structured pruning task-agnostic cho LLM (LLaMA-7B/13B, Vicuna-7B, ChatGLM-6B): tự động tìm các cấu trúc phụ thuộc nhau (dependency graph), chấm điểm cả nhóm bằng thông tin gradient (Taylor), phục hồi nhanh bằng LoRA trên 50k mẫu.
Vị trí trong lộ trình: Đọc sau 20 (DepGraph, nền tảng "nhóm phụ thuộc"), 07 (Molchanov, importance bằng Taylor), 04/05 (cắt filter/channel) và 25 (ZipLM). Bài này là phương pháp structured cho LLM đầu tiên có công cụ tự động, và là baseline mà FLAP (28) và SliceGPT (29) so sánh. Bài kế: 27 (LLM Surgeon), 28 (FLAP).
Chương nền liên quan: nền 03: Độ quan trọng của trọng số, nền 04: Cắt hạt, cắt khối và mẫu 2, nền 05: Cắt rồi huấn luyện lại

> Ghi chú học tập bằng tiếng Việt, không phải bản dịch. Số liệu lấy từ paper (một vài bảng phụ lục bị lỗi định dạng nên được ghi chú rõ).

1. Bối cảnh

Trước bài này, nén mô hình ngôn ngữ (BERT, v.v.) thường **task-specific**: tinh chỉnh trên một tác vụ rồi nén (cho một bài GLUE), hoặc chưng cất tổng quát (TinyBERT) tốn nhiều tài nguyên kém (paper nêu: khoảng 14 GPU-ngày cho general distillation của TinyBERT). Với LLM có hai trở ngại:

- **Kho dữ liệu huấn luyện khổng lồ** (hàng nghìn tỷ token, đôi khi không công khai), nên khó dùng lại để phục hồi mô hình sau cắt.
- **Thời gian post-training sau cắt quá dài.**

Bài báo đặt bài toán là **task-agnostic compression**: giảm cỡ mô hình nhưng giữ được tính "giải mọi tác vụ", dùng ít dữ liệu và thời gian. Hai khái niệm nền:

- **Structured pruning:** xoá cả hàng/cột/neuron/head thay vì từng trọng số, nên ma trận thật sự nhỏ lại.
- **Cấu trúc kết nối (coupled structures):** trong mạng, các tensor ràng buộc nhau. Ví dụ neuron j trung gian của MLP gồm hàng j của up/gate và cột j của down; xoá một nửa mà không xoá nửa kia là vô nghĩa hoặc sai kích thước.

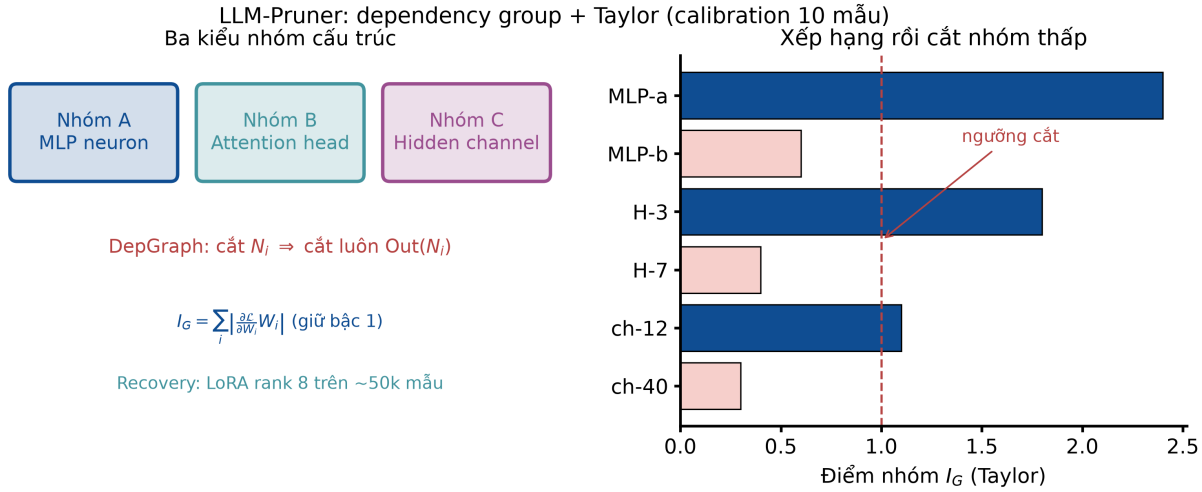
2. Ý tưởng chính

LLM-Pruner làm ba bước: (1) **Discovery:** từ định nghĩa phụ thuộc giữa các neuron, chạy lan truyền tự động để gom mọi trọng số bị ràng buộc vào cùng một nhóm, nên bạn không cần tự phân tích từng kiến trúc; (2) **Estimation:** chấm điểm cả nhóm (không phải từng ma trận) bằng đóng góp của nhóm vào loss next-token, ước lượng trên chỉ 10 câu mẫu công khai, dùng thông tin gradient vì dữ liệu không thuộc phân phối huấn luyện nên gradient không bằng 0; (3) **Recovery:** tinh chỉnh bằng LoRA 2 epoch trên Alpaca làm sạch (khoảng 50k mẫu, 3 giờ, 1 GPU). Kết quả: bỏ 20% tham số LLaMA-7B còn 94.97% hiệu năng zero-shot trung bình.

3. Phương pháp chi tiết

3.1 Discovery: đồ thị phụ thuộc

Gọi N_i, N_j là hai neuron, $\text{In}(N_i)$ và $\text{Out}(N_i)$ lần lượt là tập neuron trỏ đến và đi từ N_i. Hai quy tắc phụ thuộc (có hướng):



LLM-Pruner: nhóm phụ thuộc (MLP/head/channel), điểm Taylor bậc nhất I_G, rồi recovery LoRA. (minh hoạ, tự tính)

$$N_j \in \text{Out}(N_i) \wedge \text{Deg}^-(N_j) = 1 \;\Rightarrow\; N_j \text{ phụ thuộc } N_i$$
$$N_i \in \text{In}(N_j) \wedge \text{Deg}^+(N_i) = 1 \;\Rightarrow\; N_i \text{ phụ thuộc } N_j$$

Ở đây Deg^- là bậc vào, Deg^+ là bậc ra. Ý nghĩa: nếu một neuron chỉ nhận đầu vào từ duy nhất một neuron khác (hoặc chỉ đưa đầu ra cho duy nhất một neuron), bỏ neuron kia thì nó cũng phải bỏ. Lấy từng neuron làm **trigger**, kích hoạt các neuron phụ thuộc, các neuron mới lại trở thành trigger, cho đến khi hết thì thu được một nhóm. Với LLaMA có ba loại nhóm (Fig. 2):

- **Loại A (MLP):** neuron trung gian = hàng của gate và up, cột của down.
- **Loại B (Multi-head attention):** một head = đầu ra tương ứng của Q, K, V và đầu vào tương ứng của ma trận out-projection.
- **Loại C (Channel-wise):** chiều ẩn (hidden dimension) của residual stream; nhóm này đi qua mọi layer, embedding, các norm, đầu LM.

Phần cài đặt gọi hai chiến lược: **Block** (cắt nhóm loại A và B, tức MLP và head ở các layer giữa) và **Channel** (chỉ cắt nhóm C, tức giảm chiều ẩn).

3.2 Estimation: điểm của một nhóm

Với $D = \{x_i, y_i\}_{i=1}^N$ ($N = 10$, lấy từ BookCorpus, mỗi câu cắt 128 token) và nhóm $G = \{W_i\}_{i=1}^M$, độ quan trọng của một ma trận W_i là thay đổi loss khi xoá nó, khai triển Taylor bậc hai:

$$I_{W_i} = |\Delta\mathcal{L}(D)| = \left|\mathcal{L}_{W_i}(D) - \mathcal{L}_{W_i=0}(D)\right| = \left|\underbrace{\frac{\partial \mathcal{L}^\top}{\partial W_i} W_i}_{\neq 0} - \frac{1}{2} W_i^\top H W_i + O(\|W_i\|^3)\right|$$

Điểm khác với OBD/OBS: các phương pháp cổ điển bỏ số hạng bậc nhất vì mô hình đã hội tụ trên dữ liệu huấn luyện (gradient ≈ 0). Ở đây dữ liệu calibration **không** lấy từ dữ liệu huấn luyện gốc nên gradient khác

0, và LLM thì tính Hessian là bất khả thi, nên **giữ số hạng bậc nhất**. Với từng tham số k (mức element) có thể thêm số hạng bậc hai xấp xỉ bằng Fisher:

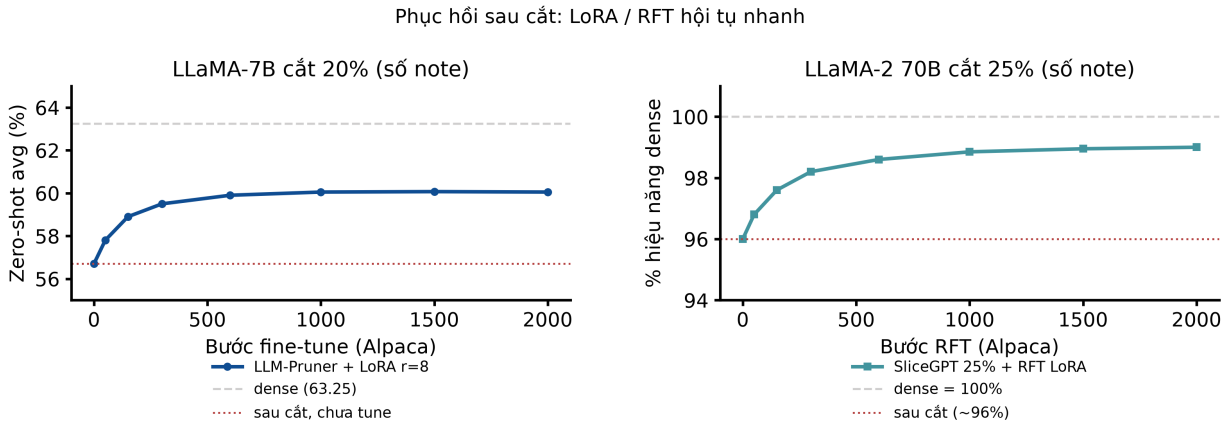
$$I_{W_i^k} \approx \left| \frac{\partial \mathcal{L}}{\partial W_i^k} W_i^k - \frac{1}{2} \sum_{j=1}^{N} \left(\frac{\partial \mathcal{L}(D_j)}{\partial W_i^k} W_i^k \right)^2 \right|$$

Các biến thể trong ablation: **Vector** (điểm cho cả ma trận, Eq. 3), **Element1** (điểm theo từng tham số, chỉ bậc nhất), **Element2** (theo tham số, có thêm bậc hai Fisher).

Gộp điểm trong nhóm (bốn cách): Sum $I_G = \sum_i I_{W_i}$; Prod; Max; Last-only (chỉ lấy ma trận chạy cuối trong nhóm, lý do: xoá cấu trúc cuối nhóm tương đương xoá mọi kết quả tính trong nhóm). Chọn mặc định **Sum**. Sau đó xếp hạng nhóm trong từng module và xoá nhóm điểm thấp theo tỷ lệ định trước (local pruning; global thử ở phụ lục và hơi kém).

3.3 Recovery: LoRA

$f(x) = (W + \Delta W)x + b = (Wx + b) + PQx$, chỉ huấn luyện P, Q (rank 8). Sau huấn luyện gộp PQ vào W nên không tăng tham số. LoRA áp dụng cho mọi ma trận tuyến tính (cả đã cắt lẫn chưa).



Đường cong phục hồi: LLM-Pruner+LoRA (7B 20%) và SliceGPT+RFT (70B 25%) hội tụ về gần dense. (minh hoạ, tự tính)

```
LLM-Pruner(model, calib(10 câu), ratio, tune_data):
    groups = discover_dependency_groups(model)        # A: MLP, B: head, C: channel
    loss   = next_token_loss(model, calib); loss.backward()   # lấy gradient
    for g in groups:
        score[g] = aggregate(importance(W_i, grad_i) for W_i in g)   # Sum
    prune lowest-score groups per module until target ratio
        # Block: bỏ 3 layer đầu và layer cuối khỏi diện cắt
    LoRA fine-tune (rank 8, lr 1e-4, 2 epoch, ~50k mẫu Alpaca)
```

Chi tiết cấu hình (phụ lục): với chiến lược Block trên LLaMA-7B, giữ nguyên **3 layer đầu và layer cuối** (tác động quá lớn), mỗi module MLP có 11,008 nhóm; để đạt 20% tham số, cắt **25% nhóm ở layer 5 đến 30**; để đạt 50% tham số, cắt **60% nhóm ở layer 4 đến 30**. Tỷ lệ cắt nhóm cao hơn tỷ lệ tham số vì embedding và các layer bị loại không bị cắt.

4. Thí nghiệm và kết quả chính

Mô hình: LLaMA-7B, LLaMA-13B, Vicuna-7B (LLaMA tinh chỉnh hội thoại), ChatGLM-6B (song ngữ). **Đánh giá:** zero-shot 7 bộ phân loại (BoolQ, PIQA, HellaSwag, WinoGrande, ARC-e, ARC-c, OpenbookQA),

trung bình 7 tác vụ, cộng PPL WikiText2 và PTB. Dòng LLaMA-7B^⋆ là số tự chạy lại của tác giả (63.25), thấp hơn số chính thức (68.59) vì prompt khác.

Thống kê mô hình (Table 3, đo bằng 64 token đầu vào, A5000):

Mô hình	Chiến lược	Tỷ lệ	#Params	MACs	Bộ nhớ	Latency
LLaMA-7B	gốc	0%	6.74B	424.02G	12884.5 MiB	69.32 s
LLaMA-7B	Channel	20%	5.39B	339.36G	10363.6 MiB	61.50 s
LLaMA-7B	Block	20%	5.42B	339.60G	10375.5 MiB	58.55 s
Vicuna-7B	Channel	50%	3.37B	212.58G	6556.3 MiB	40.11 s
Vicuna-7B	Block	50%	3.35B	206.59G	6533.9 MiB	37.54 s

(Tham số giảm 20% nhưng latency chỉ giảm khoảng 15% với Block: 69.32 s xuống 58.55 s, theo phép tính từ các số trên.)

LLaMA-7B, 20% tham số (Table 1; trung bình zero-shot / PPL WikiText2):

Cấu hình	Wiki2 PPL	PTB PPL	Trung bình 7 tác vụ
LLaMA-7B^⋆ (gốc)	12.62	22.14	63.25
L2 (không tune)	582.41	1022.17	42.65
Random (không tune)	27.51	43.19	52.69
Channel (không tune)	74.63	153.75	45.38
Vector (không tune)	22.28	41.78	53.25
Element2 (không tune)	19.77	36.66	56.82
Element1 (không tune)	19.09	34.21	56.69
Channel (có LoRA)	22.02	38.67	55.57
Vector (có LoRA)	18.84	33.05	57.23
Element2 (có LoRA)	17.37	30.39	**60.07**
Element1 (có LoRA)	17.58	30.11	59.23

Không tune giữ 89.8% hiệu năng gốc (56.82/63.25); có tune Element2 giữ **94.97%** (60.07/63.25). Điểm bậc hai (Element2) tốt nhất nhưng Element1 (rẻ hơn nhiều) chỉ kém chút, nên tác giả nghiêng về bậc nhất. ChatGLM-6B khác: điểm theo tham số (Element1) hỏng (Table 5, không tune 36.45), điểm Vector tốt nhất (cắt 10%: 46.16 không tune, 47.34 sau tune, so với gốc 47.05).

LLaMA-13B (Table 2, dùng Element1): gốc 64.97 (PPL 11.58). Cắt 20% không tune: Block 61.79 (PPL 16.01), Channel 50.29, L2 51.08, Random 57.09; sau tune: Block **64.02** (PPL 15.18), Channel 60.24, L2 62.12, Random 59.78.

Vicuna-7B 20% (Table 4): gốc 62.78; không tune Element2 56.83, Element1 56.25; có LoRA Element2 57.78, Element1 57.71, Vector 56.64 (92.03% của gốc, tính theo câu của paper cho điểm tốt nhất).

Mức cắt 50%: paper thừa nhận "suy giảm lớn" (xem mục Giới hạn). Từ bảng phụ lục C.5 (định dạng bảng bị rối, ta đọc được) trung bình khoảng 46 so với 63.25 gốc, và Channel không tune có PPL cỡ 10^4.

5. Phân tích / ablation

- **Cần dependency (Table 6):** nếu cắt mỗi ma trận độc lập, không tôn trọng nhóm: PPL WikiText2 **68378** (không tune) và **13307** (có tune), trung bình 38.32 và 38.10, so với có dependency 19.09 và 17.58 (56.69

và 59.23). Đây là nhóm ablation quan trọng nhất của paper: pruning cấu trúc mà phá nhóm thì không phục hồi được.

- **Block so với Channel (Table 1, 2, Fig. 3):** cắt Channel (chiều ẩn đồng đều ở mọi layer) tệ hơn Block rõ rệt. Lý do tác giả đưa ra: tầm quan trọng không phân bố đều giữa các layer; layer đầu và cuối rất quan trọng (Fig. 3: chỉ bỏ nhóm ở một layer, layer 0 và cuối gây tăng PPL lớn), mà Channel buộc phải cắt chúng.
- **Tỷ lệ cắt (Fig. 4):** theo mô tả, L2 (không dữ liệu) sụp quanh 20%, trong khi LLM-Pruner cho phép cắt tới khoảng 60% để đạt cùng mức PPL (với LLaMA); Vicuna cắt 10% bằng L2 tương đương cắt 60% bằng LLM-Pruner. (Số đọc từ hình, nên chỉ nêu xu hướng.)
- **Gộp điểm (Table 7, LLaMA-7B không tune):** Sum PPL 66.13/164.25; Max 62.59/144.38 (PPL tốt nhất nhưng zero-shot tệ nhất); Prod 77.63/192.88; Last-only 130.00/170.88 (zero-shot tốt nhất nhưng sinh văn bản tệ nhất). Chọn Sum làm thoả hiệp.
- **Global so với local (Table 14):** Element1 local avg 56.69 (PPL 19.09), global 56.49 (20.84); local nhỉnh hơn vì điểm khác layer không so sánh được.
- **Số bước tune (Fig. 5, mục C.4):** PPL giảm nhanh trong khoảng 300 bước rồi tăng nhẹ; huấn luyện quá lâu thì overfit Alpaca và hại các tác vụ tổng quát. Dùng 2.59M mẫu thay vì 50k: avg 62.36 so với 59.23 (LLaMA-5.4B, phụ lục C.1, so 50k mẫu Alpaca với 2.59M mẫu từ một nguồn khác).
- **Chưng cất kiểu DistilBERT (chọn 1 trong 2 layer), Table 8:** 44.64 (3.50B) so với LLM-Pruner 48.88 (3.35B).
- **Huấn luyện từ đầu (Table 9):** LLaMA-3B từ LLM-Pruner (3.6B) trung bình 50.30 so với StableLM-3B 45.84 (đều tune Alpaca), nhưng tác giả thừa nhận không phải lúc nào cũng thắng mô hình 3B huấn luyện từ đầu (chênh lệch lớn về kích thước kho huấn luyện).
- **Kết hợp lượng tử hoá (Table 13):** LLM.int8() trên mô hình cắt vẫn giữ avg 62.30 (so với 62.36 trước lượng tử), bộ nhớ 5444 MiB.

6. Giới hạn

Do tác giả nêu: (a) ở mức cắt cao như 50% hiệu năng sụt mạnh, nén LLM mạnh vẫn là thách thức; (b) mô hình cắt có lúc sinh câu vô nghĩa hoặc lặp token, nhất là với văn bản dài; (c) tune quá nhiều có thể overfit tập tune; (d) không phải lúc nào cũng hơn mô hình cùng cỡ huấn luyện từ đầu.

Do ta nhận xét:

- Điểm là **bậc nhất, chỉ bằng 10 câu**; số hạng bậc nhất ($\frac{\partial \mathcal{L}}{\partial W} W$) có thể ồn và không phản ánh tương tác giữa các nhóm được cắt cùng nhau. Không có bước bù tuyến tính (khác ZipLM/LLM Surgeon/SliceGPT) nên toàn bộ phần hư hại để cho LoRA sửa.
- Phục hồi chỉ bằng LoRA rank 8 và SFT trên Alpaca; không dùng distillation từ mô hình gốc. Do đó mô hình cắt bị "lệch" về phân phối Alpaca, và tác giả không đo trên kiểu tác vụ khác.
- Latency giảm thấp hơn nhiều so với tham số (15% với 20% tham số) vì chỉ đo 64 token, 1 GPU; không có phân tích theo từng phần cứng như ZipLM.
- Bảng phụ lục (C.5) bị rối trong bản text nên ta chỉ nêu xu hướng.

7. Liên hệ với các bài khác

- **20 (DepGraph):** cùng nhóm tác giả (Fang, Ma, Wang). LLM-Pruner áp dụng DepGraph cho LLM; **30 (MaskLLM)** cũng cùng nhóm Fang & Wang.
- **07 (Molchanov 2019):** điểm bậc nhất theo Taylor cho khối cấu trúc; LLM-Pruner thêm gộp nhóm và dùng cho LLM với dữ liệu rất ít.
- **01 (OBD/OBS):** LLM-Pruner mượn khai triển Taylor nhưng *giữ* số hạng bậc nhất vì dữ liệu ngoài phân phối.
- **25 (ZipLM), 27 (LLM Surgeon), 28 (FLAP), 29 (SliceGPT):** cùng nhắm structured pruning LLM,

đều có thành phần bù/tái tạo mà LLM-Pruner thiếu. FLAP chỉ ra LLM-Pruner (kể cả có LoRA) kém FLAP không cần retrain.

- **38 (Sheared LLaMA), 39 (Minitron):** so sánh "cắt rồi huấn luyện tiếp bằng nhiều token" với cách LoRA 50k mẫu của bài này.
- **43 (Pruning vs scratch):** câu hỏi cắt có hơn huấn luyện từ đầu hay không, bài này chỉ có một so sánh nhỏ (Table 9).

8. Gợi ý áp dụng

- **Layer đầu và cuối nhạy cảm:** paper giữ 3 layer đầu và layer cuối, và cho thấy Channel (cắt chiều ẩn đồng đều) tệ hơn Block.

9. Tóm tắt công thức / khái niệm

Khái niệm	Nội dung
Phụ thuộc	$N_j \in \text{Out}(N_i) \wedge \text{Deg}^-(N_j) = 1 \Rightarrow N_j$ phụ thuộc N_i (và quy tắc đối xứng với Deg^+)
Nhóm A/B/C	MLP / head attention / kênh ẩn (channel-wise)
Điểm ma trận	$I_{W_i} = \lvert\frac{\partial \mathcal{L}^\top}{\partial W_i} W_i - \frac{1}{2} W_i^\top H W_i + \ldots\rvert$
Điểm tham số	bậc nhất + Fisher bậc hai (Element1 / Element2)
Gộp nhóm	Sum (mặc định), Prod, Max, Last-only
Phục hồi	LoRA rank 8, lr 1e-4, 2 epoch, ~50k mẫu Alpaca, ~3 giờ 1 GPU
Cấu hình cắt LLaMA-7B	20% tham số: 25% nhóm ở layer 5 đến 30; 50%: 60% ở layer 4 đến 30; giữ 3 layer đầu và layer cuối
Con số chính	20% tham số: 56.69 (không tune) và 60.07 (có LoRA, Element2) so với 63.25 gốc; 94.97%

10. Kết luận ngắn

LLM-Pruner xây dựng quy trình cắt tỉa có cấu trúc cho LLM: tìm nhóm phụ thuộc, chấm điểm nhóm bằng Taylor bậc nhất trên 10 mẫu, phục hồi bằng LoRA trong 3 giờ. Ở 20% nó đủ dùng (còn khoảng 95% hiệu năng), ở 50% thì không. Đóng góp bền nhất là công cụ dựng nhóm phụ thuộc và kết quả "phá nhóm là hỏng", còn phần chấm điểm không bù là chỗ các bài sau (25, 27, 28, 29) cải thiện.

Link gốc: https://arxiv.org/abs/2305.11627

Bài 27

The LLM Surgeon (2023)

Tác giả: Tycho F.A. van der Ouderaa, Markus Nagel, Mart van Baalen, Yuki M. Asano, Tijmen Blankevoort (Imperial College London; Qualcomm AI Research; QUVA Lab, University of Amsterdam).
Venue / arXiv: arXiv:2312.17244 (28/12/2023, bản v2 03/2024). Bản text không ghi tên hội nghị.
Nguồn: https://arxiv.org/abs/2312.17244
Chủ đề: Pruning bậc hai kiểu OBS/OBD cho LLM với xấp xỉ độ cong Kronecker-factored (KFAC) của Fisher có tính cả gradient; hỗ trợ structured (hàng/cột), semi-structured (N:M), unstructured; phân bổ thưa toàn cục theo ngưỡng; cập nhật trọng số có tương quan; cắt nhiều "shot".
Vị trí trong lộ trình: Đọc sau 01 (OBD/OBS), 22 (OBC), 23 (SparseGPT), 25 (ZipLM) và 26 (LLM-Pruner). Bài này là nỗ lực "làm OBS đúng nghĩa" cho LLM: dùng loss thật (có gradient) thay vì chỉ lỗi tái tạo từng layer. Bài kế: 28 (FLAP), 29 (SliceGPT, được trích như concurrent work).

> Ghi chú học tập bằng tiếng Việt, không phải bản dịch. Số liệu lấy từ paper.

1. Bối cảnh

OBD/OBS (1989, 1992; bài 01) viết pruning thành bài toán tối ưu có ràng buộc: xấp xỉ loss bằng khai triển Taylor bậc hai quanh trọng số đã huấn luyện, rồi chọn trọng số xoá và cập nhật các trọng số còn lại để loss tăng ít nhất. Nút thắt là **Hessian** có kích thước (số tham số)^2, không thể lưu cho mạng thật. Các hướng giải:

- **EigenDamage** (Wang 2019): phân rã Kronecker của Hessian chia khối đường chéo.
- **OBC, SparseGPT** (bài 22, 23): chỉ dùng độ cong của lỗi tái tạo bình phương đầu ra *của từng layer*, **bỏ qua gradient** liên hệ chi phí cục bộ với loss đích. Theo tác giả, xấp xỉ loss như vậy kém chính xác, và các phương pháp này không mở rộng dễ sang structured.

Một vài khái niệm cần có:

- **Fisher information matrix** (ma trận thông tin Fisher): kỳ vọng tích ngoài gradient của log-likelihood; với mô hình huấn luyện bằng negative log-likelihood, Fisher chính bằng Hessian (theo bài báo) và luôn nửa xác định dương.
- **KFAC** (Kronecker-factored approximate curvature): xấp xỉ khối Fisher của một lớp thành tích Kronecker $G \otimes A$ của hai ma trận nhỏ: A từ activation đầu vào, G từ gradient đầu ra. Nghịch đảo dễ: $(G \otimes A)^{-1} = G^{-1} \otimes A^{-1}$.
- **Laplace approximation:** xấp xỉ hậu nghiệm bằng Gaussian quanh nghiệm, với hiệp phương sai là Hessian nghịch đảo; paper dùng góc nhìn này để giải thích tại sao OBS là "tối đa hoá likelihood dưới ràng buộc".

2. Ý tưởng chính

Xấp xỉ hàm mất mát quanh mô hình đã huấn luyện bằng một dạng bậc hai, với độ cong được ước lượng bằng KFAC. Xoá một hàng hoặc cột của ma trận trọng số là một ràng buộc; từ xấp xỉ đó có thể tính (1) **chi phí** (loss tăng bao nhiêu nếu xoá) và (2) **cập nhật tối ưu** cho mọi trọng số còn lại. LLM Surgeon áp dụng đúng công thức này lên LLM, bổ sung ba thứ: ngưỡng chi phí **toàn cục** để mỗi lớp tự bị cắt nhiều hay ít tuỳ độ nhạy; cập nhật trọng số **tính cả tương quan giữa nhiều hàng/cột bị xoá cùng lúc**; cắt theo **nhiều shot** (cắt một ít, cập nhật độ cong, cắt tiếp). Có thể đổi thêm thời gian tính để lấy chất lượng cao hơn.

3. Phương pháp chi tiết

3.1 Bài toán có ràng buộc

Tham số $\theta \in \mathbb{R}^P$, loss $\mathcal{L}(\theta) = -\log p(\theta|D)$. Khai triển bậc hai quanh θ^* (giả sử gradient ≈ 0 ban đầu):

$$-\log q(\theta) \approx -\log p(D|\theta^*) - (\theta-\theta^*)^\top \nabla\mathcal{L}(\theta^*) - \tfrac{1}{2}(\theta-\theta^*)^\top H(\theta-\theta^*).$$

Với $H = F$ (Fisher), xoá tập chỉ số K là bài toán:

$$\min_{\Delta\theta} \tfrac{1}{2}\Delta\theta^\top F \Delta\theta \quad \text{s.t.} \quad e_k^\top \Delta\theta + e_k^\top \theta = 0,\ \forall k \in K.$$

Giải bằng nhân tử Lagrange, với $E_K = [e_1 \dots e_K]$ là ma trận chọn các tham số bị xoá:

$$\boxed{\mathcal{L} = \tfrac{1}{2}(E_K\theta)^\top (E_K F^{-1} E_K^\top)^{-1}(E_K\theta)} \qquad \boxed{\Delta\theta = -F^{-1}E_K^\top (E_K F^{-1} E_K^\top)^{-1} E_K\theta}$$

(Eq. 7, 8). Với Fisher xấp xỉ KFAC theo từng layer: $F_l \approx G_l \otimes A_l$, $G_l = \frac{1}{\sqrt{N}}\sum_n g_{l,n} g_{l,n}^\top$ (gradient theo đầu ra $y = Wa$, kích thước $R \times R$), $A_l = \frac{1}{\sqrt{N}}\sum_n a_{l,n} a_{l,n}^\top$ (activation đầu vào, $C \times C$).

3.2 Chi phí xoá một phần tử, một hàng, một cột

Giả sử chi phí của các phần tử/hàng/cột là độc lập khi xếp hạng. Thay E thích hợp vào Eq. 7 (Eq. 10):

$$L_k = \frac{1}{2}\frac{(\theta_k)^2}{[G^{-1}\otimes A^{-1}]_{kk}}, \qquad L_r = \frac{1}{2}\frac{\theta_r^\top A\,\theta_r}{[G^{-1}]_{rr}}, \qquad L_c = \frac{1}{2}\frac{\theta_c^\top G\,\theta_c}{[A^{-1}]_{cc}}$$

Ở đây $\theta_r \in \mathbb{R}^C$ là hàng r của W và $\theta_c \in \mathbb{R}^R$ là cột c. Đọc công thức: xoá hàng r (một output) làm đầu ra đó mất đi, mức hại là năng lượng $\theta_r^\top A\theta_r$ của đầu ra, chia cho $[G^{-1}]_{rr}$, đại lượng nhỏ khi output r **có thể được bù bởi các output khác** trong không gian gradient; tương tự cột c (một input) có chi phí $\theta_c^\top G\theta_c$ chia cho $[A^{-1}]_{cc}$, nhỏ khi kênh đầu vào đó gần như là tổ hợp tuyến tính của kênh khác. L_k trùng OBS; L_r, L_c gần với "Structured Brain Surgeon" của EigenDamage nhưng viết cho hàng/cột.

3.3 Ngưỡng toàn cục và phân bổ động

Thay vì cắt đều mỗi layer (như SparseGPT), chọn **ngưỡng τ chung** sao cho phần trọng số bị xoá bằng $1-\alpha$ (α là kích thước đích, tỉ lệ còn lại). Với unstructured: sắp xếp mọi L_k trên toàn mạng. Với N:M: lấy tổng M chi phí thấp nhất trong mỗi khối. Với structured: sắp xếp L_r, L_c có trọng số theo số phần tử mỗi hàng/cột (hàng dài tốn nhiều tham số). Các layer nhạy bị cắt ít, layer bền bị cắt nhiều.

3.4 Cập nhật trọng số có tương quan

Cộng các cập nhật độc lập cho từng phần tử là gần đúng. Paper tính **cập nhật chung** cho toàn bộ tập bị xoá:

- **Tổng quát:** cần nghịch đảo ma trận $K \times K$ (K = số tham số bị xoá cùng lúc); với giới hạn m tham số tương quan trong một nhóm để kiểm soát chi phí.
- **Structured:** nhờ cấu trúc Kronecker, chỉ phải nghịch đảo ma trận $R' \times R'$ (khi xoá R' hàng) hoặc $C' \times C'$ (khi xoá C' cột), thay vì $R'C \times R'C$. Hai trường hợp đặc biệt khi xoá *một* hàng r hoặc *một* cột c (Eq. 13), chỉ cần phép chia vô hướng:

$$\text{xoá hàng } r:\ \Delta\theta = -\frac{G^{-1}e_r \otimes \theta_r}{[G^{-1}]_{rr}}, \qquad \text{xoá cột } c:\ \Delta\theta = -\frac{\theta_c \otimes A^{-1}e_c}{[A^{-1}]_{cc}}.$$

Ở dạng ma trận, xoá cột c: $\Delta W = -\theta_c \, (A^{-1} e_c)^\top / [A^{-1}]_{cc}$ (ta viết lại từ Eq. 13 cho dễ hình dung): các cột còn lại được cộng thêm một lượng tỉ lệ cột c, trọng số theo hàng c của A^{-1}, giống bước bù của OBS/SparseGPT/ZipLM. Bản tổng quát cho nhiều hàng/cột có ở Eq. 12 (ta không chép lại vì bản text của công thức bị lệch thứ tự).

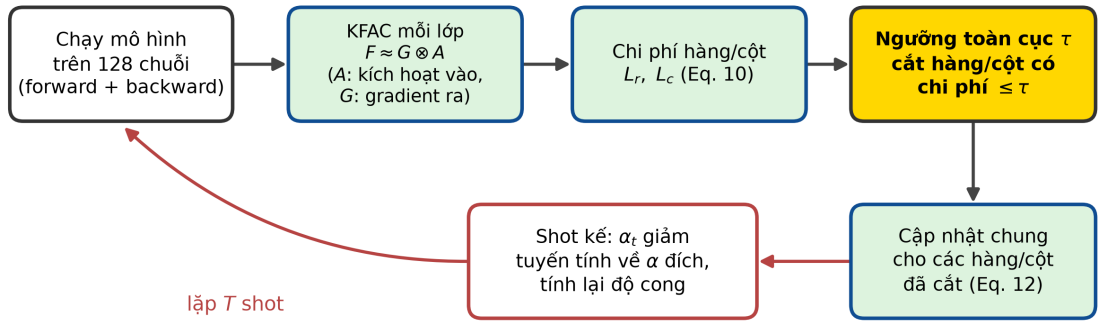
3.5 Nhiều shot

Mô hình bậc hai chỉ đúng cục bộ; bước nhảy lớn làm xấp xỉ sai. Do đó cắt trong T shot theo lịch tuyến tính α_t, giữa các shot tính lại độ cong (và cập nhật trọng số). Thực nghiệm: nhiều shot tốt hơn đơn điệu, sparsity cao cần nhiều shot hơn. Cấu hình: 40 shot cho mục tiêu 0.5 (structured), báo cáo các mức trung gian: $T = 8$ cho 0.9, 16 cho 0.8, 24 cho 0.7, 32 cho 0.6; N:M và unstructured dùng $T = 5$.

3.6 Hiệu chỉnh bậc nhất bằng LoRA xen kẽ

Mô hình có thể không ở cực tiểu, hoặc loss dùng khi nén khác loss huấn luyện, hoặc đã cắt nhiều shot, nên gradient $\neq 0$. Giữa các shot, tuỳ chọn thêm cập nhật LoRA ($W + UV$) và hấp thụ vào W (nên tổng các cập nhật rank thấp có rank cao hơn từng cái).

```
LLM_Surgeon_structured(theta0, alpha_target, D, T):
    for t in 1..T:
        # alpha_t: lịch tuyến tính từ 1 xuống alpha_target
        (G, A) = KFAC factors per layer from D (forward + backward)   # activation,
        ↪  output-gradient
        L_r, L_c = row/column costs (Eq.10)
        tau      = global threshold s.t. fraction (1 - alpha_t) of weights is cut
        remove rows/cols with cost <= tau
        Delta    = correlated update for removed rows/cols (Eq.12)
        theta   += Delta
        (optional) low-rank (LoRA) first-order update, absorbed into theta
    return theta
```



Chi phí hàng/cột nhỏ khi đầu ra (hoặc kênh vào) đó được các đầu ra (kênh) khác bù được trong không gian gradient.

Hình 53: Một shot của LLM Surgeon (KFAC, chi phí hàng/cột, ngưỡng toàn cục, cập nhật chung) và vòng lặp nhiều shot (minh hoạ, tự tính)

Cách đọc hình: hàng trên đi từ dữ liệu hiệu chuẩn tới việc chọn hàng/cột cần cắt bằng một ngưỡng chung cho cả mạng. Hàng dưới là bước bù trọng số rồi quay lại shot sau với độ cong được tính lại. Ô vàng là chỗ khác SparseGPT: lớp nhạy bị cắt ít, lớp bền bị cắt nhiều.

4. Thí nghiệm và kết quả chính

Mô hình: OPT 125m, 1.3b, 2.7b, 6.7b và Llama-v2 7B (13B ở phụ lục). **Dữ liệu nén:** 128 chuỗi 2048 token từ train WikiText-2; đánh giá test PPL WikiText-2 (cũng thử C4 và tiếng Pháp/Đức/Ý ở phụ lục). **Baseline:** Magnitude; L-OBD (chỉ activation, đường chéo $I \otimes A$); K-OBD (cùng KFAC nhưng độc lập hoàn toàn, **không cập nhật** trọng số còn lại); SparseGPT (unstructured/N:M).

Structured, test PPL WikiText-2 (Table 1; "size" = % tham số còn lại):

Phương pháp	Size	OPT-125m	OPT-1.3b	OPT-6.7b	Llama-v2 7B
Dense	100%	27.65	14.62	10.86	5.12
Magnitude	90%	767.2	894.4	3464	36746
L-OBD	90%	33.3	20.76	27.20	14259
K-OBD (không update)	90%	27.97	14.68	10.53	5.48
K-OBD	80%	29.89	15.63	11.28	9.14
K-OBD	70%	36.54	18.29	13.03	15.43
K-OBD	50%	75.95	37.68	25.54	46.64
LLM Surgeon (tương quan trong hàng/cột)	80%	29.37	15.27	11.22	7.29
LLM Surgeon (tương quan trong hàng/cột)	50%	51.48	23.81	15.38	25.62
LLM Surgeon (tương quan đầy đủ)	90%	28.01	14.70	10.77	5.25
LLM Surgeon (tương quan đầy đủ)	80%	28.73	15.12	11.02	6.18
LLM Surgeon (tương quan đầy đủ)	70%	31.82	16.24	11.64	7.83
LLM Surgeon (tương quan đầy đủ)	60%	38.47	18.45	12.58	10.39
LLM Surgeon (tương quan đầy đủ)	50%	49.78	22.95	14.90	15.38

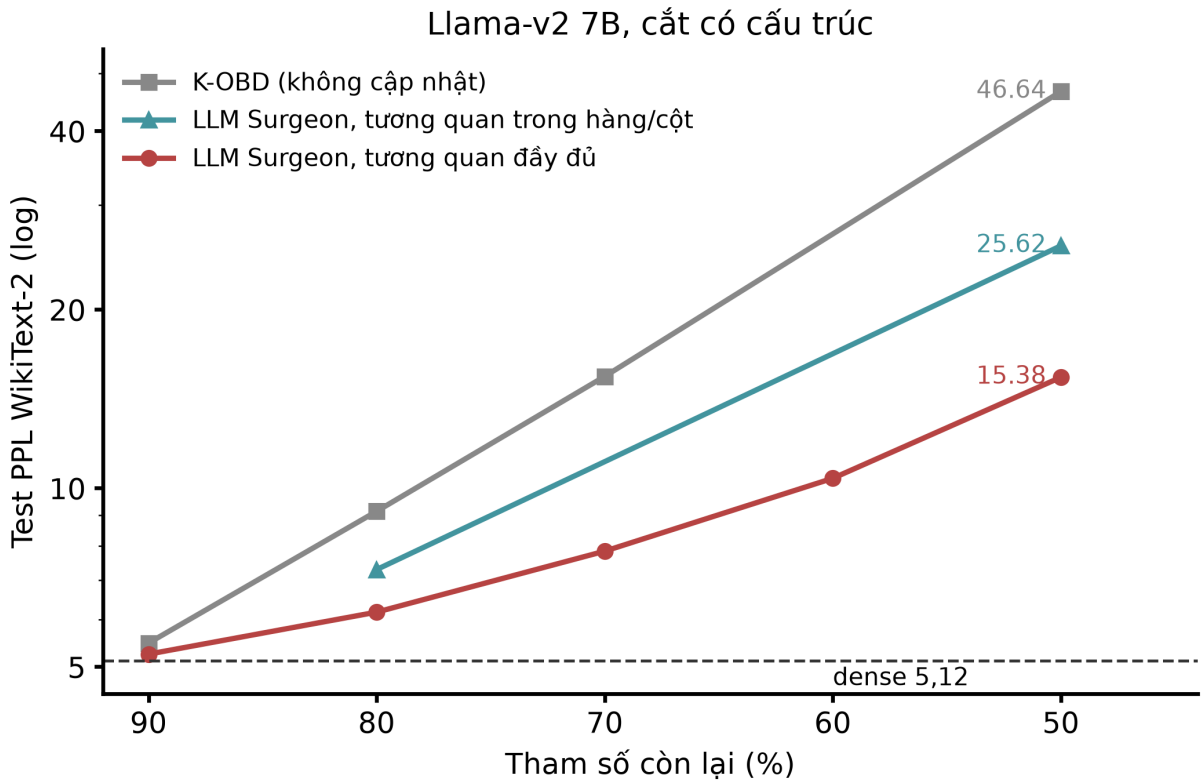
Paper kết luận: nhiều tương quan hơn thì tốt hơn, nhất là trên Llama-v2; cắt tới 20 đến 30% hàng/cột với suy giảm nhỏ; **cắt 50% không tốt hơn một mô hình nhỏ cùng cỡ** (tác giả viết vậy). Hình 1 cho thấy nội suy giữa các cỡ mô hình OPT pretrained.

Cách đọc hình: trục dọc là thang log, đường nét đứt là PPL của mô hình dense (5,12), trục ngang đảo chiều để đi từ nhẹ tới nặng. Ở 50% còn lại, không cập nhật (K-OBD) cho 46,64, cập nhật trong hàng/cột cho 25,62, cập nhật đầy đủ cho 15,38. Chênh lệch này là phần đóng góp của việc bù trọng số có tương quan.

Llama-v2 13B structured (Table 7 PPL; Table 8 zero-shot trung bình 10 tác vụ): dense PPL 4.547 và avg 74.07%; LLM Surgeon 90%: 4.692 / 74.28%; 80%: 5.286 / 72.12%; 70%: 6.207 / 67.51%; 60%: 7.245 / 62.48%; 50%: 9.428 / 57.59%. K-OBD cùng các mức: 4.908, 6.294, 10.08, 13.06, 16.06 (PPL) và avg 74.00, 71.11, 60.76, 54.37, 51.50. Tác giả: cắt 13B 20% mà giảm dưới 2% trên downstream.

Semi-structured 2:4 (Table 3; PPL): OPT-125m / 1.3b / 2.7b / 6.7b: LLM Surgeon 44.64 / 25.10 / 14.64 / 12.10 so với SparseGPT 45.51 / 29.44 / 14.92 / 13.01.

Unstructured (Table 4): Llama-v2 7B: LLM Surgeon (nhiều shot, tương quan đầy đủ) 5.13 (90%), 5.20 (80%), 5.36 (70%), 5.66 (60%), 6.08 (50%) so với SparseGPT 5.49, 5.58, 5.71, 5.94, 6.51. Trên OPT: nhỉnh hơn hoặc ngang SparseGPT (ví dụ OPT-125m 90%: 27.69 so với 27.93; OPT-6.7b 90%: cùng 10.86).



Hình 54: Llama-v2 7B cắt có cấu trúc: PPL WikiText-2 theo phần tham số còn lại, theo Table 1 của ghi chú (vẽ lại từ số liệu trong ghi chú)

Chi phí tính (phụ lục H, Table 11/12): Llama-v2 7B structured: LLM Surgeon 17 giờ 08 phút trên 4 H100 80GB (K-OBD 16 giờ 58 phút); 13B: 1 ngày 9 giờ 26 phút trên 8 H100 80GB. Unstructured Llama-v2 7B: LLM Surgeon 2 ngày 8 giờ 16 phút (4 H100) so với SparseGPT dưới 5 phút trên 1 GPU 32GB.

5. Phân tích / ablation

- **Tương quan trong cập nhật (Table 1):** từ K-OBD (không cập nhật) đến cập nhật trong hàng/cột đến cập nhật đầy đủ, PPL giảm đều; đối với Llama-v2 7B ở 50%: 46.64, 25.62, 15.38. Nghĩa là **phần bù (cập nhật trọng số) đóng góp nhiều** khi cắt structured mạnh.
- **Gradient (G) quan trọng:** K-OBD (có G) tốt hơn L-OBD (chỉ A) nhiều (ví dụ 90% Llama-v2 7B: 5.48 so với 14259). Paper nhấn mạnh việc concurrent work SliceGPT "bỏ qua thông tin gradient, nên kém hơn" (nhận định của tác giả; paper không so trực tiếp bằng số liệu).
- **Số shot (Table 9, OPT-1.3b structured):** 90%: 6/8/10 shot cho PPL 14.70 / 14.70 / 14.72; 80%: 12/16/20 shot cho 15.14 / 15.12 / 15.08; chênh lệch rất nhỏ ở các mức này (ở 90% gần như không đổi; ở 80% giảm nhẹ). Paper nói mức thưa cao cần nhiều shot hơn; bảng đầy đủ có cả các mức thấp hơn mà ta không chép.
- **LoRA xen kẽ (Table 2, OPT-125m structured):** có ích với mô hình nhỏ nhất (ví dụ 50%: 49.78 xuống 36.50) nhưng **không ích với mô hình lớn**; giả thuyết của tác giả: mô hình lớn dễ overfit vài batch WikiText-2.
- **Phân bổ sparsity học được (Fig. 4, OPT-125m):** cắt nhiều hơn ở layer đầu và ít hơn ở layer giữa; cắt nhiều FC hơn attention, nhưng độ chênh nhỏ hơn các phương pháp khác.
- **Downstream:** hiệu năng zero-shot tương quan tốt với PPL và LLM Surgeon hơn K-OBD (Table 5, 6 ở phụ lục; bản text của Table 5 không nêu rõ mô hình nên ta chỉ dựa vào Table 8 cho 13B).

6. Giới hạn

Do tác giả nêu:

- Chi phí tính toán cao: unstructured mất hơn 2 ngày trên 4 H100 (SparseGPT dưới 5 phút); họ lập luận nén chỉ làm một lần, nhưng thừa nhận đây là đánh đổi tính toán lấy chất lượng, và khuyến nghị dùng phương pháp này cho structured.
- Giả thiết gradient bằng 0 (cực tiểu) có thể không đúng; LoRA xen kẽ là giải pháp tuỳ chọn và chỉ ích cho mô hình nhỏ.
- Cắt structured 50% không hơn mô hình nhỏ cùng cỡ.

Do ta nhận xét:

- Thí nghiệm chủ yếu trên WikiText-2 (cả nén lẫn đánh giá): dùng cùng phân phối cho nén và test có thể làm số đẹp hơn; phụ lục có thử C4 và ngôn ngữ khác, nhưng bản text mà ta đọc chỉ là bảng.
- Phép cắt hàng và cột được xử lý **trong từng ma trận**; bản text chính không nói rõ cách đảm bảo kích thước nhất quán giữa các ma trận nối nhau qua residual stream (ví dụ cột của ma trận sau phải khớp hàng của ma trận trước). Cần xem phụ lục A.3, A.4 hoặc mã nguồn trước khi dùng cho mục đích triển khai (tốc độ).
- Độ cong KFAC giả định độc lập giữa activation và gradient, và bỏ qua tương quan giữa các layer; chất lượng xấp xỉ trên các kiến trúc khác chưa được kiểm tra.
- Cần backprop cho cả mô hình nên cần bộ nhớ lớn (paper dùng 4 đến 8 H100 80GB cho 7B/13B).
- Không đo tốc độ chạy thật của mô hình đã cắt.

7. Liên hệ với các bài khác

- **01 (OBD/OBS):** nền lý thuyết; paper viết lại bằng ngôn ngữ Laplace/Gaussian.
- **22, 23 (OBC, SparseGPT):** dùng độ cong của lỗi tái tạo layer; LLM Surgeon thêm G (gradient) và cắt toàn cục. Bảng 3 và 4 so trực tiếp.
- **24 (Wanda):** nhận định của paper (phụ lục H) rằng các thuật toán xét ít tương quan, thậm chí bỏ gradient (Wanda, SparseGPT), rất nhanh nhưng không đủ linh hoạt cho structured.
- **25 (ZipLM):** cùng công thức bù OBS cho nhóm cột; ZipLM chỉ có độ cong cục bộ (không G), nhưng thêm latency table; LLM Surgeon có ngưỡng toàn cục theo chi phí (không theo ms).
- **26 (LLM-Pruner):** điểm bậc nhất bằng gradient trên 10 mẫu, không bù; LLM Surgeon cho cả điểm lẫn bù dựa trên Fisher.
- **29 (SliceGPT):** concurrent; cả hai xoá hàng/cột nhưng LLM Surgeon xoá trong cấu trúc có sẵn, SliceGPT xoay trước để xoá các chiều của residual stream.
- **28 (FLAP):** bù bằng bias thay vì cập nhật trọng số.

8. Gợi ý áp dụng

- Về khái niệm giống điểm sau-bù của ZipLM (25), khác ở chỗ (i) đo bằng loss thật nhờ G chứ không chỉ sai số đầu ra layer, (ii) khi xoá nhiều hàng/cột cùng lúc, chi phí vẫn được cộng độc lập rồi mới tính cập nhật chung, nên xếp hạng bỏ qua tương tác giữa các cấu trúc bị xoá.

9. Thí nghiệm gợi ý

Tỷ số hai đại lượng là ước lượng thực nghiệm cho “bao nhiêu hại là tuyến tính được”.

2. **Bản đồ độ nhạy toàn cục:** chạy bước tính chi phí (không cập nhật) cho mọi hàng/cột của một mô hình nhỏ, sắp xếp toàn cục, và vẽ phần trăm bị cắt ở mỗi layer/loại ma trận để so với phân bổ đều (so Fig. 4).

10. Tóm tắt công thức / khái niệm

Khái niệm	Nội dung
Bài toán	min $\frac{1}{2}\Delta\theta^\top F\Delta\theta$ s.t. $e_k^\top(\theta+\Delta\theta)=0$
Chi phí tổng quát	$\mathcal{L}=\frac{1}{2}(E\theta)^\top(EF^{-1}E^\top)^{-1}(E\theta)$
Cập nhật tổng quát	$\Delta\theta=-F^{-1}E^\top(EF^{-1}E^\top)^{-1}E\theta$
KFAC	$F_l\approx G_l\otimes A_l$; G từ gradient đầu ra, A từ activation đầu vào
Chi phí hàng / cột	$L_r=\frac{1}{2}\theta_r^\top A\theta_r/[G^{-1}]_{rr}$; $L_c=\frac{1}{2}\theta_c^\top G\theta_c/[A^{-1}]_{cc}$
Ngưỡng toàn cục	τ chung, cắt hàng/cột có chi phí $\leq\tau$ cho đến khi còn α
Multi-shot	T shot, lịch α_t tuyến tính, tính lại độ cong mỗi shot
Con số chính	Llama-v2 7B structured: 5.25 (90%), 6.18 (80%), 7.83 (70%); 13B: PPL 5.286 ở 80%, avg 72.12 so với 74.07 gốc; thời gian 17h08 (7B) trên 4 H100

11. Kết luận ngắn

LLM Surgeon áp dụng tinh thần của OBS cho LLM bằng KFAC có gradient, ngưỡng toàn cục, cập nhật có tương quan và nhiều shot, cho kết quả cắt có cấu trúc khả quan ở mức cắt 20 đến 30% (và SOTA cho unstructured/2:4 lúc công bố). Đổi lại, nó tốn tính toán rất nhiều, và thông điệp quan trọng cho ta là: ở structured, **bù trọng số có tương quan và thông tin gradient đều đáng kể**; chúng giải thích nhiều chênh lệch giữa các phương pháp cắt rẻ tiền và đắt tiền.

Link gốc: https://arxiv.org/abs/2312.17244

Bài 28

Fluctuation-based Adaptive Structured Pruning for Large Language Models (2023)

Tác giả: Yongqi An, Xu Zhao, Tao Yu, Ming Tang, Jinqiao Wang (Institute of Automation, Chinese Academy of Sciences; UCAS; Wuhan AI Research; Objecteye Inc.).
Venue / arXiv: arXiv:2312.11983 (19/12/2023). Bản text mang bản quyền AAAI 2024 (Association for the Advancement of Artificial Intelligence).
Nguồn: https://arxiv.org/abs/2312.11983
Chủ đề: Structured pruning LLM **không cần retrain**: điểm "fluctuation" (phương sai của kênh đầu vào nhân chuẩn cột trọng số), phân bổ cấu trúc toàn cục thích nghi bằng chuẩn hoá điểm, và bù lỗi bằng bias lấy từ giá trị trung bình (baseline) của kênh bị cắt.
Vị trí trong lộ trình: Đọc sau 24 (Wanda), 26 (LLM-Pruner), 25 (ZipLM). FLAP lấy khung "điểm dựa trên activation" của Wanda, chuyển sang cắt cột/đầu (structured) và thay LoRA của LLM-Pruner bằng một phép bù cực rẻ. Bài kế: 29 (SliceGPT), 31 (ShortGPT).
Chương nền liên quan: nền 03: Độ quan trọng của trọng số

> Ghi chú học tập bằng tiếng Việt, không phải bản dịch. Số liệu lấy từ paper (có ghi chú những chỗ văn bản và bảng của paper không khớp nhau).

1. Bối cảnh

Với LLM, huấn luyện lại là xa xỉ nên cần **retraining-free pruning**. Nhưng các phương pháp retraining-free trước đó (Wanda, SparseGPT) đều **unstructured**: cần phần cứng/kernel thưa đặc biệt mới nhanh. Structured pruning (xoá cả hàng/cột) giảm tham số và thời gian chạy trên mọi phần cứng, nhưng cần ba thứ mà paper liệt kê:

1. một **metric cấu trúc** đánh giá cả cột/hàng (các phương pháp unstructured chấm từng phần tử);
2. cách **tìm cấu trúc toàn cục** (mỗi lớp và mỗi module nên cắt bao nhiêu);
3. **bù** để giảm tổn thất mà không huấn luyện lại. LLM-Pruner (26) làm structured nhưng phải tính gradient toàn cục và LoRA tinh chỉnh, nên khó mở rộng.

Khái niệm: paper phân biệt hai nguyên tắc lựa chọn cục bộ: **low-damage** (ít hại, tinh thần OBD, ví dụ Wanda) và **easy-recoverability** (dễ khôi phục, tinh thần OBS, ví dụ SparseGPT: tìm phần mà trọng số khác bù được). FLAP thuộc nhóm "dễ khôi phục".

Quan sát then chốt: trong LLaMA, **một số kênh của hidden state có độ ổn định theo mẫu cấu trúc** (Fig. 2): giá trị của kênh đó ít đổi giữa các mẫu và token, nên thay nó bằng giá trị trung bình thì đầu ra gần như không đổi.

2. Ý tưởng chính

Một cột j của ma trận trọng số nhân với kênh đầu vào x_j. Nếu x_j gần như hằng số (phương sai nhỏ qua dữ liệu), ta có thể xoá cột đó và **cộng một bias cố định** $W_{:,j}\bar{x}_j$ vào đầu ra, thay cho việc tính $W_{:,j}x_j$; sai khác chỉ là $W_{:,j}(x_j - \bar{x}_j)$ có kích thước nhỏ. Vậy điểm "dễ cắt" của cột j chính là *phương sai của x_j nhân bình phương chuẩn của cột $W_{:,j}$*. Để so sánh được giữa các lớp và giữa attention với MLP, chuẩn hoá điểm (mean 0, độ lệch chuẩn 1) theo từng lớp và từng module, rồi lấy ngưỡng toàn cục, nhờ đó tự động quyết định layer/module nào bị cắt nhiều. Tất cả chỉ cần một forward pass, 3 đến 5 phút trên một GPU cho LLaMA-7B.

3. Phương pháp chi tiết

3.1 Bài toán layer-wise

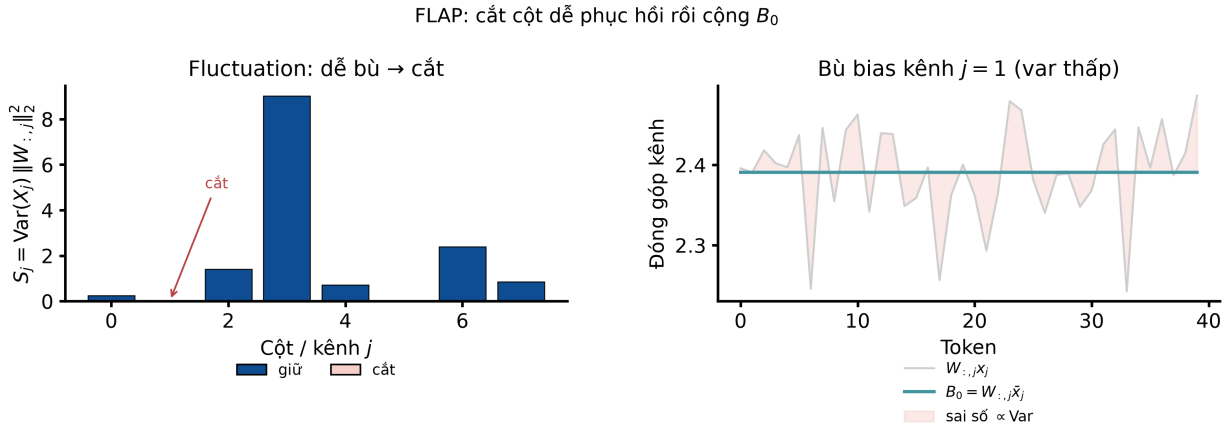
Cho đầu vào X^ℓ ($N \times C_{in} \times L$) và trọng số W^ℓ ($C_{out} \times C_{in}$); mask $M^\ell \in \mathbb{R}^{C_{in}}$ chọn cột giữ lại:

$$\arg \min_{M^\ell, \widehat{W}^\ell} \|W^\ell X^\ell - (M^\ell \odot \widehat{W}^\ell) X^\ell\|_2^2.$$

Với self-attention, cột được cắt theo nhóm (một đầu, ví dụ group size 128).

3.2 Bù bằng bias dựa trên giá trị trung bình

Phân tách đầu ra: $W^\ell X^\ell = (M^\ell \odot W^\ell)X^\ell + ((1 - M^\ell) \odot W^\ell)X^\ell$ (phần giữ, phần bị xoá). Gọi giá trị baseline của kênh j là trung bình trên mọi mẫu và vị trí:



FLAP: điểm $S = \mathrm{Var}(X)\,\|W\|_2^2$ chọn cột dễ bù; thay Wx bằng bias $B_0 = W\bar{x}$. (minh hoạ, tự tính)

$$\bar{X}^\ell_{:,j,:} = \frac{1}{NL} \sum_{n=1}^{N} \sum_{k=1}^{L} X^\ell_{n,j,k}.$$

Phần bị xoá được xấp xỉ bằng bias cố định (Eq. 4):

$$\boxed{B_0^\ell = W^\ell((1 - M^\ell) \odot \bar{X}^\ell), \qquad W^\ell X^\ell \approx (M^\ell \odot W^\ell)X^\ell + B_0^\ell}$$

Bias này có hạng 1 (cùng một vector cho mọi token), nên tác giả coi nó có vai trò gần với LoRA (rank $r = 1$). Nếu layer gốc không có bias thì thêm bias mới (phép cộng một vector).

3.3 Fluctuation metric

Điểm cho nhóm cột $W^\ell_{:,j}$ (Eq. 5):

$$\boxed{S^\ell_{:,j} = \frac{1}{N-1} \sum_{n=1}^{N} (X^\ell_{n,j,:} - \bar{X}^\ell_{:,j,:})^2 \cdot \|W^\ell_{:,j}\|_2^2}$$

tức *phương sai mẫu của kênh* j (với hiệu chỉnh Bessel $\frac{1}{N-1}$) nhân với bình phương chuẩn cột j của trọng số. Nhận xét: đây đúng bằng sai số bình phương kỳ vọng $\mathbb{E}\|W_{:,j}(x_j - \bar{x}_j)\|^2 = \mathrm{Var}(x_j)\,\|W_{:,j}\|^2$ khi thay kênh

bằng trung bình của nó, bỏ qua tương quan với các kênh khác. Nghĩa là điểm là *sai số còn lại sau khi bù bằng hằng số*, và xếp theo đó là chọn cột "dễ phục hồi" nhất.

Thống kê mean và variance được cập nhật theo kiểu streaming bằng thuật toán Welford (không lưu lại toàn bộ activation).

3.4 Tìm cấu trúc thích nghi toàn cục

Độ lớn điểm khác nhau nhiều giữa các layer và giữa attention với MLP (Fig. 3), nên không thể xếp hạng chung. Chuẩn hoá điểm trong từng layer và từng module:

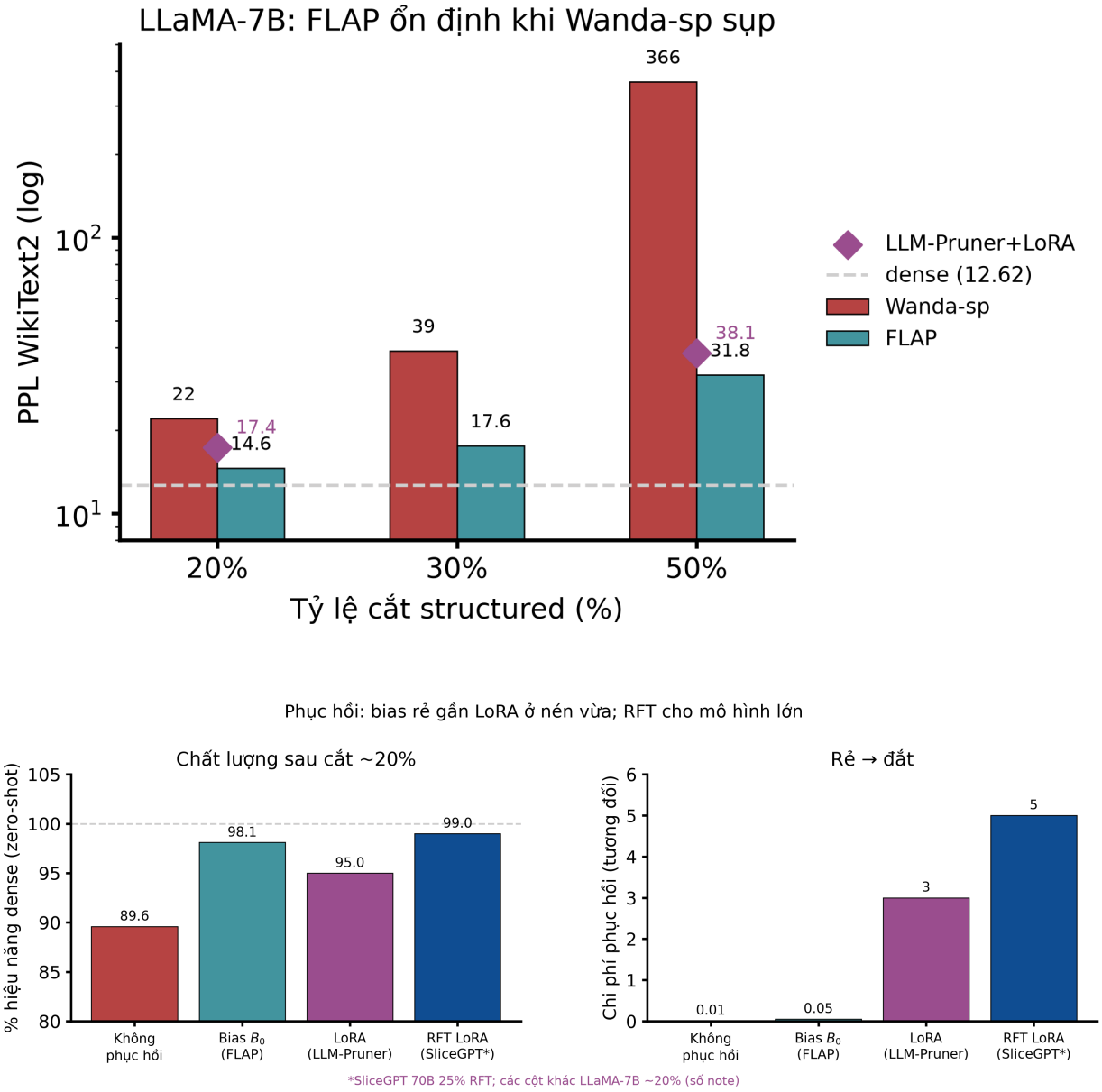
$$\widehat{S}^{\ell}_{:,j} = \frac{S^{\ell}_{:,j} - \mathbb{E}[S^{\ell}_{:,j}]}{\left(\mathbb{E}[(S^{\ell}_{:,j} - \mathbb{E}[S^{\ell}_{:,j}])^2]\right)^{1/2}}$$

Với attention, lấy điểm theo cột rồi chuẩn hoá, sau đó cộng điểm các cột của cùng một head; nhân hệ số chuẩn hoá (ví dụ 512/3) để việc bỏ một head và bỏ một neuron MLP tính theo cùng "số phần tử bị xoá". Sau đó gộp tất cả điểm đã chuẩn hoá của mọi layer và module, chọn top-K theo tỷ lệ cắt toàn cục để ra mask.

```
FLAP(model, calib, p):          # p = tỷ lệ cắt toàn cục
    for each layer l, module m in {attn, mlp}:
        # một forward pass, thống kê streaming (Welford)
        mean_x[l,m] (C_in,), var_x[l,m] (C_in,)
        S[l,m][j] = var_x[l,m][j] * ||W[l,m][:, j]||^2
        S_hat[l,m] = (S - mean(S)) / std(S)          # chuẩn hoá trong từng module
    S_all = concat(S_hat[l,m] for all l,m)           # head attention: cộng các cột + hệ
    ↪  số
    mask  = top-(1-p) fraction of S_all              # một ngưỡng toàn cục
    for each (l,m):
        B0[l,m] = W[l,m][:, pruned] @ mean_x[l,m][pruned]
        W[l,m]  = W[l,m][:, kept];  bias[l,m] += B0
    return pruned model
```

4. Thí nghiệm và kết quả chính

Mô hình: LLaMA-7B/13B/30B/65B (và Vicuna-7B ở phụ lục). **Đánh giá:** PPL WikiText2 và trung bình 7 tác vụ zero-shot (cùng bộ với LLM-Pruner). **Baseline:** Wanda-sp (Wanda mở rộng sang structured: điểm cột = $\sum_i |W_{ij}| \cdot \|X_j\|_2$) và LLM-Pruner (có và không LoRA, dấu * = có LoRA). Phụ lục A: "đã thử sửa SparseGPT sang structured nhưng không thu được kết quả hợp lý". **Calibration:** 1024 mẫu. **Phần cứng:** một A100 40GB.



So sánh phục hồi: không bù / bias FLAP / LoRA / RFT — chất lượng vs chi phí. (minh hoạ, tự tính)

PPL WikiText2 LLaMA-7B: FLAP ổn định tới 50% trong khi Wanda-sp sụp (366). (minh hoạ, tự tính)

PPL WikiText2 (Table 1):

Tỷ lệ	Phương pháp	7B	13B	30B	65B
0%	Dense	12.62	10.81	9.11	8.21
20%	Wanda-sp	22.12	16.83	11.66	11.76
20%	LLM-Pruner	19.77	16.01	-	-
20%	LLM-Pruner*	17.37	15.18	-	-
20%	**FLAP**	**14.62**	**13.66**	**10.86**	**9.79**
30%	Wanda-sp	38.88	22.89	14.90	14.64
30%	**FLAP**	**17.62**	**15.65**	**12.49**	**10.90**
50%	Wanda-sp	366.43	160.49	67.46	42.85
50%	LLM-Pruner	112.44	-	-	-

Tỷ lệ	Phương pháp	7B	13B	30B	65B
50%	LLM-Pruner*	38.12	-	-	-
50%	**FLAP**	**31.80**	**24.20**	**19.36**	**15.30**

(Lưu ý: đoạn văn của paper nói LLM-Pruner ở 50% có PPL 130.97, còn 39.02 sau LoRA; bảng ghi 112.44 và 38.12. Hai chỗ không khớp nhau; ta dùng số trong bảng.)

Zero-shot LLaMA-7B (Table 2, trung bình 7 tác vụ): dense 63.25.

Tỷ lệ	Wanda-sp	LLM-Pruner	LLM-Pruner (LoRA)	FLAP
20%	63.35	56.82	60.07	62.08
50%	37.83	41.58	48.98	50.37

Ở 20% Wanda-sp thậm chí hơi vượt dense (63.35), mà tác giả coi là dấu hiệu dư thừa cấu trúc; nhưng ở 50% Wanda-sp sụp. LLaMA-13B 20% (Table 5): dense 64.97; LLM-Pruner 61.79, LLM-Pruner (LoRA) 64.02, FLAP 64.52.

Tốc độ (Table 4, A100, LLaMA-7B):

Tỷ lệ	Phương pháp	#Params	Bộ nhớ	Token/s
0%	gốc	6.74B	12916.5 MiB	25.84
20%	Wanda (unstructured)	6.74B	12916.5 MiB	25.67 (khoảng 0%)
20%	LLM-Pruner	5.42B	10387.2 MiB	32.57 (+26%)
20%	FLAP	5.07B	9726.2 MiB	33.90 (+31%)
50%	LLM-Pruner	3.35B	6547.1 MiB	40.95 (+58%)
50%	FLAP	3.26B	6268.2 MiB	42.88 (+66%)

(Văn bản của paper mô tả “giảm tham số 52% và tăng tốc 66% ở 20%, 25% và 31% ở 50%”: hai cặp bị đảo ngược so với bảng. Theo bảng: 20% cho +31%, 50% cho +66%.) Fig. 7: mô hình cắt hưởng lợi nhiều hơn khi batch lớn, vì chưa chạm nút cổ chai throughput.

5. Phân tích / ablation

- **Metric × cấu trúc (Table 3, PPL LLaMA-7B 50%, đã có bù bias):** thử 3 metric: WIFN (chuẩn activation nhân |W|, kiểu low-damage), IFV (chỉ phương sai activation), WIFV (phương sai × chuẩn cột, của FLAP); và 4 cấu trúc toàn cục: UL-UM (đều cho layer và module), UL-MM (đều layer, tỷ lệ module thủ công), AL-MM (thích nghi layer, thủ công module), AL-AM (thích nghi cả hai).

Metric	UL-UM	UL-MM	AL-MM	AL-AM
WIFN	84.79	128.75	34.50	34.09
IFV	55.41	48.87	35.72	33.33
WIFV (FLAP)	57.57	38.31	34.82	**31.80**

AL-AM tốt nhất với mọi metric, tức **phân bổ thích nghi quan trọng hơn chọn metric**; FLAP tốt nhất ở cấu hình mặc định.

- **Bù bias (Fig. 5):** so PPL có và không có bù ở các tỷ lệ; lợi ích tăng theo tỷ lệ cắt (chỉ có số trong hình, không có số bảng nên ta không chép). Tác giả kết luận bias compensation có thể thay phần lớn retraining/PEFT.
- **Số mẫu calibration (Fig. 6):** hiệu năng tốt dần khi tăng mẫu; mặc định 1024; cả quy trình cắt LLaMA-7B xong trong 3 đến 5 phút trên một GPU.
- **Tỷ lệ cắt (Fig. 4):** FLAP ổn định khi tăng tỷ lệ, Wanda-sp suy giảm mạnh, LLM-Pruner cần LoRA để chịu được 50%.

6. Giới hạn

Do tác giả nêu: bài chủ yếu giải thích bằng quan sát thực nghiệm; phần kết nói bias compensation "có thể phần lớn thay" retrain nhưng không có mục Limitations riêng. Họ cũng nói thử mở rộng SparseGPT sang structured nhưng không ra kết quả hợp lý.

Do ta nhận xét:

- **Bù chỉ là hằng số** (cùng một vector cho mọi token). Nếu kênh bị cắt có phương sai lớn hoặc tương quan mạnh với kênh còn lại (cái mà ZipLM/LLM Surgeon/SliceGPT bù được), FLAP không bù. Điểm cũng bỏ qua tương quan giữa các kênh (chỉ dùng phương sai từng kênh).
- Giả định "kênh ổn định" có thể là đặc trưng của LLaMA; sau LayerNorm/RMSNorm hoặc với activation dạng SwiGLU gating thì khác. Fig. 2 chỉ là minh hoạ cho một số kênh, không phải thống kê.
- Đánh giá chủ yếu bằng PPL WikiText2 (+7 tác vụ trắc nghiệm); không đánh giá sinh văn bản dài hay tác vụ nặng, và hầu hết số liệu ở LLaMA gốc (không phải mô hình chỉ thị).
- Văn bản và bảng có một vài chỗ không khớp (đã ghi ở trên), nên nên kiểm tra mã nguồn khi tái hiện.
- Không bàn đến việc kết hợp với fine-tune/distillation (dù bù bias hẳn không cản trở).

7. Liên hệ với các bài khác

- **24 (Wanda):** cùng ý "activation-aware"; Wanda-sp là baseline structured và sụp ở 50%. FLAP khác ở chỗ dùng **phương sai** quanh trung bình (không phải chuẩn ℓ_2 thô) và có bù bias.
- **26 (LLM-Pruner):** FLAP không retrain mà vượt cả LLM-Pruner có LoRA; đối trọng "bù bằng phép tính" so với "bù bằng huấn luyện".
- **23 (SparseGPT), 22 (OBC), 25 (ZipLM), 27 (LLM Surgeon):** bù đầy đủ bằng Hessian nghịch đảo. FLAP là phiên bản "bậc 0" của việc đó: chỉ bù thành phần hằng số.
- **29 (SliceGPT):** bù bằng phép biến đổi tuyến tính tối ưu (PCA) rồi cắt; so với FLAP là bù tuyến tính đầy đủ thay vì hằng số.
- **31 (ShortGPT) đến 34 (SLEB):** cắt cả layer; FLAP là phía cắt cấu trúc mịn (cột/đầu).
- **06 (ThiNet):** cũng bù sau cắt kênh (bằng bình phương tối thiểu); FLAP đơn giản hơn nhiều.

8. Gợi ý áp dụng

- Chỗ này là gợi ý thí nghiệm, không phải kết quả của paper.
- **Nhận xét:** ở 50% FLAP vẫn còn PPL 31.80 so với 12.62 của dense (LLaMA-7B, không retrain), tức là bù hằng số không đủ để cắt mạnh mà không cần huấn luyện thêm; distillation vẫn cần. Paper không so sánh với cắt cả layer, nên đây chỉ là mốc tham khảo.
- **Tốc độ batch 1:** Table 4 đo token/s (batch không nêu; Fig. 7 có batch 1 đến 8), tăng tốc 31% ở 20% tham số và 66% ở 50%. Với batch-1 trên GPU nhỏ cần đo lại.

9. Thí nghiệm gợi ý

2. **Bù hằng số so với bù tuyến tính đầy đủ:** cắt 30% neuron MLP của một layer; so sai số đầu ra layer trong ba trường hợp: không bù, bù bias (FLAP), bù OBS (ZipLM-style).

3. **Chuẩn hoá thích nghi:** so cắt đều 30% mọi layer và cắt bằng ngưỡng toàn cục trên điểm đã chuẩn hoá; báo cáo CER và tốc độ batch-1.

10. Tóm tắt công thức / khái niệm

Khái niệm	Nội dung
Điểm fluctuation	$S^{\ell}_{:,j} = \frac{1}{N-1}\sum_n (X_{n,j,:} - \bar{X}_{:,j,:})^2 \cdot \|W_{:,j}\|_2^2$
Bias bù	$B_0 = W((1-M)\odot \bar{X})$; $WX \approx (M \odot W)X + B_0$
Chuẩn hoá	$\widehat{S} = (S - \mathbb{E}[S])/\text{std}(S)$ theo từng layer, từng module
Cấu trúc	AL-AM: thích nghi qua layer và module bằng một ngưỡng toàn cục
Streaming	Welford cho mean/variance
Cấu hình	1024 calibration, một A100 40GB, 3 đến 5 phút (LLaMA-7B)
Con số chính	LLaMA-7B 50%: PPL 31.80 (LLM-Pruner+LoRA 38.12, Wanda-sp 366.43), zero-shot 50.37 so với 48.98 và 37.83; 20%: PPL 14.62

11. Kết luận ngắn

FLAP cho thấy structured pruning không huấn luyện lại khả thi nếu ba thứ khớp nhau: điểm đo "độ dễ bù" (phương sai kênh × chuẩn cột), phân bổ thích nghi toàn cục qua chuẩn hoá điểm, và phép bù bias rẻ tiền.

Link gốc: https://arxiv.org/abs/2312.11983

Bài 29

SliceGPT: Compress Large Language Models by Deleting Rows and Columns (2024)

Tác giả: Saleh Ashkboos, Maximilian L. Croci, Marcelo Gennari do Nascimento, Torsten Hoefler, James Hensman (ETH Zurich; Microsoft Research; Microsoft).
Venue / arXiv: arXiv:2401.15024 (26/01/2024, bản v2 02/2024). Published as a conference paper at ICLR 2024.
Nguồn: https://arxiv.org/abs/2401.15024
Chủ đề: Cắt **chiều nhúng (embedding / residual-stream dimension)** của Transformer sau huấn luyện: xoay không gian bằng ma trận trực giao theo PCA (computational invariance) rồi xoá các hàng/cột tương ứng thành phần chính nhỏ, thu được ma trận dày nhỏ hơn.
Vị trí trong lộ trình: Đọc sau 06 (ThiNet, cắt kênh rồi bù tuyến tính), 23 (SparseGPT), 26 (LLM-Pruner), 27 (LLM Surgeon). Bài này khác cả nhóm: thay vì chọn cấu trúc để xoá, nó **đổi hệ trục** để cấu trúc xoá được trở nên ít quan trọng. Bài kế: 30 (MaskLLM), 31 (ShortGPT).

> Ghi chú học tập bằng tiếng Việt, không phải bản dịch. Số liệu lấy từ paper.

1. Bối cảnh

Pruning sau huấn luyện (SparseGPT, Wanda; bài 23, 24) tạo ma trận **thưa** nhưng cần cấu trúc dữ liệu phụ và kernel đặc biệt; hướng 2:4 (bài 18) chỉ tăng tốc hạn chế và chất lượng giảm. Structured pruning kiểu LLM-Pruner (26) và LLM Surgeon (27) cần tinh chỉnh hoặc nhiều tính toán. Bài báo nêu rằng baseline đơn giản "xoá cột/hàng có chuẩn nhỏ nhất" rất kém: WikiText-2 PPL "vọt lên hàng nghìn" chỉ sau khi xoá vài cột.

Các khái niệm cần có:

- **RMSNorm / LayerNorm:** chuẩn hoá từng hàng (từng token) của tín hiệu. RMSNorm chia hàng cho chuẩn L2 của nó, rồi nhân hệ số. LayerNorm = trừ trung bình hàng rồi chia độ lệch chuẩn.
- **Signal matrix X:** ma trận kích thước (độ dài chuỗi N) $\times$ (chiều nhúng D) chảy qua các block.
- **Residual stream:** đường skip cộng đầu ra từng block vào tín hiệu chính.
- **PCA:** chiếu dữ liệu lên các hướng có phương sai lớn nhất.
- **Ma trận trực giao Q:** $Q^\top Q = QQ^\top = I$, không đổi chuẩn vector.

2. Ý tưởng chính

Giữa các block của một Transformer dùng RMSNorm, ta có thể **xoay** tín hiệu bằng một ma trận trực giao Q trước RMSNorm và xoay ngược bằng $Q^\top$ sau RMSNorm mà mô hình tính ra đúng như cũ (vì phép chuẩn hoá chỉ phụ thuộc chuẩn của hàng, mà phép xoay không đổi chuẩn). Điều này gọi là **computational invariance**. Chọn Q là các thành phần chính (PCA) của tín hiệu giữa các block, thì sau khi xoay, năng lượng tập trung vào vài chiều đầu; các chiều cuối ít quan trọng nên **xoá được** bằng cách bỏ hàng của W_{in} và cột của W_{out} tương ứng. Kết quả là mọi ma trận nhỏ hơn và tín hiệu giữa các block cũng nhỏ hơn (giảm chiều nhúng), nên tăng tốc bằng kernel dày thông thường và cần ít GPU hơn. Không cần tinh chỉnh (nhưng tinh chỉnh nhẹ vẫn cải thiện).

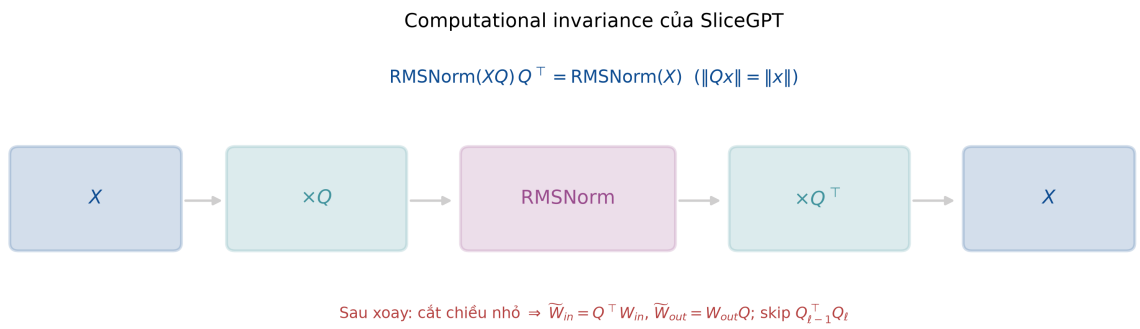
3. Phương pháp chi tiết

3.1 Cấu trúc Transformer dưới dạng $\sigma(XW_{in} + b_{in})W_{out} + b_{out}$

Mỗi khối attention hoặc FFN đều viết được dạng trên: W_{in} là ghép của W_k, W_q, W_v (attention) hoặc W_1 (và gate) (FFN), σ là phép phi tuyến theo điểm (hoặc multi-head attention), W_{out} là ma trận đầu ra. Giữa các khối có RMSNorm và đường residual (Algorithm 1: $X \leftarrow \text{Norm}_\ell(X + Z)$).

3.2 Computational invariance

RMSNorm áp dụng $x \leftarrow x/\|x\|$ cho từng hàng. Với Q trực giao:



Computational invariance: $\text{RMSNorm}(XQ)Q^\top = \text{RMSNorm}(X)$ rồi cắt chiều nhỏ của residual. (minh hoạ, tự tính)

$$\boxed{\text{RMSNorm}(XQ)\,Q^\top = \text{RMSNorm}(X)} \qquad (2)$$

(vì $\|Qx\| = \|x\|$). **Định lý 1:** mạng sau đây tương đương mạng gốc nếu thay

$$\widetilde{W}_{embd} = W_{embd}Q, \quad \widetilde{W}^\ell_{in} = Q^\top W^\ell_{in}, \quad \widetilde{W}^\ell_{out} = W^\ell_{out}Q, \quad \tilde{b}^\ell_{out} = Q^\top b^\ell_{out}, \quad \widetilde{W}_{head} = Q^\top W_{head}$$

(với b_{in}, b_{head} giữ nguyên). Tức là mọi thứ đọc từ residual stream thì nhận $Q^\top$, mọi thứ ghi vào thì nhân Q, và residual stream được "xoay" nhất quán từ embedding đến đầu LM.

LayerNorm sang RMSNorm. Mạng dùng LayerNorm (OPT, Phi-2) phải đổi trước: hấp thụ hệ số $\text{diag}(\alpha)$ vào W_{in} kế tiếp, và việc trừ trung bình (ma trận $M = I - \frac{1}{D}\mathbf{1}\mathbf{1}^\top$) vào W_{out} của block trước; W_{embd} cũng được trừ trung bình, W_{head} được co theo hệ số cuối. Phép đổi này không đổi đầu ra.

3.3 Mỗi block một phép xoay, và đường skip

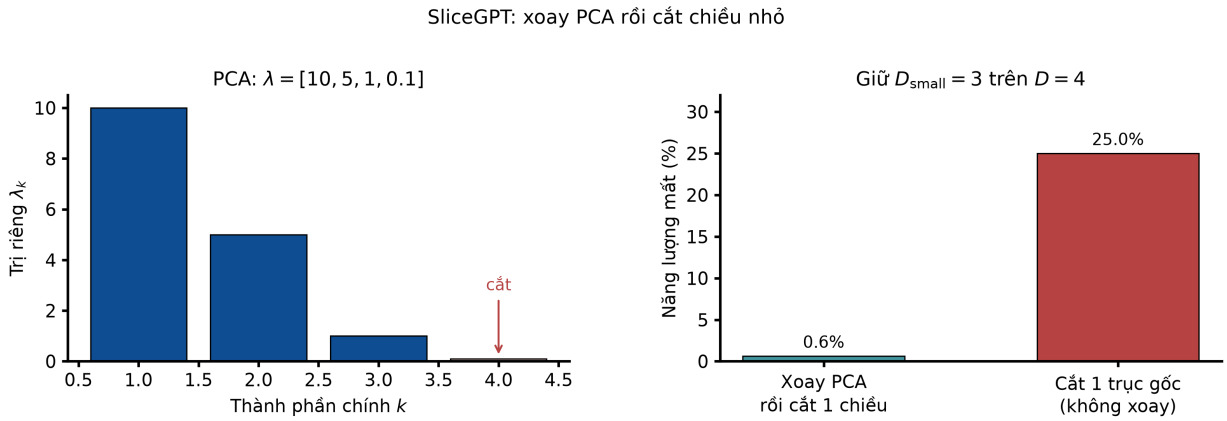
Tín hiệu ở các block không cùng hệ trục, nên PCA cho **một** Q_ℓ **riêng cho mỗi block**. Khi Q_ℓ khác nhau, đường skip (cộng đầu vào block vào đầu ra) bị lệch hệ trục, nên phải chèn một phép biến đổi tuyến tính vào đường skip:

$$\text{residual}: \; X \leftarrow X\,(Q^\top_{\ell-1}Q_\ell)$$

Ma trận $D \times D$ này **không tính trước được** vào trọng số khác và tạo ra một chi phí nhỏ (nhưng thật) $D \times D$; paper nói vẫn thấy tăng tốc tổng thể. Sau khi slice, nó thành ma trận $D_{small} \times D_{small}$ (xoá cả hàng lẫn cột).

3.4 Chọn Q_ℓ bằng PCA và cắt

Chạy một tập calibration qua mô hình (đã đổi sang RMSNorm). Gọi $X_{\ell,i}$ là đầu ra RMSNorm thứ ℓ của chuỗi i:



Phổ PCA $\lambda = [10, 5, 1, 0.1]$: xoay rồi cắt mất $\approx 0.6\%$ năng lượng, cắt trục gốc mất $\approx 25\%$. (minh hoạ, tự tính)

$$C_\ell = \sum_i X_{\ell,i}^\top X_{\ell,i}, \qquad Q_\ell = \text{eigenvectors}(C_\ell) \text{ xếp theo trị riêng giảm dần.}$$

Dùng đầu ra của mạng *đã biến đổi* để tính Q của các block sau (làm tuần tự). PCA chuẩn: $Z = XQD$, $\widetilde{X} = ZD^\top Q^\top$, trong đó D là ma trận xoá (D_{small} cột của ma trận đơn vị $D \times D$). Phép tái tạo này **tối ưu L2**: QD là ánh xạ tuyến tính (hạng D_{small}) cực tiểu hoá $\|X - \widetilde{X}\|^2$. Vì không bao giờ dựng X đầy đủ, ta áp D ngay vào các phép toán trước và sau: **xoá hàng của W_{in} và cột của W_{out}, W_{embd}**, và cả hàng lẫn cột của $Q_{\ell-1}^\top Q_\ell$ trên đường skip.

```
SliceGPT(model, calib, keep_ratio):
    model = layernorm_to_rmsnorm(model)          # hấp thụ diag(alpha), ma trận M
    X = embed(calib)
    for each block l = 0..L:
        Xn  = RMSNorm(X)
        C_l = sum_i Xn_i^T Xn_i                  # (D x D) hiệp phương sai
        Q_l = eigvecs(C_l)  sorted desc          # FP64 để tránh sai số
        # xoay:  W_in <- Q_l^T W_in ;  W_out_prev <- W_out_prev Q_l ;  skip: Q_{l-1}^T
        ↪  Q_l
        # cắt:   giữ D_small chiều đầu của Q_l
        X = forward_block_with_rotation(X, Q_l)  # dùng đầu ra đã xoay cho block kế
    slice rows of W_in, cols of W_out, W_embd; rows+cols of skip matrices
    # (tuỳ chọn) recovery fine-tuning bằng LoRA
```

Ví dụ nhỏ tự dựng (không phải của paper): $D = 4$ và dữ liệu tín hiệu có trị riêng $[10, 5, 1, 0.1]$. Xoay theo vector riêng rồi giữ 3 chiều đầu: năng lượng mất đi là $0.1/16.1 \approx 0.6\%$. Nếu cắt thẳng 1 trong 4 trục gốc (không xoay) mà năng lượng rải đều trên 4 trục (khoảng 25% mỗi trục) thì mất tới khoảng 25%. Chính phép xoay biến "mọi trục đều quan trọng" thành "vài trục rất quan trọng, vài trục gần như vô nghĩa".

Slicing theo mức khác nhau giữa các layer. Thay vì cắt đều, đặt phần phương sai muốn bỏ ở mỗi PCA (phụ lục A.4): xem mục 5.

4. Thí nghiệm và kết quả chính

Thiết lập: Hugging Face + PyTorch; Q tính trên một H100 80GB (khoảng 3.5 giờ cho LLaMA-2 70B), **độ chính xác kép** cho PCA. **Mô hình:** OPT (125M đến 66B), LLaMA-2 (7B, 13B, 70B), Phi-2 (chỉ zero-shot). **Calibration:** WikiText-2 train hoặc Alpaca train; cỡ 1024 mẫu, độ dài 2048 cho kết quả chính. **Baseline:** SparseGPT 2:4 (vì là sơ đồ thưa duy nhất có tăng tốc); thử cả "xoá cột chuẩn nhỏ" nhưng PPL lên hàng nghìn

nên bỏ. **Tỷ lệ cắt** tính trên **toàn bộ tham số kể cả embedding**.

PPL WikiText-2 (Table 1):

Phương pháp	OPT-125M	OPT-6.7B	OPT-66B	LLaMA-2 7B	LLaMA-2 13B	LLaMA-2 70B
Dense	27.64	10.85	9.33	5.47	4.88	3.32
SparseGPT 2:4	45.07	13.00	10.22	8.69	7.07	4.98
SliceGPT 10%	29.34	10.92	9.43	5.89	5.21	3.69
SliceGPT 20%	34.26	11.48	9.57	6.64	5.81	4.25
SliceGPT 25%	37.74	11.90	9.68	7.24	6.30	4.60
SliceGPT 30%	43.98	12.51	9.85	8.12	6.99	5.05

Nhận xét của paper: SparseGPT 2:4 kém SliceGPT 25% trên mọi LLaMA-2; với OPT, SliceGPT 30% vượt 2:4 ở mọi cỡ trừ 2.7B; OPT nén tốt hơn LLaMA-2; mô hình càng lớn càng nén tốt hơn.

Zero-shot (5 tác vụ: PIQA, WinoGrande, HellaSwag, ARC-e, ARC-c): lệch dense ít khi mô hình lớn. Ở 25% slice: OPT-66B giữ 99%, LLaMA-2 70B giữ 96%, Phi-2 giữ 87% hiệu năng dense (phần Kết luận). **Recovery fine-tuning (RFT) bằng LoRA** (rank 32, alpha 10; khoảng 1k chuỗi WikiText-2 hoặc 5k chuỗi Alpaca): LLaMA-2 70B cắt 25% đạt 99% dense và Phi-2 90%; LLaMA-2 70B cắt 30% với RFT Alpaca: 74.3% so với 76.6% dense (mô hình khoảng 51.6B tham số); Phi-2 cắt 25% với RFT: 65.2% so với 72.2% dense (khoảng 2.2B tham số, giữ 90.3%). Không tìm được cấu hình RFT cải thiện OPT. RFT trên WikiText-2 yếu hơn Alpaca nhiều (tác giả đổ cho độ giống giữa Alpaca và các tác vụ).

Tốc độ:

- Throughput H100 80GB (độ dài 128, tăng batch tối đa): mô hình cắt 25% đạt tới **1.55×**; cắt 50% thì mô hình lớn nhất chỉ cần 1 GPU thay vì 2, throughput tăng **3.13×** và **1.87×** (tức 6.26× và 3.75× với số GPU cố định).
- Sinh 1 token, batch 1, độ dài 128 (Table 2): LLaMA-2 70B trên A100 40GB: dense 125 ms/4 GPU (500 GPUms), cắt 25%: 110 ms/3 GPU (330 GPUms, còn 66%); trên Quadro RTX6000 24GB: dense 252 ms/7 GPU (1764 GPUms), 25%: 215 ms/5 GPU (1075 GPUms, còn 64%). OPT 66B tương tự (A100: 456 giảm còn 306 GPUms). Tốc độ đơn lẻ nhanh hơn 16 đến 17% (RTX6000) và 11 đến 13% (A100).
- Thời gian nhân ma trận (phụ lục, Table 12): LLaMA-2 7B tổng 3.99 ms (dense), 2.70 ms (2:4, 1.48×), 2.99 ms (SliceGPT 25%, 1.33×), 2.06 ms (SliceGPT 50%, 1.94×); LLaMA-2 70B 25%: 1.32× bằng SparseGPT 2:4 nhưng PPL 4.60 so với 4.98.
- **Chi phí cắt** (Table 3, cắt 30% + RFT Alpaca): LLaMA-2 7B 0h44m + 0h23m = 1h07m (1 H100); 13B 1h08m + 0h44m = 1h52m; 70B 3h31m + 1h35m (4 H100) = 5h06m; Phi-2 0h49m + 1h59m = 2h48m (1 V100 32GB).

5. Phân tích / ablation

- **Độ chính xác số học của PCA (phụ lục A.2, Table 4):** dùng FP32 thay FP64 làm hỏng mô hình lớn: LLaMA-2 70B 25%: PPL 7.01 (FP32, calibration 128 mẫu) so với 4.89 (FP64, Table 5 cùng cỡ calibration); 7B 25%: 8.52 so với 7.56. Hệ quả thực hành: phải dùng độ chính xác kép.
- **Kích thước và độ dài calibration (A.3):** nên dùng ít nhất 128 mẫu; chuỗi dài hơn (tới 4096) cho PPL tốt hơn; mặc định 1024 mẫu và độ dài 2048. Giảm cỡ calibration (128 mẫu) làm PPL xấu đi trên mọi mô hình (Table 5: LLaMA-2 7B 25% 7.56 so với 7.24).
- **Phổ trị riêng (A.4, Fig. 8):** phổ của OPT 6.7B suy giảm nhanh hơn LLaMA-2 7B (LLaMA-2 có phân phối trị riêng "dẹt" hơn, ít thành phần chính vượt trội), giải thích vì sao OPT nén tốt hơn. Phổ giảm nhanh hơn ở các layer đầu, gợi ý cắt nhiều hơn ở layer đầu.

- **Cắt khác nhau theo layer (Table 6):** đặt phần phương sai cần bỏ ở mỗi PCA để tổng khoảng 25%. OPT cải thiện nhẹ (ví dụ 6.7B: 12.10 xuống 11.94 với 24.7% tổng; 13B: 11.04 xuống 10.76), còn LLaMA-2 thì **xấu đi** (7B: 6.84 lên 7.63; 13B 6.00 lên 6.17; 70B 4.44 lên 4.63). Vì vậy bản chính dùng cắt đều.
- **Chọn dữ liệu RFT:** Alpaca tốt hơn WikiText-2 rõ rệt.

6. Giới hạn

Do tác giả nêu:

- Với mô hình $\leq$ 13B, **mô hình dense nhỏ hơn cùng số tham số tốt hơn** mô hình lớn bị cắt (tác giả nghĩ điều này sẽ không còn đúng lâu).
- Mô hình cắt có nhiều tham số hơn mô hình cắt bằng SparseGPT cùng độ nén nhưng cho phép batch lớn hơn và không có overhead cấu trúc thưa; kết hợp hai cách là hướng mở.
- Có thể có cách tính Q tốt hơn PCA.

Do ta nhận xét:

- PCA tối ưu cho tái tạo **tín hiệu** X (L2), không phải cho **loss**. Một thành phần chính nhỏ có thể vẫn quan trọng cho đầu ra; không có gradient trong tiêu chí.
- $Q_{\ell-1}^\top Q_\ell$ trên đường skip làm tăng số tham số và chi phí; "25% slicing" chưa tương đương 25% FLOPs (Table 12 cho 1.30 đến 1.33× nhân ma trận ở 25%, trong khi chi phí đầu cuối và overhead khác nhau).
- Kết quả tốt nhất cho OPT và mô hình lớn; với LLaMA-2 7B ở 30% PPL 8.12 so với 5.47: hơn 48% tăng, còn lớn.
- Cắt **chiều chung của residual stream**: cắt mọi layer cùng lúc; không cho kiểm soát cắt riêng MLP hay riêng attention.
- Không thử trên kiến trúc lai (SSM, linear-attention), mô hình thị giác hay đa phương thức.

7. Liên hệ với các bài khác

- **06 (ThiNet):** paper trích ThiNet là phương pháp gần nhất ("áp dụng phép tuyến tính giữa các layer"), nhưng ThiNet cần fine-tune và nhắm mô hình nhỏ hơn hàng bậc.
- **22, 23 (OBC, SparseGPT) và 27 (LLM Surgeon):** bù trọng số bằng nghịch đảo Hessian trong cùng ma trận; SliceGPT bù bằng cách chọn hệ trục. LLM Surgeon (concurrent) so sánh bằng cách nói SliceGPT bỏ qua gradient.
- **25 (ZipLM):** cùng cắt chiều trung gian của FFN và head nhưng giữ nguyên chiều residual; SliceGPT cắt đúng chiều residual mà ZipLM không đụng đến.
- **26 (LLM-Pruner):** nhóm "Channel" (chiều ẩn) của LLM-Pruner tệ; SliceGPT là cách làm đúng việc đó có bù.
- **28 (FLAP):** bù hằng số; SliceGPT bù tuyến tính đầy đủ qua Q.
- **18 (2:4):** SliceGPT dùng 2:4 SparseGPT làm baseline tốc độ.
- **31 đến 36 (ShortGPT, v.v.):** cắt theo chiều sâu (layer); SliceGPT cắt theo chiều rộng của residual. Đáng so khi tính ngân sách.
- **36 (ReplaceMe):** theo ta hiểu là thay layer bị bỏ bằng một ánh xạ tuyến tính; tinh thần "xấp xỉ tuyến tính tối ưu" tương tự (ta chưa đối chiếu bài 36 khi viết note này).

8. Gợi ý áp dụng

- PCA trên tín hiệu residual cho **ánh xạ tuyến tính hạng** D_{small} **tối ưu theo L2** (QD cực tiểu hoá $\|X - XQDD^\top Q^\top\|^2$). Đúng nghĩa đen đây là "ánh xạ tuyến tính tối ưu tại điểm cắt" cho điểm cắt là *chiều của residual stream*. Phần năng lượng thấp bị bỏ không có nghĩa là loss không đổi, nên cần kiểm chứng bằng loss/CER.

- Ma trận $Q_{\ell-1}^\top Q_\ell$ trên đường skip là một **ánh xạ tuyến tính thực sự được thêm vào mô hình** như phần bù.
- Cần kiểm tra thủ công: các norm bên trong block (nằm sau W_{in}) không cản trở; các norm có tham số ở residual phải hấp thụ được; ViT dùng LayerNorm cần đổi sang RMSNorm trước. Đây là suy luận, chưa được paper kiểm tra cho kiến trúc này.
- **Giới hạn phần cứng:** SliceGPT giảm D (nhỏ hơn cả bộ nhớ activation và KV/state), không đổi số layer; batch-1 latency phụ thuộc số lần launch kernel nhiều hơn số FLOPs, nên cần đo trước khi kỳ vọng (xem 25 về depth/width ở batch nhỏ).

9. Thí nghiệm gợi ý

Dốc nhanh nghĩa là có chỗ cắt chiều residual; dẹt (như LLaMA-2) nghĩa là khó. Đây là thí nghiệm rẻ nhất để biết SliceGPT có hợp không.

10. Tóm tắt công thức / khái niệm

Khái niệm	Nội dung
Computational invariance	RMSNorm$(XQ)Q^\top =$ RMSNorm(X) với Q trực giao
Biến đổi mạng	$\widetilde{W}_{embd} = W_{embd}Q$, $\widetilde{W}_{in} = Q^\top W_{in}$, $\widetilde{W}_{out} = W_{out}Q$, $\tilde{b}_{out} = Q^\top b_{out}$, $\widetilde{W}_{head} = Q^\top W_{head}$
Skip khi Q_ℓ khác nhau	nhân thêm $Q_{\ell-1}^\top Q_\ell$ ($D \times D$, không tính trước được)
PCA	$C_\ell = \sum_i X_{\ell,i}^\top X_{\ell,i}$, Q_ℓ = vector riêng sắp theo trị riêng giảm dần
Slicing	$\widetilde{X} = XQDD^\top Q^\top$; xoá hàng W_{in}, cột W_{out}, W_{embd}, hàng+cột của ma trận skip
LayerNorm sang RMSNorm	hấp thụ diag(α) vào W_{in}, $M = I - \frac{1}{D}\mathbf{1}\mathbf{1}^\top$ vào W_{out}
Cấu hình	calibration 1024 mẫu x 2048 token, PCA FP64, RFT LoRA r=32
Con số chính	LLaMA-2 70B 25%: PPL 4.60 (dense 3.32, SparseGPT 2:4 4.98); chi phí tính 66% (A100) / 64% (RTX6000); throughput 25% tới 1.55×

11. Kết luận ngắn

SliceGPT biến "xoá chiều residual" thành khả thi nhờ tính bất biến tính toán: xoay theo PCA rồi xoá các thành phần nhỏ, kèm một ánh xạ tuyến tính $D_{small} \times D_{small}$ ở mỗi đường skip. Kết quả tốt với mô hình lớn (70B 25% gần dense, nhanh hơn, ít GPU hơn), kém hơn với mô hình nhỏ.

Link gốc: https://arxiv.org/abs/2401.15024

Bài 30

MaskLLM: Learnable Semi-Structured Sparsity for Large Language Models (2024)

Tác giả: Gongfan Fang, Hongxu Yin, Saurav Muralidharan, Greg Heinrich, Jeff Pool, Jan Kautz, Pavlo Molchanov, Xinchao Wang (NVIDIA; National University of Singapore).
Venue / arXiv: arXiv:2409.17481 (26/09/2024, bản v2 12/2024). NeurIPS 2024 (ghi ở trang đầu bản text).
Nguồn: https://arxiv.org/abs/2409.17481
Chủ đề: Học mask thưa N:M (cụ thể 2:4) cho LLM bằng huấn luyện đầu cuối với Gumbel-Softmax, **giữ nguyên (đóng băng) trọng số**; mask prior từ phương pháp one-shot; chuyển giao mask sang miền/tác vụ mới.
Vị trí trong lộ trình: Đọc sau 18 (NVIDIA 2:4 sparsity), 23 (SparseGPT), 24 (Wanda) và 16/17 (Movement pruning, CoFi: học mask bằng gradient). Bài này là phiên bản "học bằng loss thật" thay cho "chấm điểm cục bộ" ở chuẩn N:M. Bài kế: 31 (ShortGPT) mở nhóm cắt theo chiều sâu.
Chương nền liên quan: nền 02: Ma trận thưa và chi phí thật

> Ghi chú học tập bằng tiếng Việt, không phải bản dịch. Số liệu lấy từ paper.

1. Bối cảnh

N:M sparsity (thưa bán cấu trúc): trong mỗi nhóm M trọng số liên tiếp, chỉ giữ N giá trị khác 0 (ví dụ 2:4: mỗi 4 trọng số giữ 2). GPU NVIDIA đời mới có sparse tensor core tăng tốc pattern này (bài 18). Khó khăn với LLM: số tổ hợp mask khổng lồ (paper nêu: LLaMA-2 7B toàn bộ 2:4 có 1.6 tỷ mask 2:4 phải chọn cho các lớp dày). Phương pháp hiện hành (SparseGPT, Wanda) dùng ít mẫu calibration và điểm quan trọng thủ công. Bài báo chỉ ra hai vấn đề:

1. **Calibration nhỏ không đại diện** cho kiến thức của LLM đã huấn luyện trên dữ liệu đa dạng; tăng calibration vượt 256 mẫu không cải thiện thêm (theo thí nghiệm của họ).
2. **Điểm quan trọng thủ công là proxy có sai số** so với mức thay đổi thực sự của hàm mất mát (độ lớn, gradient, Hessian đều xấp xỉ).

Khái niệm: **Gumbel-Softmax** là kỹ thuật lấy mẫu từ phân phối rời rạc (categorical) theo cách khả vi, bằng cách cộng nhiễu Gumbel vào logit và dùng softmax có nhiệt độ thay cho argmax; nhờ đó gradient chảy được về xác suất. **Frozen weights:** trọng số không được cập nhật, chỉ học mask.

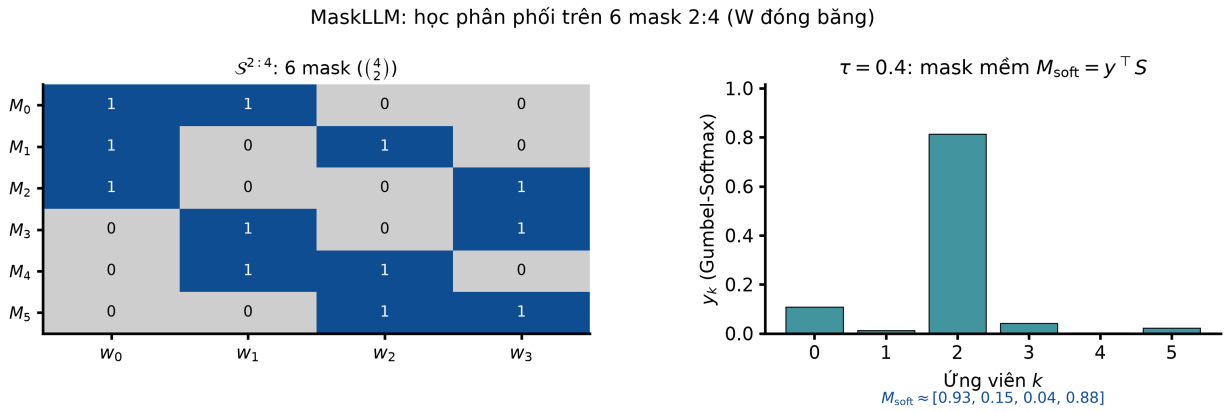
2. Ý tưởng chính

Với mẫu 2:4, mỗi nhóm bốn trọng số có $\binom{4}{2} = 6$ mặt nạ khả dĩ. Gán mỗi nhóm 6 logit học được (một phân phối categorical). Trong lúc huấn luyện, lấy mẫu một mask "mềm" bằng Gumbel-Softmax (trung bình có trọng số của 6 mask ứng viên), nhân vào trọng số đóng băng, tính loss ngôn ngữ trên dữ liệu lớn, backprop về các logit. Sau huấn luyện, mỗi nhóm chọn mask có logit lớn nhất. Có thể khởi tạo logit từ mask do Magnitude/SparseGPT/Wanda ("mask prior") để hội tụ nhanh hơn. Kết quả: LLaMA-2 7B 2:4 đạt PPL 6.72 (SparseGPT 10.42, dense 5.12) mà không sửa trọng số nào.

3. Phương pháp chi tiết

3.1 Bài toán chọn mask

Tập ứng viên cho 2:4 (6 mask):



Sáu ứng viên 2:4 và Gumbel-Softmax tạo mask mềm khi W đóng băng. (minh hoạ, tự tính)

$$\mathcal{S}^{2:4} = \{[1,1,0,0],[1,0,1,0],[1,0,0,1],[0,1,0,1],[0,1,1,0],[0,0,1,1]\}.$$

Mục tiêu (Eq. 3): chọn mask cho mọi khối W_i để cực tiểu loss ngôn ngữ:

$$\{\mathcal{M}_i^*\} = \arg\min_{\mathcal{M}_i \in \mathcal{S}^{2:4}} \mathbb{E}_{x\sim p(x)}\Big[\mathcal{L}_{LM}\big(x; \{W_i \odot \mathcal{M}_i\}\big)\Big].$$

Nhìn theo xác suất (Eq. 4): học phân phối $p(\mathcal{M}_i)$ sao cho **kỳ vọng loss khi lấy mẫu mask** là nhỏ nhất.

3.2 Lấy mẫu khả vi

Gumbel-Max: $y = \text{onehot}(\arg\max_i[\log p_i + g_i])$, $g_i = -\log(-\log \epsilon_i)$, $\epsilon_i \sim U(0,1)$. Thay argmax bằng softmax nhiệt độ τ để khả vi (Eq. 6):

$$\tilde{y}_i = \frac{\exp((\log p_i + g_i)/\tau)}{\sum_j \exp((\log p_j + g_j)/\tau)}.$$

Mask mềm là trung bình có trọng số của các ứng viên (Eq. 7): $\widetilde{\mathcal{M}} = \sum_{i=1}^{|\mathcal{S}|} \tilde{y}_i \hat{\mathcal{M}}_i$. Khi $\tau \to 0$, $\tilde{y}$ tiến về one-hot.

Tham số hoá: học logit π_i với **hệ số tỉ lệ** κ: $p_i = \exp(\pi_i\kappa)/\sum_j \exp(\pi_j\kappa)$. κ cân bằng độ lớn logit so với nhiễu Gumbel, tức điều khiển mức ngẫu nhiên khi lấy mẫu.

3.3 Chính quy hoá trọng số còn lại

Khi mask loại bỏ trọng số, gradient truyền qua layer có thể nhỏ đi (trọng số còn lại có độ lớn nhỏ), cản trở học mask. Thêm số hạng thưởng độ lớn (Eq. 8):

$$\min_{\{p_\pi(\mathcal{M}_i)\}} \mathbb{E}_{x,\widetilde{\mathcal{M}}_i\sim p_\pi}\Big[\mathcal{L}_{LM}\big(x; \{W_i \odot \widetilde{\mathcal{M}}_i\}\big) - \lambda \sum_i \|W_i \odot \widetilde{\mathcal{M}}_i\|_2^2\Big]$$

3.4 Mask prior (chuyển giao sparsity)

Cho mask có sẵn $\mathcal{M}_0$ (từ magnitude, SparseGPT, Wanda hoặc từ quá trình học khác). Độ tương tự với từng ứng viên (Eq. 9, 10):

$$\text{sim}(\mathcal{M}_0, \hat{\mathcal{M}}_i) = \mathcal{M}_0\hat{\mathcal{M}}_i^\top - N/2, \qquad \pi_i' = \pi_i + \sigma(\pi)\cdot \text{sim}(\mathcal{M}_0, \hat{\mathcal{M}}_i)\cdot\alpha$$

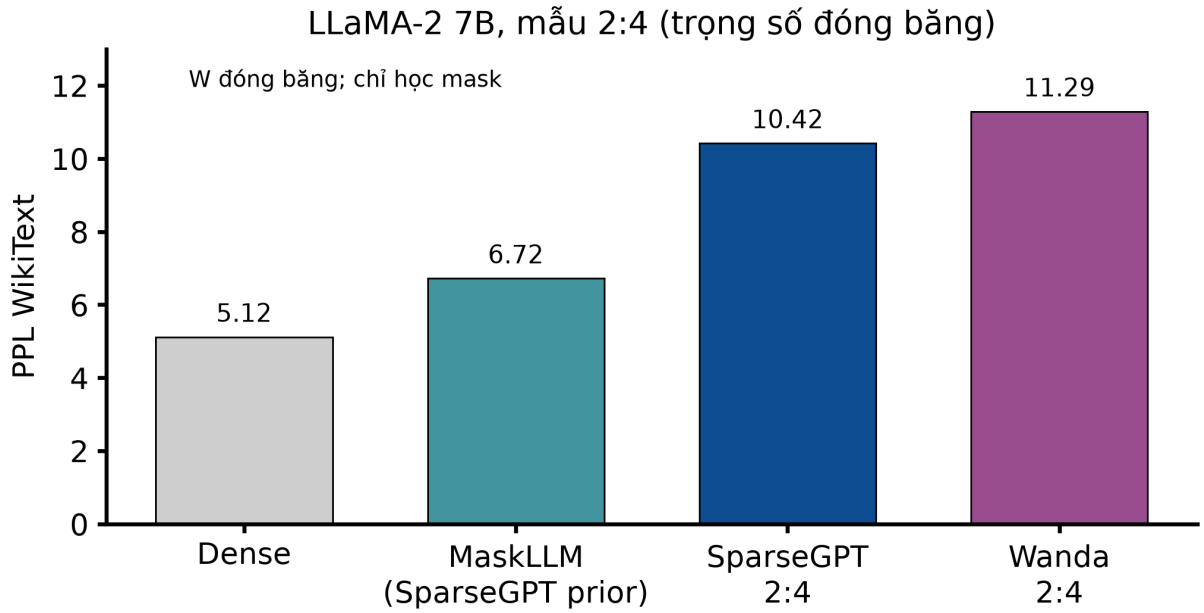
($\sigma(\pi)$ là độ lệch chuẩn của logit; α là độ mạnh của prior; $\alpha = 0$ là học thuần không prior). Mask ứng viên giống mask prior được tăng xác suất ban đầu.

```
MaskLLM(W frozen, D, prior_mask M0 optional, steps=2000):
    S = 6 candidate 2:4 masks (matrix 6 x 4)
    for each group of 4 consecutive weights: logits pi ~ N(0, 0.01)   # 6 logits per
    ↪  group
    if M0: pi += std(pi) * (M0 @ S^T - 2/2) * alpha                    # alpha=3
    ↪  (SparseGPT prior)
    for step in 1..steps:
        x ~ data
        for each group: g ~ Gumbel; y = softmax((pi*kappa + g)/tau)
        M_soft = y @ S                                  # (4,)  mask mềm
        loss = L_LM(x; W * M_soft) - lambda * ||W * M_soft||^2
        update pi (AdamW)          # kappa: 1e2 -> 5e2 ;  tau: 4 -> 0.05
    final mask of each group = S[argmax pi]
```

Với miền/tác vụ mới: khởi tạo từ mask chung đã học (làm prior) rồi học tiếp vài trăm bước trên dữ liệu miền đó (**transfer learning of sparsity**).

4. Thí nghiệm và kết quả chính

Mô hình: LLaMA-2 7B, 13B, Nemotron-4 15B, và hai mô hình nội bộ GPT-3 843M, GPT-3 2B (huấn luyện bằng Megatron); LLaMA-3 8B ở phụ lục. **Cài đặt:** học logit **2000 bước**, batch toàn cục 256 (tức 512k mẫu), trọng số đóng băng; AdamW lr 5e-4 (GPT-3: 1e-3), wd 0.1; khởi tạo logit $\mathcal{N}(0, 0.01)$; κ tăng 1e2 đến 5e2; τ 4 xuống 0.05; prior SparseGPT với $\alpha = 3$; hệ số chính quy 1e-5. Dữ liệu LLaMA-2: tập pha trộn 69 miền (CUDA, VHDL, Reddit, ...). **Đánh giá:** PPL WikiText-2, 7 tác vụ zero-shot (HellaSwag, RACE, PIQA, WinoGrande, ARC-E, ARC-C, OBQA). Baseline one-shot calibrate bằng C4 (SparseGPT có cập nhật trọng số).



LLaMA-2 7B 2:4: MaskLLM PPL 6.72 vs SparseGPT 10.42 / Wanda 11.29 (dense 5.12). (minh hoạ, tự tính)

2:4 với trọng số đóng băng (Table 1, trích):

Mô hình	Phương pháp	PPL Wikitext	Zero-shot trung bình
LLaMA-2 7B	Dense	5.12	57.16
	Magnitude	54.71	46.19
	SparseGPT	10.42	47.16
	Wanda	11.29	45.98
	MaskLLM	**6.72**	**52.09**
LLaMA-2 13B	Dense	4.57	59.60
	SparseGPT	8.20	52.32
	Wanda	8.47	50.61
	MaskLLM	**5.85**	**56.74**
Nemotron-4 15B	Dense	5.78	61.47
	SparseGPT	13.38	51.13
	Wanda	25.05	45.82
	MaskLLM	**7.31**	**56.74**
GPT3 2B	Dense	9.35	48.90
	SparseGPT	22.14	39.54
	MaskLLM	**11.42**	**44.82**
GPT3 843M	Dense	12.42	41.92
	SparseGPT	38.78	35.22
	MaskLLM	**15.39**	**39.10**

LLaMA-3 8B (Table 12): dense 5.76; SparseGPT 17.64; Wanda 23.40; Magnitude 2.61E+03; MaskLLM **8.50** (prior SparseGPT, học trên C4). **LLaMA-2 13B so với các phương pháp khác (Table 13):** SparseGPT 8.32, Wanda 8.27, ADMM-Iter 7.78, GBLM 8.80, RIA 8.41, Pruner-Zero 7.41, MaskLLM 5.85 (số lấy từ bài gốc của từng phương pháp).

Mask riêng cho miền (Table 4, GPT3-2B, PPL trung bình 8 miền): dense 7.42; SparseGPT 18.80; SparseGPT-Update 15.36; Wanda 23.36; **MaskLLM 7.39** (ngang dense, "lossless"). LLaMA2-7B 8 miền: dense 4.80; SparseGPT 6.64; SparseGPT-Update 6.32; Wanda 8.04; MaskLLM 4.90.

Chuyển giao (Table 5, GPT-3 2B): mask chung 10.61, mask học từ đầu cho miền 7.51, **mask chuyển giao 7.39** (dense 7.42).

Lưu trữ và tốc độ (Table 6, LLaMA-2 7B, A6000, batch 1): fine-tune cả mô hình cho mỗi tác vụ cần 16 bit/tham số; mask học được cần **0.65 bit/tham số** ($\log_2 6/4 \approx 0.646$, mã hoá số học; giảm 25 lần), bộ nhớ mô hình 73%, nhanh **1.4×**. Phụ lục (Table 16, TensorRT-LLM): LLaMA-2 7B tốc độ token/s/GPU dense so với 2:4: 61.74 so với 86.92 (1.41×), 59.18 so với 82.11 (1.39×), 57.55 so với 78.81 (1.37×), 55.40 so với 75.10 (1.36×); LLaMA-2 13B 1.50× đến 1.57×.

Chi phí huấn luyện: 64 GPU A100 với tensor parallel 8; LLaMA-2 7B 1,280 GPU-giờ; LLaMA-2 13B 2,304 GPU-giờ; Nemotron-4 2,304 GPU-giờ.

5. Phân tích / ablation

- **Mở rộng theo lượng dữ liệu (Fig. 4, LLaMA-2 7B):** SparseGPT bão hoà sau 256 mẫu calibration; MaskLLM cải thiện liên tục tới 512k mẫu. MaskLLM cần 128 mẫu để dựng prior, vượt SparseGPT sau khoảng 1280 mẫu (5 bước cập nhật với batch 256). Nếu chỉ 1 đến 2 bước thì không bằng one-shot.
- **Mask prior (Table 2, PPL Wikitext, LLaMA-2 7B):** prior magnitude 54.71 sang mask học 6.77; SparseGPT 10.46 sang **6.72**; Wanda 11.29 sang 6.80; không prior: **9.12**. Mô hình nhỏ hơn: GPT-3

843M không prior 18.62 so với SparseGPT-prior 15.39; GPT-3 2B 14.31 so với 11.59. Tức học thuần vẫn thắng baseline, nhưng prior rẻ giúp nhiều khi chỉ có 2000 bước.

- **Tính ngẫu nhiên khi lấy mẫu (Fig. 5, Table 9 trên GPT-3 843M):** κ quá nhỏ (nhiễu lấn át) hội tụ chậm (1 đến 5: PPL 5.97E+06); κ quá lớn (1e5 đến 5e5) không khám phá (PPL 24.81); chọn 1e2 đến 5e2 (15.39). Nhiệt độ τ ban đầu 1: 17.52; 2: 16.69; **4: 15.39**; 10: 15.68. Độ mạnh prior α: 0: 18.62; 1: 23.65; 2: 15.59; **3: 15.39**; 5: 15.48.
- **Chính quy hoá độ lớn (Table 3, GPT-3 2B):** mask-only 11.59 sang 11.42 (có reg); miền con 7.61 sang 7.39; fine-tune sau cắt 10.21 sang 9.96. Phụ lục F: reg làm gradient lớn hơn; norm trọng số mô hình thưa tăng 1.36 đến 1.41 lần so với dense; mask học được có thể chọn trọng số nhỏ hơn magnitude (chuẩn L1 thấp hơn khoảng 10%).
- **Dữ liệu học mask:** học trên C4 (Table 11) chỉ kém dữ liệu pha trộn một chút: LLaMA-2 7B 6.79 so với 6.72 (SparseGPT 10.42 so với 9.88).
- **Giữ layer nhạy cảm dày (Table 15, GPT-3 2B, PPL Story/OpenWeb):** full sparsity 15.58/13.84; bỏ qua layer cuối 15.18/13.61; bỏ 4 layer cuối 15.07/13.24; 8 layer cuối 14.95/12.92 (dense 14.76/12.05).
- **ViT-B/16 (phụ lục J, ImageNet-1K top-1):** dense 79.15; Magnitude 65.92; Wanda 63.28; SparseGPT 71.52; MaskLLM 1 epoch 76.23; **20 epoch 79.46** (+0.31 so với dense, không cập nhật trọng số). Tác giả coi là dấu hiệu của hiện tượng Lottery Ticket (bài 08) trong ViT.

6. Giới hạn

Do tác giả nêu: huấn luyện mask tốn tài nguyên hơn nhiều phương pháp one-shot (có thể tạo mask hiệu quả); cải thiện hiệu quả huấn luyện là việc cần làm (phụ lục K).

Do ta nhận xét:

- Chi phí là hàng nghìn GPU-giờ A100 (1,280 cho 7B), trong khi SparseGPT/Wanda chỉ vài phút; số "6.72" cần so với chi phí này. Bộ nhớ: 6 logit cho mỗi 4 trọng số (1.5 logit mỗi trọng số) cộng trạng thái AdamW, nên nặng hơn nhiều so với tham số đóng băng (ước tính, không phải số của paper).
- Chỉ N:M (2:4); tăng tốc phụ thuộc phần cứng có sparse tensor core và cài đặt (1.36 đến 1.57× trên A6000 ở batch 1, không phải mức giảm 2×); không giảm số tham số một cách thực chất.
- Khác biệt huấn luyện/suy luận: khi huấn luyện dùng mask mềm (trung bình có trọng số), khi suy luận dùng mask cứng argmax; lệch này được kiểm soát bằng $\tau \to 0.05$ và κ tăng nhưng không có thí nghiệm riêng đo độ lệch (nhận xét thêm).
- So sánh với SparseGPT/Wanda dùng dữ liệu rất khác nhau (512k mẫu pha trộn miền so với 128 đến 256 mẫu); tác giả có thêm thí nghiệm học 1280 mẫu và C4 để giảm sự khác biệt này.
- Đánh giá bằng PPL và zero-shot; không đánh giá tác vụ sinh dài hay tác vụ nghiệp vụ như OCR.

7. Liên hệ với các bài khác

- **18 (2:4 sparsity):** nền tảng phần cứng/định dạng; MaskLLM học mask 2:4 tốt hơn so với chọn theo độ lớn rồi fine-tune.
- **23, 24 (SparseGPT, Wanda):** baseline; cũng là nguồn prior. MaskLLM chứng minh dùng chúng làm khởi tạo vẫn có ích.
- **16 (Movement pruning), 17 (CoFi):** cũng học mask bằng gradient với loss thật; CoFi học mask cho cấu trúc (head, FFN, layer) bằng hard-concrete; MaskLLM học mask N:M khả vi bằng Gumbel-Softmax và đóng băng trọng số.
- **08, 09 (Lottery Ticket, linear mode connectivity):** kết quả ViT "lossless chỉ bằng mask" liên hệ với giả thuyết mạng con.
- **20 (DepGraph), 26 (LLM-Pruner):** cùng nhóm Fang/Wang; MaskLLM là hướng semi-structured học được thay cho structured.

- **39 (Minitron), 44 (Nemotron-H):** NVIDIA; Nemotron-4 15B được dùng như một mô hình thí nghiệm ở đây.
- **43 (Pruning vs scratch):** hỏi lại "nén dựa trên mô hình có sẵn so với huấn luyện lại", bài này cho thấy có thể "chỉ học mask" mà không đổi trọng số.

8. Gợi ý áp dụng

- Có thể là bước *cộng thêm* sau khi đã cắt cấu trúc nếu GPU hỗ trợ.
- **Ý tưởng chuyển được (gián tiếp):**
  1. **Chọn bằng loss thật thay vì proxy cục bộ.** Kết quả 6.72 so với 10.42 (SparseGPT có cập nhật trọng số) cho thấy chọn mask bằng loss đầu cuối thắng cả phương pháp chấm điểm có bù cục bộ trong thiết lập N:M. Chú ý chi phí và khác biệt dữ liệu như ở mục 6. Bảng 2 cho thấy prior làm cải thiện lớn khi ngân sách bước ít (9.12 xuống 6.72).
  2. **Chính quy hoá giữ độ lớn của phần còn lại** để gradient không tắt: nếu học cổng với trọng số đóng băng thì cần kiểm tra điều này.
  3. **Mask riêng cho miền** (0.65 bit/tham số): một bản trọng số chung cộng nhiều mask cho từng loại tài liệu (ví dụ chữ in, chữ viết tay, ngôn ngữ). Với cấu trúc, tương tự là mỗi miền chọn một tập neuron khác nhau của cùng mô hình (một bản trọng số), nhưng đây chỉ là ý tưởng.
- Cần thận trọng khi suy ra cho cắt cấu trúc, nơi chọn lựa ít linh hoạt hơn (xoá cả hàng/cột) và bù có thể đóng vai trò lớn hơn (xem 27, 29).

9. Thí nghiệm gợi ý

2. **Ngân sách bước:** lặp lại thí nghiệm 1 với 0 (chỉ prior), 50, 500, 2000 bước để xem đường cong "prior so với học" tương tự Fig. 4.
3. **Mask theo miền:** học cổng riêng cho 2 miền tài liệu (ví dụ bảng biểu và văn bản thường) từ cùng prior chung; đo mức trùng nhau giữa các cổng (như Fig. 8 trong phụ lục về khác biệt mask).

10. Tóm tắt công thức / khái niệm

Khái niệm	Nội dung
Ứng viên 2:4	$\lvert\mathcal{S}\rvert = \binom{4}{2} = 6$ mask mỗi nhóm 4 trọng số
Mục tiêu	$\min_{\mathcal{M}} \mathbb{E}_x \mathcal{L}_{LM}(x; W \odot \mathcal{M})$ với W đóng băng
Gumbel-Softmax	$\tilde{y}_i = \dfrac{\exp((\log p_i + g_i)/\tau)}{\sum_j \exp((\log p_j + g_j)/\tau)}$
Mask mềm	$\widetilde{\mathcal{M}} = \sum_i \tilde{y}_i \hat{\mathcal{M}}_i$
Tham số hoá	$p_i = \text{softmax}(\pi\kappa)$, κ điều khiển mức ngẫu nhiên
Chính quy hoá	trừ $\lambda \sum \lVert W \odot \widetilde{\mathcal{M}} \rVert_2^2$ ($\lambda = 10^{-5}$)
Mask prior	$\pi'_i = \pi_i + \sigma(\pi) \cdot (\mathcal{M}_0 \hat{\mathcal{M}}_i^\top - N/2) \cdot \alpha$
Cấu hình	2000 bước, batch 256, AdamW 5e-4, κ 1e2 đến 5e2, τ 4 đến 0.05, $\alpha = 3$
Con số chính	LLaMA-2 7B 2:4: PPL 6.72 (SparseGPT 10.42, Wanda 11.29, dense 5.12); 1.4×, 73% bộ nhớ, 0.65 bit/tham số cho mask miền; 1,280 GPU-giờ

11. Kết luận ngắn

MaskLLM biến việc chọn mask 2:4 thành bài toán học phân phối bằng Gumbel-Softmax với loss ngôn ngữ thật trên dữ liệu lớn, trọng số đóng băng, có prior từ phương pháp one-shot và chuyển giao theo miền. Nó vượt rõ SparseGPT/Wanda với chi phí hàng nghìn GPU-giờ.

Link gốc: https://arxiv.org/abs/2409.17481

Bài 31

ShortGPT: Layers in Large Language Models are More Redundant Than You Expect (2024)

Tác giả: Xin Men, Mingyu Xu, Qingyu Zhang, Bingning Wang, Hongyu Lin, Yaojie Lu, Xianpei Han, Weipeng Chen (Baichuan Inc., ISCAS).
Venue / arXiv: arXiv:2403.03853 (v3, 11/10/2024). Bản text không ghi tên hội nghị.
Nguồn: https://arxiv.org/abs/2403.03853
Chủ đề: Depth pruning (cắt cả layer) cho LLM, chỉ số Block Influence (BI), không cần gradient, không re-train.
Vị trí trong lộ trình: Đọc trước: 26 LLM-Pruner, 29 SliceGPT (hai đối thủ "width" mà bài này đánh bại). Bài này mở đầu nhóm depth pruning (31–37): đưa ra heuristic đơn giản nhất ("layer nào có đầu ra giống đầu vào thì bỏ"). Đọc tiếp: 32 (cùng ý tưởng, thêm healing bằng QLoRA), 34 SLEB, 35 LaCo, 36 ReplaceMe (thay layer bị bỏ bằng ánh xạ tuyến tính).
Chương nền liên quan: nền 06: Cắt lớp và độ sâu trong LLM

> Tài liệu học tập tiếng Việt, ghi chú diễn giải, không phải bản dịch paper.

1. Bối cảnh

- **Structured pruning** (cắt có cấu trúc) = bỏ nguyên cả cụm tham số (kênh, head, layer) để mô hình thật sự nhỏ/nhanh trên GPU phổ thông, khác với unstructured (đặt từng trọng số = 0, khó tăng tốc).
- Trước bài này, các phương pháp cho LLM (LLM-Pruner dùng gradient để chọn nhóm coupled; SliceGPT dùng PCA để giảm chiều embedding) cắt theo chiều **rộng** (width). Chúng khá phức tạp và mất nhiều chất lượng ở tỉ lệ ~25%.
- **Pre-norm**: LayerNorm/RMSNorm đặt *trước* attention/FFN, đầu ra là $x_{l+1} = x_l + f_l(\text{norm}(x_l))$ (có đường cộng residual). Hầu hết LLM hiện đại (Llama, Mamba, RWKV) dùng pre-norm.
- Câu hỏi của bài báo: nếu mỗi lớp chỉ "cộng thêm" một lượng nhỏ vào residual, có phải nhiều layer gần như không làm gì và bỏ được?

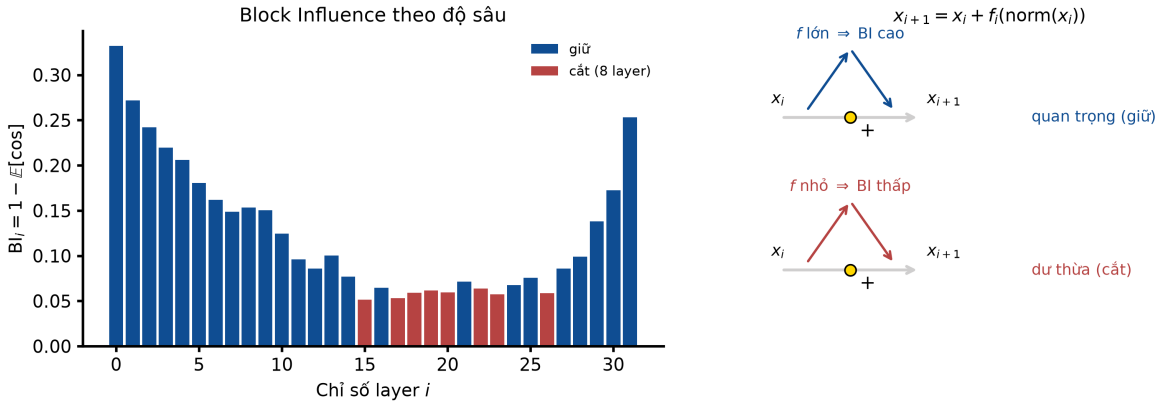
2. Ý tưởng chính

Bài báo ghi nhận (Figure 1) rằng bỏ **riêng một** layer ở giữa/sau của Llama2-7B hoặc Baichuan2-7B gần như không làm hỏng perplexity hay MMLU; lớp đầu và lớp cuối quan trọng hơn. Họ đo "layer này làm thay đổi hidden state bao nhiêu" bằng **Block Influence (BI)** = 1 − cosine giữa đầu vào và đầu ra của lớp. BI thấp nghĩa là layer gần như là hàm đồng nhất, nên bỏ. Chỉ cần tính BI trên một tập calibration nhỏ không nhãn, sắp xếp, xoá các lớp BI thấp nhất. Không gradient, không fine-tune. Kết quả: bỏ ~25% tham số mà giữ ~90% hiệu năng trung bình (trên Llama2-13B), tốt hơn LLM-Pruner/SliceGPT/LaCo.

3. Phương pháp chi tiết

3.1 Block Influence

$$\boxed{\text{BI}_i = 1 - \mathbb{E}_{X,t}\,\frac{X_{i,t}^{\top} X_{i+1,t}}{\|X_{i,t}\|_2\,\|X_{i+1,t}\|_2}}$$



Block Influence theo độ sâu: layer BI thấp (gần đồng nhất trên residual) được ưu tiên cắt. (minh hoạ, tự tính)

$X_{i,t}$ là hidden state tại token t vào layer i. Lấy kỳ vọng trên các token và các mẫu calibration. Cosine = 1 nghĩa là layer không đổi *hướng* của vector (BI = 0).

Một layer có thể cộng một vector cùng hướng nhưng rất to vẫn có BI thấp. Paper không bàn điều này.

3.2 Vì sao pre-norm làm layer sâu giống hàm đồng nhất (Phụ lục A, lập luận cho init ngẫu nhiên)

Theo Lemma 1 (Xiong et al. 2020) chuẩn của hidden state tăng theo độ sâu, $\|x_L\| = \Theta(\sqrt{L})$, trong khi $\|f_L(x_L)\| = O(1)$. Do đó

$$\cos(x_{L+1}, x_L) \geq \frac{\|x_L\|}{\|x_{L+1}\|} - \frac{\|f_L\|}{\|x_{L+1}\|} = \Theta\left(\sqrt{\frac{L}{L+1}}\right) - O\left(\sqrt{\frac{1}{L+1}}\right) \to 1.$$

Paper nói rõ đây chỉ là phân tích cho mô hình khởi tạo ngẫu nhiên; họ minh hoạ bằng thực nghiệm (Figure 2) rằng mô hình pre-norm tự huấn luyện (200B token) có cosine cao, còn post-norm thì phân kỳ sau ~26B token.

3.3 Thuật toán

```
Input: model M (L layers), calibration set D (vd. PG19), số layer cần xoá n
1. for x in D: chạy M, lưu hidden state vào/ra từng layer i
2. BI_i = 1 - mean cosine(X_i, X_{i+1})    # trên mọi token, mọi mẫu
3. sắp xếp layer tăng dần theo BI
4. xoá n layer có BI nhỏ nhất (không nhất thiết liền kề)
5. trả về model đã rút gọn (không retrain)
```

Ví dụ nhỏ: 32 layer, muốn bỏ 9. Với Llama-2-7B, các layer bị bỏ ở Table 9 là 27, 26, 25, 28, 24, 29, 23, 21, 22 (thứ tự theo BI tăng dần), tức gần như một khối liền ở phần sau-giữa của mạng.

4. Thí nghiệm và kết quả chính

Mô hình: Llama2-7B, Llama2-13B, Baichuan2-7B, Baichuan2-13B (+ RWKV-7B, Mamba-2.8B, Llama2-7B-GPTQ). **Calibration/PPL:** PG19. **Benchmark** theo LaCo: CMNLI, HellaSwag, PIQA, CHID, WSC, CommonSenseQA, BoolQ, Race-H/M, XSum, C3, MMLU, CMMLU. Cột “Per.” = % hiệu năng so với dense. Không dùng post-training cho bất kỳ baseline nào.

Bảng 2 (trích, giá trị Ave. và Per.):

Model	Method	Ratio	Ave.	Per.
Llama2-7B	Dense	0%	47.78	100.00
	LLM-Pruner	27.0%	34.78	72.79
	SliceGPT	26.4%	32.84	68.73
	LaCo	27.1%	38.41	80.39
	ShortGPT	27.1%	41.24	86.31
Llama2-13B	Dense	0%	55.14	100.00
	LaCo	24.6%	47.62	86.36
	ShortGPT	24.6%	50.53	91.64
Baichuan2-7B	ShortGPT	24.2%	49.08	85.10
Baichuan2-13B	ShortGPT	24.7%	54.43	87.35

Ví dụ nổi bật: bỏ 10/40 layer của Llama2-13B, MMLU từ 55.0 xuống 52.2 trong phần Introduction (bảng 2 ghi 54.69 cho cấu hình ShortGPT 24.6% ở cột MMLU; hai con số khác cấu hình/phiên bản chạy, paper không giải thích rõ).

Các số khác:

- Llama-7B vs LLM-Pruner (Table 7, ShortGPT ratio 21.9%): ShortGPT avg 58.12 so với LLM-Pruner 56.82 ở ratio 20% (baseline 63.25).
- Vs SliceGPT trên Llama-2-70B (Table 8), 25%: SliceGPT avg 69.75, ShortGPT 69.79; ở 30%: 66.11 vs 69.4. Baseline 76.57.
- **Non-Transformer (Table 3):** Mamba-2.8B bỏ 25% giữ Per. 90.42; RWKV-7B bỏ 25% chỉ còn 73.16 (ít dư thừa hơn).
- **Quantization (Table 4, Llama2-7B-GPTQ):** bỏ 25.0% (24 layer) throughput 5045.59 token/s (1.16x), MMLU 41.68 so với 43.17; bỏ 27.1% (23 layer) 5146.99 token/s (1.19x), PPL 40.78. Table 5: thứ tự "prune rồi quantize" MMLU 41.2, "quantize rồi prune" 42.4.
- **Post-training (Table 6, Llama2-7B):** thay layer bị xoá bằng MLP gated nhẹ (hidden 2048), train 50B token: Ave. từ 41.22 (ratio 27.1%) lên 43.16 (ratio 24.0%, vì MLP thêm tham số). XSum 0.67 -> 4.89, CHID 24.68 -> 31.82.

5. Phân tích / ablation

- **Metric (Fig. 5, Table 10):** so BI với Sequential (bỏ từ đầu), Reverse-order (bỏ từ cuối), Relative Magnitude ($\|f(x)\|/\|x + f(x)\|$). BI tốt nhất; Relative Magnitude cạnh tranh; chuẩn của hidden state theo layer cho MMLU tốt nhưng perplexity kém. Các số cụ thể chỉ có trong hình, không có trong text.
- **BI tương quan với PPL khi xoá từng layer** (Fig. 3) – tương quan dương.
- **Last layer đặc biệt (Table 1):** Llama2-7B PPL 7.60 (dense); bỏ cả layer cuối 13.37; bỏ chỉ attention của layer cuối 7.65; bỏ chỉ FFN của layer cuối 12.35. Giả thuyết của tác giả: FFN cuối gần như là một phần của classifier cùng LM head.
- **MMLU rơi đột ngột** ở một số tỉ lệ cụ thể (Fig. 5): giả thuyết có "layer then chốt". Paper không điều tra thêm.
- Paper cho rằng cắt thô (layer) tốt hơn cắt mịn (kênh/neuron) vì khi bỏ riêng từng layer sâu mô hình thay đổi rất ít, nên khó định nghĩa độ quan trọng của đơn vị nhỏ hơn. Đây là **giả thuyết**, chưa được kiểm chứng riêng.

6. Giới hạn

Do tác giả nêu:

- Layer removal làm hỏng **tác vụ sinh** (XSum, C3) nhiều hơn tác vụ trắc nghiệm: ở 25% trên model 7B, điểm sinh gần về 0 (XSum 0.67 với Llama2-7B, 0.04 với Baichuan2-7B). Giả thuyết: lỗi tích luỹ khi sinh; mô hình lớn (13B) bền hơn. Chưa giải thích được.
- RWKV ít dư thừa hơn Mamba/Transformer, chưa lý giải.

Nhận xét:

- Mỗi layer được chấm **độc lập**; không xét tương tác khi bỏ nhiều layer cùng lúc (37 sẽ chỉ ra điểm này quan trọng).
- Chỉ đo benchmark ngắn/trắc nghiệm; không đo độ trễ batch-1 thực, không có latency ngoài bảng throughput GPTQ.
- Calibration chỉ PG19 (văn xuôi sách); không thử miền khác (OCR, tài liệu).
- Bảng 2 và đoạn Introduction dùng hai con số MMLU khác nhau cho cùng ý (55.0 -> 52.2 và 54.69), cần cẩn thận khi trích.

7. Liên hệ với các bài khác

- **26 LLM-Pruner, 29 SliceGPT:** đối thủ width; ShortGPT cho thấy ở ~25% depth tốt hơn width (trong benchmark của paper).
- **35 LaCo:** cũng cắt layer, nhưng *gộp* layer sau vào layer trước thay vì xoá; ShortGPT hơn LaCo ở bảng 2.
- **32 Deeper layers:** cùng kết luận, nhưng chọn *khối liền* bằng độ tương tự góc và có healing QLoRA.
- **34 SLEB:** chọn lặp từng layer bằng perplexity thực (thay vì BI tĩnh).
- **36 ReplaceMe:** phần 4.6 (thay bằng MLP nhẹ + train) là tiền thân của ý "thay khối bị cắt bằng ánh xạ".
- **37:** chứng minh rằng chấm điểm từng layer không có repair có thể bỏ lỡ tối ưu khi dùng repair tuyến tính.

8. Gợi ý áp dụng

- **Nhưng:** decoder lai (linear-attention + full-attention) không đảm bảo giả định "layer sâu gần đồng nhất". Cần kiểm chứng; Mamba trong paper cho thấy ổn, nhưng đó là SSM thuần.
- Bài 36/37 mới xét map tuyến tính tổng quát.
- Điều cảnh báo: kết quả generative tụt mạnh.

9. Thí nghiệm gợi ý

1. Tính Block Influence cho từng block và so với mức tăng loss khi bỏ riêng từng block.
2. So sánh Block Influence, SLEB và cách chọn khối liền dưới cùng ngân sách cắt; đánh giá cả tác vụ sinh và độ trễ.

10. Tóm tắt công thức / khái niệm

Khái niệm	Ý nghĩa
BI_i	$1 - \text{cosine}$(đầu vào, đầu ra layer i) trung bình trên token
Pre-norm	Norm trước sub-layer, giữ đường residual
Layer removal	Xoá layer BI thấp nhất, không retrain
Relative Magnitude	$\|f(x)\|/\|x + f(x)\|$, metric đối chứng
Per.	% điểm trung bình so với dense

11. Kết luận ngắn

ShortGPT là baseline depth-pruning đơn giản nhất: một forward, một cosine, không huấn luyện. Mạnh ở benchmark trắc nghiệm ở ~25%, yếu ở tác vụ sinh, và không mô hình hoá tương tác giữa các lớp bị bỏ cũng như repair.

Link gốc: https://arxiv.org/abs/2403.03853

Bài 32

The Unreasonable Ineffectiveness of the Deeper Layers (2024)

Tác giả: Andrey Gromov, Kushal Tirumala, Hassan Shapourian, Paolo Glorioso, Daniel A. Roberts (Meta FAIR, UMD, Cisco, Zyphra, MIT, Sequoia Capital).
Venue / arXiv: arXiv:2403.17887 (v2, 03/2025); "Published as a conference paper at ICLR 2025".
Nguồn: https://arxiv.org/abs/2403.17887
Chủ đề: Cắt một **khối liền** các layer của LLM dựa trên khoảng cách góc giữa biểu diễn, rồi "heal" (chữa) bằng QLoRA ít token.
Vị trí trong lộ trình: Đọc trước: 31 ShortGPT (cùng ý tưởng, đo từng layer). Bài này thêm: (a) chọn *khối n layer liền kề* thay vì n layer rời; (b) bước healing ngắn; (c) phát hiện "QA benchmark bền nhưng loss next-token xuống dốc đều". Đọc tiếp: 33 Shortened LLaMA (so sánh cách retrain), 34 SLEB, 36 ReplaceMe, 37 (repair đổi layout tối ưu).
Chương nền liên quan: nền 06: Cắt lớp và độ sâu trong LLM

> Tài liệu học tập tiếng Việt, ghi chú diễn giải, không phải bản dịch paper. Hầu hết kết quả của paper nằm trong **hình** (không có bảng số trong text), nên phần số liệu dưới đây chủ yếu là ngưỡng/xu hướng mà tác giả viết ra bằng chữ.

1. Bối cảnh

- Pruning LLM sau pretraining thường cắt theo chiều rộng (head, neuron, kênh). Bài này tự hỏi một câu hỏi "khoa học" hơn: **tri thức được lưu ở đâu trong trọng số?** Nếu bỏ một lớp mà benchmark hỏi-đáp không đổi thì layer đó không cần để lưu tri thức cho các câu hỏi ấy.
- **Healing**: sau khi cắt, có "mismatch" ở chỗ nối (layer sau nhận đầu vào khác kỳ vọng). Fine-tune nhẹ để vá chỗ nối này gọi là heal.
- **QLoRA**: lượng tử hoá mô hình xuống 4 bit và chỉ huấn luyện các adapter hạng thấp (LoRA, cộng thêm ma trận BA nhỏ vào trọng số). Nhờ đó mỗi thí nghiệm chạy trên một GPU A100 40GB.
- **Residual stream**: $x^{(\ell+1)} = x^{(\ell)} + f(x^{(\ell)}, \theta^{(\ell)})$; đầu ra cuối là tổng của embedding và đóng góp mọi layer.

2. Ý tưởng chính

Khi biểu diễn thay đổi chậm theo độ sâu ($x^{(\ell)} \approx x^{(\ell-1)} + \epsilon$) việc xoá một khối lớp liên tiếp có thể chỉ gây thay đổi nhỏ. Do đó đo **khoảng cách góc** giữa đầu vào layer ℓ và đầu vào layer $\ell + n$; chọn ℓ^* làm khoảng cách đó nhỏ nhất; xoá n layer từ ℓ^*; tuỳ chọn heal bằng QLoRA. Kết quả chính: với Llama-2-70B bỏ được "khoảng một nửa" layer trước khi độ chính xác QA sụp xuống ngẫu nhiên. Tuy nhiên, loss next-token xấu đi đều đặn theo tỉ lệ cắt, và các tác vụ suy luận (GSM8K, HellaSwag) hỏng ngay khi cắt chút ít.

3. Phương pháp chi tiết

3.1 Trực giác (Eq. 1–5)

Nếu xoá layer $\ell - 1$ thì layer ℓ nhận $x^{(\ell-1)}$ thay vì $x^{(\ell)}$:

$$x^{(\ell+1)} = x^{(\ell-1)} + f(x^{(\ell-1)}, \theta^{(\ell)}).$$

Mismatch này nhỏ nếu biểu diễn hội tụ thành hàm biến thiên chậm. Để cắt n layer trước layer ℓ cần $x^{(\ell)} \approx x^{(\ell-n)} + \epsilon$. Ba giả thuyết paper kiểm chứng: (0) cắt được layer của residual net; (1) cắt layer sâu dễ hơn layer

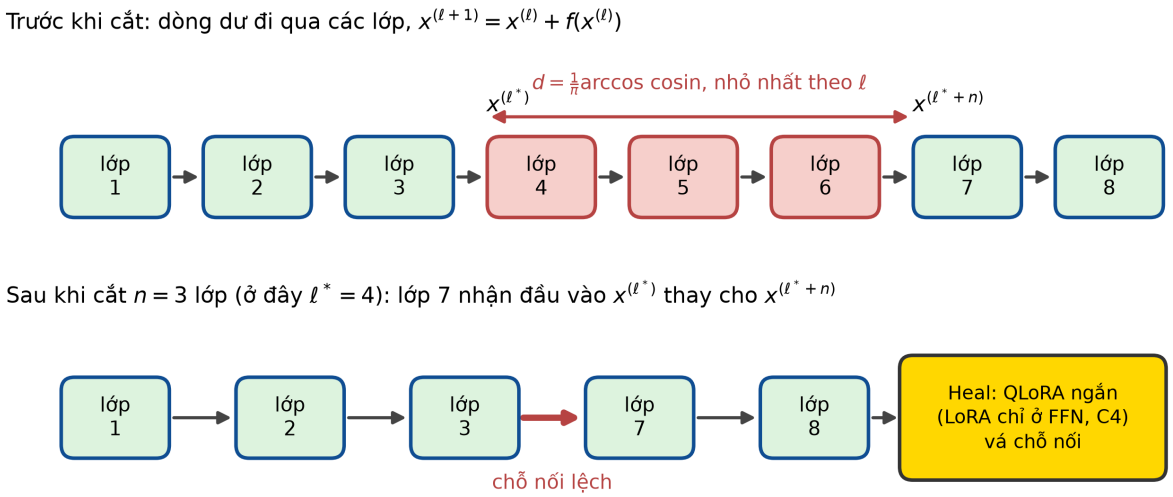
nông; (2) khối cắt thành công có đầu ra giống đầu vào.

3.2 Khoảng cách góc và chọn khối

$$d(x^{(\ell)}, x^{(\ell+n)}) = \frac{1}{\pi} \arccos\left(\frac{x_T^{(\ell)} \cdot x_T^{(\ell+n)}}{\|x_T^{(\ell)}\| \, \|x_T^{(\ell+n)}\|} \right), \qquad \ell^*(n) = \arg\min_{\ell} d(x^{(\ell)}, x^{(\ell+n)})$$

T là **token cuối** của chuỗi (vì causal mask nên chỉ embedding này phụ thuộc cả chuỗi). Cộng/trung bình trên nhiều mẫu (heat-map dùng 10k mẫu C4 validation). Paper nói rõ lựa chọn góc thay vì cosine "không quan trọng".

```
Input: model L layers, số layer cần cắt n, tập dữ liệu trung tính (C4)
1. for mỗi ℓ in 0..L-n:  d[ℓ] = mean_samples angular_distance(x[ℓ], x[ℓ+n])  # x[ℓ] =
↪  input của layer ℓ
2. ℓ* = argmin d
3. xoá layer ℓ* .. ℓ*+n-1 (nối input của layer ℓ* thẳng vào layer ℓ*+n)
4. (tuỳ chọn) heal: QLoRA trên C4
```



Dữ liệu chọn khối: token cuối của mỗi chuỗi, trung bình trên mẫu trung tính (C4).

Hình 55: Chọn khối n lớp liền nhau theo khoảng cách góc, xoá khối rồi heal chỗ nối bằng QLoRA (minh hoạ, tự tính)

Cách đọc hình: hàng trên là mạng 8 lớp với khối 3 lớp (tô đỏ) có đầu vào đầu khối và đầu vào sau khối gần nhau nhất. Hàng dưới là mạng sau cắt: lớp 7 phải đọc đầu vào của lớp 4 cũ, mũi tên đỏ đậm là chỗ nối lệch mà healing nhẹ cần vá. Số lớp và vị trí trong hình chỉ để minh hoạ.

3.3 Heuristic đơn giản hơn

Cắt n layer sâu nhất **trừ layer cuối**: bỏ layer $L-n$ đến $L-1$ (không cần chạy model gốc trên GPU). Ý nghĩa: ablation cho việc tối ưu khối.

3.4 Healing (Appendix B)

QLoRA 4-bit; LoRA chỉ gắn vào các module **FFN**; dữ liệu C4; 5000 bước, batch toàn cục 16; cosine LR với 100 bước warmup; 164M token (model ≤7B, seq 2048) hoặc 328M token (≥13B, seq 4096). Bảng hyperparameter của paper: Llama-2-7B rank 2, LR 3e-4; Llama-2-13B rank 64; Llama-2-70B rank 8, LR 3e-5;

Mistral-7B rank 4, LR 3e-6; Qwen và Phi-2 rank 64. Chọn rank theo quy trình: bắt đầu 64, nếu heal làm xấu hơn không heal thì quét rank.

Gợi ý trong footnote 8 (chưa thử): thêm loss phụ thầy-trò $\mathcal{L}_{aux} \sim \|x^{(\ell^*+n)}(\theta_0) - x^{(\ell^*)}(\theta)\|^2$ để ép đầu vào của layer sau khớp với mô hình gốc.

4. Thí nghiệm và kết quả chính

Mô hình: Llama-2 7B/13B/70B, Qwen-7B/14B, Mistral-7B, Phi-2 (2.7B). **Eval:** MMLU 5-shot, BoolQ 0-shot, loss next-token trên C4 validation (chuẩn hoá bằng $\log V$); sau đó CoT-MMLU, GSM8K, HellaSwag.

Các con số paper nêu bằng chữ (đọc từ đồ thị, không có bảng):

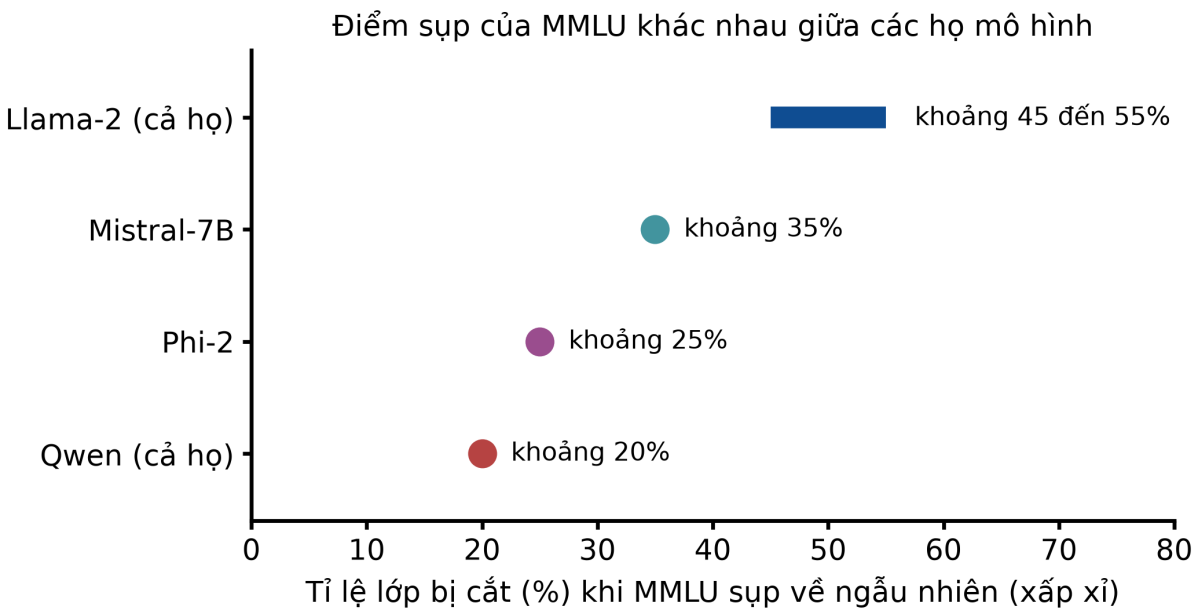
Mô hình	Tỉ lệ layer cắt thì MMLU sụp về ngẫu nhiên (xấp xỉ)
Llama-2 family	khoảng 45%–55%
Mistral-7B	khoảng 35%
Phi-2	khoảng 25%
Qwen family	khoảng 20%

- Dạng đường cong: vùng phẳng (gần như không mất) rồi **chuyển pha** sang ngẫu nhiên (25% với MMLU 4 lựa chọn).
- Healing chỉ cải thiện **khiêm tốn** accuracy QA và đẩy điểm sụp sang phải một chút.
- **Loss next-token**: không heal thì cũng sụp gần chỗ QA sụp; heal xong thì loss gần mức chưa cắt và tăng chậm, gần tuyến tính theo tỉ lệ cắt, **liên tục** qua điểm mà QA sụp (đến ít nhất 80% với Llama-2-70B theo chú thích Fig. 1).
- Mô hình lớn/sâu hơn (Llama-2-13B, 70B) bền hơn; tác giả đưa 2 giả thuyết: mô hình nhỏ bị overtrain nên ít dư thừa hơn, hoặc mô hình sâu chịu mất nhiều layer tuyệt đối hơn.
- **Heuristic "cắt layer sâu nhất"**: không heal thì rất tệ; sau heal gần như ngang chiến lược similarity (similarity hơi tốt hơn trước điểm chuyển pha; heuristic có thể đẩy điểm chuyển pha hơi xa hơn). Loss hầu như chồng lên nhau, similarity nhỉnh hơn một chút.
- Kết luận của tác giả: mục đích của fine-tune sau cắt là **vá giao diện**, không phải học thêm tri thức.
- **Reasoning (Llama-2-70B, Fig. 6):** GSM8K và HellaSwag giảm ngay với bất kỳ mức cắt nào; CoT-MMLU có vùng phẳng giống MMLU. MMLU không CoT ~69% so với CoT-MMLU ~43% ở 0% cắt.

Cách đọc hình: các số là ước lượng đọc từ đồ thị của paper nên chỉ nên xem là thứ tự cỡ. Llama-2 chịu cắt tới khoảng 45 đến 55% lớp, còn Qwen chỉ khoảng 20%. Điểm sụp này là của câu hỏi trắc nghiệm, còn loss next-token thì đi xuống đều qua cùng khoảng đó.

5. Phân tích / ablation

- **Chuẩn dữ liệu chọn khối:** thông thường dùng C4 (trung tính); có thể dùng dữ liệu đúng tác vụ đích (đề xuất, chưa đánh giá kỹ).
- **LoRA rank (Appendix C.3):** rank thấp hơn thường cải thiện nhẹ; hành vi định tính giống nhau giữa các rank. Có ablation về prompt và seed (Appendix C.1, C.2); text không đủ để trích số.
- **Layer cuối đặc biệt:** khối chứa layer cuối luôn có khoảng cách cực đại hoặc gần cực đại, nên không bao giờ cắt layer cuối. Ngoại lệ: Phi-2 và các khối lớn của Llama-2-7B cho thấy vài layer cuối cũng quan trọng.
- **Qwen "kỳ lạ":** heat-map có các "đảo" tương tự cao ở khối nông, tác giả cho rằng điều này giải thích vùng bền ngắn hơn.
- Giả thuyết tác giả: layer nông lưu/truy xuất tri thức; layer sâu dùng cho tính toán bậc cao (suy luận toán). Đây là **giả thuyết phù hợp dữ liệu**, chưa được chứng minh.



Hình 56: Tỉ lệ lớp bị cắt khi MMLU sụp về mức ngẫu nhiên theo từng họ mô hình (số xấp xỉ trong ghi chú) (vẽ lại từ số liệu trong ghi chú)

6. Giới hạn

Do tác giả nêu:

- Chưa biết chiến lược cắt tốt hơn hoặc cách heal tốt hơn; chưa giải thích vì sao heal xoá chuyển pha ở loss nhưng không ở QA.
- Eval mới có QA, một ít suy luận; cần eval toàn diện hơn; chưa rõ mô hình over-trained hay distilled có khó cắt hơn không.
- Dùng QLoRA vì giới hạn tính toán; rank chọn bằng quét hạn chế do ngân sách.

Nhận xét:

- Chỉ tính khoảng cách ở **token cuối**; không rõ độ ổn định với token khác (vd. chuỗi OCR dài).
- Kết quả là đồ thị, không bảng; rất khó tái lập con số chính xác từ text.
- Không đo latency/throughput thật; "heal bằng 164M–328M token C4" khác nhiều so với hồi phục bằng distillation.
- Chỉ LLM Transformer thuần, không có hybrid linear-attention.

7. Liên hệ với các bài khác

- **31 ShortGPT:** cùng bản chất (so biểu diễn vào/ra), nhưng ShortGPT cắt layer rời theo BI không heal; bài này cắt khối liền và heal. Cả hai kết luận layer sâu dư.
- **33 Shortened LLaMA:** so sánh kỹ cách retrain (LoRA vs continued pretraining) sau depth pruning, bổ sung bài này.
- **34 SLEB:** chọn lặp từng block bằng perplexity thực, không đo góc.
- **36 ReplaceMe:** thay vì heal bằng LoRA toàn mô hình, tìm ánh xạ tuyến tính đóng ở chỗ nối (dạng đóng, không train).
- **37:** chỉ ra khi có repair tuyến tính thì layout tối ưu chuyển từ rời sang khối liền, nhất quán với lựa chọn "khối liền" ở đây.
- **39–40 Minitron:** đối chiếu depth pruning kết hợp distillation.

8. Thí nghiệm gợi ý

2. Cắt khối ℓ^* vs khối sâu nhất (trừ layer cuối), rồi heal bằng feature-matching tại điểm nối (Eq. 8) với 1–2k bước; so CER.
3. Đo CER theo số layer cắt n để xem có "chuyển pha" như MMLU không, hay đi xuống mượt như loss C4.

9. Tóm tắt công thức / khái niệm

Khái niệm	Ý nghĩa
$d(x^{(\ell)}, x^{(\ell+n)})$	khoảng cách góc (chia π) giữa input layer ℓ và $\ell + n$
$\ell^*(n)$	điểm bắt đầu khối n layer có khoảng cách nhỏ nhất
Healing	QLoRA ngắn (FFN-only LoRA, C4) để vá chỗ nối
Simple heuristic	cắt n layer sâu nhất, trừ layer cuối
Chuyển pha	QA accuracy phẳng rồi rơi về ngẫu nhiên tại tỉ lệ cắt phụ thuộc mô hình

10. Kết luận ngắn

Cắt một khối liền layer sâu rồi heal nhẹ cho QA khá bền, nhưng loss next-token và suy luận thì không. Đây là bằng chứng sớm rằng thước đo đánh giá quyết định mức "cắt được bao nhiêu", điều quan trọng khi đánh giá OCR bằng CER thay vì trắc nghiệm.

Link gốc: https://arxiv.org/abs/2403.17887

Bài 33

Shortened LLaMA: Depth Pruning for Large Language Models with Comparison of Retraining Methods (2024)

Tác giả: Bo-Kyeong Kim, Geonmin Kim, Tae-Ho Kim, Thibault Castells, Shinkook Choi, Junho Shin, Hyoung-Kyu Song (Nota Inc., Captions).
Venue / arXiv: arXiv:2402.02834 (v2, 23/06/2024). Bản text không ghi tên hội nghị.
Nguồn: https://arxiv.org/abs/2402.02834
Chủ đề: Depth pruning (xoá Transformer block) so với width pruning, xét **độ trễ batch-1**; so sánh LoRA với continued pretraining (CPT) để hồi phục.
Vị trí trong lộ trình: Đọc trước: 26 LLM-Pruner, 28 FLAP (đối thủ width), 31 ShortGPT, 32 Deeper layers. Bài này thêm hai thứ quan trọng: (1) bằng chứng đo thật rằng **width pruning không tăng tốc ở batch nhỏ**; (2) bảng so sánh phương pháp hồi phục theo mức cắt. Đọc tiếp: 34 SLEB, 36 ReplaceMe, 39 Minitron.
Chương nền liên quan: nền 06: Cắt lớp và độ sâu trong LLM, nền 07: Đo đúng một kết quả pruning

> Tài liệu học tập tiếng Việt, ghi chú diễn giải, không phải bản dịch paper. Số liệu lấy từ paper; bảng đã được đọc kỹ nhưng text PDF hai cột bị lẫn nên chỉ trích những dòng đọc chắc chắn.

1. Bối cảnh

- **Width pruning** thu nhỏ ma trận bằng cách bỏ attention head hoặc neuron FFN nhưng giữ số lớp. **Depth pruning** bỏ cả block hoặc lớp, còn kích thước các ma trận được giữ lại không đổi.
- **Memory-bound**: khi sinh từng token với batch nhỏ, thời gian bị chi phối bởi việc đọc trọng số từ bộ nhớ GPU, không phải bởi số phép nhân. GPU ít được dùng hết sức mạnh tính toán. Tăng batch giúp nhưng dễ OOM trên GPU nhỏ.
- Các phương pháp lúc đó (LLM-Pruner, FLAP, Wanda-sp) làm width pruning và thường chỉ báo cáo độ chính xác, ít đo tốc độ thật. Depth pruning bị coi là kém vì đơn vị cắt thô.

2. Ý tưởng chính

Tác giả chọn **Transformer block** (MHA + FFN) làm đơn vị cắt, chấm điểm từng block bằng một tiêu chí rẻ (perplexity khi bỏ block đó, hoặc Taylor bậc 1 có bảo vệ), bỏ các block ít quan trọng nhất trong **một lần** (one-shot). Các phép đo cho thấy: ở batch-1, cắt chiều rộng gần như không giảm latency (thậm chí tăng vì kích thước ma trận không thân thiện GPU), còn bỏ cả block thì giảm số lần truy cập bộ nhớ nên nhanh thật. Phần thứ hai: so sánh LoRA với CPT; với mức cắt nhẹ LoRA đủ, với mức cắt nặng (dưới ~3.7B từ 6.7B) cần CPT toàn tham số.

3. Phương pháp chi tiết

3.1 Tiêu chí chấm block

Ký hiệu $W^{k,n}_{i,j}$ là trọng số của phép tuyến tính loại k (Q/K/V/O, up/gate/down) trong block n.

- **Magnitude (Mag):** $I^n_{\text{Mag}} = \sum_k \sum_i \sum_j |W^{k,n}_{i,j}|$ (tính ở mức neuron đầu ra rồi cộng; theo mô tả paper).
- **Taylor:** thay đổi loss khi đặt trọng số = 0, bỏ số hạng bậc hai:

$$L(W) - L(W=0) \approx \frac{\partial L}{\partial W} W, \qquad I^n_{\text{Taylor}} = \sum_k \sum_i \sum_j \left| \frac{\partial L(D)}{\partial W^{k,n}_{i,j}} W^{k,n}_{i,j} \right|$$

- **Mag+ / Taylor+:** như trên nhưng **bảo vệ 4 block đầu và 2 block cuối** (không cho vào danh sách ứng viên), vì xếp hạng thuần tuý hay chọn block đầu nhưng bỏ chúng thì hỏng.
- **PPL:**

$$I^n_{\text{PPL}} = \exp\Big(-\tfrac{1}{SL} \sum_s \sum_l \log p_{\theta_n}(x_l^{(s)} \mid x_{<l}^{(s)}) \Big)$$

với θ_n = mô hình bỏ block n. Block có PPL sau khi bỏ **thấp** hơn thì ít quan trọng hơn. Chỉ cần forward, không gradient. Không cần bảo vệ block đầu/cuối: PPL tự phát hiện chúng quan trọng.

3.2 One-shot

```
1. calibration D = 10 mẫu BookCorpus, độ dài 128
2. for n in blocks: tính I_n (PPL khi bỏ block n, hoặc Taylor+)
3. sắp xếp tăng dần, bỏ các block ít quan trọng nhất đến khi đạt ngân sách tham số
4. retrain: LoRA (nhẹ) hoặc CPT (nặng) hoặc CPT -> LoRA
```

Có thể làm lặp (như SLEB, cập nhật điểm sau mỗi lần bỏ) nhưng tác giả quan sát retraining quan trọng hơn lặp hay một lần, đặc biệt khi cắt nặng, và lặp tốn thời gian hơn.

3.3 Hồi phục

- **LoRA:** rank 8, LR 10^{-4}, batch 64, 2 epoch, trên 50K mẫu Alpaca đã làm sạch; LoRA gắn vào mọi ma trận chiếu. Cắt 20% mô hình 7B: ~2 giờ, 22GB VRAM (một A100).
- **CPT:** cập nhật toàn tham số trên SlimPajama (627B token); AdamW (0.9, 0.95), weight decay 0.1, LR 10^{-4}, batch toàn cục 512, 8 H100, tối đa ~2 tuần mỗi kích thước. Thời gian/token thực tế: 5.5B: 6 ngày, 37B token; 3.7B: 8 ngày, 74B token; 2.7B: 12 ngày, 150B token; 1.5B: 11 ngày, 271B token.
- **CPT -> LoRA:** CPT xong rồi LoRA trên dữ liệu instruction.

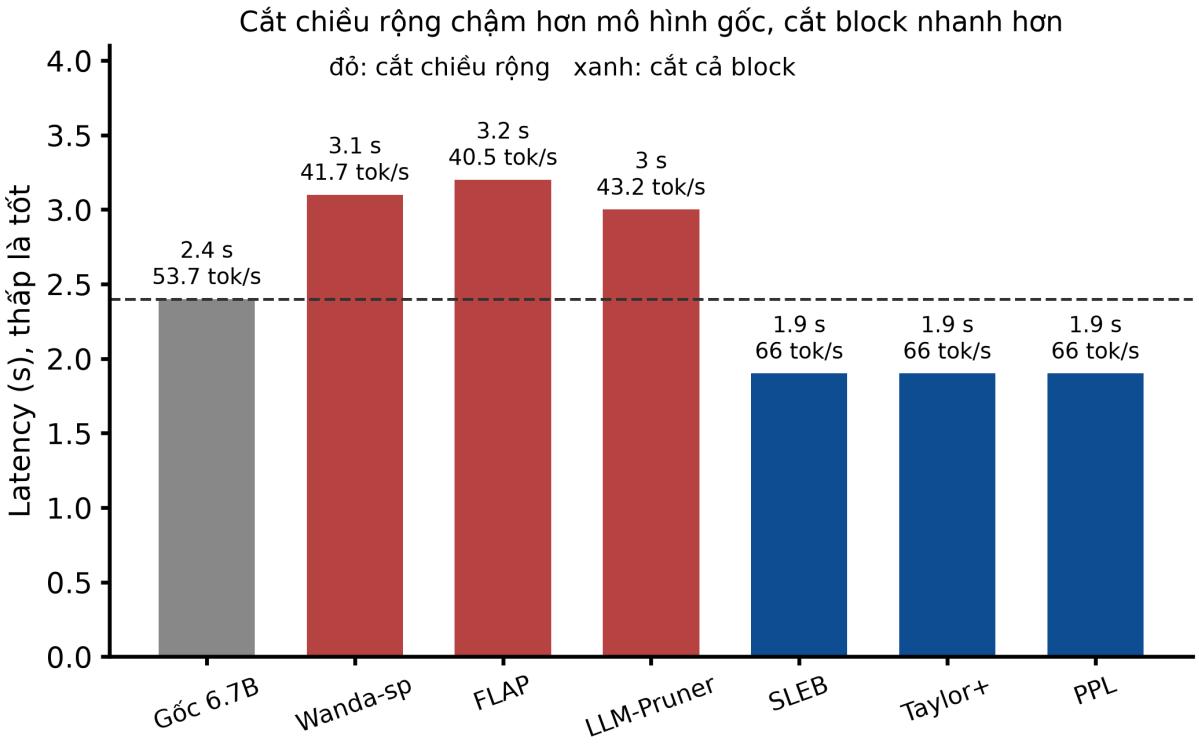
4. Thí nghiệm và kết quả chính

Mô hình: LLaMA-7B, Vicuna-7B/13B-v1.3. **Baseline width:** LLM-Pruner, FLAP, Wanda-sp. **Baseline depth:** SLEB (không retrain). **Eval:** 7 tác vụ commonsense zero-shot (BoolQ, PIQA, HellaSwag, WinoGrande, ARC-e, ARC-c, OBQA), PPL WikiText2/PTB. **Tốc độ:** 12 token vào, 128 token ra, **batch 1**, một GPU.

4.1 Cắt vừa + LoRA (Table 2, LLaMA-7B, 20% cắt, 5.5B)

Phương pháp	PPL WikiText2	PPL PTB	Ave Acc (%)	Latency H100 (s)	Throughput H100 (tok/s)
Gốc 6.7B	12.6	22.1	66.3	2.4	53.7
Wanda-sp	21.4	47.2	51.8	3.1	41.7
FLAP	17.0	30.1	59.5	3.2	40.5
LLM-Pruner	17.6	30.4	61.8	3.0	43.2
SLEB	18.5	31.6	57.6	1.9	66.0
Ours Taylor+	20.2	32.3	63.5	1.9	66.0
Ours PPL	17.7	30.7	61.9	1.9	66.0

Điểm mấu chốt: mọi phương pháp width **chậm hơn mô hình gốc** (latency 3.0–3.2 s so với 2.4 s) dù ít tham số hơn. Trên RTX3090 24GB, gốc 5.1 s (25.0 tok/s), depth 4.5 s (28.4 tok/s), width 6.0–7.7 s. Với 35% cắt (4.5B) depth: latency 1.6 s, 80.1 tok/s trên H100. Với Vicuna-13B gốc, RTX3090 báo OOM; bản depth cắt 21% chạy được (5.4 s, 23.9 tok/s).



Hình 57: Latency batch 1 trên H100 của LLaMA-7B sau khi cắt 20%: cắt chiều rộng so với cắt block, theo Table 2 của ghi chú (vẽ lại từ số liệu trong ghi chú)

Cách đọc hình: thanh xám là mô hình gốc, đường nét đứt là 2,4 s. Ba phương pháp cắt chiều rộng (đỏ) chậm hơn mô hình gốc dù ít tham số hơn, còn cắt cả block (xanh) giảm xuống 1,9 s. Ở batch 1 thời gian bị chi phối bởi việc đọc trọng số và số lần gọi lớp, không chỉ bởi tổng tham số.

4.2 Cắt mạnh: LoRA vs CPT (Table 3, Vicuna-7B)

PPL WikiText2 / Ave Acc (%) tương ứng 5.5B (20%), 3.7B (45%), 2.7B (60%), 1.5B (80%):

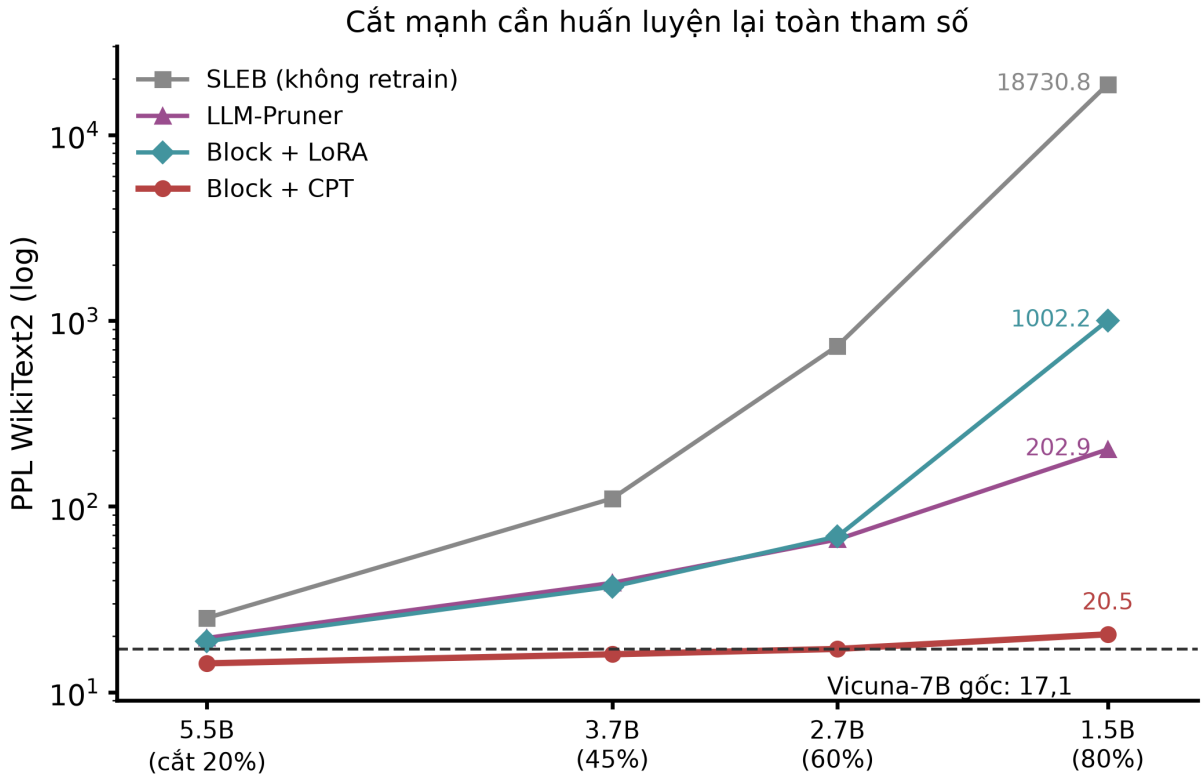
Phương pháp	PPL (5.5B, 3.7B, 2.7B, 1.5B)	Ave Acc (5.5B, 3.7B, 2.7B, 1.5B)
LLM-Pruner	19.6, 38.8, 66.4, 202.9	60.1, 50.1, 44.3, 38.4
SLEB	25.1, 110.4, 731.5, 18730.8	55.6, 40.2, 39.1, 37.4
Ours, LoRA	18.8, 37.0, 68.9, 1002.2	60.7, 47.0, 40.1, 37.1
Ours, CPT	14.3, 16.0, 17.1, 20.5	61.5, 57.1, 55.0, 49.2
Ours, CPT->LoRA	14.8, 16.5, 17.8, 21.1	63.1, 57.4, 55.0, 49.0

Throughput batch-1 H100 của bản depth: 66.0, 84.0, 107.4, 182.5 tok/s (tức 1.2x, 1.6x, 2.0x, 3.4x so với gốc 53.7); width chỉ ~40–45 tok/s. Mô hình gốc: PPL 17.1 trên WikiText2 (dòng Vicuna-7B Original), Ave Acc 65.9.

Ví dụ sinh văn bản (Table 4, 2.7B, cắt 60%): các baseline (Wanda-sp, FLAP, LLM-Pruner, SLEB) sinh rác; mô hình CPT sinh văn bản trôi chảy. **Khởi tạo CPT từ mô hình đã cắt tốt hơn train từ đầu cùng kiến trúc** (Fig. 5, 2.7B).

4.3 Lượng tử hoá

GPTQ 4-bit áp dụng lên mô hình depth-pruned sau CPT giảm VRAM mà không giảm zero-shot đáng kể (Fig. 6; số chi tiết ở Appendix D, chưa đọc trong bản text này).



Hình 58: PPL WikiText2 của Vicuna-7B theo cỡ mô hình sau cắt: SLEB, LLM-Pruner, block với LoRA và block với CPT, theo Table 3 của ghi chú (vẽ lại từ số liệu trong ghi chú)

Cách đọc hình: trục dọc là thang log, trục ngang đi từ cắt nhẹ (5.5B) tới cắt rất mạnh (1.5B). Với LoRA, PPL vọt lên 1002,2 ở 1.5B, còn CPT giữ ở 20,5, gần mức 17,1 của mô hình gốc. Đây là lý do cắt nặng cần huấn luyện lại toàn tham số với nhiều dữ liệu.

5. Phân tích / ablation

Tiêu chí (Table 5, LLaMA-7B, 20% cắt 5.5B): Mag: PPL 7720.7, Acc 34.4; Mag+: 19.4, 56.1; Taylor: 3631.7, 35.5; Taylor+: 20.2 (PTB 32.3), Acc 63.5; PPL: 17.7 (PTB 30.7), Acc 61.9. Ở 35% (4.5B): Taylor+ PPL 33.2, Acc 55.4; PPL 23.1, Acc 55.2. Kết luận tác giả: Taylor+ tốt về accuracy commonsense; PPL tốt hơn về chất lượng sinh/perplexity. Không có "+" thì chọn nhầm block đầu và sập.

Đơn vị cắt (Table 6): cắt cả block vs cắt riêng MHA/FFN. Với model >5B, cắt từng MHA/FFN cho acc tốt hơn chút nhưng PPL xấu hơn (5.7B: acc 63.1 vs 62.8, PPL 20.8 vs 16.9); với model <5B, block tốt hơn mọi mặt (ví dụ ~4.5B/4.6B: block PPL 23.1 acc 55.2, module PPL 38.9 acc 52.5). Giải thích (giả thuyết của tác giả): MHA và FFN phối hợp; cắt module riêng có thể tạo ra các FFN liên tiếp; cắt block thì block lân cận bù được.

Lượng calibration (Table 7): 10, 50, 100, 1000 mẫu cho kết quả gần như không đổi; Taylor+ tụt acc ở 1000 mẫu (61.7 so với 63.5), chưa lý giải.

Cần lưu ý: trên chính bảng của họ, CPT mới là yếu tố tạo khác biệt, không phải tiêu chí chọn block.

6. Giới hạn

Do tác giả nêu: chưa thử mô hình >13B; CPT thiết yếu cho cắt nặng và rất tốn (~2 tuần x 8 H100); chưa khám phá corpus/hyperparameter khác; chưa đánh giá độ phù hợp với sở thích người dùng (safety, helpfulness) sau khi cắt/retrain/lượng tử hoá.

Nhận xét:

- Chỉ đo batch 1 với 12 token vào / 128 ra, trên GPU datacenter (H100, A100) và RTX3090..
- Mọi so sánh "ít tham số" theo tổng tham số; kết luận width chậm là do cài đặt kích thước ma trận không chia hết cho 8 (FFN-D lẻ như 7156) trong HF Transformers, một phần có thể sửa bằng làm tròn kích thước (nhận xét; tác giả có nêu vấn đề chia hết cho 8 nhưng không thử sửa).
- Chỉ đánh giá commonsense zero-shot và PPL, không đánh giá tác vụ sinh dài có cấu trúc.

7. Liên hệ với các bài khác

- **26 LLM-Pruner, 28 FLAP, 29 SliceGPT:** đối thủ width; bài này cho thấy lợi thế depth về latency batch nhỏ.
- **31 ShortGPT, 32 Deeper layers:** cùng họ depth; bài này không dùng cosine/góc mà dùng PPL và Taylor+.
- **34 SLEB:** đối thủ trực tiếp (retrain-free, lặp); bài này thêm retrain và thấy cắt nặng thì SLEB không đủ.
- **43 Pruning vs scratch, 38 Sheared LLaMA:** bằng chứng "khởi tạo từ mô hình đã cắt tốt hơn train từ đầu" tương đồng.
- **39 Minitron:** cùng triết lý depth + retrain quy mô lớn; Minitron dùng distillation thay vì CPT thuần.

8. Gợi ý áp dụng

- Cần đo lại thay vì suy ra.
- Nên đánh giá bằng **PPL block-ablation** (một forward/block) trước khi dùng Taylor; nhỏ gọn, ít giả định. Với decoder ~20 layer chỉ cần 20 forward.
- Quy tắc bảo vệ block đầu (4) và cuối (2) trong Mag+/Taylor+ hữu ích khi tự cài đặt.

9. Thí nghiệm gợi ý

2. PPL block-ablation (từng block của decoder, và từng block ViT), vẽ biểu đồ như Fig. 4, so với BI (bài 31).

10. Tóm tắt công thức / khái niệm

Khái niệm	Ý nghĩa
I^n_{PPL}	PPL trên calibration khi bỏ block n (thấp = ít quan trọng)
Taylor+	$\lvert\partial L/\partial W \cdot W\rvert$ cộng theo block, bảo vệ 4 block đầu + 2 block cuối
CPT	continued pretraining toàn tham số trên corpus lớn
CPT->LoRA	CPT rồi LoRA instruction
Memory-bound	độ trễ do đọc bộ nhớ, không do FLOPs

11. Kết luận ngắn

Trong thiết lập đo với batch nhỏ, việc xoá cả block mới đem lại tăng tốc; cắt vừa thì LoRA đủ, cắt mạnh thì phải huấn luyện lại toàn tham số với nhiều dữ liệu.

Link gốc: https://arxiv.org/abs/2402.02834

Bài 34

SLEB: Streamlining LLMs through Redundancy Verification and Elimination of Transformer Blocks (2024)

Tác giả: Jiwon Song, Kyungseok Oh, Taesu Kim, Hyungjun Kim, Yulhwa Kim, Jae-Joon Kim (Seoul National University, SqueezeBits, Sungkyunkwan University).
Venue / arXiv: arXiv:2402.09025 (v6, 13/12/2024); "Proceedings of the 41st International Conference on Machine Learning, Vienna, Austria. PMLR 235, 2024" (ICML 2024).
Nguồn: https://arxiv.org/abs/2402.09025
Chủ đề: Depth pruning không cần train: xoá lần lượt từng transformer block, mỗi lần chọn lại block bằng perplexity trên mô hình đã bị cắt (greedy lặp).
Vị trí trong lộ trình: Đọc trước: 23 SparseGPT, 18 (2:4 sparsity), 29 SliceGPT (các đối thủ), 31 ShortGPT. Bài này đưa ra **ý then chốt: tầm quan trọng của một block thay đổi khi các block khác bị bỏ**, nên phải chọn lặp. Đọc tiếp: 33 (đã so SLEB ở cắt nặng), 36 ReplaceMe, đặc biệt 37 (bài này dùng "greedy với chi phí ngang SLEB" nhưng chấm điểm *dưới repair*).
Chương nền liên quan: nền 06: Cắt lớp và độ sâu trong LLM

Tài liệu học tập tiếng Việt, ghi chú diễn giải, không phải bản dịch paper. Mọi số liệu lấy từ paper.

1. Bối cảnh

- **2:4 sparsity**: trong mỗi nhóm 4 trọng số liên tiếp, 2 cái bằng 0 (50% thưa). GPU NVIDIA đời mới có Tensor Core hỗ trợ, về lý thuyết nhanh gấp 2. Bài báo đo (Fig. 3, RTX A6000) rằng với batch từ 1 đến 128 thì 2:4 **không** tăng tốc phép nhân ma trận; chỉ khi batch lớn hơn mới đạt khoảng 1.5x.
- **Channel-wise pruning** (LLM-Pruner, SliceGPT): cắt hàng/cột ma trận để giữ ma trận dày. Nhưng tăng tốc nhân ma trận chưa chắc thành tăng tốc end-to-end vì còn LayerNorm, attention...
- **Early exit**: dừng không chạy các block còn lại khi đủ tự tin. Cần dự đoán động, khó với batch, không giảm bộ nhớ, và phải train; Fig. 5 ước lượng ngay cả với bộ dự đoán lý tưởng vẫn phải chạy ~90% block.
- **Quan sát về residual**: $x_{i+1} = T_{i+1}(x_i) + x_i$, nên đầu ra các block liên tiếp giống nhau (cosine cao, Fig. 6).

2. Ý tưởng chính

Dùng **transformer block** làm đơn vị cắt tĩnh để tăng tốc tỉ lệ thuận với số block bỏ, ở mọi batch size, và giảm cả bộ nhớ. Câu hỏi là chọn block nào. Hai thử nghiệm thất bại dẫn đến thuật toán: (1) cosine giữa vào và ra của block làm tăng perplexity mạnh vì thay đổi nhỏ ở block sớm bị khuếch đại phía sau; (2) chọn k block có "perplexity khi bỏ riêng" thấp nhất cũng hỏng vì các block bị chọn nằm liền nhau (3,4,5,6,7,8,10 ở OPT-6.7B) và bỏ cả chuỗi hại nhiều hơn tổng từng cái. Giải pháp: **bỏ từng block một, sau mỗi lần tính lại độ quan trọng trên mô hình đã bỏ**. Không huấn luyện gì cả.

3. Phương pháp chi tiết

Ba metric (với A_j, B_j là đầu vào/ra block j, M là mô hình, M_j là mô hình bỏ block j, M' là mô hình đã bỏ một số block):

$$\text{Metric}_j^1 = 1 - \text{similarity}(A_j, B_j), \qquad \text{similarity}(x_i, x_j) = \frac{x_i \cdot x_j}{\|x_i\|\|x_j\|}$$

$$\text{Metric}_j^2 = -\frac{1}{K}\sum_k \log p_{M_j}(w_k \mid w_{<k})$$

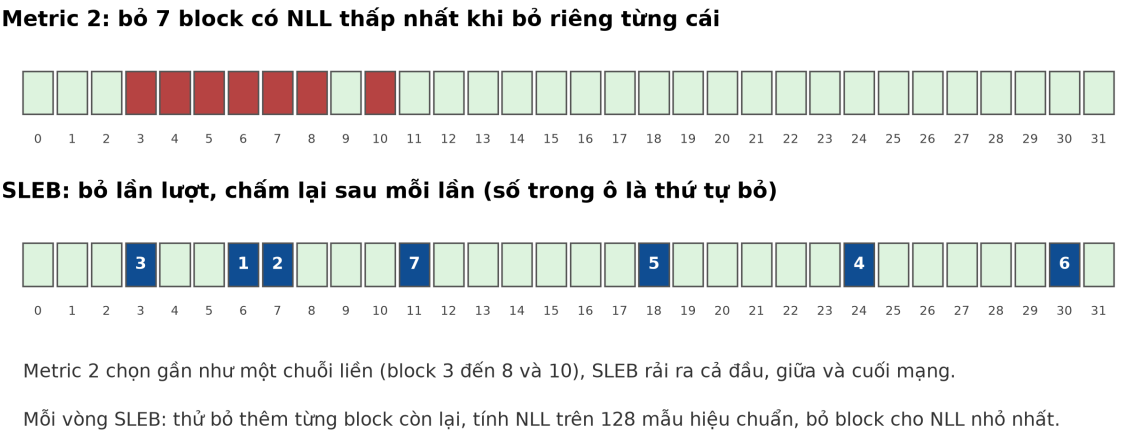
$$\boxed{\text{Metric}_j^3(M') = -\frac{1}{K}\sum_k \log p_{M'_j}(w_k \mid w_{<k})}$$

Metric 3 là negative log-likelihood (log của perplexity) trên calibration khi bỏ thêm block j khỏi mô hình hiện tại M'. Block có giá trị nhỏ nhất bị bỏ.

```
SLEB (Algorithm 1)
M <- mô hình gốc;  C <- calibration;  N <- số block;  n <- số block cần bỏ
for i in 0..n-1:                      # lặp n lần
    minS = +inf
    for j in 0..N-i-1:                # mọi block còn lại
        S = Metric3_j(M, C)           # NLL trên C khi bỏ thêm block j
        if S < minS: minS, idx = S, j
    M <- remove_block(M, idx)
return M                              # không fine-tune
```

Chi phí: $O(n \cdot N)$ lần forward trên calibration. Làm tròn lên số block cần bỏ nếu tích không nguyên. Calibration: 128 mẫu ngẫu nhiên từ tập train WikiText-2. SLEB cần 2 GPU cho OPT-66B/LLaMA-70B, 1 GPU cho mô hình nhỏ hơn; LLaMA-2-70B nén xong trong khoảng 1.5 giờ.

Ví dụ thứ tự bỏ trên OPT-6.7B (7 block): 6, 7, 3, 24, 18, 30, 11, tức không còn là chuỗi liền kề như Metric 2 (3,4,5,6,7,8,10).



Hình 59: Thứ tự bỏ 7 block của OPT-6.7B theo Metric 2 (chấm một lần) và theo SLEB (lặp), số block lấy từ ghi chú (vẽ lại từ số liệu trong ghi chú)

Cách đọc hình: mỗi ô là một block đánh số từ 0 đến 31 (cách đánh số này do hình chọn). Hàng trên là 7 block có NLL thấp nhất khi bỏ riêng lẻ, gần như một chuỗi liền; hàng dưới là thứ tự SLEB bỏ, số trong ô là lượt bỏ. Chấm lại sau mỗi lần bỏ làm các block được chọn rải rộng hơn.

4. Thí nghiệm và kết quả chính

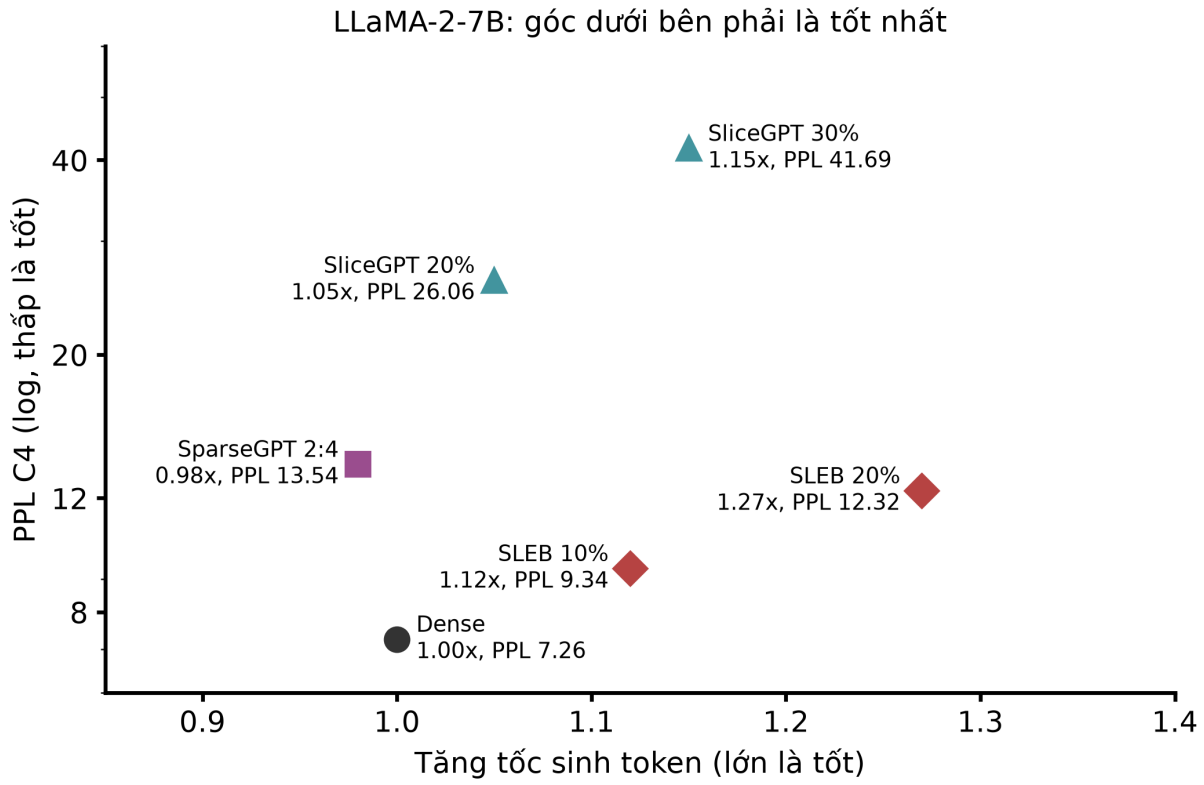
Mô hình: OPT 6.7B/13B/30B/66B, LLaMA-2 7B/13B/70B. **Sparsity:** 10% và 20% (làm tròn lên số block). **Baseline:** SparseGPT, Wanda, DSnoT (2:4, 50%), LLM-Pruner, SliceGPT (channel 20–30%). **Eval:** PPL

(C4, WikiText-2), 5 tác vụ zero-shot (PIQA, WinoGrande, HellaSwag, ARC-e, ARC-c), đo tốc độ trên 2 A100 với LLaMA-2-70B.

Table 1 (PPL C4 và throughput token-generation, batch/chuỗi 128 token):

Phương pháp	Đơn vị	Sparsity	Throughput	Tăng tốc	OPT-6.7B	LLaMA-2-7B	LLaMA-2-70B
Dense	-	0%	299 tok/s	1.00x	12.71	7.26	5.71
SparseGPT	2:4	50%	293	0.98x	16.42	13.54	8.16
SliceGPT	channel	20%	314	1.05x	23.76	26.06	15.84
SliceGPT	channel	30%	343	1.15x	33.43	41.69	25.79
SLEB	block	10%	336	1.12x	13.84	9.34	6.32
SLEB	block	20%	381	1.27x	15.99	12.32	7.31

Wanda và DSnoT 2:4 sập trên OPT-66B (PPL 8414.05 và 8360.81); SLEB giữ 10.99 -> 11.25 (10%) và 12.54 (20%).



Hình 60: LLaMA-2-7B: tăng tốc sinh token và PPL C4 của dense, SparseGPT 2:4, SliceGPT và SLEB, theo Table 1 của ghi chú (vẽ lại từ số liệu trong ghi chú)

Cách đọc hình: góc dưới bên phải là tốt nhất (nhanh hơn và PPL thấp hơn). SLEB 10% và 20% nằm ở vùng đó với tăng tốc 1,12x và 1,27x, còn SparseGPT 2:4 gần như không nhanh hơn (0,98x). Các mức cắt khác nhau nên SLEB 10% và SliceGPT 20% không phải cặp so sánh ngang bằng.

Table 2 (zero-shot trung bình %, latency prompt processing 2048 token, LLaMA-2-70B):

Phương pháp	Latency (ms)	Speedup	OPT-6.7B	LLaMA-2-7B	LLaMA-2-13B	LLaMA-2-70B
Dense	1718.4	1.00x	60.70	69.00	71.76	76.57
SparseGPT 2:4	1555.5	1.10x	54.94	58.23	63.06	71.87
SliceGPT 20%	1658.7	1.04x	56.31	58.17	63.45	72.34
SLEB 10%	1529.1	1.12x	60.00	62.24	66.77	73.14
SLEB 20%	1364.1	1.26x	57.61	56.80	62.96	70.81

Lưu ý: SLEB ở 10% sparsity so với SliceGPT ở 20% là so sánh khác mức cắt; tác giả nhấn mạnh SLEB tăng tốc nhiều hơn dù cắt ít tham số hơn. Ở 20% so với SliceGPT 20%: acc LLaMA-2-7B 56.80 so với 58.17 (SliceGPT cao hơn); PPL SLEB tốt hơn nhiều.

Tương thích lượng tử hoá: SLEB 20% + AWQ 4-bit không làm PPL xấu thêm đáng kể (Fig. 10). **Fine-tuning (Appendix B.8):** LoRA rank 8, 1 epoch trên Alpaca, trên LLaMA-2-7B: SLEB 10% PPL C4 9.17 -> 9.01 (sau FT); SLEB 20% 11.38 -> 10.50. (Bảng 13 trong bản text bị lệch cột; ví dụ hàng "SLEB 20% FT" lặp lại đúng các số của "SliceGPT 20% FT" ở các cột accuracy, và dòng SLEB 10% không FT ghi PPL 9.17 khác 9.34 trong Table 1; không dùng bảng này để trích accuracy.)

5. Phân tích / ablation

- **Vị trí block bị bỏ phụ thuộc mô hình (Fig. 8, Table 4):** OPT-13B thường chọn block sớm; LLaMA-2-13B chọn block giữa-sau (10% bỏ block 31, 27, 13, 12; 20% thêm 33, 29, 14, 2). LLaMA-2-7B 10%: 14, 23, 11, 24. LLaMA-2-70B 10%: 33, 29, 61, 11, 55, 30, 59, 28. Dòng "đừng giả định layer sâu luôn bỏ được".
- **Phụ thuộc calibration (Fig. 9):** SLEB ít phụ thuộc loại calibration (WikiText-2 hay C4) nhất; SliceGPT nhạy nhất khi calibration khác miền eval. Tác giả giải thích: SLEB dùng thông tin toàn mạng.
- **Early exit (Table 5–6):** ngay cả thoát sớm lý tưởng cũng bỏ qua ít block; số tăng throughput nhỏ ở batch lớn.
- **So với bỏ chuỗi cuối (Fig. 4):** "Early Exit" (bỏ block cuối) và "Chunk Best" (khối liền tốt nhất) đều tăng perplexity mạnh hơn nhiều khi số block bỏ tăng, trên OPT-6.7B/13B (không retrain). Đối chiếu bài 32: bài đó có healing.
- **Chi phí:** thời gian chạy thuật toán nằm trong khoảng các phương pháp khác (Appendix A.1, chưa trích bảng).

6. Giới hạn

Do tác giả nêu/ngầm: Sparsity chỉ 10–20%; ở mức cao hơn PPL tăng và tác giả không đề xuất cách hồi phục (retraining chỉ thử bằng LoRA trong phụ lục). Metric 2 khi bỏ nhiều block hỏng ở OPT-6.7B (đã nói).

Nhận xét:

- Chấm điểm bằng PPL trên văn bản thuần (WikiText-2); chưa kiểm tra với tác vụ sinh có cấu trúc.
- Greedy không đảm bảo tối ưu toàn cục; bài 37 đo chính xác khoảng cách tới tối ưu của loại heuristic này.
- Không có bước repair, nên không tận dụng khả năng ánh xạ tuyến tính sửa lỗi (bài 36/37).
- Bài 33 đã chỉ ra SLEB không retrain thì thua nhiều khi cắt mạnh (ví dụ Vicuna-7B 3.7B: Acc 40.2 và PPL 110.4 so với CPT 57.1 và 16.0).
- Throughput đo trên 2 A100 và chuỗi 128 token; không đo batch-1 GPU nhỏ.

7. Liên hệ với các bài khác

- **31 ShortGPT:** chấm từng layer một lần (BI, tĩnh), không lặp; SLEB là phản biện trực tiếp rằng độ quan trọng không cộng tính.
- **32 Deeper layers:** chọn khối liền; SLEB cho thấy Chunk Best (khối liền) kém hơn greedy rời nếu không heal.
- **33 Shortened LLaMA:** dùng làm baseline và nhấn mạnh phải retrain khi cắt mạnh.
- **35 LaCo, 36 ReplaceMe:** các cách thay thế "xoá" bằng gộp/ánh xạ.
- **37:** kết quả then chốt: dưới repair tuyến tính, optimum đổi từ rời sang liền khối; do đó chấm SLEB (không repair) lệch khỏi optimum có repair. Họ dùng greedy "chi phí ngang SLEB" nhưng chấm sau repair.
- **18 (2:4), 23 SparseGPT:** SLEB chứng minh 2:4 không nhanh ở batch nhỏ.

8. Thí nghiệm gợi ý

1. So sánh thứ tự bỏ block do SLEB, Block Influence và phương pháp chọn khối liền đưa ra trên cùng tập hiệu chuẩn.
2. Lặp lại lựa chọn tham lam với một bước repair tuyến tính sau mỗi lần bỏ; kiểm tra thứ tự có thay đổi hay không.

9. Tóm tắt công thức / khái niệm

Khái niệm	Ý nghĩa
Metric 1	$1 - \cos(A_j, B_j)$, thất bại vì khuếch đại phía sau
Metric 2	NLL khi bỏ riêng block j từ mô hình gốc
Metric 3	NLL khi bỏ block j khỏi mô hình đã bỏ các block trước (dùng trong SLEB)
Early exit	dừng động giữa chừng, khác SLEB là tĩnh
2:4	2 trên 4 trọng số bằng 0, chỉ nhanh ở batch lớn

10. Kết luận ngắn

SLEB cho thấy: độ quan trọng của block thay đổi theo ngữ cảnh các block còn lại, nên chọn lặp thay vì chấm một lần. Không cần huấn luyện, tăng tốc tỉ lệ với số block bỏ, nhưng chỉ cắt nhẹ (10–20%) và không tận dụng bước repair.

Link gốc: https://arxiv.org/abs/2402.09025

Bài 35

LaCo: Large Language Model Pruning via Layer Collapse (2024)

Tác giả: Yifei Yang, Zouying Cao, Hai Zhao (Shanghai Jiao Tong University).
Venue / arXiv: arXiv:2402.11187 (v2, 15/10/2024). Bản text không ghi tên hội nghị.
Nguồn: https://arxiv.org/abs/2402.11187
Chủ đề: Depth pruning bằng cách **gộp tham số** các layer phía sau vào một layer phía trước ("layer collapse"), không train, giữ nguyên cấu trúc mô hình.
Vị trí trong lộ trình: Đọc trước: 31 ShortGPT (đối thủ/được so sánh; bảng 2 của ShortGPT dùng benchmark của LaCo), 26 LLM-Pruner, 29 SliceGPT. Bài này đại diện cho nhánh "thay vì xoá, hãy hợp nhất". Đọc tiếp: 36 ReplaceMe (cũng "bù" cho phần bị xoá nhưng bằng ma trận tuyến tính tính từ dữ liệu), 37 (đo chính xác khi nào bù có ích).
Chương nền liên quan: nền 06: Cắt lớp và độ sâu trong LLM

> Tài liệu học tập tiếng Việt, ghi chú diễn giải, không phải bản dịch paper. Mọi con số lấy từ paper (các bảng đã được đọc kỹ; cột lệch được ghi chú).

1. Bối cảnh

- **Model merging** (hợp nhất mô hình): trộn tham số của nhiều mô hình/checkpoint thành một bằng cộng/trung bình có trọng số hoặc cộng "vector hiệu" (task vector). LaCo mượn ý này để hợp nhất các lớp *trong cùng* mô hình.
- Tác giả điểm yếu các hướng khác: lượng tử hoá cần phần cứng; distillation cần huấn luyện lại; unstructured pruning cần phần cứng thưa và hay mất chất lượng; structured pruning (LLM-Pruner, SliceGPT) thay đổi kích thước bên trong, làm khó dùng lại trong hạ tầng sẵn có.
- Layer pruning cho LLM ở thời điểm đó: early exit/layer skipping đều đòi huấn luyện tham số mới để quyết định bỏ layer nào.

2. Ý tưởng chính

Quan sát: tham số của lớp liền kề trong LLM khá giống nhau (khoảng cách L2 giữa các ma trận Q/K/V và MLP của hai layer liền kề thường không quá 200, so với ma trận 4096x4096 là rất nhỏ trên mỗi phần tử) và biểu diễn đầu ra của các lớp từ 3 đến 28 gần như cosine = 1. Vậy có thể cộng "hiệu tham số" của m layer phía sau vào một lớp rồi xoá m layer đó. Việc gộp được thử tuần tự từ trên xuống, và chỉ chấp nhận khi biểu diễn đầu ra (cuối mạng) trên vài câu calibration vẫn giống mô hình gốc ở mức cosine không dưới ngưỡng T.

3. Phương pháp chi tiết

3.1 RDSC Layer Merge (Reserving-Differences-while-Seeking-Common)

Với layer l và m layer liền sau, mọi tham số (attention và MLP, từng ma trận độc lập) được gộp:

$$\theta_l^* = \theta_l + \sum_{k=1}^{m} (\theta_{l+k} - \theta_l)$$

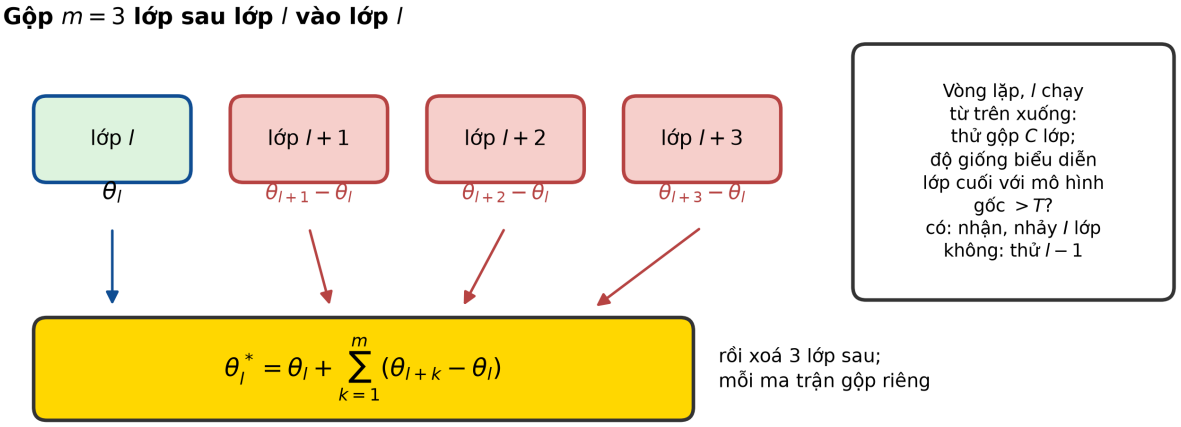
Sau đó xoá các layer $l+1, \ldots, l+m$. "Reserving differences" = giữ các hiệu $\theta_{l+k} - \theta_l$; "seeking common" = lấy θ_l làm nền chung. Chú ý (suy luận): công thức cộng hiệu **không chia trung bình**, nên nếu các layer gần giống nhau thì $\theta_l^* \approx \theta_l + \sum_k \epsilon_k$ có thể tích luỹ lệch; mô hình vẫn hoạt động nhờ các hiệu nhỏ.

3.2 Thuật toán (Algorithm 1)

```
Input: model M, C = số layer gộp mỗi lần, [L,H] = vùng layer, I = khoảng cách tối thiểu
↪  giữa hai lần gộp,
       D = vài câu calibration, T = ngưỡng cosine
M* <- M ; l <- H - C
while l >= L:
    K <- min(C-1, số layer của M* - l)
    Mtmp <- RDSC_merge(M*, l, K)          # gộp K layer sau l vào l, xoá K layer đó
    s <- mean over D of cos( out_last(Mtmp), out_last(M) )   # tương tự biểu diễn layer
    ↪  cuối
    if s > T:  M* <- Mtmp ; l <- l - I     # chấp nhận, nhảy lên I layer
               if l > số layer của M*: l <- số layer của M* - C
    else:      l <- l - 1                  # từ chối, thử vị trí kề trên
return M*
```

Điểm khác ShortGPT/SLEB: ngưỡng T thay cho "số layer cần bỏ" nên **không điều khiển trực tiếp tỉ lệ cắt** (tác giả nêu đây là hạn chế). Độ phức tạp tệ nhất $O(H \times \|D\|)$ forward; Llama2-13B 40 layer, 10 câu: tối đa 400 lần xử lý câu, vài phút trên 1 GPU.

Hyperparameter bảng 9 (L, H, I, C, T) cho các tỉ lệ khác nhau, ví dụ Llama2-7B 27.1%/23 layer: C=4, L=1, H=32, I=2, T=0.65. Calibration: 10 câu English Wikipedia cho Llama2; 5 câu En + 5 câu Zh cho Baichuan2.



Không có số lớp đích: tỉ lệ cắt phụ thuộc ngưỡng T (giảm T thì cắt nhiều hơn).

Khác xoá thuần ($m = 1$): lớp l mang thêm các hiệu tham số của các lớp bị bỏ.

Hình 61: RDSC layer merge của LaCo: cộng hiệu tham số của m lớp sau vào lớp l rồi xoá các lớp đó, kèm vòng chấp nhận theo ngưỡng cosine (minh hoạ, tự tính)

Cách đọc hình: mỗi lớp bị gộp đóng góp một hiệu tham số so với lớp nền l, tổng của chúng cộng vào θ_l (ô vàng). Khung bên phải là vòng lặp: chỉ nhận phép gộp khi biểu diễn lớp cuối còn giống mô hình gốc trên ngưỡng T. Vì không có số lớp đích, tỉ lệ cắt do T quyết định.

4. Thí nghiệm và kết quả chính

Mô hình: Llama2-7B/13B, Baichuan2-7B/13B (base). **Benchmark (OpenCompass):** CMNLI, HellaSwag, PIQA, CHID, WSC (PPL và GEN), CSQA, BoolQ, MMLU, CMMLU, Race-H/M, XSum, C3; không train sau cắt. **Baseline:** LLM-Pruner, SliceGPT, tỉ lệ cắt bằng hoặc hơi thấp hơn LaCo. "Dense*" = bản tái lập của tác giả.

Table 2 (điểm trung bình và % so với Dense*):

Mô hình	Phương pháp	Avg.	Per.
Llama2-7B	Dense*	46.55	100%
	LLM-Pruner (27.0%)	32.36	67.79%
	SliceGPT (26.4%)	31.87	67.37%
	LaCo (27.1%, 23/32 layer)	37.46	80.28%
Llama2-13B	Dense*	55.50	100%
	LaCo (24.6%, 30/40 layer)	47.55	85.21%
Baichuan2-7B	Dense*	56.52	100%
	LaCo (24.2%, 23/32)	41.79	73.26%
Baichuan2-13B	Dense*	60.70	100%
	LaCo (24.7%, 30/40)	53.94	87.94%

LLM-Pruner/SliceGPT trên Baichuan2 chỉ còn 40–70%. Ở các benchmark sinh (CHID, XSum, WSC-GEN) các baseline hay cho 0.00 (suy sụp, lặp từ vô nghĩa, Table 23), còn LaCo ổn định hơn (không có benchmark nào dưới 30% so với Dense*, Appendix D.4).

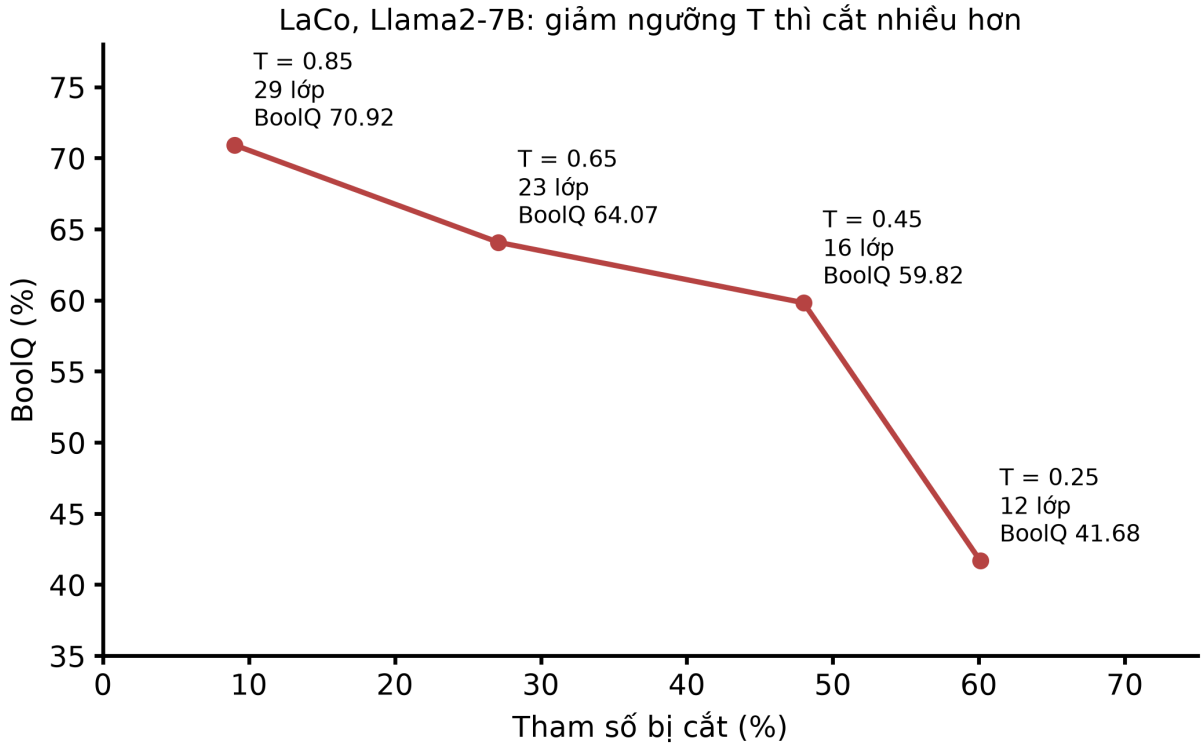
Perplexity (Table 3, Llama2-7B 27% trên 500 câu Wikipedia, 512 token): Dense 4.46; LaCo 13.93; LLM-Pruner 17.30; SliceGPT 14.51. **Thời gian cắt (Table 4):** LaCo 14.7 s, LLM-Pruner 15.9 s, SliceGPT 313 s. **Bộ nhớ/tốc độ Llama2-13B (Table 5):** bộ nhớ LaCo 19422 MB so với Dense 25902 MB; tốc độ LaCo 38.65 tok/s so với Dense 29.98 (LLM-Pruner 27.15, chậm hơn Dense; SliceGPT 35.16).

Post-training (4.1): Llama2-7B đã cắt (~5B tham số), huấn luyện toàn tham số trên ~1.0B token tiếng Anh (4 A100, ~28 giờ): loss hội tụ ~1.6 (so với 1.75 báo cáo của Llama2-7B); Baichuan2-7B ~1.25B token, loss ~2.0 (so với 1.9). Điểm trung bình (Table 6): Llama2-7B 37.46 -> 40.33 sau post-train. Baichuan2-7B: 41.79 -> 40.46 (kết quả lẫn lộn, tác giả giả thuyết do phân phối dữ liệu post-train khác dữ liệu pretrain gốc). So sánh với LLM-Pruner trên cùng dữ liệu post-train (Appendix D.3): LaCo tốt hơn và tốn ít tài nguyên hơn (chưa trích số). **Re-pruning:** cắt tiếp mô hình đã post-train xuống 17 layer (giữ 55% tham số): avg 32.40, nhưng tác giả nói giữ ~70% hiệu năng gốc (32.40/46.55 = 69.6%).

5. Phân tích / ablation

- **Tỉ lệ cắt (Table 7, Llama2-7B):** 12.0%/28 layer Avg 36.13; 27.1%/23 Avg 37.46; 45.0%/17 Avg 30.00. Với 13B: 14.6%/34 Avg 53.89; 24.7%/30 Avg 47.55; 49.7%/20 Avg 38.27. Hai điểm đầu của 7B có Avg gần nhau (36.13 so với 37.46, thứ tự không đơn điệu; paper nói hiệu năng "không giảm đáng kể" từ ~10–15% tới ~25%). Ở ~50% mô hình giữ khoảng 70% hiệu năng (theo lời tác giả), không sập.
- **Ngưỡng T (Table 13, Llama2-7B):** T=0.85 -> 9.0%/29 layer (BoolQ 70.92); T=0.65 -> 27.1%/23 (64.07); T=0.45 -> 48.0%/16 (59.82); T=0.25 -> 60.1%/12 (41.68).
- **C (Table 10):** C nhỏ (1–2) hại: BoolQ 52.57 (C=1) và 63.12 (C=2); C=3..6 tương đương (63.98–64.15); khuyến nghị C trong 4–7. C quá lớn làm cosine rơi quá nhanh dưới ngưỡng, dừng sớm.
- **Drop vs Merge (Table 15):** m=1 (chỉ xoá, không gộp) BoolQ 52.57, PIQA 68.17, HellaSwag 48.61 so với gộp 64.07, 69.80, 55.69. Tức là gộp tốt hơn xoá thuần trong thiết lập của họ.
- **Interval I (B.2):** I trong 1–3 ổn, I=4 tệ vì đẩy việc gộp về phía đầu mạng; tác giả chọn I=2. Quan sát chung: sửa tham số của layer gần đầu vào (layer 1–12) làm giảm hiệu năng.
- **Dữ liệu calibration (Table 12):** ba lần chọn 10 câu khác nhau cho kết quả gần nhau (BoolQ 63.70–64.50).
- **Metric tương tự (Table 14):** KL và CKA gần như hằng số (0.00 và 1.00) nên không phân biệt được, chọn cosine.

- **Iterative vs rule-based (Table 16):** quy tắc cố định (gộp 29–32, 21–24, 13–16) kém LaCo (Llama2-7B BoolQ 63.49 vs 64.07; Baichuan2-7B 52.94 vs 56.15) nhưng vẫn hơn các baseline nhiều benchmark.



Hình 62: LaCo trên Llama2-7B: ngưỡng T, tỉ lệ tham số bị cắt, số lớp còn lại và BoolQ, theo Table 13 của ghi chú (vẽ lại từ số liệu trong ghi chú)

Cách đọc hình: mỗi chấm là một ngưỡng T, ghi kèm số lớp còn lại và điểm BoolQ. Hạ T từ 0,85 xuống 0,25 làm tỉ lệ cắt tăng từ 9,0% lên 60,1% và BoolQ giảm từ 70,92 xuống 41,68. Điểm BoolQ rơi nhanh ở đoạn cuối, khi chỉ còn 12 lớp.

6. Giới hạn

Do tác giả nêu: không điều khiển trực tiếp tỉ lệ cắt; phải chỉnh T và các hyperparameter; động cơ mượn từ model merging nhưng **chưa có chứng minh lý thuyết**; có thể có cách gộp tốt hơn; dữ liệu post-training hạn chế nên Baichuan2 không cải thiện.

Nhận xét:

- Công thức cộng-hiệu hợp lý nhờ các layer lân cận có tham số gần nhau, điều không đảm bảo ở kiến trúc khác (chẳng hạn, các lớp khác loại có thể có hình dạng tham số không tương thích).
- Kết quả "gộp tốt hơn xoá" (Table 15) chỉ xuất hiện ở một cấu hình (Llama2-7B, ba benchmark); ShortGPT bảng 2 cho thấy xoá theo BI còn tốt hơn LaCo (Per. 86.31 vs 80.39 trên Llama2-7B), nên "gộp" không phải lợi thế tuyệt đối.
- Chuẩn cosine trên biểu diễn layer cuối với vài câu: có thể không nhạy với lỗi trên đuôi phân phối.
- Không đánh giá bằng tác vụ sinh dài có cấu trúc (như OCR).

7. Liên hệ với các bài khác

- **31 ShortGPT:** cùng benchmark; ShortGPT (xoá theo BI) hơn LaCo trên 4/4 mô hình ở bảng 2 của ShortGPT.

- **32 Deeper layers:** đề xuất healing; LaCo thay healing bằng một bước gộp tham số "miễn phí" nhưng tuỳ ý.
- **34 SLEB:** chọn lặp bằng PPL thật; LaCo cũng lặp nhưng với tiêu chí biểu diễn cuối.
- **36 ReplaceMe:** cũng bù cho phần bị xoá, nhưng bằng một ma trận tuyến tính ước lượng từ dữ liệu (không phải cộng tham số).
- **37:** nếu repair tuyến tính đổi optimum thì "gộp" của LaCo là một dạng repair tham số-khoảng-cách, không tối ưu theo dữ liệu.
- **09 Linear mode connectivity:** gợi ý lý do cộng/ghép tham số có thể hoạt động khi các layer "nằm cùng một thung lũng"; đây là liên hệ được nêu thêm, paper không viện dẫn.

8. Gợi ý áp dụng

- Điều đáng học: (a) dùng **ngưỡng tương tự biểu diễn** để dừng, (b) khái niệm "bù" khi bỏ layer.

9. Thí nghiệm gợi ý

2. So ba cách cùng bỏ 2 block liền nhau: xoá thuần, RDSC (m=1,2), và ánh xạ tuyến tính tối ưu (bài 36), đo CER trước khi distill.
3. Ablation C/T: quét T để đạt các tỉ lệ cắt xấp xỉ nhau, vẽ CER theo tỉ lệ.

10. Tóm tắt công thức / khái niệm

Khái niệm	Ý nghĩa
RDSC merge	$\theta_l^* = \theta_l + \sum_{k=1}^{m}(\theta_{l+k} - \theta_l)$ rồi xoá m layer sau
C	số layer gộp mỗi bước
I	khoảng cách tối thiểu giữa hai lần gộp
T	ngưỡng cosine biểu diễn layer cuối so với mô hình gốc
Drop (m=1)	chỉ xoá, kém hơn merge trong ablation

11. Kết luận ngắn

LaCo là phương pháp depth pruning đơn giản, nhanh (14.7 s), không huấn luyện, giữ nguyên cấu trúc; giữ ~73–88% hiệu năng ở ~25% cắt, hơn LLM-Pruner/SliceGPT trong benchmark của họ. Tuy nhiên chưa có lý thuyết, không điều khiển được tỉ lệ cắt, và (theo ShortGPT) bị xoá-theo-BI vượt qua.

Link gốc: https://arxiv.org/abs/2402.11187

Bài 36

ReplaceMe: Network Simplification via Depth Pruning and Transformer Block Linearization (2025)

Tác giả: Dmitriy Shopkhoev, Ammar Ali, Magauiya Zhussip, Valentin Malykh, Stamatios Lefkimmiatis, Nikos Komodakis, Sergey Zagoruyko (MWS AI, ITMO University, University of Crete, Polynome...).
Venue / arXiv: arXiv:2505.02819 (v4, 19/02/2026); bản text ghi "accepted and presented at NeurIPS 2025".
Nguồn: https://arxiv.org/abs/2505.02819
Chủ đề: Depth pruning không train: thay một khối layer liền kề bằng **một ma trận tuyến tính** ước lượng từ dữ liệu calibration, rồi gộp ma trận này vào trọng số `down_proj` của MLP, nên không thêm tham số.
Vị trí trong lộ trình: Đọc trước: 32 Deeper layers (đây là "UIDL" trong bài này, baseline chính), 31 ShortGPT, 35 LaCo. Đọc tiếp: 37 (cho thấy repair tuyến tính này đối *layer nào nên cắt*), 39 Minitron.
Chương nền liên quan: nền 06: Cắt lớp và độ sâu trong LLM

Tài liệu học tập tiếng Việt, ghi chú diễn giải, không phải bản dịch paper. Số liệu lấy từ paper; bảng đã đọc kỹ.

1. Bối cảnh

- Các bài depth pruning trước (31, 32, 34) cắt xong thường cần **healing** (fine-tune, thường LoRA) để hồi phục; LLM-Streamline thay khối bị cắt bằng một mạng nhẹ (một lớp transformer hoặc FFN) rồi huấn luyện với MSE + LLM loss. Tốn thời gian và tài nguyên.
- **Least squares (bình phương tối thiểu):** cho A và B, tìm T làm $\|AT - B\|_2^2$ nhỏ nhất; nghiệm đóng $T = (A^\top A)^{-1} A^\top B$. Không cần gradient descent.
- **Calibration set:** vài nghìn mẫu không nhãn dùng để lấy hidden state, không để huấn luyện theo nghĩa thông thường.
- Quan sát từ công trình khác (do bài này trích dẫn): nhiều transformer block gần như tuyến tính trong ánh xạ embedding giữa các lớp.

2. Ý tưởng chính

Giả thuyết của tác giả: **một khối liền kề các transformer block có thể xấp xỉ bằng một phép biến đổi tuyến tính**. Cắt n block sau block i^*, rồi tìm ma trận T sao cho "đầu ra attention của block i^* cộng với MLP-output của nó nhân T" khớp với "đầu ra của block $i^* + n$ trước khi cắt". T ước lượng trên calibration bằng nghiệm đóng (L2) hoặc tối ưu số (cosine). Vì T nhân liền với `down_proj` của MLP nên có thể gộp vào, mô hình sau cắt có cấu trúc y như Transformer thường. Kết quả: cắt 25% mà không huấn luyện vẫn giữ ~90% hiệu năng.

3. Phương pháp chi tiết

3.1 Ký hiệu một block (Eq. 1–3)

$X_i \in \mathbb{R}^{N \times d}$ là đầu vào block i (N token, d chiều).

$$Y_i = X_i + \mathrm{MHA}_i(\mathrm{LN}_i^{(1)}(X_i)), \qquad M_i = \mathrm{MLP}_i(\mathrm{LN}_i^{(2)}(Y_i)), \qquad L_i = Y_i + M_i$$

Y_i = sau attention (đã cộng residual); M_i = đầu ra MLP; L_i = đầu ra block.

3.2 Chọn khối cắt (Eq. 4)

$$i^* = \arg\min_i \; h(L_i, L_{i+n})$$

h là khoảng cách trung bình trên calibration. Cosine distance cho kết quả tốt nhất (ablation brute-force ở A.10 xác nhận cosine chọn khối tối ưu hoặc gần tối ưu).

3.3 Ước lượng T (Eq. 5–8)

$$\boxed{T^* = \arg\min_T \; h(M_{i^*}T + Y_{i^*},\; L_{i^*+n})}$$

Sau đó xoá block $i^*+1, \ldots, i^*+n$. T nhân lên M_{i^*} (đầu ra MLP của block cuối cùng còn giữ), còn Y_{i^*} vẫn là đường residual.

- **Dạng L2 (nghiệm đóng):**

$$T^* = \arg\min_T \|M_i T + Y_i - L_{i+n}\|_2^2 = (M_i^\top M_i)^{-1} M_i^\top (L_{i+n} - Y_i)$$

- **Dạng cosine** (không có nghiệm đóng, tối ưu bằng Adam):

$$T^* = \arg\min_T \sum_{k=1}^{N} \left(1 - \frac{(M_{i,k}T + Y_{i,k})^\top L_{i+n,k}}{\|M_{i,k}T + Y_{i,k}\|\,\|L_{i+n,k}\|}\right)$$

- **Dạng tiết kiệm bộ nhớ (Eq. 8):** $\arg\min_T \cos(M_i T,\; L_{i+n} - Y_i)$ chỉ cần lưu M_i và $L_{i+n} - Y_i$; A.11 báo ảnh hưởng không đáng kể.
- **Regularization (Eq. 9):** thêm $\alpha R(T)$ với L1/L2. Quan sát: tăng chính xác benchmark nhưng tăng perplexity (trade-off).
- **Gộp (fuse):** vì T và `down_proj` là hai phép tuyến tính liên tiếp, $W'_{down} = W_{down}T$ (theo mô tả: ghép đại số vào ma trận `down_proj` của MLP).
- **Nhiều ma trận (Multi-LT, mục 2.4):** chia các block bị bỏ thành nhiều nhóm không chồng, mỗi nhóm một T; nhóm kề nhau thì gộp. Biến thể "Multi_LT_NC" ép các nhóm không liên tiếp.

```
ReplaceMe
Input: model, calibration set D (vd. 8k mẫu SlimOrca), số block n cần bỏ
1. chạy model trên D, lưu L_i cho mọi i
2. i* = argmin_i cosine_dist(L_i, L_{i+n})              # chọn khối
3. lưu M_{i*}, Y_{i*}, L_{i*+n} trên D
4. T = lstsq(M_{i*}, L_{i*+n} - Y_{i*})                  # L2: nghiệm đóng
   hoặc: T = Adam(cos loss), khởi tạo T = I, lr 1e-4, batch 1024, 10 epoch
5. W_down[i*] <- W_down[i*] @ T                          # gộp vào MLP, 0 tham số mới
6. xoá block i*+1 .. i*+n
```

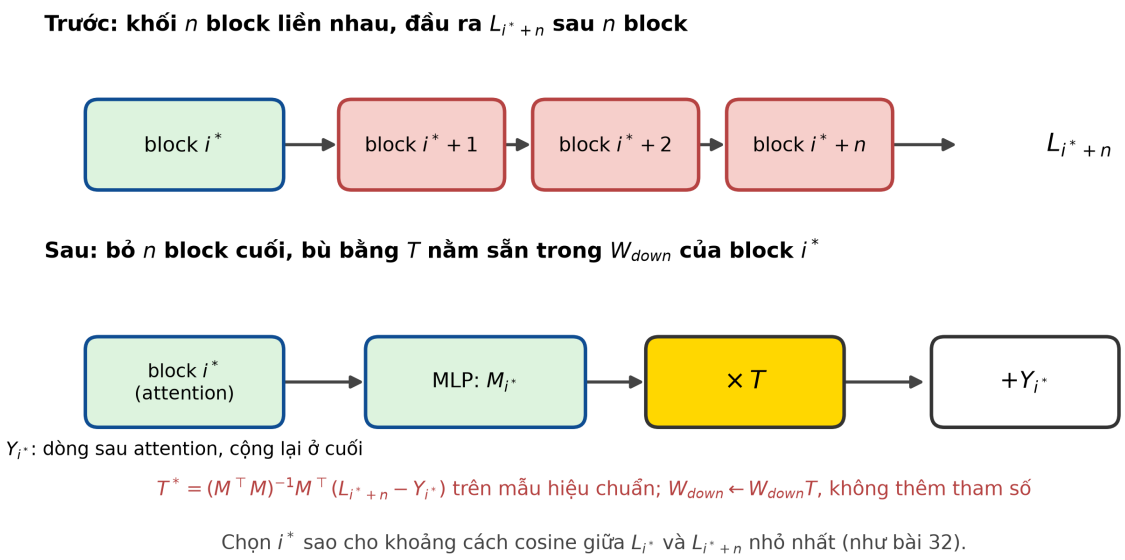
Lưu ý: T là ánh xạ $d \times d$ từ đầu ra MLP của block cuối được giữ lại; nó không trực tiếp ánh xạ đầu vào X_{i^*+1} của khối bị cắt sang đầu ra L_{i^*+n}.

Chi tiết cấu hình (A.15): Adam, LR 10^{-4}, batch 1024, 10 epoch, loss cosine, khởi tạo T là ma trận đơn vị, không bias. Chạy 1 GPU A100 40GB (có thử cả P100 trên Kaggle).

Cách đọc hình: hàng trên là khối n block cần cắt với đầu ra L_{i^*+n}; hàng dưới là block còn lại, trong đó đầu ra MLP được nhân thêm T rồi cộng lại với Y. T khớp phần đầu ra mà các block bị bỏ đã thêm vào, nên có nghiệm đóng. Vì T nhân liền với `down_proj`, kết quả không thêm tham số.

4. Thí nghiệm và kết quả chính

Mô hình: Llama-2-7B, Llama-3-8B-Instruct (chính); Qwen2.5-7B, Falcon-11B, Mistral-7B, Llama-3.2-1B, Llama-3-70B; CLIP ViT-L/14 (vision encoder). **Benchmark:** CMNLI, HellaSwag, PIQA, CHID, WSC, MMLU, CMMLU, Race-H/M, C3 (theo LLM-Streamline); thêm Winogrande, BoolQ, OpenBookQA, SciQ, Lambada-OpenAI (perplexity). RP = % so với baseline.



Hình 63: ReplaceMe: bỏ n block rồi dùng ma trận T ước lượng bằng bình phương tối thiểu, gộp vào down_proj của block còn lại (minh hoạ, tự tính)

Table 1 (Llama-2-7B, cắt 25%; baseline Avg 45.3):

Phương pháp	Train-free	AVG	RP
LLM-Streamline*	không	41.6	92.0%
LLM-Pruner*	không	35.4	78.2%
SliceGPT*	không	35.1	77.5%
LaCo*	không	37.4	82.7%
UIDL*	không	40.9	90.3%
ReplaceMe (Cosine)	**có**	**41.9**	**92.5%**
ReplaceMe (LS)	có	40.7	89.9%

(* = số lấy từ paper LLM-Streamline; các baseline đều có bước train/heal, ReplaceMe thì không.) Ghi chú: LLM-Streamline đạt RP 92.0% với train, ReplaceMe cosine 92.5% không train, tức hơn nhẹ; hơn UIDL 2.2 điểm RP.

Table 2 (Llama-3-8B-Instruct, cắt 25%, không heal; Avg-acc trên Race, Winogrande, PIQA, BoolQ, OBQA, SciQ):

Phương pháp	Linear transform	PPL Lambada	Avg-acc	RP
Baseline		3.11	0.7	100%
SVD-LLM	không	29.90	0.59	85.3%
LLM-Pruner	không	12.31	0.60	85.3%
UIDL	Identity	2216.96	0.58	82.5%
ReplaceMe	LS	20.23	0.63	89.9%
ReplaceMe	Cosine	15.88	0.63	90.9%
ReplaceMe	Multi_LT_NC (Cosine)	13.95	0.63	90.0%

Mức nén cao (Table 3, avg acc, Llama 3 8B Instruct): UIDL: 59.9, 50.4, 41, 30.4; ReplaceMe: 64.6, 60.7, 51, 38.6 (nén 12.5%, 25%, 37.5%, 50%). Tác giả thừa nhận ở mức nén cao vẫn cần healing.

Quy mô (Table 9): Llama-3.2-1B-Instruct cắt 25%: RP 0.8771 (0.6098 -> 0.5348); Llama-3.1-8B-Instruct 25%: RP 0.9184; Llama-3-70B-Instruct 25%: RP 0.9664, 37.5%: RP 0.9060. Xu hướng: mô hình càng lớn càng nén được nhiều ở cùng RP.

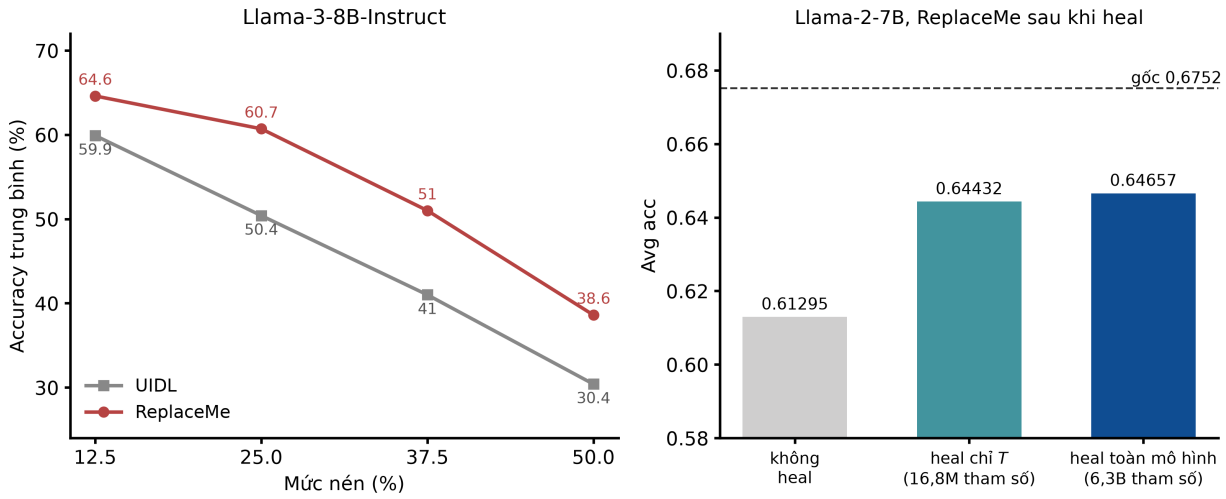
Chi phí so với UIDL (Fig. 2/4): ReplaceMe nhanh hơn và ít phát thải hơn UIDL có LoRA healing (các tỉ số trong hình: 169x, 107x, 6.6x, 3.5x...; hình bị lẫn, không trích chắc chắn từng cột).

ViT (CLIP-L/14, Table 6, ReplaceMe LS): nén 13%: MS-COCO text recall@5 0.767 (gốc 0.794, UIDL 0.745), Cifar10 acc1 0.939 (gốc 0.956, UIDL 0.927), VTAB/EuroSAT acc1 0.552 (gốc 0.625, UIDL 0.490). Nén 25%: text recall@5 0.556 vs UIDL 0.515; Cifar10 acc1 0.780 vs 0.693; VTAB/EuroSAT acc1 0.395 vs 0.381. Calibration 8000 mẫu MIMIC. Tức: tốt hơn UIDL, giữ khá ở 13%, tụt ở 25%.

5. Phân tích / ablation

- **Dữ liệu calibration (Table 4, Llama-3-8B-Instruct, 25%):** FineWeb 8k (plain text): LS Avg-acc 0.56, PPL 26.74, RP 80.47%; SlimOrca 8k (instruction): LS 0.62, PPL 21.21, RP 89.59%; Cosine SlimOrca: 0.63, PPL 15.90, RP 90.67%; “orca_generated” (đầu ra do chính mô hình sinh): PPL thấp nhất (13.24–13.58) nhưng Avg-acc thấp hơn (0.61). Trộn đa ngôn ngữ (66 ngôn ngữ) ngang SlimOrca. Kết luận tác giả: dữ liệu instruction chất lượng cao quan trọng nhất; dữ liệu sinh tự thân cho PPL tốt nhưng benchmark kém hơn.
- **Calibration đúng tác vụ (A.17, Table 17, SciQ, Llama3-8B-Instruct 25%):** gốc 0.93; ReplaceMe với calib SciQ 0.89; calib Orca 0.858; UIDL 0.687.
- **Cấu trúc của T (Table 7):** Generic Avg 0.62, PPL 21.21, RP 89.59; Diagonal 0.62, PPL 89.09, RP 88.42; Orthonormal 0.60, PPL 700.57, RP 85.67. Tác giả suy ra “co giãn (scaling) quan trọng hơn quay/phản xạ” để hồi phục.
- **Regularization (Table 5):** L2 với LS gần như không đổi; cosine + L1 (10^{-4}) tăng Avg-acc 0.634 -> 0.638 nhưng PPL 15.9 -> 22.1.
- **Solver (Table 8):** Adam, NLCG, L-BFGS cho kết quả gần nhau (Avg 0.634/0.630/0.634); Trust-NCG kẹt cực tiểu cục bộ (0.620, PPL 47.0). MSE+Adam gần như trùng LS.
- **Độ ổn định (A.5):** 8 layer bị cắt trên Llama-3-8B-Instruct: Avg-acc trung bình 60.71 (độ lệch chuẩn 0.08), PPL 15.90 (std 0.05).
- **Số ma trận LT (Table 16, tổng 8 layer cắt):** Llama-3-8B-Instruct: 1 LT 0.6244 / PPL 21.2; 2 LT 0.6296 / 18.99; 4 LT 0.6245 / 16.07; 8 LT 0.6092 / 37.98. Mistral-7B: 1 LT 0.6545; 2 LT 0.6227; 4 LT 0.6568; 8 LT 0.6510. Kết luận tác giả: 2–4 LT là cân bằng, khác biệt nhỏ.
- **LT gộp vs LT là block riêng (Table 10):** gộp vào MLP: Avg 0.634, PPL 15.875; block riêng: 0.640, PPL 13.481.
- **Healing (Table 13, Llama-2-7B):** gốc Avg 0.6752 / PPL 3.3973; ReplaceMe không heal 0.61295 / 22.3586; heal chỉ T (16.8M tham số) 0.64432 / 5.3965; heal toàn mô hình (6.3B tham số) 0.64657 / 4.9138. Tức heal chỉ T gần bằng full fine-tune với chi phí rất nhỏ.
- **Chọn khối (A.10, Fig. 5–6):** cosine distance nhỏ tương quan với accuracy cao; L2 distance kém dự báo; CKA và CCA cũng xét.

Cách đọc hình: bên trái, ReplaceMe cao hơn UIDL ở cả bốn mức nén, khoảng cách từ 4,7 điểm ở 12,5% lên 10,3 điểm ở 25%. Bên phải, heal chỉ ma trận T (16,8M tham số) đạt 0,64432, gần heal toàn mô hình (0,64657) và còn thấp hơn mô hình gốc 0,6752. Hai bảng thuộc hai mô hình khác nhau nên không đọc chéo.



Hình 64: Trái: accuracy trung bình của UIDL và ReplaceMe theo mức nén trên Llama-3-8B-Instruct; phải: ReplaceMe trên Llama-2-7B sau khi heal, theo Table 3 và Table 13 của ghi chú (vẽ lại từ số liệu trong ghi chú)

6. Giới hạn

Do tác giả nêu: hiệu quả chỉ rõ trong dải nén "hợp lý" phụ thuộc kích thước và kiến trúc; nén cao cần retrain hoặc ít nhất chỉnh T (healing chỉ-LT là lối thoát rẻ).

Nhận xét:

- Hầu hết đánh giá là benchmark trắc nghiệm và perplexity Lambada; không có tác vụ sinh dài. PPL Lambada tăng từ 3.11 lên 15.88–20.23 ở 25% cho thấy mô hình ngôn ngữ vẫn xấu đi nhiều dù accuracy trắc nghiệm giữ ~90%.
- Chọn khối **không xét repair** (Eq. 4 dùng khoảng cách trước khi có T); bài 37 chỉ ra sai khác này có thể lớn.
- Chỉ một khối liền kề (hoặc vài nhóm); không có tìm kiếm tổ hợp.
- Chỉ Transformer chuẩn; T nhân lên M_i, đòi hỏi block có MLP với down_proj để gộp.
- Số liệu Table 1 của đối thủ lấy từ paper khác (dấu *), không chạy lại.

7. Liên hệ với các bài khác

- **32 Deeper layers (UIDL):** baseline chính. UIDL = chọn khối bằng khoảng cách góc, nối thẳng (Identity), rồi heal LoRA. ReplaceMe thay "nối thẳng + LoRA" bằng "T tối ưu + không train", và ablation cho thấy "Linear transform: Identity" của UIDL có PPL 2216.96.
- **31 ShortGPT, 35 LaCo:** baseline đã có trong Table 1 (LaCo 82.7%).
- **34 SLEB:** greedy chọn từng block bằng NLL; ReplaceMe chọn khối bằng cosine rồi sửa bằng T.
- **37:** xét đúng điều bài này bỏ qua: nếu biết bước repair là T thì chọn khối thế nào.
- **29 SliceGPT:** cũng dùng phép biến đổi tuyến tính (phép chiếu PCA) và gộp vào trọng số; điểm chung là "sửa bằng phép tuyến tính được gộp".
- **39 Minitron:** phương pháp dùng 100B token để hồi phục (theo mô tả của bài này), đối lập với training-free.

8. Gợi ý áp dụng

- Bạn có thể báo cáo hai số này cho từng ứng viên khối cắt.

9. Thí nghiệm gợi ý

2. So ba cách nối cho cùng khối: Identity, LS-T, cosine-T; đo CER trước distill. ## 10. Tóm tắt công thức / khái niệm

Khái niệm	Ý nghĩa
$i^* = \arg\min_i h(L_i, L_{i+n})$	chọn khối n block có đầu ra gần đầu vào nhất
$T^* = (M^\top M)^{-1} M^\top (L_{i+n} - Y)$	nghiệm LS đóng cho ánh xạ tuyến tính thay khối
Cosine objective	tối ưu số bằng Adam, tốt hơn LS (RP 90.9% vs 89.9% trên Llama-3-8B)
Fuse	$W_{down} \leftarrow W_{down} T$, không thêm tham số
Multi-LT	nhiều nhóm block, mỗi nhóm một T
RP	% hiệu năng so với baseline

11. Kết luận ngắn

Trong các thí nghiệm của ReplaceMe, một ma trận tuyến tính ước lượng từ vài nghìn mẫu có thể bù phần lớn tổn thất khi bỏ 25% layer, không cần train; với nén cao hơn vẫn cần healing nhưng chỉ cần heal T. Điểm yếu: chọn khối không xét tới bước sửa và đánh giá thiên về trắc nghiệm.

Link gốc: https://arxiv.org/abs/2505.02819

Bài 37

Repair Changes Which Layers to Remove: Exact Optima for Depth Pruning of Language Models (2026)

Tác giả: Jianchang Su, Dawei Xiang, Yifan Zhang, Wei Zhang (University of Connecticut).
Venue / arXiv: arXiv:2610.30552 (v1, 22/10/2026, phân loại cs.DC). Bản text không ghi hội nghị; xem như preprint.
Nguồn: https://arxiv.org/abs/2610.30552
Chủ đề: Depth pruning kèm **repair tuyến tính** (ma trận least-squares kiểu ReplaceMe): liệt kê chính xác mọi tập layer có thể cắt ở độ chi tiết thô, cho thấy tập tối ưu **thay đổi theo repair**; đề xuất chọn layer *dưới repair sẽ dùng* (R-GS) và một điểm số bậc hai (R-FS).
Vị trí trong lộ trình: Đọc trước: 36 ReplaceMe (bắt buộc: bài này dùng nguyên repair của nó), 34 SLEB (R-GS là SLEB + repair), 31 ShortGPT, 32 Deeper layers. Đọc tiếp: 39 Minitron, 43 Pruning vs scratch (về hồi phục bằng training).
Chương nền liên quan: nền 05: Cắt rồi huấn luyện lại, nền 06: Cắt lớp và độ sâu trong LLM, nền 07: Đo đúng một kết quả pruning

> Tài liệu học tập tiếng Việt, ghi chú diễn giải, không phải bản dịch paper. Phần "Điều gì được đo" liệt kê chính xác đại lượng và thiết lập của paper. Số liệu lấy từ paper. Paper này dùng tên mô hình "Qwen3.8-27B" như trong văn bản; tên được giữ nguyên.

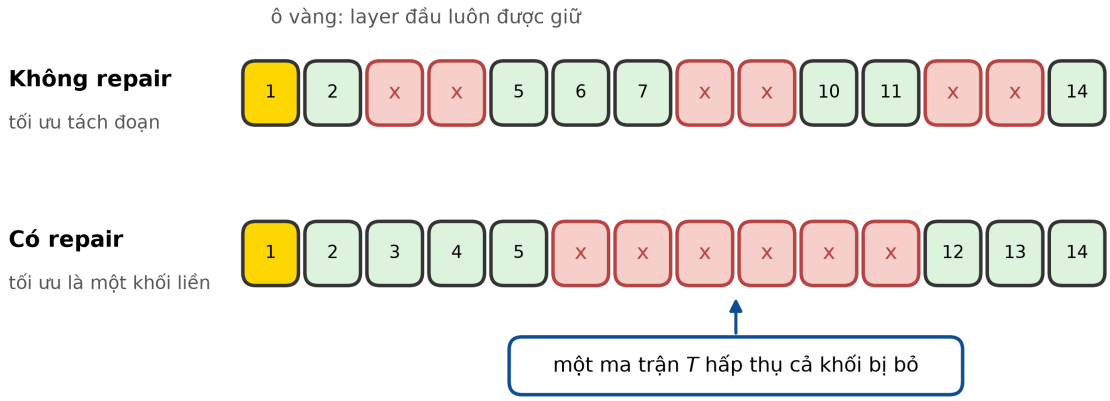
1. Bối cảnh

- **Depth pruning** bỏ nguyên layer. Với L layer và bỏ n layer có $\binom{L}{n}$ cách chọn; chấm điểm mỗi cách đòi chạy mô hình đã cắt, nên các bài trước dùng **proxy**: độ giống giữa vào/ra layer (ShortGPT, bài 31), khoảng cách góc của một khối liền (bài 32, ReplaceMe), tham lam theo perplexity (SLEB, bài 34), tìm kiếm tiến hoá (EvoPress), ước lượng Shapley...
- **Repair tuyến tính (bài 36):** thay một "run" các lớp bị bỏ bằng ma trận $T \in \mathbb{R}^{d\times d}$ khớp bằng bình phương tối thiểu trên activation calibration, gộp vào `down_proj` của lớp được giữ ngay trước đó; số tham số bằng mô hình đã cắt.
- **Khoảng cách của proxy tới tối ưu thật chưa từng được biết**, vì chưa ai tính tối ưu thật.
- **Möbius transform / phân rã tương tác:** từ hàm tập $c(S)$ (tăng loss khi bỏ tập S) tách ra phần riêng từng phần tử và phần tương tác của mọi tập con; bỏ là "cộng tính" khi mọi tương tác bằng 0. **Super-additive** (siêu cộng tính) ở đây: bỏ hai layer gần nhau tốn hơn tổng chi phí từng layer.

2. Ý tưởng chính

Bài báo gồm ba đóng góp. (1) Ở độ chi tiết "nhóm 2 layer" đủ nhỏ để **liệt kê hết** (khoảng 53.000 mô hình đã cắt trên 4 họ mô hình, có và không có repair), nên biết tối ưu chính xác. (2) Phát hiện: không repair thì tối ưu *tách thành 2–3 đoạn rời* (vì bỏ hai layer gần nhau đắt hơn tổng), còn có repair thì tối ưu *thành một khối liền* (vì một ma trận T hấp thụ tương tác trong đoạn). Hệ quả: các phương pháp chọn layer **không tính tới repair** bỏ lỡ tối ưu có repair tới 189%. (3) Sửa: chọn layer **dưới đúng repair sẽ dùng** (tham lam với repair: R-GS, chi phí ngang SLEB), hoặc dùng điểm bậc hai tính từ mô hình nguyên (R-FS, một forward + một backward).

Cách đọc hình: hàng trên là dạng tối ưu khi không có repair, hàng dưới là dạng tối ưu khi có repair. Ô đỏ là nhóm bị bỏ, ô vàng là layer đầu luôn được giữ. Vị trí các ô chỉ để minh hoạ khái niệm, không phải nghiệm của một mô hình cụ thể.



Ô đỏ có dấu x: nhóm bị bỏ; số trong ô là chỉ số nhóm layer. Vị trí là ví dụ minh hoạ, không phải kết quả của một mô hình cụ thể.

Hình 65: Cùng ngân sách bỏ 6 trong 14 nhóm layer: không repair thì tối ưu tách thành các đoạn rời, có repair thì thành một khối liền do một ma trận T hấp thụ (minh hoạ, tự tính)

3. Điều gì được đo (thiết lập chính xác)

Phần này trình bày các đại lượng và thiết lập được đánh giá trong bài báo.

Đối tượng cắt. Mô hình decoder-only L layer, residual stream $h_\ell = h_{\ell-1} + f_\ell(h_{\ell-1})$. Layer chia thành G **nhóm liền kề** 1 hoặc 2 layer. Một **run** = tập cực đại các nhóm bị bỏ kề nhau. Layer đầu luôn được giữ (repair của run bắt đầu từ layer đầu không thể gộp vào layer trước; mọi nhóm đầu trong các thí nghiệm liệt kê cũng là nhóm "critical").

Repair. Với run $a..b$, chỉ **một ma trận $T_{a,b}$ cho mỗi run**, tác động lên đầu ra MLP m_{a-1} của layer được giữ ngay trước run, đầu ra thành $y + m_{a-1}T$ với $y = h_{a-1} - m_{a-1}$. Ước lượng bằng ridge:

$$\boxed{T_{a,b} = \arg\min_T \|M_{a-1}T - (H_b - Y_{a-1})\|_F^2 + \lambda\|T\|_F^2} \qquad \text{(Eq. 1)}$$

hàng của M_{a-1}, Y_{a-1}, H_b là activation **của mô hình nguyên** tại các token calibration. $\lambda = 10^{-3}\times$ trung bình đường chéo của $M^\top M$ (không ridge cho cấu hình "mặc định ReplaceMe": dùng pseudo-inverse float64 vì MLP-output của Gemma trên SlimOrca bị thiếu hạng). Maps được tính **một lần** từ activation đã cache của mô hình nguyên (tại mỗi run, độc lập; đây là một xấp xỉ: mỗi map giả định phần mạng còn lại nguyên vẹn).

Hàm mục tiêu và dữ liệu.

- *Loss increase* của tập con = loss token trung bình của mô hình đã cắt trừ loss của mô hình nguyên, trên **16 chuỗi WikiText-2 calibration, mỗi chuỗi 256 token**. Mọi phương pháp chọn layer trên các chuỗi này.
- Báo cáo **perplexity (PPL)** trên 16 chuỗi *khác* của tập test WikiText-2. Maps fit trên 128 chuỗi WikiText-2 train.
- **Accuracy** = trung bình trên ARC-Easy, ARC-Challenge, HellaSwag, PIQA, WinoGrande, mỗi tác vụ 1.000 mẫu (prompt lm-evaluation-harness); một sai số chuẩn khoảng 0.7 điểm. BoolQ báo riêng vì pruning dịch prior yes/no tới 20 điểm.
- Biến thể "task objective" (R-GS-T): cross-entropy của lựa chọn đúng trên **200 mẫu train** mỗi tác vụ (tập rời với mẫu đánh giá). Biến thể MMLU: 5 câu hỏi/môn từ validation (285 câu), đánh giá trên test (50 câu/môn, 2.850 câu).

Giới hạn phép đo: bài báo không đánh giá CER hay tác vụ sinh, chỉ thử một loại repair bằng bình phương tối thiểu của ReplaceMe và không cắt neuron MLP.

Liệt kê chính xác (Table 14). Qwen3-1.7B (28 layer, 14 nhóm, mọi tập đến 6 nhóm bị bỏ; thêm độ chi tiết 1 layer, 28 nhóm, đến 3 layer); Gemma-3-4B (34 layer, 17 nhóm, đến 6); Llama-3.1-8B-Instruct (32 layer, 16 nhóm, đến 6); Qwen3-14B (40 layer, 20 nhóm, đến 4). Mỗi tập được tính trên mọi chuỗi calibration và test, không và có repair; khoảng 53.000 tập.

Đánh giá thêm (không liệt kê được): Qwen3.8-27B (64 layer, 3 layer linear-attention trên mỗi layer full-attention, không dùng cho bất kỳ quyết định thiết kế nào), bỏ 25% và 37.5% layer ở độ chi tiết 1 layer; Llama-3.1-8B, Gemma, Qwen3 ở các ngân sách tương tự. Baselines: ShortGPT, LaCo, ReplaceMe (khối cosine), Prune&Comp, SLEB; **tất cả đều được repair bằng cùng các map least-squares (+R)**, nên các phương pháp chỉ khác nhau ở *layer nào được chọn*. (LaCo: bỏ layer mà tiêu chí gộp của nó chọn, không gộp tham số.)

4. Phương pháp chi tiết

4.1 Phân rã tương tác

$c(S)$ = tăng loss khi bỏ S, $c(\varnothing) = 0$.

$$\mu(S) = \sum_{R \subseteq S} (-1)^{|S|-|R|} c(R), \qquad c(S) = \sum_{R \subseteq S} \mu(R)$$

$\mu(\{i\})$ là chi phí riêng; $\mu(\{i,j\})$ là số hạng cặp (dương = siêu cộng tính). **Tỉ lệ tương tác** = trung bình $|c(S) - \sum_{i \in S} c(\{i\})|$ chia cho trung bình $|c(S)|$.

4.2 Hai kiểu tương tác tìm thấy (mục 4)

- **Layer "critical":** nhóm mà bỏ riêng đã làm perplexity tăng hơn gấp đôi: 4/14 nhóm ở Qwen3-1.7B, 5/17 ở Gemma, 3/16 ở Llama, 3/20 ở Qwen3-14B, thường là các nhóm đầu. Số hạng cặp của chúng lớn và **âm** (vì loss bão hoà khi mô hình đã hỏng) và mọi tối ưu chính xác đều **giữ** chúng.
- **Siêu cộng tính cục bộ:** với nhóm không critical, $\mu(\{i,j\}) > 0$ và **suy giảm theo khoảng cách** $|i-j|$ với độ dài suy giảm khớp 2.9 layer (Gemma) đến 6.4 layer (Llama); gần 0 sau khoảng 10 layer. Số hạng cặp giải thích 52% đến 76% tính phi cộng tính tại 3 nhóm bị bỏ.

4.3 Thuật toán chọn dưới repair (R-GS, Algorithm 1)

```
Input: layer 1..L, ngân sách n, mục tiêu J, activation của mô hình nguyên (đã cache)
S <- {}
for k = 1..n:
    for mỗi layer i còn giữ, i != 1:
        với mỗi run a..b của S U {i}: fit T_{a,b} theo Eq.1 (từ activation đã cache)
        J_i <- J(mô hình bỏ S U {i}, đã repair)
    S <- S U { argmin_i J_i }
return S, gộp các map vào down_proj
```

Chi phí: $\sum_{k=0}^{n-1}(L-k)$ lần đánh giá (bằng SLEB). Trên Llama-3.1-8B bỏ 12/32 layer, một RTX A6000: R-GS 384 giây (319 mô hình đã cắt), SLEB+repair 343, LaCo 487, Prune&Comp 56, R-FS 33, ReplaceMe 8, ShortGPT 1; mỗi phương pháp có repair cần thêm 29 giây thu activation.

4.4 Điểm bậc hai R-FS (mục 6.2)

$z \in \mathbb{R}^{N \times d}$ là residual stream sau khối $a..b$ trong mô hình nguyên; mô hình đã repair cho $z + \delta$ với $\delta_t = y_t + m_t T_{a,b} - z_t$ (phần dư của repair tại token t); các layer sau khối không đổi.

Mệnh đề 1: gọi $p_s(z)$ là phân phối token kế tiếp, $D(\delta) = \sum_s \mathrm{KL}(p_s(z) \,\|\, p_s(z+\delta))$. Khi đó $D(0) = 0$, $\nabla D(0) = 0$ và

$$D(\delta) = \tfrac{1}{2}\,\mathrm{vec}(\delta)^\top F\,\mathrm{vec}(\delta) + O(\|\delta\|^3), \quad F = \sum_s \mathbb{E}_{y\sim p_s}\left[\nabla_z \log p_s(y) \nabla_z \log p_s(y)^\top\right]$$

(F là Fisher information của mạng phía sau theo residual stream). Thay kỳ vọng bằng token quan sát (empirical Fisher) và giữ một khối đường chéo hạng một mỗi vị trí ($F \approx \mathrm{blockdiag}_t(g_t g_t^\top)$, $g_t = \nabla_{z_t} \sum_s \ell_s$) ta được

$$\boxed{s(a,b) = \sum_{t=1}^{N} (g_t \cdot \delta_t)^2} \qquad \text{(Eq. 4, R-FS)}$$

Đọc nôm na: **phần dư sau repair, đo bằng thước loss** (hướng nào của residual stream làm loss nhạy thì phạt hướng đó nặng). Khác với ReplaceMe: LS giảm $\|\delta\|_2$ (chuẩn Euclid, mọi hướng như nhau); R-FS cân phần dư theo độ nhạy của loss. Paper nối với *balanced truncation* (Moore 1981). Chỉ cần một forward và một backward trên mô hình nguyên.

5. Thí nghiệm và kết quả chính

5.1 Cấu trúc tối ưu chính xác

Hình 1/Phần 4: Không repair, khối liền tốt nhất tệ hơn tối ưu tới 6.9 lần. Với repair của ReplaceMe, khối liền tốt nhất **bằng** tối ưu ở 9/18 ngân sách và trong vòng 5% ở 13/18.

Table 1: perplexity vượt tối ưu của khối liền tốt nhất, không repair (số nhóm bị bỏ 3/4/5/6):

Mô hình	3	4	5	6
Qwen3-1.7B	0%	28%	77%	588%
Gemma-3-4B	64%	252%	512%	447%
Llama-3.1-8B	32%	34%	76%	190%
Qwen3-14B	0%	18%	-	-

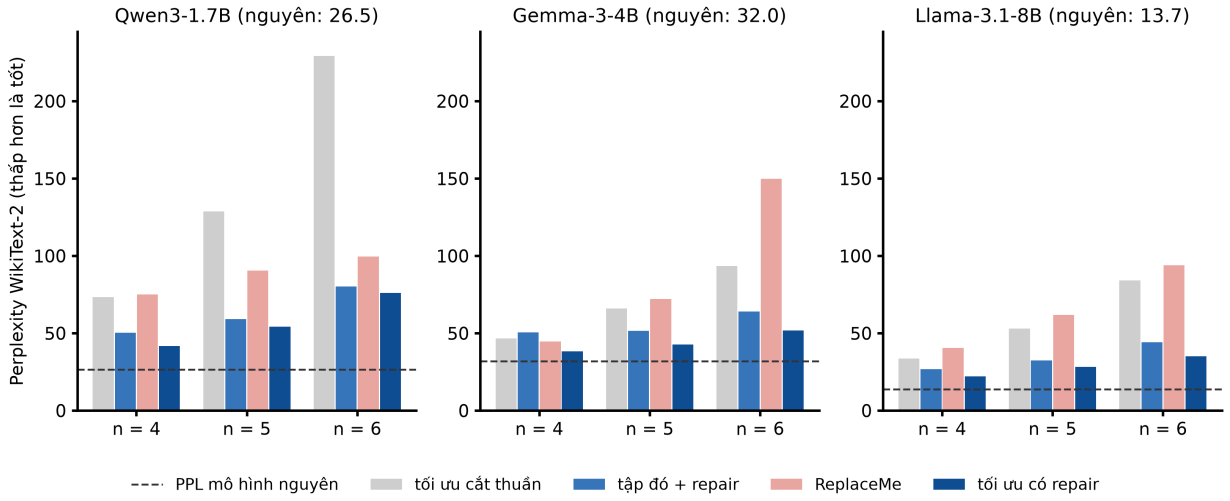
Từ 4 nhóm trở lên, mọi tối ưu chính xác ở mọi họ **tách bỏ thành 2 hoặc 3 đoạn**. Chấm một lần (one-shot) chọn các nhóm có chi phí riêng nhỏ nhất, chúng tụ lại gần nhau nên gần liền nhau, tệ hơn tối ưu 33% (Qwen3-1.7B) tới 581% (Gemma) ở 4 nhóm. SLEB (đánh giá mô hình thật) tìm được tối ưu ở **13/20** ngân sách.

Table 2: PPL ở ngân sách nặng nhất được liệt kê (n = số nhóm bị bỏ; trong ngoặc là PPL mô hình nguyên). Các cột: tối ưu của cắt thuần "as is" / chính tập đó sau repair / lựa chọn của ReplaceMe / tối ưu chính xác dưới repair.

Mô hình	n	Tối ưu cắt thuần	Tập đó + repair	ReplaceMe	Tối ưu có repair
Qwen3-1.7B (26.5)	4	73.4	50.3	75.1	41.7
	5	128.7	59.2	90.6	54.3
	6	229.2	80.2	99.5	76.0
Gemma-3-4B (32.0)	4	46.7	50.6	44.7	38.3
	5	65.9	51.6	72.2	42.8
	6	93.5	64.1	149.8	51.9

Mô hình	n	Tối ưu cắt thuần	Tập đó + repair	ReplaceMe	Tối ưu có repair
Llama-3.1-8B (13.7)	4	33.6	26.8	40.4	22.2
	5	53.0	32.5	61.9	28.3
	6	84.2	44.1	94.0	35.1
Qwen3-14B (15.0)	4	23.1	25.6	21.3	19.1

Kết luận của paper: (i) lấy tập tối ưu cho cắt thuần rồi repair thì kém tối ưu có repair 5% tới 33%; (ii) tiêu chí cosine của ReplaceMe kém 11% tới 189%. Với repair, hấp thụ tương tác trong run: tỉ lệ tương tác ở ngân sách nặng nhất giảm 62% -> 36% (Qwen3-1.7B), 66% -> 52% (Gemma), 57% -> 28% (Llama), 25% -> 15% (Qwen3-14B); nhưng các run được repair riêng vẫn tương tác (R^2 của tổng chi phí repair từng run là 0.8–0.9 ở 2 nhóm nhưng chỉ -0.09 tới 0.44 ở 6 nhóm).



Hình 66: Perplexity ở ba ngân sách nặng nhất được liệt kê của Table 2: tối ưu cắt thuần, chính tập đó sau repair, lựa chọn của ReplaceMe và tối ưu có repair (vẽ lại từ số liệu trong ghi chú)

Cách đọc hình: mỗi nhóm cột là một ngân sách n (số nhóm bị bỏ), đường nét đứt là perplexity của mô hình nguyên. Cột xanh đậm (tối ưu chọn dưới repair) thấp nhất ở hầu hết nhóm, còn cột hồng (ReplaceMe) có thể cao hơn nhiều, ví dụ 149.8 trên Gemma-3-4B ở $n = 6$. Cột xám so với cột xanh nhạt cho thấy repair giúp ra sao khi giữ nguyên tập đã chọn cho cắt thuần.

5.2 Kết quả chính cho các phương pháp chọn (Table 3)

Mọi phương pháp dưới đây dùng cùng map LS (+R), chọn trên văn bản WikiText-2 (khối trên). PPL WikiText-2 / Avg accuracy 5 tác vụ (%).

Mô hình, số layer bỏ	Nguyên	ShortGPT+R	ReplaceMe	SLEB+R	R-FS	R-GS
Qwen3-1.7B, 7/28	26.5 / 60.0	77.4 / 46.1	53.7 / 45.6	40.3 / 45.9	38.7 / 44.9	37.7 / 44.2
Gemma-3-4B, 8/34	32.0 / 69.8	44.7 / 50.8	44.7 / 50.8	50.5 / 54.7	38.2 / 48.1	35.9 / 50.6
Gemma-3-4B, 13/34	32.0 / 69.8	87.2 / 42.7	150.7 / 41.5	69.1 / 44.7	97.6 / 40.6	51.1 / 44.7

Mô hình, số layer bỏ	Nguyên	ShortGPT+R	ReplaceMe	SLEB+R	R-FS	R-GS
Llama-3.1-8B, 8/32	13.7 / 72.4	39.4 / 59.5	40.4 / 57.2	23.6 / 56.2	24.3 / 51.2	22.6 / 55.4
Llama-3.1-8B, 12/32	13.7 / 72.4	94.0 / 46.8	94.0 / 46.8	37.8 / 48.4	95.9 / 39.4	34.4 / 49.0
Qwen3-14B, 10/40	15.0 / 73.0	48.2 / 49.2	41.9 / 51.4	27.2 / 49.1	21.9 / 53.0	21.9 / 54.4
Qwen3.8-27B, 16/64	10.1 / 72.8	34.8 / 40.6	28.7 / 42.2	13.2 / 65.7	13.4 / 66.4	13.3 / 64.9
Qwen3.8-27B, 24/64	10.1 / 72.8	42.4 / 38.6	36.9 / 42.2	17.1 / 57.4	15.7 / 55.5	16.0 / 57.9

(Bảng đầy đủ có thêm Qwen3-1.7B 10/28, Qwen3-14B 15/40, LaCo+R, Prune&Comp+R; trích đủ nằm trong Table 3 của paper.) Dòng "R-GS so với ReplaceMe" của paper: PPL giảm 30%, 21%, 20%, 66%, 44%, 63%, 48%, 21% trên 8 thiết lập của 4 họ nhỏ; 54% và 56% trên Qwen3.8-27B; **Avg accuracy** đổi thêm -1.5, +0.2, -0.2, +3.2, -1.8, +2.2, +3.0, +0.6, +22.7, +15.7 điểm. Tức: **WikiText-PPL cải thiện nhất quán, accuracy lẫn lộn khi mục tiêu là WikiText** (tăng rõ chỉ ở 27B và vài ngân sách nặng). Paper tóm tắt: R-GS có PPL thấp nhất ở 7/8 thiết lập; 20% tới 66% dưới ReplaceMe, 4% tới 29% dưới SLEB có repair; 20% tới 73% dưới Prune&Comp và 37% tới 97% dưới LaCo (cả hai có repair).

Với mục tiêu tác vụ (R-GS-T, khối dưới của Table 3): chọn bằng cross-entropy của đáp án đúng trên 200 mẫu train/tác vụ. Avg accuracy vs ReplaceMe cùng dữ liệu tác vụ: Qwen3-1.7B 47.1 vs 43.8 (+3.3) và 41.7 vs 40.9 (+0.9); Gemma 59.9 vs 45.8 (+14.1) và 51.2 vs 40.6 (+10.6); Llama 61.2 vs 58.9 (+2.3) và 51.9 vs 46.8 (+5.1); Qwen3-14B 59.5 vs 51.4 (+8.1) và 47.9 vs 38.8 (+9.1); Qwen3.8-27B 66.8 vs 42.2 (+24.6) và 59.8 vs 42.2 (+17.6). Paper: tốt hơn ReplaceMe ở cả 8 thiết lập của họ nhỏ (0.9 tới 14.1 điểm) và hơn ở cả 2 ngân sách của 27B (lên tới 25 điểm). Trên Gemma 8/34 giữ 86% độ chính xác mô hình nguyên so với 73% của ReplaceMe. Trên 27B giữ 92% (16/64) và 82% (24/64). Chú ý: PPL của R-GS-T thường **tệ hơn** (ví dụ Qwen3-1.7B 7/28: 64.5 vs 53.7 của ReplaceMe dùng WikiText); mục tiêu quyết định năng lực nào được giữ.

5.3 R-FS

So với tăng loss chính xác của mọi khối layer không critical: khối có R-FS thấp nhất nằm trong 5% khối tốt nhất ở **19/22** độ dài khối; cosine distance của ReplaceMe chỉ ở **2** (Fig. 5b). Tương quan Spearman với tăng loss repair: 0.53 tới 0.86 (R-FS), -0.21 tới 0.36 (cosine). Trong Table 3, khối R-FS (không đánh giá mô hình đã cắt) trong vòng 3% R-GS ở 4/8 thiết lập; ở ngân sách nặng (Gemma 13/34: PPL 97.6 vs 51.1; Llama 12/32: 95.9 vs 34.4) kém xa vì tối ưu không còn là một khối. Paper: R-FS là mặc định tốt khi một khối đủ, R-GS cho ngân sách nặng.

6. Phân tích / ablation

- **Ablation R-GS (Table 4, 8 thiết lập nhỏ):** tỉ số PPL so với R-GS (trung bình hình học / tệ nhất), accuracy trung bình, số lần đánh giá: R-GS 1.00 / 1.00 / 48.1 / 303. Bỏ repair trong tìm kiếm (SLEB+R): 1.19 / 1.41 / 48.0 / 303. Bỏ tìm kiếm, chấm một lần + repair: 1.21 / 1.73 / 46.5 / 34. Bỏ tìm kiếm, tiêu chí khối (ReplaceMe): 1.73 / 2.95 / 47.3 / 0. **Cắt theo R-GS nhưng bỏ repair khi triển khai:** 3.98 / 14.9 (PPL tăng gấp bốn). Cải tiến "shift" (dời nhóm) 0.995 / 1.00 / 48.4. Kết luận: repair *trong* tìm kiếm là thành phần quan trọng nhất; accuracy trung bình hầu như không đổi giữa các biến thể (46.5–48.4).
- **Calibration khác (mục 7.4, Table 6–7):** dùng nguyên cấu hình mặc định của ReplaceMe (fit trên hội thoại SlimOrca, không ridge): R-GS có PPL thấp hơn 16% tới 73% trên Llama và Gemma. Chọn bằng mục tiêu SlimOrca (rời với dữ liệu fit): PPL thấp hơn ở 7/8 thiết lập, accuracy ở 6/8.

- **Sau hồi phục (mục 7.5, Fig. 6):** C4 PPL của R-GS thấp hơn ReplaceMe ở mọi mô hình/ngân sách trước hồi phục; sau 200 bước LoRA trên C4 (3 seed) R-GS vẫn thấp hơn ở 7/8, lần lượt 16% tới 30% (Llama), 2.5% tới 19% (Gemma), 1% tới 2% (Qwen3-1.7B): khoảng cách co lại nhưng còn.
- **MMLU (Table 12):** mục tiêu chọn quyết định layer nào được giữ. MMLU 5-shot cần các layer giữa ít ảnh hưởng PPL WikiText: trên Llama và Qwen3-14B, tập WikiText của R-GS và SLEB bỏ chúng và chỉ đạt 24% tới 28% (gần ngẫu nhiên 25%), trong khi ShortGPT (khối sâu) giữ 67.1% và 68.4% ở 25%. Với mục tiêu MMLU, R-GS bỏ layer sâu và đạt cao nhất ở 7/8 thiết lập (ví dụ Qwen3-14B 15 layer 53.3% so với tối đa 26.7% của phương pháp khác). Khoảng cách giữa PPL và accuracy: Phụ lục F cũng cho thấy WikiText-objective bỏ layer giữa mà WinoGrande cần (WinoGrande Llama: ShortGPT/ReplaceMe 71.1/69.7 so với 54.4 của R-GS).
- **Cơ chế siêu cộng tính (Phụ lục C, Table 5):** ba giả thuyết bị loại (thay layer bằng cập nhật trung bình vẫn còn số hạng cặp; "các layer sau khôi phục" tương quan Spearman chỉ +0.17 so với +0.72 của khoảng cách; độ cong hàm loss; tách logit). Chỉ **tương đồng chức năng** có hỗ trợ một phần (CKA tuyến tính của hai cập nhật, tương quan riêng phần với số hạng cặp: Qwen3-1.7B +0.37, Llama +0.36, Qwen3-14B +0.11, Gemma -0.06). Kết luận paper: tương tác đến từ phép tính bị mất, chưa giải thích đầy đủ.
- **Tìm kiếm không repair (Phụ lục D):** cải tiến SLEB (lặp + dời) tìm được tối ưu ở 16 ngân sách trong mọi seed (+2 nữa ở một phần seed).
- **Kích thước calibration:** với 16 chuỗi, tối ưu trên calibration trùng tối ưu trên test ở 13/17 thiết lập; 64 chuỗi: 100% ở Qwen3-1.7B và Gemma.
- **Fit repair khác:** hai cách fit map sau trên mạng đã repair không cải thiện; fit hướng tới activation mạng nguyên làm tập nhiều run tệ hơn 1% tới 16%; fit hướng tới đầu ra của run, tính trên input của mạng đã repair: tốt hơn khoảng 5% trên Qwen3-1.7B, trong ±1.5% trên Llama. Chênh lệch nhỏ hơn nhiều so với chênh lệch giữa các cách chọn.
- **Chi phí suy luận (Table 11, bf16, RTX A6000):** bỏ 25% layer ~1.3x tăng tốc (prefill 2048 token, decode 256 token ở batch 1 và 16), 37.5% ~1.55x. Ví dụ Llama-3.1-8B 8/32: trọng số 16.1 -> 12.6 GB, KV 128 -> 96 KB/token, tăng tốc 1.32 (prefill), 1.30 (batch 1), 1.31 (batch 16); 12/32: 16.1 -> 10.8 GB, 1.58, 1.54, 1.55. Qwen3.8-27B bỏ 24/64: 53.8 -> 35.6 GB, vừa một GPU 48GB. Gemma-3 không gộp được map (chuẩn hoá post-feedforward), nên có thêm một ma trận $d \times d$ cho mỗi run.

7. Giới hạn

Do tác giả nêu:

- Liệt kê chính xác chỉ ở độ chi tiết 2 layer đến 6 nhóm, hoặc 1 layer đến 3 layer; ở ngân sách nặng chỉ so sánh các phương pháp với nhau.
- Kết luận cấu trúc là về **perplexity**: ở độ chi tiết 2 layer, tối ưu PPL là tập chính xác nhất (hoặc trong 1 sai số chuẩn) ở 10/14 trường hợp; ở 4 trường hợp còn lại có tập khác chính xác hơn 2–7 điểm.
- Chỉ thử một repair (LS của ReplaceMe); các repair mới (LinearPatch, Prune&Comp, shifted maps) chưa kiểm; không so với BlockPruner và SubFit.
- Hồi phục chỉ một độ dài: 200 bước LoRA. Chỉ tiếng Anh và tác vụ trắc nghiệm; đa ngôn ngữ và sinh mở chưa kiểm.
- Cấu hình ReplaceMe mẫu fit trên hội thoại tới 1.024 token với mô hình 4-bit; bài này fit trên khối 256 token ở bfloat16 (khác với bài gốc).

Nhận xét:

- Chọn và đánh giá chủ yếu bằng WikiText-2 (16 chuỗi x 256 token): rất nhỏ, và mục tiêu WikiText lẫn lộn về accuracy (Table 3). Kết luận "layout tối ưu thành khối liền khi có repair" được chứng minh trên PPL, 4 họ, 2 nhóm layer; chưa biết nó giữ với CER của OCR.
- Maps được fit độc lập từ mô hình nguyên (cache); paper thừa nhận các run được repair vẫn tương tác

(R^2 giảm), nên repair chưa "hấp thụ" mọi tương tác.
- Đề xuất bậc hai R-FS dựa trên Fisher kinh nghiệm hạng một; chỉ xác nhận thực nghiệm (Fig. 5), và nó thất bại ở ngân sách nặng.
- Các tên mô hình/tham chiếu trong bài khá mới (2026); các số liệu chưa được kiểm chứng độc lập.

8. Liên hệ với các bài khác

- **36 ReplaceMe:** bài này nhận repair của nó làm đối tượng nghiên cứu và chứng minh tiêu chí chọn khối (cosine, Eq. 4 của ReplaceMe) lệch khỏi tối ưu có repair 11% tới 189%.
- **34 SLEB:** R-GS = SLEB + repair trong vòng lặp, cùng chi phí; SLEB cũng đã được chứng minh chọn tối ưu cắt-thuần ở 13/20 ngân sách.
- **31 ShortGPT, 32 Deeper layers, 35 LaCo:** các kiểu chọn proxy hoặc khối liền; bài này cho thấy "khối liền" chỉ gần tối ưu khi có repair (nhất quán với lựa chọn khối liền của 32 và ReplaceMe), còn ShortGPT chọn gần-liền nên tệ nếu không repair.
- **33 Shortened LLaMA:** bằng chứng tốc độ batch-1; bài này xác nhận tăng tốc ~1.3x/1.55x khi cắt 25%/37.5%.
- **01 OBD/OBS, 22 OBC, 23 SparseGPT:** R-FS dùng cùng họ khai triển bậc hai/Fisher (OBD/OBS, Fisher pruning, Taylor importance) nhưng áp dụng cho *sai số còn lại sau repair*.
- **27 LLM Surgeon, 07 Molchanov importance:** Fisher kinh nghiệm và Taylor importance liên quan.
- **39–40 Minitron:** hồi phục bằng distillation sau khi chọn; bài này chỉ thử 200 bước LoRA.

9. Gợi ý áp dụng

Mức liên quan: rất cao.

- Ngược lại, thước Euclid của LS giữ phần "sửa được". Hiện tượng bạn thấy (cắt neuron MLP phục hồi kém hơn cắt layer) có thể hiểu: khối layer có phần dư repair nhỏ đối với một map tuyến tính, còn cắt neuron thì không có map $d \times d$ tương đương.
- Với distillation (không phải map LS), chưa rõ tối ưu có cùng cấu trúc "khối liền" hay không; đây là câu hỏi mở đáng thử.
- **Mục tiêu quyết định giữ cái gì.** Table 12 cho thấy WikiText-PPL giữ lại khác với MMLU hoặc WinoGrande. Đây là điểm bạn phải tự kiểm chứng.
- **Phù hợp hybrid:** có kiểm chứng Qwen3.8-27B (3 linear-attention : 1 full-attention); R-GS hơn ReplaceMe ở đó vì ReplaceMe chọn các layer đầu.
- **Chi phí:** R-GS tốn $\sim n \cdot L$ lần đánh giá mô hình cắt; decoder ~20 layer thì rẻ.

10. Tóm tắt công thức / khái niệm

Khái niệm	Ý nghĩa
Run	tập cực đại các nhóm bị bỏ kề nhau, mỗi run một map T
$T_{a,b}$ (Eq. 1)	ridge LS: $\arg\min \|M_{a-1}T - (H_b - Y_{a-1})\|_F^2 + \lambda\|T\|_F^2$, gộp vào `down_proj`
$\mu(S)$	số hạng tương tác Möbius; cộng tính khi mọi $\mu = 0$
Critical group	bỏ riêng làm PPL tăng hơn gấp đôi, luôn được giữ ở tối ưu
Siêu cộng tính cục bộ	bỏ hai layer gần nhau tốn hơn tổng riêng; suy giảm trong 3–6 layer
R-GS	tham lam đánh giá mỗi ứng viên sau repair; chi phí = SLEB
R-FS	$s = \sum_t (g_t \cdot \delta_t)^2$, phần dư repair cân bằng Fisher, một forward + backward
R-GS-T	R-GS với mục tiêu tác vụ (CE đáp án đúng trên 200 mẫu)

11. Kết luận ngắn

Nghiên cứu đo nghiệm tối ưu của depth pruning trong không gian tìm kiếm được xét và cho thấy tập layer tốt nhất phụ thuộc vào bước repair đi kèm: không repair thì tách đoạn, có repair tuyến tính thì thành khối liền. Chọn layer dưới đúng repair sẽ triển khai hạ PPL 20–66% so với ReplaceMe ở cùng chi phí với SLEB, và mục tiêu chọn (PPL, tác vụ, MMLU) quyết định năng lực nào còn lại. Giới hạn: một loại repair, đánh giá trắc nghiệm/PPL, không có tác vụ sinh hay OCR.

Link gốc: https://arxiv.org/abs/2610.30552

Bài 38

Sheared LLaMA: Accelerating Language Model Pre-training via Structured Pruning (2023)

Tác giả: Mengzhou Xia, Tianyu Gao, Zhiyuan Zeng, Danqi Chen (Princeton NLP).
Venue / arXiv: arXiv:2310.06694 (v1: 10/10/2023; bản cuối 04/2024); ICLR 2024.
Nguồn: https://arxiv.org/abs/2310.06694
Chủ đề: Structured pruning có mục tiêu (targeted) cho LLM + tiếp tục pre-train (continued pre-training) + dynamic batch loading; "prune rồi train tiếp" thay cho train từ đầu.
Vị trí trong lộ trình: Đọc trước: 17 (CoFi, cùng nhóm tác giả, mask học được), 26 (LLM-Pruner), 25 (ZipLM). Bài này thêm: (1) ép mô hình sau prune về đúng một *kiến trúc đích* chuẩn, (2) chứng minh rằng bước train tiếp với lượng token lớn (50B) mới là thứ tạo ra mô hình cạnh tranh, (3) cân bằng dữ liệu theo domain. Tiếp theo: 39 (Minitron) làm cùng ý tưởng nhưng dùng distillation và so sánh depth/width kỹ hơn.

> Ghi chú học tập bằng tiếng Việt, diễn giải và tóm tắt, không phải bản dịch.

1. Bối cảnh

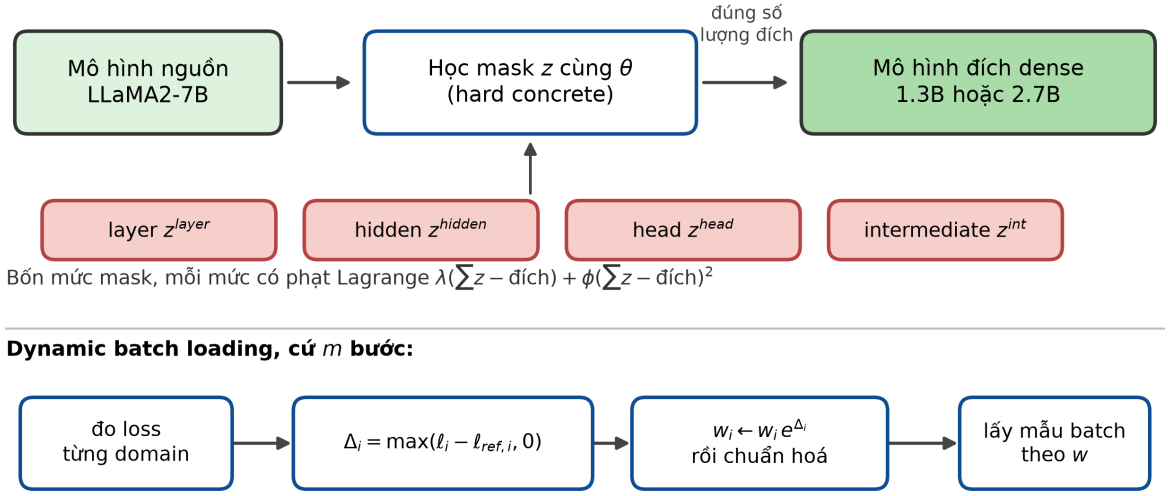
- Train một LLM cỡ vừa (1-3B) từ đầu cần hàng nghìn tỷ token (TinyLlama-1.1B: 3T token). Trong khi đó đã có sẵn các mô hình lớn mạnh (LLaMA2-7B, 2T token). Câu hỏi: có thể cắt mô hình lớn thành mô hình nhỏ với chi phí thấp hơn không?
- **Structured pruning** (cắt có cấu trúc): bỏ nguyên cả head, cả layer, cả chiều trung gian của FFN... sao cho ma trận còn lại vẫn *dense* (đặc), chạy nhanh trên GPU thường. Khác với unstructured (xoá từng trọng số, ma trận thưa khó tăng tốc).
- Hai vấn đề của các phương pháp trước (CoFi, LLM-Pruner):
  1. Không chỉ định cấu trúc đích: kết quả có kiến trúc "lạ" (mỗi lớp số head khác nhau; LLM-Pruner ra intermediate size nhỏ hơn hidden size), nên suy luận chậm và khó huấn luyện tiếp.
  2. Sau khi prune chỉ tinh chỉnh ít; paper cho rằng cần bước **continued pre-training** (train tiếp bằng loss language modeling trên lượng lớn dữ liệu) mới đạt chất lượng "general-purpose".
- Tác giả quan sát: khi tiếp tục huấn luyện mô hình đã prune trên dữ liệu pre-training gốc, loss giảm *không đều* giữa các domain so với huấn luyện từ đầu. Pruning giữ lại nhiều kiến thức ở domain "dễ/ít entropy" (GitHub) hơn domain "khó/lớn" (C4).

2. Ý tưởng chính

Hai thành phần gọi chung là **LLM-shearing**. (1) *Targeted structured pruning*: học các mask (0/1) trên layer, hidden dim, head, intermediate dim, kèm ràng buộc Lagrange ép tổng số phần tử giữ lại đúng bằng kiến trúc đích (ví dụ lấy cấu hình Pythia-1.4B hoặc INCITE-Base-3B). Kết quả là mô hình dense, đồng nhất giữa các lớp. (2) *Dynamic batch loading*: trong lúc prune và huấn luyện tiếp, cứ mỗi m bước đo loss trên từng domain, domain nào còn xa "loss tham chiếu" thì được lấy nhiều dữ liệu hơn. Với chỉ 0.4B token cho prune và 50B token cho tiếp tục huấn luyện (khoảng 3% chi phí so với huấn luyện từ đầu), mô hình 1.3B và 2.7B vượt các mô hình cùng cỡ.

Cách đọc hình: nửa trên là bước pruning, mask z được học cùng trọng số θ và bị ép về đúng số lượng của kiến trúc đích. Nửa dưới là vòng lặp mỗi m bước: domain nào còn cao hơn loss tham chiếu thì được nhân trọng số e^{Δ_i} rồi chuẩn hoá. Hai phần chạy cùng lúc trong lúc prune và train tiếp.

3. Phương pháp chi tiết



Hình 67: Sơ đồ LLM-shearing: mask học được trên bốn mức hạt với ràng buộc Lagrange về kiến trúc đích, và dynamic batch loading chỉnh trọng số domain (minh hoạ, tự tính)

3.1 Mask ở nhiều mức hạt

Mô hình nguồn M_S có L_S layer (mỗi layer = 1 MHA + 1 FFN), hidden d_S, H_S head, intermediate m_S. Các biến mask:

Mức	Mask
Layer	$z^{layer} \in \mathbb{R}^{L_S}$
Hidden dimension (dùng chung mọi layer)	$z^{hidden} \in \mathbb{R}^{d_S}$
Head	$z^{head} \in \mathbb{R}^{H_S}$ (mỗi layer)
Intermediate dim của FFN	$z^{int} \in \mathbb{R}^{m_S}$ (mỗi layer)

Mask nhân vào output tương ứng: head thì nhân vào output của head; intermediate thì nhân vào activation trung gian; layer thì nhân vào output layer; hidden thì nhân vào cả output của head lẫn MLP. Nhân mask = 0 tương đương xoá tham số.

3.2 Hard concrete (l0 regularization)

Mask không phải biến rời rạc nên khó đạo hàm. Dùng phân bố **hard concrete** (Louizos et al. 2018): một xấp xỉ liên tục, có khối lượng xác suất tập trung ở 0 và 1.

$$u \sim U(0,1), \quad s = \sigma\Big(\tfrac{1}{\beta}\big(\log \tfrac{u}{1-u} + \log \alpha\big)\Big), \quad \bar{s} = s(\zeta - \gamma) + \gamma, \quad z = \min(1, \max(0, \bar{s}))$$

Siêu tham số paper dùng: $\beta = 0.83,\ \zeta = 1.1,\ \gamma = -0.1$. α là tham số học được của từng mask.

3.3 Ràng buộc kiến trúc đích bằng Lagrange

Thay vì đặt một mức sparsity tổng, paper đặt ràng buộc cho từng loại cấu trúc. Ví dụ với số head đích H_T trên một layer:

$$\tilde{\mathcal{L}}^{head}(\lambda, \phi, z) = \lambda^{head} \cdot \Big(\sum z^{head} - H_T\Big) + \phi^{head} \cdot \Big(\sum z^{head} - H_T\Big)^2$$

Tương tự cho layer, hidden, intermediate. Bài toán min-max:

$$\min_{\theta,z} \max_{\lambda,\phi} \mathcal{L}_{prune}(\theta, z, \lambda, \phi) = \mathcal{L}(\theta, z) + \sum_{j=1}^{L_S} \tilde{\mathcal{L}}_j^{head} + \sum_{j=1}^{L_S} \tilde{\mathcal{L}}_j^{int} + \tilde{\mathcal{L}}^{layer} + \tilde{\mathcal{L}}^{hidden}$$

$\mathcal{L}(\theta, z)$ là loss language modeling của mô hình có mask. Trọng số θ và mask z được học *cùng nhau*; λ, ϕ đi theo hướng ngược (gradient ascent) để ép ràng buộc được thoả. Sau pruning, giữ các thành phần có điểm mask cao nhất đúng theo số lượng đích rồi bỏ phần còn lại. Mask thường hội tụ về 0/1 nên hầu như không cần "hấp thụ" mask vào trọng số.

Ghi chú chi tiết (Appendix D): LLaMA2-7B dùng FFN dạng GLU (3 ma trận gate/up/down). Head dim kế thừa từ nguồn (128), nên số head đích = hidden đích / head dim nguồn.

3.4 Dynamic batch loading

Gom dữ liệu thành k domain (CommonCrawl, C4, GitHub, Wikipedia, Books, ArXiv, StackExchange). Mỗi domain có tập validation riêng và một **loss tham chiếu** ℓ_{ref}. Mặc định tham chiếu là loss dự đoán từ scaling law (fit trên LLaMA2-7B/13B/70B: $L(N, D) = E + A/N^{\alpha} + B/D^{\beta}$, suy ra loss của LLaMA2-1.3B/2.7B giả định). Phương án phụ: dùng loss của chính mô hình nguồn (source reference), kém hơn chút, đặc biệt ở toán và code, nhưng dùng được khi không có họ mô hình nhiều cỡ.

```
Input: k domain D_i, validation D_i^val, trọng số đầu w0, loss tham chiếu l_ref,
       số bước T, chu kỳ đánh giá m
for t = 1..T:
    if t mod m == 0:
        l_t[i]  = loss(model, D_i^val)            # (với pruning: dùng L_prune)
        Δ_t[i]  = max(l_t[i] - l_ref[i], 0)        # domain nào còn "nợ" loss
        α[i]    = w_{t-m}[i] * exp(Δ_t[i])         # exponential ascent
        w_t     = α / sum(α)                       # chuẩn hoá
    batch B ~ sampling theo w_t
    cập nhật (θ, z, φ, λ) hoặc θ trên B
```

Giai đoạn prune dùng trọng số RedPajama gốc làm w_0; giai đoạn train tiếp dùng trọng số cuối của giai đoạn prune.

Ví dụ nhỏ: nếu C4 còn cao hơn tham chiếu 0.1 và GitHub đã thấp hơn tham chiếu ($\Delta = 0$), sau một lần cập nhật trọng số C4 nhân với $e^{0.1} \approx 1.105$ còn GitHub giữ nguyên rồi chuẩn hoá, nên C4 được lấy nhiều dữ liệu hơn.

3.5 Thiết lập huấn luyện (Table 7)

	Pruning	Continued pre-training
Ngân sách	0.4B token	50B token
LR của z, ϕ, λ	1.0	không có
LR của θ	1e-4	1e-4
Batch (token)	131K	1M
Chu kỳ đánh giá m	50 bước	400 bước
Số bước	3,200	51,200
GPU	8	16
Throughput	15K tok/s	145K (1.3B) / 77K (2.7B)

Pruning chậm khoảng 5 lần so với train LM thường, nên chỉ dùng ngân sách nhỏ.

4. Thí nghiệm và kết quả chính

Nguồn: LLaMA2-7B. Đích: 1.3B (kiến trúc Pythia-1.4B) và 2.7B (kiến trúc INCITE-Base-3B). Dữ liệu: RedPajama, chuỗi 4,096 token. Sheared-LLaMA-1.3B: 24 layer, hidden 2048, intermediate 5504, 16 head; 2.7B: 32 layer, hidden 2560, intermediate 6912, 20 head (Table 8).

Bảng downstream (Table 2, trích; số copy từ paper, cột Average là trung bình của 11 tác vụ):

Mô hình (token train)	Average
LLaMA2-7B (2T)	64.6
OPT-1.3B (300B)	48.2
Pythia-1.4B (300B)	48.9
TinyLlama-1.1B (3T)	50.0
Sheared-LLaMA-1.3B (50B)	**51.0**
OPT-2.7B (300B)	51.4
Pythia-2.8B (300B)	52.5
INCITE-Base-3B (800B)	54.7
OpenLLaMA-3B-v1 (1T)	55.1
OpenLLaMA-3B-v2 (1T)	55.7
Sheared-LLaMA-2.7B (50B)	**56.7**

Vài cột riêng: Sheared-2.7B HellaSwag (10-shot) 70.8, ARC-C (25-shot) 41.2, BoolQ 73.7, MMLU (5-shot) 26.4, NQ 16.5; Sheared-1.3B MMLU 25.7, NQ 9.6. MMLU của các mô hình nhỏ đều gần mức ngẫu nhiên (~25-27%).

- Instruction tuning (10,000 cặp ShareGPT, GPT-4 làm giám khảo, hoán vị thứ tự để bù position bias): Sheared-1.3B thắng Pythia-1.4B 57.4% vs 42.7%; Sheared-2.7B thắng INCITE-Base-3B 63.5% vs 36.6%, OpenLLaMA-v1 56.6% vs 43.5%, OpenLLaMA-v2 54.3% vs 45.8%.
- Chi phí: 0.4B + 50B token, tức khoảng 3% (1/32) của OpenLLaMA-3B-v2 hay khoảng 5% ngân sách pre-training của OpenLLaMA theo lời tác giả. Tác giả nêu cả hiệu quả mẫu: 1.3B vượt TinyLlama dù TinyLlama dùng 3T token.

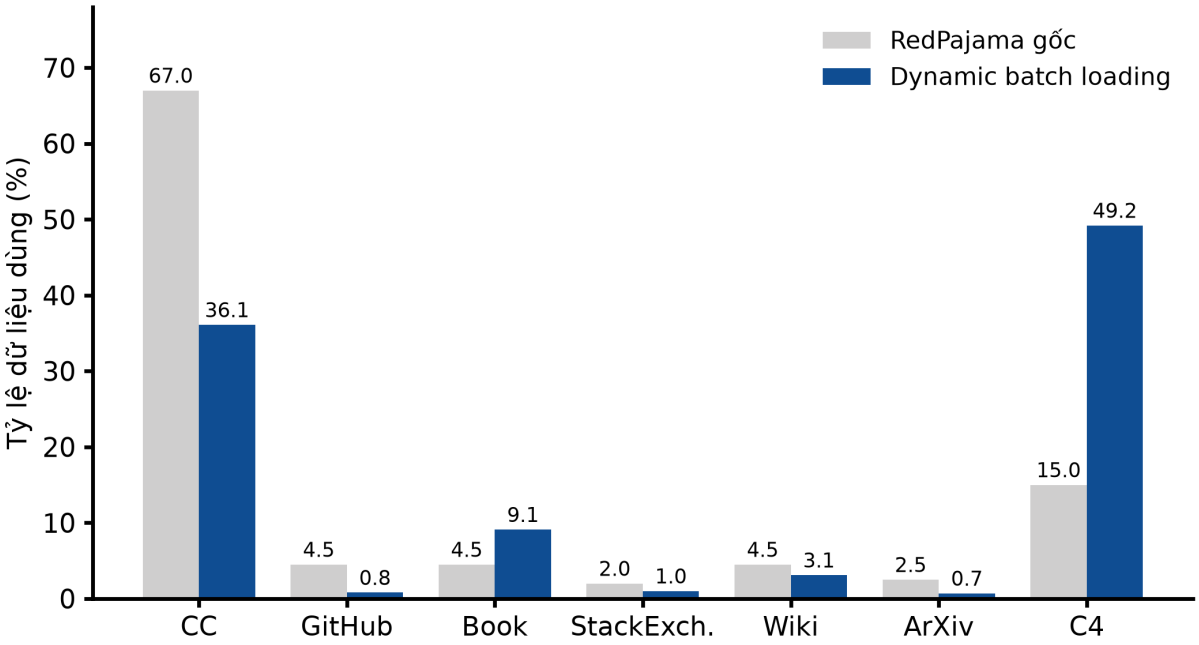
Dynamic batch loading (Table 3, tỷ lệ dữ liệu thực dùng, 1.3B):

	CC	GitHub	Book	StackExch.	Wiki	ArXiv	C4
RedPajama gốc	67.0%	4.5%	4.5%	2.0%	4.5%	2.5%	15.0%
Dynamic	36.1%	0.8%	9.1%	1.0%	3.1%	0.7%	49.2%

Cách đọc hình: mỗi cặp cột là một domain, cột xám là tỷ lệ gốc và cột xanh là tỷ lệ sau khi dynamic batch loading điều chỉnh. C4 tăng từ 15.0% lên 49.2% trong khi CommonCrawl giảm từ 67.0% xuống 36.1%. Các domain nhỏ như GitHub và ArXiv bị giảm xuống dưới 1%.

5. Phân tích / ablation

- **Targeted vs CoFi (Table 4):** CoFi (không đồng nhất giữa layer) có PPL tốt hơn một chút nhưng chậm hơn: 1.3B: PPL 9.1 / 51 tok/s (CoFi) so với 10.3 / 58 tok/s (targeted); 2.7B: 7.0 / 37 so với 7.7 / 43 (A100, batch 1, sinh tới 512 token). Targeted cần thêm ~0.5B token train tiếp để bắt kịp PPL của CoFi.



Hình 68: Tỷ lệ dữ liệu thực dùng theo domain khi huấn luyện 1.3B, RedPajama gốc so với dynamic batch loading (Table 3) (vẽ lại từ số liệu trong ghi chú)

- **Targeted vs LLM-Pruner (Table 11-12):** LLM-Pruner ra cấu hình 32 layer, 7 head, intermediate 2201, hidden 4096 (1.6B); Sheared (1.3B) 24 layer, 16 head, intermediate 5504, hidden 2048. Sheared: 58 tok/s sinh, 139K tok/s train, PPL 6.85; LLM-Pruner: 43 tok/s, 83K tok/s, PPL 7.09 (sau ~6B token).
- **Chia ngân sách (Table 5, tổng 5B token):** tăng token cho pruning thì PPL sau prune giảm (12.99 -> 10.29 -> 9.01 -> 8.04 khi pruning dùng 0.2/0.4/0.8/1.6B) và PPL sau train tiếp cũng giảm nhẹ (7.46 -> 7.32 -> 7.23 -> 7.08). Họ vẫn chọn 0.4B vì pruning đắt hơn nhiều.
- **Dynamic loading** làm loss giảm đều giữa các domain và tăng downstream (Fig. 4-5). Chỉ loại các domain "dễ" khỏi giai đoạn prune không giải quyết được (Appendix F.8): chênh lệch loss đều hơn sau prune nhưng lại phân kỳ sau train tiếp.
- **Scaling reference vs source reference:** scaling tốt hơn một chút.
- **Prune từ LLaMA1 vs LLaMA2:** LLaMA2 hơn nhưng chênh lệch không lớn.
- **So với train tiếp một mô hình nhỏ có sẵn (F.7):** INCITE-Base-3B khởi đầu cao hơn nhưng bão hoà; Sheared-2.7B nhanh chóng vượt. Tác giả chú thích rằng nếu mô hình nhỏ có sẵn gần ngang mô hình nguồn thì nó có thể là điểm khởi đầu tốt hơn.
- **Pythia-410M -> 160M (F.5):** với 10B token đạt xấp xỉ Pythia-160M train từ đầu (43.51 vs 43.56); với 33B token đạt 45.78. Tác giả nói đây là kết quả sơ bộ.
- **Toán/code (F.3, Table 13):** yếu: GSM8K của Sheared-2.7B là 2.4-2.7 (EM) so với 13.7 của LLaMA2-7B; HumanEval Pass@1 3.7-4.9.

6. Giới hạn

Tác giả nêu:

- Phụ thuộc vào có sẵn mô hình và dữ liệu pre-training mã nguồn mở; domain không có trong dữ liệu thì khó hồi phục.
- Chỉ thử mô hình nguồn 7B (và LLaMA1-13B, Pythia-410M ở phụ lục); khẳng định "tổng quát hoá" chưa được kiểm chứng.
- Khả năng code/toán còn yếu (do ít dữ liệu code trong RedPajama và pruning).

Nhận xét:

- Loss dùng khi train tiếp chỉ là LM loss trên one-hot token; không dùng logits của mô hình lớn. Paper nhắc distillation chỉ ở related work (cho rằng tốn chi phí chạy teacher); Minitron (39) sau đó cho thấy distillation giúp đáng kể.
- Mask được học ở bước pruning với chỉ 0.4B token; chưa có so sánh depth vs width có hệ thống (kiến trúc đích lấy từ mô hình có sẵn).
- Cần 16 GPU và 50B token: cùng bậc "huấn luyện lớn", không phải fine-tune nhẹ.

7. Liên hệ với các bài khác

- **17 CoFi:** cùng tác giả đầu; Sheared-LLaMA là bản "targeted" và quy mô LLM của CoFi (hard concrete + Lagrange).
- **26 LLM-Pruner, 25 ZipLM, 28 FLAP, 29 SliceGPT:** các phương pháp prune LLM với ít/không train tiếp; Sheared cho rằng thiếu bước continued pre-training thì mất nhiều.
- **11 Rethinking the value of pruning:** quan điểm "prune chỉ là tìm kiến trúc" so với "giữ trọng số làm khởi tạo"; F.7 và F.5 của Sheared nghiêng về phía trọng số được kế thừa có giá trị.
- **39-40 Minitron:** cùng pipeline prune + retrain, nhưng dùng distillation, importance dựa trên activation (không cần học mask) và nghiên cứu độ sâu vs độ rộng.
- **43 Pruning vs scratch (2026):** kiểm tra lại câu hỏi nền tảng của Sheared "prune có thật sự tiết kiệm hơn train từ đầu".

8. Gợi ý áp dụng

- Bài 39 mới làm điều đó.

9. Tóm tắt công thức / khái niệm

Khái niệm	Ý nghĩa
Structured pruning	Xoá nguyên head/layer/chiều, giữ ma trận dense
Targeted pruning	Ép kết quả khớp đúng kiến trúc đích cho trước
Hard concrete	Mask xấp xỉ liên tục, tập trung ở 0/1, tham số β, ζ, γ
Lagrange multipliers λ, ϕ	Phạt tuyến tính + bậc hai để $\sum z$ = số lượng đích
Continued pre-training	Tiếp tục train LM loss sau khi cắt để hồi phục
Dynamic batch loading	$w_t \propto w_{t-m} \exp(\max(\ell_t - \ell_{ref}, 0))$
Scaling reference	Loss dự đoán $L(N, D) = E + A/N^{\alpha} + B/D^{\beta}$ theo domain

10. Kết luận ngắn

Sheared-LLaMA cho thấy cắt LLaMA2-7B xuống 1.3B/2.7B rồi tiếp tục huấn luyện 50B token cho mô hình tốt hơn các mô hình cùng cỡ huấn luyện bằng nhiều token hơn nhiều lần, miễn là (a) kiến trúc đích chuẩn và (b) dữ liệu tiếp tục huấn luyện được cân bằng theo domain. Những hạn chế còn lại là: loss chỉ là LM loss, chưa có distillation và chưa phân tích depth vs width.

Link gốc: https://arxiv.org/abs/2310.06694

Bài 39

Compact Language Models via Pruning and Knowledge Distillation (2024)

Tác giả: Saurav Muralidharan, Sharath Turuvekere Sreenivas (đóng góp ngang nhau), Raviraj Joshi, Marcin Chochowski, Mostofa Patwary, Mohammad Shoeybi, Bryan Catanzaro, Jan Kautz, Pavlo Molchanov (NVIDIA).
Venue / arXiv: arXiv:2407.14679 (v1: 19/07/2024; bản v2: 04/11/2024). Paper không ghi tên hội nghị trong văn bản được trích.
Nguồn: https://arxiv.org/abs/2407.14679
Chủ đề: Pipeline "prune rồi retrain bằng distillation" cho LLM (họ Minitron, cắt từ Nemotron-4 15B xuống 8B và 4B); so sánh depth vs width, importance dựa trên activation, các loss distillation, tìm kiến trúc nhẹ (lightweight NAS).
Vị trí trong lộ trình: Đọc trước: 38 (Sheared LLaMA, cũng prune + train tiếp nhưng chỉ dùng LM loss), 31 (ShortGPT, chỉ số Block Importance), 33 (Shortened LLaMA, chỉ số PPL cho layer), 35 (LaCo, "layer collapse"). Bài này thêm: bộ "best practices" được kiểm chứng bằng ablation, đặc biệt (a) width thắng depth *chỉ sau khi retrain*, (b) distillation bằng KL trên logits thắng huấn luyện thường. Tiếp theo: 40 (áp dụng cho Llama-3.1 8B), 41-42 (Flextron, Puzzle).
Chương nền liên quan: nền 05: Cắt rồi huấn luyện lại

> Ghi chú học tập bằng tiếng Việt, diễn giải và tóm tắt, không phải bản dịch. Một số bảng/hình trong bản text bị lệch cột; chỗ nào không chắc, ghi rõ.

1. Bối cảnh

- Các hãng thường huấn luyện cả một họ mô hình (ví dụ Llama-2 7B/13B/70B), mỗi cỡ từ đầu với hàng nghìn tỷ token. Rất tốn.
- Ý tưởng thay thế: huấn luyện *một* mô hình lớn, rồi **prune** (cắt bớt tham số) để lấy các mô hình nhỏ, sau đó **retrain** (train lại ngắn để hồi phục độ chính xác) bằng một phần nhỏ dữ liệu.
- **Knowledge distillation (KD)**: thay vì học từ nhãn thật (token kế tiếp), mô hình nhỏ (student) học bắt chước đầu ra của mô hình lớn (teacher), thường bằng cách khớp phân phối xác suất trên từ vựng. Tín hiệu từ teacher "giàu" hơn one-hot (cho biết token nào gần đúng), nên học nhanh hơn.
- Trước bài báo này, các công trình prune LLM (LLM-Pruner, SliceGPT, LaCo, ShortGPT, Sheared LLaMA) chỉ tập trung một trục (depth *hoặc* width) và huấn luyện lại bằng loss thường hoặc chỉ tinh chỉnh ít. Tác giả nói họ là những người đầu tiên dùng distillation từ teacher chưa nén để huấn luyện lại mô hình đã prune có cấu trúc.
- Hạn chế của phương pháp mask-học-được (Sheared, LLM-Pruner, SliceGPT): cần gradient (tốn bộ nhớ) và nhiều dữ liệu. Minitron muốn chỉ dùng **forward pass**.

2. Ý tưởng chính

Tính độ quan trọng của từng layer, head, neuron MLP, kênh embedding chỉ bằng activation trên 1024 mẫu (không cần gradient). Xếp hạng, cắt bỏ phần ít quan trọng, rồi huấn luyện lại mô hình đã cắt bằng KL-divergence trên logits so với teacher (chính là mô hình gốc). Khi cần chọn kiến trúc, liệt kê <20 cấu hình có cùng số tham số, huấn luyện lại nhẹ mỗi cái (~1.8B token) rồi chọn cái tốt nhất để huấn luyện tiếp. Kết quả: Minitron 8B (cắt từ Nemotron-4 15B) dùng 94B token (khoảng 40 lần ít hơn Nemotron-3 8B) mà ngang hoặc hơn nhiều mô hình cùng cỡ. Hai phát hiện "ngược trực giác": thứ hạng các lựa chọn cắt *đảo* sau vài trăm bước huấn luyện lại, và width thắng depth chỉ sau huấn luyện lại.

3. Phương pháp chi tiết

3.1 Ký hiệu

MLP: $\text{MLP}(X) = \delta(XW_1^\top)W_2$, với d_{model} (embedding) và d_{hidden} (chiều ẩn MLP). MHA: $\text{Concat}(\text{head}_1..\text{head}_L)W_O$. LayerNorm có γ, β.

3.2 Độ quan trọng dựa trên activation (chỉ forward)

Dùng tập calibration D 1024 mẫu ngẫu nhiên từ dữ liệu train. Với batch B và chuỗi S:

$$F_{head}^{(i)} = \sum_{B,S} \|\text{Attn}(XW_{Q,i}, XW_{K,i}, XW_{V,i})\|_2, \quad F_{neuron}^{(i)} = \sum_{B,S} X\,W_1^{i\top}, \quad F_{emb}^{(i)} = \sum_{B,S} \text{LN}(X)_i$$

Tức là: head quan trọng nếu output attention có chuẩn lớn; neuron MLP quan trọng nếu activation lớn; kênh embedding quan trọng nếu output LayerNorm của kênh đó lớn. Phép "cộng" trên batch và chuỗi không phải lúc nào cũng tối ưu, nên paper thử 3 hàm gộp: mean(|S|), L2 norm, variance. Điểm theo layer được cộng để ra điểm toàn mạng cho từng trục.

Depth (layer) có 2 chỉ số:

1. *PPL*: bỏ riêng layer đó và đo perplexity tăng bao nhiêu (từ Shortened LLaMA, bài 33).
2. *Block Importance (BI)* (từ ShortGPT, bài 31): khoảng cách cosine giữa input và output của layer:

$$\text{BI}_i = 1 - \mathbb{E}_{X,t}\frac{X_{i,t}^\top X_{i+1,t}}{\|X_{i,t}\|_2\|X_{i+1,t}\|_2}$$

BI tính cho mọi layer trong một lần forward, nhanh hơn PPL; có thể mở rộng cho cụm layer liên tiếp.

3.3 Cắt mô hình

Xếp hạng theo điểm, rồi cắt trực tiếp ma trận trọng số: neuron/head thì cắt hàng/cột trong MLP/MHA; kênh embedding thì cắt chiều embedding của mọi ma trận MLP, MHA và LayerNorm.

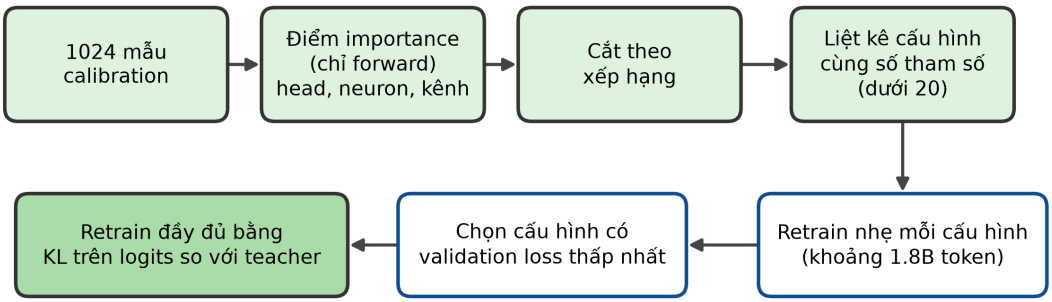
Gộp phần dư của head bị cắt: khi cắt L head xuống K head, head mới thứ i là $\text{head}_i + (\text{head}_i - \text{head}_{2K-i+1})$ cho $i \in [K-(L-K), K]$ (tương tự Layer Collapse của LaCo cho depth); với GQA chỉ áp dụng cho query head. Paper nói có tăng độ chính xác nhưng không ablation riêng ở phần đã đọc.

3.4 Lightweight NAS (Figure 3)

```
Input: ngân sách tham số (vd 8B ± 5%), không gian tìm kiếm
  Layers ∈ [29..32], Heads ∈ {32,48}, MLP expansion ∈ {2.5,3,3.5,4}, Embedding ∈
  ↪ {4096,4680,5120,5632,6144}  (Table 12, cho 8B)
1. Liệt kê mọi cấu hình thoả ngân sách        # 15 ứng viên cho 8B, 18 cho 4B
2. Với mỗi ứng viên: cắt (từ xếp hạng importance) + retrain nhẹ ~1.8B token (400 bước)
3. Chọn ứng viên có validation loss thấp nhất, retrain đầy đủ
```

Không dùng genetic/Bayesian search vì không gian đã đủ nhỏ. Tác giả nói LoRA cũng có thể dùng ở bước retrain nhẹ (chưa thử).

Cách đọc hình: hàng trên đi từ dữ liệu calibration đến danh sách cấu hình cùng số tham số, hàng dưới từ retrain nhẹ đến retrain đầy đủ. Bước chọn cấu hình nằm sau retrain nhẹ chứ không ngay sau khi cắt, vì thứ hạng còn đổi trong vài trăm bước đầu.



Hình 69: Quy trình Minitron: importance chỉ bằng forward, cắt, liệt kê cấu hình, retrain nhẹ để xếp hạng rồi retrain đầy đủ bằng KL (minh hoạ, tự tính)

3.5 Retrain bằng distillation

Phân phối của token x_i với nhiệt độ τ: $p(x_i, \tau) = \exp(x_i/\tau) / \sum_j \exp(x_j/\tau)$.

$$L_{logits} = \frac{1}{l} \sum_{k=1}^{l} \text{Loss}\big(p_t^k(x,\tau),\, p_s^k(x,\tau)\big)$$

$$L_{is} = \frac{1}{l} \sum_{i=1}^{l} \sum_{k \in H} \text{Loss}_k\big(h_t^{ki}, h_s^{ki}\big)$$

$$\boxed{L = L_{CLM} + L_{logits} + \alpha\, L_{is}}, \quad \alpha = \frac{L_{logits}}{L_{is}} \text{ (tính động)}$$

L_{CLM} là cross-entropy với nhãn thật; L_{is} là loss trên trạng thái trung gian (intermediate states: embedding, output các block...), khác chiều giữa student và teacher được xử lý bằng một phép biến đổi tuyến tính học chung, hidden states lấy sau LayerNorm. Do độ lớn L_{logits} và L_{is} chênh nhiều nên đặt α theo tỉ lệ động hiệu quả hơn hằng số. **Nhưng phát hiện cuối cùng**: dùng *chỉ* L_{logits} (KLD) là tốt nhất trong hầu hết trường hợp (xem phần 5).

4. Depth vs width và việc đảo thứ hạng sau retrain (chi tiết)

4.1 Thiết kế thí nghiệm

Cắt Nemotron-4 15B xuống cỡ Nemotron-3 8B theo các cách khác nhau, sau đó retrain với ~1.8B token (400 bước) và đo hai thứ: *distillation loss* (KL trên logits, trước → sau retrain) và *LM validation loss* sau retrain. Ký hiệu cột trong Table 1: DEP = cắt depth, MLP = cắt neuron MLP, ATT = cắt head, EMB = cắt kênh embedding.

Table 1 (số copy từ paper):

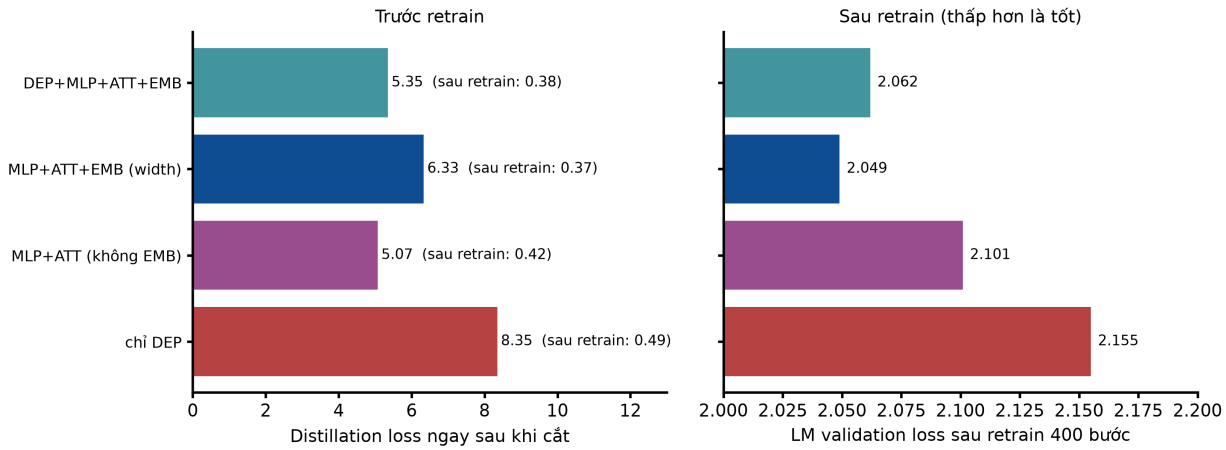
DEP	MLP	ATT	EMB	Distillation loss (trước → sau retrain)	LM val loss
có	có	có	có	5.35 → 0.38	2.062

DEP	MLP	ATT	EMB	Distillation loss (trước → sau retrain)	LM val loss
không	có	có	có	6.33 → 0.37	**2.049**
không	có	có	không	5.07 → 0.42	2.101
có	không	không	không	8.35 → 0.49	2.155
Train từ đầu (random init)				12.27 → 2.34	3.953

(Hàng cuối: ghi chú của paper nói hàng này cho biết sự thay đổi loss với Nemotron-3 8B; bản text hơi khó đọc nên diễn giải ở đây là: khởi tạo ngẫu nhiên và train cùng ngân sách thì loss rất tệ.)

Đọc bảng:

- **Ngay sau khi cắt (trước retrain)**, cấu hình "chỉ MLP + ATT" (5.07) và "depth + tất cả" (5.35) có distillation loss *thấp hơn* "width đầy đủ" (6.33). Nghĩa là nhìn ngay sau cắt, giữ embedding nguyên và/hoặc có cắt layer có vẻ tốt hơn.
- **Sau retrain 400 bước**, thứ hạng đảo: width đầy đủ (MLP+ATT+EMB, không cắt layer) cho LM loss thấp nhất 2.049; thêm cắt depth thì 2.062; chỉ cắt MLP+ATT (không cắt EMB) 2.101; chỉ cắt depth 2.155.
- Hai "đảo" khác nhau, đều được tác giả nêu: (i) cắt neuron + head thôi thì ban đầu hơn cắt cả embedding, nhưng sau vài bước retrain thì đảo; (ii) width hơn depth, nhưng chỉ sau retrain.



Hình 70: Bốn kiểu cắt Nemotron-4 15B xuống cỡ 8B: distillation loss ngay sau khi cắt (kèm giá trị sau retrain) và LM validation loss sau retrain 400 bước (Table 1) (vẽ lại từ số liệu trong ghi chú)

Cách đọc hình: panel trái xếp theo distillation loss ngay sau khi cắt, nơi cấu hình không cắt embedding (5.07) và cấu hình có cắt layer (5.35) trông tốt hơn width đầy đủ (6.33). Panel phải là LM validation loss sau retrain, nơi width đầy đủ thấp nhất (2.049) nên thứ hạng đã đảo. Trục phải bắt đầu từ 2.0 nên chênh lệch nhìn lớn hơn thực tế.

4.2 Depth-only so với width-only (Table 10)

Bỏ 16 layer ít quan trọng nhất của 15B theo 2 chỉ số, so với cắt width, so với kết hợp (bỏ 4 layer + width):

Mô hình Minitron 8B	Tham số	LM loss (sau retrain 1.8B token)
Depth (PPL)	9.39B	2.155
Depth (BI)	9.39B	2.177

Mô hình Minitron 8B	Tham số	LM loss (sau retrain 1.8B token)
Width	7.74B	**2.049**
Depth + Width	7.91B	2.062

Tác giả nhấn mạnh: width-only có *ít tham số nhất* mà vẫn tốt nhất. Hai biến thể depth còn nhiều tham số hơn (9.39B) nhưng loss kém hơn rõ. Hình 6: so width-alone (32 layer) với width + 28 layer, thứ hạng đảo trước bước ~200 (~0.8B token) của retrain, nên "cần một giai đoạn retrain nhẹ trước khi so sánh".

4.3 Hệ quả thực hành mà tác giả rút ra

- **Best practice #4:** "Prefer width pruning over depth for the model scales we consider (≤ 15B)".
- Không so sánh các lựa chọn cắt ngay sau khi cắt; phải retrain nhẹ trước khi xếp hạng (best practice #9). Hình 9 (retrain 15 ứng viên 8B): thứ hạng thay đổi đáng kể đến khoảng bước 300 rồi mới ổn định.
- Cắt sâu quá nhiều thì cũng mất: Phụ lục A.5 (hình 7): với depth, Hellaswag và PIQA giữ được tới 31 layer, MMLU giữ tới 26 layer rồi giảm mạnh ở ~20 layer. Nghĩa là vài layer có thể bỏ gần như không mất mát với retrain tối thiểu (khớp với ShortGPT).
- Khi kiến trúc đích vẫn dùng bản depth-pruned: dùng one-shot (bỏ 16 layer một lần rồi distill 30.2B token) tốt hơn lặp (bỏ 1 layer, distill 1.8B, lặp 16 lần), kể cả khi lặp nhận 86.4B token.

4.4 Giới hạn của kết luận depth vs width

- Chỉ là một họ mô hình (Nemotron-4, transformer thuần, ≤15B), một lượng retrain 1.8B token (so sánh chính), và tập đánh giá là loss/benchmark ngôn ngữ. Ngoài ra, cấu hình "depth-only" ở đây nặng hơn (9.39B so với 7.74B), nên là so sánh không cân bằng tham số (nhưng thiên về bất lợi cho width, nên kết luận width thắng càng mạnh). Đây là một nhận xét thêm.
- Paper chưa phân tích *tại sao*; ghi nhận chỉ là thực nghiệm.

5. Các lựa chọn loss distillation (chi tiết)

5.1 Distillation so với huấn luyện thường (Table 11)

Cắt xuống 4B, so các cách (∗ = cùng lượng compute; distillation tốn thêm forward của teacher nên nhận ít token hơn):

Cách	Token	Hellaswag	MMLU
4B random init	150B∗	46.22	24.36
4B random init	400B	48.23	26.24
4B Pruned (cắt từ 15B), retrain thường	150B∗	50.85	24.57
4B Pruned-Distill (cắt từ 15B)	100B∗	51.04	37.81
4B Pruned-Distill (cắt từ Minitron 8B)	100B∗	52.04	42.45

Tác giả diễn giải: prune + distill hơn train từ đầu cùng compute +4.8% Hellaswag, +13.5% MMLU; cắt từ 8B thay vì 15B thêm +1% và +4.6%. MMLU của random-init và pruned-không-distill đều gần ngẫu nhiên (~24-26%).

5.2 Hàm khoảng cách trên logits (Bảng 15-16, thí nghiệm trên Nemotron-3 8B và Minitron 8B-Depth-pruned)

Thử KLD (forward KL), reverse KLD (R-KLD), MSE, cosine; ngoài ra WSL-KD cũng kém hơn KLD.

Loss (Table 15, Nemotron-3 8B)	LM loss	WikiText PPL
$L_{CLM} + L_{logits}$ (MSE)	2.144	9.007
$L_{CLM} + L_{logits}$ (R-KLD)	2.140	9.008
$L_{CLM} + L_{logits}$ (Cosine)	2.134	8.965
$L_{CLM} + L_{logits}$ (KLD)	2.117	8.791
L_{logits} (KLD) *chỉ* logits	2.107	8.720

Table 16 (Minitron 8B-Depth-pruned): L_{logits} (R-KLD) 2.665 so với KLD 2.155. R-KLD (reverse, "mode-seeking") được báo là tốt cho SFT/instruction ở các công trình khác, nhưng với retrain mô hình *base* sau prune thì forward KLD tốt nhất. Kết luận: **dùng chỉ forward-KLD, không cộng** L_{CLM}.

5.3 Tham số logit-KD khác

- Nhiệt độ $\tau \in \{0.1, 0.5, 1.0, 3.0\}$: tốt nhất $\tau = 1.0$. Lập luận của tác giả: logit của LLM có entropy cao hơn phân loại ảnh (vốn "nhọn"), nên không cần làm mềm bằng $\tau > 1$.
- Top-K (giữ K logit lớn nhất của teacher): K nhỏ (<=100) làm giảm mạnh độ chính xác; K lớn thì không hơn việc không dùng top-K. Bỏ top-K.

5.4 Loss trên trạng thái trung gian L_{is} (Appendix A.4, Table 17-18)

Quan sát: output của block (L_o) giúp ích; 1-2 layer cuối "chuyên biệt hoá" nên ánh xạ cặp layer (last-2):(last-2) tốt nhất; loss embedding L_{emb} giúp; loss attention-relation (L_{att} từ Q/K/V) không cải thiện; loss trên input MLP không khác; ánh xạ nhiều layer teacher vào 1 layer student không hơn hoặc xấu đi; cosine là tốt nhất cho phần này. Ví dụ Table 17 (Minitron 8B-Depth-pruned): L_{logits} một mình 2.155; $+L_o(29{:}13)$ 2.145; $+L_o(28{:}12) + L_{emb}$ hoặc $L_o(29{:}13) + L_{emb}$: 2.141.

Nhưng với mô hình *không cắt depth nhiều* (Minitron 8B và 4B), Table 18 cho: $L_{logits} + L_{is}$ 58.0/73.6/26.8 (MMLU/HellaSwag/HumanEval, 18.9B token) so với L_{logits} 58.3/73.8/26.2; tăng token lên 94B với L_{logits} cho 62.8/79.7/30.4. Nên:

- **Best practice #6:** dùng logit + trạng thái trung gian + embedding khi *cắt depth mạnh*.
- **Best practice #7:** chỉ dùng logit khi không cắt depth nhiều.

(Chú ý: tác giả nói nghiên cứu "depth-pruned" mạnh dùng L_{is} giúp chút ít; bảng 17 chênh nhỏ, ~0.01-0.015 trong LM loss.)

6. Thí nghiệm và kết quả chính

Mô hình nguồn: Nemotron-4 15B (15.6B tham số), đích ~8B và ~4B. Dữ liệu: bộ 8T token pre-training của Nemotron-4 + dữ liệu continued-training (CT); ablation dùng bộ 8T; mô hình cuối dùng cả hai. Retrain nhẹ: 1.8B token (400 bước). Tối ưu hoá theo Nemotron-4, cosine LR từ 2e-4 xuống 4.5e-7. Chạy trên 16 node DGX A100 (8x A100 80GB). Triển khai trong Megatron-LM.

Kiến trúc (Table 5):

Mô hình	Layers	Hidden	Att. heads	Query groups	MLP hidden	Params
Nemotron-4 15B	32	6144	48	8	24576	15.6B
Nemotron-3 8B	32	4096	32	32	16384	8.5B
Minitron 8B	32	4096	48	8	16384	8.27B
Minitron 4B	32	3072	24	8	9216	4.19B

Kết quả cuối Minitron 8B *không cắt layer nào* (vẫn 32 layer): toàn bộ nén đến từ width (hidden, MLP, head), nhất quán với best practice #4.

Minitron 8B vs các mô hình khác (Table 2, token train: Minitron 94B so với Nemotron-3 8B 3.8T, Llama-3 8B >15T, Llama-2 7B 2T, Mistral 7B 8T, Gemma 7B 6T, Nemotron-4 15B 8T):

Benchmark	Llama-3 8B	Llama-2 7B	Mistral 7B	Gemma 7B	Nemotron-4 15B	Nemotron-3 8B	Minitron 8B
Winogrande (5)	77.6	74	78.5	78	83.6	75.9	79.0
ARC-C (25)	57.8	53	60.3	61	58.8	52.8	52.6
MMLU (5)	65.3	46	64.1	64	66.6	54.7	63.8
HellaSwag (10)	82.1	79	83.2	82	84.6	78.5	80.7
GSM8K (5)	50.3	14	37	50	48.5	24.0	51.3
TruthfulQA (0)	43.9	39	42.6	45	40.7	36.5	42.6
XLSum en (20%)	30.9	31	4.80	17	32	30.9	31.2
MBPP (0)	42.4	20	38.8	39	38	27.04	35.2
HumanEval (0)	28.1	12	28.7	32	35.4	20.7	31.6

Minitron 4B (Table 3, 4.2B, 94B token): Winogrande 74.0, ARC-C 50.9, MMLU 58.6, HellaSwag 75.0, GSM8K 24.1, TruthfulQA 42.9, XLSum 29.5, MBPP 28.2, HumanEval 23.3. So với Phi-2 2.7B (MMLU 57.5, GSM8K 55, HumanEval 50) và Gemma 2.5B (MMLU 42): Minitron 4B nhỉnh hơn ở MMLU, kém ở GSM8K/code.

So với các mô hình đã prune (Table 4): cỡ 8B: LLM-Pruner/SliceGPT/LaCo/ShortGPT (~9.8B) MMLU lần lượt 25.2/37.1/45.9/54.7 so với Minitron 8B 63.8 (HellaSwag 67.8/55.7/64.4/66.6 so với 80.7). Cỡ 4B: MMLU 23.33/28.92/26.45/43.96 so với 58.6 (Sheared-LLaMA 2.7B: MMLU 26.4). Chú ý các đối thủ lớn hơn về số tham số (non-embedding 9.5B so với 6.2B).

Instruction-tuned (Minitron 4B-instruct): MT-Bench 6.46 (Phi-2 4.29, StableLM 2 Chat 5.42, Gemma-2B-IT 5.19...); IFEval prompt-level strict 68.76 / loose 73.01 / instruction-level loose 81.29; ChatRAG-Bench trung bình 41.11 (Gemma-2B-IT 33.31); BFCL v2 trung bình 53.09 (Gemma-2B-IT 41.63, Llama-3-8B-instruct 50.51).

Tiết kiệm chi phí cho cả họ: FLOPs mỗi bước cho 15B/8B/4B là 4.4e17 / 2.5e17 / 1.2e17. Nếu 8B và 4B cần 40 lần ít token hơn: tiết kiệm 1.8 lần cho toàn họ Nemotron-4.

7. Phân tích / ablation còn lại

- **Hàm gộp importance (Table 13, Nemotron-15B cắt về 8B, không retrain):** (batch=L2, seq=mean) cho LM loss 7.18 (8T) / 7.23 (WikiText2); (mean, mean) 7.21 / 6.89; tệ nhất (var, var) 10.55 / 11.14; (L2, L2) 8.73 / 8.37. Hình 5: sau retrain 400 bước, (L2,mean) vẫn tốt hơn (L2,L2): thứ hạng giữ nguyên. Chọn (L2, mean). **Best practice #2.**
- **Importance lặp (Table 14):** cắt embedding một lần (T=1) hay lặp T=2, T=4: loss zero-shot ban đầu 5.43/5.55/5.24 nhưng sau retrain đều 1.92. Không có lợi ích. **Best practice #3.**
- **Cắt một lần hay lặp theo kích thước (Best practice #8):** 15B → 4B là giảm 73.3% trọng số; cắt 15B → 8B (~46%) rồi 8B → 4B (~50%) cho MMLU cao hơn ~12% so với one-shot (Table 11, hai hàng cuối). Teacher cuối vẫn tốt hơn là 15B chứ không phải 8B.

- **Lặp qua các chiều:** cắt embedding trước rồi MLP+ATT (retrain sau mỗi bước) *không* hơn cắt cả ba một lần (Hình 8).
- **Search với retrain (#9):** thứ hạng 15 ứng viên 8B thay đổi tới ~300 bước rồi ổn.
- **Đa pha (#10, Table 20):** cắt checkpoint pha 2 (đã qua dữ liệu sạch) và retrain chỉ bằng dữ liệu pha 2 (94B token: MMLU 61.9, HumanEval 30.5) tốt hơn cắt pha 1 rồi retrain với cả hai pha (113B: MMLU 54.7, HumanEval 25.6); HellaSwag 80.1 vs 80.3 và PIQA 76.7 vs 77.2 gần như ngang.
- **Cắt từ mô hình gần kích thước đích nhất (#8)** và **luôn chỉ dùng distillation (#5)** như đã nêu.

Danh sách 10 best practices (nguyên văn rút gọn): (1) train mô hình lớn nhất, prune+distill lặp xuống nhỏ hơn; (2) importance (batch=L2, seq=mean) cho width, PPL/BI cho depth; (3) importance one-shot; (4) ưu tiên width hơn depth (≤15B); (5) retrain hoàn toàn bằng KLD; (6) logit+trung gian+embedding khi cắt depth nhiều; (7) chỉ logit khi không cắt nhiều depth; (8) cắt từ mô hình gần kích thước đích; (9) retrain nhẹ để ổn định xếp hạng khi search; (10) nếu có pre-train đa pha, cắt checkpoint pha cuối.

8. Giới hạn

Tác giả nêu hoặc ngầm thừa nhận:

- Phạm vi kết luận “width > depth” đặt rõ là cho quy mô ≤15B.
- Không thể thử loss attention-score do abstraction của FlashAttention.
- Chưa thử LoRA/PEFT ở bước retrain nhẹ; chưa dùng genetic/Bayesian search.

Nhận xét:

- Chỉ một họ mô hình, transformer thuần (không hybrid/SSM/linear-attention); không có bài kiểm tra cho VLM hay đầu vào ảnh.
- Các bảng so sánh với đối thủ đã prune có số tham số không bằng nhau, và đối thủ không được retrain với distillation; chưa tách đóng góp của “importance” và “distillation”.
- 94B token vẫn là rất lớn so với ngân sách nhỏ; paper không cho đường cong lợi ích theo token ở mức vài tỷ.
- Nhiều kết luận (ví dụ #3, #5) dựa trên một vài cấu hình; không có thống kê nhiều seed.
- Số liệu “Nemotron-3 8B loss” ở hàng cuối Table 1 hơi mơ hồ do định dạng text.

9. Liên hệ với các bài khác

- **38 Sheared LLaMA:** cùng triết lý “prune rồi train tiếp”, nhưng Sheared học mask (gradient, đắt), chỉ LM loss; Minitron dùng activation importance (forward) và KD. Bảng 4 của Minitron so trực tiếp với Sheared.
- **31 ShortGPT, 33 Shortened LLaMA, 35 LaCo, 32 Deeper layers:** các chỉ số cho depth (BI, PPL, layer collapse) được dùng lại; Minitron cho thấy depth-only *bị thắng* bởi width khi được retrain.
- **34 SLEB, 37 Repair changes which layers:** liên quan đến việc chọn layer bỏ có thay đổi sau sửa chữa; phát hiện “thứ hạng đảo sau retrain” của Minitron nhất quán với ý này.
- **28 FLAP, 29 SliceGPT, 26 LLM-Pruner:** các chỉ số importance cho width; Minitron là ví dụ “chỉ forward”.
- **40 Llama-3.1-Minitron, 41 Flextron, 42 Puzzle:** các bài kế tiếp; Puzzle thay việc “chọn tay” bằng tìm kiếm cấu hình từng layer.

10. Gợi ý áp dụng

- Cần kiểm tra thay vì tin một trong hai bên.
- Chỉ là giả thuyết.

11. Thí nghiệm gợi ý

Vẽ đường cong để xem có đảo thứ hạng không và ở bước nào.

2. **Loss ablation nhỏ:** cùng một mô hình đã cắt, so KL-only, KL+CE, reverse-KL, và KL + cosine trên hidden cuối (như Table 15/17), cùng số bước, báo CER.

12. Tóm tắt công thức / khái niệm

Khái niệm	Nội dung
Activation-based importance	Chuẩn activation theo head/neuron/kênh trên 1024 mẫu, chỉ forward
Aggregation (batch, seq)	(L2, mean) tốt nhất sau retrain
Block Importance	$1 - \mathbb{E}\cos(X_i, X_{i+1})$
Gộp head	$\text{head}_i + (\text{head}_i - \text{head}_{2K-i+1})$
Lightweight NAS	liệt kê <20 cấu hình, retrain 1.8B token, xếp hạng sau ~300 bước
Loss tổng	$L = L_{CLM} + L_{logits} + \alpha L_{is}$, $\alpha = L_{logits}/L_{is}$; thực hành: chỉ L_{logits} (KLD, $\tau = 1$)
Width > depth	Table 1/10: width 2.049 vs depth 2.155, chỉ sau retrain
One-shot > iterative	cho importance, depth, cross-axis
Cost	40x ít token hơn cho mỗi mô hình thêm; 1.8x rẻ hơn cho cả họ

13. Kết luận ngắn

Minitron cho thấy prune (activation-based, forward-only) + huấn luyện lại bằng KL-distillation trên logits là một phương pháp hiệu quả, với bằng chứng ablation cho từng lựa chọn. Một lưu ý quan trọng là: **không đánh giá lựa chọn cắt ngay sau khi cắt**, vì thứ hạng đảo sau vài trăm bước retrain; và trong thiết lập này width thắng depth.

Link gốc: https://arxiv.org/abs/2407.14679

Bài 40

LLM Pruning and Distillation in Practice: The Minitron Approach (2024)

Tác giả: Sharath Turuvekere Sreenivas, *Saurav Muralidharan*, Raviraj Joshi, Marcin Chochowski, Ameya Sunil Mahabaleshwarkar, Gerald Shen, Jiaqi Zeng, Zijia Chen, Yoshi Suhara, Shizhe Diao, Chenhan Yu, Wei-Chun Chen, Hayley Ross, Oluwatobi Olabiyi, Ashwath Aithal, Oleksii Kuchaiev, Daniel Korzekwa, Pavlo Molchanov, Mostofa Patwary, Mohammad Shoeybi, Jan Kautz, Bryan Catanzaro (NVIDIA). (* đóng góp ngang nhau)
Venue / arXiv: arXiv:2408.11796 (v1: 21/08/2024; bản v4: 09/12/2024). Là technical report của NVIDIA; không ghi hội nghị.
Nguồn: https://arxiv.org/abs/2408.11796
Chủ đề: Áp dụng công thức Minitron lên Llama 3.1 8B (xuống 4B, hai biến thể depth và width) và Mistral NeMo 12B (xuống 8B) khi *không có dữ liệu pre-training gốc*; đề xuất "teacher correction" và chỉ số layer dựa trên tác vụ downstream.
Vị trí trong lộ trình: Đọc trước: 39 (Minitron, bắt buộc; bài này bỏ qua phần giải thích lại). Bài này thêm: (1) teacher correction, (2) cách chọn khối layer liên tiếp theo Winogrande, (3) bằng chứng thứ hai về width > depth về độ chính xác nhưng depth > width về tốc độ. Tiếp theo: 41 (Flextron), 42 (Puzzle).

> Ghi chú học tập bằng tiếng Việt, diễn giải và tóm tắt, không phải bản dịch. Số liệu lấy từ paper (một số bảng bị lệch cột trong bản text; đã đối chiếu).

1. Bối cảnh

- Bài 39 cho thấy prune + chưng cất rẻ hơn huấn luyện từ đầu rất nhiều, nhưng giả định có **dữ liệu pre-training gốc** của mô hình để distill. Các mô hình mở mạnh (Llama 3.1, Mistral NeMo) lại huấn luyện trên dữ liệu riêng, không công bố. Câu hỏi: nếu chỉ có một tập dữ liệu khác (ở đây là tập continued-training của Nemotron-4), distillation còn hiệu quả không?
- **Distribution shift (lệch phân phối)** giữa dữ liệu teacher đã học và dữ liệu đang dùng để distill: tác giả giả thuyết rằng phân phối sub-word token khác nhau khiến teacher đưa ra "hướng dẫn" (logits) kém tối ưu.

2. Ý tưởng chính

Trước khi prune, **fine-tune nhẹ teacher** (~100B token) trên chính tập dữ liệu sẽ dùng để distill; đây là **teacher correction**. Sau đó prune (activation-based, theo Minitron) và chưng cất bằng forward-KL trên logits. Ngoài ra đề xuất chọn layer để bỏ bằng *độ chính xác downstream* (Winogrande) trên các **khối layer liên tiếp** thay vì dùng LM loss/PPL, vì LM loss chọn ra những layer rời rạc nhưng cho kết quả downstream tệ. Áp dụng để tạo Llama-3.1-Minitron-4B (width và depth) và MN-Minitron-8B.

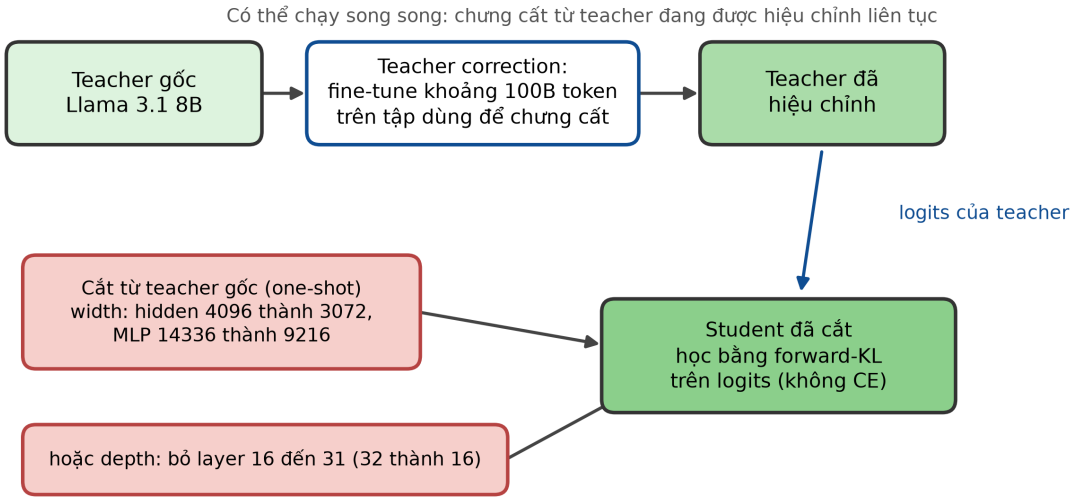
3. Phương pháp chi tiết

3.1 Pipeline (Hình 1)

```
1. Teacher correction : fine-tune teacher gốc ~100B token trên tập D_distill   (lr = 1/5
↪  peak, warm-up 120 bước)
2. Pruning            : importance (activation, 1024 mẫu calibration) -> xếp hạng -> cắt
↪  (one-shot)
```

```
                        width : L2 (batch) + mean (seq) ; depth : bỏ khối layer liên
                        ↪  tiếp theo Winogrande
3. Distillation       : student (đã cắt) học từ teacher đã hiệu chỉnh, chỉ forward-KL
↪  trên logits
```

Bỏ bước NAS của Minitron; dùng cấu hình thủ công lấy cảm hứng từ Minitron 4B/8B. Depth được xem là trường hợp riêng: **không kết hợp depth với các trục khác**.



Hình 71: Quy trình khi không có dữ liệu pre-training gốc: teacher correction, cắt width hoặc depth, rồi chưng cất bằng forward-KL (minh hoạ, tự tính)

Cách đọc hình: hàng trên là bước hiệu chỉnh teacher bằng khoảng 100B token trên chính tập dùng để chưng cất. Hàng dưới là hai kiểu cắt (width hoặc depth) cho student, student học từ logits của teacher đã hiệu chỉnh. Ghi chú ở đầu hình nhắc rằng có thể chạy hiệu chỉnh và chưng cất song song.

3.2 Teacher correction

- Fine-tune teacher gốc ~100B token với learning rate bằng 1/5 mức đỉnh, cùng batch size, min LR và lịch decay của mô hình gốc.
- Tác dụng lên teacher: ảnh hưởng nhỏ đến accuracy downstream (có tác vụ lên, có tác vụ xuống, Bảng 1); tác giả giả thuyết phần xuống là do dữ liệu fine-tune.
- Có thể làm **song song** với distillation (prune teacher gốc rồi distill từ teacher đang được hiệu chỉnh liên tục) và cho kết quả ngang với dùng teacher đã hiệu chỉnh xong (Hình 5).

3.3 Chọn layer để bỏ (depth)

Hai chỉ số được xét: LM validation loss/PPL và accuracy tác vụ downstream. Không dùng Block Importance (BI) vì "đã được chỉ ra là kém hơn PPL". Hai quan sát dẫn tới quy tắc cuối:

1. Importance theo LM loss/PPL không cho mô hình cắt tốt nhất trên downstream.
2. Bỏ khối *liên tiếp* tốt hơn bỏ rời rạc (đồng ý với Gromov et al., bài 32).

Quy tắc: bỏ khối layer liên tiếp làm Winogrande giảm ít nhất. Với Llama 3.1 8B (32 layer), kết quả là bỏ layer 16-31 (nửa sau, giữ lại 16 layer).

3.4 Cấu hình đích (Table 3)

	Llama-3.1-Minitron 4B-Width	4B-Depth	MN-Minitron-8B
Tổng tham số	4.5B	4.5B	8.4B
Không tính embedding	3.7B	3.5B	7.3B
Hidden size	3072	4096	4096
MLP hidden	9216	14336	11520
Depth	32	16	40
Query heads / nhóm / head dim	32 / 8 / 128	32 / 8 / 128	32 / 8 / 128
Vocabulary	128256	128256	131072

Nguồn: Llama 3.1 8B (hidden 4096 -> 3072, MLP 14336 -> 9216, head và depth giữ nguyên cho bản Width; depth 32 -> 16 cho bản Depth). MN-Minitron-8B từ Mistral NeMo 12B: hidden 5120 -> 4096, MLP 14336 -> 11520.

3.5 Distillation và siêu tham số (Table 4)

Chỉ forward-KL trên logits, *bỏ hẳn* cross-entropy LM (theo best practice của bài 39).

	Llama-3.1-Minitron	MN-Minitron
Peak LR	1e-4	1e-4
Min LR	1e-5	4.5e-7
Warm-up	40 bước	60 bước
Decay	cosine	cosine
Global batch	1152	768
Context	8192	8192
Tổng token	94B	380B

Phần cứng: 32 node DGX H100. Dữ liệu: Nemotron-4 continued-training (CT) dataset cho cả teacher correction, pruning và distillation.

4. Thí nghiệm và kết quả chính

Model base (Table 1, trích; đánh giá do tác giả chạy, trừ Gemma2 2B*):

Benchmark	Gemma2 2B*	Minitron 4B	Llama-3.1-Minitron 4B-Depth	4B-Width	Llama 3.1 8B	MN-Minitron 8B	Mistral NeMo 12B-Base	12B-FT
Tham số tổng	2.6B	4.2B	4.5B	4.5B	8B	8.4B	12.2B	12.2B
Token train	2T	94B	94B	94B	15T	380B	-	+0.1T
Winogrande (5)	70.9	74.0	72.1	73.5	77.3	80.4	82.2	82.7
ARC-C (25)	55.4	50.9	52.6	55.6	57.9	64.4	65.1	62.3
MMLU (5)	51.3	58.6	58.7	60.5	65.3	69.5	69.0	70.1
HellaSwag (10)	73.0	75.0	73.2	76.1	81.8	83.0	85.2	85.3

Benchmark	Gemma2 2B*	Minitron 4B	Llama-3.1-Minitron 4B-Depth	4B-Width	Llama 3.1 8B	MN-Minitron 8B	Mistral NeMo 12B-Base	12B-FT
GSM8K (5)	23.9	24.1	16.8	41.2	48.6	58.5	56.4	55.7
TruthfulQA (0)	-	42.9	38.2	42.9	45.0	47.6	49.8	48.3
MBPP (0)	29.0	28.2	30.7	32.4	42.3	43.8	42.6	47.9

(Bảng gốc có thêm Gemma 7B và Mistral 7B và hàng XLSum, HumanEval; bản text hơi lệch cột nên ở đây chỉ trích các cột đã đối chiếu chắc chắn. Cột "12B-FT" là Mistral NeMo 12B sau teacher correction.)

Điểm nhấn do tác giả nêu:

- MN-Minitron-8B vượt Llama 3.1 8B với 380B token (so với 15T), tức khoảng 40 lần ít hơn. Hai bài kiểm tra GSM8K (55.7% -> 58.5%) và HumanEval (23.8% -> 36.2%) cao hơn cả teacher Mistral NeMo 12B; tác giả cho là có ảnh hưởng của dữ liệu.
- Llama-3.1-Minitron-4B dùng 94B token, khoảng 150 lần ít hơn 15T của Llama 3.1 8B; cả hai biến thể hơn Minitron 4B đời trước (ở nhiều tác vụ).
- Width vs depth cho cùng số tham số: MMLU 60.5% vs 58.7%; **GSM8K 41.2% vs 16.8%** (suy luận toán bị ảnh hưởng lớn khi bỏ nửa số layer); khoảng cách thu hẹp sau instruction tuning.

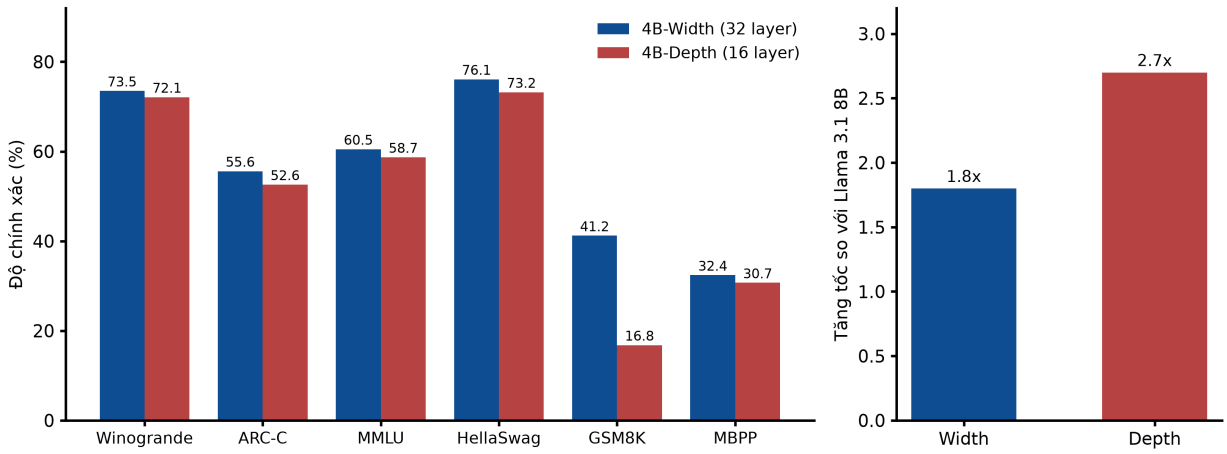
Model instruct (Table 2, trích):

Benchmark	Phi-2 2.7B	Gemma2 2B	Qwen2 1.5B	Minitron 4B	LL-Minitron 4B-Depth	4B-Width	Llama 3.1 8B	MN-Minitron 8B
MT-Bench	5.14	7.44	5.49	6.46	6.19	6.88	7.78	7.86
MMLU (5)	56.8	56.9	55.6	59.3	61.21	59.89	69.4*	70.4
GSM8K (0)	19.9	52.2	27.2	65.1	71.11	79.76	83.8	87.1
GPQA (0)	28.8	25.9	28.1	29.5	32.59	30.36	30.4*	31.5
IFEval	44.0	64.5	39.8	75.3	66.77	79.54	80.4*	84.4
BFCLv2 (live)	38.7	40.2	39.9	53.1	55.89	55.0	44.3	67.6

Instruction tuning dùng NeMo-Aligner: SFT toán + code, SFT instruction, rồi hai vòng Reward-aware Preference Optimization (RPO). Tác giả: mô hình aligned của họ hơn các mô hình cùng cỡ ở hầu hết benchmark, ngoại trừ HumanEval và MBPP; Llama-3.1-Minitron-4B thua Gemma2 ở MT-Bench.

Tốc độ (TensorRT-LLM, FP8, một H100 80GB, batch 32 cho 8B-12B và 64 cho 4B): 4B-Depth nhanh nhất, trung bình 2.7x so với Llama 3.1 8B; 4B-Width 1.8x (tóm tắt Insights ghi 1.7x ở một chỗ và 1.8x ở chỗ khác); FP8 nhanh hơn BF16 1.4x.

Cách đọc hình: panel trái so hai bản cùng số tham số, bản width cao hơn ở cả sáu benchmark và hơn hẳn ở GSM8K (41.2 so với 16.8). Panel phải là tăng tốc suy luận, bản depth nhanh hơn (2.7x so với 1.8x). Đây là đánh đổi độ chính xác lấy tốc độ.



Hình 72: Llama-3.1-Minitron 4.5B bản Width và Depth: độ chính xác trên sáu benchmark (Table 1) và tăng tốc suy luận so với Llama 3.1 8B (vẽ lại từ số liệu trong ghi chú)

5. Phân tích / ablation

- **Teacher correction (Hình 4-5):** distill từ teacher đã hiệu chỉnh cho loss validation thấp hơn rõ so với teacher gốc; tổng kết "Insights": hiệu chỉnh teacher trên dữ liệu distill cho **giảm hơn 6% LM validation loss**; không làm hỏng độ tối ưu của việc pruning; có thể làm song song.
- **Prune + distill vs lựa chọn khác (Hình 6, MN-Minitron-8B):** so (a) khởi tạo ngẫu nhiên + distillation, (b) prune ngẫu nhiên + distillation, (c) prune theo importance + LM loss thường, (d) prune theo importance + distillation. Kết luận: prune importance cho điểm khởi đầu tốt hơn nhiều so với ngẫu nhiên, và distillation hơn huấn luyện thường; hai lợi ích trực giao.
- **Width vs depth (Hình 7):** width có loss khởi đầu thấp hơn và luôn thắng depth với cùng số tham số trong suốt quá trình distill.
- **Chỉ số depth (Hình 8-9):** loại 16 layer rời rạc (chọn theo tác động nhỏ nhất lên LM loss) cho validation loss tốt hơn loại khối liên tiếp, **nhưng** accuracy Winogrande của mô hình loại 16 layer "tốt nhất theo importance" chỉ ngẫu nhiên 0.5, còn loại layer 16-31 liên tiếp được 0.595. Chênh lệch này vẫn còn sau distillation. Layer đầu và cuối là quan trọng nhất theo LM loss. (Đây là ví dụ rõ về: chỉ số proxy tốt không đồng nghĩa với downstream tốt.)
- **Giữ attention heads khi cắt width:** "retaining attention heads and pruning the other dimensions (MLP intermediate, embedding channels)" cho accuracy cao hơn (insight 3 cho width).

6. Giới hạn

Tác giả nêu:

- Teacher correction dùng ~100B token vẫn lớn; mong tối ưu bằng ít token hơn, LoRA hoặc chỉ tinh chỉnh LayerNorm (để dành cho công việc sau).
- Bỏ NAS; cấu hình chọn thủ công.
- Một số tác vụ của teacher giảm sau hiệu chỉnh (giả thuyết do dữ liệu).

Nhận xét:

- Cắt depth chỉ thử một kiểu (nửa sau 16-31) và chỉ chọn theo Winogrande, một tác vụ nhỏ; có nguy cơ overfit vào benchmark chọn.
- Không có ablation tách riêng "teacher correction" với các lượng token khác nhau, nên khó biết cần bao nhiêu cho miền hẹp.
- Vẫn là transformer thuần, văn bản; không có mô hình hybrid hoặc VLM.

- 94B-380B token distill vẫn ngoài tầm với ngân sách nhỏ.

7. Liên hệ với các bài khác

- **39 Minitron:** bài tiền nhiệm; ở đây các best practice được dùng lại gần như nguyên.
- **32 Deeper layers (Gromov et al.), 31 ShortGPT, 33 Shortened LLaMA:** nguồn của ý "bỏ khối layer liên tiếp, thường ở nửa sau". Bài này bất đồng nhẹ với ShortGPT về BI (không dùng BI).
- **37 Repair changes which layers:** cùng tinh thần "chọn layer bằng proxy trước khi sửa chữa có thể sai".
- **38 Sheared LLaMA:** cũng chia nhỏ domain/dữ liệu khi train tiếp; ở đây giải quyết khác: hiệu chỉnh teacher.
- **41 Flextron, 42 Puzzle:** thay cấu hình thủ công bằng tìm kiếm tự động theo tốc độ/bộ nhớ.

8. Gợi ý áp dụng

- Nên fine-tune nhẹ teacher trên tập distill trước (hoặc song song), đo CER của teacher trước/sau.
- Phần ViT: ít liên quan.

9. Thí nghiệm gợi ý

3. **Parallel correction:** chạy distill từ teacher đang fine-tune song song và so với teacher đã chỉnh trước (tái hiện Hình 5 ở quy mô nhỏ).

10. Tóm tắt công thức / khái niệm

Khái niệm	Nội dung
Teacher correction	Fine-tune teacher ~100B token trên tập distill (lr 1/5 đỉnh) trước khi distill
Corrected teacher song song	Prune teacher gốc, distill từ teacher được chỉnh liên tục, ngang kết quả
Loss	Chỉ forward-KL trên logits, không có CE
Chọn layer	Khối liên tiếp làm Winogrande giảm ít nhất; không dùng PPL, không dùng BI
Width pruning	L2 (batch) + mean (seq), one-shot, giữ attention heads
Kết quả	Llama-3.1-Minitron-4B: 94B token; MN-Minitron-8B: 380B token
Tốc độ	4B-Depth ~2.7x, 4B-Width ~1.8x (batch 32/64, H100, FP8)

11. Kết luận ngắn

Bài này biến công thức Minitron thành quy trình thực tế khi không có dữ liệu gốc: hiệu chỉnh teacher trước, chọn layer theo downstream và theo khối liên tiếp, rồi chưng cất chỉ bằng KL trên logits.

Link gốc: https://arxiv.org/abs/2408.11796

Bài 41

Flextron: Many-in-One Flexible Large Language Model (2024)

Tác giả: Ruisi Cai, Saurav Muralidharan, Greg Heinrich, Hongxu Yin, Zhangyang Wang, Jan Kautz, Pavlo Molchanov (NVIDIA; University of Texas at Austin).
Venue / arXiv: arXiv:2406.10260 (v1: 11/06/2024; bản v2: 28/08/2024); Proceedings of ICML 2024 (PMLR 235), theo trang đầu của paper.
Nguồn: https://arxiv.org/abs/2406.10260
Chủ đề: Biến một LLM đã train thành mạng "elastic" lồng nhau (nested): một bộ trọng số, nhiều mô hình con theo ngân sách độ trễ; router chọn mô hình con tĩnh (theo ngân sách) hoặc động (theo từng token); surrogate model để huấn luyện router.
Vị trí trong lộ trình: Đọc trước: 39 (Minitron: importance dựa trên activation, cùng nhóm tác giả), 17 (CoFi), 38 (Sheared LLaMA). Bài này khác hướng "prune một lần ra một mô hình": nó tạo **cả họ mô hình con trong một checkpoint**. Tiếp theo: 42 (Puzzle, tìm kiến trúc không đồng nhất theo từng layer trên phần cứng đích).

> Ghi chú học tập bằng tiếng Việt, diễn giải và tóm tắt, không phải bản dịch. Số liệu lấy từ paper.

1. Bối cảnh

- Muốn phục vụ nhiều thiết bị (điện thoại, laptop, GPU đám mây) thì phải huấn luyện hoặc prune nhiều mô hình khác kích thước. Tốn.
- **Elastic network / supernet** (Once-for-All, SortedNet, MatFormer): một mạng chứa nhiều mạng con lồng nhau; triển khai chỉ việc "cắt lát" theo ngân sách. Vấn đề: kiến trúc không chuẩn và thường huấn luyện lâu hơn một mô hình bình thường.
- **Mixture-of-Experts (MoE)**: nhiều "chuyên gia" (expert) trong một lớp, router chọn expert theo từng token (input-adaptive). Nhưng thường chỉ áp dụng cho FFN, expert cùng kích thước, trọng số lưu riêng nên tốn bộ nhớ/băng thông.
- **Router**: mạng nhỏ đưa ra quyết định chọn nhánh nào. Khó huấn luyện vì gradient từ loss cuối đến router yếu; trong MoE có hiện tượng **expert collapse** (router luôn chọn một đường).
- Flextron muốn lấy điểm tốt của cả ba: elastic lồng nhau (một bộ trọng số), MoE (input-adaptive), và hiểu ngân sách độ trễ.

2. Ý tưởng chính

Lấy một LLM đã huấn luyện, xếp hạng head và neuron MLP theo độ quan trọng rồi **hoán vị** để phần quan trọng nằm đầu. Khi đó "mô hình con" chỉ cần lấy N head đầu và d neuron đầu của mỗi lớp (lồng nhau, dùng chung trọng số, chỉ cần slicing). Tiếp tục huấn luyện (continued training) với việc mỗi bước lấy mẫu ngẫu nhiên vài mô hình con cùng huấn luyện, để mọi mô hình con đều chạy tốt. Sau đó huấn luyện **router** nhận vào ngân sách độ trễ (và, nếu muốn, hidden state của token) để chọn độ rộng từng layer. Router được huấn luyện gián tiếp qua một **surrogate model** (mô hình thay thế dự đoán loss của LLM chỉ từ logit của router). Chỉ dùng 7.63% token so với pre-train gốc, kết quả vượt nhiều mô hình tương đương huấn luyện riêng.

3. Phương pháp chi tiết

3.1 Cấu trúc elastic lồng nhau

Mỗi layer có K ứng viên W_i^j, nên có K^N mạng con (ví dụ paper nhắc 4^{64} cho LLaMA2-7B 32 layer: MLP và MHA mỗi cái 4 mức).

Elastic MLP (lấy cảm hứng MatFormer):

$$\boxed{\text{MLP}_j(x) = \sigma(X \cdot I_{d_j} W^{(1)\top}) \cdot I_{d_j} W^{(2)}}$$

I_{d_j} là ma trận đường chéo $D \times D$ có d_j phần tử đầu bằng 1; thực tế là slicing $W^{(1)}[0 : d_j, :]$. Ràng buộc $d_1 < d_2 < \cdots < d_K = D$. Chọn $K = 4$ (ví dụ 25%, 50%, 75%, 100% độ rộng).

Elastic MHA: dùng d_j head đầu:

$$\text{MHA}_j(x) = \text{Concat}(\text{head}_1, \ldots, \text{head}_{d_j}) \cdot I_{d_j H} W^O$$

Paper cho rằng đây là công trình đầu tiên hỗ trợ cả MLP và MHA elastic cùng lúc. MHA ảnh hưởng nhiều tới độ trễ và KV cache.

3.2 Xếp hạng + hoán vị (importance)

Dùng 512 mẫu; độ quan trọng là tổng chuẩn L1 của activation (khác Minitron dùng L2/mean):

$$F^{(i)}_{head} = \sum_X \|\text{Attn}(XW_{Q,i}, XW_{K,i}, XW_{V,i})\|_1, \qquad F^{(i)}_{neuron} = \sum_X \|X^\top W^{(1),r}\|_1$$

Sau đó sắp xếp lại các hàng/cột trọng số theo thứ tự giảm dần importance, nên mô hình con nhỏ luôn giữ phần quan trọng nhất.

3.3 Elastic continued training

```
Input: LLM đã train, 512 mẫu để xếp hạng
1. permute head/neuron theo importance (giảm dần)
2. for step in 1..80000:                          # batch 256
       lấy mẫu ngẫu nhiên k=3 mạng con (one-hot s_i cho mỗi layer) + mạng đầy đủ
       L_joint = sum_{j=0}^{k-1} L(M_j(x), y)   # tích luỹ gradient cho 4 mạng, cập nhật
       ↪  1 lần
```

Lấy mẫu vì số mạng con quá lớn. Dùng 89.9B token (7.54% pre-train GPT-3, 4.50% với Llama-2).

3.4 Router và ràng buộc độ trễ

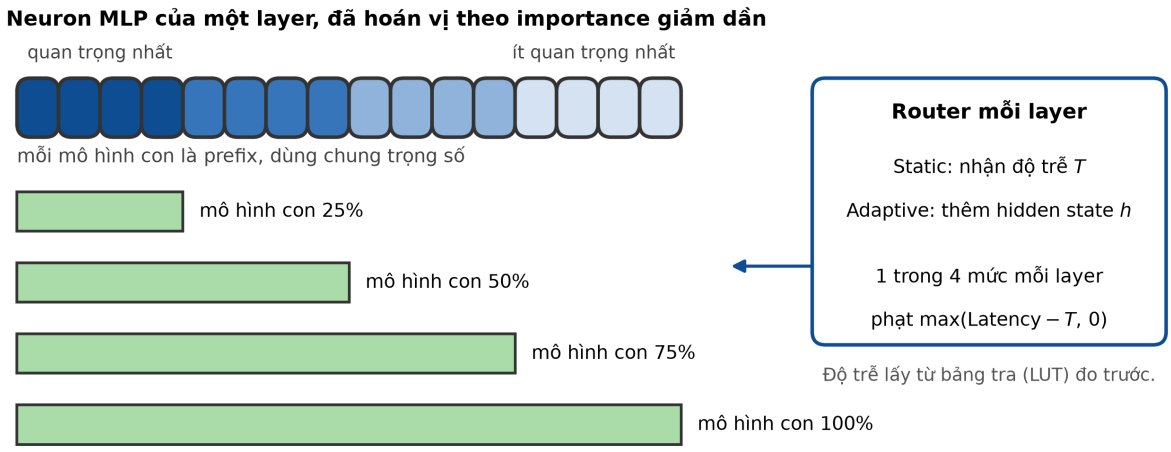
Bài toán: $\min_{S_t} \sum_t \mathcal{L}_{CE}(M_{s_t})$ với $\text{Latency}(M_{s_t}) \le T_t$, đưa về Lagrange:

$$\boxed{\mathcal{L} = \sum_t \mathcal{L}_{CE}(M_{s_t}) + \lambda \cdot \mathcal{T}_T(M_{s_t}), \quad \mathcal{T}_T = \sum_t \max(\text{Latency}(M_{s_t}) - T_t, 0)}$$

Độ trễ thật được lấy qua **lookup table (LUT)** đo trước. Ràng buộc có thể thay bằng bộ nhớ GPU.

- **Flextron-Static:** router mỗi layer là MLP nhỏ nhận *độ trễ mục tiêu* T và cho logit trên 4 ứng viên: $s_i = \arg\max R_i(T)$. Mọi token dùng chung một mô hình con.
- **Flextron-Adaptive (dynamic):** router nhận thêm hidden state h_i của token: $s_i = \arg\max R_i(T, h_i)$, với $R_i(T, h_i) = \sigma(T \cdot W + h_i W^\top_{H_i}) W_{R_i}$, chiều nhúng $U \le 128$. Quyết định theo từng token.

Cách đọc hình: dải ô phía trên là neuron MLP đã sắp từ quan trọng nhất đến ít quan trọng nhất. Mỗi thanh xanh lá là một mô hình con lấy phần đầu của dải, nên mọi mô hình con dùng chung trọng số. Khung bên phải mô tả router chọn một trong bốn mức cho mỗi layer theo ngân sách độ trễ.



Hình 73: Mô hình con lồng nhau của Flextron: neuron hoán vị theo importance, mô hình con là prefix 25/50/75/100%, router chọn mức cho từng layer (minh hoạ, tự tính)

3.5 Surrogate model (SM)

Vì gradient từ loss cuối về router yếu, thêm một mạng nhỏ dự đoán *loss LM của cả LLM chỉ từ logit router*: $r = \text{Concat}(R_0(T), \ldots, R_{N-1}(T))$, $S(r) = \sigma(rW_{S1}^{\top})W_{S2}$. Quy trình (Hình 3):

1. LLM đóng băng; router luôn nhận gradient từ latency loss; SM được cập nhật để dự đoán LM loss (MSE).
2. Khi sai số MSE của SM dưới ngưỡng, router còn được cập nhật bằng cách *giảm output của SM* (SM làm proxy khả vi cho loss thật).
3. Sau khi xong, bỏ SM và tinh chỉnh chung router + LLM (joint tuning).

Router tuning chỉ 1000 bước, batch 256, 1.049B token (static router chỉ cần dữ liệu một domain Wikipedia vì xếp hạng mô hình con ổn định giữa các domain; input-adaptive dùng một phần dữ liệu pre-train). Với Llama-2 thêm 2.62B token từ dữ liệu continued-training Nemotron-4 cho joint tuning.

4. Thí nghiệm và kết quả chính

Mô hình nền: GPT-3 2B (24 layer, hidden 2048, 16 head, 1.2B non-embedding), GPT-3 8B (32 layer, hidden 4096, 32 head, 6.4B non-embedding, vocab 320,000), Llama-2-7B (6.5B non-embedding). Context 4096.

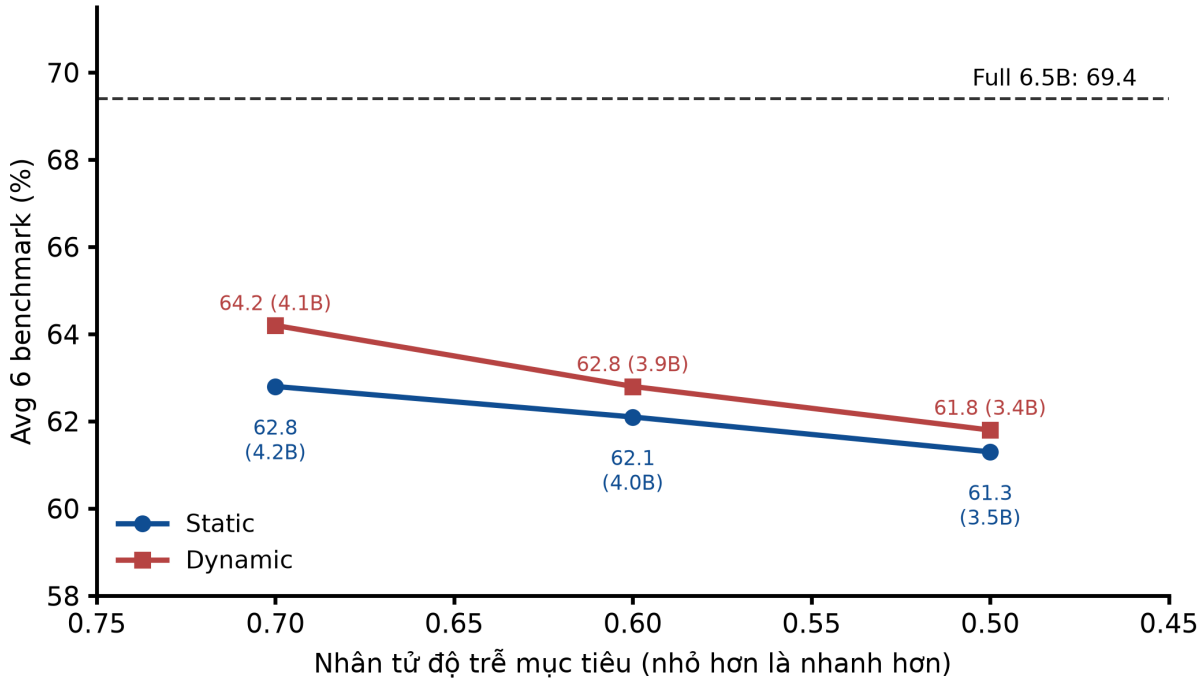
Bảng downstream (Table 1; "#Params" là non-embedding; model dynamic dùng số tham số kích hoạt trung bình; ×0.7 nghĩa là độ trễ còn 0.7):

Flextron-Llama2-7B:

	#Params	ARC-E	LAMBADA	PIQA	Winogrande	MMLU (5)	HellaSwag (10)	Avg
Full	6.5B	75.1	71.5	77.5	69.1	45.1	78.1	69.4
Static-0.7×	4.2B	65.8	64.2	75.6	62.3	41.9	67.1	62.8
Dynamic-0.7×	4.1B	68.6	65.1	76.1	63.7	42.2	69.4	64.2
Static-0.6×	4.0B	66.1	63.8	75.0	62.1	37.7	68.0	62.1
Dynamic-0.6×	3.9B	67.1	63.8	74.9	62.2	39.4	69.7	62.8

	#Params	ARC-E	LAMBADA	PIQA	Winogrande	MMLU (5)	HellaSwag (10)	Avg
Static-0.5×	3.5B	65.9	61.7	74.8	61.9	35.9	67.6	61.3
Dynamic-0.5×	3.4B	66.5	62.9	74.1	62.0	36.8	68.5	61.8

(Đơn vị %.) So sánh trong cùng bảng: Llama2-7B gốc 69.2 (avg); OpenLLaMA-3Bv2 (3.2B) 60.3; OpenLLaMA-7Bv2 (6.5B) 66.0; Sheared-LLaMA-2.7B (2.5B) avg 62.1 (MMLU 26.4); Pythia-2.8B 54.5. Các phương pháp nén: LLM-Pruner 4.5B (MMLU 23.9), Compresso 4.5B (25.9), LaCo 4.7B (26.5), SliceGPT 4.8B (28.9). Flextron-Llama2-7B ở 0.5× (3.5B) có MMLU 35.9% (static) / 36.8% (dynamic), cao hơn các phương pháp nén kia trong bảng. Bản Flextron-8B (GPT-3 8B): Full 66.8 avg, Static-0.5× (3.4B) 59.1, Dynamic-0.5× 60.0.



Hình 74: Flextron-Llama2-7B: trung bình sáu benchmark của bản Static và Dynamic ở các nhân tử độ trễ 0.7, 0.6, 0.5 (Table 1) (vẽ lại từ số liệu trong ghi chú)

Cách đọc hình: trục ngang đảo chiều, càng sang phải mô hình càng nhanh; số trong ngoặc là số tham số non-embedding. Bản Dynamic cao hơn Static ở cả ba mức nhưng khoảng cách thu hẹp từ 1.4 điểm ở 0.7 xuống 0.5 điểm ở 0.5. Đường nét đứt là mô hình đầy đủ (69.4).

Độ trễ thật (Table 2; TensorRT-LLM, A100 80GB, batch 1, prompt 8 / sinh 512; "tham số / giây"):

	Full	0.7×	0.6×	0.5×
Flextron-8B	6.4B / 10.43s	4.1B / 8.02s	3.9B / 6.39s	3.4B / 5.48s
Flextron-Llama2-7B	6.5B / 9.64s	4.1B / 7.09s	3.9B / 5.41s	3.4B / 4.91s

Lưu ý của paper: phần embedding chiếm 17.4% độ trễ đầy đủ ở GPT-3-8B (1.82s, vocab 320k) và 7.2% ở Llama2-7B (0.69s) - phần này không được cắt nên độ trễ giảm ít hơn tỷ lệ tham số.

Chi phí (Table 4): elastic CT 89.9B token (7.54% pre-train GPT-3 1.1T; 4.50% Llama-2 2T), router tuning 1.049B (0.09%/0.18% kèm joint tuning 2.62B).

Scaling law (Eq. 12): $L(N) = (N/N_c)^{-\alpha_N} + E_N$. Hệ số khớp (Table 3): Matformer $N_c = 1.680, \alpha_N = 52.74, E_N = 1.729$ (thứ tự cột trong bảng text là N_C, α_N, E_N; bản text khó đọc nên có thể bị đổi chỗ nhãn); Flextron-static 1.465/38.57/1.733; input-adaptive 1.289/25.52/1.729. Paper dùng chúng như hướng dẫn chọn mô hình theo ngân sách; không giải thích thêm trong phần đã đọc.

5. Phân tích / ablation

- **Router gán nhiều tính toán hơn cho MLP ở layer sâu:** thay layer đầy đủ bằng 75%/50% độ rộng (GPT3-2B): (1) thay MLP gây tụt hiệu năng nhiều hơn MHA; (2) thay layer sâu, đặc biệt MLP sâu, tụt mạnh. Cấu trúc router học được (Hình 7) khớp với điều này. Ở ngân sách thấp, nên dùng MHA nhẹ (ít head).
- **MHA vs MLP độ trễ:** ở GPT3-2B, mỗi layer MHA 3.830 ms, MLP 3.016 ms dù MHA ít tham số hơn; do đó elastic MHA có ích ở vùng độ trễ cao, và là điểm Flextron hơn MatFormer (chỉ elastic MLP).
- **Router động gán nhiều tính toán hơn cho mẫu khó** (Hình 8-9, GPT3-2B): dữ liệu code (HTML, Java) có đường suy giảm hiệu năng phẳng (cần ít tham số); đa ngôn ngữ cần nhiều hơn. Với mô hình độ trễ 61.8%, gần một nửa số token HTML chọn ứng viên nhỏ nhất ở layer đầu, còn Books3 thường chọn layer đầy đủ. PPL HTML 1.571 so với Books3 11.64.
- **Quỹ đạo elastic CT (Hình 11):** hiệu năng mô hình đầy đủ không giảm; mọi mô hình con hội tụ đồng bộ; mô hình cỡ trung ổn định hơn (có thể vì được lấy mẫu nhiều hơn).
- **Router training dynamics (Hình 12):** ba giai đoạn: SM tuning (L2 loss giảm nhanh, LM loss hơi tăng); joint tuning; router tuning (LM loss tiếp tục giảm).
- **Router học được vs ngẫu nhiên (Hình 13):** mô hình con ngẫu nhiên ở độ trễ 65% có loss trung bình từ 2.32 đến 3.06; router tìm được mô hình nằm ở vùng tốt nhất.
- **Hoán vị trọng số theo importance là cần thiết (Table 5, Wikipedia PPL khi chỉ dùng nửa đầu):** MHA: 184.5 (không sắp xếp) vs 179.9 (có sắp xếp); MLP: 1902.0 vs 193.6; mô hình đầy đủ 9.144. Nghĩa là sắp xếp giúp MLP rất nhiều, MHA ít hơn; cả hai vẫn xa mô hình đầy đủ khi chưa train.
- **So sánh với train từ đầu (Hình 5):** Flextron "superior to Matformer and even end-to-end-trained smaller models (843M)" theo caption của paper.

6. Giới hạn

Tác giả nêu/ngầm hiểu:

- Router động cần thêm bộ nhớ và độ phức tạp triển khai; hiệu quả phụ thuộc vào LUT độ trễ phù hợp phần cứng.
- Embedding không cắt, chiếm tỷ lệ độ trễ đáng kể (17.4%).

Nhận xét:

- Chỉ độ rộng (số head, neuron MLP) là elastic, không có depth; cấu trúc lồng nhau buộc mọi mô hình con dùng *prefix* của thứ tự importance, không cho chọn tập con tuỳ ý.
- Quy mô 89.9B token cho CT vẫn lớn; mô hình con ở 0.5× giảm MMLU nhiều (Llama-2: 45.1 -> 35.9-36.8).
- Router dynamic theo token khó tận dụng trên GPU với batch lớn và kernel tĩnh; lợi ích độ trễ đo trong paper là cho batch 1 và các cấu hình uniform/static. Dynamic: paper báo "số tham số kích hoạt trung bình", không phải độ trễ đo thật cho dynamic (Table 2 chỉ liệt kê mô hình đầy đủ và các mức giảm).
- Đánh giá chủ yếu trên benchmark ngôn ngữ, không phải tác vụ trích xuất có cấu trúc.

7. Liên hệ với các bài khác

- **39 Minitron:** cùng nhóm; importance bằng activation; Minitron chọn một cấu hình và retrain bằng distillation, còn Flextron giữ nhiều cấu hình lồng nhau trong một checkpoint (và dùng CE thường chứ không distill).
- **38 Sheared LLaMA, 17 CoFi, 25 ZipLM:** đối thủ so sánh; học cấu hình bằng mask vs router.
- **42 Puzzle:** hướng chọn cấu hình theo layer dựa trên phần cứng; Puzzle chọn một mô hình, Flextron chọn lúc chạy.
- **13 Sixteen heads, 14 Voita heads:** bằng chứng head có thể bỏ; Flextron sắp xếp head theo importance.
- **18 2:4 sparsity, 19 HALP:** liên quan ràng buộc độ trễ qua LUT (HALP dùng LUT cho CNN).

8. Gợi ý áp dụng

- Router động và surrogate model vì vậy ít cần thiết; **ít liên quan** về mặt triển khai.
- **Nhận xét:** Flextron cho thấy MLP layer sâu nhạy cảm hơn MLP layer nông và MHA ít nhạy hơn MLP. Gợi ý (chỉ giả thuyết) rằng nên cắt MLP *không đồng đều theo độ sâu*: giữ MLP layer sâu, cắt nhiều hơn ở layer nông và ở attention.
- **Phần ViT:** có thể áp dụng tương tự slimmable network cho ViT (đã có tiền lệ CNN), nhưng paper này không kiểm chứng.

9. Thí nghiệm gợi ý

3. **Độ nhạy theo độ sâu:** thay MLP từng layer bằng 50% độ rộng (như Hình 6), vẽ tụt CER theo chỉ số layer để xem layer nào phải giữ.

10. Tóm tắt công thức / khái niệm

Khái niệm	Nội dung
Nested elastic MLP/MHA	Mô hình con = d_j neuron/head đầu của ma trận đã hoán vị, dùng chung trọng số
Importance	$\sum \|\cdot\|_1$ activation của head/neuron, 512 mẫu
Joint loss	$\sum_j L(M_j(x), y)$ trên $k = 3$ mạng con ngẫu nhiên + mạng đầy đủ
Latency loss	$\max(\text{Latency} - T_t, 0)$ qua LUT
Static / Adaptive router	$\arg\max R_i(T)$ / $\arg\max R_i(T, h_i)$
Surrogate model	MLP nhỏ dự đoán LM loss từ logit router để huấn luyện router
Cost	89.9B token CT + ~1B token router

11. Kết luận ngắn

Flextron biến một LLM thành họ mô hình lồng nhau với cả head và neuron MLP elastic, có router chọn mô hình con theo độ trễ và theo token, huấn luyện router qua surrogate model.

Link gốc: https://arxiv.org/abs/2406.10260

Bài 42

Puzzle: Distillation-Based NAS for Inference-Optimized LLMs (2024)

Tác giả: Akhiad Bercovich, *Tomer Ronen*, Talor Abramovich, Nir Ailon, Nave Assaf, Mohammad Dabbah, Ido Galil, Amnon Geifman, Yonatan Geifman, Izhak Golan, Netanel Haber, Ehud Karpas, Roi Koren, Itay Levy, Pavlo Molchanov, Shahar Mor, Zach Moshe, Najeeb Nabwani, Omri Puny, Ran Rubin, Itamar Schen, Ido Shahaf, Oren Tropp, Omer Ullman Argov, Ran Zilberstein, Ran El-Yaniv (NVIDIA). (* đóng góp ngang nhau)
Venue / arXiv: arXiv:2411.19146 (v1: 28/11/2024; bản v5: 03/06/2025). Paper không nêu tên hội nghị trong văn bản được trích.
Nguồn: https://arxiv.org/abs/2411.19146
Chủ đề: Tìm kiến trúc không đồng nhất theo từng layer (neural architecture search, NAS) cho LLM đã train, nhận biết phần cứng; blockwise local distillation (BLD) + quy hoạch số nguyên hỗn hợp (MIP) + global knowledge distillation (GKD); sản phẩm: Llama-3.1-Nemotron-51B và Llama-3.3-Nemotron-49B từ Llama-70B.
Vị trí trong lộ trình: Đọc trước: 39-40 (Minitron: prune đồng nhất + KD), 41 (Flextron: elastic), 31/34 (bỏ layer). Bài này thêm: coi mỗi layer là một ô ghép (attention x FFN, mỗi ô có nhiều biến thể rẻ hơn kể cả bỏ hẳn), đo chi phí *thật trên phần cứng đích* và chọn tổ hợp tối ưu toàn cục bằng MIP. Tiếp theo: 43 (so sánh prune vs train từ đầu), 44-47 (hybrid/Mamba, Jet-Nemotron).

Ghi chú học tập bằng tiếng Việt, diễn giải và tóm tắt, không phải bản dịch. Số liệu lấy từ paper.

1. Bối cảnh

- LLM thường gồm các lớp giống hệt nhau (thiết kế vì dễ huấn luyện và dễ scale), không phải vì tối ưu suy luận. Nhưng sau khi huấn luyện, nhiều tham số và phép tính dư thừa.
- **Neural Architecture Search (NAS)**: tự động tìm kiến trúc tốt trong một không gian cấu hình. Rất tốn nếu mỗi ứng viên phải huấn luyện hoặc đánh giá cả mô hình. NAS **phân rã (decomposed)** trong thị giác máy tính (DNA, DONNA, LANA): huấn luyện từng *khối* riêng để mô phỏng khối của mô hình cha, rồi ước lượng chất lượng của một kiến trúc bằng tổng điểm từng khối, thay vì huấn luyện cả mô hình.
- Số tham số không phản ánh độ nhanh thực. Thời gian chạy phụ thuộc phần cứng (số SM, băng thông bộ nhớ, tensor core, FP8...), batch size, độ dài chuỗi, và KV cache (bộ nhớ lưu key/value của các token đã sinh, tăng tuyến tính theo batch x độ dài).
- Mục tiêu: biến mô hình cha mạnh (open-weights, dữ liệu huấn luyện đóng) thành mô hình con tối ưu cho một GPU/kịch bản cụ thể, chỉ cần trọng số mô hình cha.

2. Ý tưởng chính

Xem mỗi lớp là một "ô" của trò ghép hình. Với mỗi lớp, tạo một thư viện các mảnh thay thế rẻ hơn cho attention (GQA với ít KV head hơn, một lớp tuyến tính, hoặc bỏ hẳn) và FFN (rút gọn chiều trung gian, lớp tuyến tính, hoặc bỏ hẳn). Train từng mảnh riêng, song song, để bắt chước mảnh gốc (BLD). Đo điểm "chất lượng mất" của từng mảnh khi thay riêng mảnh đó vào mô hình cha. Dùng MIP để chọn đúng một mảnh cho mỗi lớp sao cho tổng chất lượng tốt nhất mà vẫn thoả ràng buộc thông lượng, độ trễ, bộ nhớ (đo thật trên phần cứng). Cuối cùng ghép lại và huấn luyện toàn mạng ngắn bằng distillation (GKD) để các khối "khớp" với nhau. Kết quả: Nemotron-51B giữ 98.4% độ chính xác của Llama-3.1-70B-Instruct với 2.17x thông lượng, chạy trên một H100.

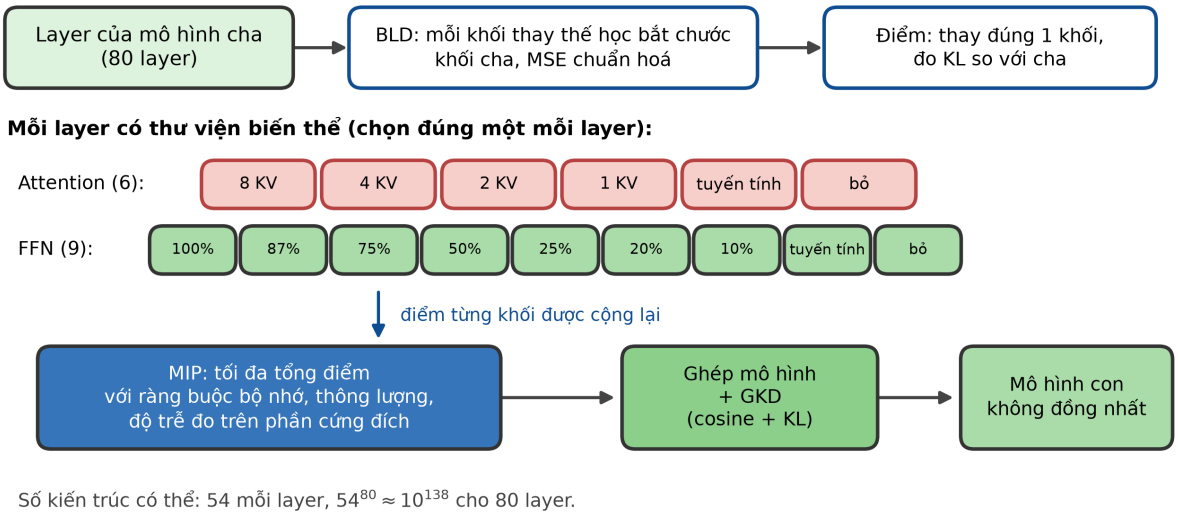
3. Phương pháp chi tiết

3.1 Không gian tìm kiếm

Mỗi layer của mô hình cha (Llama-3.1-70B có 80 layer) được thay bằng đúng một layer con; subblock trong một layer phải cùng kích thước vào/ra với mô hình cha.

Subblock	Lựa chọn
Attention (6)	GQA với 8, 4, 2, 1 KV head; một lớp tuyến tính; no-op (bỏ hẳn)
FFN (9)	Giữ nguyên chiều trung gian, hoặc rút còn khoảng 87%, 75%, 50%, 25%, 20%, 10%; lớp tuyến tính; no-op

Mỗi layer có 6 x 9 = 54 cấu hình; 80 layer cho $54^{80} \approx 10^{138}$ kiến trúc.



Hình 75: Luồng của Puzzle: thư viện khối thay thế mỗi layer, BLD, chấm điểm bằng thay một khối, MIP chọn đúng một biến thể mỗi layer rồi GKD (minh hoạ, tự tính)

Cách đọc hình: hàng trên là tạo khối thay thế và chấm điểm từng khối. Giữa hình là thư viện 6 biến thể attention và 9 biến thể FFN cho mỗi layer. Hàng dưới là MIP chọn tổ hợp theo ràng buộc phần cứng đo thật, ghép lại và chưng cất toàn mạng.

3.2 Blockwise Local Distillation (BLD)

Mỗi khối con được train *độc lập và song song* để bắt chước khối cha tương ứng, nhận đầu vào là activation của mô hình cha (không phải đầu ra của khối con trước đó). Loss là MSE chuẩn hoá:

$$\mathcal{L}_{BLD} = \frac{\text{MSE}(o_p, o_c)}{\text{MSE}(o_p, 0)}$$

o_p, o_c là output khối cha và khối con. Ưu điểm: mỗi khối chỉ học một hàm đơn giản nên ổn định, học rate cao hơn được, cần chỉ ~1B token (Distillation Mix: 224B token từ FineWeb, Dolma, Buzz-V1.2, dùng 1B token cho BLD). Nhược điểm: không đảm bảo tương thích giữa các khối kế nhau (lỗi có thể tích luỹ), nên phải có GKD cuối.

Coupled vs decoupled BLD: coupled huấn luyện mọi cặp $[a_j, f_k]$ (tốn $m \cdot n \cdot l$ khối); decoupled huấn luyện $[a_j, f_{parent}]$ và $[a_{parent}, f_k]$ riêng (đóng băng nửa còn lại) rồi ghép sau, chỉ $(m+n) \cdot l$ khối. Ví dụ $m = n = 20$, $l = 80$: 3,200 so với 32,000 khối. Có thể kết hợp: chạy decoupled trước để thu hẹp không gian, rồi coupled trên không gian hẹp (xem phần 5).

3.3 Khởi tạo không cần train (Channel Contribution)

- FFN rút gọn: xếp hạng kênh theo đóng góp vào output FFN: $C_i(X) = \| \sum_k X_k W^{down}_{k,:} - \sum_{k \neq i} X_k W^{down}_{k,:} \|_2 = |X_i| \cdot \|W^{down}_{i,:}\|_2$, lấy trung bình trên token của tập calibration, bỏ kênh đóng góp thấp nhất.
- FFN -> tuyến tính: khởi tạo bằng tích các ma trận up và down (bỏ qua cổng gating).
- Attention giảm KV head: mean-pool các projection K/V thành một (theo GQA gốc).
- Attention -> tuyến tính: dùng tích $W_V W_O$ (mô phỏng mỗi token chỉ attend chính nó).

3.4 Chấm điểm và đo chi phí

- **Replace-1-block score:** dựng mô hình giống cha nhưng thay đúng *một* khối bằng khối thư viện, đo chất lượng trên toàn mô hình. Chất lượng kiến trúc ứng viên được ước lượng bằng *tổng* điểm các khối (nhanh, <1 giây mỗi ứng viên). Ba thước đo xét tới: độ chính xác downstream (đắt), LM loss, và **KL divergence** giữa phân phối token kế tiếp của mô hình đã thay và mô hình cha (paper cho rằng KL tốt nhất).
- **Chi phí:** đo trực tiếp thời gian prefill/generate của từng biến thể khối trên phần cứng đích (nhiều độ dài chuỗi, batch, mức lượng tử hoá); bộ nhớ gồm tham số (cố định) và KV cache (tăng theo batch x độ dài).

3.5 Tìm kiếm bằng MIP

Bài toán knapsack có nhóm: chọn đúng một biến thể mỗi layer.

$$\max_x \sum_{i=1}^{L} \sum_{j=1}^{K_i} \text{score}(i,j)\, x_{i,j}$$

$$\begin{aligned}
\text{s.t.} \quad & \sum_{i,j} [\text{mem}_{params}(i,j) + b \cdot \text{mem}_{kv}(i,j)] x_{i,j} \leq \text{Mem}_{max}, \\
& \frac{b \cdot \text{seq_len}}{\sum_{i,j} \text{runtime}(i,j,b) x_{i,j}} \geq \text{Thr}_{min}, \\
& \sum_{i,j} \text{runtime}(i,j,b) x_{i,j} \leq \text{Lat}_{max}, \\
& \sum_j x_{i,j} = 1, \quad x_{i,j} \in \{0,1\}.
\end{aligned}$$

Nếu điểm là "tác động tiêu cực" (LM loss, KL), cực tiểu thay vì cực đại. Không đặt ràng buộc số tham số, nó giảm tự nhiên. b không phải biến, nên giải nhiều lần với nhiều b và chọn batch cho kiến trúc tốt nhất. Dùng python-mip, nghiệm trong vài giây. Để lấy nhiều nghiệm khác nhau, thêm ràng buộc đa dạng $\sum x_{i,j} y_{i,j} \leq \alpha L$ với mỗi nghiệm cũ y.

Tác giả nhấn mạnh tính "phụ thuộc phần cứng": với thiết bị bộ nhớ nhỏ và batch 1, thuật toán ưu tiên cắt FFN (giảm tham số) và bỏ qua tối ưu KV cache; trên H100 có FP8 thì tính chất khác; băng thông NVLink hay PCIe cũng ảnh hưởng.

3.6 Uptraining toàn cục (GKD)

Train toàn mô hình con với mô hình cha làm teacher. Thử tổ hợp ba loss: LM loss (CE nhãn), cosine similarity giữa hidden state của từng layer cha-con, và KL divergence trên logits. Thực nghiệm (Table 1) cho thấy LM loss không cần và còn gây hại:

$$\boxed{\mathcal{L}_{GKD} = \mathcal{L}_{cosine} + \mathcal{L}_{KLD}}, \quad \mathcal{L}_{cosine} = \sum_{l=1}^{L} \left(1 - \frac{h_c^l \cdot h_p^l}{\|h_c^l\| \|h_p^l\|}\right)$$

Mô hình cuối train 45B token. Giải thích tác giả: bỏ LM loss giúp giảm tác động của việc không có dữ liệu gốc của mô hình cha (open-weights, closed-data).

3.7 Hỗ trợ suy luận

Mô hình không đồng nhất đòi hỏi engine hỗ trợ: tác giả sửa TensorRT-LLM để hỗ trợ số KV head thay đổi theo layer (paged KV cache), khối tuyến tính và no-op, cùng FP8 cho trọng số, activation và KV cache.

4. Thí nghiệm và kết quả chính

Nemotron-51B (từ Llama-3.1-70B-Instruct) so với mô hình cha (Table 2):

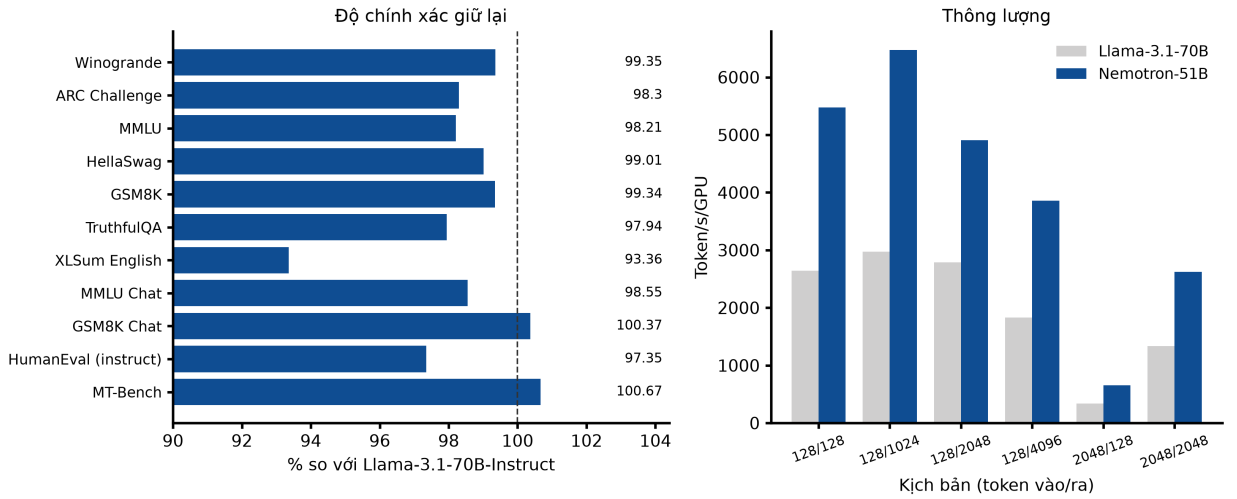
Benchmark	Llama-3.1-70B-Instruct	Nemotron-51B	% giữ lại
Winogrande	85.08	84.53	99.35
ARC Challenge	70.39	69.20	98.30
MMLU	81.66	80.20	98.21
HellaSwag	86.44	85.58	99.01
GSM8K	92.04	91.43	99.34
TruthfulQA	59.86	58.63	97.94
XLSum English	33.86	31.61	93.36
MMLU Chat	81.76	80.58	98.55
GSM8K Chat	81.58	81.88	100.37
HumanEval (instruct)	75.85	73.84	97.35
MT-Bench	8.93	8.99	100.67

Trung bình giữ 98.4%. Người đánh giá (blind test, Hình 4): hai mô hình tương đương.

Thông lượng (Table 3; tokens/giây/GPU H100 SXM, FP8 cả trọng số, activation và KV cache; chọn TP tối ưu mỗi mô hình):

Kịch bản (in/out)	Nemotron-51B	Llama-3.1-70B	Tăng tốc
Chatbot 128/128	5478 (TP1)	2645 (TP1)	2.07
Text gen 128/1024	6472 (TP1)	2975 (TP4) / 1274 (TP1)	2.17 / 5.08
Long text 128/2048	4910 (TP2)	2786 (TP4)	1.76
Inference-time compute 128/4096	3855 (TP2)	1828 (TP4)	2.11
Summarization/RAG 2048/128	653 (TP1)	339 (TP4) / 301 (TP1)	1.92 / 2.17
Stress test 2048/2048	2622 (TP2)	1336 (TP4)	1.96

Nemotron-51B tối ưu ở TP=1, batch 256 (một H100); Llama-70B tối ưu ở TP=4, batch 384.



Hình 76: Nemotron-51B so với Llama-3.1-70B-Instruct: phần trăm độ chính xác giữ lại theo benchmark (Table 2) và thông lượng theo kịch bản (Table 3) (vẽ lại từ số liệu trong ghi chú)

Cách đọc hình: panel trái là tỷ lệ giữ lại so với mô hình cha, đa số trên 97%, thấp nhất là XLSum English (93.36). Panel phải là thông lượng token/s/GPU H100 ở FP8; Llama-70B dùng TP1 ở kịch bản 128/128 và TP4 ở các kịch bản còn lại, theo bảng. Cột xanh luôn cao hơn cột xám.

Kiến trúc tìm được (Hình 6, Nemotron-51B): giảm tính toán ở nhiều layer, cả đầu (0-15) lẫn cuối (45-70); vùng giữa (16-42) giữ nguyên. Layer đầu giảm cân bằng giữa attention và FFN; layer cuối giảm attention mạnh hơn.

Ngữ cảnh dài và dẫn xuất khác:

- Nemotron-51B chỉ train tới chuỗi 8K nhưng giữ >96% độ chính xác cha ở 16K (RULER); giảm sau 64K.
- Nemotron-49B-Base (từ Llama-3.3-70B-Instruct), uptrain thêm 5B token ở 64K và 5B ở 128K: giữ 100.65% (4K) ... 98.29% (64K, điểm 87.39 so với 88.91) và 94.96% ở 128K (49.62 so với 52.25).
- Alignment nhẹ (25M token, RLHF): MMLU 80.98, MT-Bench 9.21, Arena Hard 82.1 (trước: 81.03 / 8.77 / 65.8; Llama-3.3-70B-Instruct: 81.66 / 8.84 / 71.70).
- Mô hình gọn từ Llama-3.1-8B-Instruct tối ưu cho RTX 4090 (Table 6): thông lượng 5856, accuracy 73.98 ((MT-Bench x 10 + MMLU)/2) so với Llama-3.2-3B-Instruct 5737 / 70.36 và cha 3385 / 76.40.
- Áp dụng cho Llama-3.1-405B (ràng buộc 1.5x giảm độ trễ, vừa một node 8xH100 hoặc một B100): giữ 99.5% độ chính xác (trung bình qua MMLU, MT-Bench, MMLU-Pro, HumanEval, Arena Hard). "miniPuzzle" trên Nemotron-H-56B-Base ra mô hình 47B giữ 99.94% (RTX 5090, ngữ cảnh 1M); miniPuzzle dùng khối đồng nhất ở mọi layer.
- **Ngân sách token cho GKD (Table 7):** Nemotron-49B-Base: 8.68B token cho 99.63% (80.73 / 8.87); 2.9B token cho 98.47% (80.72 / 8.675); 0.72B token cho 97.79% (80.4 / 8.59). Nemotron-51B với 3.7B token đạt 98.8% (MMLU + MT-Bench).

5. Phân tích / ablation

- **Coupled vs decoupled (Table 8, Llama-3.1-8B, 4090):** decoupled thu hẹp không gian từ (6 attention, 12 FFN) xuống (4, 3), sau đó coupled BLD (384 khối) tăng accuracy từ 73.10 lên 73.98 với thông lượng tương đương (5834 → 5856).
- **Thành phần dữ liệu (Table 9, trước GKD):** train/chấm điểm bằng Project Gutenberg (văn học) đạt MT-Bench 7.98, MMLU 74.84, MMLU-STEM 64.5 so với Distillation Mix 8.61 / 78.39 / 70.35. Tức

giữ 92.7% (MT-Bench), 95.5% (MMLU), 91.7% (STEM). Pipeline khá bền với dữ liệu hẹp.

- **Kích thước BLD (Table 10):** 0.25B/0.5B/1.0B token: MMLU 78.27/78.44/78.54, MT-Bench 8.51/8.86/8.98. Lợi ích giảm dần sau 0.5B.
- **Thước đo điểm khối:** KL divergence cho kiến trúc tốt hơn LM loss (Hình 7). Dùng accuracy downstream (nửa MMLU) để chấm điểm giúp tối ưu cho tác vụ đó (66.24 so với 64.94 trên nửa test) nhưng MT-Bench giảm (7.57 so với 8.06).
- **Không gian chỉ có no-op (Table 12):** MMLU 75.40 (thông lượng 5604) so với 78.39 (5500) của không gian đầy đủ (pre-uptraining). Đa dạng biến thể có giá trị.
- **MIP vs greedy (Table 13):** greedy MMLU 70.74 so với MIP 78.39 ở cùng thông lượng ~5500.
- **Tham số lớn nhất, không dùng điểm dữ liệu (Table 14):** MMLU 23.12 so với 78.39.
- **Kiến trúc ngẫu nhiên (Table 15, 10B token):** Nemotron-51B 79.7 MMLU / 8.89 MT-Bench (98.61% cha); chọn ngẫu nhiên từ thư viện 66.02 / 8.20 (86.58%); hoàn toàn ngẫu nhiên 23.13 / 0.89 (18.73%).
- **GKD quan trọng (Table 16):** 70B: không GKD MMLU 78.39, MT-Bench 8.67; có GKD 80.20 / 8.99. 8B: không GKD 65.25 / 7.29; có 65.46 / 8.25. MT-Bench hồi phục nhiều hơn MMLU, nhất là ở 8B.
- **Loss GKD (Table 1, ~5B token):** KLD + cosine (không LM loss) MMLU 79.61, MT-Bench 8.87 là tốt nhất. Các tổ hợp có LM loss mà không có KLD khiến KL validation tăng (đánh dấu * trong paper).
- **Phân tích MIP (Hình 8):** FFN không bao giờ bị bỏ hoàn toàn, trong khi attention bị bỏ hẳn ở nhiều layer kể cả ở ràng buộc nhẹ; có hiện tượng "tối ưu cục bộ ngược" (layer 1-4 đôi khi chọn khối nặng hơn khi mục tiêu thông lượng cao hơn).
- **So sánh phương pháp khác (Table 17):** Nemotron-51B giữ 99.49% (MMLU 80.20, MT-Bench 8.99) so với Wanda 2:4 không train 92.23% (72.99 / 8.39) và low-rank + distill 88.96% (72.87 / 8.01); Wanda + distill chỉ cải thiện MMLU lên 73.69. Tác giả nói Minitron, ShortGPT, SlimGPT là các tập con của không gian Puzzle (Minitron thường đồng nhất qua các layer; ShortGPT chỉ bỏ layer).

6. Giới hạn

Tác giả nêu/ngầm hiểu:

- BLD không đảm bảo tương thích giữa các khối; cần GKD. Subblock phải giữ nguyên chiều vào/ra (không giảm hidden size).
- Mỗi layer cha chỉ thành đúng một layer con (có thể mở rộng nhóm nhiều layer, chưa làm).
- Cần engine suy luận hỗ trợ khối không đồng nhất (phải sửa TensorRT-LLM).
- Điểm replace-1-block cộng tuyến tính, bỏ qua tương tác giữa các khối (suy ra từ cách làm; GKD là biện pháp bù).

Nhận xét:

- Mọi kết quả ở quy mô 8B-405B với batch lớn; tối ưu thông lượng. Chưa có bằng chứng cho batch 1 hay mô hình dưới 1B.
- Đánh giá chủ yếu MMLU và MT-Bench, benchmark chat; không có tác vụ trích xuất/OCR.
- Tổng chi phí vẫn đáng kể (45B token GKD cho bản chính; bản nhẹ hơn tới 0.7-3.7B token cho 97.8%-98.8%).

7. Liên hệ với các bài khác

- **39-40 Minitron:** Minitron prune đồng nhất theo trục (width) + KD; Puzzle cho phép từng layer chọn mức giảm riêng và dùng chi phí đo thật. Cả hai dùng KL/cosine trên logits/hidden; Puzzle bỏ LM loss giống Minitron.
- **31 ShortGPT, 34 SLEB, 35 LaCo, 33 Shortened LLaMA:** bỏ layer là một lựa chọn "no-op" trong Puzzle; Puzzle còn cho bỏ riêng attention hay FFN (và thấy attention bỏ dễ hơn FFN).
- **29 SliceGPT, 28 FLAP, 26 LLM-Pruner:** cắt width; trong Puzzle chỉ là một loại biến thể.

- **37 Repair changes which layers:** liên quan ý "khối nào bỏ được thay đổi sau khi sửa"; Puzzle giải quyết bằng GKD cuối.
- **23-24 SparseGPT, Wanda; 18 2:4 sparsity:** Table 17 so Wanda 2:4 với Puzzle.
- **41 Flextron:** hướng khác: elastic lúc chạy vs. chọn tĩnh một kiến trúc theo phần cứng.
- **44 Nemotron-H:** miniPuzzle áp dụng cho Nemotron-H-56B (hybrid).
- **17 CoFi, 19 HALP:** HALP/độ trễ qua LUT là tiền thân của việc dùng chi phí đo thật.

8. Gợi ý áp dụng

- Paper cho thấy KL tốt hơn LM loss làm chỉ số chọn.
- **GKD loss:** cosine hidden + KL logits, không CE; trùng khuyến nghị Minitron.

9. Thí nghiệm gợi ý

3. **BLD vs không BLD:** với một vài khối thay thế, so khởi tạo không train (Channel Contribution) với BLD vài chục triệu token trước khi KD toàn mô hình.

10. Tóm tắt công thức / khái niệm

Khái niệm	Nội dung
Search space	6 attention x 9 FFN mỗi layer; $54^{80} \approx 10^{138}$ kiến trúc
BLD loss	$\text{MSE}(o_p, o_c)/\text{MSE}(o_p, 0)$
Decoupled BLD	$(m+n)l$ khối thay vì $m \cdot n \cdot l$
Channel Contribution	$C_i = \lvert X_i \rvert \cdot \lVert W^{down}_{i,:} \rVert_2$
Replace-1-block score	Thay đúng 1 khối trong mô hình cha, đo KL/LM loss; cộng điểm các khối
MIP	Knapsack nhóm: 1 biến thể/layer; ràng buộc bộ nhớ, thông lượng, độ trễ
GKD	$\mathcal{L} = \mathcal{L}_{cosine} + \mathcal{L}_{KLD}$, không LM loss
Kết quả chính	Nemotron-51B: 98.4% độ chính xác, 2.17x thông lượng, 1 GPU H100

11. Kết luận ngắn

Puzzle coi suy luận hiệu quả là bài toán chọn kiến trúc theo phần cứng thật: tạo thư viện khối thay thế bằng distillation cục bộ rẻ tiền, chấm điểm bằng KL, chọn tổ hợp toàn cục bằng MIP, rồi hàn gắn bằng distillation toàn mạng.

Link gốc: https://arxiv.org/abs/2411.19146

Bài 43

Small LLMs: Pruning vs. Training from Scratch (2026)

Tác giả: Yufeng Xu, Taiming Lu, Kunjun Li, Jiachen Zhu, Mingjie Sun, Zhuang Liu (Princeton University; New York University; Carnegie Mellon University).
Venue / arXiv: arXiv:2606.14150 (v1: 12/06/2026; bản v3: 29/06/2026). Paper không nêu tên hội nghị trong văn bản được trích. Mã nguồn: github.com/zlab-princeton/pruning-vs-scratch.
Nguồn: https://arxiv.org/abs/2606.14150
Chủ đề: So sánh có kiểm soát số token giữa "prune mô hình lớn rồi retrain" và "train mô hình nhỏ cùng kiến trúc từ khởi tạo ngẫu nhiên", trên Llama-3.1-8B với 6 phương pháp prune (depth, width, 2:4, unstructured) và nhiều tỉ lệ prune; nhắc lại câu hỏi của "Rethinking the value of network pruning" ở quy mô LLM.
Vị trí trong lộ trình: Đọc trước: 11 (Rethinking the value of pruning, cùng nhóm tác giả, kết luận gốc cho CNN), 08 (Lottery Ticket), 38-40 (Sheared LLaMA, Minitron). Bài này cho phép *đo* xem phần "prune" trong "prune + retrain" có thực sự đáng tiền không. Tiếp theo: 44-47 (hybrid/Mamba và các hướng prune khác).
Chương nền liên quan: nền 07: Đo đúng một kết quả pruning

> Ghi chú học tập bằng tiếng Việt, diễn giải và tóm tắt, không phải bản dịch. Số liệu lấy từ paper (một vài bảng trong bản text có sai khác nhỏ giữa các bảng; sẽ chỉ rõ).

1. Bối cảnh

- Các bài trước trong lộ trình (Sheared LLaMA 38, Minitron 39-40, Puzzle 42) đều nói: prune mô hình lớn rồi huấn luyện lại bằng một phần nhỏ token cho mô hình nhỏ tốt, rẻ hơn huấn luyện từ đầu. Nhưng thường so sánh với *các mô hình cộng đồng* huấn luyện với dữ liệu/công thức khác (Pythia, TinyLlama, Gemma...), không phải một baseline "train từ đầu cùng kiến trúc, cùng dữ liệu, cùng số token".
- **Khởi tạo ngẫu nhiên (random initialization / scratch)**: trọng số bắt đầu từ phân bố ngẫu nhiên, học từ đầu. **Khởi tạo bằng prune**: trọng số bắt đầu từ mô hình lớn đã cắt.
- Liu et al. (2019), bài 11: với CNN và structured pruning, huấn luyện từ đầu kiến trúc đã cắt ngang hoặc hơn prune + fine-tune; nghĩa là cái có giá trị là *kiến trúc* tìm được, không phải trọng số thừa hưởng (coi prune như NAS). Với unstructured pruning, huấn luyện từ đầu kém hơn ở quy mô lớn.
- Câu hỏi của bài báo: kết luận đó có đúng cho LLM (mô hình/dữ liệu lớn hơn, kiến trúc khác) không? Tác giả đổi cách nhìn: **prune là một chiến lược khởi tạo, không phải công cụ nén**, nên baseline đúng là mô hình cùng cỡ huấn luyện từ đầu trên cùng luồng dữ liệu.
- Hai khái niệm token: *training token budget* (số token dùng để huấn luyện mô hình nhỏ) và *total token budget* (cả quy trình, gồm token pretrain mô hình cha cộng token retrain).

2. Ý tưởng chính

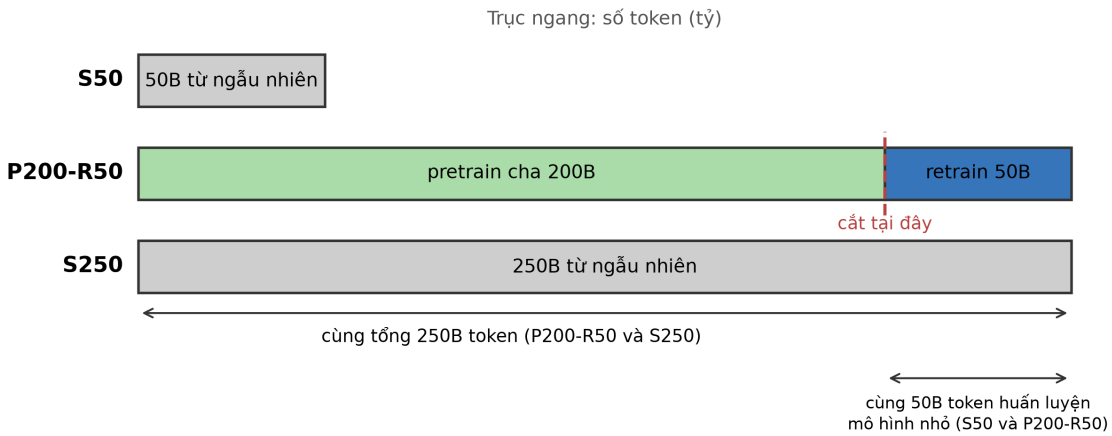
Thiết kế hai so sánh công bằng theo token. (1) **Equal training token budget**: mô hình nhỏ huấn luyện 50B token từ ngẫu nhiên (S50) so với mô hình cắt từ cha rồi huấn luyện lại 50B token (P200-R50), cô lập tác dụng của khởi tạo. (2) **Equal total token budget**: baseline huấn luyện từ đầu được cấp *cả* 250B token mà quy trình prune đã tiêu (200B pretrain cha + 50B retrain), tức S250. Kết quả: (1) prune thắng ở mọi phương pháp, nhưng lợi thế co lại khi tỉ lệ prune tăng và gần như biến mất ở 81.3% (depth); (2) khi baseline được cấp đủ tổng token thì huấn luyện từ đầu bắt kịp hoặc vượt prune có cấu trúc (structured), còn prune mịn (sparse) vẫn giữ lợi thế nhẹ. Khuyến nghị: có cha mạnh + ngân sách token hạn chế thì prune; token dồi dào thì huấn luyện từ đầu ở độ chi tiết thô cũng cạnh tranh.

3. Phương pháp chi tiết

3.1 Ký hiệu thiết kế

- S_N: train kiến trúc đích từ ngẫu nhiên trong NB token.
- P_{200}-R_N: pretrain mô hình cha (kiến trúc Llama-3.1-8B) 200B token, prune ở tỉ lệ đã chọn, rồi retrain NB token.
- Meta-R_N: như trên nhưng bắt đầu từ checkpoint Llama-3.1-8B chính thức của Meta (làm điểm tham chiếu).

Hai so sánh: **S50 vs P200-R50** (equal training budget) và **S250 vs P200-R50** (equal total budget). Dữ liệu: DCLM-Baseline-1.0, token hoá bằng tokenizer Llama-3.1, chunk 8192 token. 200B token pretrain và 50B token retrain là hai tập con không giao nhau; S250 dùng đúng hợp của hai tập đó; S50 dùng đúng 50B token của giai đoạn retrain. Nhờ vậy khác biệt chỉ đến từ khởi tạo.



Hình 77: Hai cách so sánh công bằng theo token: cùng 50B token huấn luyện (S50 và P200-R50) và cùng tổng 250B token (P200-R50 và S250) (minh hoạ, tự tính)

Cách đọc hình: độ dài thanh tỉ lệ với số token. P200-R50 gồm 200B token pretrain mô hình cha rồi cắt và retrain 50B token. S50 chỉ so được với phần 50B token cuối, còn S250 mới ngang tổng chi phí.

3.2 Công thức huấn luyện

Lingua recipe (AdamW, $\beta = (0.9, 0.95)$, eps 1e-8, weight decay 0.01, cosine decay xuống 10% đỉnh, warm-up 5%, batch 512, chuỗi 8192). Với mỗi thí nghiệm, quét LR đỉnh trong {1e-5, 3e-5, 1e-4, 3e-4, 1e-3} trên một đoạn ngắn và chọn LR cho train loss thấp nhất (tác giả nhấn mạnh: LR retrain phải chọn cẩn thận để tránh quên thảm hoạ, catastrophic forgetting). **Loss là LM loss thường; không distillation.**

3.3 Sáu phương pháp prune (Hình 2, Phụ lục C)

Phương pháp	Đơn vị bị cắt	Tiêu chí
Minitron-depth (D)	cả layer (một khối liên tiếp)	tăng validation loss khi thay khối $[i, i+k)$ bằng identity: $i^\star = \arg\min_i L(\theta_{[i:i+k]\to id}) - L(\theta)$
Minitron-width (W)	kênh hidden, kênh MLP	$s_j = \|X_{:,j}\|_2$ (chuẩn activation)
FLAP	kênh hidden/MLP	phương sai đặc trưng đầu vào nhân chuẩn cột trọng số: $s_j = \frac{1}{N-1}\sum_n (X_{n,j} - \bar{X}_{:,j})^2 \cdot \|W_{:,j}\|_2^2$
Sheared LLaMA	layer, hidden, MLP, head	mask học được + Lagrangian (bài 38)
Wanda	từng trọng số	$s_{ij} = \|W_{ij}\| \cdot \|X_j\|_2$, chọn trong từng hàng output

Phương pháp	Đơn vị bị cắt	Tiêu chí
SparseGPT	từng trọng số	tái tạo output từng layer (OBS), $\min \|XW^\top - X(\hat{W} \odot m)^\top\|_F^2$

Wanda và SparseGPT được chạy ở hai dạng: 2:4 (semi-structured, mỗi 4 trọng số giữ 2) và 50% unstructured (ký hiệu -U). Tỉ lệ prune chính là 50% (8.0B -> ~4.5B tham số). Với Minitron còn thử 62.5%, 75%, 81.3%.

Cấu hình đích (Table 1; Llama-3.1-8B gốc: 32 layer, 32 head, hidden 4096, FFN 14336, 8.0B):

Phương pháp	Tỉ lệ	Layers	Heads	Hidden	FFN	Tham số
Minitron-D	50% / 62.5% / 75% / 81.3%	16 / 8 / 4 / 2	32	4096	14336	4.5B / 2.8B / 1.9B / 1.5B
Minitron-W	50%	32	32	3072	9216	4.5B
Minitron-W	62.5%	32	32	2432	6144	3.1B
Minitron-W	75%	32	32	1792	6016	2.0B
Minitron-W	81.3%	32	32	1536	4736	1.5B
FLAP	50%	32	20	4096	6656	4.5B
Sheared LLaMA	50%	32	32	3456	7552	4.6B
Wanda / SparseGPT	50%	32	32	4096	14336	4.5B (số khác 0)

Khi phương pháp không xác định kiến trúc duy nhất (Minitron-W, FLAP, Sheared), tác giả liệt kê các cấu hình cùng số tham số, cắt không retrain, và chọn cấu hình có validation loss thấp nhất trên tập calibration (Phụ lục D.3). Với Minitron-W: giữ 32 head, ràng buộc $2h \leq \text{MLP} \leq 4h$.

3.4 Đánh giá

Perplexity (C4, WikiText-103, WikiText-2, CNN/DailyMail; 256 chuỗi 8192 token) và 8 benchmark zero-shot (WinoGrande, ARC-C, ARC-E, HellaSwag, PIQA, SciQ, BoolQ, OBQA), báo trung bình. Độ lệch chuẩn đánh giá mỗi benchmark: WinoG ±1.3, ARC-C ±1.4, ARC-E ±1.0, HellaSwag ±0.4, PIQA ±1.0, SciQ ±0.8, BoolQ ±0.8, OBQA ±2.2 (điểm); chênh lệch nhỏ hơn mức này coi là nhiễu. Nhiều giá trị nên đọc trong ngữ cảnh đó (đặc biệt OBQA, BoolQ).

4. Thí nghiệm và kết quả chính

Mô hình cha pretrain của chính tác giả (P200): chỉ 200B token, nên yếu hơn Llama-3.1-8B của Meta: điểm trung bình 68.6 so với 74.0 (Table 10).

4.1 Tổng quan ở tỉ lệ 50% (Table 2; WikiText-2 PPL và Avg accuracy trên 8 benchmark)

	PPL			Acc		
Phương pháp	PPL S50	P200-R50	PPL S250	Acc S50	P200-R50	Acc S250
Minitron-D	10.77	9.41	9.01	60.7	64.4	66.2
Minitron-W	11.17	9.01	8.69	59.9	66.3	67.5
FLAP	11.34	7.97	8.82	57.7	66.5	65.1
Sheared LLaMA	11.28	8.63	8.85	58.4	65.9	65.3
Wanda-2:4	10.55	9.15	8.60	61.5	66.7	66.2

Phương pháp	PPL S50	PPL P200-R50	PPL S250	Acc S50	Acc P200-R50	Acc S250
Wanda-U	10.66	8.37	8.57	61.6	68.1	67.2
SparseGPT-2:4	10.54	8.90	8.65	62.1	66.7	67.1
SparseGPT-U	10.47	8.44	8.59	61.9	67.5	66.3

Đọc bảng:

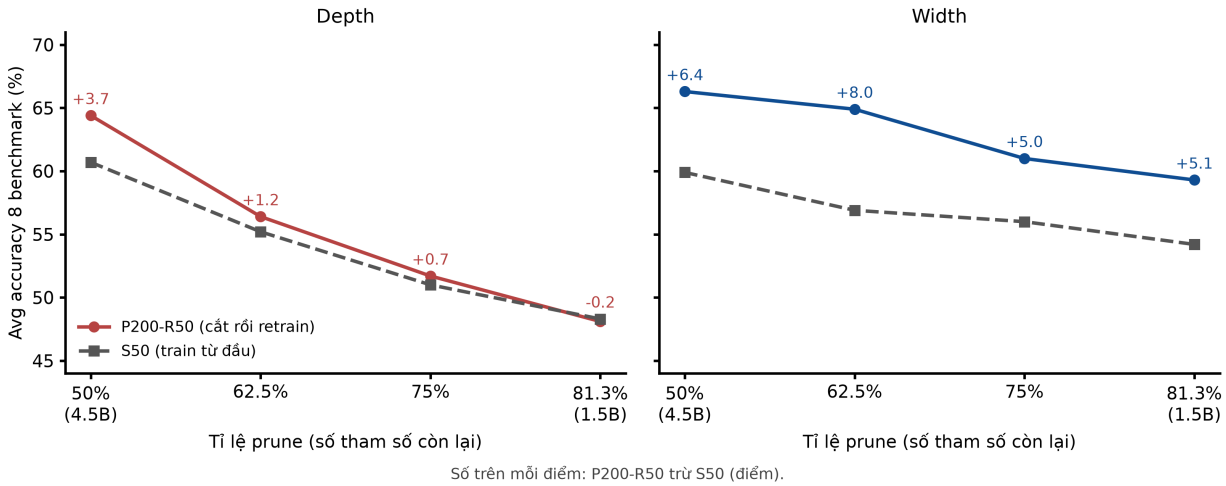
- **Cột P200-R50 so với S50:** pruning thắng ở mọi phương pháp, cả PPL lẫn accuracy (chênh accuracy từ +3.7 điểm ở Minitron-D đến +8.8 ở FLAP).
- **Cột P200-R50 so với S250:** structured: S250 vượt hoặc ngang (Minitron-D 66.2 vs 64.4; Minitron-W 67.5 vs 66.3; FLAP và Sheared ngược lại: P200-R50 nhỉnh hơn 66.5 vs 65.1 và 65.9 vs 65.3). Sparse: Wanda-U 68.1 vs 67.2, SparseGPT-U 67.5 vs 66.3 (pruning nhỉnh hơn); 2:4: 66.7 vs 66.2 (Wanda), 66.7 vs 67.1 (SparseGPT, S250 nhỉnh hơn).
- Perplexity: FLAP có PPL WikiText-2 thấp nhất trong số mô hình prune (7.97). Phụ lục cũng lưu ý kết luận về lợi thế của sparse phụ thuộc benchmark; không nên xem đây là kết quả đúng cho mọi phép đo.
- Mô hình chính xác nhất: Wanda-U 68.1%.

4.2 Finding 1: cùng training token budget, prune thắng nhưng thu hẹp theo tỉ lệ prune

Các số cho Minitron (Table 11, accuracy trung bình 8 benchmark; Δ = P200-R50 trừ S50):

Tỉ lệ (tham số)	Depth S50	Depth P200-R50	Depth Δ	Width S50	Width P200-R50	Width Δ
50% (4.5B)	60.7	64.4	+3.7	59.9	66.3	+6.4
62.5%	55.2	56.4	+1.2	56.9	64.9	+8.0
75%	51.0	51.7	+0.7	56.0	61.0	+5.0
81.3% (1.5B)	48.3	48.1	-0.2	54.2	59.3	+5.1

- **Depth:** lợi thế co về ~0 ở 81.3% (PPL WikiText-2 ngang nhau, 25.16 vs 25.48; accuracy 48.3 vs 48.1).
- **Width:** vẫn dương ở mọi tỉ lệ (+5.1 ở 81.3%), nghĩa là cắt width giữ được nhiều "kiến thức" từ cha hơn cắt layer ở tỉ lệ cao.



Hình 78: Avg accuracy 8 benchmark của S50 và P200-R50 theo tỉ lệ prune, cho Minitron depth và width (Table 11) (vẽ lại từ số liệu trong ghi chú)

Cách đọc hình: số trên mỗi điểm là chênh lệch P200-R50 trừ S50. Ở depth lợi thế co từ +3.7 xuống -0.2 khi prune 81.3%, còn ở width vẫn giữ +5.1. Khoảng cách giữa hai đường là phần được hưởng từ trọng số thừa hưởng của mô hình cha.

- Ví dụ per-benchmark ở 50%: Minitron-D thắng S50 trên cả 8 benchmark, +4.5 ARC-C, +6.1 HellaSwag; Minitron-W +8.2 ARC-C, +10.2 HellaSwag. Paper giải thích Minitron-W giữ nhiều năng lực của cha hơn Minitron-D (theo Minitron).
- Sparse: tinh hơn thì thắng nhiều hơn. Wanda: PPL WikiText-2 tốt hơn S50 1.40 (2:4) so với 2.30 (unstructured); SparseGPT: ARC-C +2.0% (2:4) so với +8.1% (unstructured).
- **Điều quan trọng về xếp hạng trước retrain:** Table 10 cho thấy P200 + Minitron-W ngay sau khi cắt có PPL C4 24686.5 và WikiText-2 36761.5 (gần như hỏng), nhưng retrain 50B token phục hồi hoàn toàn (66.3 avg); tác giả kết luận PPL ngay sau prune là chỉ báo kém của chất lượng sau retrain (giống phát hiện "đảo thứ hạng" của Minitron, bài 39). So sánh: P200 + Minitron-D sau cắt: PPL C4 504.0; avg 41.5. Sheared-LLaMA sau cắt: avg 63.3 (P200), Wanda-U: 64.3, SparseGPT-U: 65.4.

4.3 Finding 2: cho scratch đủ tổng token thì structured bị bắt kịp

- Minitron-D: S250 vượt P200-R50 trên mọi benchmark (PPL WikiText-2 thấp hơn 0.40; accuracy cao hơn 0.6-4.0 điểm; trung bình +1.8 trong Table 16 với Δ = scratch - pruned).
- Minitron-W: biên nhỏ hơn (OBQA: từ 4.0 xuống 2.2) nhưng S250 vẫn nhỉnh hơn (+1.2 avg).
- FLAP và Sheared LLaMA: P200-R50 vẫn nhỉnh hơn S250 ở accuracy trung bình (FLAP 66.5 vs 65.1, Sheared 65.9 vs 65.3) và ở PPL (FLAP 7.97 vs 8.82). Lợi thế còn sót của FLAP là BoolQ (+8.7 so với S250), của Sheared là OBQA (+3.8), theo Phụ lục H.4.
- Sparse (2:4): S250 PPL tốt hơn nhưng thua P200-R50 trên 6/8 benchmark (Wanda-2:4: −1.8 WinoG, −1.2 ARC-C). Unstructured (Wanda-U): S250 thua trên 6/8 benchmark, chỉ BoolQ và OBQA tốt hơn (+0.4, +0.8).
- Cũng trong Phụ lục: FLAP (structured) có PPL tốt nhất tổng thể, vượt mọi sparse, là một phản ví dụ cho khẩu hiệu "sparse > structured".

4.4 Finding 3: tinh hơn thì khởi tạo tốt hơn nhưng phải trả bằng hiệu quả phần cứng

Cùng tỉ lệ prune, hạt càng mịn càng thắng scratch nhiều. Nhưng trên TPU v4-256 (không có sparse tensor core), cả 2:4 và unstructured chạy tốc độ như mô hình dense (1.0x). Table 9 (mô hình P200, FLOPs lý thuyết một forward chuỗi 2048; "speedup" là tăng tốc đo trên TPU):

Phương pháp	Tỉ lệ	FLOPs (TF)	Speedup
Base	0%	32.9	1.0x
Minitron-D	50 / 62.5 / 75 / 81.3%	17.5 / 9.8 / 6.0 / 4.1	1.9x / 4.2x / 7.2x / 11.2x
Minitron-W	50 / 62.5 / 75 / 81.3%	19.1 / 12.6 / 9.8 / 7.9	1.4x / 2.0x / 2.4x / 2.9x
FLAP	50%	17.7	1.5x
Sheared LLaMA	50%	18.9	1.4x
Wanda, SparseGPT (50% hoặc 2:4)	50%	16.5	1.0x

Cột Speedup có vẻ đo tốc độ bước huấn luyện; các giá trị 4.2–11.2 lần của depth cao hơn đáng kể so với tỷ lệ FLOPs $32.9/9.8 \approx 3.4$. Cách đo chưa rõ, vì vậy chỉ nên xem đây là thứ tự tương đối. Bài báo lưu ý rằng 2:4 cần GPU có sparse tensor core (như A100/H100) và cuSPARSELt để tăng tốc suy luận; dạng thưa không cấu trúc thì không.

4.5 Token scaling (Tables 12-15, 4 phương pháp structured, 50%, trung bình accuracy)

Minitron-depth: S10 49.8 -> P200-R10 60.7; S30 55.5 -> 62.6; S50 60.7 -> 64.4; S250 66.0 -> 67.0 (P200-R250); S500 67.3 -> 68.1 (P200-R500). Minitron-width: S10 46.8 -> 63.3; S50 59.0 -> 65.7; S250 67.4 -> 68.3; S500 68.5 -> 69.6. FLAP: S500 66.9 -> 70.7. Sheared LLaMA: S500 67.0 -> 69.6 ($\Delta \geq +2.6$ ở mọi budget, là phương pháp structured duy nhất không bao giờ vượt về 0). Với cùng số token retrain N (không phải tổng token), pruned luôn dẫn nhưng khoảng cách co lại.

Meta-RN (cắt từ checkpoint Meta thật) đứng đầu hơn P200-RN ở budget thấp (Minitron-D ΔS10 +12.1 vs +10.9), khoảng cách đóng dần khi N tăng; tác giả giải thích là tín hiệu còn lại của hỗn hợp pretrain gốc của Meta.

5. Phân tích / ablation

- **Tỉ lệ prune:** lợi thế của prune giảm khi tỉ lệ tăng; depth mất sạch ở 81.3%, width vẫn giữ +5.1.
- **Độ mịn (granularity):** unstructured > 2:4 > width > depth về lợi thế so với scratch cùng tỉ lệ.
- **Token retrain:** lợi thế co theo N nhưng với avg accuracy không biến mất đến 500B (Minitron-D +0.8, Minitron-W +1.1, FLAP +3.8, Sheared +2.6 so với S500). Phụ lục H.3 viết rằng BoolQ là benchmark đầu tiên đổi dấu khi N tăng (−2.0, −5.4, −4.6 ở 500B) và suy ra BoolQ thưởng quy mô dữ liệu hơn khởi tạo. **Nhưng khi đối chiếu bảng, các giá trị −2.0 / −5.4 / −4.6 nằm ở cột WinoGrande (Minitron-D) và OBQA (Minitron-W, FLAP), còn cột BoolQ ở 500B vẫn dương (+0.4, +7.0, +13.5).** Có vẻ phần chữ gán nhầm tên benchmark; ta nên tin bảng và nhớ rằng WinoG và OBQA mới là các điểm đổi dấu (OBQA nhiễu ±2.2).
- **Giải thích cơ chế (Phụ lục G, tác giả đề xuất, dựa trên tài liệu có sẵn, chưa kiểm chứng trực tiếp):** (1) Lottery Ticket và Evci et al.: tối ưu mô hình thưa với mask cố định từ ngẫu nhiên khó (kẹt cực tiểu cục bộ); (2) Liu et al. 2019: trọng số của mô hình unstructured sau fine-tune lệch với bản scratch nhiều hơn structured, nên giá trị trọng số thừa hưởng, không chỉ mẫu thưa, là nguồn lợi thế.
- **Nội dung "cái gì được chuyển từ cha":** tác giả suy ra rằng thông tin ở mức trọng số được chuyển sang (sparse), còn hình dạng kiến trúc thì không (structured, có thể bù bằng train lâu); đề xuất công thức bất đối xứng: structured chỉ đáng khi token là ràng buộc chính.

6. Giới hạn

Tác giả nêu (Phụ lục A):

1. Chỉ một họ mô hình (Llama-3.1-8B), không thử Qwen/Gemma/OLMo hoặc cỡ nhỏ/lớn hơn.
2. Một corpus (DCLM); khoảng cách sparse-structured có thể đổi theo phân phối dữ liệu; Meta-RN không kiểm soát được hỗn hợp pretrain của Meta.
3. **Không có baseline knowledge distillation:** so với LM-loss retraining thường, không có pipeline distillation như Minitron/Sheared, có thể thu hẹp hoặc xoá khoảng cách cho structured.
4. Một lần xáo trộn dữ liệu, không đo phương sai giữa các lần chạy.
5. Một kiểu khởi tạo ngẫu nhiên chuẩn (không µP, không depth-aware scaling, không warm-start từ mô hình nhỏ).

Nhận xét:

- Mô hình cha P200 chỉ pretrain 200B token, yếu hơn nhiều so với một mô hình cha thực tế (68.6 so với 74.0 avg); cha yếu làm lợi thế của prune bị đánh giá thấp, hoặc ngược lại, khó nói. Paper có Meta-RN để cho thấy xu hướng không đổi.
- Có vài sai khác nhỏ giữa các bảng cho cùng một thí nghiệm (ví dụ Minitron-W 50%: Table 2/16/11 báo S50 avg 59.9, P200-R50 66.3; Table 13 báo 59.0 và 65.7; FLAP: Table 2 báo 66.5, Table 14 báo 67.0), có thể do lần đánh giá khác nhau; chênh nhỏ so với các khoảng cách chính, nhưng cần lưu ý khi trích số.

- Tập benchmark là trắc nghiệm ngôn ngữ ngắn (nhiễu ±1-2 điểm); chênh lệch dưới 1 điểm trong kết luận về sparse ở S250 nên được coi là chưa chắc chắn.
- Số liệu "speedup" ở Table 9 mô tả tốc độ trên TPU huấn luyện chứ không phải độ trễ suy luận trên GPU của người dùng.

7. Liên hệ với các bài khác

- **11 Rethinking the value of network pruning (Liu et al. 2019), 08 Lottery Ticket, 09 Linear mode connectivity:** bài này là phiên bản LLM của câu hỏi ở bài 11: kết luận chia theo độ mịn, đúng như bài 11 gợi ý (structured: giá trị ở kiến trúc; unstructured: giá trị ở trọng số).
- **38 Sheared LLaMA, 39-40 Minitron:** các phương pháp được thử lại trong cùng điều kiện; bài này xác nhận prune có lợi khi token ít, nhưng ngầm nói rằng phần lớn lợi thế của Sheared/Minitron so với "mô hình cộng đồng" có thể do thừa hưởng token của cha (hàng nghìn tỷ) chứ không chỉ do thuật toán.
- **42 Puzzle:** Puzzle dùng token cực ít (45B) từ cha 15T token; bài này cho thấy lợi thế đó phụ thuộc vào việc chi phí cha đã "trả xong".
- **23-24 SparseGPT, Wanda; 18 2:4 sparsity:** các baseline sparse; kết quả cho thấy chúng tạo khởi tạo mạnh nhất nhưng ít lợi suy luận trên phần cứng không hỗ trợ.
- **33-34 Shortened LLaMA, SLEB, 31 ShortGPT; 37 Repair:** depth pruning: bài này cho thấy depth pruning mất lợi thế khi tỉ lệ cao.
- **47 Jet-Nemotron, 44-46:** các hướng khác về thiết kế mô hình nhỏ.

8. Gợi ý áp dụng

- Trong Table 11, nếu ghép lại: P200-R50 avg (cùng 4.5B) width 66.3 vs depth 64.4, tức width hồi phục tốt hơn depth sau 50B token LM-loss. Cần thí nghiệm trực tiếp.
- **Post-prune loss không đáng tin để chọn cấu hình** (Table 10: PPL 36761 mà phục hồi hoàn toàn). Nên chọn bằng retrain ngắn, không bằng điểm ngay sau cắt.
- Kết quả "sparse thắng" nhìn chung ít liên quan.
- **Cảnh báo chung:** đo kết quả bằng CER/WER chứ không bằng benchmark trắc nghiệm của paper.

9. Thí nghiệm gợi ý

Đây là kiểm tra trực tiếp xem prune còn đáng không khi đã có teacher và ở tỉ lệ cao.

10. Tóm tắt công thức / khái niệm

Khái niệm	Nội dung
Pruning as initialization	Trọng số đã cắt chỉ là điểm bắt đầu cho mô hình nhỏ; baseline đúng là cùng kiến trúc train từ ngẫu nhiên
S_N / P_{200}-R_N / Meta-R_N	scratch NB token / pretrain cha 200B, prune, retrain NB / như vậy nhưng từ checkpoint Meta
Equal training budget	S50 vs P200-R50
Equal total budget	S250 vs P200-R50 (250 = 200 pretrain + 50 retrain)
Minitron-depth	bỏ khối liên tiếp làm tăng val loss ít nhất
Minitron-width	giữ kênh có chuẩn activation $\|X_{:,j}\|_2$ lớn
Wanda	$\|W_{ij}\| \cdot \|X_j\|_2$
Finding 1	prune thắng ở cùng token train; lợi thế co khi tỉ lệ tăng, mất ở 81.3% với depth
Finding 2	scratch được cấp đủ tổng token bắt kịp structured; sparse còn lợi thế nhẹ

Khái niệm	Nội dung
Finding 3	độ mịn cao hơn cho khởi tạo tốt hơn nhưng ít lợi suy luận

11. Kết luận ngắn

Khi tính cả chi phí token của mô hình cha, lợi thế của “prune rồi retrain” so với huấn luyện từ đầu thu hẹp lại: structured pruning bị bắt kịp ở 250B token, chỉ sparse pruning giữ được chút lợi thế (nhưng khó tăng tốc). Khi token là ràng buộc chính và cha đã có sẵn thì prune thắng rõ, với lợi thế giảm theo tỉ lệ prune.

Link gốc: https://arxiv.org/abs/2606.14150

Bài 44

Nemotron-H: A Family of Accurate and Efficient Hybrid Mamba-Transformer Models (2025)

Tác giả: NVIDIA (đội Nemotron; phần nén mô hình có Sreenivas, Muralidharan, Bercovich, Taghibakhshi, Molchanov và cộng sự).
Venue / arXiv: arXiv:2504.03624 (v4, 5/9/2025). Bài không nêu venue hội nghị; đây là technical report.
Nguồn: https://arxiv.org/abs/2504.03624
Chủ đề: Kiến trúc hybrid Mamba-2 / attention / FFN, và **MiniPuzzle**: nén Nemotron-H-56B xuống 47B bằng pruning (cắt layer + cắt neuron FFN) rồi distillation.
Vị trí trong lộ trình: Đọc trước: 39 Minitron, 40 Llama-3.1-Minitron, 42 Puzzle (MiniPuzzle là "Minitron đơn giản + Puzzle linh hoạt"), 33–34 (cắt layer). Bài này thêm: lần đầu áp công thức pruning + distillation lên một hybrid Mamba-Transformer **quy mô lớn** và cho biết *cắt cái gì, giữ cái gì*. Tiếp theo: 45 Minitron-SSM (cắt cả bên trong Mamba), 46 Mamba-Shedder, 47 Jet-Nemotron.

> Ghi chú học tập bằng tiếng Việt, không phải bản dịch. Đây là technical report dài (FP8 training, dữ liệu, VLM, instruct...); **ghi chú này chỉ tập trung vào kiến trúc (tóm tắt) và Mục 3 "Compression and Distillation"**.

1. Bối cảnh

- **Self-attention** cần $O(n)$ phép tính cho mỗi token được sinh (n = số token đã thấy) và phải lưu **KV cache** (bộ nhớ key/value của mọi token cũ), vì vậy độ trễ và bộ nhớ tăng khi ngữ cảnh dài.
- **Mamba-2** (một loại *state space model*, SSM): thay vì nhìn lại mọi token, nó nén lịch sử vào một **trạng thái cố định kích thước** (constant memory per token). Rẻ ở context dài, nhưng khả năng "tra cứu chính xác" yếu hơn attention. Do đó có xu hướng **hybrid**: phần lớn layer là Mamba, giữ ít layer attention.
- Câu hỏi của bài: (a) hybrid có đạt độ chính xác của Transformer cùng cỡ không? (b) làm sao tạo ra một biến thể **nhỏ hơn** (cho GPU RTX 5090 32 GiB) mà không phải huấn luyện lại từ đầu?
- Tiền đề cần nhớ: **pruning** = bỏ bớt thành phần của mô hình đã huấn luyện; **distillation** = cho mô hình nhỏ (student) bắt chước phân phối đầu ra của mô hình lớn (teacher). Hai thứ kết hợp gọi là "compression via pruning and distillation" (Minitron, bài 39).

2. Ý tưởng chính

Nemotron-H là họ mô hình (8B, 47B, 56B) trong đó khoảng 8% layer là self-attention, phần còn lại xen kẽ Mamba-2 và FFN. Để có bản 47B từ bản 56B, tác giả đề xuất **MiniPuzzle** gồm ba bước: (1) đo độ quan trọng của từng *layer* và từng *neuron FFN* chỉ bằng forward pass, (2) liệt kê khoảng 400 kiến trúc ứng viên thoả ràng buộc bộ nhớ rồi chọn dần xuống 130 rồi 3 ứng viên (NAS có điều kiện), (3) chưng cất ngắn (~7B token) để chọn ứng viên tốt nhất, rồi chưng cất dài (63B token). Kết quả: 47B gần như không mất độ chính xác so với 56B, nhanh hơn 1.2x ở context dài, dùng ít hơn khoảng 300x số token so với huấn luyện từ đầu (theo bài báo).

3. Kiến trúc hybrid của Nemotron-H (tóm tắt)

	Số layer	Hidden	FFN dim	Q heads	KV heads	State dim	Mamba groups
Nemotron-H-8B	52	4096	21504	32	8	128	8

	Số layer	Hidden	FFN dim	Q heads	KV heads	State dim	Mamba groups
Nemotron-H-56B	118	8192	32768	64	8	256	8

- Mỗi "layer" là **một** trong ba loại: Mamba-2, self-attention, hoặc FFN (khác Transformer chuẩn, nơi một block gồm attention + FFN). Có skip connection quanh mỗi layer.
- Số layer attention khoảng 8% tổng (4/52 ở 8B, 10/118 ở 56B), **rải đều** trong mô hình; phần còn lại chia đều Mamba-2 và FFN (56B: 54 Mamba-2 + 54 FFN + 10 attention = 118).
- Ràng buộc thiết kế: layer đầu là Mamba-2, layer cuối là FFN, attention luôn đứng trước FFN. Không dùng position embedding. FFN dùng squared ReLU, GQA với 8 KV head.
- Có bản đối chứng Transformer cùng dữ liệu (Nemotron-T-8B). Tốc độ: 56B sinh nhiều hơn 2.4x token/s/GPU so với Qwen-2.5-72B và Llama-3.1-70B ở input 65536 / output 1024 (paper nêu 2.9x sau khi distill sang 47B).

4. MiniPuzzle: phương pháp chi tiết

Mục tiêu triển khai: chạy **context 1 triệu token** trên một RTX 5090 (32 GiB) với trọng số FP4. Bản 56B riêng trọng số đã cần khoảng 29.5 GiB, chỉ còn 2.5 GiB cho KV cache và activation, nên cần nhỏ hơn.

4.1 Hai trục được cắt (và hai trục không cắt)

- **Cắt:** (a) toàn bộ *layer* (depth) thuộc cả ba loại; (b) *neuron ẩn của FFN* (width của FFN).
- **Đã thử nhưng bỏ:** cắt *Mamba heads* và *Mamba head dimension*: "both axes caused severe accuracy degradation (particularly head dimension pruning)". Đây là lý do sang bài 45 (Minitron-SSM) xử lý riêng bài toán cắt trong Mamba.
- Tác giả không dùng độ quan trọng dựa trên gradient (không khả thi ở quy mô này); chỉ dùng forward pass.

4.2 Độ quan trọng

Layer importance (theo Puzzle, bài 42): MSE giữa activation ngay trước LM head của mô hình đầy đủ và của mô hình bị bỏ layer đó, trung bình trên 128 mẫu huấn luyện:

$$\boxed{\mathrm{Imp}(\ell) = \mathbb{E}_x \left\| h^{\text{full}}_{\text{pre-head}}(x) - h^{\text{drop } \ell}_{\text{pre-head}}(x) \right\|_2^2}$$

Neuron FFN importance (theo Minitron, bài 39): $\mathrm{FFN}(X) = \delta(XW_1^\top)W_2$ với δ = squared ReLU. Với neuron i (hàng i của W_1), điểm là tổng hợp activation của nó trên batch B và chuỗi S:

$$F^{(i)}_{\text{neuron}} = \sum_{B,S} \delta(X(W_1^i)^\top)$$

Dùng 1024 mẫu hiệu chỉnh (calibration), tổng hợp bằng mean và ℓ_2-norm theo kết luận của Minitron.

Quan sát trên Hình 9 của paper: layer quan trọng tập trung ở **đầu và cuối** mô hình (giống Minitron); đáng chú ý, dù chỉ có 10 layer attention, **một số attention layer lại thuộc nhóm ít quan trọng nhất** (đặc biệt layer 84, attention thứ 7), trong khi layer 40, 51, 62, 73 quan trọng hơn lân cận.

4.3 Conditional NAS (3 bước)

1. **Liệt kê ứng viên.** Quét lưới số layer cho từng loại (attention, FFN, Mamba-2) và FFN hidden dim $\in \{24576, 25600, \ldots, 32768\}$. Với mỗi cấu hình, giữ các layer/neuron điểm cao nhất; FFN của mọi

layer cắt về cùng một độ rộng. Chỉ giữ kiến trúc có bộ nhớ < **31.7 GiB** (FP4, context 1M token), còn khoảng **400** ứng viên.

2. **Xếp hạng nhẹ.** Chấm bằng validation corpus 1M token với hai metric: *next-token accuracy* và *next-token parent agreement* (child đồng ý với parent về token đoán tham lam bao nhiêu lần). Giữ top **130**.
3. **Chọn ứng viên.** Benchmark trên một tập tác vụ, chọn **3** ứng viên; distill ngắn mỗi cái ~7B token để chọn ứng viên cuối.

4.4 Distillation

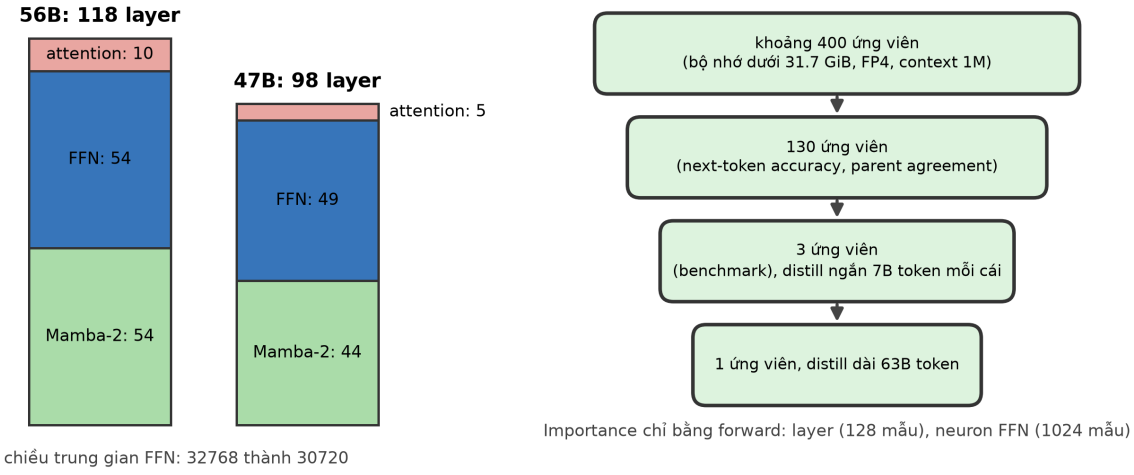
Logit-based, chỉ dùng **forward KL** (teacher = 56B). Tất cả chạy FP8, temperature 1.0, độ dài chuỗi 8192, batch 768 mẫu. Chi tiết loss lấy từ Mục 3 của Minitron.

4.5 Pseudo-code

```
MiniPuzzle(parent, memory_budget):
  # 1. Importance (chỉ forward)
  for each layer l: I_layer[l] = MSE(parent_out, parent_out_without_l)   # 128 mẫu
  for each FFN neuron j: I_neuron[j] = agg_{B,S} activation_j            # 1024 mẫu
  # 2. Conditional NAS
  C = []
  for (n_attn, n_ffn, n_mamba, d_ffn) in grid:
      child = keep top-n layers per type by I_layer, keep top-d_ffn neurons per FFN
      if memory_FP4(child, ctx=1M) < 31.7 GiB: C.append(child)          # ~400
  C = top_130(C, by=[next_token_acc, parent_agreement])                  # 1M-token val
  C = top_3(C, by=benchmark_subset)
  # 3. Distillation
  for c in C: distill(c, teacher=parent, tokens=7B)                      # chọn cái tốt
  ↪  nhất
  best = argmax avg_benchmark(C)
  distill(best, teacher=parent, tokens=63B)                              # bản 47B cuối
```

5. Thí nghiệm và kết quả chính

Cấu trúc 47B cuối (Mục 3.5): giữ 5 attention layer (từ 10), 44 Mamba-2 (từ 54) và 49 FFN (từ 54); chiều trung gian FFN giảm từ 32768 xuống 30720. Tổng số layer là 98, so với 118 ban đầu.



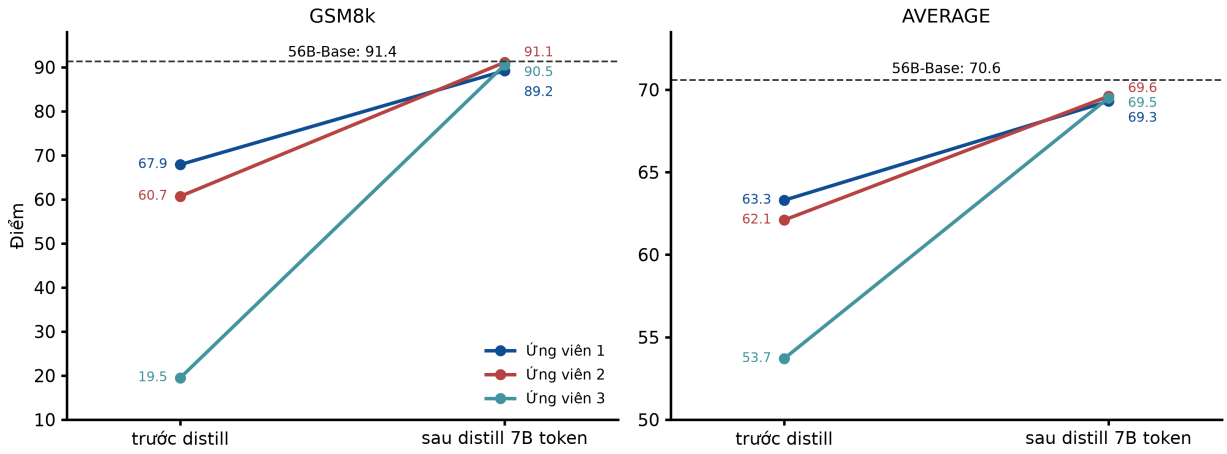
Hình 79: MiniPuzzle: số layer mỗi loại của 56B và 47B, cùng phễu chọn ứng viên từ khoảng 400 xuống 1 (minh hoạ, tự tính)

Cách đọc hình: bên trái là cột xếp chồng số layer Mamba-2, FFN và attention trước và sau khi nén (118 thành 98). Bên phải là phễu chọn ứng viên theo bộ nhớ, rồi theo hai metric, rồi theo benchmark kèm distill ngắn, cuối cùng là distill dài một ứng viên.

Ứng viên trước/sau distill ngắn (Bảng 6, định dạng "trước → sau"):

Benchmark	56B-Base	Cand. 1	Cand. 2	Cand. 3
MMLU	84.2	81.5 → 83.7	80.8 → 83.2	81.3 → 83.6
Commonsense (avg)	67.6	64.2 → 66.6	63.8 → 67.2	64.5 → 66.8
GSM8k	91.4	67.9 → 89.2	60.7 → 91.1	19.5 → 90.5
Code (avg)	67.0	52.0 → 64.9	52.7 → 64.6	23.0 → 64.9
AVERAGE	70.6	63.3 → 69.3	62.1 → 69.6	53.7 → 69.5

Ứng viên 2 được chọn (trung bình sau distill cao nhất, 69.6) cho đợt distill 63B token.



Hình 80: Ba ứng viên 47B trước và sau distill ngắn 7B token: GSM8k và điểm AVERAGE (Bảng 6) (vẽ lại từ số liệu trong ghi chú)

Cách đọc hình: mỗi đường nối điểm trước distill với điểm sau distill của một ứng viên, đường nét đứt là 56B-Base. Ứng viên 3 kém hẳn trước distill (GSM8k 19.5) nhưng sau distill ba ứng viên gần nhau (AVERAGE 69.3 đến 69.6). Vì vậy điểm ngay sau khi cắt không dự đoán chắc chắn điểm sau distill.

56B vs 47B cuối (Bảng 4):

Task	56B-Base	47B-Base
MMLU-Pro (5-shot COT)	60.5	61.8
MMLU (5-shot)	84.2	83.6
GSM8k (8-shot COT)	93.7	93.3
MATH (4-shot COT)	59.4	57.4
MATH Level 5	35.2	34.1
MMLU STEM	80.6	79.8
HumanEval	60.4	61.0
HumanEval+	54.3	56.1

Tốc độ: 1.2x suy luận context dài so với 56B; 2.9x token/s/GPU so với Qwen-2.5-72B / Llama-3.1-70B. Chi phí nén: ~7B token (chọn ứng viên) + 63B token (distill chính).

6. Phân tích / ablation

- **Distill ngắn quan trọng để chọn ứng viên**: trước distill, ứng viên 3 kém hẳn (GSM8k 19.5 so với 60-68), nhưng sau 7B token cả ba gần bằng nhau (GSM8k 89.2 đến 91.1). Tức là điểm "trước distill" **không dự đoán chắc chắn** điểm sau distill; paper nói bước này "crucial" để chọn đúng.
- **Lựa chọn cuối phụ thuộc mục tiêu**: paper lưu ý ứng viên tốt nhất thay đổi tuỳ ưu tiên (code hay commonsense).
- **Điểm quan trọng không đồng nghĩa với "attention luôn quan trọng"**: một số attention layer bị xếp hạng thấp, và 47B giữ đúng một nửa số attention.
- **Trục Mamba không cắt được bằng cách đơn giản**: nguyên văn "severe accuracy degradation" (không có bảng số trong bài này).

7. Giới hạn

Paper nêu / ngụ ý:

- Chỉ cắt hai trục (layer, FFN neuron); trục Mamba bị loại vì suy giảm nặng.
- Importance chỉ dựa vào forward pass; không so sánh với phương pháp có gradient.

Nhận xét:

- Không có ablation tách riêng "cắt layer" vs "cắt FFN neuron" trong bài này; ta không biết trục nào đóng góp bao nhiêu vào độ giảm.
- Không có ablation về hàm mất mát distill (chỉ forward KL).
- Kết quả đo trên bản 56B/47B; chưa rõ chuyển được xuống mô hình 1B hay nhỏ hơn.

8. Liên hệ với các bài khác

- **39, 40 Minitron / Llama-3.1-Minitron:** nguồn của importance neuron FFN và cách distill; MiniPuzzle giữ sự đơn giản đó.
- **42 Puzzle:** nguồn của layer importance bằng MSE và ý tưởng NAS có điều kiện theo ràng buộc phần cứng.
- **45 Minitron-SSM:** giải quyết đúng chỗ MiniPuzzle bỏ cuộc (cắt Mamba heads/channels).
- **46 Mamba-Shedder:** cắt block/SSM trong Mamba không cần huấn luyện lại.
- **33, 34 (Shortened LLaMA, SLEB), 31 ShortGPT:** cách đo độ quan trọng layer; ở đây dùng MSE đầu ra thay vì cosine/perplexity.
- **43 Pruning vs scratch:** con số "~300x ít token hơn so với huấn luyện từ đầu" là một dữ kiện ủng hộ pruning + distill.

9. Gợi ý áp dụng

- Cách đo độ quan trọng bằng MSE đầu ra khi bỏ layer và danh sách ứng viên có ràng buộc bộ nhớ có thể dùng nguyên.
- **Điều đáng chú ý:** attention layer **không** mặc nhiên là quan trọng nhất; nên đo từng layer thay vì "giữ hết attention".
- Bài này không so sánh trực tiếp hai trục cắt neuron và cắt layer. (Bài 49 trên VLM cho thấy chiều hướng phụ thuộc mức nén.)

10. Thí nghiệm gợi ý

1. Đo độ quan trọng theo từng layer, rồi so sánh với tác động khi bỏ riêng layer đó.
2. Chọn các kiến trúc ứng viên dưới cùng ngân sách tham số; so thứ hạng trước và sau một giai đoạn chưng cất ngắn.

11. Tóm tắt khái niệm

Khái niệm	Ý nghĩa
Mamba-2	Layer SSM, trạng thái cố định kích thước, rẻ ở context dài
Hybrid	Trộn Mamba-2 + attention + FFN; ~8% attention
MiniPuzzle	Importance → Conditional NAS → Distill (Minitron + Puzzle)
Layer importance	MSE trước LM head khi bỏ layer (128 mẫu)
Neuron importance	Activation tổng hợp của neuron FFN (1024 mẫu)
Parent agreement	Tỉ lệ child và parent đoán cùng token kế tiếp
Ràng buộc	< 31.7 GiB, FP4, context 1M token
Kết quả	56B → 47B (5 attn, 44 Mamba, 49 FFN; FFN 30720), 1.2x nhanh hơn, 63B token

12. Kết luận ngắn

Nemotron-H cho thấy hybrid Mamba-Transformer đạt độ chính xác ngang Transformer cùng cỡ nhưng nhanh hơn nhiều ở context dài. MiniPuzzle cho thấy có thể nén hybrid lớn bằng cách **cắt layer + neuron FFN, tránh trục Mamba**, rồi dùng chưng cất ngắn để chọn kiến trúc và chưng cất dài để phục hồi; 47B gần như ngang 56B. Kết quả gợi ý rằng nên chọn ứng viên *sau* chưng cất ngắn, thận trọng với điểm số ngay sau khi cắt.

Link gốc: https://arxiv.org/abs/2504.03624

Bài 45

Minitron-SSM: Efficient Hybrid Language Model Compression through Group-Aware SSM Pruning (2025)

Tác giả: Ali Taghibakhshi, Sharath Turuvekere Sreenivas, Saurav Muralidharan, Marcin Chochowski, Yashaswi Karnati (đồng tác giả chính) cùng Raviraj Joshi, Ameya Sunil Mahabaleshwarkar, Zijia Chen, Yoshi Suhara, Oluwatobi Olabiyi, Daniel Korzekwa, Mostofa Patwary, Mohammad Shoeybi, Jan Kautz, Bryan Catanzaro, Ashwath Aithal, Nima Tajbakhsh, Pavlo Molchanov (NVIDIA).
Venue / arXiv: arXiv:2504.11409 (v2, 31/10/2025). Bài không nêu venue.
Nguồn: https://arxiv.org/abs/2504.11409
Chủ đề: Cắt tỉa có cấu trúc cho hybrid Mamba-2 + attention: cắt Mamba heads / head channels theo **nhóm (group-aware)**, kết hợp cắt FFN, embedding, layer; phục hồi bằng distillation. Nén Nemotron-H 8B → 4B.
Vị trí trong lộ trình: Đọc trước: 39 Minitron, 28 FLAP, 44 Nemotron-H. Bài này thêm: giải đúng phần mà MiniPuzzle (44) phải bỏ, tức là cắt *bên trong* khối Mamba mà không phá cấu trúc SSM. Tiếp theo: 46 Mamba-Shedder (cắt cả khối/mô-đun SSM, không huấn luyện), 47 Jet-Nemotron.

Ghi chú học tập bằng tiếng Việt, không phải bản dịch.

1. Bối cảnh

- Minitron (39) cho thấy cắt tỉa có cấu trúc (neuron, head, layer) kết hợp chưng cất có hiệu quả trên Transformer. Nhưng với **hybrid** (Mamba + attention) thì "rất ít được khảo sát".
- Khó khăn: trong Transformer, neuron FFN và kênh embedding có tính **hoán vị tương đương (permutation equivariant)**: đổi thứ tự neuron rồi đổi thứ tự tương ứng ở ma trận sau thì kết quả y nguyên, nên ta có thể *sắp xếp theo điểm quan trọng rồi cắt phần đuôi*. Khối Mamba **không** có tính chất này một cách tự do (xem dưới), nên cắt kiểu cũ sẽ làm hỏng phép tính SSM.
- Tiền đề SSM: $h_t = A_t h_{t-1} + B_t x_t$, $y_t = C_t^\top h_t + D_t x_t$ (Mamba có tham số phụ thuộc đầu vào). Mamba-2 chia kênh thành **heads** (m_h head, mỗi head m_d kênh) và chia sẻ B, C theo **groups** (g nhóm, trạng thái d_s).

2. Ý tưởng chính

Một layer Mamba-2 có m_h heads, nhưng các head được gom thành g nhóm dùng chung B và C. Nếu sắp xếp lại head giữa các nhóm khác nhau, phép "broadcast" của $B_t x_t$ bị đổi nghĩa. Vì vậy tác giả sắp xếp và cắt head **chỉ trong từng nhóm** (group-aware), và cắt *cùng một tập kênh* cho mọi head. Đi kèm là công thức chấm điểm theo kích hoạt (không cần gradient) cho head/kênh Mamba, FFN neuron, kênh embedding, cộng với tìm kiếm kiến trúc nhỏ rồi distill. Kết quả: Nemotron-H 8B → 4B giữ hơn 96% độ chính xác, nhanh khoảng 2x so với mô hình cùng cỡ, dùng tới 40x ít token hơn.

3. Phương pháp chi tiết

3.1 Layer Mamba và ràng buộc nhóm

Từ đầu vào X (sau LayerNorm), năm phép chiếu: $z = W_z\text{LN}(X)$, $x = W_x\text{LN}(X)$ (kích thước $m_h \times m_d$), $B = W_B\text{LN}(X)$, $C = W_C\text{LN}(X)$ (kích thước $g \times d_s$), $dt = W_{dt}\text{LN}(X)$ (m_h). x, B, C qua conv1d, rồi $\tilde{y} = \text{SSM}(\hat{x}, \hat{B}, \hat{C}, A, D, dt)$, rồi $y = W_O(\text{RMSNorm}(\tilde{y}, z))$.

Phép $B_t x_t$ reshape B thành $g \times d_s$ và **broadcast** lên $x \in \mathbb{R}^{m_h \times m_d}$. Gọi $\mathcal{G}_g$ là tập head thuộc nhóm g. Hoán vị hợp lệ phải thoả:

$$\boxed{\mathcal{P}(h) \in \mathcal{G}_g \quad \forall h \in \mathcal{G}_g}$$

(chỉ hoán vị trong nhóm). Ví dụ trong paper: hoán đổi head H3 và H8 nằm ở hai nhóm khác nhau thì $B_t x_t$ mới **không** phải một hoán vị của $B_t x_t$ gốc.

Nhất quán kênh: kênh k của head phải được giữ hoặc bỏ **đồng loạt** cho mọi head (tập kênh chung).

3.2 Chấm điểm và xếp hạng head / kênh Mamba

Điểm kích hoạt (dùng đầu ra của W_x): $s = \mathrm{LN}(X)W_x^\top$, tổng hợp theo batch B và độ dài L (mean cho một trục, ℓ_2 cho trục kia, theo Minitron):

1. Điểm kênh: $s_d = \|s_{:,d}\|_2$; giữ top-k_d kênh $\mathcal{D}_{top}$.
2. Điểm head (chỉ trên kênh được giữ): $f_h = \|s_{h,\mathcal{D}_{top}}\|_2$.
3. Trong mỗi nhóm, sắp xếp head theo f_h, giữ k_g head tốt nhất mỗi nhóm; ghép lại thành xếp hạng cuối.

```
GroupAwareMambaRank(s in R^{m_h x m_d}, k_d, {k_g}):
  s_d  = ||s[:,d]||_2 for each channel d
  D    = topk(s_d, k_d)
  f_h  = ||s[h, D]||_2 for each head h
  for g in groups:
      R_g = argsort_desc(f_h for h in G_g)[:k_g]
  return concat(R_g), D
# Cắt: W in {W_x, W_z, W_O, W_A, W_D, W_dt, conv1d} giữ các hàng/cột theo R, D
```

3.3 FFN, embedding, layer, FLAP

- **Neuron FFN:** $F^{(i)}_{neuron} = \sum_{B,L} X(W_1^i)^\top$ (như Minitron). **Kênh embedding:** $F^{(i)}_{emb} = \sum_{B,L} \mathrm{LN}(X)_i$, gộp trên mọi layer dùng kênh đó (FFN, Mamba, attention, LayerNorm).
- **Depth:** độ quan trọng layer = KL divergence giữa logit mô hình bỏ layer đó và mô hình đầy đủ, trung bình 256 mẫu.
- **FLAP mở rộng (28):** $S_j = \|W_j\|_2 \cdot \mathrm{Var}(X_j)$, áp lên đầu vào của OutProj của SSM. Dùng để so sánh với phương pháp ℓ_2 ở trên.
- **Attention không cắt**: chỉ chiếm ~8% số layer.

3.4 Phục hồi bằng distillation

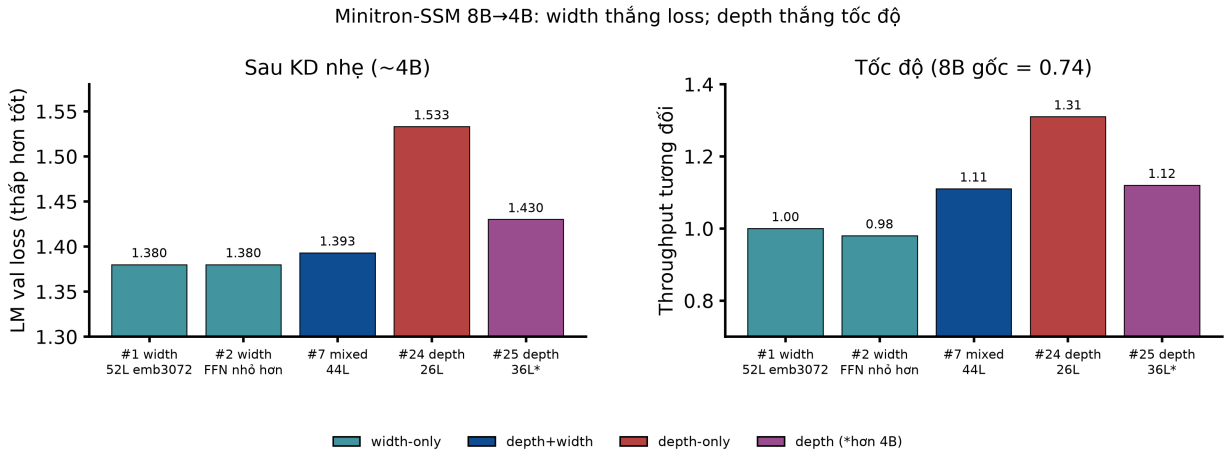
Logit-based, forward KL: $L_{logits} = \frac{1}{L}\sum_k \mathrm{FKLD}(p_t^k(x,\tau), p_s^k(x,\tau))$ với teacher = 8B gốc. Tham số: batch 768, độ dài 8192, cosine LR (bắt đầu 1.6e-4), warmup 60 bước. Dữ liệu: mẫu từ Phase 3 mixture của Nemotron-H.

3.5 Tìm kiếm kiến trúc (ngân sách 4B)

Không gian: số layer 26-52, embedding 3072-4096, FFN 9984-21504, Mamba heads 64-128, head channels 32-64; hơn một trăm ứng viên. Quy trình: (1) loss zero-shot trên 1024 mẫu; (2) top-22 ứng viên distill nhẹ 3.8B token; (3) chọn ứng viên tốt nhất (tie-break bằng throughput/latency) và distill dài ~380B token.

4. Thí nghiệm và kết quả chính

Mô hình: Nemotron-H 8B (52 layer, embedding 4096, FFN 21504, 128 Mamba heads, 64 head channels) → 4B. Kiểm tra thêm trên Mamba2-1.3B → 780M.



Minitron-SSM 8B → 4B: width-only loss 1.380 thắng depth-only 1.533; depth cho throughput cao hơn. (minh hoạ, tự tính)

Cấu hình cuối (ứng viên #1, Bảng 1): 52 layer, embedding 3072, FFN 12288, 112 Mamba heads, 64 head channels; LM val loss sau KD nhẹ **1.380**; throughput tương đối **1** (8B gốc: 0.74).

Trích Bảng 1 (sau KD nhẹ, ~4B tham số; loss thấp hơn là tốt):

#	Layers	Emb	FFN	Heads	Channels	LM loss	Rel. throughput
1	52	3072	12288	112	64	1.380	1
2	52	3072	10752	128	64	1.380	0.98
7	44	3072	14592	128	64	1.393	1.11
24	26	4096	21504	128	64	1.533	1.31
25*	36	4096	21504	128	64	1.430	1.12

(* nhiều tham số hơn 4B.) Ứng viên #1 và #2 cùng loss; chọn #1 vì throughput cao hơn nhờ ít Mamba head hơn.

Base 4B vs mô hình cùng cỡ (Bảng 4, trích): trung bình Nemotron-H-4B **64.5** (0.38T token) so với Qwen-2.5-3B 60.3 (18T), Zamba-2-2.7B 55.2 (3T), Falcon-3-3B 54.7 (0.1T), Llama-3.2-3B 51.1 (9T); 8B gốc 66.7 (15T). Ví dụ: MMLU 68.1 (4B) vs 72.7 (8B); GSM8K 69.6 vs 77.9; HumanEval 59.8 vs 57.3.

Tóm tắt trong paper: giữ >96% độ chính xác của 8B, throughput ~1.4x so với 8B, ~2.2x throughput và ~1.8x thấp latency so với Phi-4-4B (model đứng thứ hai); an toàn: Garak 67.77 (so với 70.75 của 8B), AEGIS 98.17 (so với 99.83).

Mamba2-1.3B → 780M (10.5B token): mô hình nén 780M ngang trung bình với bản 1.3B gốc và hơn bản 780M huấn luyện từ đầu 300B token (Bảng 8; chi tiết một số dòng: ARC-C 34.2 vs 28.6/33.2, HellaSwag 50.4 vs 54.7/59.9; nghĩa là không thắng mọi dòng).

5. Phân tích / ablation

- **Width-only thắng depth-only ở nén 50%:** ứng viên width-only #1 (loss 1.380) tốt hơn nhiều depth-only #24 (1.533). Ngay cả mô hình chỉ cắt depth giữ 36 layer (#25, nhiều hơn ~1.4x tham số) vẫn kém ứng viên width-only tệ nhất (#23). Tức *độ sâu quan trọng*.
- **Nhưng depth-only cho tốc độ cao nhất** (#24: 1.31x). Kết hợp depth + width (#7 với 44 layer) vẫn kém #1 về loss.

- **Độ nhạy theo trục:** độ sâu nhạy nhất với độ chính xác, tiếp theo là Mamba heads, còn FFN và embedding ít ảnh hưởng nhất. Ngược lại, cắt thành phần Mamba cho mô hình nhanh hơn cắt FFN/embedding (tương quan tham số Mamba–latency +0.94, –throughput –0.86).
- **Cắt Mamba heads tốt hơn cắt head channels** (LM loss, latency, throughput đều tốt hơn). Bài suy đoán (hypothesis) rằng Mamba chịu được việc cắt head vì có nhiều head hơn (128 so với 32 attention heads).
- **Điểm kích hoạt:** lấy từ đầu ra W_x thường cho zero-shot loss thấp nhất (ví dụ 3.56 so với 4.11 của W_z và 3.79 của W_O ở cấu hình đầu bảng 2).
- **FLAP vs ℓ_2 (Bảng 3):** trước KD kết quả lẫn lộn; sau KD ngang nhau cho ứng viên #1, FLAP không có lợi thế rõ.
- **Cắt depth thuần túy tụt ở toán và code:** cắt 4-26 layer + 126B token distill, benchmark "kiến thức lõi" hầu như không đổi nhưng math/code tụt nhiều, nhất là dưới 44 layer.
- **Layer importance (Hình 4):** quan trọng nhất ở đầu và cuối; **attention layer đầu tiên là một trong những layer ít quan trọng nhất**, các attention layer khác quan trọng hơn lân cận; có "mẫu răng cưa" MLP so với Mamba.

6. Giới hạn

Paper nêu / ngụ ý:

- Attention không được cắt (chỉ ~8% layer), nên chưa biết cách xử lý khi attention chiếm tỉ lệ lớn.
- Distillation cho long-context mới là "future work".

Nhận xét:

- Kết quả trên một họ mô hình (Nemotron-H/Mamba2); chưa kiểm tra trên các kiến trúc khác có ràng buộc chia sẻ tham số khác.
- So sánh với mô hình cộng đồng khác nhau cả dữ liệu lẫn token, nên "thắng" không chỉ nhờ pruning.
- Throughput đo tương đối trên phần cứng và cài đặt của tác giả.

7. Liên hệ với các bài khác

- **39 Minitron:** cùng khung importance bằng kích hoạt + KD; Minitron-SSM mở rộng sang Mamba.
- **44 Nemotron-H / MiniPuzzle:** MiniPuzzle bỏ trục Mamba vì suy giảm nặng; bài này cho thấy làm đúng ràng buộc nhóm thì cắt Mamba heads dùng được.
- **46 Mamba-Shedder:** cách tiếp cận khác (bỏ cả khối hoặc mô-đun SSM, training-free) và cũng thấy Mamba-2 chịu cắt SSM tốt hơn cắt khối.
- **28 FLAP:** nguồn gốc metric biến thiên x chuẩn cột.
- **31–34, 36 (cắt layer):** kết quả "depth nhạy hơn width" ủng hộ nhận định về độ sâu.
- **17 CoFi, 20 DepGraph:** nhắc lại ý "nhóm phụ thuộc phải cắt cùng nhau"; ràng buộc nhóm B/C ở đây là một dạng đó.

8. Gợi ý áp dụng

- Đáng kiểm tra riêng, đừng giả định.

9. Thí nghiệm gợi ý

3. So sánh điểm kích hoạt (ngõ ra proj x) với FLAP cho head tuyến tính; kỳ vọng ngang nhau như Bảng 3.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / ý
SSM chọn lọc	$h_t = A_t h_{t-1} + B_t x_t,\ y_t = C_t^\top h_t + D_t x_t$
Ràng buộc nhóm	$\mathcal{P}(h) \in \mathcal{G}_g$ cho mọi $h \in \mathcal{G}_g$
Điểm kênh / head	$s_d = \|s_{:,d}\|_2$; $f_h = \|s_{h,\mathcal{D}_{top}}\|_2$
Neuron FFN	$F^{(i)} = \sum_{B,L} X (W_1^i)^\top$
FLAP cho SSM	$S_j = \|W_j\|_2 \,\mathrm{Var}(X_j)$
Độ quan trọng layer	KL giữa logit bỏ layer và đầy đủ
Loss KD	forward KL trên logit
Cấu hình 4B	52 layer, emb 3072, FFN 12288, 112 heads, 64 channels

11. Kết luận ngắn

Với kiến trúc hybrid Mamba được nghiên cứu, phương pháp cắt các head Mamba **trong từng nhóm** và cắt cùng tập kênh cho mọi head. Phối hợp với cắt FFN + embedding và distill, Nemotron-H 8B → 4B giữ >96% độ chính xác với ~40x ít token hơn so với các mô hình cùng cỡ. Hai bài học tổng quát: (1) độ sâu nhạy nhất, nhưng cắt chiều rộng cho chất lượng tốt hơn ở cùng số tham số; (2) tốc độ thật phụ thuộc trục cắt (Mamba/layer đẩy nhanh hơn FFN/embedding).

Link gốc: https://arxiv.org/abs/2504.11409

Bài 46

Mamba-Shedder: Post-Transformer Compression for Efficient Selective Structured State Space Models (2025)

Tác giả: J. Pablo Muñoz, Jinjie Yuan (đồng tác giả chính), Nilesh Jain (Intel Labs / Intel Corporation).
Venue / arXiv: arXiv:2501.17088 (v1, 28/1/2025). Bản này không nêu tên venue.
Nguồn: https://arxiv.org/abs/2501.17088
Chủ đề: Cắt tỉa có cấu trúc **không cần huấn luyện** cho Mamba-1, Mamba-2 và hybrid (Zamba2, Hymba): bỏ cả block Mamba, mô-đun SSM, MHA, MLP, hoặc nhóm kênh MLP.
Vị trí trong lộ trình: Đọc trước: 34 SLEB, 31 ShortGPT (cắt block theo độ quan trọng), 44-45 (hướng NVIDIA). Bài này thêm: khảo sát **độ nhạy** của từng cấp độ chi tiết trong mô hình SSM và hybrid, và cho thấy Mamba-1 vs Mamba-2 phản ứng ngược nhau. Tiếp theo: 47 Jet-Nemotron.

> Ghi chú học tập bằng tiếng Việt, không phải bản dịch.

1. Bối cảnh

- **Mamba-1** (S6) và **Mamba-2** (SSD, "state space duality") là các lớp chuỗi có chi phí tuyến tính theo độ dài và trạng thái cố định khi sinh. Mamba-2 đổi ma trận A từ đường chéo thành "vô hướng nhân đơn vị" và đưa vào khái niệm *head*.
- **Hybrid:** Zamba2-2.7B (54 layer: 45 khối Mamba-2 đơn + 9 layer lai gồm Mamba-2 và khối Transformer dùng chung); Hymba-1.5B (Mamba và attention chạy *song song* trong cùng một "hybrid head").
- Pruning cho Transformer đã nhiều; cho SSM thì rất ít. Mamba-Shedder hỏi: những phần nào của mô hình SSM **thừa**, tức bỏ đi mà ít ảnh hưởng?
- **Cắt tỉa training-free:** chỉ chấm điểm rồi bỏ, không cập nhật trọng số (khác với bài 44-45 có distillation lớn).

2. Ý tưởng chính

Thuật toán tham lam đơn giản: ở mỗi bước, thử bỏ từng cấu trúc ứng viên (khối Mamba, mô-đun SSM, MHA, MLP, hay một nhóm kênh MLP), đo xem **perplexity** trên một tập hiệu chỉnh nhỏ (Alpaca) tăng bao nhiêu, rồi bỏ cấu trúc làm tăng ít nhất. Lặp lại t bước. Có thể cắt tuần tự theo cấp: trước cấu trúc lớn (khối), sau đó cấu trúc nhỏ (SSM, kênh MLP). Cuối cùng có thể tinh chỉnh nhẹ 2 epoch để phục hồi. Phát hiện chính: Mamba-1 chịu bỏ nguyên khối tốt hơn, Mamba-2 chịu bỏ mô-đun SSM tốt hơn; hybrid cần cắt đa cấp.

3. Phương pháp chi tiết

Thuật toán 1 (Block / Module Pruning):

$$\boxed{M_{\min} = \arg\min_{M_i \in \mathcal{M}} S_i, \quad S_i = \text{Importance}(M_i, m, C, \phi)}$$

với $\mathcal{M}$ là tập cấu trúc ứng viên, C tập hiệu chỉnh, ϕ metric (ở đây là perplexity). Bỏ $M_{\min}$, lặp t lần.

Thuật toán 2 (MLP Channel Pruning): như trên nhưng ứng viên là "g kênh cuối" của từng MLP (kích thước nhóm kênh g); bỏ nhóm làm tăng perplexity ít nhất.

```
Shedder(model m, candidates M, calib C, steps t):
  for k in 1..t:
      for each M_i in M:
```

```
          S_i = PPL(m without M_i, C)        # càng thấp càng thừa
      M_min = argmin_i S_i
      m = m without M_min
  return m
# Đa cấp: "&" = các loại ứng viên chấm cùng một bước; "+" = các giai đoạn nối tiếp
# vd: (Mamba block & MLP & MHA) + MLP channel + SSM
```

Các cấu trúc ứng viên: toàn bộ khối Mamba; mô-đun SSM (S6 hoặc SSD) bên trong khối Mamba (giữ lại linear projection và conv); với hybrid thêm khối Transformer, MHA, MLP, nhóm kênh MLP; với Hymba bỏ cả khối hybrid-head. Hiệu chỉnh bằng bộ Alpaca (theo BlockPruner); đo trên lm-eval-harness (Lambada PPL + 6 tác vụ: HellaSwag, PIQA, ARC-e, ARC-c, WinoGrande, OBQA).

4. Thí nghiệm và kết quả chính

Mô hình: Mamba-2.8B (64 khối S6), Mamba2-2.7B (64 khối SSD), Zamba2-2.7B (54 layer), Falcon-Mamba-7B, Hymba-1.5B-Base. Mỗi khối Mamba bỏ đi giảm 0.04B tham số. Không fine-tune, trừ khi ghi chú.

Bỏ nguyên khối Mamba (Bảng 1; Avg = trung bình 7 tác vụ):

Mô hình	Bỏ khối	Tỉ lệ	Lambada PPL	Avg
Mamba-2.8B	0	0%	4.23	59.9
	7/64	10.43%	4.94	57.7 (-2.2)
	14/64	20.86%	7.51	53.8 (-6.1)
Mamba2-2.7B	0	0%	4.10	60.2
	7/64	10.42%	8.43	56.2 (-4.0)
	14/64	20.83%	11.53	52.8 (-7.4)
Zamba2-2.7B	0	0%	4.01	67.2
	7/54	10.38%	6.80	60.5 (-6.7)
	14/54	20.77%	15.8	52.9 (-14.3)

Chỉ bỏ mô-đun SSM (Bảng 2):

Mô hình	Bỏ SSM	Lambada PPL	Avg
Mamba-2.8B	16/64	9.23	49.8 (-10.1)
	24/64	22.55	44.1 (-15.8)
Mamba2-2.7B	16/64	4.26	59.8 (-0.4)
	24/64	14.95	55.5 (-4.7)
Zamba2-2.7B	16/54	4.14	66.4 (-0.8)
	24/54	5.46	64.7 (-2.5)

Hybrid Zamba2, đa cấp (Bảng 3, trích, ngân sách ~10% / ~15%):

Chiến lược	Tỉ lệ	SSM bổ sung	PPL	Avg
Mamba block & Transformer block	10.40%	0	9.18	57.6 (-9.6)
Mamba block & MLP & MHA	10.33%	0	5.01	64.5 (-2.7)
... + MLP channel	10.27%	0	5.45	66.3 (-0.9)
... + MLP channel + SSM	10.27%	18/54	5.18	65.9 (-1.3)

Chiến lược	Tỉ lệ	SSM bổ sung	PPL	Avg
Mamba block & MLP & MHA + MLP channel	15.48%	0	7.39	62.3 (-4.9)
... + SSM	15.48%	18/54	7.43	61.3 (-5.9)

Mô hình khác: Hymba-1.5B bỏ 7/32 khối: Avg 61.7 (từ 63.8; trung bình 5 tác vụ). Falcon-Mamba-7B: bỏ 10/64 khối PPL 3.15→4.97, Avg 72.0→66.4; bỏ 20 khối PPL vọt lên 39.31, Avg 55.7; bỏ SSM 20/64 PPL 14.14, Avg 55.1.

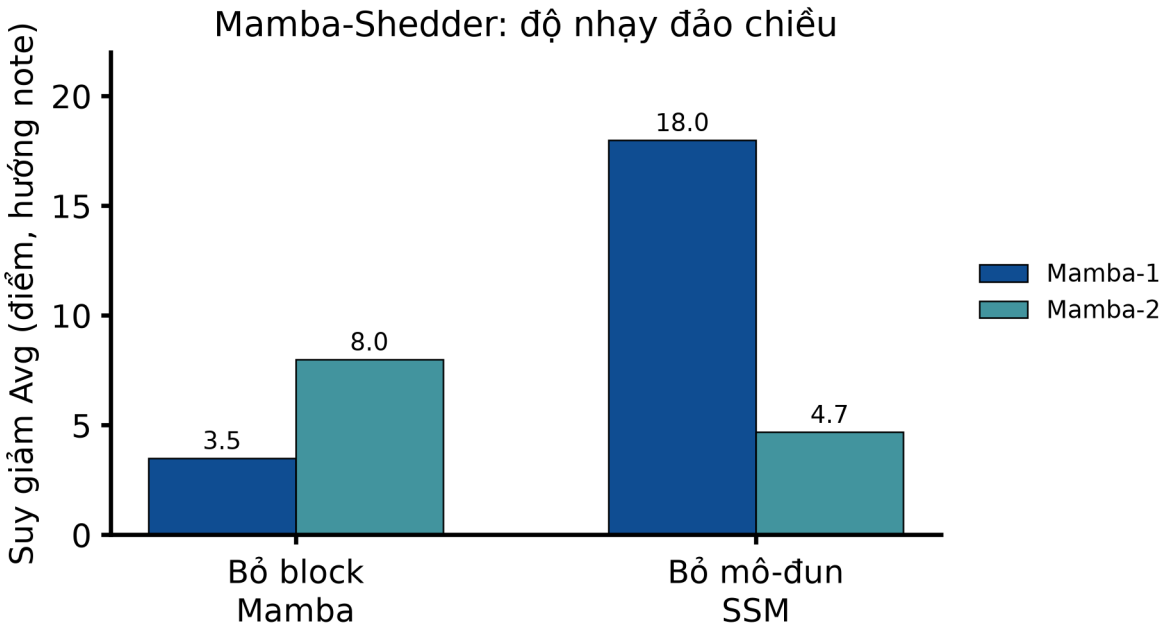
Tăng tốc (V100 32GB, batch 1, prompt 512):

Mô hình / cấu hình	Prefill	Decode
Mamba-2.8B, bỏ 7 khối	1.12x	1.13x
Mamba-2.8B, bỏ 14 khối	1.31x	1.29x
Mamba2-2.7B, bỏ 24 SSM	1.20x	1.18x
Zamba2 (15.48%, không SSM)	1.16x	1.34x
Zamba2 (15.48% + 18 SSM)	1.25x	1.39x
Hymba-1.5B, bỏ 7 khối	1.15x	1.24x

Fine-tune phục hồi (Alpaca đã làm sạch, 2 epoch): Zamba2 ở 10.27%, 18 SSM: PPL 5.18 → 4.58, Avg 65.9 → 67.0 (dense 67.2). Ở 15.48%: PPL 7.43 → 5.88, Avg 61.3 → 64.4. Hymba bỏ 7 khối: Avg 61.7 → 63.7 (dense 63.8). Mamba2-2.7B bỏ 20 SSM: PPL 5.89 → 4.44, Avg 58.6 → 59.6.

5. Phân tích / ablation

- **Mamba-1 vs Mamba-2 ngược nhau:** Mamba-1 chịu bỏ khối tốt hơn (PPL 4.23→7.51 ở 20.86%) nhưng rất nhạy khi bỏ SSM (Avg 44.1 ở 24 SSM); Mamba-2 ngược lại (bỏ 24 SSM chỉ giảm 4.7 điểm). Tác giả đoán các thay đổi trong Mamba-2 (giới hạn tính biểu đạt để tăng hiệu quả huấn luyện) làm nó nhạy hơn với bỏ khối nhưng bền hơn với bỏ SSM; đây là giả thuyết, không được kiểm chứng bằng ablation riêng.
- **Hybrid chịu bỏ SSM tốt nhất:** Zamba2 bỏ 12/54 SSM gần như không đổi PPL (4.01 → 4.02). Tác giả cho rằng bản chất hybrid giúp (giả thuyết).
- **Cắt đa cấp tốt hơn đơn cấp:** chỉ bỏ khối Mamba và khối Transformer nguyên vẹn là tệ nhất; tách thành MHA/MLP, rồi cắt thêm kênh MLP, là tốt nhất ở cùng tỉ lệ.
- **SSM có thể thừa thật:** thêm bước bỏ SSM sau khi đã cắt MLP kênh giảm PPL ở mức 10% (5.45 → 5.18), gợi ý một số SSM là dư thừa sau giai đoạn hai.
- **Đánh đổi tốc độ vs độ chính xác:** bỏ khối Mamba cho speedup lớn hơn bỏ SSM, nhưng bỏ SSM bảo vệ perplexity tốt hơn; lựa chọn tuỳ mục tiêu.
- **Fine-tune ngắn phục hồi gần như đầy đủ** ở mức 10% (Zamba2, Hymba).



Mamba-1 chịu bỏ block; Mamba-2 chịu bỏ SSM (note §5)

Mamba-1 chịu bỏ block hơn; Mamba-2 chịu bỏ mô-đun SSM hơn (hướng số note). (minh hoạ, tự tính)

6. Giới hạn

Paper nêu: đây là công trình đầu tiên khảo sát bỏ cấu trúc trong Mamba/hybrid; còn nhiều câu hỏi về tỉ lệ SSM/Transformer phù hợp; cần thêm nghiên cứu về đánh đổi hiệu quả-độ chính xác trước khi triển khai ở nơi ảnh hưởng đến con người. Không gian tìm kiếm đa cấp mới tuần tự, chưa chấm các cấp trong cùng một bước (future work).

Nhận xét:

- Thuật toán tham lam theo perplexity trên Alpaca: có thể thiên về tác vụ ngôn ngữ chung; chưa kiểm tra trên tác vụ chuyên biệt (OCR/tài liệu).
- Mỗi bước phải chạy lại mô hình cho mọi ứng viên: tốn tính toán khi mô hình lớn (paper không nêu chi phí).
- Chỉ mô hình ≤ 7B, tác vụ đa chọn/zero-shot; không có math/code/long-context.
- Chỉ khi fine-tune trên cùng dữ liệu Alpaca thì phục hồi mới được báo cáo; chưa có distillation.
- Chỉ số tham số và một GPU V100 cho tốc độ; chưa rõ trên GPU nhỏ hiện đại.

7. Liên hệ với các bài khác

- **31 ShortGPT, 34 SLEB, 33 Shortened LLaMA:** cùng triết lý bỏ khối bằng điểm quan trọng; Mamba-Shedder dùng perplexity (giống SLEB/BlockPruner).
- **45 Minitron-SSM:** bài đó cắt *bên trong* Mamba (heads/channels) có distillation; bài này bỏ cả mô-đun, training-free. Kết luận chung: Mamba-2 chịu cắt trong-SSM khá tốt.
- **44 Nemotron-H:** không cắt Mamba; bài 45 và bài này cho thấy có hướng khác.
- **29 SliceGPT, 26 LLM-Pruner:** các phương pháp cắt cho Transformer; Mamba-Shedder là phiên bản tương tự cho SSM.
- **47 Jet-Nemotron:** cũng khảo sát loại layer nào cần giữ trong hybrid.

8. Gợi ý áp dụng

- **Bài học đáng dùng:** cắt **đa cấp và tuần tự** (khối trước, kênh sau) tốt hơn chỉ cắt một cấp.

9. Thí nghiệm gợi ý

2. Thử cắt hai giai đoạn: bỏ 2-3 layer rồi cắt nhóm kênh MLP, so với chỉ bỏ layer hoặc chỉ cắt MLP ở cùng ngân sách tham số; so sánh trước và sau distill ngắn.
3. Đo xem điểm "PPL tăng khi bỏ layer" có dự đoán được điểm sau distill hay không (như bài học "điểm trước distill không chắc dự đoán" ở bài 44).

10. Tóm tắt khái niệm

Khái niệm	Ý nghĩa
S6 / SSD	Lõi SSM của Mamba-1 / Mamba-2
Bỏ khối Mamba	Mất ~0.04B tham số mỗi khối ở các mô hình 2.7B
Bỏ SSM	Giữ lại projection và conv, bỏ phần quét trạng thái
Channel group g	Nhóm kênh MLP bị bỏ cùng nhau
"&" vs "+"	cùng bước chấm điểm vs các giai đoạn nối tiếp
Chấm điểm	Perplexity trên Alpaca khi bỏ cấu trúc
Kết quả	tới 1.4x tăng tốc (Zamba2 decode 1.39x)

11. Kết luận ngắn

Mô hình SSM và hybrid có dư thừa có thể bỏ mà ít ảnh hưởng. Mamba-1 chịu bỏ khối, Mamba-2 chịu bỏ mô-đun SSM; hybrid hưởng lợi từ cắt đa cấp (khối, MLP, MHA, kênh MLP, SSM). Cắt training-free đã có lợi (tăng tốc tới ~1.4x), và tinh chỉnh 2 epoch đưa độ chính xác gần với mô hình đầy đủ ở tỉ lệ ~10%.

Link gốc: https://arxiv.org/abs/2501.17088

Bài 47

Jet-Nemotron: Efficient Language Model with Post Neural Architecture Search (2025)

Tác giả: Yuxian Gu, Qinghao Hu, Shang Yang, Haocheng Xi, Junyu Chen, Song Han, Han Cai (NVIDIA).
Venue / arXiv: arXiv:2508.15884 (v3, 28/9/2025). Bài không nêu venue.
Nguồn: https://arxiv.org/abs/2508.15884
Vị trí trong lộ trình: Đọc trước: 42 Puzzle, 44 Nemotron-H (hybrid ~8% attention), 31-34 (layer importance). Bài này thêm: học vị trí full-attention bằng super-network + beam search, thay vì rải đều; và phát hiện các attention layer đóng góp rất không đều và cho các năng lực khác nhau.

> Ghi chú học tập bằng tiếng Việt, không phải bản dịch. Bài này **không phải bài pruning thuần**: nó là kiến trúc/NAS bắt đầu từ mô hình đã huấn luyện, nhưng phép "bỏ hầu hết full-attention layer và thay bằng layer rẻ" là một dạng nén có cấu trúc nên được đưa vào lộ trình.

1. Bối cảnh

- **Full attention** có độ phức tạp $O(n^2)$ và KV cache lớn. **Linear attention** (RWKV7, RetNet, Mamba2, GLA, DeltaNet, Gated DeltaNet...) là $O(n)$, trạng thái nhỏ. Hybrid: giữ **vài** full-attention layer để giữ khả năng tra cứu/ MMLU, còn lại là linear.
- Hai vấn đề của hybrid trước đây: (1) độ chính xác vẫn thua các mô hình full-attention mạnh trên MMLU(-Pro), toán, retrieval, code, long-context; (2) **vị trí** của các full-attention layer thường chọn *đều đặn* (uniform) mà không có cơ sở.
- Huấn luyện kiến trúc mới từ đầu rất đắt, nên bài đề xuất khám phá kiến trúc *trên nền một mô hình đã huấn luyện*.
- Gloss: **NAS** (Neural Architecture Search) = tự động tìm kiến trúc. **Super-network once-for-all** = một mạng chứa nhiều đường đi (mỗi lớp có thể là full hoặc linear); huấn luyện một lần rồi trích mạng con. **KV cache** = bộ nhớ key/value của các token cũ.

2. Ý tưởng chính

PostNAS bắt đầu từ mô hình full-attention đã huấn luyện (Qwen2.5-1.5B hoặc 3B), **đóng băng toàn bộ MLP**, và chỉ thay/tối ưu khối attention theo bốn bước thô-đến-mịn: (1) học vị trí giữ full-attention (và loại bỏ phần còn lại); (2) chọn khối linear attention tốt nhất (Gated DeltaNet); (3) thiết kế khối mới JetBlock (conv động); (4) tìm siêu tham số theo throughput thật. Với bước (1): huấn luyện super-network mà mỗi lớp có thể là full hay linear, rồi dùng beam search để chọn k layer full-attention theo một mục tiêu phụ thuộc tác vụ. Kết quả là chỉ cần **2 layer full-attention trên 28 layer** để đạt độ chính xác ngang Qwen3-1.7B với throughput sinh cao hơn tới 53.6x.

3. Phương pháp chi tiết

3.1 Học vị trí full-attention (Mục 2.2, phần trọng tâm)

1. Xây **once-for-all super-network**: mô hình pretrained, mỗi layer attention được bổ sung đường linear attention (GLA được dùng cho đơn giản và nhanh hơn một chút; kết quả ổn định khi đổi loại linear attention).
2. Huấn luyện: mỗi bước **lấy mẫu ngẫu nhiên** một đường đi (một tập layer là full, còn lại linear) → mạng con; huấn luyện bằng **feature distillation loss** (khớp đặc trưng với mô hình gốc).

3. Sau huấn luyện, **beam search** chọn vị trí tối ưu cho số full-attention layer cho trước (ví dụ 2). Mục tiêu theo tác vụ:
   - MMLU: chọn cấu hình có loss thấp nhất trên đáp án đúng (tức tối đa hoá $-$loss);
   - toán và retrieval: chọn cấu hình có accuracy cao nhất.
4. Hình 5(a) trực quan hoá bằng cách cho *từng* layer làm full-attention duy nhất (các layer khác là linear) và chấm điểm.

```
PostNAS-Placement(pretrained model M, budget k, task T):
  S = M with each attention layer l augmented by linear path L_l
  freeze all MLP weights                        # chỉ học phần attention
  repeat many steps:
      path = random choice per layer in {full_l, linear_l}
      sub  = subnetwork(S, path)
      loss = feature_distill(sub, M)            # student = sub, teacher = M
      update(sub's trainable parts)
  # Tìm kiếm:
  best = beam_search(configs with exactly k full layers, objective = score_T)
  return best                                   # tập layer giữ full-attention
```

3.2 Các bước còn lại (tóm tắt)

- **Chọn khối linear** (Bảng 1, đo trên cùng khung): Gated DeltaNet đạt tổng thể tốt nhất, nhờ *data-dependent gating* (quyết định tập trung vào token hiện tại hay trạng thái lịch sử) và *delta rule* (cập nhật trạng thái bằng phần thông tin tăng thêm).
- **JetBlock:** thêm một "kernel generator" sinh **kernel conv nhân quả động** theo đầu vào, áp lên V; bỏ conv tĩnh trên Q, K. Reduction ratio 8, kích thước kernel 4.
- **Hardware-aware search:** lấy throughput sinh làm mục tiêu thay vì số tham số; giữ **kích thước cache cố định** và lưới tìm trên key dim, value dim, số head (Bảng 2).

4. Thí nghiệm và kết quả chính

Thiết lập: Jet-Nemotron-2B xây trên Qwen2.5-1.5B; Jet-Nemotron-4B trên Qwen2.5-3B. Huấn luyện hai giai đoạn: giai đoạn 1 đóng băng MLP + distillation, **50B token** (Nemotron-CC + Redstone-QA); giai đoạn 2 huấn luyện toàn mô hình với thêm dữ liệu toán/code, **350B token**. Throughput đo trên H100, context 64K (trừ khi ghi chú).

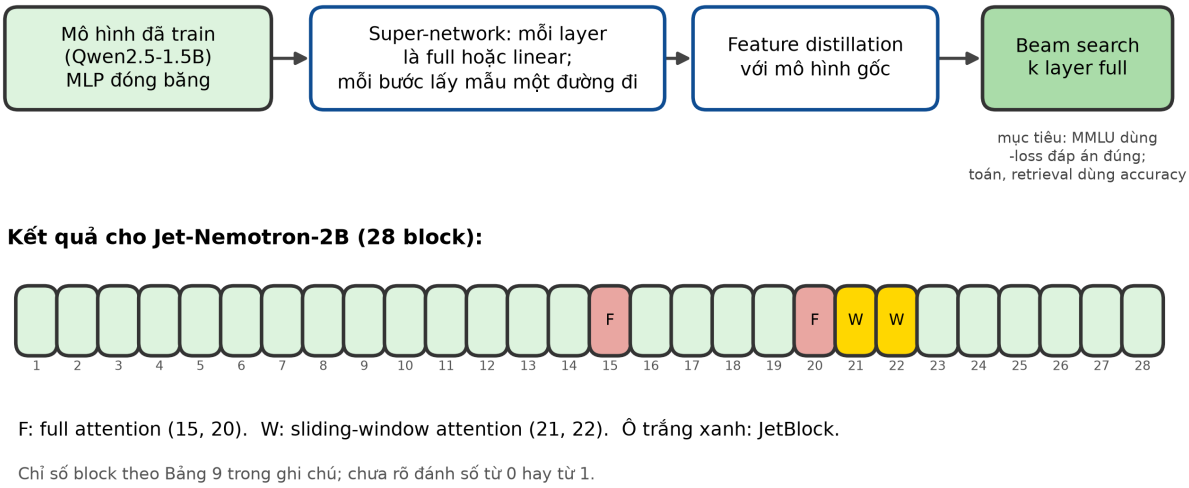
Kiến trúc cuối (Bảng 9 trong phụ lục):

	Jet-Nemotron-2B	Jet-Nemotron-4B
Tổng số block	28	36
Full attention layers	No. 15, 20	No. 18, 21, 22, 28, 33
Sliding window attention (SWA)	No. 21, 22	No. 17, 20, 23, 24, 26
Hidden / MLP size	1536 / 8960	2048 / 11008

Mọi layer còn lại là JetBlock. Lưu ý: Mục 3.1 của chính paper mô tả 4B với full-attention **18, 21, 33** và 7 SWA (6, 17, 20, 22, 23, 26, 28), không khớp với Bảng 9 ở trên. Chưa rõ bảng nào đúng; xem Giới hạn. Với 2B hai nguồn khớp nhau.

Cách đọc hình: hàng trên là các bước tìm vị trí: super-network lấy mẫu đường đi full hoặc linear, huấn luyện bằng feature distillation, rồi beam search chọn k layer full. Hàng dưới là kết quả cho bản 2B, chỉ hai layer full attention và hai layer sliding-window, còn lại là JetBlock. Chỉ số block lấy theo bảng trong ghi chú.

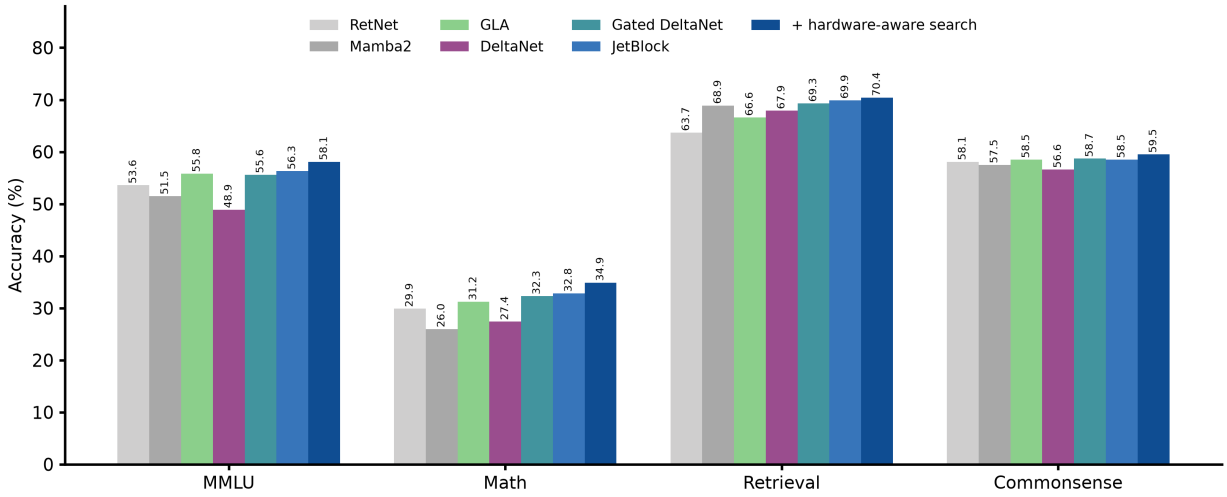
Chọn khối linear (Bảng 1, accuracy MMLU / Math / Retrieval / Commonsense):



Hình 81: PostNAS học vị trí full-attention bằng super-network và beam search, cùng bố cục 28 block của Jet-Nemotron-2B theo Bảng 9 (minh hoạ, tự tính)

Khối	MMLU	Math	Retrieval	Common.
RetNet	53.6	29.9	63.7	58.1
Mamba2	51.5	26.0	68.9	57.5
GLA	55.8	31.2	66.6	58.5
DeltaNet	48.9	27.4	67.9	56.6
Gated DeltaNet	55.6	32.3	69.3	58.7
JetBlock	56.3	32.8	69.9	58.5
+ hardware-aware search	58.1	34.9	70.4	59.5

(RWKV7 bị loại vì throughput huấn luyện thấp: 123 token/s so với ~250.)



Hình 82: Accuracy của các khối linear attention trên bốn nhóm tác vụ, kèm JetBlock và JetBlock có hardware-aware search (Bảng 1) (vẽ lại từ số liệu trong ghi chú)

Cách đọc hình: mỗi nhóm cột là một nhóm tác vụ, bảy cột là bảy khối theo thứ tự trong chú giải. Gated DeltaNet đứng cao ở hầu hết nhóm, JetBlock và bản có hardware-aware search tiếp tục tăng thêm. Ví dụ MMLU tăng từ 55.6 lên 56.3 rồi 58.1.

Mô hình cuối so với đối thủ (Bảng 3-8, trích):

Long-context (LongBench, Bảng 8): Jet-Nemotron-2B trung bình 41.1 với chỉ hai full-attention layer, xấp xỉ Qwen2.5-1.5B 39.1 và Gemma3n-E2B 40.4; Jet-Nemotron-4B 43.9.

Tốc độ: so với Qwen3-1.7B-Base ở 256K: prefill 6.14x, decode 53.6x; ở 4K decode 15.6x. Cận trên lý thuyết là 56x vì 2 full-attention layer × 2 nhóm KV so với 28 layer × 8 nhóm KV. Trên phần cứng nhỏ (context 64K): Jetson Orin 55.00 vs 6.22 tok/s (8.84x), RTX 3090 684.01 vs 105.18 (6.50x) so với Qwen2.5-1.5B.

Chi phí PostNAS cho 2B (Bảng 12): bước học vị trí full-attention: 50B token, 0.8 ZFLOPs, 808 giờ H100; chọn khối linear 3120 giờ; JetBlock 624 giờ; hardware-aware search 5616 giờ; huấn luyện cuối giai đoạn 2 7536 giờ.

5. Phân tích / ablation

Bốn "key finding" của paper:

1. **Các attention layer không đóng góp ngang nhau.** Với MMLU chỉ hai layer thật sự quan trọng; với retrieval chỉ hai đến ba layer.
2. **Layer khác nhau phụ trách năng lực khác nhau.** Layer quan trọng cho MMLU không nhất thiết quan trọng cho retrieval.
3. **Toán phức tạp hơn**, nhưng hợp của các layer top cho MMLU và retrieval đã bao phủ phần lớn layer quan trọng cho toán.
4. **Kích thước KV cache quyết định throughput sinh**, không phải số tham số (decode bị giới hạn băng thông bộ nhớ): khi cache cố định, các cấu hình có số tham số khác nhau (1.62B đến 1.98B) cho throughput tương tự (~2950-2980 tok/s ở Bảng 2).

Thêm:

- **Học vị trí tốt hơn rải đều** (Hình 5b): PostNAS vượt uniform về accuracy MMLU; các con số trong hình bị mangle trong bản text nên không trích dẫn tại đây.
- **Dùng SWA cho tác vụ trắc nghiệm:** tác giả đặt retrieval hướng dẫn chọn full-attention và MMLU hướng dẫn chọn SWA; cho rằng tác vụ trắc nghiệm chủ yếu cần tính chất "khớp mẫu" của softmax và SWA giữ được điều đó (giải thích của tác giả).
- **Tách nguồn gốc dữ liệu (Bảng 14):** khi các baseline được huấn luyện tiếp trên cùng dữ liệu của Jet-Nemotron, Jet-Nemotron-2B vẫn hơn (MMLU 59.6 vs Qwen2.5-continual 56.7, Mamba2 41.0, RWKV7 49.8).
- **PostNAS tổng:** MMLU +5.3, Math +8.4, Retrieval +7.8, Commonsense +3.2 qua bốn bước (Hình 3).
- **So với Falcon-H1 (hybrid theo head):** Jet-Nemotron-2B tương đương Falcon-H1-1.5B-deep nhưng throughput 2885 vs 66 tok/s; paper giải thích do head-wise hybrid phải chạy tuần tự Mamba2 và attention trong cùng layer.

6. Giới hạn

Paper nêu / ngụ ý:

- Kiến trúc tìm được "có thể không tối ưu nếu huấn luyện từ đầu"; giá trị nằm ở dùng được ngay và làm testbed.
- Hiệu năng prefill ở context ngắn chỉ nhỉnh hơn một chút (1.14x ở 4K); cần kernel JetBlock tối ưu hơn.

Nhận xét:

- Vị trí full-attention được chọn theo tác vụ đại diện (retrieval, MMLU); nếu tác vụ đích khác (OCR/tài liệu) có thể cần chạy lại.
- Bảng 9 và Mục 3.1 mâu thuẫn về chỉ số layer của 4B (xem trên).
- Chi phí đáng kể: hàng nghìn giờ H100 cho mỗi bước tìm kiếm; train 350B token.

- Điều kiện thực nghiệm chỉ trên Qwen2.5 (decoder-only thuần ngôn ngữ), không có VLM.
- Không phải pruning thuần: mô hình cuối được huấn luyện thêm 400B token, không chỉ phục hồi nhẹ.

7. Liên hệ với các bài khác

- **44 Nemotron-H:** cũng thấy attention layer không đồng đều (một số attention layer ít quan trọng); ở Nemotron-H attention ~8% rải đều từ đầu, còn ở đây vị trí được học và hạ xuống 2/28 (~7%).
- **45 Minitron-SSM:** thấy attention layer đầu tiên ít quan trọng; bổ sung cho Finding 1.
- **42 Puzzle:** NAS theo khối có điều kiện phần cứng; PostNAS cũng dùng "mạng con từ một mô hình cha".
- **31 ShortGPT, 34 SLEB, 35 LaCo, 36 ReplaceMe:** đo độ quan trọng layer; ở đây thay layer bằng phiên bản rẻ thay vì bỏ hẳn.
- **38 Sheared LLaMA, 41 Flextron:** cũng huấn luyện mạng con từ mô hình lớn.
- **46 Mamba-Shedder:** cùng câu hỏi "tỉ lệ SSM/attention phù hợp?"; Mamba-Shedder là cắt, bài này là thay.

8. Tóm tắt khái niệm

Khái niệm	Ý nghĩa
PostNAS	NAS trên mô hình pretrained, MLP đóng băng
Super-network once-for-all	Mỗi layer là full hoặc linear; lấy mẫu mạng con khi huấn luyện
Mục tiêu tìm kiếm	MMLU: −loss đáp án đúng; toán/retrieval: accuracy
Beam search	Tìm vị trí k full-attention layer tốt nhất
Gated DeltaNet	Linear attention với gating phụ thuộc dữ liệu + delta rule
JetBlock	Gated DeltaNet + conv động trên V
SWA	Sliding-window attention (cửa sổ 1152 ở 2B, 2048 ở 4B)
Finding 4	KV cache, không phải số tham số, quyết định throughput sinh
Kết quả 2B	2 full-attention / 28 layer; 47x throughput so với Qwen3-1.7B (64K)

9. Kết luận ngắn

Trong hybrid, không phải full-attention layer nào cũng đáng giữ, và vị trí quan trọng phụ thuộc năng lực mong muốn. Học vị trí bằng super-network + beam search vượt rải đều; chỉ 2/28 layer full-attention (cùng 2 SWA) là đủ để đạt độ chính xác ngang mô hình full-attention cùng cỡ. Gated DeltaNet là khối linear tốt nhất trong các khối thử.

Link gốc: https://arxiv.org/abs/2508.15884

Bài 48

ECoFLaP: Efficient Coarse-to-Fine Layer-Wise Pruning for Vision-Language Models (2024)

Tác giả: Yi-Lin Sung, Jaehong Yoon, Mohit Bansal (UNC Chapel Hill).
Venue / arXiv: arXiv:2310.02998 (v2, 26/1/2024); ICLR 2024 (theo đầu trang paper "Published as a conference paper at ICLR 2024").
Nguồn: https://arxiv.org/abs/2310.02998
Chủ đề: Cắt tỉa **không cấu trúc (unstructured)** một lần cho mô hình vision-language (BLIP-2, BLIP, CLIP...): xác định tỉ lệ thưa **cho từng layer** từ độ quan trọng toàn cục (xấp xỉ bậc 0 của gradient), rồi cắt theo từng layer bằng Wanda/SparseGPT.
Vị trí trong lộ trình: Đọc trước: 23 SparseGPT, 24 Wanda (hai bộ cắt "fine"), 07 Molchanov, 01 OBD/OBS (độ quan trọng $w^2/2[H^{-1}]_{ii}$ và gradient). Bài này thêm: (a) vấn đề riêng của VLM là phân phối trọng số/gradient chênh lệch giữa module thị giác và ngôn ngữ; (b) phân bổ thưa không đều giữa các layer; (c) tính gradient bằng chỉ forward. Tiếp theo: 49.

> Ghi chú học tập bằng tiếng Việt, không phải bản dịch. Bài này là pruning **không cấu trúc**: không giảm kích thước ma trận, nên không tự nhiên làm nhanh trên GPU thường.

1. Bối cảnh

- **Cắt toàn cục (global pruning):** xếp hạng mọi trọng số của cả mô hình theo một điểm chung rồi bỏ phần thấp nhất. Chính xác (điểm kiểu OBS: $w_i^2/(2[H^{-1}]_{ii})$) nhưng cần Hessian nghịch đảo của cả mô hình, bất khả thi với mô hình lớn.
- **Cắt theo từng lớp (layer-wise, one-shot):** SparseGPT (23), Wanda (24) dùng ~128 mẫu hiệu chỉnh, mỗi lớp cắt cùng tỉ lệ p% độc lập. Rẻ, nhưng **mọi layer cùng một tỉ lệ thưa** và không có cái nhìn toàn cục, nên dưới thưa cao thì kém.
- **Vấn đề của VLM:** mô hình gồm các module theo từng modality (ViT thị giác, Q-Former, LLM). Hình 1 của bài báo cho thấy độ lớn trọng số và gradient của thị giác so với ngôn ngữ chênh nhau mạnh, và điểm cục bộ của SparseGPT phân bố rất lệch giữa layer. Hậu quả: điểm của các lớp **không so sánh được** với nhau, nên không thể dùng chúng để chia tỉ lệ thưa; và cắt toàn cục bằng độ lớn (global magnitude) đồng đều thì phá module thị giác (đạt 0.0 ở 50% thưa).

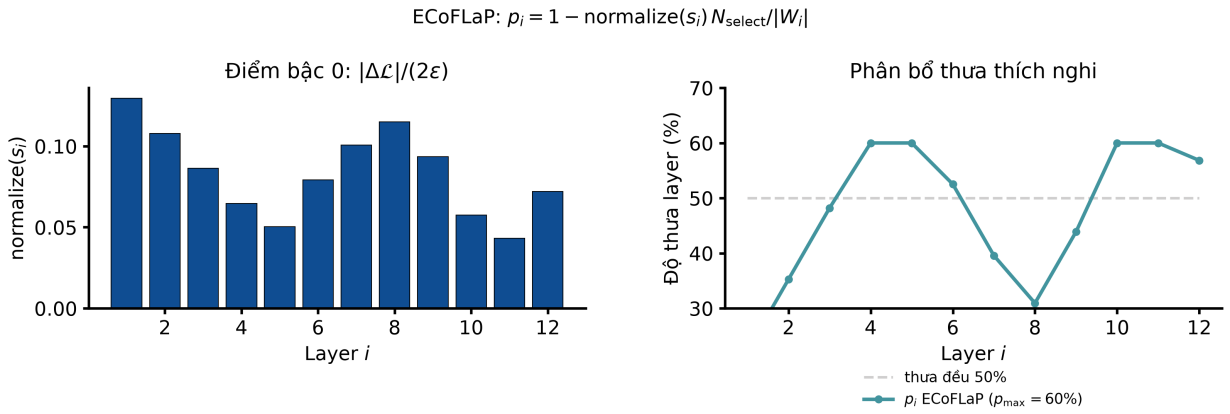
2. Ý tưởng chính

Chia hai pha thô-đến-mịn. **Thô (coarse):** tính một điểm quan trọng *toàn cục* cho mỗi lớp (hoặc block), rồi chuyển thành tỉ lệ thưa riêng của lớp đó (layer quan trọng thì giữ nhiều hơn). **Mịn (fine):** trong từng layer, dùng Wanda (hoặc SparseGPT) cắt theo tỉ lệ thưa vừa tính. Để tính điểm toàn cục mà không tốn bộ nhớ cho backprop, tác giả dùng **gradient bậc 0** (ước lượng bằng hai lần forward với nhiễu Gaussian), nên bộ nhớ gần bằng Wanda.

3. Phương pháp chi tiết

3.1 Từ điểm toàn cục sang tỉ lệ thưa từng layer

Điểm quan trọng toàn cục $s = S(W \mid D, \mathcal{L})$. Cho thưa tổng p, số trọng số cần giữ $N_{select} = (1-p)|W|$. Với layer i:



ECoFLaP phân bổ p_i theo điểm gradient bậc 0, có trần $p_{\max}$. (minh hoạ, tự tính)

$$\text{normalize}(s_i, s) = \frac{s_i}{\sum s}, \qquad \boxed{p_i = 1 - \frac{\text{normalize}(s_i, s) \cdot N_{select}}{|W_i|}}$$

Nghĩa: số trọng số được giữ ở layer i tỉ lệ với tổng điểm của layer đó. Thêm siêu tham số p_{max} (thưa tối đa mỗi layer) để tránh **layer collapse** (một layer bị cắt hết). Thực nghiệm đặt $p_{max} = p + 0.1$. Ngoài ra dùng cùng một tỉ lệ cho các layer trong cùng block (ví dụ FFN và attention của một block transformer) bằng cách cộng điểm theo block, cho kết quả bền hơn.

3.2 Gradient bậc 0 (chỉ forward)

Điểm bậc nhất cổ điển: $|W| \cdot |\nabla_W \mathcal{L}|$ (LeCun 1989). Backprop cho LLaMA 7B cần hơn 40 GB. Thay bằng:

$$\boxed{\|\nabla_{W_i}\mathcal{L}\| \approx \mathbb{E}_{d\sim D}\, \mathbb{E}_{z\sim\mathcal{N}(0,1)} \left[\left| \frac{\mathcal{L}(W_i + \epsilon z, d) - \mathcal{L}(W_i - \epsilon z, d)}{2\epsilon} \right| \right]}$$

Dùng giá trị này **trực tiếp** làm điểm của W_i (không nhân độ lớn trọng số, vì "solely using the approximated gradients works better empirically"). Quy trình theo từng layer: chọn seed, lấy nhiễu z, đổi $W \to W + \epsilon z$ và tính loss; dùng lại seed để trừ $2\epsilon z$ ($W - \epsilon z$) và tính loss; lấy hiệu chia 2ϵ (các đại lượng đều là số vô hướng). Với mạng N layer chỉ cần $(N + 1)$ lần bộ nhớ của một layer, so với $2N$ nếu lưu toàn bộ gradient. Một nhiễu mỗi mẫu là đủ.

3.3 Pseudo-code

```
ECoFLaP(model, calib D (32 mẫu), sparsity p, p_max=p+0.1):
  # Coarse
  for each layer (or block) i:
      s_i = mean over d in D of | L(W_i + eps z, d) - L(W_i - eps z, d) | / (2 eps)
      (restore W_i)
  N_select = (1-p) * total_params
  p_i = 1 - normalize(s_i) * N_select / |W_i|     # clip sao cho p_i <= p_max
  # Fine
  for layer i = 1..L:
      prune W_i to sparsity p_i using Wanda (|W| * ||input act||) or SparseGPT
```

Mô hình mục tiêu: BLIP/BLIP-2 gồm EVA-ViT + Q-Former + FlanT5 (hoặc BERT). Q-Former được giữ nguyên vì chỉ ~5% tham số.

4. Thí nghiệm và kết quả chính

Mô hình/dữ liệu: BLIP-2 (EVA-ViT + FlanT5), BLIP (ViT + BERT), CLIP, FlanT5, LLaMA-7B, EVA-ViT. Tác vụ: VQAv2, OK-VQA, GQA, NoCaps, Flickr30k (zero-shot cho BLIP-2); NLVR2, COCO captions (BLIP fine-tune); ImageNet-1k; MMLU; WikiText. Hiệu chỉnh 128 mẫu (CC3M, ImageNet, C4); ablation dùng 32 mẫu.

BLIP-2 ở thưa 50%, zero-shot (Bảng 1):

Phương pháp	Bộ nhớ (GB)	VQAv2	OK-VQA	GQA	NoCaps CIDEr	Flickr TR@1	Macro avg
Full model	-	63.1	41.1	44.1	121.7	97.6	62.1
Global magnitude	8.46	0.0	0.0	0.0	0.0	0.2	0.0
Gradient-based (lặp, global)	22.4	56.9	35.3	41.7	96.8	93.0	55.4
SparseGPT	9.04	57.4	36.1	40.8	108.1	96.4	57.4
Wanda	8.87	57.9	35.3	42.2	106.9	95.1	57.2
ECoFLaP (grad. bậc 1)	22.4	59.4	36.3	42.3	109.9	95.9	58.2
ECoFLaP (grad. bậc 0)	8.93	58.4	35.6	42.9	110.2	95.5	58.0

So với Wanda: +1.1% VQA, +2.3% captioning, +1.0% retrieval (tương đối); ở thưa 0.6, hơn Wanda 9.6% trên VQA. Dùng ~40% bộ nhớ của cắt toàn cục lặp.

CLIP ở thưa 0.4 (Bảng 2, trung bình 11 tác vụ): đầy đủ 65.3; Wanda 47.2; SparseGPT 58.7; ECoFLaP+Wanda 56.0; ECoFLaP+SparseGPT 61.0. Dùng *điểm cục bộ* để chia tỉ lệ thưa còn **tệ hơn** thưa đều (Wanda 45.3, SparseGPT 54.0).

So UPop (BLIP, thưa 50%, Bảng 3):

	NLVR2 val/test	COCO CIDEr/SPICE
Full	82.3 / 83.6	133.3 / 23.8
Không fine-tune: UPop	76.9 / 77.8	-
Không fine-tune: Wanda	78.3 / 78.1	97.1 / 18.4
Không fine-tune: ECoFLaP	79.1 / 79.2	111.0 / 20.3
Có fine-tune: UPop	80.3 / 81.1	128.9 / 23.3
Có fine-tune: ECoFLaP	81.8 / 82.5	132.3 / 23.8

LLaMA-7B, thưa 0.6, WikiText perplexity (Bảng 7): đầy đủ 7.26; Wanda 10.68; ECoFLaP bậc 1 10.16; ECoFLaP bậc 0 **9.83** (cải thiện tương đối 10.6% so với Wanda).

5. Phân tích / ablation

- **Tỉ lệ thưa tối đa p_{max} (Bảng 4, VQA, thưa 0.5):** 0.5 → 57.9; 0.6 → 58.4; 0.8 → 57.1; 1.0 (không giới hạn) → 51.6. Không giới hạn gây layer collapse.
- **Số mẫu (Bảng 5):** 16/32/64/128 mẫu cho 58.2/58.4/58.3/58.4: rất ít nhạy. **Số nhiễu (Bảng 6)** với tổng lần forward cố định: 1/4/8/32 nhiễu cho 58.4/58.1/58.0/58.1, tức nên dùng nhiều mẫu, một nhiễu mỗi mẫu.

- **Hình 6:** các biến thể ECoFLaP đều gán thưa **thấp hơn cho module thị giác, cao hơn cho module ngôn ngữ**; phân bố của bậc 0 gần với bậc 1, ủng hộ việc dùng xấp xỉ rẻ.
- **Landscape (Hình 5):** BLIP-2 nằm trong một "bồn" mượt, một cực tiểu địa phương; tác giả dùng đó để biện minh cho ước lượng bậc 0.
- **Phụ lục:** ϵ lớn hơn giúp ước lượng bậc 0 ở LLaMA (giả thuyết: khám phá vùng xa hơn).
- **Tổng quát hoá:** hiệu quả trên FlanT5/MMLU, EVA-ViT/ImageNet, đặc biệt ở thưa cao (Hình 4).

6. Giới hạn

Paper nêu / ngụ ý: tập trung vào **không cấu trúc**; tác giả nói trực tiếp rằng lợi ích tốc độ cần phần cứng/kernel thưa chuyên dụng (trong phần Related Work). Chỉ giữ Q-Former nguyên vẹn.

Nhận xét:

- Không đo latency hay tốc độ; toàn bộ kết quả là thưa trọng số, không giảm kích thước tensor.
- Cắt 50-60% trọng số không cấu trúc ≠ giảm 50% FLOPs/VRAM trên GPU thường (VRAM vẫn lưu ma trận dày nếu không nén thưa).
- Chưa có phục hồi bằng distillation; chỉ fine-tune ở BLIP (NLVR2/COCO).
- Mô hình thuộc đời BLIP-2/FlanT5, không phải VLM decoder-only hiện đại; không có OCR/tài liệu.
- Độ tin cậy của bậc 0 chỉ được biện minh bằng landscape mượt trên BLIP-2.

7. Liên hệ với các bài khác

- **23 SparseGPT, 24 Wanda:** bước "fine" của ECoFLaP; nhược điểm cố hữu là thưa đều.
- **01 OBD/OBS, 07 Molchanov:** điểm quan trọng bậc nhất/bậc hai; ECoFLaP thay backprop bằng ước lượng bậc 0.
- **29 SliceGPT, 26 LLM-Pruner, 28 FLAP:** cắt có cấu trúc; ECoFLaP không đạt tăng tốc trực tiếp như thế.
- **18 2:4 sparsity:** con đường để thưa hoá không cấu trúc mới tăng tốc phần cứng.
- **Phân bổ thưa không đều giữa layer** gần với tinh thần "layer nào đáng giữ" của 42 Puzzle, 47 Jet-Nemotron.

8. Gợi ý áp dụng

- Thưa không cấu trúc không đem lại giảm VRAM/latency trên GPU thông thường, nên ECoFLaP không phải công cụ chính.

9. Thí nghiệm gợi ý

3. Kiểm tra xem ước lượng gradient bậc 0 có xếp hạng block giống gradient thật không (tương quan hạng) trên một mô hình nhỏ.

10. Tóm tắt công thức

Khái niệm	Công thức / ý
Tỉ lệ thưa mỗi layer	$p_i = 1 - \frac{(s_i / \sum s)\, N_{select}}{\lvert W_i \rvert}$, $N_{select} = (1-p)\lvert W \rvert$
Ràng buộc	$p_i \le p_{max} = p + 0.1$
Gradient bậc 0	$\mathbb{E}\lvert \frac{\mathcal{L}(W+\epsilon z) - \mathcal{L}(W-\epsilon z)}{2\epsilon} \rvert$
Bộ nhớ	$(N+1)$ lần bộ nhớ một layer, so với $2N$
Bước mịn	Wanda hoặc SparseGPT với p_i

Khái niệm	Công thức / ý
Phát hiện	Thị giác ít thưa hơn, ngôn ngữ thưa hơn

11. Kết luận ngắn

ECoFLaP cho thấy trong VLM không nên cắt đều mọi layer: tính độ quan trọng toàn cục bằng gradient bậc 0 (rẻ như forward) để chia tỉ lệ thưa từng layer, rồi cắt bằng Wanda/SparseGPT, cải thiện rõ nhất ở thưa cao (ví dụ +9.6% VQA tương đối so với Wanda ở thưa 0.6) và ngang bộ nhớ Wanda.

Link gốc: https://arxiv.org/abs/2310.02998

Bài 49

Structural Pruning of Large Vision Language Models: A Comprehensive Study on Pruning Dynamics, Recovery, and Data Efficiency (2026)

Tác giả: Yiran Huang, Lukas Thede, Massimiliano Mancini, Wenjia Xu, Zeynep Akata (TUM, Munich Center for Machine Learning, Helmholtz Munich, Tübingen AI Center, Univ. Trento, BUPT).
Venue / arXiv: arXiv:2604.24380 (v1, 27/4/2026). Bài nói là mở rộng từ một bản hội nghị trước của chính nhóm; không nêu tên venue.
Nguồn: https://arxiv.org/abs/2604.24380
Chủ đề: Nghiên cứu thực nghiệm: cắt tỉa có cấu trúc **LLM backbone** của VLM (layerwise vs widthwise), các cách phục hồi (SFT, distillation logit/hidden state), hiệu quả dữ liệu, và quyết định chọn cấu hình theo tỉ lệ nén.
Vị trí trong lộ trình: Đọc trước: 26 LLM-Pruner (widthwise theo dependency graph), 31/33 (Block Influence, cắt layer), 39 Minitron, 20 DepGraph, 48 ECoFLaP. Bài này thêm: bức tranh thực nghiệm cho **VLM**: cắt layer thắng sau phục hồi ở nén thấp, cắt width thắng ở nén cao, và công thức phục hồi tốt nhất.

> Ghi chú học tập bằng tiếng Việt, không phải bản dịch. Mọi con số lấy từ paper.

1. Bối cảnh

- **LVLM** (Large Vision-Language Model) = vision encoder + bộ chiếu đa phương thức (multimodal projector) + LLM. LLM thường chiếm phần lớn tham số (Bảng 1: LLaVA-v1.5-7B: LLM Vicuna 6.7B của 7.0B; Mini-InternVL-4B: Phi-3-mini 3.8B của 4.1B; Bunny-3B: Phi-2 2.8B của 3.2B), nên là mục tiêu nén chính.
- Cách phổ biến để có VLM nhỏ là ghép với một mô hình ngôn ngữ nhỏ rồi huấn luyện từ đầu (MobileVLM, Bunny...): kích thước bị cố định theo SLM và tốn kém. Hướng thay thế: **nén VLM sẵn có** rồi phục hồi nhẹ.
- Thách thức riêng: cắt LLM có thể làm hỏng không chỉ khả năng ngôn ngữ mà cả **sự căn chỉnh thị giác-ngôn ngữ** đã học. Chưa có nghiên cứu hệ thống.
- Hai kiểu cắt có cấu trúc (gloss): **layerwise** = bỏ nguyên các block transformer; **widthwise** = bỏ attention head và neuron MLP bên trong mỗi lớp. Cắt có cấu trúc giảm kích thước ma trận thật, khác cắt không cấu trúc (xem 48).

2. Ý tưởng chính

Tác giả áp dụng hai kiểu cắt đã dùng cho LLM (Block Influence cho layer; dependency-graph + Taylor cho width) lên LLM backbone của ba họ VLM, rồi so sánh nhiều cách phục hồi (SFT chỉ projector, SFT projector + LLM với LoRA, KD logit KL/RKL, KD hidden state L2 và các tổ hợp), ở các mức nén 15/30/45/60% và các tỉ lệ dữ liệu 5-100%. Kết luận thực dụng: nén nhẹ (≤15%) chỉ cần SFT projector; nén vừa (≤30%) dùng cắt layer + phục hồi; nén cao (≥45%) dùng cắt width; công thức phục hồi tốt nhất là **SFT + L2 hidden-state distillation**; 5% dữ liệu đủ cho nén ≤ 45% (theo tóm tắt dưới 50%).

3. Phương pháp chi tiết

3.1 Cắt layer: Block Influence (BI)

Với layer i, đầu vào H_i và đầu ra H_{i+1} (hidden state), đo khoảng cách cosine:

$$\boxed{\mathrm{BI}_i(D) = 1 - \mathbb{E}_{X\sim D,t}\left[\frac{H_{i,t}^{\top} H_{i+1,t}}{\|H_{i,t}\|_2 \|H_{i+1,t}\|_2}\right]}$$

Layer biến đổi hidden state càng ít (BI thấp) thì càng bỏ được. Xếp hạng rồi bỏ các layer BI thấp nhất.

3.2 Cắt width: dependency graph + Taylor

Xây đồ thị phụ thuộc trong mỗi layer: N_j phụ thuộc N_i nếu $N_j \in \mathrm{Out}(N_i)$ và N_j chỉ có một đầu vào (hoặc đối xứng); khi cắt N_i phải cắt luôn các nút phụ thuộc → nhóm $\mathcal{G}$. Độ quan trọng nhóm bằng thay đổi loss khi loại trọng số, xấp xỉ Taylor bậc nhất:

$$I_{w_i^k}(X) = \left|\mathcal{L}_{CE}(X, m_\theta) - \mathcal{L}_{CE}(X, m_\theta^{w_i^k=0})\right| \approx \left|\frac{\partial \mathcal{L}_{CE}}{\partial w_i^k} w_i^k\right|, \qquad I_{\mathcal{G}}(D) = \mathbb{E}_X\Big[\sum_i \sum_k I_{w_i^k}(X)\Big]$$

Bỏ nhóm có $I_{\mathcal{G}}$ thấp nhất. Hiệu chỉnh bằng một tập nhỏ mẫu lấy từ chính dữ liệu visual instruction tuning của mô hình.

3.3 Phục hồi

Giữ vision encoder đóng băng. Ký hiệu teacher = mô hình gốc, student = mô hình đã cắt.

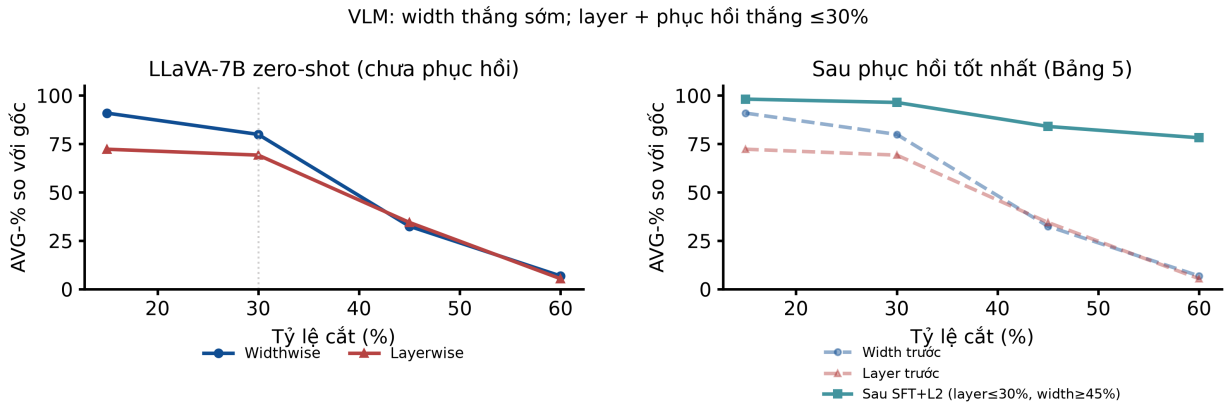
- **SFT:** $\mathcal{L}_{sft} = \mathbb{E}\,\mathcal{L}_{CE}(m^S(x_v, x_p), x_r)$. Hai chế độ: chỉ huấn luyện projector, hoặc projector + LLM (LoRA).
- **KD logit:** KL hoặc reverse KL (RKL) giữa phân phối teacher và student. KL là "mean-seeking" (phải khớp cả đuôi), RKL "mode-seeking" (chỉ phạt khi student đặt xác suất cao ở chỗ sai).
- **KD hidden state (L2):** $\mathcal{L}_{match} = \mathbb{E}\,\|H_i^S - H_i^T\|_2^2$.
- **Tổng hợp:**

$$\boxed{\mathcal{L} = \alpha\,\mathcal{L}_{sft} + \beta\,\mathcal{L}_{logits} + \gamma\,\mathcal{L}_{match}}$$

```
CompressLVLM(model, ratio r, data D):
  calib = small subset of D (<= ~10-15 samples đã đủ, xem Hình 7)
  if r small and want latency: score layers by BI(calib); drop lowest-BI layers
  else:                        build dependency groups; score by Taylor; drop lowest
  ↪  groups
  recover: freeze vision encoder
           if r <= 15%: train projector only (SFT)
           else:        train projector + LLM(LoRA) with SFT + L2(hidden states) vs
           ↪  original teacher
           data: 5%..100% of D (5% đủ khi r <= 45%)
```

4. Thí nghiệm và kết quả chính

Mô hình: LLaVA-v1.5-7B, Mini-InternVL-Chat-4B-V1.5, Bunny-v1.0-3B. **Benchmark:** MMMU, SQA, MathVista, AI2D, MME-Cognition/Perception, POPE, DocVQA, GQA (qua lmms-eval); báo cáo trung bình và % so với mô hình gốc. **Dữ liệu:** dữ liệu visual instruction tuning gốc của từng mô hình để cả hiệu chỉnh và phục hồi (5%, 10%, 20%, 100%).



LLaVA-7B: widthwise thắng zero-shot sớm; sau SFT+L2 layer≤30% phục hồi gần dense. (minh hoạ, tự tính)

Zero-shot sau cắt, chưa phục hồi (Bảng 2, AVG-% so với gốc):

Mô hình	Kiểu	15%	30%	45%	60%
LLaVA-7B	Widthwise	90.87	79.88	32.43	6.75
	Layerwise	72.22	69.18	34.40	5.34
InternVL-4B	Widthwise	94.74	73.72	40.52	7.05
	Layerwise	90.48	57.42	34.17	4.79
Bunny-3B	Widthwise	90.87	75.58	47.44	14.51
	Layerwise	88.84	69.03	45.99	11.77

Widthwise rõ ràng tốt hơn ở 15-30% khi chưa phục hồi; ở 60% cả hai sụp đổ. Khoảng cách lớn hơn ở tác vụ sinh tự do/lập luận: trên Bunny 15%, widthwise hơn layerwise chỉ 2.2% tính trung bình nhưng 31.5% trên tác vụ sinh (MathVista, DocVQA, GQA); AI2D: 65.64 (width) so với 52.73 (layer) trên Bunny ở 15%.

Phục hồi tốt nhất = Layerwise (<45%) hoặc Widthwise (≥45%) + SFT + L2 (Bảng 5, AVG-%):

Mô hình	15% (layer)	30% (layer)	45% (width)	60% (width)
LLaVA-7B	98.10	96.38	83.99	78.13
InternVL-4B	99.12	95.96	86.22	79.39
Bunny-3B	98.98	94.37	90.48	83.86

(Bảng 5 chọn kiểu cắt: layerwise cho tỉ lệ < 45%, widthwise cho ≥ 45%.)

Phần cứng (Bảng 3; bộ nhớ GiB / FLOPs T / latency ms): Ví dụ LLaVA 0%: 13.23 / 9.57 / 105. Ở 30%: widthwise 9.32 / 6.89 / 84.4; layerwise 9.44 / 6.92 / 80.7. Ở 60%: widthwise 5.31 / 4.17 / 55.4; layerwise 5.37 / 3.90 / 49.2. Bunny 30%: width 43 ms, layer 41.2 ms. InternVL 30%: width 49.2 ms, layer 48.7 ms. Bộ nhớ và FLOPs gần như như nhau giữa hai kiểu cắt (chênh <0.5% FLOPs ở LLaVA 30%), nhưng **layerwise luôn có latency thấp hơn**.

Kết hợp lượng tử hoá (Bảng 7, LLaVA, LLM.int8()): 13.2 GiB (53.7) → 7.5 GiB (52.5) chỉ lượng tử; cắt 15% + int8: 6.5 GiB (51.9); cắt 30% + int8: 5.4 GiB (50.8).

5. Phân tích / ablation

- **Giải thích của tác giả (không phải chứng minh):** layerwise gây **lệch phân phối đặc trưng** (layer N+2 nhận đầu ra layer N mà nó chưa quen), widthwise giữ nguyên "giao diện" giữa các layer; ngoài

ra mất độ sâu làm mất khả năng lập luận nhiều bước. Sau phục hồi, lệch đặc trưng có thể học lại (nên layerwise thắng ≤30%), còn thiếu độ sâu là hạn chế cấu trúc (nên widthwise thắng ở nén cao).

- **Phục hồi đảo thứ tự:** sau SFT, layerwise vượt widthwise ở nén ≤ 30%; ở ≥ 45% widthwise vượt hoặc ngang (Mini-InternVL, Bunny vượt; LLaVA ngang).
- **Nén nhẹ chủ yếu hỏng sự căn chỉnh, không phải ngôn ngữ (Bảng 4, LLaVA widthwise):** 15% nén: suy giảm đa phương thức 9.13% so với text-only chỉ 1.07%; 30%: 20.11% vs 10.38%; 45%: 67.56% vs 56.92%; 60%: 93.26% vs 93.05%. Từ đó chỉ cần SFT projector khi nén ≤ 15% (giữ hơn 95% hiệu năng ở 15% trên cả ba mô hình; ở 60% phục hồi 60-80%).
- **SFT thêm LLM** cần khi nén > 40%: ở 45% khôi phục trên 80% hiệu năng gốc.
- **KD đơn lẻ không vượt SFT** ở nén cao; thứ tự RKL > KL > L2 khi đứng riêng, nhưng **SFT + L2 vượt mọi tổ hợp** (kể cả SFT+L2+KL/RKL, là các tổ hợp bộ ba). Tác giả giải thích bằng tính trực giao của hai mục tiêu (SFT tối ưu đầu ra, L2 ràng buộc biểu diễn trong) và xung đột khi thêm ràng buộc đầu ra.
- **Hiệu quả dữ liệu (Hình 6):** với nén ≤ 45%, 5% dữ liệu đã phục hồi >90% hiệu năng đạt được với dữ liệu đầy đủ; ở 60% cần nhiều dữ liệu hơn. (Tóm tắt Mục 4.6 nói đủ khi dưới 50%.)
- **Kích thước tập hiệu chỉnh (Hình 7):** hiệu năng tăng tới khoảng 10-15 mẫu rồi bão hoà (widthwise 15%: 43.60 với 1 mẫu → 48.80 với 15 mẫu và giữ nguyên).
- **"Vùng an toàn":** ≤ 30% giữ phần lớn; 45% là điểm gãy (LLaVA 83.99%, InternVL 86.22%, Bunny 90.48%); muốn >90% nên nén dưới 30%.
- **Failure case định tính (Bảng 6):** khi nén tăng, đếm đối tượng và nhận dạng chi tiết hỏng trước; ở 60% mô hình "hallucinate" theo cấu trúc tổng thể.

6. Giới hạn

Tác giả nêu: chỉ cắt có cấu trúc, không xét unstructured/semi-structured; vision encoder đóng băng (chưa cắt ViT); sau ngưỡng nén nhất định, hiệu năng "khó phục hồi" và "extreme pruning of a large model is not a viable approach"; chỉ hai kỹ thuật cắt, ba mô hình.

Nhận xét:

- Chỉ Transformer thuần (Vicuna, Phi-3-mini, Phi-2); không có kiến trúc hybrid/linear attention, và không có OCR chuyên dụng ngoài DocVQA.
- Importance của width dùng gradient Taylor; layer dùng BI; không so sánh các thước đo khác (ví dụ MSE/KL, bài 44-45).
- Bảng 5 chọn thủ công kiểu cắt theo tỉ lệ nén, nên so sánh "cắt layer vs width" sau phục hồi cần xem Hình 3 (đồ thị) chứ không chỉ bảng.
- Latency đo cụ thể trên một cấu hình; chênh lệch layerwise vs widthwise nhỏ (vài ms) và độ lệch chuẩn đôi khi lớn (ví dụ LLaVA 15% layerwise 95 ± 8.1).

7. Liên hệ với các bài khác

- **26 LLM-Pruner, 20 DepGraph:** nguồn của cắt width bằng dependency graph + Taylor.
- **31 ShortGPT, 33 Shortened LLaMA:** nguồn của BI/cắt layer; bài này xác nhận layer cắt gây hại cho lập luận nhiều bước.
- **39 Minitron, 40:** cùng kết luận "width thắng depth, nhất là lập luận (GSM8K)" (bài này trích Sreenivas et al. để so). Bài 45 cũng thấy width-only tốt hơn depth-only ở nén 50% (loss), nhưng depth-only nhanh nhất.
- **37 Repair changes which layers:** bài này cho thấy lựa chọn "layer vs width" thay đổi theo mức phục hồi; tinh thần tương tự (xếp hạng trước phục hồi chưa chắc đúng sau phục hồi).
- **43 Pruning vs scratch:** phục hồi dữ liệu thấp ủng hộ nén từ mô hình có sẵn.
- **48 ECoFLaP:** cùng chủ đề VLM, nhưng thưa không cấu trúc.
- **47 Jet-Nemotron:** giảm số full-attention layer là một hướng giảm độ sâu "có thay thế".

8. Gợi ý áp dụng

- **Công thức phục hồi dùng được ngay:** SFT + hidden-state L2 distillation (teacher lớn hơn 4B nên cần chiếu chiều nếu khác hidden size); freeze vision encoder khi nén nhẹ, chỉ train projector trước.
- **Hiệu chỉnh:** 10-15 mẫu đã đủ cho điểm quan trọng, và 5% dữ liệu đủ phục hồi nén vừa: tiết kiệm.

9. Thí nghiệm gợi ý

1. So sánh cắt layer và cắt chiều rộng tại cùng mức nén trước và sau khi tinh chỉnh.
2. So sánh SFT kết hợp chưng cất hidden state với SFT kết hợp KL trên logits, dùng cùng ngân sách huấn luyện.
3. Đo hiệu năng theo lượng dữ liệu phục hồi và mức nén để kiểm tra độ nhạy với dữ liệu.

10. Tóm tắt khái niệm

Khái niệm	Ý nghĩa
LVLM	Vision encoder + projector + LLM
Layerwise (BI)	Bỏ layer có BI thấp: $1 - \cos(H_i, H_{i+1})$
Widthwise	Dependency graph + Taylor, bỏ head/neuron
SFT projector	Chỉ huấn luyện projector, đủ khi nén ≤ 15%
SFT+L2	Cross-entropy + khớp hidden state; công thức phục hồi tốt nhất
RKL vs KL	Reverse KL (mode-seeking) vs KL (mean-seeking)
Vùng an toàn	Nén ≤ 30% giữ phần lớn hiệu năng
Điểm gãy	~45% (83.99-90.48% còn lại sau phục hồi)
Dữ liệu	5% đủ khi nén ≤ 45%; >50% cần dữ liệu đầy đủ

11. Kết luận ngắn

Để nén VLM có sẵn: dưới ~30% nén thì cắt layer + phục hồi (chỉ projector nếu ≤15%) giữ 94-99% hiệu năng và cho latency tốt nhất; từ 45% trở lên thì cắt width và phục hồi bằng SFT + hidden-state L2 distillation; 5% dữ liệu thường đủ nếu nén dưới ~50%.

Link gốc: https://arxiv.org/abs/2604.24380